P4$_{16}$ Language Specification

The P4 Language Consortium

Version v1.2.5, 2026-10-13

Contents

1. Scope 18
2. Terms, definitions, and symbols 18
3. Overview 19
    3.1. Benefits of P4 . . . 21
    3.2. P4 language evolution: comparison to previous versions (P4 v1.0/v1.1) . . . 22
4. Architecture Model 23
    4.1. Standard architectures . . . 24
    4.2. Data plane interfaces . . . 24
    4.3. Extern objects and functions . . . 24
5. Example: A very simple switch 25
    5.1. Very Simple Switch Architecture . . . 26
    5.2. Very Simple Switch Architecture Description . . . 28
        5.2.1. Arbiter block. . . . 28
        5.2.2. Parser runtime block . . . 28
        5.2.3. Demux block . . . 29
        5.2.4. Available extern blocks. . . . 29
    5.3. A complete Very Simple Switch program . . . 29
6. P4 language definition 33
    6.1. Syntax and semantics . . . 34
        6.1.1. Grammar . . . 34
        6.1.2. Semantics and the P4 abstract machines . . . 34
            6.1.2.1. Pseudo-code snippets . . . 34
            6.1.2.2. Prose algorithms . . . 34
                6.1.2.2.1. Conditions and optional results . . . 36
    6.2. Lexical constructs. . . . 36
        6.2.1. Identifiers . . . 36
        6.2.2. Literal constants . . . 37
            6.2.2.1. Boolean literals. . . . 37
            6.2.2.2. Integer literals. . . . 37
            6.2.2.3. String literals . . . 38
        6.2.3. Comments . . . 39
        6.2.4. Optional trailing commas . . . 39
    6.3. Scoping . . . 40
        6.3.1. Name resolution . . . 40
        6.3.2. Name visibility . . . 41
    6.4. Stateful elements . . . 42
    6.5. Call convention: copy-in/copy-out . . . 42
        6.5.1. Justification. . . . 45
        6.5.2. Optional parameters . . . 46
    6.6. Overloading. . . . 46
    6.7. Naming conventions . . . 47
7. The P4 Abstract Machine 47
    7.1. Preprocessing . . . 48
    7.2. Type checking . . . 48
        7.2.1. P4 Intermediate Representation . . . 49
        7.2.2. Typing a P4 program . . . 50
        7.2.3. Typing context . . . 50
    7.3. Instantiation . . . 52

7.3.1. Control-plane names . . . 54
7.3.1.1. Computing control-plane names . . . 54
7.3.1.1.1. Value sets . . . 54
7.3.1.1.2. Tables . . . 55
7.3.1.1.3. Keys . . . 55
7.3.1.1.4. Actions . . . 55
7.3.1.1.5. Instances . . . 56
7.3.1.2. Annotations controlling naming . . . 57
7.3.1.3. Recommendations . . . 57
7.3.2. Instantiating a P4 program . . . 58
7.3.3. Instantiation context . . . 58
7.4. Runtime evaluation . . . 60
7.4.1. Evaluating a P4 program . . . 60
7.4.2. Evaluation context . . . 61
7.4.3. The parser abstract machine . . . 61
7.4.4. The match-action pipeline abstract machine . . . 62
7.4.5. Concurrency model . . . 62
7.5. Compile-time known and local compile-time known values . . . 63
8. P4 types and values 65
8.1. Semantics of types . . . 66
8.1.1. Type checking surface types . . . 66
8.1.2. Well-formedness . . . 67
8.2. Base types and values . . . 68
8.2.1. Well-formedness . . . 69
8.2.2. The void type . . . 69
8.2.2.1. Type checking . . . 69
8.2.3. Booleans . . . 69
8.2.3.1. Operations . . . 70
8.2.3.2. Type checking . . . 70
8.2.4. Errors . . . 70
8.2.4.1. Operations . . . 71
8.2.4.2. Type checking . . . 71
8.2.5. Match kinds . . . 71
8.2.5.1. Operations . . . 72
8.2.5.2. Type checking . . . 72
8.2.6. Strings . . . 72
8.2.6.1. Operations . . . 73
8.2.6.2. Type checking . . . 73
8.2.7. Integers (signed and unsigned) . . . 73
8.2.7.1. Portability . . . 74
8.2.7.2. Integer literals . . . 75
8.2.7.3. Illegal arithmetic expressions . . . 75
8.2.7.4. Arbitrary-precision integers . . . 76
8.2.7.4.1. Operations . . . 76
8.2.7.4.2. Type checking . . . 77
8.2.7.5. Fixed-width unsigned integers (bit-strings) . . . 77
8.2.7.5.1. Operations . . . 78
8.2.7.5.2. Type checking . . . 79
8.2.7.6. Fixed-width signed integers . . . 80
8.2.7.6.1. Operations . . . 80

8.2.7.6.2. Type checking. . . . 82
8.2.7.7. Dynamically-sized bit-strings . . . 82
8.2.7.7.1. Operations . . . 82
8.2.7.7.2. Type checking. . . . 83
8.3. Named types. . . . 83
8.3.1. Type definitions (type constructors) . . . 85
8.3.1.1. Well-formedness . . . 86
8.3.2. Type resoultion . . . 86
8.3.3. Type specialization . . . 87
8.3.4. Type names . . . 87
8.3.4.1. Operations. . . . 88
8.3.4.2. Type checking . . . 88
8.3.4.3. Well-formedness . . . 89
8.3.5. Specialized types . . . 89
8.3.5.1. Type checking . . . 89
8.4. Data types and values . . . 90
8.4.1. Lists . . . 90
8.4.1.1. Operations. . . . 90
8.4.1.2. Type checking . . . 91
8.4.1.3. Well-formedness . . . 91
8.4.2. Tuples . . . 91
8.4.2.1. Operations. . . . 92
8.4.2.2. Type checking . . . 92
8.4.2.3. Well-formedness . . . 92
8.4.3. Arrays . . . 92
8.4.3.1. Type checking . . . 93
8.4.3.2. Well-formedness . . . 93
8.4.4. Header stacks . . . 93
8.4.4.1. Operations. . . . 94
8.4.4.2. Type checking . . . 96
8.4.4.3. Well-formedness . . . 96
8.4.5. Structs . . . 96
8.4.5.1. Operations. . . . 97
8.4.5.2. Type checking . . . 98
8.4.5.3. Well-formedness . . . 98
8.4.6. Headers . . . 98
8.4.6.1. Operations . . . 100
8.4.6.2. Type checking . . . 101
8.4.6.3. Well-formedness . . . 101
8.4.7. Header unions. . . . 101
8.4.7.1. Operations . . . 102
8.4.7.2. Type checking . . . 106
8.4.7.3. Well-formedness . . . 106
8.4.8. Enums . . . 106
8.4.8.1. Enum type declaration without an underlying type . . . 107
8.4.8.2. Enum type declaration with an underlying type . . . 107
8.4.8.3. Operations . . . 108
8.4.8.4. Type checking . . . 111
8.4.8.5. Well-formedness . . . 111
8.5. Object types and values . . . 111

8.5.1. Extern object types . . . 111
8.5.1.1. Type checking . . . 112
8.5.1.2. Well-formedness . . . 113
8.5.2. Parser object types . . . 113
8.5.2.1. Type checking . . . 114
8.5.2.2. Well-formedness . . . 114
8.5.3. Control object types . . . 114
8.5.3.1. Type checking . . . 114
8.5.3.2. Well-formedness . . . 114
8.5.4. Package object types . . . 115
8.5.4.1. Type checking . . . 115
8.5.4.2. Well-formedness . . . 115
8.5.5. Table object types . . . 115
8.5.5.1. Well-formedness . . . 116
8.5.6. Object reference values . . . 116
8.5.7. typedef declaration . . . 117
8.5.7.1. Type checking . . . 117
8.5.7.2. Well-formedness . . . 117
8.5.8. type declaration . . . 117
8.5.8.1. Operations . . . 118
8.5.8.2. Type checking . . . 118
8.5.8.3. Well-formedness . . . 118
8.6. Synthesized types and values . . . 119
8.6.1. Default . . . 119
8.6.1.1. Operations . . . 119
8.6.1.2. Well-formedness . . . 120
8.6.2. Invalid headers . . . 120
8.6.2.1. Operations . . . 120
8.6.2.2. Well-formedness . . . 121
8.6.3. Sequences . . . 121
8.6.3.1. Operations . . . 121
8.6.3.2. Well-formedness . . . 122
8.6.4. Records . . . 122
8.6.4.1. Operations . . . 123
8.6.4.2. Well-formedness . . . 123
8.6.5. Sets . . . 124
8.6.5.1. Operations . . . 124
8.6.5.1.1. Singleton sets . . . 125
8.6.5.1.2. The universal set . . . 125
8.6.5.1.3. Masks . . . 125
8.6.5.1.4. Ranges . . . 126
8.6.5.1.5. Products . . . 126
8.6.5.2. Well-formedness . . . 126
8.6.6. Table metadata . . . 126
8.6.6.1. Operations . . . 127
8.6.6.2. Well-formedness . . . 127
8.7. Unrolling . . . 127
9. P4 callables 128
9.1. Parameters . . . 130
9.1.1. Type checking . . . 131

9.1.2. Well-formedness . . . 133
9.2. Callable definitions (callable constructors) . . . 133
9.2.1. Well-formedness . . . 135
9.2.2. Well-formedness of return types . . . 136
9.3. Actions . . . 136
9.3.1. Well-formedness . . . 138
9.4. Functions . . . 139
9.4.1. User-defined functions . . . 139
9.4.1.1. Well-formedness . . . 140
9.4.2. Extern functions . . . 140
9.4.2.1. Well-formedness . . . 141
9.5. Methods . . . 141
9.5.1. Extern methods . . . 142
9.5.1.1. Well-formedness . . . 143
9.5.2. Parser apply methods . . . 144
9.5.2.1. Well-formedness . . . 144
9.5.3. Control apply methods . . . 144
9.5.3.1. Well-formedness . . . 145
9.5.4. Table apply methods . . . 145
9.5.4.1. Well-formedness . . . 146
9.5.5. Built-in methods . . . 146
9.5.5.1. Well-formedness . . . 146
10. P4 objects and constructors 147
10.1. Objects . . . 147
10.1.1. Fully-qualified names . . . 147
10.2. Constructors . . . 147
10.2.1. Constructor parameters . . . 148
10.2.1.1. Type checking . . . 149
10.2.1.2. Well-formedness . . . 149
10.2.2. Constructor definitions . . . 150
10.2.2.1. Well-formedness . . . 151
10.3. Extern objects . . . 152
10.3.1. Well-formedness . . . 153
10.4. Parser objects . . . 153
10.4.1. Well-formedness . . . 154
10.5. Control objects . . . 155
10.5.1. Well-formedness . . . 156
10.6. Table objects . . . 157
10.6.1. Well-formedness . . . 158
10.7. Parser value set objects . . . 158
10.8. Package objects . . . 159
10.8.1. Well-formedness . . . 160
11. Declarations 161
11.1. Semantics of declarations . . . 161
11.1.1. Type checking . . . 161
11.1.2. Compile-time evaluation . . . 163
11.1.3. Runtime evaluation . . . 165
11.2. Variable declarations . . . 165
11.2.1. Type checking . . . 166
11.2.2. Runtime evaluation . . . 167

11.3. Constant declarations . . . 169
11.3.1. Type checking . . . 169
11.3.2. Compile-time evaluation . . . 170
11.3.3. Runtime evaluation . . . 171
11.4. Instantiations . . . 171
11.4.1. Objects with abstract methods . . . 172
11.4.1.1. Type checking . . . 173
11.4.1.2. Compile-time evaluation . . . 176
11.4.2. Type checking . . . 177
11.4.3. Compile-time evaluation . . . 180
11.5. Function declarations . . . 181
11.5.1. Type checking . . . 181
11.5.2. Compile-time evaluation . . . 183
11.6. Action declarations . . . 183
11.6.1. Type checking . . . 183
11.6.2. Compile-time evaluation . . . 185
11.7. Error declarations . . . 185
11.7.1. Type checking . . . 185
11.7.2. Compile-time evaluation . . . 186
11.8. Match kind declarations . . . 187
11.8.1. Type checking . . . 187
11.8.2. Compile-time evaluation . . . 188
11.9. Extern function declarations . . . 188
11.9.1. Type checking . . . 188
11.9.2. Compile-time evaluation . . . 189
11.10. Extern object declarations . . . 189
11.10.1. Extern methods . . . 189
11.10.1.1. Type checking . . . 190
11.10.1.2. Compile-time evaluation . . . 192
11.10.2. Extern constructors . . . 193
11.10.2.1. Type checking . . . 193
11.10.3. Type checking . . . 194
11.10.4. Compile-time evaluation . . . 196
11.11. Parser declarations . . . 197
11.11.1. Type checking . . . 197
11.11.2. Compile-time evaluation . . . 198
11.12. Control declarations . . . 199
11.12.1. Type checking . . . 199
11.12.2. Compile-time evaluation . . . 200
11.13. Enum type declaration . . . 201
11.13.1. Enum type declarations without an underyling type . . . 202
11.13.1.1. Type checking . . . 202
11.13.1.2. Compile-time evaluation . . . 202
11.13.2. Enum type declarations with an underlying type . . . 203
11.13.2.1. Type checking . . . 203
11.13.2.2. Compile-time evaluation . . . 205
11.14. Struct type declaration . . . 205
11.14.1. Type checking . . . 206
11.14.2. Compile-time evaluation . . . 207
11.15. Header type declaration . . . 207

11.15.1. Type checking . . . 207
11.15.2. Compile-time evaluation . . . 208
11.16. Header union type declaration . . . 209
11.16.1. Type checking . . . 209
11.16.2. Compile-time evaluation . . . 210
11.17. Typedef declaration . . . 210
11.17.1. typedef declaration . . . 211
11.17.1.1. Type checking . . . 211
11.17.1.2. Compile-time evaluation . . . 212
11.17.2. New type declaration . . . 213
11.17.2.1. Type checking . . . 213
11.17.2.2. Compile-time evaluation . . . 213
11.18. Parser type declaration . . . 213
11.18.1. Type checking . . . 214
11.18.2. Compile-time evaluation . . . 214
11.19. Control type declaration . . . 215
11.19.1. Type checking . . . 215
11.19.2. Compile-time evaluation . . . 215
11.20. Package type declaration . . . 216
11.20.1. Type checking . . . 216
11.20.2. Compile-time evaluation . . . 217
12. L-values 217
12.1. Semantics of l-values . . . 219
12.1.1. Type checking . . . 219
12.1.2. Runtime evaluation . . . 221
12.1.2.1. Creation . . . 222
12.1.2.2. Read . . . 222
12.1.2.3. Write . . . 223
12.2. Identifiers . . . 224
12.2.1. Type checking . . . 224
12.2.2. Creation . . . 224
12.2.3. Read . . . 224
12.2.4. Write . . . 224
12.3. Member accesses . . . 225
12.3.1. Type checking . . . 225
12.3.2. Creation . . . 226
12.3.3. Read . . . 227
12.3.4. Write . . . 228
12.4. Index accesses . . . 229
12.4.1. Type checking . . . 229
12.4.2. Creation . . . 231
12.4.3. Read . . . 231
12.4.4. Write . . . 232
12.5. Bit-slice accesses . . . 233
12.5.1. Type checking . . . 233
12.5.2. Creation . . . 235
12.5.3. Read . . . 235
12.5.4. Write . . . 236
12.6. Parenthesized l-values . . . 236
12.6.1. Type checking . . . 236

12.6.2. Creation . . . 237
13. Statements 237
13.1. Semantics of statements . . . 237
13.1.1. Type checking . . . 237
13.1.2. Compile-time evaluation . . . 239
13.1.3. Runtime evaluation . . . 240
13.2. Empty statements . . . 242
13.2.1. Type checking . . . 242
13.2.2. Compile-time evaluation . . . 242
13.2.3. Runtime evaluation . . . 242
13.3. Assignment statements . . . 242
13.3.1. Type checking . . . 243
13.3.2. Compile-time evaluation . . . 244
13.3.3. Runtime evaluation . . . 244
13.4. Call statements . . . 245
13.4.1. Type checking . . . 246
13.4.2. Compile-time evaluation . . . 247
13.4.3. Runtime evaluation . . . 247
13.5. Direct type invocation . . . 247
13.5.1. Type checking . . . 248
13.5.2. Compile-time evaluation . . . 249
13.6. Return statements . . . 250
13.6.1. Type checking . . . 251
13.6.2. Compile-time evaluation . . . 251
13.6.3. Runtime evaluation . . . 251
13.7. Exit statements . . . 252
13.7.1. Type checking . . . 253
13.7.2. Compile-time evaluation . . . 253
13.7.3. Runtime evaluation . . . 253
13.8. Block statements . . . 253
13.8.1. Type checking . . . 254
13.8.2. Compile-time evaluation . . . 256
13.8.3. Runtime evaluation . . . 258
13.9. Conditional statements . . . 260
13.9.1. Type checking . . . 260
13.9.2. Compile-time evaluation . . . 261
13.9.3. Runtime evaluation . . . 261
13.10. For statements . . . 262
13.10.1. 3-clause for statements . . . 263
13.10.1.1. Type checking . . . 264
13.10.1.2. Compile-time evaluation . . . 268
13.10.1.3. Runtime evaluation . . . 268
13.10.2. for-in statements . . . 272
13.10.2.1. Type checking . . . 273
13.10.2.2. Compile-time evaluation . . . 275
13.10.2.3. Runtime evaluation . . . 275
13.11. Break statements . . . 279
13.11.1. Type checking . . . 279
13.11.2. Compile-time evaluation . . . 279
13.11.3. Runtime evaluation . . . 279

13.12. Continue statements . . . 280
13.12.1. Type checking . . . 280
13.12.2. Compile-time evaluation . . . 280
13.12.3. Runtime evaluation . . . 280
13.13. Switch statements . . . 281
13.13.1. Notes common to all switch statements . . . 283
13.13.2. Switch statement on match-action table result . . . 285
13.13.2.1. Type checking . . . 285
13.13.2.2. Runtime evaluation . . . 287
13.13.3. Switch statement on numeric or enumerated type . . . 289
13.13.3.1. Type checking . . . 290
13.13.3.2. Runtime evaluation . . . 292
13.13.4. Type checking . . . 295
13.13.5. Compile-time evaluation . . . 295
13.13.6. Runtime evaluation . . . 297
14. Expressions 298
14.1. Semantics of expressions . . . 298
14.1.1. Type checking . . . 298
14.1.2. Local compile-time evaluation . . . 300
14.1.3. Compile-time evaluation . . . 301
14.1.4. Runtime evaluation . . . 303
14.2. Literal expressions . . . 305
14.2.1. Type checking . . . 306
14.2.2. Local compile-time evaluation . . . 306
14.2.3. Compile-time evaluation . . . 307
14.2.4. Runtime evaluation . . . 307
14.3. Reference expressions . . . 307
14.3.1. Type checking . . . 308
14.3.2. Local compile-time evaluation . . . 309
14.3.3. Compile-time evaluation . . . 309
14.3.4. Runtime evaluation . . . 310
14.4. Default expressions . . . 311
14.4.1. Type checking . . . 312
14.4.2. Local compile-time evaluation . . . 312
14.4.3. Compile-time evaluation . . . 312
14.4.4. Runtime evaluation . . . 312
14.5. Unary expressions . . . 313
14.5.1. Type checking . . . 313
14.5.1.1. Negation . . . 313
14.5.1.2. Bitwise complement . . . 313
14.5.1.3. Plus and minus . . . 314
14.5.2. Local compile-time evaluation . . . 314
14.5.3. Compile-time evaluation . . . 314
14.5.4. Runtime evaluation . . . 314
14.6. Binary expressions . . . 315
14.6.1. Type checking . . . 315
14.6.1.1. Plus, minus, and multiplication . . . 315
14.6.1.2. Saturating plus and minus . . . 316
14.6.1.3. Division and modulo . . . 316
14.6.1.4. Shift left and right . . . 317

14.6.1.5. Equality and inequality . . . 318
14.6.1.6. Comparison . . . 319
14.6.1.7. Bitwise and, xor, and or . . . 319
14.6.1.8. Concatenation . . . 320
14.6.1.9. Logical and, and or . . . 320
14.6.2. Local compile-time evaluation . . . 321
14.6.3. Compile-time evaluation . . . 321
14.6.4. Runtime evaluation . . . 322
14.7. Ternary expressions . . . 323
14.7.1. Type checking . . . 324
14.7.2. Local compile-time evaluation . . . 324
14.7.3. Compile-time evaluation . . . 325
14.7.4. Runtime evaluation . . . 325
14.8. Cast expressions . . . 325
14.8.1. Type checking . . . 326
14.8.2. Local compile-time evaluation . . . 326
14.8.3. Compile-time evaluation . . . 326
14.8.4. Runtime evaluation . . . 327
14.9. Invalid header expressions . . . 327
14.9.1. Type checking . . . 327
14.9.2. Local compile-time evaluation . . . 327
14.9.3. Compile-time evaluation . . . 328
14.9.4. Runtime evaluation . . . 328
14.10. Sequence expressions . . . 328
14.10.1. Type checking . . . 328
14.10.2. Local compile-time evaluation . . . 330
14.10.3. Compile-time evaluation . . . 330
14.10.4. Runtime evaluation . . . 330
14.11. Record expressions . . . 331
14.11.1. Type checking . . . 331
14.11.2. Local compile-time evaluation . . . 333
14.11.3. Compile-time evaluation . . . 334
14.11.4. Runtime evaluation . . . 334
14.12. Error access expression . . . 335
14.12.1. Type checking . . . 335
14.12.2. Local compile-time evaluation . . . 335
14.12.3. Compile-time evaluation . . . 336
14.12.4. Runtime evaluation . . . 336
14.13. Member access expressions . . . 336
14.13.1. Type checking . . . 336
14.13.2. Local compile-time evaluation . . . 339
14.13.3. Compile-time evaluation . . . 340
14.13.4. Runtime evaluation . . . 342
14.14. Index access expressions . . . 344
14.14.1. Type checking . . . 344
14.14.2. Local compile-time evaluation . . . 346
14.14.3. Compile-time evaluation . . . 347
14.14.4. Runtime evaluation . . . 348
14.15. Bitslice access expressions . . . 349
14.15.1. Type checking . . . 349

14.15.2. Local compile-time evaluation . . . 352
14.15.3. Compile-time evaluation . . . 353
14.15.4. Runtime evaluation . . . 353
14.16. Call expressions . . . 354
14.16.1. Type checking . . . 354
14.16.2. Local compile-time evaluation . . . 357
14.16.3. Compile-time evaluation . . . 359
14.16.4. Runtime evaluation . . . 360
14.17. Parenthesized expressions . . . 361
14.17.1. Type checking . . . 361
14.17.2. Local compile-time evaluation . . . 361
14.17.3. Compile-time evaluation . . . 361
14.17.4. Runtime evaluation . . . 362
14.18. Keyset expressions . . . 362
15. Arguments and type arguments 365
15.1. Semantics of arguments . . . 366
15.1.1. Type checking . . . 366
15.1.2. Local Compile-time evaluation . . . 367
15.1.3. Compile-time evaluation . . . 368
15.1.4. Runtime evaluation . . . 368
15.1.4.1. in and directionless arguments . . . 369
15.1.4.2. out and inout arguments . . . 369
15.2. Semantics of type arguments . . . 370
15.2.1. Type checking . . . 370
16. Abstract parser machine 372
16.1. Parser state machine . . . 372
16.1.1. Sub-parsers . . . 373
16.1.2. Runtime evaluation of a parser . . . 374
16.1.3. Parser state transitions . . . 375
16.2. Parser local declarations . . . 376
16.2.1. Semantics of parser local declarations . . . 376
16.2.1.1. Type checking . . . 376
16.2.1.2. Compile-time evaluation . . . 378
16.2.1.3. Runtime evaluation . . . 379
16.2.2. Constant declarations . . . 380
16.2.2.1. Type checking . . . 380
16.2.2.2. Compile-time evaluation . . . 380
16.2.2.3. Runtime evaluation . . . 380
16.2.3. Instantiations . . . 381
16.2.3.1. Type checking . . . 381
16.2.3.2. Compile-time evaluation . . . 381
16.2.4. Variable declarations . . . 381
16.2.4.1. Type checking . . . 381
16.2.4.2. Compile-time evaluation . . . 381
16.2.4.3. Runtime evaluation . . . 381
16.2.5. Parser value set declarations . . . 382
16.2.5.1. Type checking . . . 382
16.2.5.2. Compile-time evaluation . . . 382
16.3. Parser states . . . 383
16.3.1. Type checking . . . 384

16.3.2. Compile-time evaluation . . . 385
16.3.3. Runtime evaluation . . . 386
16.4. Parser statements . . . 387
16.4.1. Semantics of parser statements . . . 387
16.4.1.1. Type checking . . . 387
16.4.1.2. Compile-time evaluation . . . 388
16.4.1.3. Runtime evaluation . . . 389
16.4.2. Empty statement . . . 390
16.4.2.1. Type checking . . . 390
16.4.2.2. Compile-time evaluation . . . 390
16.4.2.3. Runtime evaluation . . . 390
16.4.3. Assignment statement . . . 390
16.4.3.1. Type checking . . . 390
16.4.3.2. Compile-time evaluation . . . 391
16.4.3.3. Runtime evaluation . . . 391
16.4.4. Call statement . . . 391
16.4.4.1. Type checking . . . 391
16.4.4.2. Compile-time evaluation . . . 392
16.4.4.3. Runtime evaluation . . . 392
16.4.5. Direct type invocation . . . 392
16.4.5.1. Type checking . . . 392
16.4.5.2. Compile-time evaluation . . . 392
16.4.6. Block statement . . . 393
16.4.6.1. Type checking . . . 393
16.4.6.2. Compile-time evaluation . . . 395
16.4.6.3. Runtime evaluation . . . 397
16.4.7. Conditional statement . . . 399
16.4.7.1. Type checking . . . 399
16.4.7.2. Compile-time evaluation . . . 400
16.4.7.3. Runtime evaluation . . . 400
16.5. Parser transition statements . . . 401
16.5.1. Type checking . . . 402
16.5.2. Runtime evaluation . . . 403
16.6. Parser select expression . . . 403
16.6.1. Type checking . . . 405
16.6.1.1. Simple keyset expressions . . . 407
16.6.1.2. Tuple keyset expressions . . . 411
16.6.2. Runtime evaluation . . . 412
16.7. verify . . . 414
16.8. Packet data extraction . . . 414
16.8.1. Fixed-width extraction . . . 415
16.8.2. Variable-width extraction . . . 416
16.8.3. Lookahead . . . 417
16.8.4. Skipping bits . . . 418
17. Abstract control machine 418
17.1. Match-action pipeline machine . . . 419
17.1.1. Sub-controls . . . 419
17.1.2. Runtime evaluation of a control . . . 420
17.2. Control local declarations . . . 421
17.2.1. Semantics of parser local declarations . . . 421

17.2.1.1. Type checking . . . 421
17.2.1.2. Compile-time evaluation . . . 422
17.2.1.3. Runtime evaluation . . . 423
17.2.2. Constant declarations . . . 425
17.2.2.1. Type checking . . . 425
17.2.2.2. Compile-time evaluation . . . 425
17.2.2.3. Runtime evaluation . . . 425
17.2.3. Instantiations . . . 425
17.2.3.1. Type checking . . . 425
17.2.3.2. Compile-time evaluation . . . 426
17.2.4. Variable declarations . . . 426
17.2.4.1. Type checking . . . 426
17.2.4.2. Compile-time evaluation . . . 426
17.2.4.3. Runtime evaluation . . . 426
17.2.5. Action declarations . . . 427
17.2.5.1. Type checking . . . 427
17.2.5.2. Compile-time evaluation . . . 427
17.2.6. Table declarations . . . 427
17.2.6.1. Type checking . . . 427
17.2.6.2. Compile-time evaluation . . . 429
17.3. Match-action tables . . . 430
17.3.1. Semantics of match-action table properties . . . 431
17.3.1.1. Type checking . . . 431
17.3.1.2. Compile-time evaluation . . . 432
17.3.2. Table keys . . . 433
17.3.2.1. Type checking . . . 435
17.3.2.2. Compile-time evaluation . . . 437
17.3.2.3. Runtime evaluation . . . 437
17.3.3. Table actions . . . 438
17.3.3.1. Type checking . . . 440
17.3.3.2. Compile-time evaluation . . . 442
17.3.4. Table default actions . . . 444
17.3.4.1. Type checking . . . 445
17.3.4.2. Compile-time evaluation . . . 447
17.3.5. Table entries . . . 447
17.3.5.1. Entry priorities . . . 449
17.3.5.1.1. Type checking . . . 452
17.3.5.2. Type checking . . . 453
17.3.5.2.1. Type checking table entry keyset . . . 455
17.3.5.2.2. Type checking table entry action reference . . . 463
17.3.5.3. Compile-time evaluation . . . 463
17.3.6. Additional properties . . . 464
17.3.6.1. Type checking . . . 464
17.3.6.2. Compile-time evaluation . . . 465
17.3.7. Match-action table invocation . . . 465
18. Casting 470
18.1. Cast chains . . . 470
18.2. Explicit casts . . . 472
18.3. Implicit casts . . . 484
18.4. Cast insertion . . . 496

18.4.1. Casts to match expected type . . . 497
18.4.2. Casts to ensure type equality . . . 497
18.4.3. Serializable enum reduction . . . 498
18.5. Performing casts . . . 499
18.6. Type equivalence . . . 513
18.6.1. Base types . . . 516
18.6.2. Named types . . . 516
18.6.3. Data types . . . 516
18.6.3.1. List types . . . 516
18.6.3.2. Tuple types . . . 516
18.6.3.3. Array types . . . 516
18.6.3.4. Header stack types . . . 517
18.6.3.5. Struct types . . . 517
18.6.3.6. Header types . . . 517
18.6.3.7. Header union types . . . 517
18.6.3.8. Enum types . . . 517
18.6.4. Object types . . . 518
18.6.4.1. Extern object types . . . 518
18.6.4.2. Parser object types . . . 520
18.6.4.3. Control object types . . . 520
18.6.4.4. Package object types . . . 520
18.6.4.5. Table object types . . . 520
18.6.5. Alias types . . . 520
18.6.5.1. typedef types . . . 520
18.6.5.2. type types . . . 521
18.6.6. Synthesized types . . . 521
18.6.6.1. Default types . . . 521
18.6.6.2. Invalid header types . . . 521
18.6.6.3. Sequence types . . . 521
18.6.6.4. Record types . . . 522
18.6.6.5. Set types . . . 522
18.6.6.6. Table metadata types . . . 522
19. Calls 522
19.1. Call targets . . . 523
19.2. Argument alignment . . . 526
19.2.1. Parameter classification . . . 526
19.2.2. Aligning arguments . . . 526
19.3. Overload resolution . . . 528
19.3.1. Positional arguments . . . 530
19.3.2. Named arguments . . . 531
19.3.3. Non-overloaded targets . . . 533
19.4. Callee resolution . . . 534
19.4.1. Resolving constructors . . . 534
19.4.2. Resolving callables . . . 536
19.4.2.1. Actions . . . 539
19.4.2.2. Functions . . . 540
19.4.2.3. Built-in methods . . . 542
19.4.2.3.1. minSizeInBits, maxSizeInBits, minSizeInBytes, and maxSizeInBytes . . . 542
19.4.2.3.2. push_front and pop_front on header stacks . . . 543
19.4.2.3.3. isValid, setValid, and setInvalid on headers and header unions . . . 545

19.4.2.4. Extern methods . . . 547
19.4.2.5. Parser apply methods . . . 549
19.4.2.6. Control apply methods . . . 551
19.4.2.7. Table apply methods . . . 552
19.4.2.8. Parenthesized callables . . . 553
19.5. Type argument inference . . . 554
19.5.1. Constraint generation . . . 556
19.5.2. Constraint merging . . . 558
19.5.3. Constraint resolution . . . 559
19.6. Call convention . . . 560
19.6.1. Type checking . . . 561
19.6.1.1. in . . . 562
19.6.1.2. out and inout . . . 563
19.6.1.3. Directionless . . . 563
19.6.2. Runtime evaluation of copy-in . . . 563
19.6.2.1. in and directionless . . . 566
19.6.2.2. inout . . . 567
19.6.2.3. out . . . 567
19.6.3. Compile-time evaluation of copy-in . . . 568
19.6.4. Runtime evaluation of copy-out . . . 570
19.6.4.1. in and directionless . . . 572
19.6.4.2. out and inout . . . 572
19.7. Constructor calls . . . 572
19.7.1. Extern objects . . . 574
19.7.2. Parser objects . . . 575
19.7.3. Control objects . . . 576
19.7.4. Package objects . . . 577
19.8. Callable calls . . . 578
19.8.1. Actions . . . 580
19.8.1.1. Type checking . . . 580
19.8.1.2. Runtime evaluation . . . 580
19.8.2. Functions . . . 582
19.8.2.1. Type checking . . . 582
19.8.2.2. Runtime evaluation . . . 582
19.8.3. Extern functions . . . 584
19.8.3.1. Type checking . . . 584
19.8.3.2. Runtime evaluation . . . 584
19.8.4. Built-in methods . . . 585
19.8.4.1. Type checking . . . 585
19.8.4.2. Runtime evaluation . . . 585
19.8.4.2.1. minSizeInBits, maxSizeInBits, minSizeInBytes, and maxSizeInBytes . . . 586
19.8.4.2.2. push_front and pop_front on header stacks . . . 586
19.8.4.2.3. isValid, setValid, and setInvalid on headers . . . 588
19.8.4.2.4. isValid on header unions . . . 589
19.8.5. Extern methods . . . 590
19.8.5.1. Type checking . . . 590
19.8.5.2. Runtime evaluation . . . 591
19.8.6. Parser apply methods . . . 592
19.8.6.1. Type checking . . . 592
19.8.6.2. Runtime evaluation . . . 593

19.8.7. Control apply methods . . . 593
19.8.7.1. Type checking . . . 593
19.8.7.2. Runtime evaluation . . . 594
19.8.8. Table apply methods . . . 595
19.8.8.1. Type checking . . . 595
19.8.8.2. Runtime evaluation . . . 596
20. Operations 596
20.1. Unary operations . . . 597
20.2. Binary operations . . . 598
20.3. Default values . . . 619
20.4. Bit-string access operations . . . 621
20.5. Compile-time size determination . . . 624
20.6. Keyset matching . . . 627
21. Annotations 629
21.1. Bodies of Unstructured Annotations . . . 630
21.1.1. Structured Annotation Examples . . . 630
21.2. Predefined annotations . . . 632
21.2.1. Optional parameter annotations . . . 632
21.2.2. Annotations on the table action list . . . 633
21.2.3. Control-plane API annotations . . . 633
21.2.3.1. Restrictions . . . 633
21.2.4. Concurrency control annotations . . . 635
21.2.5. Branch prediction annotations . . . 635
21.2.6. Value set annotations . . . 636
21.2.7. Extern function/method annotations . . . 636
21.2.8. Deprecated annotation . . . 636
21.2.9. No warnings annotation . . . 637
Appendix A: Revision History 637
A.1. Summary of changes made in unreleased version . . . 637
A.2. Summary of changes made in version 1.2.5, released October 11, 2024 . . . 637
A.3. Summary of changes made in version 1.2.4,released May 15, 2023 . . . 637
A.4. Summary of changes made in version 1.2.3, released July 11, 2022 . . . 638
A.5. Summary of changes made in version 1.2.2, released May 17, 2021 . . . 639
A.6. Summary of changes made in version 1.2.1, released June 11, 2020 . . . 640
A.7. Summary of changes made in version 1.2.0, released October 14, 2019 . . . 640
A.8. Summary of changes made in version 1.1.0, released November 26, 2017 . . . 641
A.9. Initial version 1.0.0, released May 17, 2017 . . . 641
Appendix B: P4 reserved keywords 641
Appendix C: P4 reserved annotations 642
Appendix D: P4 core library 642
Appendix E: Restrictions on compile time and run time calls 643
E.1. Restrictions on run time calls . . . 644
E.2. Restrictions on instantiations . . . 646
Appendix F: P4 grammar 648
Appendix G: $P4_{IR}$ grammar 660
Appendix H: Built-in meta-functions 671
H.1. Core notation and collections . . . 671
Appendix I: Type meta-functions 676
I.1. Free type variables . . . 676
I.2. Substitution . . . 682

Appendix J: Subexpressions 693
Appendix K: Flattening and unflattening 695
Appendix L: Lifting P4IR to surface syntax 712
Appendix M: Context and environment operations 753
M.1. Typing context. . . . 753
M.2. Instantiation context 763
M.3. Execution context 768
Appendix N: Structural projections and identifiers 777
Appendix O: Value serialization 785

Abstract

P4 is a language for programming the data plane of network devices. This document provides a precise definition of the $P4_{16}$ language, which is the 2016 revision of the P4 language (http://p4.org). The target audience for this document includes developers who want to write compilers, simulators, IDEs, and debuggers for P4 programs. This document may also be of interest to P4 programmers who are interested in understanding the syntax and semantics of the language at a deeper level.

1. Scope

This specification document defines the structure and interpretation of programs in the $P4_{16}$ language. It defines the syntax, semantic rules, and requirements for conformant implementations of the language.

It does not define:

- Mechanisms by which P4 programs are compiled, loaded, and executed on packet-processing systems,
- Mechanisms by which data are received by one packet-processing system and delivered to another system,
- Mechanisms by which the control plane manages the match-action tables and other stateful objects defined by P4 programs,
- The size or complexity of P4 programs,
- The minimal requirements of packet-processing systems that are capable of providing a conformant implementation.

It is understood that some implementations may be unable to implement the behavior defined here in all cases, or may provide options to eliminate some safety guarantees in exchange for better performance or handling larger programs. They should document where they deviate from this specification.

2. Terms, definitions, and symbols

Throughout this document, the following terms will be used:

- **Architecture**: A set of P4-programmable components and the data plane interfaces between them.
- **Control plane**: A class of algorithms and the corresponding input and output data that are concerned with the provisioning and configuration of the data plane.
- **Data plane**: A class of algorithms that describe transformations on packets by packet-processing systems.
- **Metadata**: Intermediate data generated during execution of a P4 program.
- **Packet**: A network packet is a formatted unit of data carried by a packet-switched network.
- **Packet header**: Formatted data at the beginning of a packet. A given packet may contain a sequence of packet headers representing different network protocols.
- **Packet payload**: Packet data that follows the packet headers.
- **Packet-processing system**: A data-processing system designed for processing network packets. In general, packet-processing systems implement control plane and data plane algorithms.
- **Target**: A packet-processing system capable of executing a P4 program.

All terms defined explicitly in this document should not be understood to refer implicitly to similar terms defined elsewhere. Conversely, any terms not defined explicitly in this document should be interpreted

according to generally recognizable sources—e.g., IETF RFCs.

3. Overview

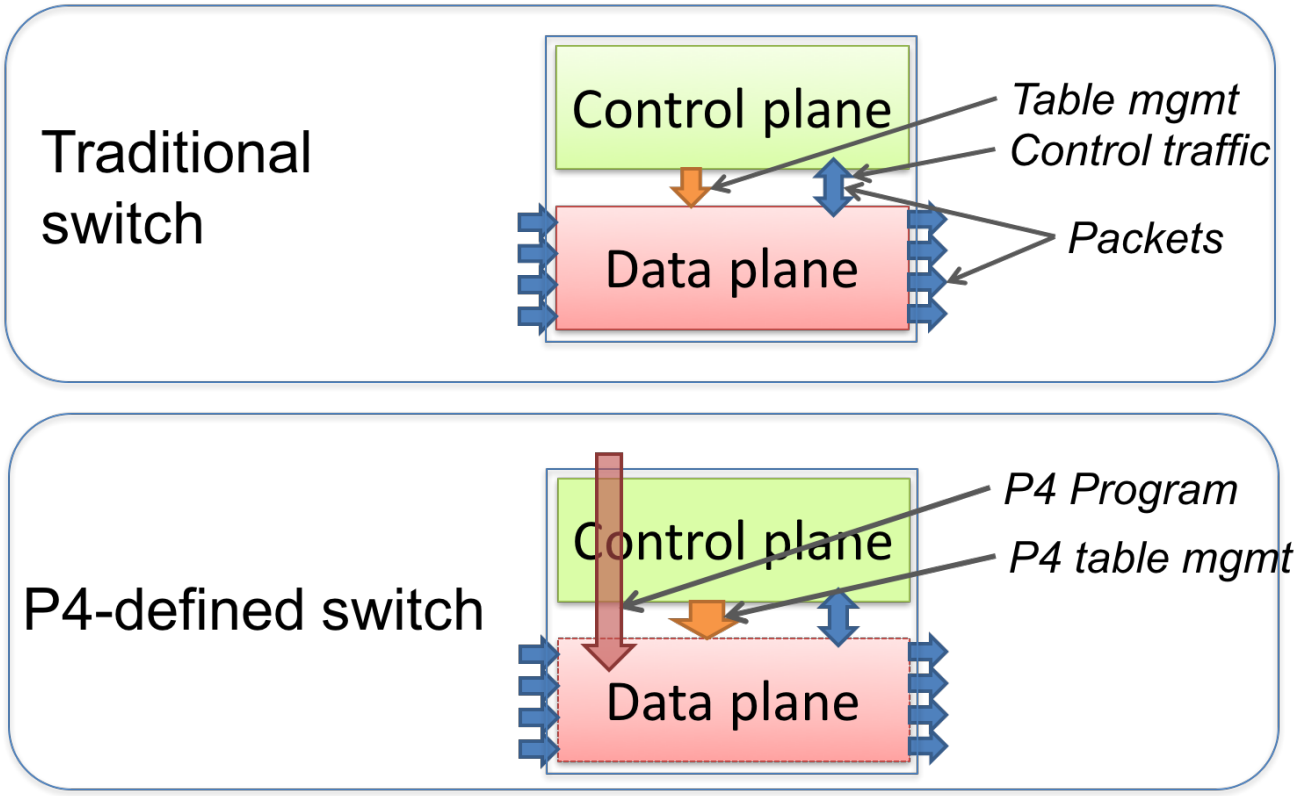

Figure 1. Traditional switches vs. programmable switches.

P4 is a language for expressing how packets are processed by the data plane of a programmable forwarding element such as a hardware or software switch, network interface card, router, or network appliance. The name P4 comes from the original paper that introduced the language, "Programming Protocol-independent Packet Processors," https://arxiv.org/pdf/1312.1719.pdf. While P4 was initially designed for programming switches, its scope has been broadened to cover a large variety of devices. In the rest of this document we use the generic term *target* for all such devices.

Many targets implement both a control plane and a data plane. P4 is designed to specify only the data plane functionality of the target. P4 programs also partially define the interface by which the control plane and the data-plane communicate, but P4 cannot be used to describe the control-plane functionality of the target. In the rest of this document, when we talk about P4 as "programming a target", we mean "programming the data plane of a target".

As a concrete example of a target, Figure 1 illustrates the difference between a traditional fixed-function switch and a P4-programmable switch. In a traditional switch the manufacturer defines the data-plane functionality. The control-plane controls the data plane by managing entries in tables (e.g. routing tables), configuring specialized objects (e.g. meters), and by processing control-packets (e.g. routing protocol packets) or asynchronous events, such as link state changes or learning notifications.

A P4-programmable switch differs from a traditional switch in two essential ways:

- The data plane functionality is not fixed in advance but is defined by a P4 program. The data plane is configured at initialization time to implement the functionality described by the P4 program (shown by the long red arrow) and has no built-in knowledge of existing network protocols.
- The control plane communicates with the data plane using the same channels as in a fixed-function device, but the set of tables and other objects in the data plane are no longer fixed, since they are defined by a P4 program. The P4 compiler generates the API that the control plane uses to

communicate with the data plane.

Hence, P4 can be said to be protocol independent, but it enables programmers to express a rich set of protocols and other data plane behaviors.

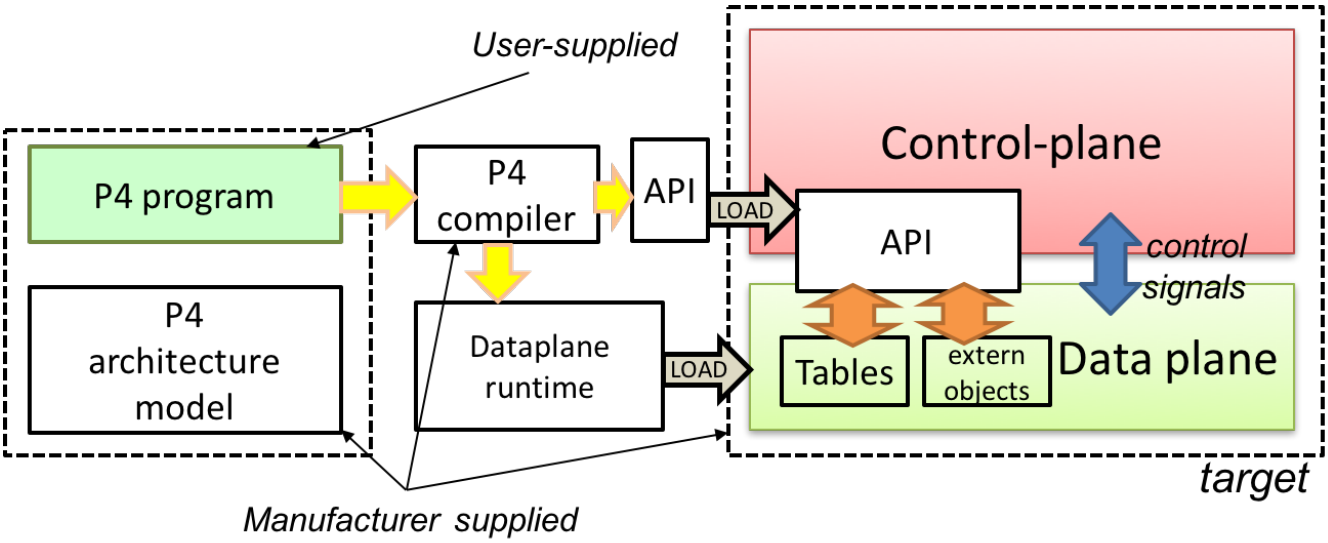

Figure 2. Programming a target with P4.

The core abstractions provided by the P4 language are:

- **Header types** describe the format (the set of fields and their sizes) of each header within a packet.
- **Parsers** describe the permitted sequences of headers within received packets, how to identify those header sequences, and the headers and fields to extract from packets.
- **Tables** associate user-defined keys with actions. P4 tables generalize traditional switch tables; they can be used to implement routing tables, flow lookup tables, access-control lists, and other user-defined table types, including complex multi-variable decisions.
- **Actions** are code fragments that describe how packet header fields and metadata are manipulated. Actions can include data, which is supplied by the control-plane at runtime.
- **Match-action units** perform the following sequence of operations:
  - Construct lookup keys from packet fields or computed metadata,
  - Perform table lookup using the constructed key, choosing an action (including the associated data) to execute, and
  - Finally, execute the selected action.
- **Control flow** expresses an imperative program that describes packet-processing on a target, including the data-dependent sequence of match-action unit invocations. Deparsing (packet reassembly) can also be performed using a control flow.
- **Extern objects** are architecture-specific constructs that can be manipulated by P4 programs through well-defined APIs, but whose internal behavior is hard-wired (e.g., checksum units) and hence not programmable using P4.
- **User-defined metadata**: user-defined data structures associated with each packet.
- **Intrinsic metadata**: metadata provided by the architecture associated with each packet—e.g., the input port where a packet has been received.

Figure 2 shows a typical tool workflow when programming a target using P4.

Target manufacturers provide the hardware or software implementation framework, an architecture definition, and a P4 compiler for that target. P4 programmers write programs for a specific architecture, which defines a set of P4-programmable components on the target as well as their external data plane

interfaces.

Compiling a set of P4 programs produces two artifacts:

- a data plane configuration that implements the forwarding logic described in the input program and
- an API for managing the state of the data plane objects from the control plane

P4 is a domain-specific language that is designed to be implementable on a large variety of targets including programmable network interface cards, FPGAs, software switches, and hardware ASICs. As such, the language is restricted to constructs that can be efficiently implemented on all of these platforms.

Assuming a fixed cost for table lookup operations and interactions with extern objects, all P4 programs (i.e., parsers and controls) execute a constant number of operations for each byte of an input packet received and analyzed. Although parsers may contain loops, provided some header is extracted on each cycle, the packet itself provides a bound on the total execution of the parser. In other words, under these assumptions, the computational complexity of a P4 program is linear in the total size of all headers, and never depends on the size of the state accumulated while processing data (e.g., the number of flows, or the total number of packets processed). These guarantees are necessary (but not sufficient) for enabling fast packet processing across a variety of targets.

P4 conformance of a target is defined as follows: if a specific target T supports only a subset of the P4 programming language, say $P4^T$, programs written in $P4^T$ executed on the target should provide the exact same behavior as is described in this document. Note that P4 conformant targets can provide arbitrary P4 language extensions and extern elements.

3.1. Benefits of P4

Compared to state-of-the-art packet-processing systems (e.g., based on writing microcode on top of custom hardware), P4 provides a number of significant advantages:

- **Flexibility**: P4 makes many packet-forwarding policies expressible as programs, in contrast to traditional switches, which expose fixed-function forwarding engines to their users.
- **Expressiveness**: P4 can express sophisticated, hardware-independent packet processing algorithms using solely general-purpose operations and table look-ups. Such programs are portable across hardware targets that implement the same architectures (assuming sufficient resources are available).
- **Resource mapping and management**: P4 programs describe storage resources abstractly (e.g., IPv4 source address); compilers map such user-defined fields to available hardware resources and manage low-level details such as allocation and scheduling.
- **Software engineering**: P4 programs provide important benefits such as type checking, information hiding, and software reuse.
- **Component libraries**: Component libraries supplied by manufacturers can be used to wrap hardware-specific functions into portable high-level P4 constructs.
- **Decoupling hardware and software evolution**: Target manufacturers may use abstract architectures to further decouple the evolution of low-level architectural details from high-level processing.
- **Debugging**: Manufacturers can provide software models of an architecture to aid in the development and debugging of P4 programs.

3.2. P4 language evolution: comparison to previous versions (P4 v1.0/v1.1)

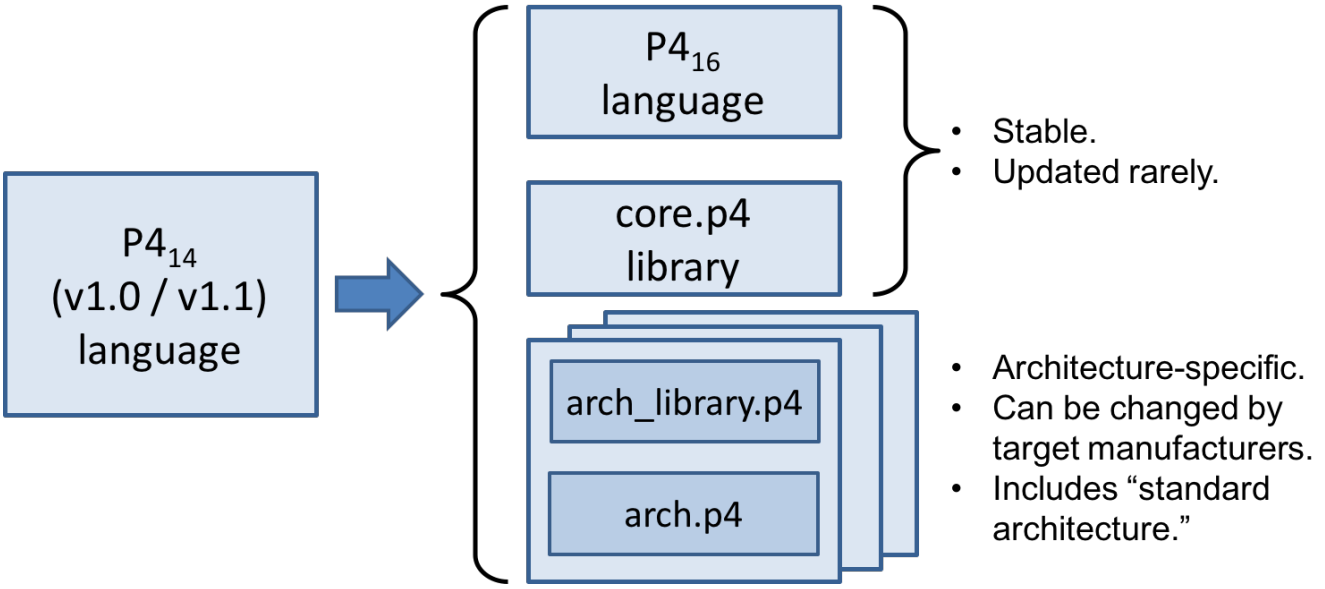

Figure 3. Evolution of the language between versions $P4_{14}$ (versions 1.0 and 1.1) and $P4_{16}$.

Compared to $P4_{14}$, the earlier version of the language, $P4_{16}$ makes a number of significant, backwards-incompatible changes to the syntax and semantics of the language. The evolution from the previous version ($P4_{14}$) to the current one ($P4_{16}$) is depicted in Figure 3. In particular, a large number of language features have been eliminated from the language and moved into libraries including counters, checksum units, meters, etc.

Hence, the language has been transformed from a complex language (more than 70 keywords) into a relatively small core language (less than 40 keywords, shown in Appendix B) accompanied by a library of fundamental constructs that are needed for writing most P4.

The v1.1 version of P4 introduced a language construct called extern that can be used to describe library elements. Many constructs defined in the v1.1 language specification will thus be transformed into such library elements (including constructs that have been eliminated from the language, such as counters and meters). Some of these extern objects are expected to be standardized, and they will be in the scope of a future document describing a standard library of P4 elements. In this document we provide several examples of extern constructs. $P4_{16}$ also introduces and repurposes some v1.1 language constructs for describing the programmable parts of an architecture. These language constructs are: parser, state, control, and package.

One important goal of the $P4_{16}$ language revision is to provide a **stable** language definition. In other words, we strive to ensure that all programs written in $P4_{16}$ will remain syntactically correct and behave identically when treated as programs for future versions of the language. Moreover, if some future version of the language requires breaking backwards compatibility, we will seek to provide an easy path for migrating $P4_{16}$ programs to the new version.

4. Architecture Model

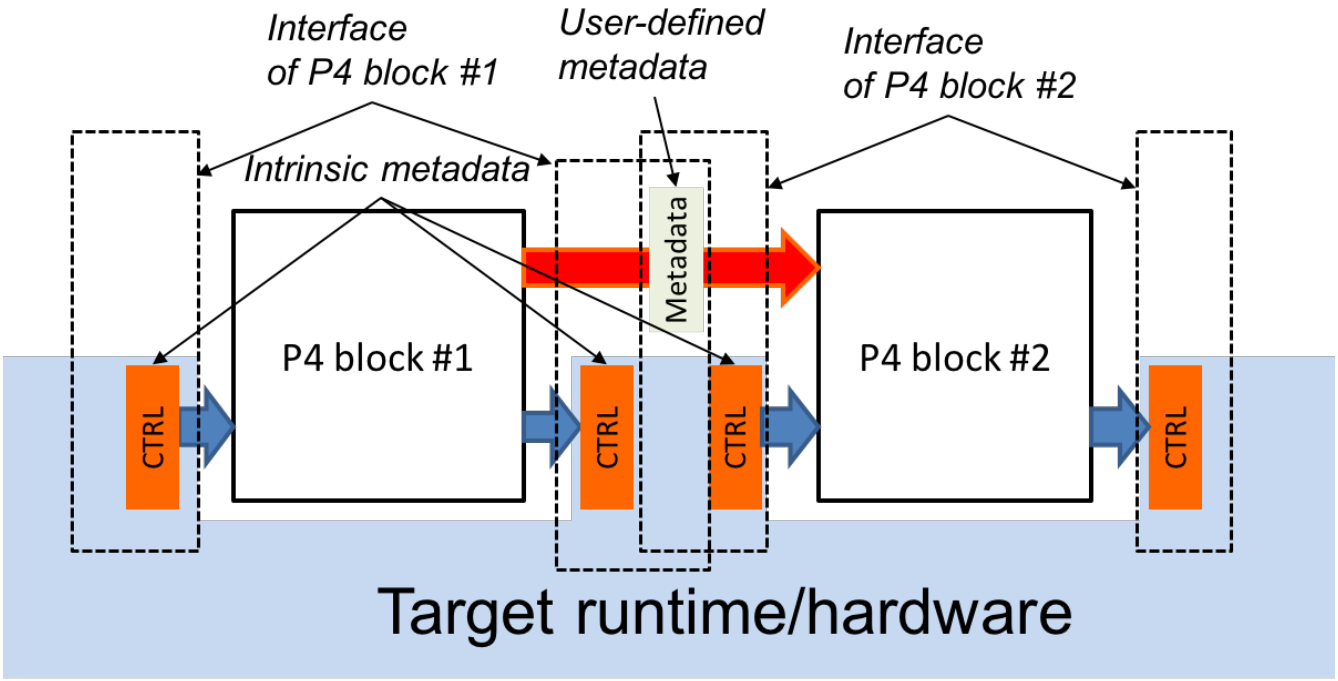

Figure 4. P4 program interfaces.

The *P4 architecture* identifies the P4-programmable blocks (e.g., parser, ingress control flow, egress control flow, deparser, etc.) and their data plane interfaces.

The P4 architecture can be thought of as a contract between the program and the target. Each manufacturer must therefore provide both a P4 compiler as well as an accompanying architecture definition for their target. (We expect that P4 compilers can share a common front-end that handles all architectures). The architecture definition does not have to expose the entire programmable surface of the data plane—a manufacturer may even choose to provide multiple definitions for the same hardware device, each with different capabilities (e.g., with or without multicast support).

Figure 4 illustrates the data plane interfaces between P4-programmable blocks. It shows a target that has two programmable blocks (#1 and #2). Each block is programmed through a separate fragment of P4 code. The target interfaces with the P4 program through a set of control registers or signals. Input controls provide information to P4 programs (e.g., the input port that a packet was received from), while output controls can be written to by P4 programs to influence the target behavior (e.g., the output port where a packet has to be directed). Control registers/signals are represented in P4 as *intrinsic metadata*. P4 programs can also store and manipulate data pertaining to each packet as *user-defined metadata*.

The behavior of a P4 program can be fully described in terms of transformations that map vectors of bits to vectors of bits. To actually process a packet, the architecture model interprets the bits that the P4 program writes to intrinsic metadata. For example, to cause a packet to be forwarded on a specific output port, a P4 program may need to write the index of an output port into a dedicated control register. Similarly, to cause a packet to be dropped, a P4 program may need to set a "drop" bit into another dedicated control register. Note that the details of how intrinsic metadata are interpreted is architecture-specific.

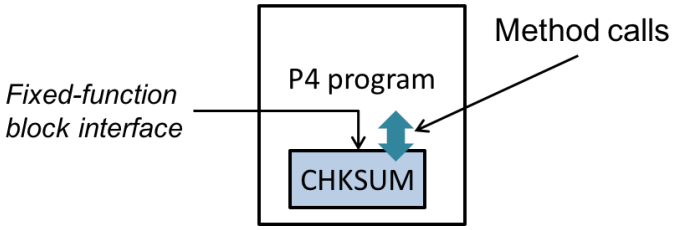

Figure 5. P4 program invoking the services of a fixed-function object.

P4 programs can invoke services implemented by extern objects and functions provided by the architecture. Figure 5 depicts a P4 program invoking the services of a built-in checksum computation unit on a target. The implementation of the checksum unit is not specified in P4, but its interface is. In general, the interface for an extern object describes each operation it provides, as well as their parameter and return types.

In general, P4 programs are not expected to be portable across different architectures. For example, executing a P4 program that broadcasts packets by writing into a custom control register will not function correctly on a target that does not have the control register. However, P4 programs written for a given architecture should be portable across all targets that faithfully implement the corresponding model, provided there are sufficient resources.

4.1. Standard architectures

We expect that the P4 community will evolve a small set of standard architecture models pertaining to specific verticals. Wide adoption of such standard architectures will promote portability of P4 programs across different targets. However, defining these standard architectures is outside of the scope of this document.

4.2. Data plane interfaces

To describe a functional block that can be programmed in P4, the architecture includes a type declaration that specifies the interfaces between the block and the other components in the architecture. For example, the architecture might contain a declaration such as the following:

```
control MatchActionPipe<H>(in bit<4> inputPort,
                           inout H parsedHeaders,
                           out bit<4> outputPort);
```

This type declaration describes a block named MatchActionPipe that can be programmed using a data-dependent sequence of match-action unit invocations and other imperative constructs (indicated by the control keyword). The interface between the MatchActionPipe block and the other components of the architecture can be read off from this declaration:

- The first parameter is a 4-bit value named inputPort. The direction in indicates that this parameter is an input that cannot be modified.
- The second parameter is an object of type H named parsedHeaders, where H is a type variable representing the headers that will be defined later by the P4 programmer. The direction inout indicates that this parameter is both an input and an output.
- The third parameter is a 4-bit value named outputPort. The direction out indicates that this parameter is an output whose value is undefined initially but can be modified.

4.3. Extern objects and functions

P4 programs can also interact with objects and functions provided by the architecture. Such objects are

described using the extern construct, which describes the interfaces that such objects expose to the data-plane.

An extern object describes a set of methods that are implemented by an object, but not the implementation of these methods (i.e., it is similar to an abstract class in an object-oriented language). For example, the following construct could be used to describe the operations offered by an incremental checksum unit:

```
extern Checksum16 {
    Checksum16();               // constructor
    void clear();               // prepare unit for computation
    void update<T>(in T data); // add data to checksum
    void remove<T>(in T data); // remove data from existing checksum
    bit<16> get(); // get the checksum for the data added since last clear
}
```

5. Example: A very simple switch

As an example to illustrate the features of architectures, consider implementing a very simple switch in P4. We will first describe the architecture of the switch and then write a complete P4 program that specifies the data plane behavior of the switch. This example demonstrates many important features of the P4 programming language.

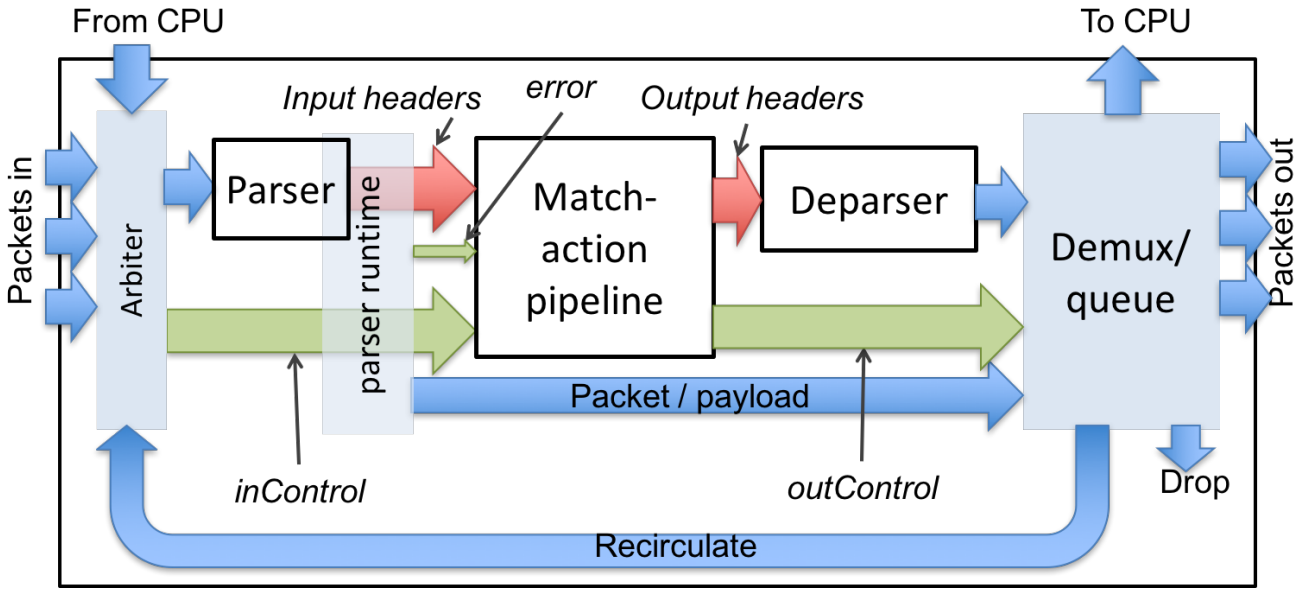

Figure 6. The Very Simple Switch (VSS) architecture.

We call our architecture the "Very Simple Switch" (VSS). Figure 6 is a diagram of this architecture. There is nothing inherently special about VSS—it is just a pedagogical example that illustrates how programmable switches can be described and programmed in P4. VSS has a number of fixed-function blocks (shown in cyan in our example), whose behavior is described in Section 5.2. The white blocks are programmable using P4.

VSS receives packets through one of 8 input Ethernet ports, through a recirculation channel, or from a port connected directly to the CPU. VSS has one single parser, feeding into a single match-action pipeline, which feeds into a single deparser. After exiting the deparser, packets are emitted through one of 8 output Ethernet ports or one of 3 "special" ports:

- Packets sent to the "CPU port" are sent to the control plane
- Packets sent to the "Drop port" are discarded

- Packets sent to the "Recirculate port" are re-injected in the switch through a special input port

The white blocks in the figure are programmable, and the user must provide a corresponding P4 program to specify the behavior of each such block. The red arrows indicate the flow of user-defined data. The cyan blocks are fixed-function components. The green arrows are data plane interfaces used to convey information between the fixed-function blocks and the programmable blocks—exposed in the P4 program as intrinsic metadata.

5.1. Very Simple Switch Architecture

The following P4 program provides a declaration of VSS in P4, as it would be provided by the VSS manufacturer. The declaration contains several type declarations, constants, and finally declarations for the three programmable blocks; the code uses syntax highlighting. The programmable blocks are described by their types; the implementation of these blocks has to be provided by the switch programmer.

```
// File "very_simple_switch_model.p4"
// Very Simple Switch P4 declaration
// core library needed for packet_in and packet_out definitions
# include <core.p4>
/* Various constants and structure declarations */
/* ports are represented using 4-bit values */
typedef bit<4> PortId;
/* only 8 ports are "real" */
const PortId REAL_PORT_COUNT = 4w8;  // 4w8 is the number 8 in 4 bits
/* metadata accompanying an input packet */
struct InControl {
    PortId inputPort;
}
/* special input port values */
const PortId RECIRCULATE_IN_PORT = 0xD;
const PortId CPU_IN_PORT = 0xE;
/* metadata that must be computed for outgoing packets */
struct OutControl {
    PortId outputPort;
}
/* special output port values for outgoing packet */
const PortId DROP_PORT = 0xF;
const PortId CPU_OUT_PORT = 0xE;
const PortId RECIRCULATE_OUT_PORT = 0xD;
/* Prototypes for all programmable blocks */
/**
 * Programmable parser.
 * @param <H> type of headers; defined by user
 * @param b input packet
 * @param parsedHeaders headers constructed by parser
 */
parser Parser<H>(packet_in b,
                 out H parsedHeaders);
/**
 * Match-action pipeline
 * @param <H> type of input and output headers
 * @param headers headers received from the parser and sent to the deparser
 * @param parseError error that may have surfaced during parsing
 * @param inCtrl information from architecture, accompanying input packet
 * @param outCtrl information for architecture, accompanying output packet
 */
control Pipe<H>(inout H headers,
                in error parseError,// parser error
                in InControl inCtrl,// input port
                out OutControl outCtrl); // output port
/**
 * VSS deparser.
 * @param <H> type of headers; defined by user
```

```
 * @param b output packet
 * @param outputHeaders headers for output packet
 */
control Deparser<H>(inout H outputHeaders,
                    packet_out b);
/**
 * Top-level package declaration - must be instantiated by user.
 * The arguments to the package indicate blocks that
 * must be instantiated by the user.
 * @param <H> user-defined type of the headers processed.
 */
package VSS<H>(Parser<H> p,
               Pipe<H> map,
               Deparser<H> d);
// Architecture-specific objects that can be instantiated
// Checksum unit
extern Checksum16 {
    Checksum16();              // constructor
    void clear();              // prepare unit for computation
    void update<T>(in T data); // add data to checksum
    void remove<T>(in T data); // remove data from existing checksum
    bit<16> get(); // get the checksum for the data added since last clear
}
```

Let us describe some of these elements:

- The included file core.p4 is described in more detail in Appendix D. It defines some standard data-types and error codes.
- bit<4> is the type of bit-strings with 4 bits.
- The syntax 4w0xF indicates the value 15 represented using 4 bits. An alternative notation is 4w15. In many circumstances the width modifier can be omitted, writing just 15.
- error is a built-in P4 type for holding error codes
- Next follows the declaration of a parser:

  ```
  parser Parser<H>(packet_in b, out H parsedHeaders);
  ```

  This declaration describes the interface for a parser, but not yet its implementation, which will be provided by the programmer. The parser reads its input from a packet_in, which is a pre-defined P4 extern object that represents an incoming packet, declared in the core.p4 library. The parser writes its output (the out keyword) into the parsedHeaders argument. The type of this argument is H, yet unknown—it will also be provided by the programmer.

- The declaration

  ```
  control Pipe<H>(inout H headers,
                  in error parseError,
                  in InControl inCtrl,
                  out OutControl outCtrl);
  ```

  describes the interface of a Match-Action pipeline named Pipe.

The pipeline receives three inputs: the headers headers, a parser error parseError, and the inCtrl control data. Figure 6 indicates the different sources of these pieces of information. The pipeline writes its outputs into outCtrl, and it must update in place the headers to be consumed by the deparser.

- The top-level package is called VSS; in order to program a VSS, the user will have to instantiate a package of this type (shown in the next section). The top-level package declaration also depends on a

type variable H:

```
package VSS<H>
```

A type variable indicates a type yet unknown that must be provided by the user at a later time. In this case H is the type of the set of headers that the user program will be processing; the parser will produce the parsed representation of these headers, and the match-action pipeline will update the input headers in place to produce the output headers.

- The package VSS declaration has three complex parameters, of types Parser, Pipe, and Deparser respectively; which are exactly the declarations we have just described. In order to program the target one has to supply values for these parameters.
- In this program the inCtrl and outCtrl structures represent control registers. The content of the headers structure is stored in general-purpose registers.
- The extern Checksum16 declaration describes an extern object whose services can be invoked to compute checksums.

5.2. Very Simple Switch Architecture Description

In order to fully understand VSS's behavior and write meaningful P4 programs for it, and for implementing a control plane, we also need a full behavioral description of the fixed-function blocks. This section can be seen as a simple example illustrating all the details that have to be handled when writing an architecture description. The P4 language is not intended to cover the description of all such functional blocks—the language can only describe the interfaces between programmable blocks and the architecture. For the current program, this interface is given by the Parser, Pipe, and Deparser declarations. In practice we expect that the complete description of the architecture will be provided as an executable program and/or diagrams and text; in this document we will provide informal descriptions in English.

5.2.1. Arbiter block

The input arbiter block performs the following functions:

- It receives packets from one of the physical input Ethernet ports, from the control plane, or from the input recirculation port.
- For packets received from Ethernet ports, the block computes the Ethernet trailer checksum and verifies it. If the checksum does not match, the packet is discarded. If the checksum does match, it is removed from the packet payload.
- Receiving a packet involves running an arbitration algorithm if multiple packets are available.
- If the arbiter block is busy processing a previous packet and no queue space is available, input ports may drop arriving packets, without indicating the fact that the packets were dropped in any way.
- After receiving a packet, the arbiter block sets the inCtrl.inputPort value that is an input to the match-action pipeline with the identity of the input port where the packet originated. Physical Ethernet ports are numbered 0 to 7, while the input recirculation port has a number 13 and the CPU port has the number 14.

5.2.2. Parser runtime block

The parser runtime block works in concert with the parser. It provides an error code to the match-action pipeline, based on the parser actions, and it provides information about the packet payload (e.g., the size of the remaining payload data) to the demux block. As soon as a packet's processing is completed by the parser, the match-action pipeline is invoked with the associated metadata as inputs (packet headers and user-defined metadata).

5.2.3. Demux block

The core functionality of the demux block is to receive the headers for the outgoing packet from the deparser and the packet payload from the parser, to assemble them into a new packet and to send the result to the correct output port. The output port is specified by the value of outCtrl.ouputPort, which is set by the match-action pipeline.

- Sending the packet to the drop port causes the packet to disappear.
- Sending the packet to an output Ethernet port numbered between 0 and 7 causes it to be emitted on the corresponding physical interface. The packet may be placed in a queue if the output interface is already busy emitting another packet. When the packet is emitted, the physical interface computes a correct Ethernet checksum trailer and appends it to the packet.
- Sending a packet to the output CPU port causes the packet to be transferred to the control plane. In this case, the packet that is sent to the CPU is the **original input packet**, and not the packet received from the deparser—the latter packet is discarded.
- Sending the packet to the output recirculation port causes it to appear at the input recirculation port. Recirculation is useful when packet processing cannot be completed in a single pass.
- If the outputPort has an illegal value (e.g., 9), the packet is dropped.
- Finally, if the demux unit is busy processing a previous packet and there is no capacity to queue the packet coming from the deparser, **the demux unit may drop the packet**, irrespective of the output port indicated.

Please note that some of the behaviors of the demux block may be unexpected—we have highlighted them in bold. We are not specifying here several important behaviors related to queue size, arbitration, and timing, which also influence the packet processing.

The arrow shown from the parser runtime to the demux block represents an additional information flow from the parser to the demux: the packet being processed as well as the offset within the packet where parsing ended (i.e., the start of the packet payload).

5.2.4. Available extern blocks

The VSS architecture provides an incremental checksum extern block, called Checksum16. The checksum unit has a constructor and four methods:

- clear(): prepares the unit for a new computation
- update<T>(in T data): add some data to be checksummed. The data must be either a bit-string, a header-typed value, or a struct containing such values. The fields in the header/struct are concatenated in the order they appear in the type declaration.
- get(): returns the 16-bit one's complement checksum. When this function is invoked the checksum must have received an integral number of bytes of data.
- remove<T>(in T data): assuming that data was used for computing the current checksum, data is removed from the checksum.

5.3. A complete Very Simple Switch program

Here we provide a complete P4 program that implements basic forwarding for IPv4 packets on the VSS architecture. This program does not utilize all of the features provided by the architecture—e.g., recirculation—but it does use preprocessor #include directives (see [sec-preprocessor]).

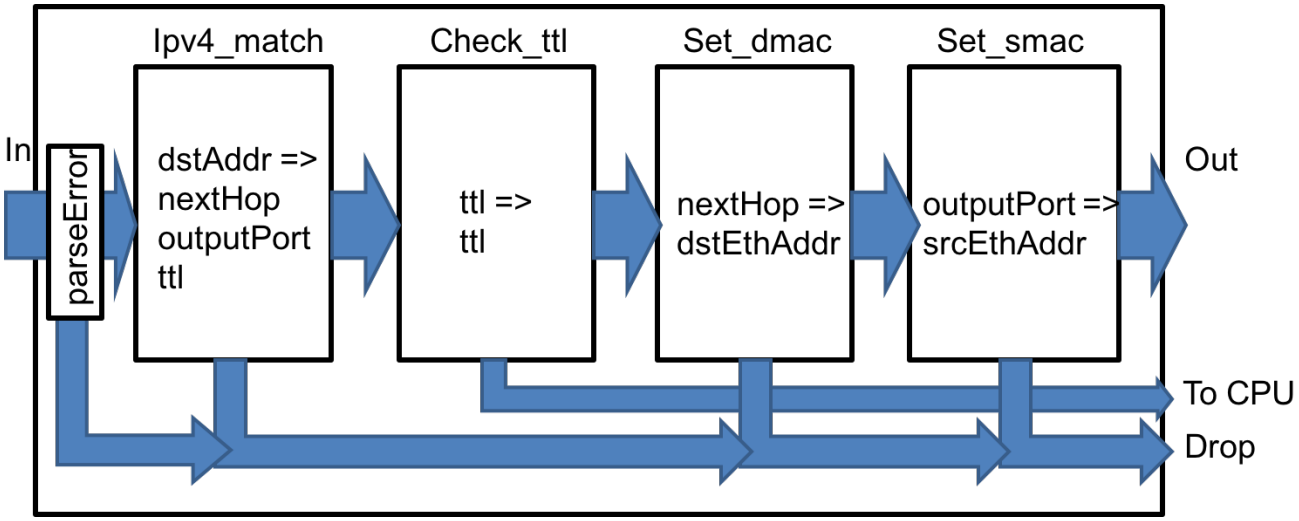

Figure 7. Diagram of the match-action pipeline expressed by the VSS P4 program.

The parser attempts to recognize an Ethernet header followed by an IPv4 header. If either of these headers are missing, parsing terminates with an error. Otherwise it extracts the information from these headers into a Parsed_packet structure. The match-action pipeline is shown in Figure 7; it comprises four match-action units (represented by the P4 table keyword):

- If any parser error has occurred, the packet is dropped (i.e., by assigning
- outputPort to DROP_PORT)
- The first table uses the IPv4 destination address to determine the
- outputPort and the IPv4 address of the next hop. If this lookup fails, the packet is dropped. The table also decrements the IPv4 ttl value.
- The second table checks the ttl value: if the ttl becomes 0, the packet is sent to the control plane through the CPU port.
- The third table uses the IPv4 address of the next hop (which was computed by the first table) to determine the Ethernet address of the next hop.
- Finally, the last table uses the outputPort to identify the source Ethernet address of the current switch, which is set in the outgoing packet.

The deparser constructs the outgoing packet by reassembling the Ethernet and IPv4 headers as computed by the pipeline.

```
// Include P4 core library
# include <core.p4>

// Include very simple switch architecture declarations
# include "very_simple_switch_model.p4"

// This program processes packets comprising an Ethernet and an IPv4
// header, and it forwards packets using the destination IP address

typedef bit<48>  EthernetAddress;
typedef bit<32>  IPv4Address;

// Standard Ethernet header
header Ethernet_h {
    EthernetAddress dstAddr;
    EthernetAddress srcAddr;
    bit<16>         etherType;
}

// IPv4 header (without options)
header IPv4_h {
```

```
    bit<4>       version;
    bit<4>       ihl;
    bit<8>       diffserv;
    bit<16>      totalLen;
    bit<16>      identification;
    bit<3>       flags;
    bit<13>      fragOffset;
    bit<8>       ttl;
    bit<8>       protocol;
    bit<16>      hdrChecksum;
    IPv4Address  srcAddr;
    IPv4Address  dstAddr;
}

// Structure of parsed headers
struct Parsed_packet {
    Ethernet_h ethernet;
    IPv4_h     ip;
}

// Parser section

// User-defined errors that may be signaled during parsing
error {
    IPv4OptionsNotSupported,
    IPv4IncorrectVersion,
    IPv4ChecksumError
}

parser TopParser(packet_in b, out Parsed_packet p) {
    Checksum16() ck;  // instantiate checksum unit

    state start {
        b.extract(p.ethernet);
        transition select(p.ethernet.etherType) {
            0x0800: parse_ipv4;
            // no default rule: all other packets rejected
        }
    }

    state parse_ipv4 {
        b.extract(p.ip);
        verify(p.ip.version == 4w4, error.IPv4IncorrectVersion);
        verify(p.ip.ihl == 4w5, error.IPv4OptionsNotSupported);
        ck.clear();
        ck.update(p.ip);
        // Verify that packet checksum is zero
        verify(ck.get() == 16w0, error.IPv4ChecksumError);
        transition accept;
    }
}

// Match-action pipeline section

control TopPipe(inout Parsed_packet headers,
                in error parseError, // parser error
                in InControl inCtrl, // input port
                out OutControl outCtrl) {
     IPv4Address nextHop;  // local variable

     /**
      * Indicates that a packet is dropped by setting the
      * output port to the DROP_PORT
      */
      action Drop_action() {
          outCtrl.outputPort = DROP_PORT;
      }

     /**
```

```
     * Set the next hop and the output port.
     * Decrements ipv4 ttl field.
     * @param ipv4_dest ipv4 address of next hop
     * @param port output port
     */
     action Set_nhop(IPv4Address ipv4_dest, PortId port) {
         nextHop = ipv4_dest;
         headers.ip.ttl = headers.ip.ttl - 1;
         outCtrl.outputPort = port;
     }

    /**
     * Computes address of next IPv4 hop and output port
     * based on the IPv4 destination of the current packet.
     * Decrements packet IPv4 TTL.
     * @param nextHop IPv4 address of next hop
     */
    table ipv4_match {
        key = { headers.ip.dstAddr: lpm; }  // longest-prefix match
        actions = {
             Drop_action;
             Set_nhop;
        }
        size = 1024;
        default_action = Drop_action;
    }

    /**
     * Send the packet to the CPU port
     */
     action Send_to_cpu() {
         outCtrl.outputPort = CPU_OUT_PORT;
     }

    /**
     * Check packet TTL and send to CPU if expired.
     */
    table check_ttl {
        key = { headers.ip.ttl: exact; }
        actions = { Send_to_cpu; NoAction; }
        const default_action = NoAction; // defined in core.p4
    }

    /**
     * Set the destination MAC address of the packet
     * @param dmac destination MAC address.
     */
     action Set_dmac(EthernetAddress dmac) {
         headers.ethernet.dstAddr = dmac;
     }

    /**
     * Set the destination Ethernet address of the packet
     * based on the next hop IP address.
     * @param nextHop IPv4 address of next hop.
     */
     table dmac {
         key = { nextHop: exact; }
         actions = {
              Drop_action;
              Set_dmac;
         }
         size = 1024;
         default_action = Drop_action;
     }

     /**
      * Set the source MAC address.
      * @param smac: source MAC address to use
```

```
        */
        action Set_smac(EthernetAddress smac) {
            headers.ethernet.srcAddr = smac;
        }

        /**
         * Set the source mac address based on the output port.
         */
        table smac {
            key = { outCtrl.outputPort: exact; }
            actions = {
                Drop_action;
                Set_smac;
            }
            size = 16;
            default_action = Drop_action;
        }

        apply {
            if (parseError != error.NoError) {
                Drop_action();  // invoke drop directly
                return;
            }

            ipv4_match.apply(); // Match result will go into nextHop
            if (outCtrl.outputPort == DROP_PORT) return;

            check_ttl.apply();
            if (outCtrl.outputPort == CPU_OUT_PORT) return;

            dmac.apply();
            if (outCtrl.outputPort == DROP_PORT) return;

            smac.apply();
    }
}

// deparser section
control TopDeparser(inout Parsed_packet p, packet_out b) {
    Checksum16() ck;
    apply {
        b.emit(p.ethernet);
        if (p.ip.isValid()) {
            ck.clear();               // prepare checksum unit
            p.ip.hdrChecksum = 16w0; // clear checksum
            ck.update(p.ip);          // compute new checksum.
            p.ip.hdrChecksum = ck.get();
        }
        b.emit(p.ip);
    }
}

// Instantiate the top-level VSS package
VSS(TopParser(),
    TopPipe(),
    TopDeparser()) main;
```

6. P4 language definition

This section provides a high-level overview of the P4 programming language, focusing on its syntax, semantics, lexical constructs, scoping rules, calling conventions, and naming conventions.

Chapter 7 introduces the P4 abstract machine, which serves as the model for describing the behavior of P4 programs. Chapter 8 through Chapter 17 define the P4 syntax and semantics in detail.

6.1. Syntax and semantics

6.1.1. Grammar

The complete grammar of $P4_{16}$ is given in Appendix F, using Yacc/Bison grammar description language. This text is based on the same grammar. We adopt several standard conventions when we provide excerpts from the grammar:

- UPPERCASE symbols denote terminals in the grammar.
- Excerpts from the grammar are given in BNF notation as follows:

```
p4program
    : /* empty */ | p4program declaration | p4program ;
    ;
```

6.1.2. Semantics and the P4 abstract machines

We describe the semantics of P4 in terms of abstract machines executing traditional imperative code.

P4 compilers are free to reorganize the code they generate in any way as long as the externally visible behaviors of the P4 programs are preserved as described by this specification where externally visible behavior is defined as:

- The input/output behavior of all P4 blocks, and
- The state maintained by extern blocks.

Throughout this specification, the P4 semantics is described using a combination of:

- P4 code snippets
- Pseudo-code snippets
- Prose algorithms

6.1.2.1. Pseudo-code snippets

Pseudo-code (mostly used for describing the semantics of various P4 constructs) are shown with fixed-size fonts as in the following example:

```
ParserModel.verify(bool condition, error err) {
    if (condition == false) {
        ParserModel.parserError = err;
        goto reject;
    }
}
```

6.1.2.2. Prose algorithms

The semantics of P4 constructs are also described using prose algorithms. For example, below is the signature of an algorithm used to describe type checking of P4 expressions.

Expr_ok:

- Typing expression, under context TC at p:
- The result is typedExpressionIR.

Detailed discussion of what cursor, typingContext, and typedExpressionIR mean is given in Section 7.2. For now, it suffices to know that Expr_ok takes an expression, a cursor, and a typingContext as inputs and results in a typedExpressionIR.

An algorithm is defined in terms of case-analysis, where each case corresponds to a variant of a P4 construct. For example, the following algorithm defines how to type check logical not expressions.

Typing '!' expression, under context TC at p:

1. Let typedExpressionIR be the result of typing expression, under context TC at p.
2. Let $typedExpressionIR'^{?}$ be the result of reducing serializable enums in typedExpressionIR until $compat_lnot is satisfied.
3. If $typedExpressionIR'^{?}$ is defined:
   a. Let $typedExpressionIR_{reduced}$ be $typedExpressionIR'^{?}$.
   b. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{reduced}$.
   c. Let $ctk_{reduced}$ be the compile-time known-ness of $typedExpressionIR_{reduced}$.
   d. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
   e. The result is '!' $typedExpressionIR_{reduced}$ annotated with expressionNoteIR.

$compat_lnot checks whether the operand is compatible with logical not. The algorithm checks that its optional result is present before binding the contained type. The following algorithm defines type checking of bitwise negation expressions.

Typing '~' expression, under context TC at p:

1. Let typedExpressionIR be the result of typing expression, under context TC at p.
2. Let $typedExpressionIR'^{?}$ be the result of reducing serializable enums in typedExpressionIR until $compat_bnot is satisfied.
3. If $typedExpressionIR'^{?}$ is defined:
   a. Let $typedExpressionIR_{reduced}$ be $typedExpressionIR'^{?}$.
   b. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{reduced}$.
   c. Let $ctk_{reduced}$ be the compile-time known-ness of $typedExpressionIR_{reduced}$.
   d. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
   e. The result is '~' $typedExpressionIR_{reduced}$ annotated with expressionNoteIR.

And so on for other variants of P4 expressions. These prose algorithms are presented throughout this specification.

6.1.2.2.1. Conditions and optional results

Some algorithms return an optional value to indicate possible failure. The following algorithm computes modulo of two natural numbers (including zero).

n_a mod n_b

1. Try:
    a. If n_b is not equal to 0: return $n_a \setminus n_b$.
2. Then, try:
    a. Check that n_b is equal to 0.
    b. Return ·.

Because the divisor may be zero, the result is optional. Now consider the case when the algorithm is called.

n mod forty-two

1. Let $nat^?$ be n mod 42.
2. Check that $nat^?$ is defined.
3. Let n_{result} be $nat^?$.
4. Return n_{result}.

The algorithm first obtains the optional result, checks that it is present, and then binds its contained value. A failed check prevents this invocation from producing a result. When another alternative can still succeed, the condition is written as an If branch instead.

6.2. Lexical constructs

All P4 keywords use only ASCII characters. All P4 identifiers must use only ASCII characters. P4 compilers should handle correctly strings containing 8-bit characters in comments and string literals. P4 is case-sensitive. Whitespace characters, including newlines are treated as token separators. Indentation is free-form; however, P4 has C-like block constructs, and all our examples use C-style indentation. Tab characters are treated as spaces.

The lexer recognizes the following kinds of terminals:

- IDENTIFIER: start with a letter or underscore, and contain letters, digits and underscores
- TYPE_IDENTIFIER: identifier that denotes a type name
- INTEGER: integer literals
- DONTCARE: a single underscore
- Keywords such as RETURN. By convention, each keyword terminal corresponds to a language keyword with the same spelling but using lowercase. For example, the RETURN terminal corresponds to the return keyword.

6.2.1. Identifiers

P4 identifiers may contain only letters, numbers, and the underscore character _, and must start with a letter or underscore. The special identifier consisting of a single underscore _ is reserved to indicate a

"don't care" value; its type may vary depending on the context. Certain keywords (e.g., apply) can be used as identifiers if the context makes it unambiguous.

```
<span id="identifier"></span>
<span id="nonTypeName"></span>
<span id="typeIdentifier"></span>
<span id="typeName"></span>
<span id="name"></span>
```

```
identifier
    : _ID text
    ;

nonTypeName
    : identifier | APPLY | KEY | ACTIONS | STATE | ENTRIES | TYPE | PRIORITY
    ;

typeIdentifier
    : _TID text
    ;

typeName = typeIdentifier

name
    : nonTypeName | typeName | LIST
    ;
```

The following helper is used to get a text representation of a name:

The name of name

1. Return name as text.

6.2.2. Literal constants

6.2.2.1. Boolean literals

```
<span id="booleanLiteral"></span>
```

```
booleanLiteral
    : TRUE | FALSE
    ;
```

There are two Boolean literal constants: true and false.

6.2.2.2. Integer literals

```
<span id="integerLiteral"></span>
```

```
integerLiteral
    : D int | nat W int | nat S int
    ;
```

Integer literals are non-negative arbitrary-precision integers. By default, literals are represented in base 10. The following prefixes must be employed to specify the base explicitly:

- `0x` or `0X` indicates base 16 (hexadecimal)
- `0o` or `0O` indicates base 8 (octal)
- `0d` or `0D` indicates base 10 (decimal)
- `0b` or `0B` indicates base 2

The width of a numeric literal in bits can be specified by an unsigned number prefix consisting of a number of bits and a signedness indicator:

- `w` indicates unsigned numbers
- `s` indicates signed numbers

Note that a leading zero by itself does not indicate an octal (base 8) constant. The underscore character is considered a digit within number literals but is ignored when computing the value of the parsed number. This allows long constant numbers to be more easily read by grouping digits together. The underscore cannot be used in the width specification or as the first character of an integer literal. No comments or whitespaces are allowed within a literal. Here are some examples of numeric literals:

```
32w255          // a 32-bit unsigned number with value 255
32w0d255        // same value as above
32w0xFF         // same value as above
32s0xFF         // a 32-bit signed number with value 255
8w0b10101010    // an 8-bit unsigned number with value 0xAA
8w0b_1010_1010 // same value as above
8w170           // same value as above
8s0b1010_1010  // an 8-bit signed number with value -86
16w0377         // 16-bit unsigned number with value 377 (not 255!)
16w0o377        // 16-bit unsigned number with value 255 (base 8)
```

6.2.2.3. String literals

```
stringLiteral
    : " text "
    ;
```

String literals are specified as an arbitrary sequence of 8-bit characters, enclosed within double quote characters `"` (ASCII code 34). Strings start with a double quote character and extend to the first double quote sign which is not immediately preceded by an odd number of backslash characters (ASCII code 92). P4 does not make any validity checks on strings (i.e., it does not check that strings represent legal UTF-8 encodings).

Since P4 does not allow strings to exist at runtime, string literals are generally passed unchanged through the P4 compiler to other third-party tools or compiler-backends. The compiler can, however, perform compile-time concatenation (constant-folding) of concatenation expressions into single literal. When such concatenation is performed, the binary representation of the string literals (excluding the quotes) is concatenated in the order they appear in the source code. There are no escape sequences that would be treated specially when strings are concatenated.

The backends and other tools can define their own handling of escape sequences (e.g., how to specify Unicode characters, or handle unprintable ASCII characters).

Here are 3 examples of string literals:

```
"simple string"
```

```
"string \" with \" embedded \" quotes"
"string with embedded
line terminator"
```

Here is an example of concatenation expression and an equivalent string literal:

```
"one string \" with a quote inside;" ++ (" " ++ "another string")
// can be constant folded to
"one string \" with a quote inside; another string"
```

6.2.3. Comments

P4 supports several kinds of comments:

- Single-line comments, introduced by `//` and spanning to the end of line,
- Multi-line comments, enclosed between `/*` and `*/`
- Nested multi-line comments are not supported.
- Javadoc-style comments, starting with `/**` and ending with `*/`

Use of Javadoc-style comments is strongly encouraged for the tables and actions that are used to synthesize the interface with the control-plane.

P4 treats comments as token separators and no comments are allowed within a token---e.g. `bi/**/t` is parsed as two tokens, `bi` and `t`, and not as a single token `bit`.

6.2.4. Optional trailing commas

The P4 grammar allows several kinds of comma-separated lists to end in an optional comma.

`<span id="trailingCommaOpt"></span>`

```
trailingCommaOpt
    : /* empty */ | ,
    ;
```

For example, the following declarations are both legal, and have the same meaning:

```
enum E {
     a, b, c
}

enum E {
     a, b, c,
}
```

This is particularly useful in combination with preprocessor directives:

```
enum E {
#if SUPPORT_A
    a,
#endif
    b,
    c,
}
```

6.3. Scoping

Scopes are used to organize P4 programs into nested namespaces. There are three kinds of scopes in P4, from outermost to innermost: the top-level scope (global namespace), the scope within a parser, control, or extern declaration (block namespace), and the scope within a block statement (local namespace). Local namespaces can be nested within other local namespaces.

```
// top-level (global) scope
const bit<8> GLOBAL = 255;

control c() {
  // block scope within a control
  bit<8> block = GLOBAL;

  apply {
    // local scope
    bit<8> local = block;
  }
}

bit<8> f() {
  // local scope
  bit<8> local = GLOBAL;
  {
    // nested local scope
    bit<8> local_nested = local;
  }
  return local;
}
```

Cursors are used to track scopes while defining the semantic rules of P4.

```
<span id="cursor"></span>
```

```
cursor
    : GLOBAL | BLOCK | LOCAL
    ;
```

6.3.1. Name resolution

P4 namespaces are organized in a hierarchical fashion.

```
<span id="prefixedNonTypeName"></span>
<span id="prefixedTypeName"></span>
```

```
prefixedNonTypeName
    : nonTypeName | _ID . nonTypeName
    ;

prefixedTypeName
    : typeName | _TID . typeName
    ;
```

Identifiers prefixed with a dot are always resolved in the top-level namespace.

```
const bit<32> x = 2;
control c() {
   int<32> x = 0;
   apply {
```

```
        x = x + (int<32>).x;  // x is the int<32> local variable,
                              // .x is the top-level bit<32> variable
    }
}
```

References to resolve an identifier are attempted inside-out, starting with the current scope and proceeding to all lexically enclosing scopes. The compiler may provide a warning if multiple resolutions are possible for the same name (name shadowing).

```
const bit<4> x = 1;
control c() {
    const bit<8> x = 8;    // x declaration shadows global x
    const bit<4> y = .x;   // reference to top-level x
    const bit<8> z = x;    // reference to c's local x
    apply {}
}
```

The order of declarations is important; with the exception of parser states, all uses of a symbol must follow the symbol's declaration. (This is a departure from $P4_{14}$, which allows declarations in any order. This requirement significantly simplifies the implementation of compilers for P4, allowing compilers to use additional information about declared identifiers to resolve ambiguities.)

See Section 14.3 for how variable references are resolved with respect to scopes.

The following helpers are used to convert the name within a prefixed name to a text:

The prefixed name of prefixedNonTypeName

1. If let nonTypeName of type nonTypeName be prefixedNonTypeName:
    a. Let nameIR be the name of nonTypeName.
    b. Return $_{\text{BARE}}$ nameIR.
2. Else:
    a. Let $_{\text{ID}}$ '.' nonTypeName be prefixedNonTypeName.
    b. Let nameIR' be the name of nonTypeName.
    c. Return '.' nameIR'.

The prefixed name of prefixedTypeName

1. If let typeName of type typeName be prefixedTypeName:
    a. Let nameIR be the name of typeName.
    b. Return $_{\text{BARE}}$ nameIR.
2. Else:
    a. Let $_{\text{TID}}$ '.' typeName be prefixedTypeName.
    b. Let nameIR' be the name of typeName.
    c. Return '.' nameIR'.

6.3.2. Name visibility

Identifiers defined in the top-level namespace are globally visible. Declarations within a parser or control

are private and cannot be referred to from outside of the enclosing parser or control.

6.4. Stateful elements

Most P4 constructs are stateless: given some inputs they produce a result that solely depends on these inputs. There are only two stateful constructs that may retain information across packets:

- tables: Tables are read-only for the data plane, but their entries can be modified by the control-plane,
- extern objects: many objects have state that can be read and written by the control plane and data plane. All constructs from the $P4_{14}$ language version that encapsulate state (e.g., counters, meters, registers) are represented using extern objects in $P4_{16}$.

In P4 all stateful elements must be explicitly allocated at compilation-time through the process called "instantiation".

In addition, parsers, control blocks, and packages may contain stateful element instantiations. Thus, they are also treated as stateful elements, even if they appear to contain no state, and must be instantiated before they can be used. However, although they are stateful, tables do not need to be instantiated explicitly—declaring a table also creates an instance of it. This convention is designed to support the common case, since most tables are used just once. To have finer-grained control over when a table is instantiated, a programmer can declare it within a control.

Recall the example in Section 5.3: TopParser, TopPipe, TopDeparser, Checksum16, and Switch are types. There are two instances of Checksum16, one in TopParser and one in TopDeparser, both called ck. The TopParser, TopDeparser, TopPipe, and Switch are instantiated at the end of the program, in the declaration of the main instance object, which is an instance of the Switch type (a package).

6.5. Call convention: copy-in/copy-out

P4 provides multiple constructs for writing modular programs: extern methods, parsers, controls, actions. All these constructs behave similarly to procedures in standard general-purpose programming languages:

- They have named and typed parameters.
- They introduce a new local scope for parameters and local variables.
- They allow arguments to be passed by binding them to their parameters.

```
<span id="parameterList"></span>
<span id="nonEmptyParameterList"></span>
<span id="parameter"></span>
<span id="direction"></span>
```

```
parameterList
    : /* empty */ | nonEmptyParameterList
    ;

nonEmptyParameterList
    : parameter | nonEmptyParameterList , parameter
    ;

parameter
    : annotationList direction type name initializerOpt
    ;

direction
    : /* empty */ | IN | OUT | INOUT
```

```
    ;
```

Invocations are executed using copy-in/copy-out semantics.

Each parameter may be labeled with a direction:

- in parameters are read-only. It is an error to use an in parameter on the left-hand side of an assignment or to pass it to a callee as a non-in argument. in parameters are initialized by copying the value of the corresponding argument when the invocation is executed.
- out parameters are, with a few exceptions listed below, uninitialized and are treated as l-values (See Chapter 12) within the body of the method or function. An argument passed as an out parameter must be an l-value; after the execution of the call, the value of the parameter is copied to the corresponding storage location for that l-value.
- inout parameters behave like a combination of in and out parameters simultaneously: On entry the value of the arguments is copied to the parameters. On return the value of the parameters is copied back to the arguments. In consequence, an argument passed as an inout parameter must be an l-value.
- The meaning of parameters with no direction depends upon the kind of entity the parameter is for:
  - For anything other than an action, e.g. a control, parser, or function, a directionless parameter means that the value supplied as an argument in a call must be a compile-time known value (see Section 7.5).
  - For an action, a directionless parameter indicates that it is "action data". See Section 9.3 for the meaning of action data, but its meaning includes the following possibilities:
    - The parameter's value is provided in the P4 program. In this case, the parameter behaves as if the direction were in. Such an argument expression need not be a compile-time known value.
    - The parameter's value is provided by the control plane software when an entry is added to a table that uses that action. See Section 9.3.
  - A directionless parameter of an extern object type is passed by reference.

Direction out parameters are always initialized at the beginning of execution of the portion of the program that has the out parameters, e.g. control, parser, action, function, etc. This initialization is not performed for parameters with any direction that is not out.

- If a direction out parameter is of type header or header_union, it is set to "invalid".
- If a direction out parameter is of type header stack, all elements of the header stack are set to "invalid", and its nextIndex field is initialized to 0 (see Section 8.4.4).
- If a direction out parameter is a compound type, e.g. a struct or tuple, other than one of the types listed above, then apply these rules recursively to its members.
- If a direction out parameter has any other type, e.g. bit<W>, an implementation need not initialize it to any predictable value.

For example, if a direction out parameter has type s2_t named p:

```
header h1_t {
    bit<8> f1;
    bit<8> f2;
}
struct s1_t {
    h1_t h1a;
    bit<3> a;
    bit<7> b;
}
struct s2_t {
```

```
    h1_t h1b;
    s1_t s1;
    bit<5> c;
}
```

then at the beginning of execution of the part of the program that has the out parameter p, it must be initialized so that p.h1b and and p.s1.h1a are invalid. No other parts of p are required to be initialized.

Arguments are evaluated from left to right prior to the invocation of the function itself. The order of evaluation is important when the expression supplied for an argument can have side-effects. Consider the following example:

```
extern void f(inout bit x, in bit y);
extern bit g(inout bit z);
bit a;
f(a, g(a));
```

Note that the evaluation of g may mutate its argument a, so the compiler has to ensure that the value passed to f for its first parameter is not changed by the evaluation of the second argument. The semantics for evaluating a function call is given by the following algorithm (implementations can be different as long as they provide the same result):

1. Arguments are evaluated from left to right as they appear in the function call expression.
2. If a parameter has a default value and no corresponding argument is supplied, the default value is used as an argument.
3. For each out and inout argument the corresponding l-value is saved (so it cannot be changed by the evaluation of the following arguments). This is important if the argument contains indexing operations into a header stack.
4. The value of each argument is saved into a temporary.
5. The function is invoked with the temporaries as arguments. We are guaranteed that the temporaries that are passed as arguments are never aliased to each other, so this "generated" function call can be implemented using call-by-reference if supported by the architecture.
6. On function return, the temporaries that correspond to out or inout arguments are copied in order from left to right into the l-values saved in Step 3.

According to this algorithm, the previous function call is equivalent to the following sequence of statements:

```
bit tmp1 = a;     // evaluate a; save result
bit tmp2 = g(a);  // evaluate g(a); save result; modifies a
f(tmp1, tmp2);    // evaluate f; modifies tmp1
a = tmp1;         // copy inout result back into a
```

To see why Step 3 in the above algorithm is important, consider the following example:

```
header H { bit z; }
H[2] s;
f(s[a].z, g(a));
```

The evaluation of this call is equivalent to the following sequence of statements:

```
bit tmp1 = a;          // save the value of a
bit tmp2 = s[tmp1].z;  // evaluate first argument
bit tmp3 = g(a);       // evaluate second argument; modifies a
```

```
f(tmp2, tmp3);          // evaluate f; modifies tmp2
s[tmp1].z = tmp2;       // copy inout result back; dest is not s[a].z
```

When used as arguments, extern objects can only be passed as directionless parameters—e.g., see the packet argument in the very simple switch example.

6.5.1. Justification

The main reason for using copy-in/copy-out semantics (instead of the more common call-by-reference semantics) is for controlling the side-effects of extern functions and methods. extern methods and functions are the main mechanism by which a P4 program communicates with its environment. With copy-in/copy-out semantics extern functions cannot hold references to P4 program objects; this enables the compiler to limit the side-effects that extern functions may have on the P4 program both in space (they can only affect out parameters) and in time (side-effects can only occur at function call time).

In general, extern functions are arbitrarily powerful: they can store information in global storage, spawn separate threads, "collude" with each other to share information—but they cannot access any variable in a P4 program. With copy-in/copy-out semantics the compiler can still reason about P4 programs that invoke extern functions.

There are additional benefits of using copy-in copy-out semantics:

- It enables P4 to be compiled for architectures that do not support references (e.g., where all data is allocated to named registers. Such architectures may require indices into header stacks that appear in a program to be compile-time known values.)
- It simplifies some compiler analyses, since function parameters can never alias to each other within the function body.

Following is a summary of the constraints imposed by the parameter directions:

- When used as arguments, extern objects can only be passed as directionless parameters.
- All constructor parameters are evaluated at compilation-time, and in consequence they must all be directionless (they cannot be in, out, or inout); this applies to package, control, parser, and extern objects. Expressions for these parameters must be supplied at compile-time, and they must evaluate to compile-time known values. See Section 10.2.1 for further details.
- For actions all directionless parameters must be at the end of the parameter list. When an action appears in a table's actions list, only the parameters with a direction must be bound. See Section 9.3 for further details.
- Actions can also be explicitly invoked using function call syntax, either from a control block or from another action. In this case, values for all action parameters must be supplied explicitly, including values for the directionless parameters. The directionless parameters in this case behave like in parameters. See Section 19.8.1 for further details.
- Default expressions are only allowed for in or directionless parameters, and the expressions supplied as defaults must be compile-time known values.
- If parameters with default values do not appear at the end of the list of parameters, invocations that use the default values must use named arguments, as in the following example:

```
extern void f(in bit a, in bit<3> b = 2, in bit<5> c);

void g() {
  f(a = 1, b = 2, c = 3);  // ok
  f(a = 1, c = 3);  // ok, equivalent to the previous call, b uses default value
  f(1, 2, 3);       // ok, equivalent to the previous call
  // f(1, 3); // illegal, since the parameter b is not the last in the list
}
```

6.5.2. Optional parameters

A parameter that is annotated with the @optional annotation is optional: the user may omit the value for that parameter in an invocation. Optional parameters can only appear for arguments of: packages, parser types, control types, extern functions, extern methods, and extern object constructors. Optional parameters cannot have default values. If a procedure-like construct has both optional parameters and default values then it can only be called using named arguments. It is recommended, but not mandatory, for all optional parameters to be at the end of a parameter list.

The implementation of such objects is not expressed in P4, so the meaning and implementation of optional parameters should be specified by the target architecture. For example, we can imagine a two-stage switch architecture where the second stage is optional. This could be declared as a package with an optional parameter:

```
package pipeline(/* parameters omitted */);
package switch(pipeline first, @optional pipeline second);

pipeline(/* arguments omitted */) ingress;
switch(ingress) main;   // a switch with a single-stage pipeline
```

Here the target architecture could implement the elided optional argument using an empty pipeline.

The following example shows optional parameters and parameters with default values.

```
extern void h(in bit<32> a, in bool b = true);  // default value

// function calls
h(10);  // same as h(10, true);
h(a = 10);  // same as h(10, true);
h(a = 10, b = true);

struct Empty {}
control nothing(inout Empty h, inout Empty m) {
   apply {}
}

parser parserProto<H, M>(packet_in p, out H h, inout M m);
control controlProto<H, M>(inout H h, inout M m);

package pack<HP, MP, HC, MC>(
    @optional parserProto<HP, MP> _parser,  // optional parameter
                              controlProto<HC, MC> _control = nothing()); // default
parameter value

pack() main;   // No value for _parser, _control is an instance of nothing()
```

6.6. Overloading

Functions, methods, and constructors are the only P4 constructs that support overloading: there can exist multiple methods with the same name in the same scope. When overloading is used, the compiler must be able to disambiguate at compile-time which method, function, or constructor is being called, either by the number of arguments or by the names of the arguments, when calls are specifying argument names. Argument type information is not used in disambiguating calls.

Notice that overloading of abstract methods, parsers, controls, or packages is not allowed:

```
parser p(packet_in p, out bit<32> value) {
  ...
```

```
}

// The following will cause an error about a duplicate declaration
//parser p(packet_in p, out Headers headers) {
//    ...
//}
```

See Section 19.3 for details on how overload is resolved.

6.7. Naming conventions

P4 provides a rich assortment of types. Base types include bit-strings, numbers, and errors. There are also built-in types for representing constructs such as parsers, pipelines, actions, and tables. Users can construct new types based on these: structures, enumerations, headers, header stacks, header unions, etc.

In this document we adopt the following conventions:

- Built-in types are written with lowercase characters—e.g., int<20>,
- User-defined types are capitalized—e.g., IPv4Address,
- Type variables are always uppercase—e.g., parser P<H, IH>(),
- Variables are uncapitalized-- e.g., ipv4header,
- Constants are written with uppercase characters—e.g., CPU_PORT, and
- Errors and enumerations are written in camel-case-- e.g. PacketTooShort.

7. The P4 Abstract Machine

A $P4_{16}$ program is a list of declarations:

```
p4program
    : /* empty */ | p4program declaration | p4program ;
    ;
```

An empty declarations is indicated with a single semicolon. (Allowing empty declarations accommodates the habits of C/C++ and Java programmers—e.g., certain constructs, like struct, do not require a terminating semicolon).

The evaluation of a P4 program is done in four stages:

- **preprocessing**: at compile time, the P4 program is preprocessed to handle directives such as #include and #define.
- **type checking**: at compile time, the P4 program is type checked according to the P4 type system.
- **instantiation**: at compile time, all stateful blocks in the P4 program are instantiated.
- **runtime evaluation**: at runtime, each P4 functional block is executed to completion, in isolation, when it receives control from the architecture.

The preprocessing stage is described in Section 7.1.

The type system is described in Section 7.2. In addition to checking the types, the type system also converts an input P4 program into an intermediate representation ($P4_{IR}$). $P4_{IR}$ is an extension of the surface syntax of P4, e.g., the types of all expressions are made explicit, implicit type casts are inserted,

and omitted type arguments are inferred. $P4_{IR}$ is described in Section 7.2.1.

The instantiation stage is described in Section 7.3. It traverses the type checked $P4_{IR}$ program, and recursively instantiates all stateful blocks. The result of the instantiation stage is a global store that holds all instantiated stateful blocks.

The runtime evaluation stage is described in Section 7.4. It describes how each P4 block, represented as stateful objects within the global store, is executed to completion. These blocks are orchestrated by the architecture to compose the overall packet processing pipeline.

7.1. Preprocessing

To aid the composition of programs from multiple source files, P4 compilers should support the following subset of the C preprocessor functionality:

- #define for defining macros (without arguments)
- #undef
- #if #else #endif #ifdef #ifndef #elif
- #include

The preprocessor should also remove the sequence backslash newline (ASCII codes 92, 10) to facilitate splitting content across multiple lines when convenient for formatting.

Additional C preprocessor capabilities may be supported, but are not guaranteed---e.g., macros with arguments. Similar to C, #include can specify a file name either within double quotes or within <>.

```
# include <system_file>
# include "user_file"
```

The difference between the two forms is the order in which the preprocessor searches for header files when the path is incompletely specified.

P4 compilers should correctly handle #line directives that may be generated during preprocessing. This functionality allows P4 programs to be built from multiple source files, potentially produced by different programmers at different times:

- the P4 core library, defined in this document,
- the architecture, defining data plane interfaces and extern blocks,
- user-defined libraries of useful components (e.g. standard protocol header definitions), and
- the P4 programs that specify the behavior of each programmable block.

The P4 language specification defines a core library that includes several common programming constructs. A description of the core library is provided in Appendix D. All P4 programs must include the core library. Including the core library is done with

```
# include <core.p4>
```

7.2. Type checking

$P4_{16}$ is a statically-typed language. Programs that do not pass the type checker are considered invalid and rejected by the compiler. P4 provides a number of base types as well as type operators that construct

derived types.

7.2.1. P4 Intermediate Representation

$P4_{IR}$ is an intermediate representation for P4 programs. $P4_{IR}$ extends the surface syntax of $P4_{16}$ with additional information obtained during type checking. They include:

- All expressions are annotated with their types.
- Implicit casts are made explicit.
- Omitted type arguments are inferred and made explicit.

$P4_{IR}$ is designed to stay as close as possible to the surface syntax of $P4_{16}$. This makes it easy to relate $P4_{IR}$ programs to their source $P4_{16}$ programs.

Take expressions as an example.

```
<span id="expression"></span>
<span id="expressionIR"></span>
<span id="expressionNoteIR"></span>
<span id="typedExpressionIR"></span>
```

```
expression
    : literalExpression | referenceExpression | defaultExpression
    | unaryExpression | binaryExpression | ternaryExpression | castExpression
    | dataExpression | accessExpression | callExpression
    | parenthesizedExpression
    ;

expressionIR
    : literalExpressionIR | referenceExpressionIR | defaultExpressionIR
    | unaryExpressionIR | binaryExpressionIR | ternaryExpressionIR
    | castExpressionIR | dataExpressionIR | accessExpressionIR
    | callExpressionIR | parenthesizedExpressionIR
    ;

expressionNoteIR
    : ( typeIR ctk )
    ;

typedExpressionIR
    : expressionIR # expressionNoteIR
    ;
```

typedExpressionIR is an intermediate representation of expression that is annotated with type information, expressionNoteIR. Nevertheless, the structure of the grammar mostly stays the same.

```
<span id="binaryExpression"></span>
<span id="binaryExpressionIR"></span>
```

```
binaryExpression
    : expression binop expression
    ;

binaryExpressionIR
    : typedExpressionIR binop typedExpressionIR
    ;
```

binaryExpressionIR is an intermediate representation of binaryExpression. The only difference is that the operands are now of type typedExpressionIR.

7.2.2. Typing a P4 program

For a well-typed $P4_{16}$ program, p4program, a type checker produces a $P4_{IR}$ program, p4programIR:

```
p4programIR
    : declarationIR* ;
    ;
```

Program_ok type checks a p4program and produces a p4programIR.

Program_ok:

- Typing p4program:
- The result is typed P4 program p4programIR.

Below is a prose algorithm that implements Program_ok. Details of what each meta-variable means are provided in the subsequent sections.

Typing p4program:

1. Let declaration* be the list of declarations in p4program.
2. Let TC_0 be an empty typing context.
3. Let context TC_1 and declarationIR* be the result of typing declaration* under context TC_0.
4. The result is typed P4 program declarationIR* ';'.

The following helper function is used to collect declarations in a p4program:

The list of declarations in p4program'

1. If p4program' matches pattern $_{EMPTY}$: return ·.
2. Else if let p4program declaration be p4program':
   a. Let declaration'* be the list of declarations in p4program.
   b. Return declaration'* concatenated with declaration.
3. Else:
   a. Let p4program ';' be p4program'.
   b. Return the list of declarations in p4program.

7.2.3. Typing context

```
globalTypingLayer = {
  CONSTRUCTOR constructorTypeDefEnv,
  TYPE typeDefEnv,
  CALLABLE callableTypeDefEnv,
  FRAME typeFrame
}

blockTypingLayer = {
  KIND blockKind,
  TYPE typeDefEnv,
  CALLABLE callableTypeDefEnv,
  FRAME typeFrame
}

localTypingLayer = {
  KIND localKind,
  TYPE typeDefEnv,
  FRAMES typeFrame*
}

typingContext = {
  CONTROL flow,
  LOOP loop,
  GLOBAL globalTypingLayer,
  BLOCK blockTypingLayer,
  LOCAL localTypingLayer
}
```

The operations on the typing context are collected in Appendix M.

A typing context is used to keep track of the types of all named declarations in scope at a given program point. It consists of three layers:

- A global typing layer, which contains all declarations at the top level of a $P4_{16}$ program.
- A block typing layer, which contains all declarations in the current block scope.
- A local typing layer, which contains all declarations in the current local scope.

Each layer contains mappings from names to types.

```
varTypeIR
    : direction typeIR ctk value?
    ;

typeFrame = map<id, varTypeIR>
```

typeFrame is a map from variable names to their types, varTypeIR. In addition to the type of the variable, varTypeIR also contains the direction of the variable (direction), whether it is a compile-time constant (ctk), and whether it is associated with a compile-time known value (value?).

The compile-time known-ness is discussed in Section 7.5.

Notice that localTypingLayer contains multiple typeFrames. This is because a local scope can be nested with block statements.

```
typeDefIR
    : nameTypeDefIR | aliasTypeDefIR | dataTypeDefIR | objectTypeDefIR
    ;

typeDefEnv = map<typeId, typeDefIR>
```

typeDefEnv is a map from type names to their type definitions, typeDefIR. typeDefIR is a type constructor, derived from user-defined type declarations.

```
callableTypeDefIR
    : actionTypeDefIR | functionTypeDefIR | methodTypeDefIR
    ;

callableTypeDefEnv = map<callableId, callableTypeDefIR>
```

Functions, methods, and actions can be defined in $P4_{16}$. These are collectively called callables. callableTypeDefEnv is a map from callable names to their type definitions, callableTypeDefIR.

```
constructorTypeDefIR
    : CONSTRUCTOR < typeParameterIR* , typeParameterIR* >
        ( constructorParameterIR* ) : typeIR
    ;

constructorTypeDefEnv = map<constructorId, constructorTypeDefIR>
```

Stateful objects can be defined in $P4_{16}$, which are instantiated using constructors. constructorTypeDefEnv is a map from constructor names to their type definitions, constructorTypeDefIR.

7.3. Instantiation

Instantiation of a program proceeds in order of declarations, starting in the top-level namespace:

- All declarations (e.g., parsers, controls, types, constants) evaluate to themselves.
- Each table evaluates to a table instance.
- Constructor invocations evaluate to stateful objects of the corresponding type. For this purpose, all constructor arguments are evaluated recursively and bound to the constructor parameters. Constructor arguments must be compile-time known values. The order of evaluation of the constructor arguments should be unimportant --- all evaluation orders should produce the same results.
- Instantiations evaluate to named stateful objects.
- The instantiation of a parser or control block recursively evaluates all stateful instantiations declared in the block.
- The result of the program's evaluation is the value of the top-level main variable.

Note that all stateful values are instantiated at compilation time.

As an example, consider the following program fragment:

```
// architecture declaration
parser P(/* parameters omitted */);
control C(/* parameters omitted */);
control D(/* parameters omitted */);

package Switch(P prs, C ctrl, D dep);

extern Checksum16 { /* body omitted */}

// user code
Checksum16() ck16; // checksum unit instance

parser TopParser(/* parameters omitted */)(Checksum16 unit) { /* body omitted */}
control Pipe(/* parameters omitted */) { /* body omitted */}
control TopDeparser(/* parameters omitted */)(Checksum16 unit) { /* body omitted */}

Switch(TopParser(ck16),
       Pipe(),
       TopDeparser(ck16)) main;
```

The evaluation of this program proceeds as follows:

1. The declarations of P, C, D, Switch, and Checksum16 all evaluate to themselves.
2. The Checksum16() ck16 instantiation is evaluated and it produces an object named ck16 with type Checksum16.
3. The declarations for TopParser, Pipe, and TopDeparser evaluate as themselves.
4. The main variable instantiation is evaluated:
   a. The arguments to the constructor are evaluated recursively.
   b. TopParser(ck16) is a constructor invocation.
      i. Its argument is evaluated recursively; it evaluates to the ck16 object.
   c. The constructor itself is evaluated, leading to the instantiation of an object of type TopParser.
   d. Similarly, Pipe() and TopDeparser(ck16) are evaluated as constructor calls.
   e. All the arguments of the Switch package constructor have been evaluated (they are an instance of TopParser, an instance of Pipe, and an instance of TopDeparser). Their signatures are matched with the Switch declaration.
   f. Finally, the Switch constructor can be evaluated. The result is an instance of the Switch package (that contains a TopParser named prs the first parameter of the Switch; a Pipe named ctrl; and a TopDeparser named dep).
5. The result of the program evaluation is the value of the main variable, which is the above instance of the Switch package.

Figure 8 shows the result of the evaluation in a graphical form. The result is always a graph of instances. There is only one instance of Checksum16, called ck16, shared between the TopParser and TopDeparser. Whether this is possible is architecture-dependent. Specific target compilers may require distinct checksum units to be used in distinct blocks.

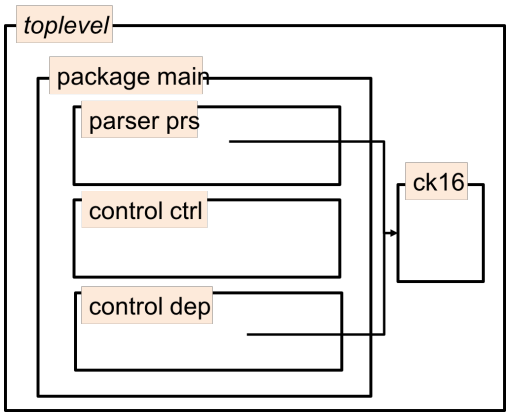

Figure 8. Evaluation result.

7.3.1. Control-plane names

Every controllable entity exposed in a P4 program must be assigned a unique, fully-qualified name, which the control plane may use to interact with that entity. The following entities are controllable.

- value sets
- tables
- keys
- actions
- extern instances

A fully qualified name consists of the local name of a controllable entity prepended with the fully qualified name of its enclosing namespace. Hence, the following program constructs, which enclose controllable entities, must themselves have unique, fully-qualified names.

- control instances
- parser instances

Evaluation may create multiple instances from one type, each of which must have a unique, fully-qualified name.

7.3.1.1. Computing control-plane names

The fully-qualified name of a construct is derived by concatenating the fully-qualified name of its enclosing construct with its local name. Constructs with no enclosing namespace, i.e. those defined at the global scope, have the same local and fully-qualified names. The local names of controllable entities and enclosing constructs are derived from the syntax of a P4 program as follows.

7.3.1.1.1. Value sets

For each value_set construct, its syntactic name becomes the local name of the value set. For example:

```
struct vsk_t {
    @match(ternary)
    bit<16> port;
}
value_set<vsk_t>(4) pvs;
```

This value_set's local name is pvs.

7.3.1.1.2. Tables

For each table construct, its syntactic name becomes the local name of the table. For example:

```
control c(/* parameters omitted */)() {
    table t { /* body omitted */ }
}
```

This table's local name is t.

7.3.1.1.3. Keys

Syntactically, table keys are expressions. For simple expressions, the local key name can be generated from the expression itself; the algorithm by which a compiler derives control-plane names for complex key expressions is target-dependent.

The spec suggests, but does not mandate, the following algorithm for generating names for some kinds of key expressions:

Kind	Example	Name
The isValid() method.	h.isValid()	"h.isValid()"
Array accesses.	header_stack[1]	"header_stack[1]"
Constants.	1	"1"
Field projections.	data.f1	"data.f1"
Slices.	f1[3:0]	"f1[3:0]"
Masks.	h.src & 0xFFFF	"h.src & 0xFFFF"

In the following example, the previous algorithm would derive for table t two keys with names data.f1 and hdrs[3].f2.

```
table t {
    keys = {
        data.f1 : exact;
        hdrs[3].f2 : exact;
    }
    actions = { /* body omitted */ }
}
```

If a compiler cannot generate a name for a key it *requires* the key expression to be annotated with a @name annotation (Section 21.2.3), as in the following example:

```
table t {
    keys = {
        data.f1 + 1 : exact @name("f1_mask");
    }
    actions = { /* body omitted */ }
}
```

Here, the @name("f1_mask") annotation assigns the local name "f1_mask" to this key.

7.3.1.1.4. Actions

For each action construct, its syntactic name is the local name of the action. For example:

```
control c(/* parameters omitted */)() {
    action a(...) { /* body omitted */ }
}
```

This action's local name is a.

7.3.1.1.5. Instances

The local names of extern, parser, and control instances are derived based on how the instance is used. If the instance is bound to a name, that name becomes its local control plane name. For example, if control C is declared as,

```
control C(/* parameters omitted */)() { /* body omitted */ }
```

and instantiated as,

```
C() c_inst;
```

then the local name of the instance is c_inst.

Alternatively, if the instance is created as an actual argument, then its local name is the name of the formal parameter to which it will be bound. For example, if extern E and control C are declared as,

```
extern E { /* body omitted */ }
control C(/* parameters omitted */)(E e_in) { /* body omitted */ }
```

and instantiated as,

```
C(E()) c_inst;
```

then the local name of the extern instance is e_in.

If the construct being instantiated is passed as an argument to a package, the instance name is derived from the user-supplied type definition when possible. In the following example, the local name of the instance of MyC is c, and the local name of the extern is e2, not e1.

```
extern E { /* body omitted */ }
control ArchC(E e1);
package Arch(ArchC c);

control MyC(E e2)() { /* body omitted */ }
Arch(MyC()) main;
```

Note that in this example, the architecture will supply an instance of the extern when it applies the instance of MyC passed to the Arch package. The fully-qualified name of that instance is main.c.e2.

Next, consider a larger example that demonstrates name generation when there are multiple instances.

```
control Callee() {
    table t { /* body omitted */ }
    apply { t.apply(); }
}
control Caller() {
```

```
    Callee() c1;
    Callee() c2;
    apply {
       c1.apply();
       c2.apply();
    }
}
control Simple();
package Top(Simple s);
Top(Caller()) main;
```

The compile-time evaluation of this program produces the structure in Figure 9. Notice that there are two instances of the table t. These instances must both be exposed to the control plane. To name an object in this hierarchy, one uses a path composed of the names of containing instances. In this case, the two tables have names s.c1.t and s.c2.t, where s is the name of the argument to the package instantiation, which is derived from the name of its corresponding formal parameter.

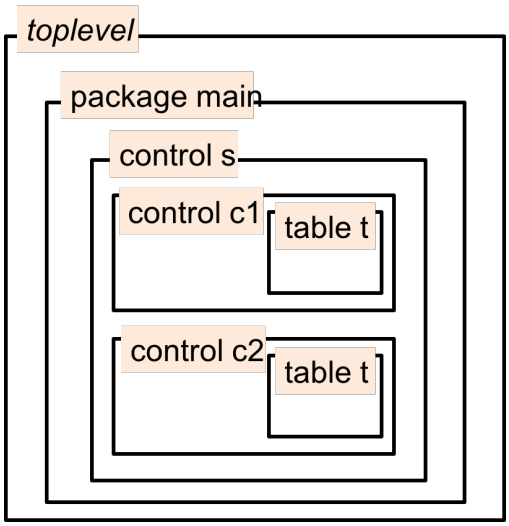

Figure 9. Evaluating a program that has several instantiations of the same component.

7.3.1.2. Annotations controlling naming

Control plane-related annotations (Section 21.2.3) can alter the names exposed to the control plane in the following ways.

- The @hidden annotation hides a controllable entity from the control plane. This is the only case in which a controllable entity is not required to have a unique, fully-qualified name.
- The @name annotation may be used to change the local name of a controllable entity.

Programs that yield the same fully-qualified name for two different controllable entities are invalid.

7.3.1.3. Recommendations

The control plane may refer to a controllable entity by a postfix of its fully qualified name when it is unambiguous in the context in which it is used. Consider the following example.

```
control c(/* parameters omitted */)() {
    action a (/* parameters omitted */) { /* body omitted */ }
    table t {
        keys = { /* body omitted */ }
        actions = { a; } }
}
c() c_inst;
```

Control plane software may refer to action c_inst.a as a when inserting rules into table c_inst.t, because it is clear from the definition of the table which action a refers to.

Not all unambiguous postfix shortcuts are recommended. For instance, consider the last example in Section 7.3.1.1.5. One might be tempted to refer to s.c1 simply as c1, as no other instance named c1 appears in the program. However, this leads to a brittle program since future modifications can never introduce an instance named c1, or include libraries of P4 code that contain instances with that name.

7.3.2. Instantiating a P4 program

```
objectId = nameIR*

object
    : externObject | parserObject | controlObject | packageObject | tableObject
    | valueSetObject
    ;
```

The stateful elements of a P4 program are represented as objects. These objects are identified by their fully-qualified names (objectId).

```
store = map<objectId, object>
```

A $P4_{16}$ program is first type checked to yield a $P4_{IR}$ program, and is then instantiated to create a store, which maps objectIds to objects.

Program_inst:

- Loading p4program:
- The result is global instantiation layer globalInstLayer and store STO.

Loading p4program:

1. Let typed P4 program declarationIR* ';' be the result of typing p4program.
2. Let IC_0 be an empty instantiation context.
3. Let STO_0 be an empty store.
4. Let context IC_1 and store STO_1 be the result of loading declarationIR*, under context IC_0, and store STO_0.
5. The result is global instantiation layer IC_1.GLOBAL and store STO_1.

7.3.3. Instantiation context

```
globalInstLayer = {
  CONSTRUCTOR constructorDefEnv,
  TYPE typeDefEnv,
  CALLABLE callableDefEnv,
  FRAME frame
}

blockInstLayer = {
  TYPE theta,
  CALLABLE callableDefEnv,
  STATE stateEnv,
  FRAME frame
}

localInstLayer = {
  TYPE theta,
  FRAMES frame*
}

instContext = {
  PATH objectId,
  GLOBAL globalInstLayer,
  BLOCK blockInstLayer,
  LOCAL localInstLayer
}
```

The operations on the instantiation context are collected in Appendix M.

An instantiation context is used to keep track of the types and compile-time known values of all named declarations in scope at a given program point.

- A global instantiation layer, which contains all declarations at the top level of a P4 program.
- A block instantiation layer, which contains all declarations in the current block scope.
- A local instantiation layer, which contains all declarations in the current local scope.

```
value
    : baseValue | dataValue | objectReferenceValue | synthesizedValue
    ;

frame = map<id, value>
```

frame is a map from variable names to their compile-time known values, value. The compile-time known-ness is discussed in Section 7.5.

```
typeDefIR
    : nameTypeDefIR | aliasTypeDefIR | dataTypeDefIR | objectTypeDefIR
    ;

typeDefEnv = map<typeId, typeDefIR>
```

typeDefEnv is a map from type names to their type definitions, typeDefIR. typeDefIR is a type constructor, derived from user-defined type declarations.

```
callableDef
    : actionDef | functionDef | methodDef
    ;

callableDefEnv = map<callableId, callableDef>
```

callableDefEnv is a map from callable names to their definitions, callableDef.

```
constructorDef
    : externObjectConstructorDef | parserObjectConstructorDef
    | controlObjectConstructorDef | packageObjectConstructorDef
    ;

constructorDefEnv = map<callableId, constructorDef>
```

Stateful objects are instantiated using constructors. constructorDefEnv is a map from constructor names to their definitions, constructorDef.

7.4. Runtime evaluation

The runtime evaluation of a P4 program is orchestrated by the architecture model. Each architecture model needs to specify the order and the conditions under which the various P4 component programs are dynamically executed. For example, in the Simple Switch example from Section 5.1 the execution flow goes Parser Pipe Deparser.

Once a P4 execution block is invoked its execution proceeds until termination according to the semantics defined in this document. P4 execution blocks are represented as objects in the global store, which is statically allocated during instantiation.

7.4.1. Evaluating a P4 program

P4 program evaluation proceeds by invoking the objects in the global store. A call to a programmable block invokes the corresponding abstract machine:

Call_eval:

- Calling callee `< typeArgumentListIR `> `( argumentListIR `), under context EC and state ARCH at p:
- The result is context EC, state ARCH, and call result callResult.

A call to an extern function invokes the corresponding extern function:

ExternFunctionCall_eval:

- Calling an extern function nameIR `( nameIR'* `), under context EC, and state ARCH:
- The result is context EC, state ARCH, and call result callResult.

A call to an extern method invokes the corresponding extern method:

ExternMethodCall_eval:

- Calling an extern method nameIR `( nameIR'* `) on object objectId, under context EC, and state ARCH:
- The result is context EC, state ARCH, and call result callResult.

7.4.2. Evaluation context

```
globalEvalLayer = globalInstLayer

blockEvalLayer = blockInstLayer

localEvalLayer = localInstLayer

evalContext = {
  GLOBAL globalEvalLayer,
  BLOCK blockEvalLayer,
  LOCAL localEvalLayer
}
```

The operations on the evaluation context are collected in Appendix M.

An evaluation context is used to keep track of the types and compile-time known values of all named declarations in scope at a given program point.

- A global evaluation layer, which contains all declarations at the top level of a P4 program.
- A block evaluation layer, which contains all declarations in the current block scope.
- A local evaluation layer, which contains all declarations in the current local scope.

The layers are the same as in the instantiation context, except that the frame is a map from variable names to their current runtime values.

The following sections describe the abstract machines that define the dynamic semantics of the programmable blocks in a P4 target.

7.4.3. The parser abstract machine

A parser starts execution in the start state and ends execution when one of the reject or accept states has been reached.

An architecture must specify the behavior when the accept and reject states are reached. For example, an architecture may specify that all packets reaching the reject state are dropped without further processing. Alternatively, it may specify that such packets are passed to the next block after the parser, with intrinsic metadata indicating that the parser reached the reject state, along with the error recorded.

7.4.4. The match-action pipeline abstract machine

We can describe the computational model of a match-action pipeline, embodied by a control block: the body of the control block is executed, similarly to the execution of a traditional imperative program:

- At runtime, statements within a block are executed in the order they appear in the control block.
- Execution of the return statement causes the immediate termination of the execution of the current control block and a return to the caller.
- Execution of the exit statement causes the immediate termination of the execution of the current control block and of all the enclosing caller control blocks.
- Applying a table executes the corresponding match-action unit.

7.4.5. Concurrency model

A typical packet processing system needs to execute multiple simultaneous logical "threads." At the very least there is a thread executing the control plane, which can modify the contents of the tables. Architecture specifications should describe in detail the interactions between the control-plane and the data-plane. The data plane can exchange information with the control plane through extern function and method calls. Moreover, high-throughput packet-processing systems may be processing multiple packets simultaneously, e.g., in a pipelined fashion, or concurrently parsing a first packet while performing match-action operations on a second packet. This section specifies the semantics of P4 programs with respect to such concurrent executions.

Each top-level parser or control block is executed as a separate thread when invoked by the architecture. All the parameters of the block and all local variables are thread-local—i.e., each thread has a private copy of these resources. This applies to the packet_in and packet_out parameters of parsers and deparsers.

As long as a P4 block uses only thread-local storage (e.g., metadata, packet headers, local variables), its behavior in the presence of concurrency is identical with the behavior in isolation, since any interleaving of statements from different threads must produce the same output.

In contrast, extern blocks instantiated by a P4 program are global, shared across all threads. If extern blocks mediate access to state (e.g., counters, registers)—i.e., the methods of the extern block read and write state, these stateful operations are subject to data races. P4 mandates that execution of a method call on an extern instance is atomic.

To allow users to express atomic execution of larger code blocks, P4 provides an @atomic annotation, which can be applied to block statements, parser states, control blocks, or whole parsers.

Consider the following example:

```
extern Register { /* body omitted */ }
control Ingress() {
  Register() r;
  table flowlet { /* read state of r in an action */ }
  table new_flowlet { /* write state of r in an action */ }
  apply {
    @atomic {
       flowlet.apply();
       if (ingress_metadata.flow_ipg > FLOWLET_INACTIVE_TIMEOUT)
          new_flowlet.apply();
```

```
}}}
```

This program accesses an extern object r of type Register in actions invoked from tables flowlet (reading) and new_flowlet (writing). Without the @atomic annotation these two operations would not execute atomically: a second packet may read the state of r before the first packet had a chance to update it.

Note that even within an action definition, if the action does something like reading a register, modifying it, and writing it back, in a way that only the modified value should be visible to the next packet, then, to guarantee correct execution in all cases, that portion of the action definition should be enclosed within a block annotated with @atomic.

A compiler backend must reject a program containing @atomic blocks if it cannot implement the atomic execution of the instruction sequence. In such cases, the compiler should provide reasonable diagnostics.

The P4 semantics described throughout this document assumes a sequential execution model. Thus, it does not describe the interactions between multiple threads accessing shared state. The purpose of this document is to clearly define the semantics of P4 constructs in isolation. The exact concurrency model, including the interactions between multiple threads, is target-dependent and beyond the scope of this document.

7.5. Compile-time known and local compile-time known values

Certain expressions in a P4 program have the property that their value can be determined at compile time. Moreover, for some of these expressions, their value can be determined only using information in the current scope. We call these *compile-time known values* and *local compile-time known values* respectively.

The following are local compile-time known values:

- Integer literals, Boolean literals, and string literals.
- Identifiers declared as constants using the const keyword.
- Identifiers declared in an error, enum, or match_kind declaration.
- The default identifier.
- The size field of a value with type header stack.
- The _ identifier when used as a select expression label.
- The expression {#} representing an invalid header or header union value.
- Identifiers that represent declared types, actions, functions, tables, parsers, controls, or packages.
- Tuple expression where all components are local compile-time known values.
- Structure-valued expressions, where all fields are local compile-time known values.
- Expressions evaluating to a list type, where all elements are local compile-time known values.
- Legal casts applied to local compile-time known values.
- Indexing a local compile-time known stack or tuple value with a local compile-time known index.
- Accessing a field of a local compile-time known struct, header, or header union value.
- The following expressions (+, -, |+|, |-|, *, /, %, !, &, |, ^, &&, ||, <<, >>, ~, >, <, ==, !=, <=, >=, ++, [:], ?:) when their operands are all local compile-time known values.
- Expressions of the form e.minSizeInBits(), e.minSizeInBytes(), e.maxSizeInBits() and e.maxSizeInBytes() where the

type of e is not generic.

The following are compile-time known values:

- All local compile-time known values.
- Constructor parameters (i.e., the declared parameters for a parser, control, etc.)
- Tuple expression where all components are compile-time known values.
- Structure-valued expressions, where all fields are compile-time known values.
- Expressions evaluating to a list type, where all elements are compile-time known values.
- Legal casts applied to compile-time known values.
- Indexing a compile-time known stack or tuple value with a compile-time known index.
- Accessing a field of a compile-time known struct, header, or header union value.
- The following expressions (+, -, |+|, |-|, *, /, %, !, &, |, ^, &&, ||, <<, >>, ~, >, <, ==, !=, <=, >=, ++, [:], ?:) when their operands are all compile-time known values.
- Expressions of the form e.minSizeInBits(), e.minSizeInBytes(), e.maxSizeInBits() and e.maxSizeInBytes() where the the type of e is generic.

Intuitively, the main difference between *compile-time known values* and *local compile-time known values* is that the former also contains constructor parameters. The distinction is important when it comes to defining the meaning of features like types. For example, in the type bit<e>, the expression e must be a local compile-time known value. Suppose instead that e were a constructor parameter—i.e., merely a compile-time known value. In this situation, it would be impossible to resolve bit<e> to a concrete type using purely local information—we would have to wait until the constructor was instantiated and the value of e known.

Local compile-time known values can be identified in the type checking phase. Compile-time known values can be identified after type checking, during the instantiation phase. Once the constructor parameters are bound to actual arguments and type parameters are bound to actual types, compile-time known values can be evaluated to concrete values.

ctk is a marker used in $P4_{IR}$ to indicate whether an expression is local compile-time known, compile-time known, or neither.

```
<span id="ctk"></span>
```

```
ctk
    : LCTK | CTK | DYN
    ;
```

The compile-time known status of a composite expression is the join of the statuses of its sub-expressions. Two statuses, and a list of statuses, are joined with:

The join of ctk and ctk'

1. If ctk matches pattern LCTK:
    a. If ctk' matches pattern LCTK: return LCTK.
    b. Else if ctk' matches pattern CTK: return CTK.
2. Else if ctk matches pattern CTK:
    a. If ctk' matches pattern LCTK: return CTK.
    b. Else if ctk' matches pattern CTK: return CTK.

3. Otherwise: return DYN.

The join of ctk'^*

1. If ctk'^* matches pattern []: return LCTK.
2. Else if ctk'^* matches pattern [_/1]:
   a. Let ctk be ctk'^*.
   b. Return ctk.
3. Else if ctk'^* matches pattern ::::
   a. Let $ctk_a :: ctk^*$ be ctk'^*.
   b. If ctk^* matches pattern [_/1]:
      i. Let ctk_b be ctk^*.
      ii. Return the join of ctk_a and ctk_b.
   c. Else if ctk^* matches pattern ::::
      i. Let $ctk_b :: ctk_c^*$ be ctk^*.
      ii. Check that ctk_c^* is a non-empty list.
      iii. Let ctk_d be the join of ctk_a and ctk_b.
      iv. Return the join of $ctk_d :: ctk_c^*$.

typedExpressionIRs in $P4_{IR}$, produced by typing expressions in $P4_{16}$, annotated with their type typeIR and a ctk marker indicating the compile-time known status.

```
expressionNoteIR
    : ( typeIR ctk )
    ;

typedExpressionIR
    : expressionIR # expressionNoteIR
    ;
```

8. P4 types and values

$P4_{16}$ is a statically-typed language. Programs that do not pass the type checker are considered invalid and rejected by the compiler. P4 provides a number of base types as well as type operators that construct derived types.

The syntax for P4 types is as follows:

```
<span id="type"></span>
<span id="typeOrVoid"></span>
```

```
type
    : baseType | namedType | arrayType | listType | tupleType
    ;

typeOrVoid
    : type | VOID | identifier
    ;
```

These types are represented in $P4_{IR}$ as follows:

```
typeIR
    : baseTypeIR | nameTypeIR | aliasTypeIR | dataTypeIR | objectTypeIR
    | synthesizedTypeIR
    ;
```

The types represent the kinds of values that can be used in P4 programs. The runtime representation of P4 values is as follows:

```
value
    : baseValue | dataValue | objectReferenceValue | synthesizedValue
    ;
```

8.1. Semantics of types

8.1.1. Type checking surface types

typeIR is constructed from type checking the surface syntax of P4 types (type) with the following relation:

Type_ok:

- Typing typeOrVoid, under context TC at p:
- The result is typeIR and fresh type variables typeId*.

Type_ok dispatch:

1. Try:
    a. If typeOrVoid matches pattern VOID: goto void
    b. Else if let baseType of type baseType be typeOrVoid:
        i. Try:
            A. If baseType matches pattern BOOL: goto boolean
            B. Else if baseType matches pattern ERROR: goto error
            C. Else if baseType matches pattern MATCH_KIND: goto matchKind
            D. Else if baseType matches pattern STRING: goto string
            E. Else if baseType matches pattern INT: goto arbitraryInt
        ii. Then, try:
            A. Goto fixedInt
        iii. Then, try:
            A. Goto fixedBit
        iv. Then, try:
            A. Goto variableBit

2. Then, try:
    a. Goto prefixedTypeName-identifier
3. Then, try:
    a. If let prefixedTypeName `< typeArgumentList `> of type specializedType be typeOrVoid: goto specializedType
    b. Else if let type `[ $expression_{size}$ `] of type arrayType be typeOrVoid:
        i. Try:
            A. Goto headerStackType
        ii. Then, try:
            A. Goto arrayType
    c. Else if let LIST `< typeArgument `> of type listType be typeOrVoid: goto listType
    d. Else if let TUPLE `< typeArgumentList `> of type tupleType be typeOrVoid: goto tupleType

8.1.2. Well-formedness

The P4 type system also imposes well-formedness constraints on types, which enforce additional restrictions on the structure of types, such as type nesting.

Type_wf:

- typeIR' is a well-formed type, with bound type variables B

Type_wf dispatch:

1. Try:
    a. If let baseTypeIR of type baseTypeIR be typeIR': goto baseTypeIR
    b. Else if let $_{\text{NAME}}$ typeId of type nameTypeIR be typeIR': goto nameTypeIR
    c. Else if let TYPEDEF typeId typeIR of type typedefTypeIR be typeIR': goto typedefTypeIR
    d. Else if let TYPE typeId typeIR of type newTypeIR be typeIR': goto newTypeIR
    e. Else if let LIST `< typeIR `> of type listTypeIR be typeIR': goto listTypeIR
    f. Else if let TUPLE `< $\text{typeIR}^*$ `> of type tupleTypeIR be typeIR': goto tupleTypeIR
    g. Else if let ARRAY typeIR `[ nat `] of type arrayTypeIR be typeIR': goto arrayTypeIR
    h. Else if let HEADER_STACK typeIR `[ nat `] of type headerStackTypeIR be typeIR': goto headerStackTypeIR
    i. Else if let STRUCT typeId `< $\text{typeArgumentIR}^*$ `> `{ ( annotationList $\text{typeIR}_{\text{field}}$ $\text{nameIR}_{\text{field}}$ ';' )$^*$ `} of type structTypeIR be typeIR': goto structTypeIR
    j. Else if let HEADER typeId `< $\text{typeArgumentIR}^*$ `> `{ ( annotationList $\text{typeIR}_{\text{field}}$ $\text{nameIR}_{\text{field}}$ ';' )$^*$ `} of type headerTypeIR be typeIR': goto headerTypeIR
    k. Else if let HEADER_UNION typeId `< $\text{typeArgumentIR}^*$ `> `{ ( annotationList $\text{typeIR}_{\text{field}}$ $\text{nameIR}_{\text{field}}$ ';' )$^*$ `} of type headerUnionTypeIR be typeIR': goto headerUnionTypeIR
2. Then, try:
    a. Goto enumTypeIR
3. Then, try:
    a. If let EXTERN typeId `< $\text{typeArgumentIR}^*$ `> `{ ( callableId ':' externMethodTypeDefIR )$^*$ `} of type

externObjectTypeIR be typeIR': goto externObjectTypeIR

b. Else if let PARSER typeId `< typeArgumentIR* `> `( parameterIR* `) of type parserObjectTypeIR be typeIR': goto parserObjectTypeIR

c. Else if let CONTROL typeId `< typeArgumentIR* `> `( parameterIR* `) of type controlObjectTypeIR be typeIR': goto controlObjectTypeIR

d. Else if let PACKAGE typeId `< typeArgumentIR* `> `{ typeIR* `} of type packageObjectTypeIR be typeIR': goto packageObjectTypeIR

e. Else if let TABLE typeId `{ tableMetadataStructTypeIR `} of type tableObjectTypeIR be typeIR': goto tableObjectTypeIR

f. Else if let defaultTypeIR of type defaultTypeIR be typeIR': goto defaultTypeIR

g. Else if let invalidHeaderTypeIR of type invalidHeaderTypeIR be typeIR': goto invalidHeaderTypeIR

h. Else if let sequenceTypeIR of type sequenceTypeIR be typeIR': goto sequenceTypeIR

i. Else if let recordTypeIR of type recordTypeIR be typeIR': goto recordTypeIR

j. Else if let SET `< typeIR `> of type setTypeIR be typeIR': goto setTypeIR

4. Then, try:

a. Goto tableMetadataTypeIR

The following tracks the set of bound type variables:

```
bound = set<typeId>
```

8.2. Base types and values

P4 supports the following built-in base types:

- bool, which represents Boolean values
- The error type, which is used to convey errors in a target-independent, compiler-managed way.
- The match_kind type, which is used for describing the implementation of table lookups,
- The string type, which can be used with compile-time known values of type string.
- int, which represents arbitrary-sized integer values
- Fixed-width signed integers represented using two's complement int<>
- Bit-strings of fixed width, denoted by bit<>
- Bit-strings of dynamically-computed width with a fixed maximum width varbit<>

```
baseType
    : BOOL | ERROR | MATCH_KIND | STRING | INT | INT < int >
    | INT < ( expression ) > | BIT | BIT < int > | BIT < ( expression ) >
    | VARBIT < int > | VARBIT < ( expression ) >
    ;
```

In addition, P4 supports void types:

```
typeOrVoid
    : type | VOID | identifier
```

```
    ;
```

- The void type, which has no values and can be used only in a few restricted circumstances.

In $P4_{IR}$, base types and values are represented as:


```
<span id="baseTypeIR"></span>
<span id="baseValue"></span>
```


```
baseTypeIR
    : voidTypeIR | boolTypeIR | errorTypeIR | matchKindTypeIR | stringTypeIR
    | integerTypeIR
    ;

baseValue
    : boolValue | errorValue | matchKindValue | stringValue | integerValue
    ;
```

8.2.1. Well-formedness

The well-formedness rules for base types are as follows:


```
<span id="Type_wf-baseTypeIR"></span>
```


baseTypeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.2.2. The void type


```
<span id="voidTypeIR"></span>
```


```
voidTypeIR
    : VOID
    ;
```

The void type is written void. It contains no values. It is not included in the production rule baseType as it can only appear in few restricted places in P4 programs.

There is no value of type void.

8.2.2.1. Type checking


```
<span id="Type_ok-void"></span>
```


Typing VOID, under context TC at p:

1. The result is VOID and fresh type variables ·.

8.2.3. Booleans


```
<span id="boolTypeIR"></span>
<span id="boolValue"></span>
```

```
boolTypeIR
    : BOOL
    ;

boolValue
    : _B bool
    ;
```

The Boolean type bool contains just two values, false and true. Boolean values are not integers or bit-strings.

8.2.3.1. Operations

The following operations are provided on Boolean expressions:

- And, denoted by &&
- Or, denoted by ||
- Negation, denoted by !
- Equality and inequality tests, denoted by == and != respectively.

The precedence of these operators is similar to C and uses short-circuited evaluation where relevant.

Additionally, the size of a boolean can be determined at compile-time ([sec-minsizeinbits]).

P4 does not implicitly cast from bit-strings to Booleans or vice versa. As a consequence, a program that is valid in a language like C such as,

```
if (x) /* body omitted */
```

(where x has an integer type) must instead be written in P4 as:

```
if (x != 0) /* body omitted */
```

See the discussion on arbitrary-precision types and implicit casts in Section 18.3 for details on how the 0 in this expression is evaluated.

8.2.3.2. Type checking

Typing BOOL, under context TC at p:

1. The result is BOOL and fresh type variables ·.

8.2.4. Errors

```
errorTypeIR
    : ERROR
    ;
```

```
errorValue
    : ERROR . nameIR
    ;
```

The error type contains opaque distinct values that can be used to signal errors. It is written as error. New elements of the error type are defined with the following syntax:

```
errorDeclaration
    : ERROR { nameList }
    ;
```

All elements of the error type are inserted into the error namespace, irrespective of the place where an error is defined. error is similar to an enumeration (enum) type in other languages. A program can contain multiple error declarations, which the compiler will merge together. It is an error to declare the same identifier multiple times.

Error declaration is described in detail in Section 11.7.

For example, the following declaration creates two elements of the error type (these errors are declared in the P4 core library):

```
error { ParseError, PacketTooShort }
```

The underlying representation of errors is target-dependent.

See Section 11.7 for the semantics of error declarations and Section 14.12 for accessing error values.

8.2.4.1. Operations

Symbolic names declared by an error declaration belong to the error namespace. The error type only supports equality (==) and inequality (!=) comparisons. The result of such a comparison is a Boolean value.

For example, the following operation tests for the occurrence of an error:

```
error errorFromParser;

if (errorFromParser != error.NoError) { /* code omitted */ }
```

8.2.4.2. Type checking

Typing ERROR, under context TC at p:

1. The result is ERROR and fresh type variables ·.

8.2.5. Match kinds

```
matchKindTypeIR
    : MATCH_KIND
    ;

matchKindValue
    : MATCH_KIND . nameIR
    ;
```

The match_kind type is very similar to the error type and is used to declare a set of distinct names that may be used in a table's key property (described in [sec-table-props]). All identifiers are inserted into the top-level namespace. It is an error to declare the same match_kind identifier multiple times.

```
matchKindDeclaration
    : MATCH_KIND { nameList trailingCommaOpt }
    ;
```

The P4 core library contains the following match_kind declaration:

```
match_kind {
   exact,
   ternary,
   lpm
}
```

Architectures may support additional match_kinds. The declaration of new match_kinds can only occur within model description files; P4 programmers cannot declare new match kinds.

Match kind declaration is described in detail in Section 11.8.

8.2.5.1. Operations

Values of type match_kind are similar to enum values. They support only assignment and comparisons for equality and inequality.

```
match_kind { fuzzy }
const bool same = exact == fuzzy;  // always 'false'
```

8.2.5.2. Type checking

Typing MATCH_KIND, under context TC at p:

1. The result is MATCH_KIND and fresh type variables ·.

8.2.6. Strings

```
stringTypeIR
```

```
    : STRING
    ;

stringValue = stringLiteral
```

The type string represents strings. The values of type string are either string literals, or concatenations of multiple string-typed expressions.

One cannot declare variables with a string type. Parameters with type string can be only directionless (see Section Section 19.6). P4 does not support string manipulation in the dataplane; the string type is only allowed for describing compile-time known values (i.e., string literals, as discussed in Section Section 6.2.2.3). Even so, the string type is useful, for example, in giving the type signature for extern functions such as the following:

```
extern void log(string message);
```

As another example, the following annotation indicates that the specified name should be used for a given table in the generated control-plane API:

```
@name("acl") table t1 { /* body omitted */ }
```

8.2.6.1. Operations

The only operation allowed on strings is concatenation, denoted by ++. For string concatenation, both operands must be strings and the result is also a string. String concatenation can only be performed at compile time.

```
extern void log(string message);

void foo(int<8> v) {
  // ...
  log("my log message " ++
      "continuation of the log message");
}
```

8.2.6.2. Type checking

Typing STRING, under context TC at p:

1. The result is STRING and fresh type variables ·.

8.2.7. Integers (signed and unsigned)

```
integerTypeIR
    : intTypeIR | fixedIntTypeIR | fixedBitTypeIR | varBitTypeIR
    ;

integerLiteral
    : D int | nat W int | nat S int
```

```
    ;

integerValue
    : integerLiteral | nat . nat V int
    ;
```

P4 supports arbitrary-size integer values. The typing rules for the integer types are chosen according to the following principles:

- **Inspired by C**: Typing of integers is modeled after the well-defined parts of C, expanded to cope with arbitrary fixed-width integers. In particular, the type of the result of an expression only depends on the expression operands, and not on how the result of the expression is consumed.
- **No undefined behaviors**: P4 attempts to avoid many of C's behaviors, which include the size of an integer (int), the results produced on overflow, and the results produced for some input combinations (e.g., shifts with negative amounts, overflows on signed numbers, etc.). P4 computations on integer types have no undefined behaviors.
- **Least surprise**: The P4 typing rules are chosen to behave as closely as possible to traditional well-behaved C programs.
- **Forbid rather than surprise**: Rather than provide surprising or undefined results (e.g., in C comparisons between signed and unsigned integers), we have chosen to forbid expressions with ambiguous interpretations. For example, P4 does not allow binary operations that combine signed and unsigned integers.

The priority of arithmetic operations is identical to C—e.g., multiplication binds tighter than addition.

The width of an integer type is given by the following; the arbitrary-precision integer type has no width:

Width of integer type integerTypeIR

1. If let intTypeIR of type intTypeIR be integerTypeIR: return ·.
2. Else if let INT `< n `> of type fixedIntTypeIR be integerTypeIR: return n.
3. Else if let BIT `< n `> of type fixedBitTypeIR be integerTypeIR: return n.
4. Else:
    a. Let VARBIT `< n `> be integerTypeIR.
    b. Return n.

8.2.7.1. Portability

No P4 target can support all possible types and operations. For example, the type bit<23132312> is legal in P4, but it is highly unlikely to be supported on any target in practice. Hence, each target can impose restrictions on the types it can support. Such restrictions may include:

- The maximum width supported
- Alignment and padding constraints (e.g., arithmetic may only be supported on widths which are an integral number of bytes).
- Constraints on some operands (e.g., some architectures may only support multiplications by small values, or shifts with small values).

The documentation supplied with a target should clearly specify restrictions, and target-specific compilers should provide clear error messages when such restrictions are encountered. An architecture may reject a well-typed P4 program and still be conformant to the P4 spec. However, if an architecture accepts a P4 program as valid, the runtime program behavior should match this specification.

8.2.7.2. Integer literals

The types of integer literals are as follows:

- An integer with no type prefix has type int.
- A non-negative integer prefixed with an integer width W and the character w has type bit<W>.
- An integer prefixed with an integer width W and the character s has type int<W>.

The table below shows several examples of integer literals and their types. For additional examples of literals see Section 6.2.2.

Literal	Interpretation
10	Type is int, value is 10
8w10	Type is bit<8>, value is 10
8s10	Type is int<8>, value is 10
2s3	Type is int<2>, value is -1 (last 2 bits), overflow warning
1w10	Type is bit<1>, value is 0 (last bit), overflow warning
1s1	Type is int<1>, value is -1, overflow warning

8.2.7.3. Illegal arithmetic expressions

Many arithmetic expressions that would be allowed in other languages are illegal in P4. To illustrate, consider the following declarations:

```
bit<8>  x;
bit<16> y;
int<8>  z;
```

The table below shows several expressions which are illegal because they do not obey the P4 typing rules. For each expression we provide several ways that the expression could be manually rewritten into a legal expression. Note that for some expression there are several legal alternatives, which may produce different results! The compiler cannot guess the user intent, so P4 requires the user to disambiguate.

Expression	Why it is illegal	Alternatives
x + y	Different widths	(bit<16>)x + y
		x + (bit<8>)y
x + z	Different signedness	(int<8>)x + z
		x + (bit<8>)z
(int<8>)y	Cannot change both sign and width	(int<8>)(bit<8>)y
		(int<8>)(int<16>)y
y + z	Different widths and signs	(int<8>)(bit<8>)y + z
		y + (bit<16>)(bit<8>)z
		(bit<8>)y + (bit<8>)z
		(int<16>)y + (int<16>)z
x << z	RHS of shift cannot be signed	x << (bit<8>)z
x < z	Different signs	x < (bit<8>)z

Expression	Why it is illegal	Alternatives
		(int<8>)x < z
1 << x	Either LHS should have a fixed width (bit shift),	32w1 << x
	Or RHS must be compile-time known (int shift)	None
~1	Bitwise operation on int	~32w1
5 & -3	Bitwise operation on int	32w5 & -3

8.2.7.4. Arbitrary-precision integers

The arbitrary-precision data type describes integers with an unlimited precision. This type is written as int.

This type is reserved for integer literals and expressions that involve only literals. No P4 runtime value can have an int type; at compile time the compiler will convert all int values that have a runtime component to fixed-width types, according to the rules described below.

The following example shows three constant definitions whose values are arbitrary-precision integers.

```
const int a = 5;
const int b = 2 * a;
const int c = b - a + 3;
```

Parameters with type int are not supported for actions. Parameters with type int for other callable entities of a program, e.g. controls, parsers, or functions, must be directionless, indicating that all calls must provide a compile-time known value as an argument for such a parameter. See Section 19.6 for more details on directionless parameters.

8.2.7.4.1. Operations

The type int denotes arbitrary-precision integers. In P4, all expressions of type int must be compile-time known values. The type int supports the following operations:

- Negation, denoted by unary -
- Unary plus, denoted by +. This operation behaves like a no-op.
- Addition, denoted by +.
- Subtraction, denoted by -.
- Comparison for equality and inequality, denoted by == and != respectively. These operations produce a Boolean result.
- Numeric comparisons <,<=,>, and >=. These operations produce a Boolean result.
- Multiplication, denoted by *.
- Truncating integer division between positive values, denoted by /.
- Modulo between positive values, denoted by %.
- Arithmetic shift left and right denoted by << and >>. These operations produce an int result. The right operand must be either an unsigned value of type bit<S> or a compile-time known value that is a non-negative integer.
- Bit slices, denoted by [H:L], where H and L must be expressions that evaluate to non-negative, local compile-time known values, and H >= L must be true. The types of H and L (which do not need to be identical) must be one of the following:

- int - an arbitrary-precision integer ([sec-arbitrary-precision-integers])
- bit<W> - a W-bit unsigned integer where W >= 0 ([sec-unsigned-integers])
- int<W> - a W-bit signed integer where W >= 1 ([sec-signed-integers])
- a serializable enum with an underlying type that is bit<W> or int<W> ([sec-enum-types]).

The result is an unsigned bit-string of width H - L + 1, including the bits numbered from L (which becomes the least significant bit of the result) to H (the most significant bit of the result) from the source operand. The conditions 0 <= L <= H are checked statically. If necessary, the source integer value that is sliced is automatically extended to have a width with H bits. Note that both endpoints of the extraction are inclusive. The bounds are required to be values that are known at compile time so that the width of the result can be computed at compile time. A slice of a negative or positive value is always a positive value.

Each operand that participates in any of these operation must have type int (except shifts). Binary operations cannot be used to combine values of type int with values of a fixed-width type (except shifts). However, the compiler automatically inserts casts from int to fixed-width types in certain situations—see Chapter 18.

All computations on int values are carried out without loss of information. For example, multiplying two 1024-bit values may produce a 2048-bit value (note that concrete representation of int values is not specified). int values can be cast to bit<w> and int<w> values. Casting an int value to a fixed-width type will preserve the least-significant bits. If truncation causes significant bits to be lost, the compiler should emit a warning.

Note: bitwise-operations (|, &, ^, ~) are not defined on expressions of type int. In addition, it is illegal to apply division and modulo to negative values. Saturating arithmetic is not supported for arbitrary-precision integers.

8.2.7.4.2. Type checking

Typing INT, under context TC at p:

1. The result is INT and fresh type variables ·.

8.2.7.5. Fixed-width unsigned integers (bit-strings)

An unsigned integer (which we also call a "bit-string") has an arbitrary width, expressed in bits. A bit-string of width W is declared as: bit<W>. W must be an expression that evaluates to a local compile-time known value (see Section 7.5) that is a non-negative integer. When using an expression for the size, the expression must be parenthesized. Bitstrings with width 0 are allowed; they have no actual bits, and can only have the value 0. See [sec-uninitialized-values-and-writing-invalid-headers] for additional details. Note that bit<W> type refers to both cases of bit<W> and bit<(expression)> where the width is a local compile-time known value.

```
const bit<32> x = 10;   // 32-bit constant with value 10.
const bit<(x + 2)> y = 15;  // 12-bit constant with value 15.
                            // expression for width must use ()
```

Bits within a bit-string are numbered from 0 to W-1. Bit 0 is the least significant, and bit W-1 is the most significant.

For example, the type bit<128> denotes the type of bit-string values with 128 bits numbered from 0 to 127,

where bit 127 is the most significant.

The type bit is a shorthand for bit<1>.

P4 architectures may impose additional constraints on bit types: for example, they may limit the maximum size, or they may only support some arithmetic operations on certain sizes (e.g., 16-, 32-, and 64- bit values).

8.2.7.5.1. Operations

This section discusses all operations that can be performed on expressions of type bit<W> for some width W, also known as bit-strings.

Arithmetic operations "wrap around", similar to C operations on unsigned values (i.e., representing a large value on W bits will only keep the least-significant W bits of the value). In particular, P4 does not have arithmetic exceptions—the result of an arithmetic operation is defined for all possible inputs.

P4 target architectures may optionally support saturating arithmetic. All saturating operations are limited to a fixed range between a minimum and maximum value. Saturating arithmetic has advantages, in particular when used as counters. The result of a saturating counter max-ing out is much closer to the real result than a counter that overflows and wraps around. According to Wikipedia Saturating Arithmetic is as numerically close to the true answer as possible; for 8-bit binary signed arithmetic, when the correct answer is 130, it is considerably less surprising to get an answer of 127 from saturating arithmetic than to get an answer of –126 from modular arithmetic. Likewise, for 8-bit binary unsigned arithmetic, when the correct answer is 258, it is less surprising to get an answer of 255 from saturating arithmetic than to get an answer of 2 from modular arithmetic. At this time, P4 defines saturating operations only for addition and subtraction. For an unsigned integer with bit-width of W, the minimum value is 0 and the maximum value is 2^W-1. The precedence of saturating addition and subtraction operations is the same as for modular arithmetic addition and subtraction.

All binary operations except shifts and concatenation require both operands to have the same exact type and width; supplying operands with different widths produces an error at compile time. No implicit casts are inserted by the compiler to equalize the widths. There are no other binary operations that accept signed and unsigned values simultaneously besides shifts and concatenation. The following operations are provided on bit-string expressions:

- Test for equality between bit-strings of the same width, designated by ==. The result is a Boolean value.
- Test for inequality between bit-strings of the same width, designated by !=. The result is a Boolean value.
- Unsigned comparisons <,>,<=,>=. Both operands must have the same width and the result is a Boolean value.

Each of the following operations produces a bit-string result when applied to bit-strings of the same width:

- Negation, denoted by unary -. The result is computed by subtracting the value from 2^W. The result is unsigned and has the same width as the input. The semantics is the same as the C negation of unsigned numbers.
- Unary plus, denoted by +. This operation behaves like a no-op.
- Addition, denoted by +. This operation is associative. The result is computed by truncating the result of the addition to the width of the output (similar to C).
- Subtraction, denoted by -. The result is unsigned, and has the same type as the operands. It is computed by adding the negation of the second operand (similar to C).
- Multiplication, denoted by *. The result has the same width as the operands and is computed by

truncating the result to the output's width. P4 architectures may impose additional restrictions—e.g., they may only allow multiplication by a non-negative integer power of two.

- Bitwise "and" between two bit-strings of the same width, denoted by &.
- Bitwise "or" between two bit-strings of the same width, denoted by |.
- Bitwise "complement" of a single bit-string, denoted by ~.
- Bitwise "xor" of two bit-strings of the same width, denoted by ^.
- Saturating addition, denoted by |+|.
- Saturating subtraction, denoted by |-|.

Bit-strings also support the following operations:

- Logical shift left and right by a non-negative integer value (which need not be a compile-time known value), denoted by << and >> respectively. In a shift, the left operand is unsigned, and right operand must be either an expression of type bit<S> or a non-negative integer value that is known at compile time. The result has the same type as the left operand. Shifting by an amount greater than or equal to the width of the input produces a result where all bits are zero.
- Extraction of a set of contiguous bits, also known as a slice, denoted by [H:L], where H and L must be expressions that evaluate to non-negative, local compile-time known values, and H >= L. The types of H and L (which do not need to be identical) must be numeric ([sec-numeric-values]). The result is a bit-string of width H - L + 1, including the bits numbered from L (which becomes the least significant bit of the result) to H (the most significant bit of the result) from the source operand. The conditions 0 <= L <= H < W are checked statically (where W is the length of the source bit-string). Note that both endpoints of the extraction are inclusive. The bounds are required to be local compile-time known values so that the width of the result can be computed at compile time. Slices are also l-values, which means that P4 supports assigning to a slice: e[H:L] = x. The effect of this statement is to set bits H through L (inclusive of both) of e to the bit-pattern represented by x, and leaves all other bits of e unchanged. A slice is always an unsigned integer. A slice may also be specified as [L+:W] where L is the lowest bit of the slice and W is the width of the slice. In this case, only W must be a non-negative local compile-time known numeric value. L must also be a numeric value and some architectures may require it to be compile-time known. It must also be non-negative and in-range for the type. Some architectures may allow variable indexing, in which case an out of range value will be equivalent to a shift of that amount. An index that is not compile-time known must be a bit<W> type in order to ensure that it is non-negative.
- Concatenation of bit-strings and/or fixed-width signed integers, denoted by ++. The two operands must be either bit<W> or int<W>, and they can be of different signedness and width. The result has the same signedness as the left operand and the width equal to the sum of the two operands' width. In concatenation, the left operand is placed as the most significant bits.

Additionally, the size of a bit-string can be determined at compile-time ([sec-minsizeinbits]).

8.2.7.5.2. Type checking

Typing baseType, under context TC at p:

1. If baseType matches pattern BIT:
   a. The result is BIT `< 1 `> and fresh type variables ·.
2. Else if let BIT `< i `> be baseType:
   a. If let n of type nat be i:
      i. The result is BIT `< n `> and fresh type variables ·.

3. Else if let BIT `< `(expression `) `> be baseType:
    a. Let typedExpressionIR be the result of typing expression, under context TC at p.
    b. If the compile-time known-ness of typedExpressionIR is LCTK:
        i. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
        ii. If let integerValue of type integerValue be value:
            A. Let nat? be the natural number representation of integerValue.
            B. If nat? is defined:
                I. Let n be nat?.
                II. The result is BIT `< n `> and fresh type variables ·.

8.2.7.6. Fixed-width signed integers

Signed integers are represented using two's complement. An integer with W bits is declared as: int<W>. W must be an expression that evaluates to a local compile-time known (see Section Section 7.5) value that is a non-negative integer. Note that int<W> type refers to both cases of int<W> and int<(expression)> where the width is a local compile-time known value.

Bits within an integer are numbered from 0 to W-1. Bit 0 is the least significant, and bit W-1 is the sign bit.

For example, the type int<64> describes the type of integers represented using exactly 64 bits with bits numbered from 0 to 63, where bit 63 is the most significant (sign) bit.

P4 architectures may impose additional constraints on signed types: for example, they may limit the maximum size, or they may only support some arithmetic operations on certain sizes (e.g., 16-, 32-, and 64- bit values).

A signed integer with width 1 can only have two legal values: 0 and -1.

8.2.7.6.1. Operations

This section discusses all operations that can be performed on expressions of type int<W> for some W. Recall that the int<W> denotes signed W-bit integers, represented using two's complement.

In general, P4 arithmetic operations do not detect "underflow" or "overflow": operations simply "wrap around", similar to C operations on unsigned values. Hence, attempting to represent large values using W bits will only keep the least-significant W bits of the value.

P4 supports saturating arithmetic (addition and subtraction) for signed integers. Targets may optionally reject programs using saturating arithmetic. For a signed integer with bit-width of W, the minimum value is -2^(W-1) and the maximum value is 2^(W-1)-1.

P4 also does not support arithmetic exceptions. The runtime result of an arithmetic operation is defined for all combinations of input arguments.

All binary operations except shifts and concatenation require both operands to have the same exact type (signedness) and width and supplying operands with different widths or signedness produces a compile-time error. No implicit casts are inserted by the compiler to equalize the types. Except for shifts and concatenation, P4 does not have any binary operations that operate simultaneously on signed and unsigned values.

Note that bitwise operations on signed integers are well-defined, since the representation is mandated to be two's complement.

The int<W> datatype supports the following operations; all binary operations require both operands to

have the exact same type. The result always has the same width as the left operand.

- Negation, denoted by unary -.
- Unary plus, denoted by +. This operation behaves like a no-op.
- Addition, denoted by +.
- Subtraction, denoted by -.
- Comparison for equality and inequality, denoted == and != respectively. These operations produce a Boolean result.
- Numeric comparisons, denoted by <,<=,> and >=. These operations produce a Boolean result.
- Multiplication, denoted by *. Result has the same width as the operands. P4 architectures may impose additional restrictions—e.g., they may only allow multiplication by a power of two.
- Bitwise "and" between two bit-strings of the same width, denoted by &.
- Bitwise "or" between two bit-strings of the same width, denoted by |.
- Bitwise "complement" of a single bit-string, denoted by ~.
- Bitwise "xor" of two bit-strings of the same width, denoted by ^.
- Saturating addition, denoted by |+|.
- Saturating subtraction, denoted by |-|.

The int<W> datatype also support the following operations:

- Arithmetic shift left and right denoted by << and >>. The left operand is signed and the right operand must be either an unsigned number of type bit<S> or a compile-time known value that is a non-negative integer. The result has the same type as the left operand. Shifting left produces the exact same bit pattern as a shift left of an unsigned value. Shift left can thus overflow, when it leads to a change of the sign bit. Shifting by an amount greater than the width of the input produces a "correct" result:
  - all result bits are zero when shifting left
  - all result bits are zero when shifting a non-negative value right
  - all result bits are one when shifting a negative value right
- Extraction of a set of contiguous bits, also known as a slice, denoted by [H:L], where H and L must be expressions that evaluate to non-negative, local compile-time known values, and H >= L must be true. The types of H and L (which do not need to be identical) must be numeric ([sec-numeric-values]). The result is an unsigned bit-string of width H - L + 1, including the bits numbered from L (which becomes the least significant bit of the result) to H (the most significant bit of the result) from the source operand. The conditions 0 <= L <= H < W are checked statically (where W is the length of the source bit-string). Note that both endpoints of the extraction are inclusive. The bounds are required to be values that are known at compile time so that the width of the result can be computed at compile time. Slices are also l-values, which means that P4 supports assigning to a slice: e[H:L] = x. The effect of this statement is to set bits H through L of e to the bit-pattern represented by x, and leaves all other bits of e unchanged. A slice of a signed integer is treated as an unsigned integer.
- Concatenation of bit-strings and/or fixed-width signed integers, denoted by ++. The two operands must be either bit<W> or int<W>, and they can be of different signedness and width. The result has the same signedness as the left operand and the width equal to the sum of the two operands' width. In concatenation, the left operand is placed as the most significant bits.

Additionally, the size of a fixed-width signed integer can be determined at compile-time ([sec-minsizeinbits]).

8.2.7.6.2. Type checking

Typing baseType, under context TC at p:

1. If let INT `< i `> be baseType:
    a. If let n of type nat be i:
        i. If n is greater than 0:
            A. The result is INT `< n `> and fresh type variables ·.
2. Else if let INT `< `(expression `) `> be baseType:
    a. Let typedExpressionIR be the result of typing expression, under context TC at p.
    b. If the compile-time known-ness of typedExpressionIR is LCTK:
        i. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
        ii. If let integerValue of type integerValue be value:
            A. Let $nat^?$ be the natural number representation of integerValue.
            B. If $nat^?$ is defined:
                I. Let n be $nat^?$.
                II. If n is greater than 0:
                    1. The result is INT `< n `> and fresh type variables ·.

8.2.7.7. Dynamically-sized bit-strings

Some network protocols use fields whose size is only known at runtime (e.g., IPv4 options). To support restricted manipulations of such values, P4 provides a special bit-string type whose size is set at runtime, called a varbit.

The type varbit<W> denotes a bit-string with a width of at most W bits, where W is a local compile-time known value (see Section Section 7.5) that is a non-negative integer. For example, the type varbit<120> denotes the type of bit-string values that may have between 0 and 120 bits. Most operations that are applicable to fixed-size bit-strings (unsigned numbers) **cannot** be performed on dynamically sized bit-strings. Note that varbit<W> type refers to both cases of varbit<W> and varbit<(expression)> where the width is a local compile-time known value.

P4 architectures may impose additional constraints on varbit types: for example, they may limit the maximum size, or they may require varbit values to always contain an integer number of bytes at runtime.

8.2.7.7.1. Operations

To support parsing headers with variable-length fields, P4 offers a type varbit. Each occurrence of the type varbit has a statically-declared maximum width, as well as a dynamic width, which must not exceed the static bound. Prior to initialization a variable-size bit-string has an unknown dynamic width.

Variable-length bit-strings support a limited set of operations:

- Assignment to another variable-sized bit-string. The target of the assignment must have the same static width as the source. When executed, the assignment sets the dynamic width of the target to the dynamic width of the source.
- Comparison for equality or inequality with another varbit field. Two varbit fields can be compared only if they have the same type. Two varbits are equal if they have the same dynamic width and all the bits

up to the dynamic width are the same.

The following operations are *not* supported directly on a value of type varbit, but instead on any type for which extract and emit operations are supported (e.g. a value with type header) that may contain a field of type varbit. They are mentioned here only to ease finding this information in a section dedicated to type varbit.

- Parser extraction into a header containing a variable-sized bit-string using the two-argument extract method of a packet_in extern object (see [sec-packet-extract-two]). This operation sets the dynamic width of the field.
- The emit method of a packet_out extern object can be performed on a header and a few other types (see [sec-deparse]) that contain a field with type varbit. Such an emit method call inserts a variable-sized bit-string with a known dynamic width into the packet being constructed.

Additionally, the maximum size of a variable-length bit-string can be determined at compile-time ([sec-minsizeinbits]).

8.2.7.7.2. Type checking

Typing baseType, under context TC at p:

1. If let VARBIT `< i `> be baseType:
   a. If let n of type nat be i:
      i. The result is VARBIT `< n `> and fresh type variables ·.
2. Else if let VARBIT `< `(expression `) `> be baseType:
   a. Let typedExpressionIR be the result of typing expression, under context TC at p.
   b. If the compile-time known-ness of typedExpressionIR is LCTK:
      i. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
      ii. If let integerValue of type integerValue be value:
         A. Let $nat^?$ be the natural number representation of integerValue.
         B. If $nat^?$ is defined:
            I. Let n be $nat^?$.
            II. The result is VARBIT `< n `> and fresh type variables ·.

8.3. Named types

```
prefixedTypeName
    : typeName | _TID . typeName
    ;

specializedType
    : prefixedTypeName < typeArgumentList >
    ;

namedType
    : prefixedTypeName | specializedType
```

```
    ;
```

Named types are types that reference another type by name. User-defined types can be introduced with their names. For example, the following declares a new struct type S:

```
struct S {
    bit<32> x;
    bit<32> y;
}
```

This type can then be referenced by its name, as can be seen in the parameter type of the following example:

```
void f(S s) {}
```

Here, S is a named type (typeName) that refers to the struct type defined earlier.

Types such as struct, header, and header union types can be generic. In order to use such a generic type it must be specialized with appropriate type arguments. For example,

```
// generic structure type
struct S<T> {
    T field;
    bool valid;
}

struct G<T> {
    S<T> s;
}

// specialize S by replacing 'T' with 'bit<32>'
const S<bit<32>> s = { field = 32w0, valid = false };
// Specialize G by replacing 'T' with 'bit<32>'
const G<bit<32>> g = { s = { field = 0, valid = false } };

// generic header type
header H<T> {
    T field;
}

// Specialize H by replacing 'T' with 'bit<8>'
const H<bit<8>> h = { field = 1 };
// Header stack produced from a specialization of a generic header type
H<bit<8>>[10] stack;

// Generic header union
header_union HU<T> {
    H<bit<32>> h32;
    H<bit<8>>  h8;
    H<T>       ht;
}

// Header union with a type obtained by specializing a generic header union type
HU<bit> hu;
```

To handle named types used as return types, the following production is also used:

```
identifier
    : _ID text
    ;
```

```
typeOrVoid
    : type | VOID | identifier
    ;
```

8.3.1. Type definitions (type constructors)

```
typeDeclaration
    : derivedTypeDeclaration | typedefDeclaration | parserTypeDeclaration
    | controlTypeDeclaration | packageTypeDeclaration
    ;
```

typeDeclaration introduces a user-defined type definition. These can be referenced by name. Generic type definitions can be specialized with type arguments to produce a type instance.

The internal representation of type definitions is as follows:

```
nameTypeDefIR = nameTypeIR

aliasTypeDefIR = aliasTypeIR

dataTypeDefIR
    : structTypeDefIR | headerTypeDefIR | headerUnionTypeDefIR | enumTypeDefIR
    ;

objectTypeDefIR
    : externObjectTypeDefIR | parserObjectTypeDefIR | controlObjectTypeDefIR
    | packageObjectTypeDefIR | tableObjectTypeDefIR
    ;

typeDefIR
    : nameTypeDefIR | aliasTypeDefIR | dataTypeDefIR | objectTypeDefIR
    ;
```

For example, structTypeDeclaration introduces a struct type definition, structTypeDefIR:

```
structTypeDefIR
    : STRUCT typeId < typeParameterIR* > { fieldTypeIR* }
    ;

structTypeDeclaration
    : annotationList STRUCT name typeParameterListOpt { typeFieldList }
    ;
```

Some type definitions like struct and header type definitions can be polymorphic, meaning they can have type variables as parameters. On the other hand, type definitions such as enum and type alias definitions can only be monomorphic. Below summarizes which kinds of type definitions can be polymorphic:

typeDefIR is polymorphic

1. Let bool be typeDefIR is monomorphic.
2. Return ~bool.

typeDefIR is monomorphic

1. If typeDefIR has type nameTypeDefIR: return true.
2. Else if typeDefIR has type aliasTypeDefIR: return true.
3. Else if typeDefIR has type enumTypeDefIR: return true.
4. Else if typeDefIR has type tableObjectTypeDefIR: return true.
5. Otherwise: return false.

8.3.1.1. Well-formedness

The well-formedness of type definitions is defined as:

TypeDef_wf:

- typeDefIR is a well-formed type definition, with bound type variables B

typeDefIR is a well-formed type definition, with bound type variables B:

1. Let ($\text{typeParameterIR}_{expl}^{*}$, $\text{typeParameterIR}_{impl}^{*}$) be the type parameters of typeDefIR.
2. Let $\text{typeParameterIR}^{*}$ be $\text{typeParameterIR}_{expl}^{*}$ concatenated with $\text{typeParameterIR}_{impl}^{*}$.
3. Check that the elements of $\text{typeParameterIR}^{*}$ are distinct.
4. Let typeIR_{base} be the underlying type of typeDefIR.
5. Let B_{inner} be the union of the sets B and `{ $\text{typeParameterIR}^{*}$ `}.
6. If typeIR_{base} is a well-formed type, with bound type variables B_{inner}: then, the relation holds.

8.3.2. Type resoultion

Notice that P4 has struct types, but the surface syntax does not have an explicit construct for struct types. Instead, struct types are always referenced by name. To easily handle named types in $P4_{IR}$, the internal representation of types contain representations for the referred types.

```
structTypeIR
    : STRUCT typeId < typeArgumentIR* > { fieldTypeIR* }
```

```
    ;
```

For example, a struct type is represented as structTypeIR in $P4_{IR}$. Thus, after type checking, a named type in $P4_{16}$ is resolved to its referred type in $P4_{IR}$. For named types referring to monomorphic type definitions, the resolved type is simply the type defined by the type definition.

```
enum E_t { A, B, C; }
void f(E_t e) {}
```

In the above example, the named type E_t is resolved to the enum type enumTypeIR.

```
struct S<T> { T t; }
void f(S<bit<32>> s) {}
```

In the above example, the named type S<bit<32>> is resolved to a structTypeIR, where it specializes a structTypeDefIR with the type argument bit<32>. Type specialization is discussed in detail in the next section.

8.3.3. Type specialization

typeDefIR$_{base}$ specialized by typeArgumentIR*

1. Let (typeParameterIR$_{expl}{}^{*}$, typeParameterIR$_{impl}{}^{*}$) be the type parameters of typeDefIR$_{base}$.
2. Let typeParameterIR* be typeParameterIR$_{expl}{}^{*}$ concatenated with typeParameterIR$_{impl}{}^{*}$.
3. Check that the length of typeParameterIR* is equal to the length of typeArgumentIR*.
4. Let theta be `{ ( typeParameterIR ':' typeArgumentIR )$^{*}$ `}.
5. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.
6. Let typeIR$_{spec}$ be typeIR$_{base}$ substituted by theta.
7. Return typeIR$_{spec}$.

8.3.4. Type names

```
prefixedTypeName
    : typeName | _TID . typeName
    ;

identifier
    : _ID text
    ;
```

A prefixedTypeName and identifier may refer to either a monomorphic type definition or a polymorphic type definition without type arguments. The names can be prefixed by a dot (.) to refer to the top-level namespace.

On type checking, the referenced type definition is looked up by name. If the referenced type definition is monomorphic, then the resolved type is simply the type defined by the type definition. If the referenced type definition is polymorphic (but without type arguments), then the resolved type is a specialized type with no type arguments.

In some cases, the reference may be a type parameter. In other words, the type is indeed a variable type, not a concrete type until it is bound to a type argument. These are internally represented as:

```
nameTypeDefIR = nameTypeIR

nameTypeIR
    : _NAME typeId
    ;
```

8.3.4.1. Operations

Because functions and methods can be generic, they offer the possibility of declaring values with types that are type variables:

```
void f<T>() {
   T x; // the type of x is T, a type variable
}
```

The type of such objects is not known until the function is specialized with specific type arguments.

Currently the only operations that are available for such values are assignment (explicit through =, or implicit, through argument passing). This behavior is similar to languages such as Java, and different from languages such as C++.

A future version of P4 may introduce a notion of *type constraints* which would enable more operations on such values. Because of this limitation, such values are currently of limited utility.

8.3.4.2. Type checking

Typing typeOrVoid, under context TC at p:

1. If let $_{ID}$ typeId of type identifier be typeOrVoid:
   a. Let typeIR and fresh type variables $typeId_{fresh}^{*}$ be the result of typing $_{TID}$ typeId, under context TC at p.
   b. The result is typeIR and fresh type variables $typeId_{fresh}^{*}$.
2. Else if let prefixedTypeName of type prefixedTypeName be typeOrVoid:
   a. Let prefixedNameIR be the prefixed name of prefixedTypeName.
   b. Let $typeDefIR'^{?}$ be the type definition of prefixedNameIR from the p layer of TC.
   c. If $typeDefIR'^{?}$ is defined:
      i. Let typeDefIR be $typeDefIR'^{?}$.
      ii. If typeDefIR is monomorphic:
         A. Let typeIR be the underlying type of typeDefIR.
         B. The result is typeIR and fresh type variables ·.
      iii. Else:
         A. Let typeIR be typeDefIR specialized by ·.
         B. The result is typeIR and fresh type variables ·.

8.3.4.3. Well-formedness

NAME typeId is a well-formed type, with bound type variables B:

1. If typeId is an element of the set B: then, the relation holds.

8.3.5. Specialized types

A generic type may be specialized by specifying arguments for its type variables. In cases where the compiler can infer type arguments, type specialization is not necessary. When a type is specialized, all its type variables must be bound.

```
specializedType
    : prefixedTypeName < typeArgumentList >
    ;
```

For example, the following extern declaration describes a generic block of registers, where the type of the elements stored in each register is an arbitrary T.

```
extern Register<T> {
    Register(bit<32> size);
    T read(bit<32> index);
    void write(bit<32> index, T value);
}
```

The type T has to be specified when instantiating a set of registers, by specializing the Register type:

```
Register<bit<32>>(128) registerBank;
```

The instantiation of registerBank is made using the Register type specialized with the bit<32> bound to the T type argument.

8.3.5.1. Type checking

Typing prefixedTypeName `< typeArgumentList `>, under context TC at p:

1. Let prefixedNameIR be the prefixed name of prefixedTypeName.
2. Let $typeDefIR'^{?}$ be the type definition of prefixedNameIR from the p layer of TC.
3. Check that $typeDefIR'^{?}$ is defined.
4. Let typeDefIR be $typeDefIR'^{?}$.
5. Check that typeDefIR is polymorphic.
6. Let $typeIR_{arg}^{*}$ and fresh type variables $typeId_{fresh}^{*}$ be the result of typing typeArgumentList, under context TC at p.
7. Let typeIR be typeDefIR specialized by $typeIR_{arg}^{*}$.
8. The result is typeIR and fresh type variables $typeId_{fresh}^{*}$.

8.4. Data types and values

```
dataTypeIR
    : listTypeIR | tupleTypeIR | arrayTypeIR | headerStackTypeIR | structTypeIR
    | headerTypeIR | headerUnionTypeIR | enumTypeIR
    ;

dataValue
    : listValue | tupleValue | arrayValue | headerStackValue | structValue
    | headerValue | headerUnionValue | enumValue
    ;
```

Data types represent the types that aggregate other types, including lists, tuples, header stacks, structs, headers, header unions, and enums.

8.4.1. Lists

A list holds zero or more values, where every element must have the same type. The type of a list where all elements have type `T` is written as

```
list<T>
```

```
listType
    : LIST < typeArgument >
    ;
```

Internally, list types and values are represented as follows:

```
listTypeIR
    : LIST < typeIR >
    ;

listValue
    : LIST [ value* ]
    ;
```

8.4.1.1. Operations

The value of a list is written using curly braces, with each element separated by a comma. The left curly brace is preceded by a (list<T>) where `T` is the list element type. Such a value can be passed as an argument, e.g. to extern constructor functions.

```
struct pair_t {
    bit<16> a;
    bit<32> b;
}

extern E {
```

```
    E(list<pair_t> data);
    void run();
}

control c() {
    E((list<pair_t>) {{2, 3}, {4, 5}}) e;
    apply {
        e.run();
    }
}
```

Additionally, the size of a list can be determined at compile-time ([sec-minsizeinbits]).

8.4.1.2. Type checking

Typing LIST `< typeArgument `>, under context TC at p:

1. Let $\text{typeIR}_{\text{arg}}$ and fresh type variables $\text{typeId}_{\text{fresh}}^{*}$ be the result of typing typeArgument, under context TC at p.
2. The result is LIST `< $\text{typeIR}_{\text{arg}}$ `> and fresh type variables $\text{typeId}_{\text{fresh}}^{*}$.

8.4.1.3. Well-formedness

LIST `< typeIR `> is a well-formed type, with bound type variables B:

1. If typeIR can be nested in a list type:
    a. If typeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.4.2. Tuples

A tuple is similar to a struct, in that it holds multiple values. The type of tuples with n component types T1, …, Tn is written as

```
tuple<T1, /* more fields omitted */, Tn>
```

The type tuple<> is a tuple type with no components.

```
tupleType
    : TUPLE < typeArgumentList >
    ;
```

Internally, tuple types and values are represented as follows:

```
tupleTypeIR
```

```
    : TUPLE < typeIR* >
    ;

tupleValue
    : TUPLE ( value* )
    ;
```

8.4.2.1. Operations

The fields of a tuple can be accessed using array index syntax x[0], x[1]. The indexes **must** be local compile-time known values, to enable the type-checker to identify the field types statically.

Tuples can be compared for equality using == and !=; two tuples are equal if and only if all their fields are respectively equal.

Currently tuple fields are not left-values, even if the tuple itself is. (i.e., a tuple can only be assigned monolithically, and the field values cannot be changed individually.) This restriction may be lifted in a future version of the language.

8.4.2.2. Type checking

Typing TUPLE `< typeArgumentList `>, under context TC at p:

1. Let $\text{typeIR}_{\text{arg}}^{*}$ and fresh type variables $\text{typeId}_{\text{fresh}}^{*}$ be the result of typing typeArgumentList, under context TC at p.
2. The result is TUPLE `< $\text{typeIR}_{\text{arg}}^{*}$ `> and fresh type variables $\text{typeId}_{\text{fresh}}^{*}$.

8.4.2.3. Well-formedness

TUPLE `< $\text{typeIR}^{*}$ `> is a well-formed type, with bound type variables B:

1. If typeIR can be nested in a tuple type, for all typeIR in typeIR^{*}:
    a. If typeIR is a well-formed type, with bound type variables B, for all typeIR in typeIR^{*}: then, the relation holds.

8.4.3. Arrays

An array is a fixed-size vector of elements of the same type. It is defined as:

```
arrayType
    : type [ expression ]
    ;
```

where typeRef refers to the element type and the expression is a compile-time known value that is the size of the array. For an array type such as arr[n], n-1 is the maximum defined index.

Internally, array types and values are represented as follows:

```
arrayTypeIR
    : ARRAY typeIR [ nat ]
    ;

arrayValue
    : ARRAY [ value* ]
    ;
```

8.4.3.1. Type checking

Typing type `[ $expression_{size}$ `], under context TC at p:

1. Let $typeIR_{base}$ and fresh type variables $typeId_{fresh}^{*}$ be the result of typing type, under context TC at p.
2. Let $typeIR_{base'}$ be $typeIR_{base}$ with typedefs unrolled.
3. Check that $_{typeIR}base'$~ has type headerTypeIR \/ $typeIR_{base'}$ has type headerUnionTypeIR.
4. Let $typedExpressionIR_{size}$ be the result of typing $expression_{size}$, under context TC at p.
5. Check that the compile-time known-ness of $typedExpressionIR_{size}$ is LCTK.
6. Let value be the result of local compile-time evaluation of $typedExpressionIR_{size}$, under context TC at p.
7. Check that value has type integerValue.
8. Let $integerValue_{size}$ be value.
9. Let $nat^{?}$ be the natural number representation of $integerValue_{size}$.
10. Check that $nat^{?}$ is defined.
11. Let n_{size} be $nat^{?}$.
12. Check that n_{size} is greater than 0.
13. The result is ARRAY $typeIR_{base}$ `[ $n_{size}$ `] and fresh type variables $typeId_{fresh}^{*}$.

8.4.3.2. Well-formedness

ARRAY typeIR `[ nat `] is a well-formed type, with bound type variables B:

1. If typeIR can be nested in an array type:
    a. If typeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.4.4. Header stacks

A header stack represents an array of headers or header unions. A header stack type is defined as:

```
arrayType
    : type [ expression ]
    ;
```

where namedType refers to a header or header union type. For a header stack hs[n], the term n is the maximum defined size, and must be a local compile-time known value that is a positive integer. Nested header stacks are not supported.

Internally, header stack types and values are represented as follows:

```
headerStackTypeIR
    : HEADER_STACK typeIR [ nat ]
    ;

headerStackValue
    : HEADER_STACK [ value* ( nat ) ]
    ;
```

At runtime a stack contains n values with type namedType, only some of which may be valid. The valid elements of a header stack need not be contiguous.

For example, the following declarations,

```
header Mpls_h {
    bit<20> label;
    bit<3>  tc;
    bit     bos;
    bit<8>  ttl;
}
Mpls_h[10] mpls;
```

introduce a header stack called mpls containing ten entries, each of type Mpls_h.

8.4.4.1. Operations

P4 provides a set of computations for manipulating header stacks. A header stack hs of type h[n] can be understood in terms of the following pseudocode:

```
// type declaration
struct hs_t {
  bit<32> nextIndex;
  bit<32> size;
  h[n] data;  // Ordinary array
}

// instance declaration and initialization
hs_t hs;
hs.nextIndex = 0;
hs.size = n;
```

Intuitively, a header stack can be thought of as a struct containing an ordinary array of headers hs and a counter nextIndex that can be used to simplify the construction of parsers for header stacks, as discussed below. The nextIndex counter is initialized to 0.

Given a header stack value hs of size n, the following expressions are legal:

- hs[index]: produces a reference to the header at the specified position within the stack; if hs is an l-value, the result is also an l-value. The header may be invalid. Some implementations may impose the constraint that the index expression must be a compile-time known value. A P4 compiler must

give an error if an index that is a compile-time known value is out of range.

Accessing a header stack hs with an index less than 0 or greater than or equal to hs.size results in an undefined value. See [sec-uninitialized-values-and-writing-invalid-headers] for more details.

The index is an expression that must be of numeric types ([sec-numeric-values]).

- hs.size: produces a 32-bit unsigned integer that returns the size of the header stack (a local compile-time known value).
- assignment from a header stack hs into another stack requires the stacks to have the same types and sizes. All components of hs are copied, including its elements and their validity bits, as well as nextIndex.

To help programmers write parsers for header stacks, P4 also offers computations that automatically advance through the stack as elements are parsed:

- hs.next: produces a reference to the element with index hs.nextIndex in the stack. May only be used in a parser. If the stack's nextIndex counter is greater than or equal to size, then evaluating this expression results in a transition to reject and sets the error to error.StackOutOfBounds. If hs is an l-value, then hs.next is also an l-value.
- hs.last: produces a reference to the element with index hs.nextIndex - 1 in the stack, if such an element exists. May only be used in a parser. If the nextIndex counter is less than 1, or greater than size, then evaluating this expression results in a transition to reject and sets the error to error.StackOutOfBounds. Unlike hs.next, the resulting reference is never an l-value.
- hs.lastIndex: produces a 32-bit unsigned integer that encodes the index hs.nextIndex - 1. May only be used in a parser. If the nextIndex counter is 0, then evaluating this expression produces an undefined value.

Finally, P4 offers the following computations that can be used to manipulate the elements at the front and back of the stack:

- hs.push_front(int count): shifts hs "right" by count. The first count elements become invalid. The last count elements in the stack are discarded. The hs.nextIndex counter is incremented by count. The count argument must be a compile-time known value that is a positive integer. The return type is void.
- hs.pop_front(int count): shifts hs "left" by count (i.e., element with index count is copied in stack at index 0). The last count elements become invalid. The hs.nextIndex counter is decremented by count. The count argument must be a compile-time known value that is a positive integer. The return type is void.

The following pseudocode defines the behavior of push_front and pop_front:

```
void push_front(int count) {
    for (int i = this.size-1; i >= 0; i -= 1) {
        if (i >= count) {
            this[i] = this[i-count];
        } else {
            this[i].setInvalid();
        }
    }
    this.nextIndex = this.nextIndex + count;
    if (this.nextIndex > this.size) this.nextIndex = this.size;
    // Note: this.last, this.next, and this.lastIndex adjust with this.nextIndex
}

void pop_front(int count) {
    for (int i = 0; i < this.size; i++) {
        if (i+count < this.size) {
            this[i] = this[i+count];
        } else {
            this[i].setInvalid();
        }
```

```
    }
    if (this.nextIndex >= count) {
        this.nextIndex = this.nextIndex - count;
    } else {
        this.nextIndex = 0;
    }
    // Note: this.last, this.next, and this.lastIndex adjust with this.nextIndex
}
```

Similar to structs and headers, the size of a header stack is a compile-time known value (Section [sec-minsizeinbits]).

Two header stacks can be compared for equality (==) or inequality (!=) only if they have the same element type and the same length. Two stacks are equal if and only if all their corresponding elements are equal. Note that the nextIndex value is not used in the equality comparison.

8.4.4.2. Type checking

Typing type `[ $expression_{size}$ `], under context TC at p:

1. Let $typeIR_{base}$ and fresh type variables $typeId_{fresh}^{*}$ be the result of typing type, under context TC at p.
2. Let $typeIR_{base'}$ be $typeIR_{base}$ with typedefs unrolled.
3. If $typeIR_{base'}$ has type headerTypeIR or $typeIR_{base'}$ has type headerUnionTypeIR:
    a. Let $typedExpressionIR_{size}$ be the result of typing $expression_{size}$, under context TC at p.
    b. If the compile-time known-ness of $typedExpressionIR_{size}$ is LCTK:
        i. Let value be the result of local compile-time evaluation of $typedExpressionIR_{size}$, under context TC at p.
        ii. If let $integerValue_{size}$ of type integerValue be value:
            A. Let $nat^{?}$ be the natural number representation of $integerValue_{size}$.
            B. If $nat^{?}$ is defined:
                I. Let n_{size} be $nat^{?}$.
                II. If n_{size} is greater than 0:
                    1. The result is HEADER_STACK $typeIR_{base}$ `[ $n_{size}$ `] and fresh type variables $typeId_{fresh}^{*}$.

8.4.4.3. Well-formedness

HEADER_STACK typeIR `[ nat `] is a well-formed type, with bound type variables B:

1. If typeIR can be nested in a header stack type:
    a. If typeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.4.5. Structs

P4 struct types are defined with the following syntax:

```
typeField
    : annotationList type name ;
    ;

structTypeDeclaration
    : annotationList STRUCT name typeParameterListOpt { typeFieldList }
    ;
```

This declaration introduces a new struct type definition with the specified name in the current scope. Field names have to be distinct. An empty struct (with no fields) is legal.

```
structTypeDefIR
    : STRUCT typeId < typeParameterIR* > { fieldTypeIR* }
    ;
```

For example, the structure Parsed_headers below contains the headers recognized by a simple parser:

```
header Tcp_h { /* fields omitted */ }
header Udp_h { /* fields omitted */ }
struct Parsed_headers {
    Ethernet_h ethernet;
    Ip_h       ip;
    Tcp_h      tcp;
    Udp_h      udp;
}
```

Struct types can be used by referring to their names. Generic struct types can be used by specializing their type parameters with type arguments. Internally, struct types and values are represented as follows:

```
fieldTypeIR
    : annotationList typeIR nameIR ;
    ;

structTypeIR
    : STRUCT typeId < typeArgumentIR* > { fieldTypeIR* }
    ;

fieldValue
    : value nameIR ;
    ;

structValue
    : STRUCT typeId { fieldValue* }
    ;
```

8.4.5.1. Operations

The only operation defined on expressions whose type is a struct is field access, written using dot (".") notation—e.g., s.field. If s is an l-value, then s.field is also an l-value. P4 also allows copying structs using assignment when the source and target of the assignment have the same type. Finally, structs can be initialized with a sequence expression, as discussed in Section 8.6.3, or with a record expression, as described in Section 8.6.4. Both of these cases must initialize all fields of the structure. The size of a struct can be determined at compile-time ([sec-minsizeinbits]).

Two structs can be compared for equality (==) or inequality (!=) only if they have the same type and all of their fields can be recursively compared for equality. Two structures are equal if and only if all their corresponding fields are equal.

The following example shows a structure initialized in several different ways:

```
struct S {
    bit<32> a;
    bit<32> b;
}
const S x = { 10, 20 };             // sequence expression
const S x = { a = 10, b = 20 };     // record expression
const S x = (S) { a = 10, b = 20 }; // record expression
```

See [sec-uninitialized-values-and-writing-invalid-headers] for a description of the behavior if struct fields are read without being initialized.

8.4.5.2. Type checking

Struct types are introduced by struct declarations. Its typing rule is explained in Section 11.14. Struct types are then used by referencing their names. Thus struct types can be obtained from named types, as illustrated in Section 8.3.

8.4.5.3. Well-formedness

STRUCT typeId `< typeArgumentIR* `> `{ ( annotationList typeIR$_{field}$ nameIR$_{field}$ ';' )* `} is a well-formed type, with bound type variables B:

1. If the elements of nameIR$_{field}$* are distinct:
    a. If typeIR$_{field}$ can be nested in a struct type, for all typeIR$_{field}$ in typeIR$_{field}$*:
        i. If typeIR$_{field}$ is a well-formed type, with bound type variables B, for all typeIR$_{field}$ in typeIR$_{field}$*: then, the relation holds.

8.4.6. Headers

The declaration of a header type is given by the following syntax:

```
typeField
    : annotationList type name ;
    ;

headerTypeDeclaration
    : annotationList HEADER name typeParameterListOpt { typeFieldList }
    ;
```

where each type is restricted to a bit-string type (fixed or variable), a fixed-width signed integer type, a boolean type, or a struct that itself contains other struct fields, nested arbitrarily, as long as all of the "leaf" types are bit<W>, int<W>, a serializable enum, or a bool. If a bool is used inside a P4 header, all implementations encode the bool as a one bit long field, with the value 1 representing true and 0 representing false. Field names have to be distinct.

A header declaration introduces a new identifier in the current scope; the type definition can be referred to using this identifier.

```
headerTypeDefIR
    : HEADER typeId < typeParameterIR* > { fieldTypeIR* }
    ;
```

A header is similar to a struct in C, containing all the specified fields. However, in addition, a header also contains a hidden Boolean "validity" field. When the "validity" bit is true we say that the "header is valid". When a local variable with a header type is declared, its "validity" bit is automatically set to false. The "validity" bit can be manipulated by using the header methods isValid(), setValid(), and setInvalid().

Header types may be empty:

```
header Empty_h { }
```

Note that an empty header still contains a validity bit.

When a struct is inside of a header, the order of the fields for the purposes of extract and emit calls is the order of the fields as defined in the source code. An example of a header including a struct is included below.

```
struct ipv6_addr {
    bit<32> Addr0;
    bit<32> Addr1;
    bit<32> Addr2;
    bit<32> Addr3;
}

header ipv6_t {
    bit<4>    version;
    bit<8>    trafficClass;
    bit<20>   flowLabel;
    bit<16>   payloadLen;
    bit<8>    nextHdr;
    bit<8>    hopLimit;
    ipv6_addr src;
    ipv6_addr dst;
}
```

Headers that do not contain any varbit field are *fixed size*. Headers containing varbit fields have *variable size*. The size (in bits) of a fixed-size header is simply the sum of the sizes of all component fields (without counting the validity bit). There is no padding or alignment of the header fields. Targets may impose additional constraints on header types—e.g., restricting headers to sizes that are an integer number of bytes.

For example, the following declaration describes a typical Ethernet header:

```
header Ethernet_h {
   bit<48> dstAddr;
   bit<48> srcAddr;
   bit<16> etherType;
}
```

The following variable declaration uses the newly introduced type Ethernet_h:

```
Ethernet_h ethernetHeader;
```

P4's parser language provides an extract method that can be used to "fill in" the fields of a header from a network packet, as described in [sec-packet-data-extraction]. The successful execution of an extract operation also sets the validity bit of the extracted header to true.

Here is an example of an IPv4 header with variable-sized options:

```
header IPv4_h {
    bit<4>       version;
    bit<4>       ihl;
    bit<8>       diffserv;
    bit<16>      totalLen;
    bit<16>      identification;
    bit<3>       flags;
    bit<13>      fragOffset;
    bit<8>       ttl;
    bit<8>       protocol;
    bit<16>      hdrChecksum;
    bit<32>      srcAddr;
    bit<32>      dstAddr;
    varbit<320>  options;
}
```

As demonstrated by a code example in [sec-packet-extract-two], another way to support headers that contain variable-length fields is to define two headers—one fixed length, one containing a varbit field—and extract each part in separate parsing steps.

Notice that the names isValid, setValid, minSizeInBits, etc. are all valid header field names.

Internally, header types and values are represented as follows:

```
headerTypeIR
    : HEADER typeId < typeArgumentIR* > { fieldTypeIR* }
    ;

headerValue
    : HEADER typeId { bool ; fieldValue* }
    ;
```

8.4.6.1. Operations

Headers provide the same operations as structs. Assignment between headers also copies the "validity" header bit.

In addition, headers support the following methods:

- The method isValid() returns the value of the "validity" bit of the header.
- The method setValid() sets the header's validity bit to "true". It can only be applied to an l-value.
- The method setInvalid() sets the header's validity bit to "false". It can only be applied to an l-value.

A header value's validity is read and invalidated with:

$isValid_header(value)

1. Check that value has type headerValue.
2. Let headerValue be value.
3. Let HEADER _ `{ $b_{valid}$ ';' _ $^{*}$ `} be headerValue.
4. Return b_{valid}.

$invalidate_header(HEADER typeId `{ _ ';' fieldValue$^{*}$ `})

1. Return HEADER typeId `{ false ';' fieldValue$^{*}$ `}.

Similar to a struct, a header object can be initialized with a sequence expression (see Section 8.6.3)—the sequence elements are assigned to the header fields in the order they appear—or with a record expression (see Section 8.6.4). When initialized the header automatically becomes valid:

```
header H { bit<32> x; bit<32> y; }
H h;
h = { 10, 12 };  // This also makes the header h valid
h = { y = 12, x = 10 };  // Same effect as above
```

Two headers can be compared for equality (==) or inequality (!=) only if they have the same type. Two headers are equal if and only if they are both invalid, or they are both valid and all their corresponding fields are equal. Furthermore, the size of a header can be determined at compile-time ([sec-minsizeinbits]).

See [sec-uninitialized-values-and-writing-invalid-headers] for a description of the behavior if header fields are read without being initialized, or header fields are written to a currently invalid header.

8.4.6.2. Type checking

Header types are introduced by header declarations. Its typing rule is explained in Section 11.15. Header types are then used by referencing their names. Thus header types can be obtained from named types, as illustrated in Section 8.3.

8.4.6.3. Well-formedness

<span id=""Type_wf-headerTypeIR">

HEADER typeId `< typeArgumentIR$^{*}$ `> `{ ( annotationList $typeIR_{field}$ $nameIR_{field}$ ';' )$^{*}$ `} is a well-formed type, with bound type variables B:

1. If the elements of $nameIR_{field}^{*}$ are distinct:
   a. If $typeIR_{field}$ can be used in a header type, for all $typeIR_{field}$ in $typeIR_{field}^{*}$:
      i. If $typeIR_{field}$ is a well-formed type, with bound type variables B, for all $typeIR_{field}$ in $typeIR_{field}^{*}$: then, the relation holds.

8.4.7. Header unions

A header union represents an alternative containing at most one of several different headers. Header unions can be used to represent "options" in protocols like TCP and IP. They also provide hints to P4

compilers that only one alternative will be present, allowing them to conserve storage resources.

A header union is defined as:

```
typeField
    : annotationList type name ;
    ;

headerUnionTypeDeclaration
    : annotationList HEADER_UNION name typeParameterListOpt { typeFieldList }
    ;
```

This declaration introduces a new type definition with the specified name in the current scope.

```
headerUnionTypeDefIR
    : HEADER_UNION typeId < typeParameterIR* > { fieldTypeIR* }
    ;
```

Each element of the list of fields used to declare a header union must be of header type. An empty list of fields is legal. Field names have to be distinct.

As an example, the type Ip_h below represents the union of an IPv4 and IPv6 headers:

```
header_union IP_h {
  IPv4_h v4;
  IPv6_h v6;
}
```

A header union is not considered a type with fixed length.

Internally, header union types and values are represented as follows:

```
headerUnionTypeIR
    : HEADER_UNION typeId < typeArgumentIR* > { fieldTypeIR* }
    ;

headerUnionValue
    : HEADER_UNION typeId { fieldValue* }
    ;
```

8.4.7.1. Operations

A variable declared with a union type is initially invalid. For example:

```
header H1 {
  bit<8> f;
}
header H2 {
  bit<16> g;
}
header_union U {
```

```
  H1 h1;
  H2 h2;
}

U u; // u invalid
```

This also implies that each of the headers h1 through hn contained in a header union are also initially invalid. Unlike headers, a union cannot be initialized. However, the validity of a header union can be updated by assigning a valid header to one of its elements:

```
U u;
H1 my_h1 = { 8w0 }; // my_h1 is valid
u.h1 = my_h1;       // u and u.h1 are both valid
```

We can also assign a sequence to an element of a header union,

```
U u;
u.h2 = { 16w1 };     // u and u.h2 are both valid
```

or set their validity bits directly.

```
U u;
u.h1.setValid();     // u and u.h1 are both valid
H1 my_h1 = u.h1;     // my_h1 is now valid, but contains an undefined value
```

Note that reading an uninitialized header produces an undefined value, even if the header is itself valid.

If u is an expression whose type is a header union U with fields ranged over by hi, then the expression u.hi evaluates to a header, and thus it can be used wherever a header expression is allowed. If u is a left-value, then u.hi is also a left-value.

The following operations:

- u.hi.setValid(): sets the valid bit for header hi to true and sets the valid bit for all other headers to false, which implies that it is unspecified what value reading any member header of u will return.
- u.hi.setInvalid(): if the valid bit for any member header of u is true then sets it to false, which implies that it is unspecified what value reading any member header of u will return.

The assignment to a union field:

```
u.hi = e;
```

has the following meaning:

- if e is valid, then it is equivalent to:

```
  u.hi.setValid();
  u.hi = e;
```

- if e is invalid, then it is equivalent to:

```
  u.hi.setInvalid();
```

Assignments between variables of the same type of header union are permitted. The assignment u1 = u2 copies the full state of header union u2 to u1. If u2 is valid, then there is some header u2.hi that is valid. The assignment behaves the same as u1.hi = u2.hi. If u2 is not valid, then u1 becomes invalid (i.e. if any header of u1 was valid, it becomes invalid).

u.isValid() returns true if any member of the header union u is valid, otherwise it returns false. setValid() and setInvalid() methods are not defined for header unions.

Supplying an expression with a union type to emit simply emits the single header that is valid, if any.

A header union value's validity is invalidated with:

$invalidate_headerUnion(HEADER_UNION typeId `{ ( $value_{field}$ $nameIR_{field}$ ';' )* `})

1. For each $value_{field}$ in $value_{field}^*$:
   - Let $value_{field_invalidated}$ be $invalidate_value($value_{field}$).

   Let $value_{field_invalidated}^*$ be the resulting list.
2. Return HEADER_UNION typeId `{ ( $value_{field\_invalidated}$ $nameIR_{field}$ ';' )* `}.

The following example shows how we can use header unions to represent IPv4 and IPv6 headers uniformly:

```
header_union IP {
    IPv4 ipv4;
    IPv6 ipv6;
}

struct Parsed_packet {
   Ethernet ethernet;
   IP ip;
}

parser top(packet_in b, out Parsed_packet p) {
    state start {
       b.extract(p.ethernet);
       transition select(p.ethernet.etherType) {
           16w0x0800 : parse_ipv4;
           16w0x86DD : parse_ipv6;
       }
   }
   state parse_ipv4 {
       b.extract(p.ip.ipv4);
       transition accept;
   }
   state parse_ipv6 {
       b.extract(p.ip.ipv6);
       transition accept;
   }
}
```

As another example, we can also use unions to parse (selected) TCP options:

```
header Tcp_option_end_h {
    bit<8> kind;
}
header Tcp_option_nop_h {
    bit<8> kind;
```

```
}
header Tcp_option_ss_h {
    bit<8>  kind;
    bit<32> maxSegmentSize;
}
header Tcp_option_s_h {
    bit<8>  kind;
    bit<24> scale;
}
header Tcp_option_sack_h {
    bit<8>         kind;
    bit<8>         length;
    varbit<256>    sack;
}
header_union Tcp_option_h {
    Tcp_option_end_h  end;
    Tcp_option_nop_h  nop;
    Tcp_option_ss_h   ss;
    Tcp_option_s_h    s;
    Tcp_option_sack_h sack;
}

typedef Tcp_option_h[10] Tcp_option_stack;

struct Tcp_option_sack_top {
    bit<8> kind;
    bit<8> length;
}

parser Tcp_option_parser(packet_in b, out Tcp_option_stack vec) {
    state start {
        transition select(b.lookahead<bit<8>>()) {
            8w0x0 : parse_tcp_option_end;
            8w0x1 : parse_tcp_option_nop;
            8w0x2 : parse_tcp_option_ss;
            8w0x3 : parse_tcp_option_s;
            8w0x5 : parse_tcp_option_sack;
        }
    }
    state parse_tcp_option_end {
        b.extract(vec.next.end);
        transition accept;
    }
    state parse_tcp_option_nop {
         b.extract(vec.next.nop);
         transition start;
    }
    state parse_tcp_option_ss {
         b.extract(vec.next.ss);
         transition start;
    }
    state parse_tcp_option_s {
         b.extract(vec.next.s);
         transition start;
    }
    state parse_tcp_option_sack {
         bit<8> n = b.lookahead<Tcp_option_sack_top>().length;
         // n is the total length of the TCP SACK option in bytes.
         // The length of the varbit field 'sack' of the
         // Tcp_option_sack_h header is thus n-2 bytes.
         b.extract(vec.next.sack, (bit<32>) (8 * n - 16));
         transition start;
    }
}
```

Similar to headers, the size of a header union is a local compile-time known value (Section [sec-minsizeinbits]).

Two header unions can be compared for equality (==) or inequality (!=) if they have the same type. The unions are equal if and only if all their corresponding fields are equal (i.e., either all fields are invalid in both unions, or in both unions the same field is valid, and the values of the valid fields are equal as headers).

8.4.7.2. Type checking

Header union types are introduced by header_union declarations. Its typing rule is explained in Section 11.16. Header union types are then used by referencing their names. Thus header union types can be obtained from named types, as illustrated in Section 8.3.

8.4.7.3. Well-formedness

HEADER_UNION typeId `< typeArgumentIR* `> `{ ( annotationList typeIR$_{field}$ nameIR$_{field}$ ';' )* `} is a well-formed type, with bound type variables B:

1. If the elements of nameIR$_{field}$* are distinct:
   a. If typeIR$_{field}$ can be used in a header union type, for all typeIR$_{field}$ in typeIR$_{field}$*:
      i. If typeIR$_{field}$ is a well-formed type, with bound type variables B, for all typeIR$_{field}$ in typeIR$_{field}$*: then, the relation holds.

8.4.8. Enums

An enumeration type is defined using the following syntax:

```
namedExpression
    : name = expression
    ;

namedExpressionList
    : namedExpression | namedExpressionList , namedExpression
    ;

enumTypeDeclaration
    : annotationList ENUM name { nameList trailingCommaOpt }
    | annotationList ENUM type name { namedExpressionList trailingCommaOpt }
    ;
```

An enum declaration introduces a new enum type definition:

```
enumTypeDefIR = enumTypeIR
```

Enums cannot be parameterized over type parameters, thus is always monomorphic.

Enums can be declared in two different ways, depending on whether or not an underlying representation type is provided. The two forms are described below.

Internally, enum types and values are represented as follows:

```
<span id="valueFieldIR"></span>
<span id="enumTypeIR"></span>
<span id="enumValue"></span>
```

```
valueFieldIR
    : nameIR = value ;
    ;

enumTypeIR
    : simpleEnumTypeIR | serializableEnumTypeIR
    ;

enumValue
    : typeId . nameIR | typeId . nameIR . value
    ;
```

8.4.8.1. Enum type declaration without an underlying type

For example, the declaration

```
enum Suits { Clubs, Diamonds, Hearths, Spades }
```

introduces a new enumeration type, which contains four elements—e.g., Suits.Clubs. An enum declaration introduces a new identifier in the current scope for naming the created type along with its distinct elements. The underlying representation of the Suits enum is not specified, so their "size" in bits is not specified (it is target-specific).

8.4.8.2. Enum type declaration with an underlying type

It is also possible to specify an enum with an underlying representation. These are sometimes called serializable enums, because headers are allowed to have fields with such enum types. This requires the programmer provide both the fixed-width unsigned (or signed) integer type and an associated integer value for each symbolic entry in the enumeration.

For example, the declaration

```
enum bit<16> EtherType {
  VLAN      = 0x8100,
  QINQ      = 0x9100,
  MPLS      = 0x8847,
  IPV4      = 0x0800,
  IPV6      = 0x86dd
}
```

introduces a new enumeration type, which contains five elements—e.g., EtherType.IPV4. This enum declaration specifies the fixed-width unsigned integer representation for each entry in the enum and provides an underlying type: bit<16>. This kind of enum declaration can be thought of as declaring a new bit<16> type, where variables or fields of this type are expected to be unsigned 16-bit integer values, and the mapping of symbolic to numeric values defined by the enum are also defined as a part of this declaration. In this way, an enum with an underlying type can be thought of as being a type derived from the underlying type carrying equality, assignment, and casts to/from the underlying type.

Compiler implementations are expected to raise an error if the fixed-width integer representation for an enumeration entry falls outside the representation range of the underlying type.

For example, the declaration

```
enum bit<8> FailingExample {
  first           = 1,
  second          = 2,
  third           = 3,
  unrepresentable = 300
}
```

would raise an error because 300, the value associated with FailingExample.unrepresentable cannot be represented as a bit<8> value.

The initializer expression must be a local compile-time known value.

Annotations, represented by the non-terminal annotationList, are described in [sec-annotations].

8.4.8.3. Operations

Symbolic names declared by an enum belong to the namespace introduced by the enum declaration rather than the top-level namespace.

```
enum X { v1, v2, v3 }
X.v1  // reference to v1
v1    // error - v1 is not in the top-level namespace
```

Similar to errors, enum expressions without a specified underlying type only support equality (==) and inequality (!=) comparisons. Expressions whose type is an enum without a specified underlying type cannot be cast to or from any other type.

An enum may also specify an underlying type, such as the following:

```
enum bit<8> E {
  e1 = 0,
  e2 = 1,
  e3 = 2
}
```

More than one symbolic value in an enum may map to the same fixed-width integer value.

```
enum bit<8> NonUnique {
  b1 = 0,
  b2 = 1,  // Note, both b2 and b3 map to the same value.
  b3 = 1,
  b4 = 2
}
```

An enum with an underlying type also supports explicit casts to and from the underlying type. For instance, the following code:

```
bit<8> x;
E a = E.e2;
E b;

x = (bit<8>) a; // sets x to 1
b = (E) x;      // sets b to E.e2
```

casts `a` (which was initialized with `E.e2`) to a `bit<8>`, using the specified fixed-width unsigned integer representation for `E.e2`, i.e. `1`. The variable `b` is then set to the symbolic value `E.e2`, which corresponds to the fixed-width unsigned integer value `1`.

Because it is always safe to cast from an `enum` to its underlying fixed-width integer type, implicit casting from an `enum` to its fixed-width (signed or unsigned) integer type is also supported (see Section 18.3):

```
bit<8> x = E.e2; // sets x to 1 (E.e2 is automatically casted to bit<8>)

E  a = E.e2
bit<8> y = a << 3; // sets y to 8 (a is automatically casted to bit<8> and then shifted)
```

Implicit casting from an underlying fixed-width type to an enum is **not** supported.

```
enum bit<8> E1 {
   e1 = 0, e2 = 1, e3 = 2
}

enum bit<8> E2 {
   e1 = 10, e2 = 11, e3 = 12
}
E1 a = E1.e1;
E2 b = E2.e2;

a = b;      // Error: b is automatically casted to bit<8>,
            // but bit<8> cannot be automatically casted to E1

a = (E1) b; // OK

a = E1.e1 + 1; // Error: E.e1 is automatically casted to bit<8>,
               // and the right-hand expression has
               // the type bit<8>, which cannot be casted to E automatically.

a = (E1)(E1.e1 + 1); // Final explicit casting makes the assignment legal

a = E1.e1 + E1.e2; // Error: both arguments to the addition are automatically
                   // casted to bit<8>. Thus the addition itself is legal, but
                   // the assignment is not

a = (E1)(E2.e1 + E2.e2); //  Final explicit casting makes the assignment legal
```

A reasonable compiler might generate a warning in cases that involve multiple automatic casts.

```
E1     a = E1.e1;
E2     b = E2.e2;
bit<8> c;

if (a > b) { // Potential warning: two automatic and different casts to bit<8>.
   // code omitted
}

c = a + b; // Legal, but a warning would be reasonable
```

Note that while it is always safe to cast from an `enum` to its fixed-width unsigned integer type, and vice versa, there may be cases where casting a fixed-width unsigned integer value to its related `enum` type produces an unnamed value.

```
bit<8> x = 5;
E e = (E) x; // sets e to an unnamed value
```

sets e to an unnamed value, since there is no symbol corresponding to the fixed-width unsigned integer value 5.

For example, in the following code, the else clause of the if/else if/else block can be reached even though the matches on x are complete with respect to the symbols defined in MyPartialEnum_t:

```
enum bit<2> MyPartialEnum_t {
    VALUE_A = 2w0,
    VALUE_B = 2w1,
    VALUE_C = 2w2
}

bit<2> y = < some value >;
MyPartialEnum_t x = (MyPartialEnum_t)y;

if (x == MyPartialEnum_t.VALUE_A) {
    // some code here
} else if (x == MyPartialEnum_t.VALUE_B) {
    // some code here
} else if (x == MyPartialEnum_t.VALUE_C) {
    // some code here
} else {
    // A P4 compiler MUST ASSUME that this branch can be executed
    // some code here
}
```

Additionally, if an enumeration is used as a field of a header, we would expect the transition select to match default when the parsed integer value does not match one of the symbolic values of EtherType in the following example:

```
enum bit<16> EtherType {
  VLAN      = 0x8100,
  IPV4      = 0x0800,
  IPV6      = 0x86dd
}

header ethernet {
  // Some fields omitted
  EtherType etherType;
}

parser my_parser(/* parameters omitted */) {
  state parse_ethernet {
    packet.extract(hdr.ethernet);
    transition select(hdr.ethernet.etherType) {
      EtherType.VLAN : parse_vlan;
      EtherType.IPV4 : parse_ipv4;
      EtherType.IPV6: parse_ipv6;
      default: reject;
  }
}
```

Any variable with an enum type that contains an unnamed value (whether as the result of a cast to an enum with an underlying type, parse into the field of an enum with an underlying type, or simply the declaration of any enum without a specified initial value) will not be equal to any of the values defined for that type. Such an unnamed value should still lead to predictable behavior in cases where any legal value would match, e.g. it should match in any of these situations:

- If used in a select expression, it should match default or _ in a key set expression.
- If used as a key with match_kind ternary in a table, it should match a table entry where the field has all bit positions "don't care".

- If used as a key with match_kind lpm in a table, it should match a table entry where the field has a prefix length of 0.

Note that if an enum value lacking an underlying type appears in the control-plane API, the compiler must select a suitable serialization data type and representation. For enum values with an underlying type and representations, the compiler should use the specified underlying type as the serialization data type and representation.

Additionally, the size of a serializable enum can be determined at compile-time. However, the size of an enum without an underlying type cannot be determined at compile-time ([sec-minsizeinbits]).

8.4.8.4. Type checking

Enum types are introduced by enum declarations. Its typing rule is explained in Section 11.13. Enum types are then used by referencing their names. Thus enum types can be obtained from named types, as illustrated in Section 8.3.

8.4.8.5. Well-formedness

typeIR' is a well-formed type, with bound type variables B:

1. If let ENUM _ `{ nameIR$_{field}^{*}$ `} of type simpleEnumTypeIR be typeIR':
   a. If the elements of nameIR$_{field}^{*}$ are distinct: then, the relation holds.
2. Else if let ENUM _ `< typeIR `> `{ ( nameIR$_{field}$ '=' _ ';' )$^{*}$ `} of type serializableEnumTypeIR be typeIR':
   a. If the elements of nameIR$_{field}^{*}$ are distinct:
      i. If typeIR can be used in a serializable enum type:
         A. If typeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.5. Object types and values

```
objectTypeIR
    : externObjectTypeIR | parserObjectTypeIR | controlObjectTypeIR
    | packageObjectTypeIR | tableObjectTypeIR
    ;

objectReferenceValue
    : REF objectId
    ;
```

Stateful objects are represented using objectTypeIR. At runtime, these objects are passed along directionless parameters as references (objectReferenceValue).

The types parser, control, and package cannot be used as types of arguments for actions, functions, methods, parsers, controls, or tables. They *can* be used as types for the arguments passed to constructors.

8.5.1. Extern object types

An extern object declaration declares an object and all methods that can be invoked to perform

computations and to alter the state of the object. Extern declarations may only appear as allowed by the architecture model and may be specific to a target.

```
externConstructorPrototype
    : annotationList typeIdentifier ( parameterList ) ;
    ;

externMethodPrototype
    : annotationList functionPrototype ;
    | annotationList ABSTRACT functionPrototype ;
    ;

externConstructorOrMethodPrototype
    : externConstructorPrototype | externMethodPrototype
    ;

externObjectDeclaration
    : annotationList EXTERN nonTypeName typeParameterListOpt
        { externConstructorOrMethodPrototypeList }
    ;
```

For example, the P4 core library introduces two extern objects packet_in and packet_out used for manipulating packets (see [sec-packet-data-extraction] and [sec-deparse]). Here is an example showing how the methods of these objects can be invoked on a packet:

```
extern packet_out {
    void emit<T>(in T hdr);
}
control d(packet_out b, in Hdr h) {
    apply {
        b.emit(h.ipv4);       // write ipv4 header into output packet
    }                         // by calling emit method
}
```

An extern object declaration introduces a new extern object type definition.

```
externObjectTypeDefIR
    : EXTERN typeId < typeParameterIR* , typeParameterIR* >
        externMethodTypeDefEnv
    ;
```

Extern object types can be referred to using their name. Internally, extern object types are represented as:

```
externObjectTypeIR
    : EXTERN typeId < typeArgumentIR* > externMethodTypeDefEnv
    ;
```

8.5.1.1. Type checking

Extern object types are introduced by extern declarations. Its typing rule is explained in Section 11.10.

Extern object types are then used by referencing their names. Thus extern object types can be obtained from named types, as illustrated in Section 8.3.

8.5.1.2. Well-formedness

EXTERN typeId `< typeArgumentIR* `> `{ ( callableId ':' externMethodTypeDefIR )* `} is a well-formed type, with bound type variables B:

1. If externMethodTypeDefIR is a well-formed callable type definition, with bound type variables B, for all externMethodTypeDefIR in externMethodTypeDefIR*: then, the relation holds.

8.5.2. Parser object types

A parser type declaration describes the signature of a parser. A parser should have at least one argument of type packet_in, representing the received packet that is processed.

```
parserTypeDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList ) ;
    ;
```

For example, the following is a type declaration of a parser type named P that is parameterized on a type variable H. That parser receives as input a packet_in value b and produces two values:

- A value with a user-defined type H
- A value with a predefined type Counters

```
struct Counters { /* Fields omitted */ }
parser P<H>(packet_in b,
            out H packetHeaders,
            out Counters counters);
```

A parser type declaration introduces a new parser object type definition.

```
parserObjectTypeDefIR
    : PARSER typeId < typeParameterIR* , typeParameterIR* > ( parameterIR* )
    ;
```

Parser types can be referred to using their name and type arguments. Internally, parser types are represented as:

```
parserObjectTypeIR
    : PARSER typeId < typeArgumentIR* > ( parameterIR* )
    ;
```

8.5.2.1. Type checking

Parser object types are introduced by parser type declarations. Its typing rule is explained in Section 11.18. Parser object types are then used by referencing their names. Thus parser object types can be obtained from named types, as illustrated in Section 8.3.

8.5.2.2. Well-formedness

PARSER typeId `< typeArgumentIR* `> `( parameterIR* `) is a well-formed type, with bound type variables B:

1. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*: then, the relation holds.

8.5.3. Control object types

A control type declaration describes the signature of a control block.

```
controlTypeDeclaration
    : annotationList CONTROL name typeParameterListOpt ( parameterList ) ;
    ;
```

Control type declarations are similar to parser type declarations.

A control type declaration introduces a new control object type definition.

```
controlObjectTypeDefIR
    : CONTROL typeId < typeParameterIR* , typeParameterIR* > ( parameterIR* )
    ;
```

Control types can be referred to using their name and type arguments. Internally, control types are represented as:

```
controlObjectTypeIR
    : CONTROL typeId < typeArgumentIR* > ( parameterIR* )
    ;
```

8.5.3.1. Type checking

Control object types are introduced by control type declarations. Its typing rule is explained in Section 11.19. Control object types are then used by referencing their names. Thus control object types can be obtained from named types, as illustrated in Section 8.3.

8.5.3.2. Well-formedness

CONTROL typeId `< typeArgumentIR* `> `( parameterIR* `) is a well-formed type, with bound type variables B:

1. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*: then, the relation holds.

8.5.4. Package object types

A package type describes the signature of a package.

```
packageTypeDeclaration
    : annotationList PACKAGE name typeParameterListOpt ( parameterList ) ;
    ;
```

This introduces a new package object type definition.

```
packageObjectTypeDefIR
    : PACKAGE typeId < typeParameterIR* , typeParameterIR* > { typeIR* }
    ;
```

Package types can be referred to using their name and type arguments. Internally, package types are represented as:

```
packageObjectTypeIR
    : PACKAGE typeId < typeArgumentIR* > { typeIR* }
    ;
```

8.5.4.1. Type checking

Package object types are introduced by package declarations. Its typing rule is explained in Section 11.20. Package object types are then used by referencing their names. Thus package object types can be obtained from named types, as illustrated in Section 8.3.

8.5.4.2. Well-formedness

PACKAGE typeId `< typeArgumentIR* `> `{ typeIR* `} is a well-formed type, with bound type variables B:

1. If typeIR is a well-formed type, with bound type variables B, for all typeIR in typeIR*: then, the relation holds.

8.5.5. Table object types

A table declaration introduces a table instance and a table type definition.

```
tableDeclaration
    : annotationList TABLE name { tablePropertyList }
    ;

tableObjectTypeDefIR = tableObjectTypeIR
```

Internally, table types are represented as:

```
tableObjectTypeIR
    : TABLE typeId { tableMetadataStructTypeIR }
    ;
```

Table metadata types are introduced in [sec-table-metadata-type-and-value].

8.5.5.1. Well-formedness

TABLE typeId `{ tableMetadataStructTypeIR `} is a well-formed type, with bound type variables B:

1. If tableMetadataStructTypeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.5.6. Object reference values

P4 objects are stateful, thus their values are references to the actual objects allocated in the global store.

```
objectId = nameIR*

objectReferenceValue
    : REF objectId
    ;
```

Objects are referred to by their fully-qualified names from the top-level scope. === Alias types

A typedef or type declaration can be used to give an alias to a type.

```
typedef
    : type | derivedTypeDeclaration
    ;

typedefDeclaration
    : annotationList TYPEDEF typedef name ; | annotationList TYPE type name ;
    ;
```

Internally, the alias is represented as:

```
aliasTypeIR
```

```
    : typedefTypeIR | newTypeIR
    ;
```

8.5.7. typedef declaration

```
typedef bit<32> u32;
typedef struct Point { int<32> x; int<32> y; } Pt;
typedef Empty_h[32] HeaderStack;
```

The two types are treated as synonyms, and all operations that can be executed using the original type can be also executed using the newly created type.

If typedef is used with a generic type the type must be specialized with the suitable number of type arguments:

```
struct S<T> {
   T field;
}

// typedef S X;  -- illegal: S does not have type arguments
typedef S<bit<32>> X;  // -- legal
```

Internally, the alias is represented as:

```
typedefTypeIR
    : TYPEDEF typeId typeIR
    ;
```

8.5.7.1. Type checking

typedefs are introduced by typedef declarations. Its typing rule is explained in Section 11.17.1. typedefs are then used by referencing their names. Thus typedef types can be obtained from named types, as illustrated in Section 8.3.

8.5.7.2. Well-formedness

TYPEDEF typeId typeIR is a well-formed type, with bound type variables B:

1. If typeIR can be used in a typedef:
   a. If typeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.5.8. type declaration

Similarly to typedef, the keyword type can be used to introduce a new type.

```
type bit<32> U32;
U32 x = (U32)0;
```

While similar to typedef, the type keyword introduces a new type which is not a synonym with the original type: values of the original type and the newly introduced type cannot be mixed in expressions.

Currently the types that can be created by the type keyword are restricted to one of: bit<>, int<>, bool, or types defined using type from such types.

One important use of such types is in describing P4 values that need to be exchanged with the control plane through communication channels (e.g., through the control-plane API or through network packets sent to the control plane). For example, a P4 architecture may define a type for the switch ports:

```
type bit<9> PortId_t;
```

This declaration will prevent PortId_t values from being used in arithmetic expressions without casts. Moreover, this declaration may enable special manipulation or such values by software that lies outside of the datapath (e.g., a target-specific toolchain could include software that automatically converts values of type PortId_t to a different representation when exchanged with the control-plane software).

Internally, the alias is represented as:

```
newTypeIR
    : TYPE typeId typeIR
    ;
```

8.5.8.1. Operations

Values with a type introduced by the type keyword provide only a few operations:

- assignment to left-values of the same type
- comparisons for equality and inequality if the original type supported such comparisons
- casts to and from the original type

```
type bit<32> U32;
U32 x = (U32)0;  // cast needed
U32 y = (U32) ((bit<32>)x + 1);  // casts needed for arithmetic
bit<32> z = 1;
bool b0 = x == (U32)z; // cast needed
bool b1 = (bit<32>)x == z;  // cast needed
bool b2 = x == y;  // no cast needed
```

8.5.8.2. Type checking

types are introduced by type declarations. Its typing rule is explained in Section 11.17.2. types are then used by referencing their names. Thus type types can be obtained from named types, as illustrated in Section 8.3.

8.5.8.3. Well-formedness

TYPE typeId typeIR is a well-formed type, with bound type variables B:

1. If typeIR can be used in a new type:

a. If typeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.6. Synthesized types and values

For the purposes of type-checking the P4 compiler can synthesize some type representations which cannot be directly expressed by users.

These types and values are represented as:

```
synthesizedTypeIR
    : defaultTypeIR | invalidHeaderTypeIR | sequenceTypeIR | recordTypeIR
    | setTypeIR | tableMetadataTypeIR
    ;

synthesizedValue
    : defaultValue | invalidHeaderValue | sequenceValue | recordValue | setValue
    | tableMetadataValue
    ;
```

8.6.1. Default

A left-value can be initialized automatically with a default value of the suitable type using:

```
defaultExpression
    : ...
    ;
```

Internally, the type and value of ... itself are represented as:

```
defaultTypeIR
    : DEFAULT
    ;

defaultValue
    : DEFAULT
    ;
```

8.6.1.1. Operations

For example:

```
struct S {
     bit<32> b32;
     bool b;
}

enum int<8> NO {
   one = 1,
```

```
    zero = 0,
    two = 2
}

enum N1 {
     A, B, C, F
}

S s0 = ...;  // initialize s0 with the default value { 0, false }
N0 n0 = ...;  // initialize n0 with the default value 0
N1 n1 = ...;  // initialize n1 with the default value N1.A
```

See Section 14.4 for semantics of default initialization.

8.6.1.2. Well-formedness

DEFAULT is a well-formed type, with bound type variables B: then, the relation holds.

8.6.2. Invalid headers

The expression {#} represents an invalid header of some type, but it can be any header or header union type. A P4 compiler may require an explicit cast on this expression in cases where it cannot determine the particular header or header union type from the context.

```
invalidHeaderExpression
    : {#}
    ;
```

Note that the # character cannot be misinterpreted as a preprocessor directive, since it cannot be the first character on a line when it occurs in the single lexical token {#}, which may not have whitespace or any other characters between those shown.

Internally, the type and value of {#} itself are represented as:

```
invalidHeaderTypeIR
    : HEADER_INVALID
    ;

invalidHeaderValue
    : {#}
    ;
```

8.6.2.1. Operations

For example:

```
header H { bit<32> x; bit<32> y; }
H h;
```

```
h = {#};   // This makes the header h become invalid
```

8.6.2.2. Well-formedness

HEADER_INVALID is a well-formed type, with bound type variables B: then, the relation holds.

8.6.3. Sequences

A sequence expression is written using curly braces, with each element separated by a comma:

```
sequenceElementExpression = expressionList

sequenceOrRecordElementExpression
    : sequenceElementExpression | recordElementExpression
    ;

sequenceOrRecordExpression
    : { sequenceOrRecordElementExpression trailingCommaOpt }
    ;
```

The type of a sequence expression is a sequence type. Its value is a sequence value.

```
sequenceTypeIR
    : SEQ < typeIR* > | SEQ < typeIR* , ... >
    ;

sequenceValue
    : SEQ ( value* ) | SEQ ( value* , ... )
    ;
```

8.6.3.1. Operations

Sequence expressions can be assigned to expressions of type tuple, struct or header, and can also be passed as arguments to methods. Sequences may be nested.

For instance, sequences can be assigned to other tuples with the same type:

```
tuple<bit<32>, bool> x = { 10, false };
```

The following program fragment uses a sequence expression to pass several header fields simultaneously to a learning provider:

```
extern LearningProvider<T> {
    LearningProvider();
    void learn(in T data);
}
```

```
LearningProvider<tuple<bit<48>, bit<32>>>() lp;

lp.learn( { hdr.ethernet.srcAddr, hdr.ipv4.src } );
```

A sequence may be used to initialize a structure if the tuple has the same number of elements as fields in the structure. The effect of such an initializer is to assign the n^{th} element of the tuple to the n^{th} field in the structure:

```
struct S {
    bit<32> a;
    bit<32> b;
}
const S x = { 10, 20 }; // a = 10, b = 20
```

Sequence expressions that have ... as their last element are allowed to give values to only a subset of the fields of the struct or header type to which it evaluates. Any field names not given a value explicitly will be given their default value (see [sec-initializing-with-default-values]).

8.6.3.2. Well-formedness

sequenceTypeIR is a well-formed type, with bound type variables B:

1. If let SEQ `< typeIR* `> be sequenceTypeIR:
   a. If typeIR is a well-formed type, with bound type variables B, for all typeIR in typeIR*: then, the relation holds.
2. Else:
   a. Let SEQ `< typeIR* ',' '...' `> be sequenceTypeIR.
   b. If typeIR is a well-formed type, with bound type variables B, for all typeIR in typeIR*: then, the relation holds.

8.6.4. Records

One can write record expressions that evaluate to a structure or header. The syntax of these expressions is given by:

```
recordElementExpression
    : name = expression | name = expression , ...
    | name = expression , namedExpressionList
    | name = expression , namedExpressionList , ...
    ;

sequenceOrRecordElementExpression
    : sequenceElementExpression | recordElementExpression
    ;

sequenceOrRecordExpression
    : { sequenceOrRecordElementExpression trailingCommaOpt }
    ;
```

Internally, the type and value of a record expression are represented as:

```
recordTypeIR
    : RECORD { fieldTypeIR* } | RECORD { fieldTypeIR* , ... }
    ;

recordValue
    : RECORD { fieldValue* } | RECORD { fieldValue* , ... }
    ;
```

8.6.4.1. Operations

The following example shows a structure-valued expression used in an equality comparison expression:

```
struct S {
    bit<32> a;
    bit<32> b;
}

void f() {
  S s;

  // Compare s with a record expression
  bool b = s == (S) { a = 1, b = 2 };
}
```

Record expressions can be used in the right-hand side of assignments, in comparisons, in field selection expressions, and as arguments to functions, method or actions. Record expressions are not left values.

Record expressions that do not have ... as their last element must provide a value for every member of the struct or header type to which it evaluates, by mentioning each field name exactly once.

Record expressions that have ... as their last element are allowed to give values to only a subset of the fields of the struct or header type to which it evaluates. Any field names not given a value explicitly will be given their default values (see [sec-initializing-with-default-values]).

The order of the fields of the struct or header type does not need to match the order of the values of the structure-valued expression.

It is a compile-time error if a field name appears more than once in the same structure-valued expression.

8.6.4.2. Well-formedness

recordTypeIR is a well-formed type, with bound type variables B:

1. If let RECORD `{ ( _ typeIR$_{field}$ nameIR$_{field}$ ';' )$^{*}$ `} be recordTypeIR:
    a. If the elements of nameIR$_{field}^{*}$ are distinct:
        i. If typeIR$_{field}$ is a well-formed type, with bound type variables B, for all typeIR$_{field}$ in typeIR$_{field}^{*}$: then, the relation holds.
2. Else:
    a. Let RECORD `{ ( _ typeIR$_{field}$ nameIR$_{field}$ ';' )$^{*}$ ',' '...' `} be recordTypeIR.
    b. If the elements of nameIR$_{field}^{*}$ are distinct:

i. If $typeIR_{field}$ is a well-formed type, with bound type variables B, for all $typeIR_{field}$ in $typeIR_{field}^{*}$: then, the relation holds.

8.6.5. Sets

```
setTypeIR
    : SET < typeIR >
    ;
```

The set type describes *sets* of values of some type T. Set types can only appear in restricted contexts in P4 programs. For example, the range expression 8w5 .. 8w8 describes a set containing the 8-bit numbers 5, 6, 7, and 8, so its type is set<bit<8>>;. This expression can be used as a label in a select expression (see [sec-select]), matching any value in this range. Set types cannot be named or declared by P4 programmers, they are only synthesized by the compiler internally and used for type-checking.

Set values are represented as:

```
setValue
    : SET { value } | SET { value* ( nat ) } | SET { value &&& value }
    | SET { value .. value } | SET { ... }
    ;
```

8.6.5.1. Operations

Some P4 expressions denote sets of values. These expressions can appear only in a few contexts—parsers and table entries. For example, the select expression ([sec-select]) has the following structure:

```
select (expression) {
   set1: state1;
   set2: state2;
   // More labels omitted
}
```

Here the expressions set1, set2, etc. evaluate to sets of values and the select expression tests whether expression belongs to the sets used as labels.

```
keysetExpression
    : simpleKeysetExpression | tupleKeysetExpression
    ;

tupleKeysetExpression
    : ( expression &&& expression ) | ( expression .. expression ) | ( DEFAULT )
    | ( _ ) | ( simpleKeysetExpression , simpleKeysetExpressionList )
    ;

simpleKeysetExpressionList
    : simpleKeysetExpression
    | simpleKeysetExpressionList , simpleKeysetExpression
```

```
    ;

simpleKeysetExpression
    : expression | expression &&& expression | expression .. expression
    | DEFAULT | _
    ;
```

The mask (`&&&`) and range (`..`) operators have the same precedence; the just above the `?:` operator.

8.6.5.1.1. Singleton sets

In a set context, expressions denote singleton sets. For example, in the following program fragment,

```
select (hdr.ipv4.version) {
   4: continue;
}
```

The label `4` denotes the singleton set containing the `int` value `4`.

8.6.5.1.2. The universal set

In a set context, the expressions `default` or `_` denote the universal set, which contains all possible values of a given type:

```
select (hdr.ipv4.version) {
   4: continue;
   _: reject;
}
```

8.6.5.1.3. Masks

The infix operator `&&&` takes two arguments of the same numeric type ([sec-numeric-values]), and creates a value of the same type. The right value is used as a "mask", where each bit set to `0` in the mask indicates a "don't care" bit. More formally, the set denoted by `a &&& b` is defined as follows:

```
a &&& b = { c where a & b = c & b }
```

For example:

```
8w0x0A &&& 8w0x0F
```

denotes a set that contains 16 different `bit<8>` values, whose bit-pattern is `XXXX1010`, where the value of an `X` can be any bit. Note that there may be multiple ways to express a keyset using a mask operator—e.g., `8w0xFA &&& 8w0x0F` denotes the same keyset as in the example above.

Similar to other binary operations, the mask operator allows the compiler to automatically insert casts to unify the argument types in certain situations (Section 18.3).

P4 architectures may impose additional restrictions on the expressions on the left and right-hand side of a mask operator: for example, they may require that either or both sub-expressions be compile-time known values.

8.6.5.1.4. Ranges

The infix operator .. takes two arguments of the same numeric type T ([sec-numeric-values]), and creates a value of the type set<T>. The set contains all values numerically between the first and the second, inclusively. For example:

```
4s5 .. 4s8
```

denotes a set of 4 consecutive int<4> values 4s5, 4s6, 4s7, and 4s8.

Similar to other binary operations, the range operator allows the compiler to automatically insert casts to unify the argument types in certain situations ([sec-implicit-casts]).

A range where the second value is smaller than the first one represents an empty set.

8.6.5.1.5. Products

Multiple sets can be combined using Cartesian product:

```
select(hdr.ipv4.ihl, hdr.ipv4.protocol) {
     (4w0x5, 8w0x1): parse_icmp;
     (4w0x5, 8w0x6): parse_tcp;
     (4w0x5, 8w0x11): parse_udp;
     (_, _): accept; }
```

The type of a product of sets is a set of sequences or tuples.

8.6.5.2. Well-formedness

SET `< typeIR `> is a well-formed type, with bound type variables B:

1. If typeIR can be used in a set type:
    a. If typeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.6.6. Table metadata

A table can be invoked by calling its apply method. Calling an apply method on a table instance returns a value with a table metadata struct type with three fields. This structure is synthesized by the compiler automatically. For each table T, the compiler synthesizes a table metadata enum and a struct, shown in pseudo-P4:

```
enum action_list(T) {
   // one field for each action in the actions list of table T
}
struct apply_result(T) {
    bool hit;
    bool miss;
    action_list(T) action_run;
}
```

8.6.6.1. Operations

The evaluation of the apply method sets the hit field to true and the field miss to false if a match is found in the lookup-table; if a match is not found hit is set to false and miss to true. These bits can be used to drive the execution of the control-flow in the control block that invoked the table:

```
if (ipv4_match.apply().hit) {
    // there was a hit
} else {
    // there was a miss
}

if (ipv4_host.apply().miss) {
    ipv4_lpm.apply(); // Look up the route only if host table missed
}
```

The action_run field indicates which kind of action was executed (irrespective of whether it was a hit or a miss). It can be used in a switch statement:

```
switch (dmac.apply().action_run) {
    Drop_action: { return; }
}
```

Internally, the type and value of table metadata are represented as:

```
tableMetadataTypeIR
    : tableMetadataEnumTypeIR | tableMetadataStructTypeIR
    ;

tableMetadataValue
    : tableMetadataEnumValue | tableMetadataStructValue
    ;
```

8.6.6.2. Well-formedness

typeIR' is a well-formed type, with bound type variables B:

1. If let TABLE_ENUM _ `{ nameIR* `} of type tableMetadataEnumTypeIR be typeIR':
   a. Check that the elements of nameIR* are distinct.
   b. Then, the relation holds.
2. Else if let TABLE_STRUCT _ `{ HIT _ ';' MISS _ ';' ACTION_RUN tableMetadataEnumTypeIR ';' `} of type tableMetadataStructTypeIR be typeIR':
   a. If tableMetadataEnumTypeIR is a well-formed type, with bound type variables B: then, the relation holds.

8.7. Unrolling

typedefs, described in Section 8.5.7, introduce indirection in types.

```
struct S {
    bit<8> t;
    int<8> u;
}

typedef S A;
```

The type A is an alias of the struct type S. Neverthelss, the underlying type of A is a structTypeIR with two fields, t and u, of types int<8> and bit<8> respectively. It is useful to be able to work with the underlying type of A, when dealing with type checking, type equivalence, and other type-based analyses.

Unrolling is the process of recursively expanding all typedefs until a type is fully expanded to its underlying type.

typeIR' with typedefs unrolled

1. If let TYPEDEF _ typeIR of type typedefTypeIR be typeIR': return typeIR with typedefs unrolled.
2. Otherwise: return typeIR'.

Aliases introduced by type declarations, described in Section 11.17.2, are unrolled similarly, expanding both type aliases and typedefs:

typeIR' with aliases unrolled

1. If let TYPE _ typeIR of type newTypeIR be typeIR': return typeIR with aliases unrolled.
2. Else if let TYPEDEF _ typeIR of type typedefTypeIR be typeIR': return typeIR with aliases unrolled.
3. Otherwise: return typeIR'.

9. P4 callables

$P4_{16}$ includes four entities that can be invoked: actions, functions, methods, and object constructors. Object constructors are explained in [sec-def-object]. The other three callable entities are collectively referred to as *callables*.

Callable types are created by the P4 compiler internally to represent the types of actions, functions, and methods during type-checking. We also call the type of a callable its signature. Libraries can contain callable declarations.

For example, consider the following declarations:

```
extern void random(in bit<5> logRange, out bit<32> value);

bit<32> add(in bit<32> left, in bit<32> right) {
   return left + right;
}
```

These declarations describe two callables:

- random, which has an extern function type, representing the following information:
  - the result type is void

    - the function has two inputs
    - the first formal parameter has direction in, type bit<5>, and name logRange
    - the second formal parameter has direction out, type bit<32>, and name value
- add, also has a function type, representing the following information:
    - the result type is bit<32>
    - the function has two inputs
    - both inputs have direction in and type bit<32>

Specifically, during type checking, callable declarations introduce a type constructor for the callable type:

```
callableTypeDefIR
    : actionTypeDefIR | functionTypeDefIR | methodTypeDefIR
    ;
```

These callable type definitions are used to construct callable types via specialization:

```
callableTypeIR
    : actionTypeIR | functionTypeIR | methodTypeIR
    ;
```

The well-formedness rules for callable types are checked by:

CallableType_wf:

- callableTypeIR is a well-formed callable type, with bound type variables B

CallableType_wf dispatch:

1. If let annotationList ACTION nameIR `(` parameterIR* `)` of type actionTypeIR be callableTypeIR: goto actionTypeIR
2. Else if let FUNCTION nameIR `(` parameterIR* `)` `:` $\text{typeIR}_{\text{ret}}$ of type definedFunctionTypeIR be callableTypeIR: goto definedFunctionTypeIR
3. Else if let EXTERN_FUNCTION nameIR `(` parameterIR* `)` `:` $\text{typeIR}_{\text{ret}}$ of type externFunctionTypeIR be callableTypeIR: goto externFunctionTypeIR
4. Else if let builtinMethodTypeIR of type builtinMethodTypeIR be callableTypeIR: goto builtinMethodTypeIR
5. Else if let externMethodTypeIR of type externMethodTypeIR be callableTypeIR: goto externMethodTypeIR
6. Else if let PARSER_APPLY `(` parameterIR* `)` of type parserApplyMethodTypeIR be callableTypeIR: goto parserApplyMethodTypeIR
7. Else if let CONTROL_APPLY `(` parameterIR* `)` of type controlApplyMethodTypeIR be callableTypeIR: goto controlApplyMethodTypeIR
8. Else:
    a. Let TABLE_APPLY `:` tableMetadataStructTypeIR be callableTypeIR.
    b. Goto tableApplyMethodTypeIR

The runtime representation of a callable is given by:

```
callableDef
    : actionDef | functionDef | methodDef
    ;
```

9.1. Parameters

```
direction
    : /* empty */ | IN | OUT | INOUT
    ;

parameter
    : annotationList direction type name initializerOpt
    ;
```

Callables can have parameters. Each parameter has a direction, a type, a name, and an optional default value.

A parameter is internally represented as:

```
parameterIR
    : annotationList direction typeIR id parameterInitializerOptIR
    ;
```

The name of a parameter, and whether it is a default or an optional parameter, are obtained with:

The identifier of _ _ _ id _

1. Return id.

_ _ _ _ parameterInitializerOptIR is a parameter with a default value

1. Let parameterInitializerIR? be parameterInitializerOptIR.
2. If parameterInitializerIR? is none: return false.
3. Otherwise: return true.

parameterIR is an optional parameter

1. Return parameterIR is optional.

The high-level overview of parameters with respect to the copy in/copy out calling convention is described in Section 6.5. Below are the formal typing and well-formedness rules for parameters that implement the restrictions described there.

9.1.1. Type checking

Parameter_ok:

- Typing parameter, under context TC at p:
- The result is parameter parameterIR, and fresh type variables $typeId^*$.

Typing a parameter produces a parameterIR if it is well-typed. A parameter is well-typed when its type is well-formed and its default value (if any) has the same type as the parameter type.

Typing annotationList direction type name initializerOpt, under context TC at p:

1. If initializerOpt matches pattern $_{\text{EMPTY}}$:
    a. Let typeIR and fresh type variables $typeId_{fresh}^*$ be the result of typing type, under context TC at p.
    b. Let bound be bound type variables from the p layer of TC.
    c. Let B be the union of the sets bound and `{ $typeId_{fresh}^*$ `}.
    d. If typeIR is a well-formed type, with bound type variables B:
        i. Let nameIR be the name of name.
        ii. Let parameterIR be annotationList direction typeIR nameIR ·.
        iii. The result is parameter parameterIR, and fresh type variables $typeId_{fresh}^*$.
2. Else:
    a. Let '=' $expression_{init}$ be initializerOpt.
    b. Let typeIR and fresh type variables $typeId_{fresh}^*$ be the result of typing type, under context TC at p.
    c. Let bound' be bound type variables from the p layer of TC.
    d. Let B be the union of the sets bound' and `{ $typeId_{fresh}^*$ `}.
    e. If typeIR is a well-formed type, with bound type variables B:
        i. Let $typedExpressionIR_{init}$ be the result of typing $expression_{init}$, under context TC at p.
        ii. Let $typeIR_{init}$ be the type of $typedExpressionIR_{init}$.
        iii. If the compile-time known-ness of $typedExpressionIR_{init}$ matches pattern CTK:
            A. Let $typedExpressionIR^?$ be $typedExpressionIR_{init}$ implicitly cast to typeIR.
            B. Check that $typedExpressionIR^?$ is defined.
            C. Let $typedExpressionIR_{init_cast}$ be $typedExpressionIR^?$.
            D. Let nameIR be the name of name.
            E. Let parameterInitializerIR be '=' $typedExpressionIR_{init_cast}$.
            F. Let parameterIR be annotationList direction typeIR nameIR parameterInitializerIR.
            G. The result is parameter parameterIR, and fresh type variables $typeId_{fresh}^*$.
        iv. Else if the compile-time known-ness of $typedExpressionIR_{init}$ matches pattern LCTK:
            A. Let $typedExpressionIR^?$ be $typedExpressionIR_{init}$ implicitly cast to typeIR.
            B.

Check that typedExpressionIR? is defined.

C. Let $typedExpressionIR_{init_cast}$ be typedExpressionIR?.

D. Let nameIR be the name of name.

E. Let $value_{init}$ be the result of local compile-time evaluation of $typedExpressionIR_{init_cast}$, under context TC at p.

F. Let parameterInitializerIR be '=' VALUE $value_{init}$.

G. Let parameterIR be annotationList direction typeIR nameIR parameterInitializerIR.

H. The result is parameter parameterIR, and fresh type variables $typeId_{fresh}$*.

A list of parameters is type checked as follows:

ParameterList_ok:

- Typing parameterList, under context TC at p:
- The result is context TC, parameterListIR, and fresh type variables typeId*.

Typing parameterList, under context TC_0 at p:

1. Let parameter* be parameterList flattened.
2. Let parameters parameterIR*, and fresh type variables $typeId_{fresh}$* be the result of typing parameter*, under context TC_0 at p.
3. Let TC_1 be $add_parameters_t(p, TC_0, parameterIR*).
4. The result is context TC_1, parameterIR*, and fresh type variables $typeId_{fresh}$*.

Parameters_ok:

- Typing parameter*, under context TC at p:
- The result is parameters parameterListIR, and fresh type variables typeId*.

Typing parameter*, under context TC at p:

1. If parameter* matches pattern []:
   a. The result is parameters ·, and fresh type variables ·.
2. Else:
   a. Let $parameter_h$ :: $parameter_t$* be parameter*.

b. Let parameter parameterIR_h, and fresh type variables $\text{typeId}_{\text{fresh_h}}^*$ be the result of typing parameter_h, under context TC at p.

c. Let parameters parameterIR_t^*, and fresh type variables $\text{typeId}_{\text{fresh_t}}^*$ be the result of typing parameter_t^*, under context TC at p.

d. Let parameterIR^* be $\text{parameterIR}_h :: \text{parameterIR}_t^*$.

e. Let $\text{typeId}_{\text{fresh}}^*$ be $\text{typeId}_{\text{fresh_h}}^*$ concatenated with $\text{typeId}_{\text{fresh_t}}^*$.

f. The result is parameters parameterIR^*, and fresh type variables $\text{typeId}_{\text{fresh}}^*$.

9.1.2. Well-formedness

ParameterType_wf:

- annotationList direction typeIR id $\text{parameterInitializerIR}^?$ is a well-formed parameter, with bound type variables B

A parameter type is well-formed when its type is well-formed. Also, the type should not be a synthesized type. Types that can only be local compile-time known or compile-time known (e.g., int type) must be directionless. A default value can be specified for in or directionless parameters only.

annotationList direction typeIR id $\text{parameterInitializerIR}^?$ is a well-formed parameter, with bound type variables B:

1. If $\text{parameterInitializerIR}^?$ matches pattern ():
   a. If typeIR is a well-formed type, with bound type variables B:
      i. Let $\text{typeIR}_{\text{unroll}}$ be typeIR with typedefs unrolled.
      ii. Check that $\text{typeIR}_{\text{unroll}}$ does not have type synthesizedTypeIR.
      iii. Check that if $\text{typeIR}_{\text{unroll}}$ is equal to INT or $\text{typeIR}_{\text{unroll}}$ has type stringTypeIR or $\text{typeIR}_{\text{unroll}}$ has type objectTypeIR, then direction is equal to $_{\text{EMPTY}}$.
      iv. Then, the relation holds.
2. Else:
   a. If typeIR is a well-formed type, with bound type variables B:
      i. Let $\text{typeIR}_{\text{unroll}}$ be typeIR with typedefs unrolled.
      ii. Check that $\text{typeIR}_{\text{unroll}}$ does not have type synthesizedTypeIR.
      iii. Check that if $\text{typeIR}_{\text{unroll}}$ is equal to INT or $\text{typeIR}_{\text{unroll}}$ has type stringTypeIR or $\text{typeIR}_{\text{unroll}}$ has type objectTypeIR, then direction is equal to $_{\text{EMPTY}}$.
      iv. Check that direction is equal to IN or direction is equal to $_{\text{EMPTY}}$.
      v. Check that annotationList does not contain an optional annotation.
      vi. Then, the relation holds.

9.2. Callable definitions (callable constructors)

Similar to how type declarations introduce type definitions (see Section 8.3.1), callable declarations introduce callable definitions. These are constructors for callable types, that are parameterized by type

parameters (if any).

```
actionTypeDefIR = actionTypeIR

functionTypeDefIR
    : definedFunctionTypeDefIR | externFunctionTypeDefIR
    ;

methodTypeDefIR
    : externMethodTypeDefIR | parserApplyMethodTypeDefIR
    | controlApplyMethodTypeDefIR | tableApplyMethodTypeDefIR
    ;

callableTypeDefIR
    : actionTypeDefIR | functionTypeDefIR | methodTypeDefIR
    ;
```

When specialized with concrete type arguments, callable type definitions yield callable types:

```
actionTypeIR
    : annotationList ACTION nameIR ( parameterIR* )
    ;

functionTypeIR
    : definedFunctionTypeIR | externFunctionTypeIR
    ;

methodTypeIR
    : builtinMethodTypeIR | externMethodTypeIR | parserApplyMethodTypeIR
    | controlApplyMethodTypeIR | tableApplyMethodTypeIR
    ;

callableTypeIR
    : actionTypeIR | functionTypeIR | methodTypeIR
    ;
```

Specialization inserts fresh type variables for any omitted implicit type arguments, yielding the callable type together with those fresh variables:

callableTypeDefIR$_{base}$ specialized by typeArgumentIR*

1. Let (typeParameterIR$_{expl}{}^{*}$, typeParameterIR$_{impl}{}^{*}$) be the type parameters of callableTypeDefIR$_{base}$.
2. Let typeParameterIR* be typeParameterIR$_{expl}{}^{*}$ concatenated with typeParameterIR$_{impl}{}^{*}$.
3. Try:
    a. If the length of typeParameterIR$_{expl}{}^{*}$ is equal to the length of typeArgumentIR*:
        i. Let typeId$_{fresh}{}^{*}$ be the length of typeParameterIR$_{impl}{}^{*}$ fresh type variables.
        ii. For each typeId$_{fresh}$ in typeId$_{fresh}{}^{*}$:
            - Let typeArgumentIR$_{impl}$ be $_{NAME}$ typeId$_{fresh}$.

        Let typeArgumentIR$_{impl}{}^{*}$ be the resulting list.

iii. If the length of $typeArgumentIR_{impl}^{*}$ is equal to the length of $typeParameterIR_{impl}^{*}$:

A. Let theta be `{ ( $typeParameterIR_{expl}$ ':' typeArgumentIR )* ++ ( $typeParameterIR_{impl}$ ':' $typeArgumentIR_{impl}$ )* `}.

B. Let $callableTypeIR_{base}$ be the underlying type of $callableTypeDefIR_{base}$.

C. Let $callableTypeIR_{spec}$ be $callableTypeIR_{base}$ substituted by theta.

D. Return ($callableTypeIR_{spec}$, $typeId_{fresh}^{*}$).

4. Then, try:

a. Check that the length of $typeParameterIR_{expl}^{*}$ is greater than 0 and the length of $typeArgumentIR^{*}$ is equal to 0.

b. Let $typeId_{fresh}^{*}$ be the length of $typeParameterIR^{*}$ fresh type variables.

c. For each $typeId_{fresh}$ in $typeId_{fresh}^{*}$:

- Let $typeArgumentIR_{fresh}$ be $_{NAME}$ $typeId_{fresh}$.

Let $typeArgumentIR_{fresh}^{*}$ be the resulting list.

d. Check that the length of $typeArgumentIR_{fresh}^{*}$ is equal to the length of $typeParameterIR^{*}$.

e. Let theta be `{ ( typeParameterIR ':' $typeArgumentIR_{fresh}$ )* `}.

f. Let $callableTypeIR_{base}$ be the underlying type of $callableTypeDefIR_{base}$.

g. Let $callableTypeIR_{spec}$ be $callableTypeIR_{base}$ substituted by theta.

h. Return ($callableTypeIR_{spec}$, $typeId_{fresh}^{*}$).

For example, a function declaration introduces a function type definition:

```
T identity<T>(in T t) { return t; }
```

identity is a generic function type definition that falls into:

```
definedFunctionTypeDefIR
    : FUNCTION nameIR < typeParameterIR* , typeParameterIR* > ( parameterIR* ) :
        typeIR
    ;
```

Notice that definedFunctionTypeDefIR does *not* include the function body. Because callable type definitions are used during type checking, the body of the callable is not part of the type definition. After specialization with proper type argument(s), it yields a function type:

```
definedFunctionTypeIR
    : FUNCTION nameIR ( parameterIR* ) : typeIR
    ;
```

9.2.1. Well-formedness

CallableTypeDef_wf:

- callableTypeDefIR is a well-formed callable type definition, with bound type variables B

callableTypeDefIR is a well-formed callable type definition, with bound type variables B:

1. Let ($typeParameterIR_{expl}^{*}$, $typeParameterIR_{impl}^{*}$) be the type parameters of callableTypeDefIR.
2. Let $typeParameterIR^{*}$ be $typeParameterIR_{expl}^{*}$ concatenated with $typeParameterIR_{impl}^{*}$.
3. Check that the elements of $typeParameterIR^{*}$ are distinct.
4. Let $callableTypeIR_{base}$ be the underlying type of callableTypeDefIR.
5. Let B_{inner} be the union of the sets B and `{ $typeParameterIR^{*}$ `}.
6. If $callableTypeIR_{base}$ is a well-formed callable type, with bound type variables B_{inner}: then, the relation holds.

9.2.2. Well-formedness of return types

ReturnType_wf:

- $typeIR_{ret}$ is a well-formed return type, with bound type variables B

$typeIR_{ret}$ is a well-formed return type, with bound type variables B:

1. If $typeIR_{ret}$ is a well-formed type, with bound type variables B:
   a. Let $typeIR_{unroll}$ be $typeIR_{ret}$ with typedefs unrolled.
   b. Check that $_{typeIR}unroll\sim$ = INT and $typeIR_{unroll}$ does not have type synthesizedTypeIR and $typeIR_{unroll}$ does not have type objectTypeIR.
   c. Then, the relation holds.

9.3. Actions

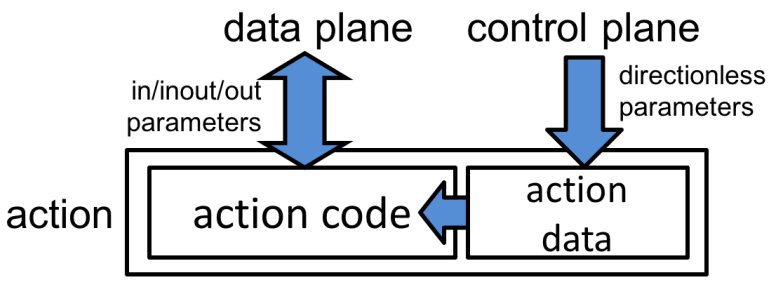

Figure 10. Actions contain code and data. The code is in the P4 program, while the data is provided in the table entries, typically populated by the control plane. Other parameters are bound by the data plane.

Actions are code fragments that can read and write the data being processed. Actions may contain data values that can be written by the control plane and read by the data plane. Actions are the main construct by which the control plane can dynamically influence the behavior of the data plane. Figure 10 shows the abstract model of an action.

```
actionDeclaration
    : annotationList ACTION name ( parameterList ) blockStatement
    ;
```

Syntactically actions resemble functions with no return value. Actions may be declared within a control block; in this case they can only be used within instances of that control block.

The following example shows an action declaration:

```
action Forward_a(out bit<9> outputPort, bit<9> port) {
    outputPort = port;
}
```

Action parameters may not have extern types. Action parameters that have no direction (e.g., port in the previous example) indicate "action data." All such parameters must appear at the end of the parameter list. When used in a match-action table (see [sec-table-action-list]), these parameters will be provided by the table entries (e.g., as specified by the control plane, the default_action table property, or the entries table property).

The body of an action consists of a sequence of statements and declarations. No table, control, or parser applications can appear within actions.

Some targets may impose additional restrictions on action bodies—e.g., only allowing straight-line code, with no conditional statements or expressions.

Actions can be executed in two ways:

- Implicitly: by tables during match-action processing.
- Explicitly: either from a control block or from another action. In either case, the values for all action parameters must be supplied explicitly, including values for the directionless parameters. In this case, the directionless parameters behave like in parameters.

Actions are introduced by action declarations. Its typing and instantiation rules are explained in Section 11.6. It introduces an action type definition:

```
actionTypeDefIR = actionTypeIR
```

Because actions cannot be generic, the action type is the same as the action type definition:

```
actionTypeIR
    : annotationList ACTION nameIR ( parameterIR* )
    ;
```

The runtime representation of an action is as follows:

```
actionDef
    : annotationList ACTION nameIR ( parameterListIR ) blockStatementIR
    ;
```

9.3.1. Well-formedness

annotationList ACTION nameIR `( parameterIR* `) is a well-formed callable type, with bound type variables B:

1. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*:
    a. Check that parameterIR is not an optional parameter, for all parameterIR in parameterIR*.
    b. For each parameterIR in parameterIR*:

    - Let _ direction typeIR _ _ be parameterIR.

    Let direction* and typeIR* be the resulting lists.

    c. Check that directionless parameters in direction* only appear at the end.
    d. Check that typeIR can be used in an action type, for all typeIR in typeIR*.
    e. Then, the relation holds.

Directionless parameters must form a trailing suffix of the parameter list, which is checked with:

Directionless parameters in direction* only appear at the end

1. Let direction'* be the reversal of direction*.
2. Return $directionless_trailing'(true, direction'*).

$directionless_trailing'(b, direction'*)

1. Try:
    a. If direction'* matches pattern []: return true.
    b. Else:
        i. Let $direction_h :: direction_t^*$ be direction'*.
        ii. If $direction_h$ does not match pattern $_{EMPTY}$:
            A. Return $directionless_trailing'(false, $direction_t^*$).
2. Then, try:
    a. If b is equal to true:
        i. Check that direction'* is a non-empty list.
        ii. Let $direction :: direction_t^*$ be direction'*.
        iii. Check that direction is $_{EMPTY}$.
        iv. Return $directionless_trailing'(true, $direction_t^*$).
    b. Else:
        i. Check that direction'* is a non-empty list.
        ii. Let $direction :: direction_t^*$ be direction'*.

iii. Check that direction is EMPTY.

iv. Return false.

9.4. Functions

Two kinds of functions are supported in P4: user-defined functions and extern functions. The former are functions whose implementation is provided in P4 source code, while the latter are functions whose implementation is provided externally by the target architecture.

The internal representation of functions is as follows:

```
functionTypeDefIR
    : definedFunctionTypeDefIR | externFunctionTypeDefIR
    ;

functionTypeIR
    : definedFunctionTypeIR | externFunctionTypeIR
    ;

functionDef
    : definedFunctionDef | externFunctionDef
    ;
```

9.4.1. User-defined functions

Functions can only be declared at the top level and all parameters must have a direction. P4 functions are modeled after functions as found in most other programming languages, but the language does not permit recursive functions.

```
functionPrototype
    : typeOrVoid name typeParameterListOpt ( parameterList )
    ;

functionDeclaration
    : annotationList functionPrototype blockStatement
    ;
```

Here is an example of a function that returns the maximum of two 32-bit values:

```
bit<32> max(in bit<32> left, in bit<32> right) {
   return (left > right) ? left : right;
}
```

A function returns a value using the return statement. A function with a return type of void can simply use the return statement with no arguments. A function with a non-void return type must return a value of the suitable type on all possible execution paths.

The typing and instantiation rules for function declarations is explained in Section 11.5. This introduces a function type definition, which can be specialized to a function type.

```
definedFunctionTypeDefIR
    : FUNCTION nameIR < typeParameterIR* , typeParameterIR* > ( parameterIR* ) :
        typeIR
    ;

definedFunctionTypeIR
    : FUNCTION nameIR ( parameterIR* ) : typeIR
    ;
```

Also, the runtime representation of a defined function is as follows:

```
definedFunctionDef
    : FUNCTION nameIR < typeParameterListIR > ( parameterListIR )
        blockStatementIR
    ;
```

9.4.1.1. Well-formedness

FUNCTION nameIR `( parameterIR* `) ':' $\text{typeIR}_{\text{ret}}$ is a well-formed callable type, with bound type variables B:

1. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*:
   a. Check that parameterIR is not an optional parameter, for all parameterIR in parameterIR*.
   b. For each parameterIR in parameterIR*:
      - Let _ _ typeIR _ _ be parameterIR.

      Let typeIR* be the resulting list.
   c. Check that typeIR can be used in a defined function type, for all typeIR in typeIR*.
   d. If $\text{typeIR}_{\text{ret}}$ is a well-formed return type, with bound type variables B: then, the relation holds.

9.4.2. Extern functions

An extern function declaration describes the name and type signature of the function, but not its implementation.

```
externDeclaration
    : externFunctionDeclaration | externObjectDeclaration
    ;
```

extern declarations introduce extern function type definitions, which can be specialized to extern function types.

```
externFunctionTypeDefIR
    : EXTERN_FUNCTION nameIR < typeParameterIR* , typeParameterIR* >
```

```
        ( parameterIR* ) : typeIR
    ;

externFunctionTypeIR
    : EXTERN_FUNCTION nameIR ( parameterIR* ) : typeIR
    ;
```

Also, the runtime representation of an extern function is as follows:

```
externFunctionDef
    : EXTERN_FUNCTION nameIR < typeParameterListIR > ( parameterListIR )
    ;
```

See Section 11.9 for the typing and instantiation rules for extern function declarations.

9.4.2.1. Well-formedness

EXTERN_FUNCTION nameIR `( parameterIR* `) ':' typeIR$_{ret}$ is a well-formed callable type, with bound type variables B:

1. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*:
   a. For each parameterIR in parameterIR*:
      - Let _ _ typeIR _ _ be parameterIR.

      Let typeIR* be the resulting list.
   b. Check that typeIR can be used in an extern function type, for all typeIR in typeIR*.
   c. If typeIR$_{ret}$ is a well-formed return type, with bound type variables B: then, the relation holds.

9.5. Methods

Objects in P4 can have methods associated with them. Specifically, extern objects have methods declared in their definitions. parser, control, and table objects have built-in apply methods.

Further, P4 allows built-in methods for certain types, such as isValid() and setValid() methods for header types.

The internal representation of methods is defined as follows:

```
methodTypeDefIR
    : externMethodTypeDefIR | parserApplyMethodTypeDefIR
    | controlApplyMethodTypeDefIR | tableApplyMethodTypeDefIR
    ;

methodTypeIR
    : builtinMethodTypeIR | externMethodTypeIR | parserApplyMethodTypeIR
    | controlApplyMethodTypeIR | tableApplyMethodTypeIR
    ;

methodDef
```

```
    : externMethodDef | parserApplyMethodDef | controlApplyMethodDef
    | tableApplyMethodDef
    ;
```

9.5.1. Extern methods

An extern object declaration declares an object and all methods that can be invoked to perform computations and to alter the state of the object.

```
externConstructorPrototype
    : annotationList typeIdentifier ( parameterList ) ;
    ;

externMethodPrototype
    : annotationList functionPrototype ;
    | annotationList ABSTRACT functionPrototype ;
    ;

externConstructorOrMethodPrototype
    : externConstructorPrototype | externMethodPrototype
    ;

externObjectDeclaration
    : annotationList EXTERN nonTypeName typeParameterListOpt
        { externConstructorOrMethodPrototypeList }
    ;
```

For example, the P4 core library introduces two extern objects packet_in and packet_out used for manipulating packets (see [sec-packet-data-extraction] and [sec-deparse]). Here is an example showing how the methods of these objects can be invoked on a packet:

```
extern packet_out {
    void emit<T>(in T hdr);
}
control d(packet_out b, in Hdr h) {
    apply {
        b.emit(h.ipv4);       // write ipv4 header into output packet
    }                         // by calling emit method
}
```

Internally, extern methods are represented as follows:

```
externMethodTypeDefIR
    : EXTERN_METHOD nameIR < typeParameterIR* , typeParameterIR* >
        ( parameterIR* ) : typeIR
    | EXTERN_METHOD ABSTRACT nameIR < typeParameterIR* , typeParameterIR* >
        ( parameterIR* ) : typeIR
    ;

externMethodTypeIR
    : EXTERN_METHOD nameIR ( parameterIR* ) : typeIR
    | EXTERN_METHOD ABSTRACT nameIR ( parameterIR* ) : typeIR
    ;

externMethodDef
    : EXTERN_METHOD nameIR < typeParameterListIR > ( parameterListIR )
        blockStatementIR?
```

```
    | EXTERN_METHOD ABSTRACT nameIR < typeParameterListIR > ( parameterListIR )
    ;
```

Typical extern object methods are implemented by the target architecture. P4 programmers can only call such methods.

Some types of extern objects may provide methods that can be implemented by the P4 programmers. Such methods are described with the abstract keyword prior to the method definition. Here is an example:

```
extern Balancer {
    Balancer();
    // get the number of active flows
    bit<32> getFlowCount();
    // return port index used for load-balancing
    // @param address: IPv4 source address of flow
    abstract bit<4> on_new_flow(in bit<32> address);
}
```

When such an object is instantiated the user has to supply an implementation of all the abstract methods (see [sec-instantion-abstract-method]).

9.5.1.1. Well-formedness

externMethodTypeIR is a well-formed callable type, with bound type variables B:

1. If let EXTERN_METHOD _ `( parameterIR* `) ':' typeIR$_{ret}$ be externMethodTypeIR:
   a. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*:
      i. For each parameterIR in parameterIR*:
         - Let _ _ typeIR _ _ be parameterIR.

         Let typeIR* be the resulting list.
      ii. Check that typeIR can be used in an extern method type, for all typeIR in typeIR*.
      iii. If typeIR$_{ret}$ is a well-formed return type, with bound type variables B: then, the relation holds.
2. Else:
   a. Let EXTERN_METHOD ABSTRACT _ `( parameterIR* `) ':' typeIR$_{ret}$ be externMethodTypeIR.
   b. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*:
      i. Check that parameterIR is not an optional parameter, for all parameterIR in parameterIR*.
      ii. For each parameterIR in parameterIR*:
         - Let _ _ typeIR _ _ be parameterIR.

         Let typeIR* be the resulting list.
      iii. Check that typeIR can be used in an extern method type, for all typeIR in typeIR*.
      iv. If typeIR$_{ret}$ is a well-formed return type, with bound type variables B: then, the relation holds.

9.5.2. Parser apply methods

P4 allows parsers to invoke the services of other parsers, similar to subroutines. To invoke the services of another parser, the sub-parser must be first instantiated; the services of an instance are invoked by calling it using its apply method.

The following example shows a sub-parser invocation:

```
parser callee(packet_in packet, out IPv4 ipv4) { /* body omitted */ }
parser caller(packet_in packet, out Headers h) {
     callee() subparser;  // instance of callee
     state subroutine {
          subparser.apply(packet, h.ipv4);  // invoke sub-parser
          transition accept;  // accept if sub-parser ends in accept state
     }
}
```

A parser apply method type definition is implicitly introduced by a parser declaration. Its semantics is explained in Section 11.11. A parser apply method is represented internally as follows:

```
parserApplyMethodTypeDefIR = parserApplyMethodTypeIR

parserApplyMethodTypeIR
    : PARSER_APPLY ( parameterIR* )
    ;

parserApplyMethodDef
    : PARSER_APPLY ( parameterListIR )
        { parserLocalDeclarationListIR ; stateEnv }
    ;
```

9.5.2.1. Well-formedness

PARSER_APPLY `( parameterIR* `) is a well-formed callable type, with bound type variables B:

1. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*:
   a. Check that parameterIR is not an optional parameter, for all parameterIR in parameterIR*.
   b. For each parameterIR in parameterIR*:
      - Let _ _ typeIR _ _ be parameterIR.

      Let typeIR* be the resulting list.
   c. Check that typeIR can be used in a parser apply method type, for all typeIR in typeIR*.
   d. Then, the relation holds.

9.5.3. Control apply methods

P4 allows controls to invoke the services of other controls, similar to subroutines. To invoke the services of another control, it must be first instantiated; the services of an instance are invoked by calling it using

its apply method.

The following example shows a control invocation:

```
control Callee(inout IPv4 ipv4) { /* body omitted */ }
control Caller(inout Headers h) {
     Callee() instance;  // instance of callee
     apply {
          instance.apply(h.ipv4);  // invoke control
     }
}
```

A control apply method type definition is implicitly introduced by a control declaration. Its semantics is explained in Section 11.12. A control apply method is represented internally as follows:

```
controlApplyMethodTypeDefIR = controlApplyMethodTypeIR

controlApplyMethodTypeIR
    : CONTROL_APPLY ( parameterIR* )
    ;

controlApplyMethodDef
    : CONTROL_APPLY ( parameterListIR )
        { controlLocalDeclarationListIR ; actionDefEnv ; controlBodyIR }
    ;
```

9.5.3.1. Well-formedness

CONTROL_APPLY `( parameterIR* `) is a well-formed callable type, with bound type variables B:

1. If parameterIR is a well-formed parameter, with bound type variables B, for all parameterIR in parameterIR*:
   a. Check that parameterIR is not an optional parameter, for all parameterIR in parameterIR*.
   b. For each parameterIR in parameterIR*:
      - Let _ _ typeIR _ _ be parameterIR.

      Let typeIR* be the resulting list.
   c. Check that typeIR can be used in a control apply method type, for all typeIR in typeIR*.
   d. Then, the relation holds.

9.5.4. Table apply methods

A table can be invoked by calling its apply method. Calling an apply method on a table instance returns a synthesized table metadata structure, explained in Section 8.6.6.

A table apply method definition is introduced by a table declaration. Its semantics rule is explained in [sec-table-declaration]. A table declaration may not be generic. Thus, the table apply method type definition does not have type parameters, and is same as the table apply method type. A table apply method is represented internally as follows:

```
tableApplyMethodTypeDefIR = tableApplyMethodTypeIR

tableApplyMethodTypeIR
    : TABLE_APPLY : tableMetadataStructTypeIR
    ;

tableApplyMethodDef
    : TABLE_APPLY { tablePropertyListIR }
    ;
```

9.5.4.1. Well-formedness

TABLE_APPLY ':' tableMetadataStructTypeIR is a well-formed callable type, with bound type variables B: then, the relation holds.

9.5.5. Built-in methods

Built-in methods are methods associated with certain types defined by the P4 language. Thus, they do not have method type definitions, and only have method types. The internal representations are:

```
builtinMethodTypeIR
    : builtinIsValidMethodTypeIR | builtinSetValidMethodTypeIR
    | builtinSetInvalidMethodTypeIR | builtinPushFrontMethodTypeIR
    | builtinPopFrontMethodTypeIR | builtinSizeMethodTypeIR
    ;
```

9.5.5.1. Well-formedness

builtinMethodTypeIR is a well-formed callable type, with bound type variables B: then, the relation holds.

The signature of a built-in method, which lists its parameters and return type, is extracted from the method type with:

$signature_of_builtinMethodTypeIR(builtinMethodTypeIR)

1. If let builtinIsValidMethodTypeIR of type builtinIsValidMethodTypeIR be builtinMethodTypeIR: return (·, BOOL).
2. Else if let builtinSetValidMethodTypeIR of type builtinSetValidMethodTypeIR be builtinMethodTypeIR: return (·, VOID).
3. Else if let builtinSetInvalidMethodTypeIR of type builtinSetInvalidMethodTypeIR be builtinMethodTypeIR: return (·, VOID).
4. Else if let builtinPushFrontMethodTypeIR of type builtinPushFrontMethodTypeIR be builtinMethodTypeIR:
    a. Return (EMPTY EMPTY INT "count" ·, VOID).

5. Else if let builtinPopFrontMethodTypeIR of type builtinPopFrontMethodTypeIR be builtinMethodTypeIR:
    a. Return (EMPTY EMPTY INT "count" ·, VOID).
6. Else:
    a. Let builtinSizeMethodTypeIR be builtinMethodTypeIR.
    b. Return (·, INT).

10. P4 objects and constructors

10.1. Objects

tables, value_sets, and extern objects are stateful entities in P4 programs. These objects should be instantiated statically prior to program execution. parsers, controls, and packages may contain stateful objects, thus they must also be instantiated.

The runtime representation of these objects is defined as follows:

```
object
    : externObject | parserObject | controlObject | packageObject | tableObject
    | valueSetObject
    ;
```

The instantiation phase (described in Section 7.3) creates objects in the source P4 program, and allocates them in the global store. At runtime (described in Section 7.4), these objects interact, as defined by the target architecture, to process packets.

10.1.1. Fully-qualified names

Instantiation may create multiple instances of a type, each of which must have a unique, fully-qualified name. Thus, objects are identified by their full-qualified names (objectId). Section 7.3.1 describes how a fully-qualified name is constructed.

```
objectId = nameIR*
```

10.2. Constructors

Objects can be instantiated in three ways:

- Using instantiations, described in Section 11.4.
- Using constructor call expressions, described in [sec-constructor-call-expression].
- Using direct type invocation statements, described in Section 13.5.

Instantiations and constructor call expressions invoke constructors, which creates an object as a result. Direct application statements are syntactic sugar for instantiations of parser and control objects.

Constructor invocation is similar to callable invocation; constructors can also be overloaded, and called using named arguments.

The following example shows a constructor invocation for setting the target-dependent implementation property of a table:

```
extern ActionProfile {
    ActionProfile(bit<32> size);  // constructor
}
table tbl {
    actions = { /* body omitted */ }
    implementation = ActionProfile(1024);  // constructor invocation
}
```

10.2.1. Constructor parameters

In order to support libraries of useful P4 components, externs, parsers, controls, and packages can additionally be parameterized through the use of constructor parameters.

Consider the parser declaration syntax:

```
parserDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { parserLocalDeclarationList parserStateList }
    ;

constructorParameterListOpt
    : /* empty */ | ( parameterList )
    ;
```

From this grammar fragment we infer that a parser declaration may have two sets of parameters:

- The runtime parser parameters (parameterList)
- Optional parser constructor parameters (constructorParameterListOpt)

Because constructors are evaluated entirely at compilation time, all constructor arguments must be expressions that can be evaluated at compilation time. In consequence, constructor parameters must be directionless (i.e., they cannot be in, out, or inout).

Consider the following example:

```
parser GenericParser(packet_in b, out Packet_header p)
                    (bool udpSupport) {   // constructor parameters
    state start {
        b.extract(p.ethernet);
        transition select(p.ethernet.etherType) {
            16w0x0800: ipv4;
        }
    }
    state ipv4 {
        b.extract(p.ipv4);
        transition select(p.ipv4.protocol) {
           6: tcp;
           17: tryudp;
        }
    }
    state tryudp {
        transition select(udpSupport) {
            false: accept;
            true : udp;
        }
    }
```

```
    state udp {
        // body omitted
    }
}
```

When instantiating the GenericParser it is necessary to supply a value for the udpSupport parameter, as in the following example:

```
// topParser is a GenericParser where udpSupport = false
GenericParser(false) topParser;
```

Internally, constructor parameters are represented as:

```
constructorParameterIR = parameterIR

parameterIR
    : annotationList direction typeIR id parameterInitializerOptIR
    ;
```

10.2.1.1. Type checking

ConstructorParameterListOpt_ok:

- Typing constructorParameterListOpt under context TC:
- The result is context TC, constructorParameterListIR, and fresh type variables typeId*.

Typing constructorParameterListOpt under context TC_0:

1. Let constructorParameter* be constructorParameterListOpt flattened.
2. Let parameters constructorParameterIR*, and fresh type variables $typeId_{fresh}$* be the result of typing constructorParameter*, under context TC_0 at BLOCK.
3. Let TC_1 be $add_parameters_t(BLOCK, TC_0, constructorParameterIR*).
4. The result is context TC_1, constructorParameterIR*, and fresh type variables $typeId_{fresh}$*.

10.2.1.2. Well-formedness

ConstructorParameterType_wf:

- constructorParameterIR is a well-formed constructor parameter, with bound type variables B

constructorParameterIR is a well-formed constructor parameter, with bound type variables B:

1. Let direction be the direction of constructorParameterIR.
2. Check that direction is EMPTY.
3. If constructorParameterIR is a well-formed parameter, with bound type variables B: then, the relation holds.

10.2.2. Constructor definitions

Similar to how type declarations introduce type definitions and callable declarations introduce callable definitions (see Section 9.2), constructor declarations introduce constructor definitions. These are parameterized by type parameters, and produces a constructor type when specialized with specific type arguments.

The runtime representation of constructors is defined as follows:

```
constructorDef
    : externObjectConstructorDef | parserObjectConstructorDef
    | controlObjectConstructorDef | packageObjectConstructorDef
    ;
```

When type arguments and constructor arguments are supplied to a constructor definition, an object of the constructed type is created.

Similar to how callable types are internally used during type checking to represent callables, constructor types internally represent the types of constructors.

```
constructorTypeIR
    : CONSTRUCTOR ( constructorParameterIR* ) : typeIR
    ;

constructorTypeDefIR
    : CONSTRUCTOR < typeParameterIR* , typeParameterIR* >
        ( constructorParameterIR* ) : typeIR
    ;
```

Specialization inserts fresh type variables for any omitted implicit type arguments, yielding the constructor type together with those fresh variables:

CONSTRUCTOR `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `( constructorParameterIR* `) ':' typeIR specialized by typeArgumentIR*

1. Let typeParameterIR* be typeParameterIR$_{expl}$* concatenated with typeParameterIR$_{impl}$*.
2. Try:
    a. If the length of typeParameterIR$_{expl}$* is equal to the length of typeArgumentIR*:
        i. Let typeId$_{fresh}$* be the length of typeParameterIR$_{impl}$* fresh type variables.
        ii. For each typeId$_{fresh}$ in typeId$_{fresh}$*:
            - Let typeArgumentIR$_{impl}$ be NAME typeId$_{fresh}$.

Let $typeArgumentIR_{impl}^{*}$ be the resulting list.

iii. If the length of $typeArgumentIR_{impl}^{*}$ is equal to the length of $typeParameterIR_{impl}^{*}$:

A. Let theta be `{ ( $typeParameterIR_{expl}$ ':' typeArgumentIR )* ++ ( $typeParameterIR_{impl}$ ':' $typeArgumentIR_{impl}$ )* `}.

B. Let $constructorTypeIR_{spec}$ be CONSTRUCTOR `( constructorParameterIR* `) ':' typeIR substituted by theta.

C. Return ($constructorTypeIR_{spec}$, $typeId_{fresh}^{*}$).

3. Then, try:

a. Check that the length of $typeParameterIR_{expl}^{*}$ is greater than 0 and the length of $typeArgumentIR^{*}$ is equal to 0.

b. Let $typeId_{fresh}^{*}$ be the length of $typeParameterIR^{*}$ fresh type variables.

c. For each $typeId_{fresh}$ in $typeId_{fresh}^{*}$:

- Let $typeArgumentIR_{fresh}$ be NAME $typeId_{fresh}$.

Let $typeArgumentIR_{fresh}^{*}$ be the resulting list.

d. Check that the length of $typeArgumentIR_{fresh}^{*}$ is equal to the length of $typeParameterIR^{*}$.

e. Let theta be `{ ( typeParameterIR ':' $typeArgumentIR_{fresh}$ )* `}.

f. Let $constructorTypeIR_{spec}$ be CONSTRUCTOR `( constructorParameterIR* `) ':' typeIR substituted by theta.

g. Return ($constructorTypeIR_{spec}$, $typeId_{fresh}^{*}$).

10.2.2.1. Well-formedness

ConstructorTypeDef_wf:

- constructorTypeDefIR is a well-formed constructor type definition, with bound type variables B

constructorTypeDefIR is a well-formed constructor type definition, with bound type variables B:

1. Let CONSTRUCTOR `< $typeParameterIR_{expl}^{*}$ ',' $typeParameterIR_{impl}^{*}$ `> `( parameterIR* `) ':' $typeIR_{object}$ be constructorTypeDefIR.
2. Check that $typeIR_{object}$ can be constructed.
3. Let $typeParameterIR^{*}$ be $typeParameterIR_{expl}^{*}$ concatenated with $typeParameterIR_{impl}^{*}$.
4. Check that the elements of $typeParameterIR^{*}$ are distinct.
5. Let B_{inner} be the union of the sets B and `{ $typeParameterIR^{*}$ `}.
6. If CONSTRUCTOR `( $parameterIR^{*}$ `) ':' $typeIR_{object}$ is a well-formed constructor type, with bound type variables B_{inner}: then, the relation holds.

The well-formedness of constructor types is defined as follows:

ConstructorType_wf:

- CONSTRUCTOR `( parameterIR* `) ':' typeIR$_{object}$ is a well-formed constructor type, with bound type variables B

ConstructorType_wf dispatch:

1. Try:
    a. Goto externObjectTypeIR
2. Then, try:
    a. Goto parserObjectTypeIR
3. Then, try:
    a. Goto controlObjectTypeIR
4. Then, try:
    a. If parameterIR* is an empty list: goto tableObjectTypeIR
5. Then, try:
    a. Goto packageObjectTypeIR

10.3. Extern objects

An extern object declaration introduces an extern object type and its associated methods. Extern object declarations can also optionally declare constructor methods; these must have the same name as the enclosing extern type, no type parameters, and no return type.

```
externConstructorPrototype
    : annotationList typeIdentifier ( parameterList ) ;
    ;

externMethodPrototype
    : annotationList functionPrototype ;
    | annotationList ABSTRACT functionPrototype ;
    ;

externObjectDeclaration
    : annotationList EXTERN nonTypeName typeParameterListOpt
        { externConstructorOrMethodPrototypeList }
    ;
```

Internally, an extern object constructor is represented as:

```
externObjectConstructorDef
    : EXTERN nameIR < typeParameterListIR > ( constructorParameterListIR )
        { externMethodPrototypeListIR }
    ;

externMethodPrototypeIR
    : annotationList functionPrototypeIR ;
    | annotationList ABSTRACT functionPrototypeIR ;
    ;
```

Extern objects are then instantiated by constructor invocations to yield:

```
externObject
    : EXTERN typeId < theta > { objectState frame externMethodDefEnv }
    ;
```

The only operation on extern objects is invoking a method of an extern object instance using a method call expression.

Controls, parsers, packages, and extern constructors can have parameters of type extern only if they are directionless parameters. Extern object instances can thus be used as arguments in the construction of such objects.

No other operations are available on extern objects, e.g., equality comparison is not defined.

10.3.1. Well-formedness

CONSTRUCTOR `(` parameterIR* `)` `:` $typeIR_{object}$ is a well-formed constructor type, with bound type variables B:

1. If parameterIR is a well-formed constructor parameter, with bound type variables B, for all parameterIR in parameterIR*:
    a. If $typeIR_{object}$ is a well-formed type, with bound type variables B:
        i. Let typeIR' be $typeIR_{object}$ with typedefs unrolled.
        ii. If typeIR' has type externObjectTypeIR:
            A. For each parameterIR in parameterIR*:
                - Let _ _ typeIR _ _ be parameterIR.

            Let typeIR* be the resulting list.

            B. If typeIR can be used in an extern constructor type, for all typeIR in typeIR*: then, the relation holds.

10.4. Parser objects

A parser declaration comprises a name, a list of parameters, an optional list of constructor parameters, local elements, and parser states (as well as optional annotations).

```
parserDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { parserLocalDeclarationList parserStateList }
    ;

parserLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | valueSetDeclaration
    ;
```

```
parserState
    : annotationList STATE name
        { parserBlockElementStatementList transitionStatement }
    ;
```

At least one state, named start, must be present in any parser. A parser may not define two states with the same name. It is also illegal for a parser to give explicit definitions for the accept and reject states—those states are logically distinct from the states defined by the programmer.

Preceding the parser states, a parser may also contain a list of local elements. These can be constants, variables, or instantiations of objects that may be used within the parser. Such objects may be instantiations of extern objects, or other parsers that may be invoked as subroutines. However, it is illegal to instantiate a control block within a parser.

The states and local elements are all in the same namespace, thus the following example will produce an error:

```
// erroneous example
parser p() {
    bit<4> t;
    state start {
       t = 1;
       transition t;
    }
    state t {  // error: name t is duplicated
       transition accept;
    }
}
```

For an example containing a complete declaration of a parser see Section 5.3.

A parser declaration introduces a parser object constructor, represented as:

```
parserObjectConstructorDef
    : PARSER < typeParameterListIR > ( parameterListIR )
        ( constructorParameterListIR )
        { parserLocalDeclarationListIR parserStateListIR }
    ;
```

Parser objects are then instantiated by constructor invocations to yield:

```
parserObject
    : PARSER < theta > ( parameterListIR )
        { frame parserLocalDeclarationListIR stateEnv }
    ;
```

10.4.1. Well-formedness

CONSTRUCTOR `( parameterIR* `) ':' typeIR$_{object}$ is a well-formed constructor type, with bound type variables B:

1. If parameterIR is a well-formed constructor parameter, with bound type variables B, for all parameterIR in parameterIR*:
    a. If parameterIR is not an optional parameter, for all parameterIR in parameterIR*:
        i. If $typeIR_{object}$ is a well-formed type, with bound type variables B:
            A. Let typeIR' be $typeIR_{object}$ with typedefs unrolled.
            B. If typeIR' has type parserObjectTypeIR:
                I. For each parameterIR in parameterIR*:
                    - Let _ _ typeIR _ _ be parameterIR.

                Let typeIR* be the resulting list.

                II. If typeIR can be used in a parser constructor type, for all typeIR in typeIR*: then, the relation holds.

10.5. Control objects

P4 parsers are responsible for extracting bits from a packet into headers. These headers (and other metadata) can be manipulated and transformed within control blocks. The body of a control block resembles a traditional imperative program. Within the body of a control block, match-action units can be invoked to perform data transformations. Match-action units are represented in P4 by constructs called tables.

Syntactically, a control block is declared with a name, parameters, optional type parameters, and a sequence of declarations of constants, variables, actions, tables, and other instantiations:

```
controlDeclaration
    : annotationList CONTROL name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { controlLocalDeclarationList APPLY controlBody }
    ;

controlLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | actionDeclaration | tableDeclaration
    ;

controlBody = blockStatement
```

It is illegal to instantiate a parser within a control block.

P4 does not support exceptional control-flow within a control block. The only statement which has a non-local effect on control flow is exit, which causes execution of the enclosing control block to immediately terminate. That is, there is no equivalent of the verify statement or the reject state from parsers. Hence, all error handling must be performed explicitly by the programmer.

A control declaration introduces a control object constructor, represented as:

```
controlObjectConstructorDef
    : CONTROL < typeParameterListIR > ( parameterListIR )
```

```
        ( constructorParameterListIR )
        { controlLocalDeclarationListIR APPLY controlBodyIR }
    ;
```

Control objects are then instantiated by constructor invocations to yield:

```
controlObject
    : CONTROL < theta > ( parameterListIR )
        { frame controlLocalDeclarationListIR actionDefEnv controlBodyIR }
    ;
```

10.5.1. Well-formedness

CONSTRUCTOR `( parameterIR* `) ':' typeIR$_{object}$ is a well-formed constructor type, with bound type variables B:

1. If parameterIR is a well-formed constructor parameter, with bound type variables B, for all parameterIR in parameterIR*:
    a. If parameterIR is not an optional parameter, for all parameterIR in parameterIR*:
        i. If typeIR$_{object}$ is a well-formed type, with bound type variables B:
            A. Let typeIR' be typeIR$_{object}$ with typedefs unrolled.
            B. If typeIR' has type controlObjectTypeIR:
                I. For each parameterIR in parameterIR*:
                    - Let _ _ typeIR _ _ be parameterIR.

                Let typeIR* be the resulting list.

                II. If typeIR can be used in a control constructor type, for all typeIR in typeIR*: then, the relation holds.

10.6. Table objects

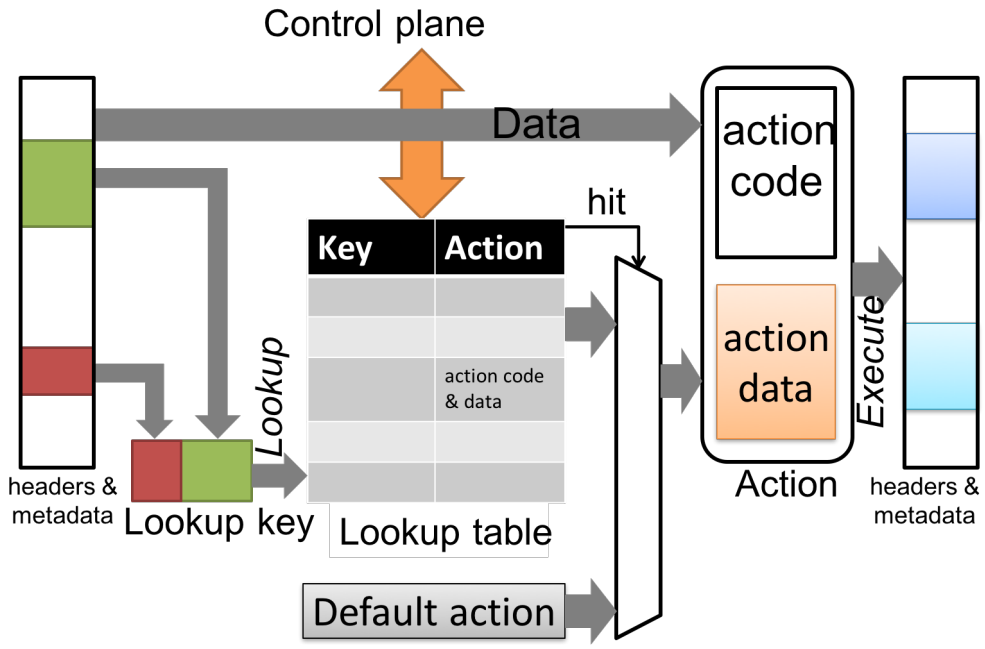

Figure 11. Match-Action Unit Dataflow.

A table describes a match-action unit. The structure of a match-action unit is shown in Figure 11. Processing a packet using a match-action table executes the following steps:

- Key construction.
- Key lookup in a lookup table (the "match" step). The result of key lookup is an "action".
- Action execution (the "action step") over the input data, resulting in mutations of the data.

A table declaration introduces a table instance. To obtain multiple instances of a table, it must be declared within a control block that is itself instantiated multiple times.

The look-up table is a finite map whose contents are manipulated asynchronously (read/write) by the target control plane, through a separate control-plane API (see Figure 11). Note that the term "table" is overloaded: it can refer to the P4 table objects that appear in P4 programs, as well as the internal look-up tables used in targets. We will use the term "match-action unit" when necessary to disambiguate.

Syntactically, a table is defined in terms of a set of key-value properties. Some of these properties are "standard" properties, but the set of properties can be extended by target-specific compilers as needed. Note that duplicated properties are invalid and the compiler should reject them.

```
<span id="tablePropertyList"></span>
<span id="tableProperty"></span>
```

```
tableDeclaration
    : annotationList TABLE name { tablePropertyList }
    ;

tablePropertyList
    : /* empty */ | tablePropertyList tableProperty
    ;

tableProperty
    : KEY = { tableKeyList } | ACTIONS = { tableActionList }
    | annotationList constOpt ENTRIES = { tableEntryList }
    | annotationList constOpt tableCustomName initializer ;
```

```
    ;
```

See Section 17.3 for details on the table properties.

A table declaration creates a table object, thus table constructors are not defined in $P4_{IR}$. Table objects are defined as:


```
<span id="tableObject"></span>
<span id="tablePropertyIR"></span>
<span id="tableObjectProperty"></span>
```


```
tableObject
    : TABLE typeId { frame tableObjectProperty }
    ;

tablePropertyIR
    : tableKeysPropertyIR | tableActionsPropertyIR
    | tableDefaultActionPropertyIR | tableEntriesPropertyIR
    | tableCustomPropertyIR
    ;

tableObjectProperty = {
  KEYS tableKeysPropertyIR,
  ACTIONS tableActionsPropertyIR,
  DEFAULT_ACTION tableDefaultActionPropertyIR,
  ENTRIES tableEntriesPropertyIR,
  CUSTOMS tableCustomPropertyIR*
}
```

10.6.1. Well-formedness


```
<span id="ConstructorType_wf-tableObjectTypeIR"></span>
```


CONSTRUCTOR `( · )` ':' $typeIR_{object}$ is a well-formed constructor type, with bound type variables B:

1. If $typeIR_{object}$ is a well-formed type, with bound type variables B:
    a. Let typeIR be $typeIR_{object}$ with typedefs unrolled.
    b. If typeIR has type tableObjectTypeIR: then, the relation holds.

10.7. Parser value set objects

In some cases, the values that determine the transition from one parser state to another need to be determined at runtime. MPLS is one example where the value of the MPLS label field is used to determine what headers follow the MPLS tag and this mapping may change dynamically at runtime. To support this functionality, P4 supports the notion of a Parser Value Set. This is a named set of values with a runtime API to add and remove values from the set.

Value sets are declared locally within a parser. They should be declared before being referenced in parser keysetExpression and can be used as a label in a select expression.

The syntax for declaring value sets is:


```
<span id="valueSetDeclaration"></span>
<span id="valueSetType"></span>
```


```
valueSetDeclaration
```

```
    : annotationList VALUE_SET < valueSetType > ( expression ) name ;
    ;

valueSetType
    : baseType | tupleType | prefixedTypeName
    ;
```

Parser Value Sets support a size argument to provide hints to the compiler to reserve hardware resources to implement the value set. Thus, the size argument must be a compile-time known value. For example, this parser value set:

```
value_set<bit<16>>(4) pvs;
```

creates a value_set of size 4 with entries of type bit<16>.

The semantics of the size argument is similar to the size property of a table. If a value set has a size argument with value N, it is recommended that a compiler should choose a data plane implementation that is capable of storing N value set entries. See "Size property of P4 tables and parser value sets" [1] for further discussion on the implementation of parser value set size.

The value set is populated by the control plane by methods specified in the P4Runtime specification.[2]

Similar to tables, a value_set declaration creates a new parser value set object. Thus, no constructor is defined for them. The object is represented as:

```
valueSetObject
    : VALUE_SET { value* ( nat ) }
    ;
```

10.8. Package objects

A package type declaration introduces a package object constructor.

```
packageTypeDeclaration
    : annotationList PACKAGE name typeParameterListOpt ( parameterList ) ;
    ;
```

All parameters of a package are evaluated at compilation time, and in consequence they must all be directionless (they cannot be in, out, or inout). Otherwise package types are very similar to parser type declarations. Packages can only be instantiated; there are no runtime behaviors associated with them.

Package object constructors are defined as follows:

```
packageObjectConstructorDef
    : PACKAGE < typeParameterListIR > ( constructorParameterListIR )
    ;
```

Package objects are then instantiated to yield:

```
packageObject
    : PACKAGE < theta > { frame }
    ;
```

10.8.1. Well-formedness

CONSTRUCTOR `( parameterIR* `) ':' $typeIR_{object}$ is a well-formed constructor type, with bound type variables B:

1. If parameterIR is a well-formed constructor parameter, with bound type variables B, for all parameterIR in parameterIR*:
    a. If $typeIR_{object}$ is a well-formed type, with bound type variables B:
        i. Let typeIR' be $typeIR_{object}$ with typedefs unrolled.
        ii. Check that typeIR' has type packageObjectTypeIR.
        iii. For each parameterIR in parameterIR*:
            - Let _ _ typeIR _ _ be parameterIR.

            Let typeIR* be the resulting list.
        iv. Check that typeIR can be used in a package constructor type, for all typeIR in typeIR*.
        v. Then, the relation holds.

[1] https://github.com/p4lang/p4-spec/blob/master/p4-16/spec/docs/p4-table-and-parser-value-set-sizes.md

[2] The P4Runtime API is defined as a Google Protocol Buffer .proto file and an accompanying English specification document here: https://github.com/p4lang/p4runtime

11. Declarations

The syntax of top-level declarations is defined as follows:

```
declaration
    : constantDeclaration | instantiation | functionDeclaration
    | actionDeclaration | errorDeclaration | matchKindDeclaration
    | externDeclaration | parserDeclaration | controlDeclaration
    | typeDeclaration
    ;

externDeclaration
    : externFunctionDeclaration | externObjectDeclaration
    ;

typeDeclaration
    : derivedTypeDeclaration | typedefDeclaration | parserTypeDeclaration
    | controlTypeDeclaration | packageTypeDeclaration
    ;
```

As described in Chapter 7, declarations are type checked, instantiated at compile time, and evaluated at runtime.

11.1. Semantics of declarations

11.1.1. Type checking

Decl_ok:

- Typing declaration under context TC:
- The result is context TC and declarationIR.

Decl_ok dispatch:

1. If let constantDeclaration of type constantDeclaration be declaration: goto constantDeclaration
2. Else if let instantiation of type instantiation be declaration: goto instantiation
3. Else if let functionDeclaration of type functionDeclaration be declaration: goto functionDeclaration
4. Else if let actionDeclaration of type actionDeclaration be declaration: goto actionDeclaration
5. Else if let ERROR `{ nameList `} of type errorDeclaration be declaration: goto errorDeclaration
6. Else if let MATCH_KIND `{ nameList trailingCommaOpt `} of type matchKindDeclaration be declaration: goto matchKindDeclaration
7. Else if let annotationList EXTERN typeOrVoid name typeParameterListOpt `( parameterList `) ';' of type externFunctionDeclaration be declaration: goto externFunctionDeclaration
8. Else if let annotationList EXTERN nonTypeName typeParameterListOpt `{ externConstructorOrMethodPrototypeList `} of type externObjectDeclaration be declaration: goto externObjectDeclaration
9.

Else if let annotationList PARSER name typeParameterListOpt `( parameterList `) constructorParameterListOpt `{ parserLocalDeclarationList parserStateList `} of type parserDeclaration be declaration: goto parserDeclaration

10. Else if let annotationList CONTROL name typeParameterListOpt `( parameterList `) constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody `} of type controlDeclaration be declaration: goto controlDeclaration
11. Else if let enumTypeDeclaration of type enumTypeDeclaration be declaration:
    a. If let annotationList ENUM name `{ $\text{nameList}_{\text{field}}$ _ `} be enumTypeDeclaration: goto enumTypeDeclaration-non-serializable
    b. Else:
        i. Let annotationList ENUM type name `{ $\text{namedExpressionList}_{\text{field}}$ _ `} be enumTypeDeclaration.
        ii. Goto enumTypeDeclaration-serializable
12. Else if let annotationList STRUCT name typeParameterListOpt `{ typeFieldList `} of type structTypeDeclaration be declaration: goto structTypeDeclaration
13. Else if let annotationList HEADER name typeParameterListOpt `{ typeFieldList `} of type headerTypeDeclaration be declaration: goto headerTypeDeclaration
14. Else if let annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList `} of type headerUnionTypeDeclaration be declaration: goto headerUnionTypeDeclaration
15. Else if let typedefDeclaration of type typedefDeclaration be declaration:
    a. If let annotationList TYPEDEF typedef name ';' be typedefDeclaration: goto typedefDeclaration-typedef
    b. Else:
        i. Let annotationList TYPE type name ';' be typedefDeclaration.
        ii. Goto typedefDeclaration-newtype
16. Else if let annotationList PARSER name typeParameterListOpt `( parameterList `) ';' of type parserTypeDeclaration be declaration: goto parserTypeDeclaration
17. Else if let annotationList CONTROL name typeParameterListOpt `( parameterList `) ';' of type controlTypeDeclaration be declaration: goto controlTypeDeclaration
18. Else:
    a. Let annotationList PACKAGE name typeParameterListOpt `( parameterList `) ';' be declaration.
    b. Goto packageTypeDeclaration

After type checking, declarations are represented in P4_{IR} as follows:

```
<span id="declarationIR"></span>
<span id="externDeclarationIR"></span>
<span id="typeDeclarationIR"></span>
```

```
declarationIR
    : constantDeclarationIR | instantiationIR | functionDeclarationIR
    | actionDeclarationIR | errorDeclarationIR | matchKindDeclarationIR
    | externDeclarationIR | parserDeclarationIR | controlDeclarationIR
    | typeDeclarationIR
    ;

externDeclarationIR
    : externFunctionDeclarationIR | externObjectDeclarationIR
    ;

typeDeclarationIR
    : derivedTypeDeclarationIR | typedefDeclarationIR | parserTypeDeclarationIR
    | controlTypeDeclarationIR | packageTypeDeclarationIR
    ;
```

A list of declarations is type checked as follows:

Decls_ok:

- Typing declaration* under context TC:
- The result is context TC and declarationIR*.

Typing declaration* under context TC:

1. If declaration* matches pattern []:
   a. The result is context TC and ·.
2. Else:
   a. Let declaration_h :: declaration_t^* be declaration*.
   b. Let context TC_1 and declarationIR_h be the result of typing declaration_h under context TC.
   c. Let context TC_2 and declarationIR_t^* be the result of typing declaration_t^* under context TC_1.
   d. The result is context TC_2 and declarationIR_h :: declarationIR_t^*.

Some declarations introduce type parameters. These are checked as follows:

TypeParameterListOpt_ok:

- Typing typeParameterListOpt, under context TC at p:
- The result is context TC and type parameters typeId*.

Typing typeParameterListOpt, under context TC_0 at p:

1. Let typeParameter* be typeParameterListOpt flattened.
2. For each typeParameter in typeParameter*:
   - Let typeId be the name of typeParameter.

   Let typeId* be the resulting list.
3. Let TC_1 be type parameters typeId* added to the p layer of TC_0.
4. The result is context TC_1 and type parameters typeId*.

11.1.2. Compile-time evaluation

Decl_inst:

- Loading declarationIR, under context IC, and store STO:
- The result is context IC and store STO.

Decl_inst dispatch:

1. If let constantDeclarationIR of type constantDeclarationIR be declarationIR: goto constantDeclarationIR
2. Else if let instantiationIR of type instantiationIR be declarationIR: goto instantiationIR
3. Else if let functionDeclarationIR of type functionDeclarationIR be declarationIR: goto functionDeclarationIR
4. Else if let actionDeclarationIR of type actionDeclarationIR be declarationIR: goto actionDeclarationIR
5. Else if let ERROR `{ nameIR* `} of type errorDeclarationIR be declarationIR: goto errorDeclarationIR
6. Else if let MATCH_KIND `{ nameIR* `} of type matchKindDeclarationIR be declarationIR: goto matchKindDeclarationIR
7. Else if let annotationList EXTERN typeIR nameIR `< typeParameterListIR$_{expl}$ ',' typeParameterListIR$_{impl}$ `> `( parameterListIR `) ';' of type externFunctionDeclarationIR be declarationIR: goto externFunctionDeclarationIR
8. Else if let annotationList EXTERN nameIR `< typeParameterListIR ',' `> `{ externConstructorPrototypeIR* externMethodPrototypeIR* `} of type externObjectDeclarationIR be declarationIR: goto externObjectDeclarationIR
9. Else if let annotationList PARSER nameIR `< typeParameterListIR$_{expl}$ ',' typeParameterListIR$_{impl}$ `> `( parameterListIR `) `( constructorParameterListIR `) `{ parserLocalDeclarationListIR parserStateListIR `} of type parserDeclarationIR be declarationIR: goto parserDeclarationIR
10. Else if let annotationList CONTROL nameIR `< typeParameterListIR$_{expl}$ ',' typeParameterListIR$_{impl}$ `> `( parameterListIR `) `( constructorParameterListIR `) `{ controlLocalDeclarationListIR APPLY controlBodyIR `} of type controlDeclarationIR be declarationIR: goto controlDeclarationIR
11. Else if let enumTypeDeclarationIR of type enumTypeDeclarationIR be declarationIR:
    a. If let annotationList ENUM nameIR `{ nameIR$_{field}$* `} be enumTypeDeclarationIR: goto enumTypeDeclarationIR-non-serializable
    b. Else:
        i. Let annotationList ENUM typeIR nameIR `{ ( nameIR$_{field}$ '=' value$_{field}$ )* `} be enumTypeDeclarationIR.
        ii. Goto enumTypeDeclarationIR-serializable
12. Else if let annotationList STRUCT nameIR `< typeParameterListIR `> `{ typeFieldIR* `} of type structTypeDeclarationIR be declarationIR: goto structTypeDeclarationIR
13. Else if let annotationList HEADER nameIR `< typeParameterListIR `> `{ typeFieldIR* `} of type headerTypeDeclarationIR be declarationIR: goto headerTypeDeclarationIR
14. Else if let annotationList HEADER_UNION nameIR `< typeParameterListIR `> `{ typeFieldIR* `} of type headerUnionTypeDeclarationIR be declarationIR: goto headerUnionTypeDeclarationIR
15. Else if let typedefDeclarationIR of type typedefDeclarationIR be declarationIR:
    a. If let annotationList TYPEDEF typedefIR nameIR ';' be typedefDeclarationIR: goto typedefDeclarationIR-typedef
    b. Else:
        i. Let annotationList TYPE typeIR nameIR ';' be typedefDeclarationIR.
        ii. Goto typedefDeclarationIR-newtype
16. Else if let annotationList PARSER nameIR `< typeParameterListIR ',' typeParameterListIR$_{inferred}$ `> `( parameterIR* `) ';' of type parserTypeDeclarationIR be declarationIR: goto parserTypeDeclarationIR
17. Else if let annotationList CONTROL nameIR `< typeParameterListIR ',' typeParameterListIR$_{inferred}$ `> `( parameterIR* `) ';' of type controlTypeDeclarationIR be declarationIR: goto controlTypeDeclarationIR

18. Else:
    a. Let annotationList PACKAGE nameIR `< typeParameterListIR ',' typeParameterListIR$_{inferred}$ `> `( parameterIR$^{*}$ `) ';' be declarationIR.
    b. Goto packageTypeDeclarationIR

Instantiation statically allocates instantiated objects in the global store. Section 11.4 describes how objects are instantiated from a declaration.

A list of declarations is instantiated as follows:

Decls_inst:

- Loading declarationIR*, under context IC, and store STO:
- The result is context IC and store STO.

Loading declarationIR*, under context IC, and store STO:

1. If declarationIR* matches pattern []:
    a. The result is context IC and store STO.
2. Else:
    a. Let declarationIR$_{h}$ :: declarationIR$_{t}^{*}$ be declarationIR*.
    b. Let context IC$_{1}$ and store STO$_{1}$ be the result of loading declarationIR$_{h}$, under context IC, and store STO.
    c. Let context IC$_{2}$ and store STO$_{2}$ be the result of loading declarationIR$_{t}^{*}$, under context IC$_{1}$, and store STO$_{1}$.
    d. The result is context IC$_{2}$ and store STO$_{2}$.

11.1.3. Runtime evaluation

All declarations except constant and variable declarations are evaluated at compile-time. See Section 11.2 for the runtime evaluation of variable declarations and Section 11.3 for the runtime evaluation of constant declarations.

The subsequent sections describe each kind of declaration in detail.

11.2. Variable declarations

Local variables are declared with a type, a name, and an optional initializer (as well as an optional annotation):

```
variableDeclaration
    : annotationList type name initializerOpt ;
```

```
    ;

initializerOpt
    : /* empty */ | initializer
    ;

initializer
    : = expression
    ;
```

Variable declarations without an initializer are uninitialized (except for headers and other header-related types, which are initialized to invalid in the same way as described for direction out parameters in Section 19.6). The language places few restrictions on the types of variables: most P4 types that can be written explicitly can be used (e.g., base types, struct, header, header stack, tuple). However, it is impossible to declare variables with type int, or with types that are only synthesized by the compiler (e.g., set) In addition, variables of type parser, control, package, or extern types must be declared using instantiations (see Section 11.4).

Reading the value of a variable that has not been initialized yields an undefined result. The compiler should attempt to detect and emit a warning in such situations.

Variable declarations can appear in the following locations within a P4 program:

- In a block statement,
- In a parser state,
- In a control block's apply sub-block,
- In the list of local declarations in a parser, and
- In the list of local declarations in a control.

Note that variable declarations *cannot* appear at the top level of a P4 program.

Variables have local scope, and behave like stack-allocated variables in languages such as C. The value of a variable is never preserved from one invocation of its enclosing block to the next. In particular, variables cannot be used to maintain state between different network packets.

11.2.1. Type checking

Variable declarations are type checked using the following relation:

VarDecl_ok:

- Typing variableDeclaration, under context TC at p:
- The result is context TC and variableDeclarationIR.

After typing, variable declarations are represented in $P4_{IR}$ as:

```
variableDeclarationIR
    : annotationList typeIR nameIR initializerOptIR ;
```

```
    ;

initializerIR
    : = typedExpressionIR
    ;
```

These are produced by the following rules:

Typing annotationList type name initializerOpt ';', under context TC_0 at p:

1. If initializerOpt matches pattern $_{\text{EMPTY}}$:
    a. Let typeIR and fresh type variables $typeId^*$ be the result of typing type, under context TC_0 at p.
    b. Check that $typeId^*$ is an empty list.
    c. Let bound be bound type variables from the p layer of TC_0.
    d. If typeIR is a well-formed type, with bound type variables bound:
        i. Check that typeIR supports assignment.
        ii. Let nameIR be the name of name.
        iii. Let TC_1 be TC_0 where nameIR to INOUT typeIR DYN · is added to the p layer.
        iv. Let variableDeclarationIR be annotationList typeIR nameIR · ';'.
        v. The result is context TC_1 and variableDeclarationIR.
2. Else:
    a. Let '=' $expression_{init}$ be initializerOpt.
    b. Let typeIR and fresh type variables $typeId^*$ be the result of typing type, under context TC_0 at p.
    c. Check that $typeId^*$ is an empty list.
    d. Let bound' be bound type variables from the p layer of TC_0.
    e. If typeIR is a well-formed type, with bound type variables bound':
        i. Check that typeIR supports assignment.
        ii. Let $typedExpressionIR_{init}$ be the result of typing $expression_{init}$, under context TC_0 at p.
        iii. Let $typedExpressionIR^?$ be $typedExpressionIR_{init}$ implicitly cast to typeIR.
        iv. Check that $typedExpressionIR^?$ is defined.
        v. Let $typedExpressionIR_{init_cast}$ be $typedExpressionIR^?$.
        vi. Let nameIR be the name of name.
        vii. Let TC_1 be TC_0 where nameIR to INOUT typeIR DYN · is added to the p layer.
        viii. Let variableDeclarationIR be annotationList typeIR nameIR '=' $typedExpressionIR_{init_cast}$ ';'.
        ix. The result is context TC_1 and variableDeclarationIR.

11.2.2. Runtime evaluation

Variable declarations are evaluated at runtime using the following relation:

VarDecl_eval:

- Evaluating variableDeclarationIR, under context EC and state ARCH at p:
- The result is context EC, state ARCH, and result declarationResult.

The result of evaluating a variable declaration is:

```
<span id="continueEmptyResult"></span>
<span id="exitResult"></span>
<span id="abortResult"></span>
<span id="declarationResult"></span>
```

```
continueEmptyResult
    : /* empty */
    ;

exitResult
    : EXIT
    ;

abortResult
    : exitResult | rejectTransitionResult
    ;

declarationResult
    : continueEmptyResult | abortResult
    ;
```

See Section 13.7 for how exitResult may occur. The relation is defined by the following rule:

```
<span id="VarDecl_eval-"></span>
```

Evaluating annotationList typeIR nameIR initializerIR? ';', under context EC_0 and state ARCH at p:

1. If initializerIR? matches pattern ():
    a. Let $typeIR_{subst}$ be typeIR substituted by bound type variables in EC_0 from the LOCAL layer.
    b. Let $value_{init}$ be the default value for type $typeIR_{subst}$.
    c. Let EC_1 be EC_0 where $_{BARE}$ nameIR to $value_{init}$ is added to the p layer.
    d. The result is context EC_1, state ARCH, and result $_{EMPTY}$.
2. Else:
    a. Let '=' $typedExpressionIR_{init}$ be initializerIR?.
    b. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating $typedExpressionIR_{init}$, under context EC_0 and state ARCH at p.
    c. If let abortResult of type abortResult be expressionResult:
        i. The result is context EC_1, state $ARCH_1$, and result abortResult.
    d. Else:
        i. Let $_{CONT}$ $value_{init}$ be expressionResult.
        ii. Let EC_2 be EC_1 where $_{BARE}$ nameIR to $value_{init}$ is added to the p layer.
        iii. The result is context EC_2, state $ARCH_1$, and result $_{EMPTY}$.

11.3. Constant declarations

Constant values are defined with the syntax:

```
constantDeclaration
    : annotationList CONST type name initializer ;
    ;

initializer
    : = expression
    ;
```

The initializer expression must be a local compile-time known value. A constant declaration introduces a constant whose value has the specified type. The following are all legal constant declarations:

```
const bit<32> COUNTER = 32w0x0;
struct Version {
    bit<32> major;
    bit<32> minor;
}
const Version version = { 32w0, 32w0 };
```

11.3.1. Type checking

Constant declarations are type checked with the relation:

ConstDecl_ok:

- Typing constantDeclaration, under context TC at p:
- The result is context TC and constantDeclarationIR.

After typing, in $P4_{IR}$, constant declarations have the syntax:

```
constantDeclarationIR
    : annotationList CONST typeIR nameIR constantInitializerIR ;
    ;

constantInitializerIR
    : = _VALUE value
    ;
```

Notice that constantInitializerIR is reduced to a value in $P4_{IR}$, where value is computed from initializer via local compile-time evaluation. It is defined by the following rule:

Typing annotationList CONST type name '=' $\text{expression}_{\text{value}}$ ';', under context TC_0 at p:

1. Let typeIR and fresh type variables typeId^* be the result of typing type, under context TC_0 at p.
2. Check that typeId^* is an empty list.
3. Let bound be bound type variables from the p layer of TC_0.
4. If typeIR is a well-formed type, with bound type variables bound:
    a. Let $\text{typedExpressionIR}_{\text{value}}$ be the result of typing $\text{expression}_{\text{value}}$, under context TC_0 at p.
    b. Check that the compile-time known-ness of $\text{typedExpressionIR}_{\text{value}}$ is LCTK.
    c. Let $\text{typedExpressionIR}^?$ be $\text{typedExpressionIR}_{\text{value}}$ implicitly cast to typeIR.
    d. Check that $\text{typedExpressionIR}^?$ is defined.
    e. Let $\text{typedExpressionIR}_{\text{value_cast}}$ be $\text{typedExpressionIR}^?$.
    f. Let value be the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{value_cast}}$, under context TC_0 at p.
    g. Let nameIR be the name of name.
    h. Let TC_1 be TC_0 where nameIR to $_{\text{EMPTY}}$ typeIR LCTK value is added to the p layer.
    i. Let constantDeclarationIR be annotationList CONST typeIR nameIR '=' $_{\text{VALUE}}$ value ';'.
    j. The result is context TC_1 and constantDeclarationIR.

When used as a top-level declaration, constant declarations are typed with:

Typing constantDeclaration under context TC_0:

1. Let context TC_1 and constantDeclarationIR be the result of typing constantDeclaration, under context TC_0 at GLOBAL.
2. The result is context TC_1 and constantDeclarationIR.

11.3.2. Compile-time evaluation

Constant declarations are evaluated during instantiation phase with:

ConstDecl_inst:

- Loading constantDeclarationIR, under context IC at p:
- The result is context IC.

It is defined by the following rule:

Loading annotationList CONST typeIR nameIR '=' $_{\text{VALUE}}$ value ';', under context IC_0 at p:

1.

Let IC_1 be IC_0 where nameIR to value is added to the p layer.

2. The result is context IC_1.

When used as a top-level declaration, constant declarations are compile-time evaluated with:

Loading constantDeclarationIR, under context IC_0, and store STO:

1. Let context IC_1 be the result of loading constantDeclarationIR, under context IC_0 at GLOBAL.
2. The result is context IC_1 and store STO.

11.3.3. Runtime evaluation

Constant declarations are evaluated at runtime using the following relation:

ConstDecl_eval:

- Evaluating constantDeclarationIR, under context EC at p:
- The result is context EC.

Evaluating annotationList CONST typeIR nameIR '=' $_{VALUE}$ value ';', under context EC_0 at p:

1. Let EC_1 be EC_0 where $_{BARE}$ nameIR to value is added to the p layer.
2. The result is context EC_1.

11.4. Instantiations

Instantiations are similar to variable declarations, but are reserved for the types with constructors (extern objects, control blocks, parsers, and packages):

```
instantiation
    : annotationList type ( argumentList ) name ;
    | annotationList type ( argumentList ) name objectInitializer ;
    ;
```

An instantiation is written as a constructor invocation followed by a name. Instantiations are always executed at compilation time (Section Section 7.3). The effect is to allocate an object with the specified name, and to bind it to the result of the constructor invocation. Note that instantiation arguments can be specified by name.

For example, a hypothetical bank of counter objects can be instantiated as follows:

```
// from target library
enum CounterType {
   Packets,
   Bytes,
   Both
}
extern Counter {
    Counter(bit<32> size, CounterType type);
    void increment(in bit<32> index);
}
// user program
control c(/* parameters omitted */) {
    Counter(32w1024, CounterType.Both) ctr;  // instantiation
    apply { /* body omitted */ }
}
```

A P4 program may not instantiate controls and parsers at the top-level. This restriction is designed to ensure that most state resides in the architecture itself, or is local to a parser or control. For example, the following program is not valid:

```
// Program
control c(/* parameters omitted */) { /* body omitted */ }
c() c1;  // illegal top-level instantiation
```

because control c1 is instantiated at the top-level. Note that top-level declarations of constants and instantiations of extern objects are permitted.

11.4.1. Objects with abstract methods

When instantiating an extern type that has abstract methods users have to supply implementations for all such methods. This is done using object initializers:

```
objectInitializer
    : = { objectDeclarationList }
    ;

objectDeclarationList
    : /* empty */ | objectDeclarationList objectDeclaration
    ;

objectDeclaration
    : functionDeclaration | instantiation
    ;
```

In $P4_{IR}$, object initializers have the syntax:

```
objectInitializerIR
    : = { objectDeclarationListIR }
    ;

objectDeclarationListIR = objectDeclarationIR*
```

```
objectDeclarationIR
    : functionDeclarationIR | instantiationIR
    ;
```

Abstract method implementations must use the same number of parameters, and the same parameter directions as those declared in the extern object declaration. Note that overloading of abstract methods is not allowed, thus an implementation could be matched to a declaration only with the method name.

Abstract method implementations can only use the supplied arguments or refer to values that are in the same initializer block or the top-level scope. When calling another method of the same instance the this keyword is used to indicate the current object instance:

```
// Instantiate a balancer
Balancer() b = {  // provide an implementation for the abstract methods
    bit<4> on_new_flow(in bit<32> address) {
        // uses the address and the number of flows to balance the load
        bit<32> count = this.getFlowCount();  // call method of the same instance
        return (address + count)[3:0];
    }
}
```

Abstract methods may be invoked by users explicitly, or they may be invoked by the target architecture. The architectural description has to specify when the abstract methods are invoked and what the meaning of their arguments and return values is; target architectures may impose additional constraints on abstract methods.

The following checks that an extern object is concrete, without an unresolved abstract method:

typeIR is an extern object type without abstract methods

1. Let typeIR' be typeIR with typedefs unrolled.
2. Return $is_concrete_extern_object'(typeIR').

$is_concrete_extern_object'(typeIR)

1. If typeIR does not have type externObjectTypeIR: return true.
2. Else:
    a. Let EXTERN _ `< _* `> `{ ( _ ':' externMethodTypeDefIR )* `} be typeIR.
    b. If $is_concrete_extern_object''(externMethodTypeDefIR), for all externMethodTypeDefIR in externMethodTypeDefIR*: return true.
3. Otherwise: return false.

$is_concrete_extern_object''(externMethodTypeDefIR)

1. If externMethodTypeDefIR matches pattern EXTERN_METHOD % `< % ',' % `> `( % `) ':' %: return true.
2. Otherwise: return false.

11.4.1.1. Type checking

An object declaration is type checked with the relation:

ObjectDecl_ok:

- Typing objectDeclaration under context TC with typeFrame and externMethodTypeDefEnv at p:
- The result is typeFrame and externMethodTypeDefEnv of the object declaration and objectDeclarationIR.

It is defined by the following rules:

Typing objectDeclaration under context TC_0 with typeFrame and externMethodTypeDefEnv at p:

1. If let instantiation of type instantiation be objectDeclaration:
   a. Let context TC_1 and instantiationIR be the result of typing instantiation, under context TC_0 at p.
   b. Let nameIR be the name of instantiationIR.
   c. Let $varTypeIR^{\iota?}$ be the type of variable $_{BARE}$ nameIR from the p layer of TC_1.
   d. Check that $varTypeIR^{\iota?}$ is defined.
   e. Let varTypeIR be $varTypeIR^{\iota?}$.
   f. Check that ~$in_set<id>(nameIR, $dom_map<id, varTypeIR>(typeFrame)).
   g. Let $typeFrame_{init}$ be the map typeFrame with the value for nameIR is updated to varTypeIR.
   h. The result is $typeFrame_{init}$ and externMethodTypeDefEnv of the object declaration and instantiationIR.
2. Else:
   a. Let annotationList typeOrVoid name typeParameterListOpt `( parameterList `) blockStatement be objectDeclaration.
   b. Let TC_1 be TC_0 with BLOCK.KIND set to EXTERN.
   c. Let TC_2 be TC_1 with BLOCK.FRAME set to typeFrame.
   d. Let context TC_3 and type parameters $typeId_{expl}^*$ be the result of typing typeParameterListOpt, under context TC_2 at LOCAL.
   e. Let context TC_4, $parameterIR^*$, and fresh type variables $typeId_{impl}^*$ be the result of typing parameterList, under context TC_3 at LOCAL.
   f. Let $typeIR_{ret}$ and fresh type variables $typeId^*$ be the result of typing typeOrVoid, under context TC_3 at LOCAL.
   g. Check that $typeId^*$ is an empty list.
   h. Let TC_5 be TC_4 with LOCAL.KIND set to EXTERN_METHOD ':' $typeIR_{ret}$.
   i. Let TC_6 be TC_5[CONTROL = CONT] with LOOP set to OUT.
   j. Let context TC_7 and blockStatementIR be the result of typing blockStatement under context TC_6.
   k. Check that TC_7.CONTROL is equal to RET or $typeIR_{ret}$ is equal to VOID.
   l. Let callableId be the callable identifier for name `( parameterList `).
   m. Let nameIR be the name of name.
   n. Let externMethodTypeDefIR be EXTERN_METHOD nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ':' $typeIR_{ret}$.
   o. Let bound be bound type variables from the p layer of TC_0.
   p. If externMethodTypeDefIR is a well-formed callable type definition, with bound type variables bound:

i. Let externMethodTypeDefEnv$_{init}$ be the map externMethodTypeDefEnv with the value for callableId is updated to externMethodTypeDefIR.

ii. Let functionDeclarationIR be annotationList typeIR$_{ret}$ nameIR `< typeId$_{expl}^{*}$ ',' typeId$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) blockStatementIR.

iii. The result is typeFrame and externMethodTypeDefEnv$_{init}$ of the object declaration and functionDeclarationIR.

A list of object declarations is type checked with the relation:

ObjectDeclList_ok:

- Typing objectDeclarationList under context TC at p:
- The result is typeFrame and externMethodTypeDefEnv of the object declaration block and objectDeclarationListIR.

Typing objectDeclarationList under context TC at p:

1. Let objectDeclaration* be objectDeclarationList flattened.
2. Let typeFrame and externMethodTypeDefEnv of the object declaration block and objectDeclarationListIR be the result of typing objectDeclaration* under context TC with `{ · `} and `{ · `} at p.
3. The result is typeFrame and externMethodTypeDefEnv of the object declaration block and objectDeclarationListIR.

ObjectDecls_ok:

- Typing objectDeclaration* under context TC with typeFrame and externMethodTypeDefEnv at p:
- The result is typeFrame and externMethodTypeDefEnv of the object declaration block and objectDeclarationListIR.

Typing objectDeclaration* under context TC with typeFrame and externMethodTypeDefEnv at p:

1. If objectDeclaration* matches pattern []:
   a. The result is typeFrame and externMethodTypeDefEnv of the object declaration block and ·.
2. Else:
   a. Let objectDeclaration$_h$ :: objectDeclaration$_t^{*}$ be objectDeclaration*.
   b. Let typeFrame$_1$ and externMethodTypeDefEnv$_1$ of the object declaration and objectDeclarationIR$_h$ be the result of typing objectDeclaration$_h$ under context TC with typeFrame and externMethodTypeDefEnv at p.

c. Let typeFrame_2 and $\text{externMethodTypeDefEnv}_2$ of the object declaration block and $\text{objectDeclarationIR}_t^*$ be the result of typing $\text{objectDeclaration}_t^*$ under context TC with typeFrame_1 and $\text{externMethodTypeDefEnv}_1$ at p.

d. The result is typeFrame_2 and $\text{externMethodTypeDefEnv}_2$ of the object declaration block and $\text{objectDeclarationIR}_h :: \text{objectDeclarationIR}_t^*$.

11.4.1.2. Compile-time evaluation

At compile-time, an object declaration is loaded with the relation:

ObjectDecl_inst:

- Loading objectDeclarationIR under instantiation context IC and store STO:
- The result is the updated instantiation context IC and store STO.

It is defined by the following rules:

Loading objectDeclarationIR under instantiation context IC_0 and store STO:

1. If let functionDeclarationIR of type functionDeclarationIR be objectDeclarationIR:
   a. Let context IC_1 be the result of loading functionDeclarationIR, under context IC_0 at BLOCK.
   b. The result is the updated instantiation context IC_1 and store STO.
2. Else:
   a. Let instantiationIR be objectDeclarationIR.
   b. Let context IC_1, store STO_1, and reference to object _ be the result of loading instantiationIR, under context IC_0 and store STO at BLOCK.
   c. The result is the updated instantiation context IC_1 and store STO_1.

A list of object declarations is loaded with the relation:

ObjectDecls_inst:

- Loading objectDeclarationListIR under instantiation context IC and store STO:
- The result is the updated instantiation context IC and store STO.

Loading $\text{objectDeclarationIR}^*$ under instantiation context IC and store STO:

1. If $\text{objectDeclarationIR}^*$ matches pattern []:

a. The result is the updated instantiation context IC and store STO.

2. Else:

   a. Let $objectDeclarationIR_h :: objectDeclarationIR_t^*$ be $objectDeclarationIR^*$.

   b. Let the updated instantiation context IC_1 and store STO_1 be the result of loading $objectDeclarationIR_h$ under instantiation context IC and store STO.

   c. Let the updated instantiation context IC_2 and store STO_2 be the result of loading $objectDeclarationIR_t^*$ under instantiation context IC_1 and store STO_1.

   d. The result is the updated instantiation context IC_2 and store STO_2.

11.4.2. Type checking

Instantiations are type checked with the relation:

InstDecl_ok:

- Typing instantiation, under context TC at p:
- The result is context TC and instantiationIR.

After type checking, in $P4_{IR}$, instantiations have the syntax:

```
instantiationIR
    : annotationList typeIR constructorTargetIR ( argumentListIR ) nameIR
        objectInitializerOptIR ;
    ;
```

Typing instantiation, under context TC_0 at p:

1. If let annotationList type `( argumentList `) name ';' be instantiation:

   a. If let prefixedTypeName of type prefixedTypeName be type:

      i. Let context TC_1 and instantiationIR be the result of typing annotationList prefixedTypeName `< $_{EMPTY}$ `> `( argumentList `) name ';' under context TC_0 at p.

      ii. The result is context TC_1 and instantiationIR.

   b. Else if let prefixedTypeName `< typeArgumentList `> of type specializedType be type:

      i. Let context TC_1 and instantiationIR be the result of typing annotationList prefixedTypeName `< typeArgumentList `> `( argumentList `) name ';' under context TC_0 at p.

      ii. The result is context TC_1 and instantiationIR.

2. Else:

   a. Let annotationList type `( argumentList `) name '=' `{ objectDeclarationList `} ';' be instantiation.

   b. If let prefixedTypeName of type prefixedTypeName be type:

      i. Let context TC_1 and instantiationIR be the result of typing annotationList prefixedTypeName `< $_{EMPTY}$ `> `(

argumentList `) name '=' `{ objectDeclarationList `} ';' under context TC_0 at p.

ii. The result is context TC_1 and instantiationIR.

c. Else if let prefixedTypeName `< typeArgumentList `> of type specializedType be type:

i. Let context TC_1 and instantiationIR be the result of typing annotationList prefixedTypeName `< typeArgumentList `> `( argumentList `) name '=' `{ objectDeclarationList `} ';' under context TC_0 at p.

ii. The result is context TC_1 and instantiationIR.

When used as a top-level declaration, instantiations are typed with:

Typing instantiation under context TC_0:

1. Let context TC_1 and instantiationIR be the result of typing instantiation, under context TC_0 at GLOBAL.
2. The result is context TC_1 and instantiationIR.

There are two sub-relations, depending on whether object initializers were provided or not.

When object initializers are absent

InstDecl_non_objectInitializer_ok:

- Typing annotationList prefixedTypeName `< typeArgumentList `> `( argumentList `) name ';' under context TC at p:
- The result is context TC and instantiationIR.

Typing annotationList prefixedTypeName `< typeArgumentList `> `( argumentList `) name ';' under context TC_0 at p:

1. Check that annotationList has at most one name annotation.
2. Let $typeArgumentIR^*$ and fresh type variables $typeId_{impl}{}^*$ be the result of typing typeArgumentList, under context TC_0 at p.
3. Let $argumentIR^*$ be the result of typing argumentList, under context TC_0 at p.
4. Let prefixedNameIR be the prefixed name of prefixedTypeName.
5. Let constructorTargetIR be prefixedNameIR `< $typeArgumentIR^*$ `>.
6. Let constructorTypeIR with fresh type variables $typeId_{inserted}{}^*$, defaults $id_{default}{}^*$ and optionals $id_{optional}{}^*$ be the result of finding type of constructor call constructorTargetIR `( $argumentIR^*$ `), under cursor p and context TC_0.
7. Let $typeArgumentIR_{infer}{}^*$ be $typeArgumentIR^*$ concatenated with (NAME $typeId_{inserted}$)*.
8. Let $typeId_{infer}{}^*$ be $typeId_{impl}{}^*$ concatenated with $typeId_{inserted}{}^*$.
9. Let constructed type $typeIR_{object}$ with inferred $typeArgumentIR_{inferred}{}^*$ and casted $argumentIR_{cast}{}^*$ be the result of typing instantiation constructorTypeIR `< $typeArgumentIR_{infer}{}^*$ `> `( $argumentIR^*$ `) with omitted type arguments $typeId_{infer}{}^*$, defaults $id_{default}{}^*$ and optionals $id_{optional}{}^*$, under cursor p, context TC_0, and NAMED instantiation site context.
10. Check that $typeIR_{object}$ is an extern object type without abstract methods.

11. Let nameIR be the name of name.
12. Let TC_1 be TC_0 where nameIR to $_{EMPTY}$ $typeIR_{object}$ CTK · is added to the p layer.
13. Let $constructorTargetIR_{inferred}$ be prefixedNameIR `< $typeArgumentIR_{inferred}^{*}$ `>.
14. Let instantiationIR be annotationList $typeIR_{object}$ $constructorTargetIR_{inferred}$ `( $argumentIR_{cast}^{*}$ `) nameIR · ';'.
15. The result is context TC_1 and instantiationIR.

When object initializers are present

InstDecl_objectInitializer_ok:

- Typing annotationList prefixedTypeName `< typeArgumentList `> `( argumentList `) name '=' `{ objectDeclarationList `} ';' under context TC at p:
- The result is context TC and instantiationIR.

Typing annotationList prefixedTypeName `< typeArgumentList `> `( argumentList `) name '=' `{ objectDeclarationList `} ';' under context TC_0 at p:

1. Check that annotationList has at most one name annotation.
2. Let $typeArgumentIR^{*}$ and fresh type variables $typeId_{impl}^{*}$ be the result of typing typeArgumentList, under context TC_0 at p.
3. Let $argumentIR^{*}$ be the result of typing argumentList, under context TC_0 at p.
4. Let prefixedNameIR be the prefixed name of prefixedTypeName.
5. Let constructorTargetIR be prefixedNameIR `< $typeArgumentIR^{*}$ `>.
6. Let constructorTypeIR with fresh type variables $typeId_{inserted}^{*}$, defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$ be the result of finding type of constructor call constructorTargetIR `( $argumentIR^{*}$ `), under cursor p and context TC_0.
7. Let $typeArgumentIR_{infer}^{*}$ be $typeArgumentIR^{*}$ concatenated with ($_{NAME}$ $typeId_{inserted}$)*.
8. Let $typeId_{infer}^{*}$ be $typeId_{impl}^{*}$ concatenated with $typeId_{inserted}^{*}$.
9. Let constructed type $typeIR_{object}$ with inferred $typeArgumentIR_{inferred}^{*}$ and casted $argumentIR_{cast}^{*}$ be the result of typing instantiation constructorTypeIR `< $typeArgumentIR_{infer}^{*}$ `> `( $argumentIR^{*}$ `) with omitted type arguments $typeId_{infer}^{*}$, defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$, under cursor p, context TC_0, and NAMED instantiation site context.
10. Let typeIR be $typeIR_{object}$ with typedefs unrolled.
11. Check that typeIR has type externObjectTypeIR.
12. Let EXTERN $typeId_{extern}$ `< $typeIR_{arg}^{*}$ `> $externMethodTypeDefEnv_{extern}$ be typeIR.
13. Let TC_1 be TC_0 where "this" to $_{EMPTY}$ $typeIR_{object}$ CTK · is added to the LOCAL layer.
14. Let $typeFrame_{init}$ and $externMethodTypeDefEnv_{init}$ of the object declaration block and $objectDeclarationIR^{*}$ be the result of typing objectDeclarationList under context TC_1 at p.
15. Let $externMethodTypeDefEnv_{init_subst}$ be substitution of abstract methods in $externMethodTypeDefEnv_{extern}$ by implementations in $externMethodTypeDefEnv_{init}$.
16. Let $typeIR_{object_init}$ be EXTERN $typeId_{extern}$ `< $typeIR_{arg}^{*}$ `> $externMethodTypeDefEnv_{init_subst}$.
17. Check that $typeIR_{object_init}$ is an extern object type without abstract methods.

18. Let nameIR be the name of name.
19. Let TC_2 be TC_0 where nameIR to $_{EMPTY}$ $typeIR_{object_init}$ CTK · is added to the p layer.
20. Let $constructorTargetIR_{inferred}$ be prefixedNameIR `< $typeArgumentIR_{inferred}^{*}$ `>.
21. Let instantiationIR be annotationList $typeIR_{object_init}$ $constructorTargetIR_{inferred}$ `( $argumentIR_{cast}^{*}$ `) nameIR '=' `{ $objectDeclarationIR^{*}$ `} ';'.
22. The result is context TC_2 and instantiationIR.

11.4.3. Compile-time evaluation

Instantiations are evaluated at compile-time with the relation:

InstDecl_inst:

- Loading instantiationIR, under context IC and store STO at p:
- The result is context IC, store STO, and reference to object constantDeclarationIR.

Loading annotationList typeIR constructorTargetIR `( argumentListIR `) nameIR objectInitializerOptIR ';', under context IC_0 and store STO_0 at p:

1. Let prefixedNameIR `< typeArgumentListIR `> be constructorTargetIR.
2. Let constructor constructorDef `< $typeArgumentListIR_{inst}$ `> with defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$ be the result of finding constructor for prefixedNameIR `< typeArgumentListIR `> `( argumentListIR `) under context IC_0 at p.
3. Try:
   a. Let IC_{inner} be IC_0 with nameIR added to the path.
   b. Let store STO_1 and instantiated object be the result of instantiating constructorDef `< $typeArgumentListIR_{inst}$ `> `( argumentListIR `) with defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$, under context IC_{inner} and store STO_0 at p.
   c. If let EXTERN typeId `< theta `> `{ objectState frame externMethodDefEnv `} of type externObject be object:
      i. Let IC_{decl} be an instantiation context same as IC_0 up to the GLOBAL layer.
      ii. Let objectDeclarationListIR be the list of object declarations in objectInitializerOptIR.
      iii. Let the updated instantiation context IC_{decl_post} and store STO_2 be the result of loading objectDeclarationListIR under instantiation context IC_{decl} and store STO_1.
      iv. Let $frame_{merged}$ be the merged frame of frame and IC_{decl_post}.BLOCK.FRAME.
      v. Let $externMethodDefEnv_{merged}$ be the merged extern method definition environment of externMethodDefEnv and IC_{decl_post}.BLOCK.CALLABLE.
      vi. Let $object_{merged}$ be EXTERN typeId `< theta `> `{ objectState $frame_{merged}$ $externMethodDefEnv_{merged}$ `}.
      vii. Let objectId be IC_0.PATH concatenated with nameIR.
      viii. Let STO_3 be STO_2 where objectId to $object_{merged}$ is added.
      ix. Let IC_1 be IC_0 where nameIR to REF objectId is added to the p layer.
      x. Let constantDeclarationIR be $_{EMPTY}$ CONST typeIR nameIR '=' $_{VALUE}$ REF objectId ';'.
      xi. The result is context IC_1, store STO_3, and reference to object constantDeclarationIR.

4. Then, try:
    a. Let nameIR' be $name_annotation_default(annotationList, nameIR).
    b. Let IC_{inner} be IC_0 with nameIR' added to the path.
    c. Let store STO_1 and instantiated object be the result of instantiating constructorDef `< $typeArgumentListIR_{inst}$ `> `( argumentListIR `) with defaults $id_{default}^*$ and optionals $id_{optional}^*$, under context IC_{inner} and store STO_0 at p.
    d. Check that object does not have type externObject.
    e. Let objectId be IC_0.PATH concatenated with nameIR.
    f. Let STO_2 be STO_1 where objectId to object is added.
    g. Let IC_1 be IC_0 where nameIR to REF objectId is added to the p layer.
    h. Let constantDeclarationIR be $_{EMPTY}$ CONST typeIR nameIR '=' $_{VALUE}$ REF objectId ';'.
    i. The result is context IC_1, store STO_2, and reference to object constantDeclarationIR.

When used as a top-level declaration, instantiations are evaluated with:

Loading instantiationIR, under context IC_0, and store STO:

1. Let context IC_1, store STO_1, and reference to object _ be the result of loading instantiationIR, under context IC_0 and store STO at GLOBAL.
2. The result is context IC_1 and store STO_1.

11.5. Function declarations

Function declarations introduce user-defined functions into a P4 program. Section 9.4.1 describes the semantics of user-defined functions in P4.

```
functionPrototype
    : typeOrVoid name typeParameterListOpt ( parameterList )
    ;

functionDeclaration
    : annotationList functionPrototype blockStatement
    ;
```

11.5.1. Type checking

Function declarations are type checked with the relation:

FuncDecl_ok:

- Typing functionDeclaration, under context TC at p:
- The result is context TC and functionDeclarationIR.

After type checking, function declarations are represented in $P4_{IR}$ as:

```
functionPrototypeIR
    : typeIR nameIR < typeParameterListIR , typeParameterListIR >
        ( parameterListIR )
    ;

functionDeclarationIR
    : annotationList functionPrototypeIR blockStatementIR
    ;
```

They are produced by the following rule:

Typing annotationList typeOrVoid name typeParameterListOpt `( parameterList `) blockStatement, under context TC_0 at p:

1. Let context TC_1 and type parameters $typeId_{expl}^*$ be the result of typing typeParameterListOpt, under context TC_0 at LOCAL.
2. Let $typeIR_{ret}$ and fresh type variables $typeId^*$ be the result of typing typeOrVoid, under context TC_1 at LOCAL.
3. Check that $typeId^*$ is an empty list.
4. Let context TC_2, $parameterIR^*$, and fresh type variables $typeId_{impl}^*$ be the result of typing parameterList, under context TC_1 at LOCAL.
5. Let TC_3 be TC_2 with LOCAL.KIND set to FUNCTION ':' $typeIR_{ret}$.
6. Let TC_4 be TC_3[CONTROL = CONT] with LOOP set to OUT.
7. Let context TC_5 and blockStatementIR be the result of typing blockStatement under context TC_4.
8. Check that TC_5.CONTROL is equal to RET or $typeIR_{ret}$ is equal to VOID.
9. Let callableId be the callable identifier for name `( parameterList `).
10. Let nameIR be the name of name.
11. Let definedFunctionTypeDefIR be FUNCTION nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ':' $typeIR_{ret}$.
12. Let bound be bound type variables from the p layer of TC_0.
13. If definedFunctionTypeDefIR is a well-formed callable type definition, with bound type variables bound:
    a. Let TC_6 be TC_0 where callableId to definedFunctionTypeDefIR is added as an overloaded callable to the p layer.
    b. Let functionDeclarationIR be annotationList $typeIR_{ret}$ nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) blockStatementIR.
    c. The result is context TC_6 and functionDeclarationIR.

When used as a top-level declaration, function declarations are typed with:

Typing functionDeclaration under context TC_0:

1. Let context TC_1 and functionDeclarationIR be the result of typing functionDeclaration, under context TC_0 at GLOBAL.
2. The result is context TC_1 and functionDeclarationIR.

11.5.2. Compile-time evaluation

At compile-time, function declarations are loaded into the context with the relation:

FuncDecl_inst:

- Loading functionDeclarationIR, under context IC at p:
- The result is context IC.

It is defined by the following rule:

Loading annotationList typeIR nameIR `< typeParameterListIR$_{expl}$ ',' typeParameterListIR$_{impl}$ `> `( parameterListIR `) blockStatementIR, under context IC$_0$ at p:

1. Let callableId be the callable identifier for nameIR `( parameterListIR `).
2. Let typeParameterListIR be typeParameterListIR$_{expl}$ concatenated with typeParameterListIR$_{impl}$.
3. Let definedFunctionDef be FUNCTION nameIR `< typeParameterListIR `> `( parameterListIR `) blockStatementIR.
4. Let IC$_1$ be IC$_0$ where callableId to definedFunctionDef is added as an overloaded callable to the p layer.
5. The result is context IC$_1$.

When used as a top-level declaration, function declarations are loaded with:

Loading functionDeclarationIR, under context IC$_0$, and store STO:

1. Let context IC$_1$ be the result of loading functionDeclarationIR, under context IC$_0$ at GLOBAL.
2. The result is context IC$_1$ and store STO.

11.6. Action declarations

action declarations introduce actions:

```
actionDeclaration
    : annotationList ACTION name ( parameterList ) blockStatement
    ;
```

Section 9.3 describes the semantics of actions.

11.6.1. Type checking

Action declarations are type checked with the relation:

ActionDecl_ok:

- Typing actionDeclaration, under context TC at p:
- The result is context TC and actionDeclarationIR.

After type checking, action declarations are represented in $P4_{IR}$ as:

```
actionDeclarationIR
    : annotationList ACTION nameIR ( parameterListIR ) blockStatementIR
    ;
```

These are produced by the following rule:

Typing annotationList ACTION name `( parameterList `) blockStatement, under context TC_0 at p:

1. Check that annotationList has at most one name annotation.
2. Let TC_1 be TC_0 with LOCAL.KIND set to ACTION.
3. Let TC_2 be TC_1[CONTROL = CONT] with LOOP set to OUT.
4. Let context TC_3, parameterIR*, and fresh type variables typeId* be the result of typing parameterList, under context TC_2 at LOCAL.
5. Check that typeId* is an empty list.
6. Let context _ and blockStatementIR be the result of typing blockStatement under context TC_3.
7. Let callableId be the callable identifier for name `( parameterList `).
8. Let nameIR be the name of name.
9. Let actionTypeDefIR be annotationList ACTION nameIR `( parameterIR* `).
10. Let bound be bound type variables from the p layer of TC_0.
11. If actionTypeDefIR is a well-formed callable type definition, with bound type variables bound:
    a. Let TC_4 be TC_0 where callableId to actionTypeDefIR is added as a non-overloaded callable to the p layer.
    b. Let actionDeclarationIR be annotationList ACTION nameIR `( parameterIR* `) blockStatementIR.
    c. The result is context TC_4 and actionDeclarationIR.

When used as a top-level declaration, action declarations are typed with:

Typing actionDeclaration under context TC_0:

1. Let context TC_1 and actionDeclarationIR be the result of typing actionDeclaration, under context TC_0 at GLOBAL.
2. The result is context TC_1 and actionDeclarationIR.

11.6.2. Compile-time evaluation

At compile-time, action declarations are loaded into the context with the relation:

ActionDecl_inst:

- Loading actionDeclarationIR, under context IC at p:
- The result is context IC.

It is defined by the following rule:

Loading annotationList ACTION nameIR `( parameterListIR `) blockStatementIR, under context IC_0 at p:

1. Let callableId be the callable identifier for nameIR `( parameterListIR `).
2. Let actionDef be annotationList ACTION nameIR `( parameterListIR `) blockStatementIR.
3. Let IC_1 be IC_0 where callableId to actionDef is added as a non-overloaded callable to the p layer.
4. The result is context IC_1.

When used as a top-level declaration, action declarations are loaded with:

Loading actionDeclarationIR, under context IC_0, and store STO:

1. Let context IC_1 be the result of loading actionDeclarationIR, under context IC_0 at GLOBAL.
2. The result is context IC_1 and store STO.

11.7. Error declarations

```
errorDeclaration
    : ERROR { nameList }
    ;
```

error declarations introduce error elements. See Section 8.2.4 for more details on error elements.

11.7.1. Type checking

After type checking, an error declaration is represented as:

```
errorDeclarationIR
    : ERROR { nameListIR }
```

```
    ;
```

Typing ERROR `{ nameList `} under context TC_0:

1. Let name* be nameList flattened.
2. For each name in name*:
    - Let nameIR be the name of name.

    Let nameIR* be the resulting list.
3. Check that the elements of nameIR* are distinct.
4. For each nameIR in nameIR*:
    - Let $nameIR_{error}$ be "error." concatenated with nameIR.

    Let $nameIR_{error}$* be the resulting list.
5. For each nameIR in nameIR*:
    - Let $value_{error}$ be ERROR '.' nameIR.

    Let $value_{error}$* be the resulting list.
6. For each $value_{error}$ in $value_{error}$*:
    - Let varTypeIR be $_{EMPTY}$ ERROR LCTK $value_{error}$.

    Let varTypeIR* be the resulting list.
7. Let TC_1 be TC_0 where each of $nameIR_{error}$* to each of varTypeIR* are added to the GLOBAL layer.
8. The result is context TC_1 and ERROR `{ nameIR* `}.

11.7.2. Compile-time evaluation

At comile-time, error declarations are loaded into the context by:

Loading ERROR `{ nameIR* `}, under context IC_0, and store STO:

1. For each nameIR in nameIR*:
    - Let $nameIR_{error}$ be the concatenation of ["error.", nameIR].

    Let $nameIR_{error}$* be the resulting list.
2. For each nameIR in nameIR*:
    - Let $value_{error}$ be ERROR '.' nameIR.

    Let $value_{error}$* be the resulting list.
3.

Let IC_1 be IC_0 where each of $nameIR_{error}^*$ to each of $value_{error}^*$ are added to the GLOBAL layer.

4. The result is context IC_1 and store STO.

11.8. Match kind declarations

```
matchKindDeclaration
    : MATCH_KIND { nameList trailingCommaOpt }
    ;
```

Similar to error declarations, match_kind declarations introduce match kinds. See Section 8.2.5 for more details on match kinds.

11.8.1. Type checking

After type checking, an match_kind declaration is represented as:

```
matchKindDeclarationIR
    : MATCH_KIND { nameListIR }
    ;
```

Typing MATCH_KIND `{ nameList trailingCommaOpt `} under context TC_0:

1. Let $name^*$ be nameList flattened.
2. For each name in $name^*$:

   - Let nameIR be the name of name.

   Let $nameIR^*$ be the resulting list.

3. Check that the elements of $nameIR^*$ are distinct.
4. For each nameIR in $nameIR^*$:

   - Let $value_{match_kind}$ be MATCH_KIND '.' nameIR.

   Let $value_{match_kind}^*$ be the resulting list.

5. For each $value_{match_kind}$ in $value_{match_kind}^*$:

   - Let varTypeIR be $_{EMPTY}$ MATCH_KIND LCTK $value_{match_kind}$.

   Let $varTypeIR^*$ be the resulting list.

6. Let TC_1 be TC_0 where each of $nameIR^*$ to each of $varTypeIR^*$ are added to the GLOBAL layer.
7. The result is context TC_1 and MATCH_KIND `{ $nameIR^*$ `}.

11.8.2. Compile-time evaluation

At comile-time, match kind declarations are loaded into the context by:

Loading MATCH_KIND `{ nameIR* `}, under context IC_0, and store STO:

1. For each nameIR in nameIR*:
    - Let $value_{match_kind}$ be MATCH_KIND '.' nameIR.

   Let $value_{match_kind}$* be the resulting list.
2. Let IC_1 be IC_0 where each of nameIR* to each of $value_{match_kind}$* are added to the GLOBAL layer.
3. The result is context IC_1 and store STO.

11.9. Extern function declarations

```
externFunctionDeclaration
    : annotationList EXTERN functionPrototype ;
    ;
```

An extern function declaration introduces an extern function. See Section 9.4.2 for how extern functions are internally represented.

11.9.1. Type checking

After type checking, an extern function declaration is represented in $P4_{IR}$ as:

```
externFunctionDeclarationIR
    : annotationList EXTERN functionPrototypeIR ;
    ;
```

Typing annotationList EXTERN typeOrVoid name typeParameterListOpt `( parameterList `) ';' under context TC_0:

1. Let context TC_1 and type parameters $typeId_{expl}$* be the result of typing typeParameterListOpt, under context TC_0 at LOCAL.
2. Let context TC_2, parameterIR*, and fresh type variables $typeId_{impl}$* be the result of typing parameterList, under context TC_1 at LOCAL.
3. Let $typeIR_{ret}$ and fresh type variables typeId* be the result of typing typeOrVoid, under context TC_1 at LOCAL.
4. Check that typeId* is an empty list.
5. Let callableId be the callable identifier for name `( parameterList `).
6. Let nameIR be the name of name.
7.

Let externFunctionTypeDefIR be EXTERN_FUNCTION nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ':' $typeIR_{ret}$.

8. Let bound be bound type variables from the GLOBAL layer of TC_0.
9. If externFunctionTypeDefIR is a well-formed callable type definition, with bound type variables bound:
    a. Let TC_4 be TC_0 where callableId to externFunctionTypeDefIR is added as an overloaded callable to the GLOBAL layer.
    b. Let externFunctionDeclarationIR be annotationList EXTERN $typeIR_{ret}$ nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ';'.
    c. The result is context TC_4 and externFunctionDeclarationIR.

11.9.2. Compile-time evaluation

At comile-time, extern function declarations are loaded into the context by:

Loading annotationList EXTERN typeIR nameIR `< $typeParameterListIR_{expl}$ ',' $typeParameterListIR_{impl}$ `> `( parameterListIR `) ';', under context IC_0, and store STO:

1. Let callableId be the callable identifier for nameIR `( parameterListIR `).
2. Let typeParameterListIR be $typeParameterListIR_{expl}$ concatenated with $typeParameterListIR_{impl}$.
3. Let externFunctionDef be EXTERN_FUNCTION nameIR `< typeParameterListIR `> `( parameterListIR `).
4. Let IC_1 be IC_0 where callableId to externFunctionDef is added as an overloaded callable to the GLOBAL layer.
5. The result is context IC_1 and store STO.

11.10. Extern object declarations

An extern object declaration introduces an extern object.

```
externObjectDeclaration
    : annotationList EXTERN nonTypeName typeParameterListOpt
        { externConstructorOrMethodPrototypeList }
    ;

externConstructorOrMethodPrototype
    : externConstructorPrototype | externMethodPrototype
    ;

externConstructorOrMethodPrototypeList
    : /* empty */
    | externConstructorOrMethodPrototypeList externConstructorOrMethodPrototype
    ;
```

See Section 10.3 for how extern objects are internally represented. An extern object declaration includes its constructors and methods, to be explained in the following sections.

11.10.1. Extern methods

Extern objects can have methods, which are declared using the following syntax:

```
externMethodPrototype
    : annotationList functionPrototype ;
    | annotationList ABSTRACT functionPrototype ;
    ;
```

Whether an extern prototype is a constructor or a method prototype is tested with:

$is_externConstructorPrototype(externConstructorOrMethodPrototype)

1. If externConstructorOrMethodPrototype has type externConstructorPrototype: return true.
2. Otherwise: return false.

$is_externMethodPrototype(externConstructorOrMethodPrototype)

1. Let bool be $is_externConstructorPrototype(externConstructorOrMethodPrototype).
2. Return ~bool.

An extern object body mixes constructor and method prototypes; they are separated with:

externConstructorOrMethodPrototypeList split into constructors and methods

1. Let externConstructorOrMethodPrototype* be externConstructorOrMethodPrototypeList flattened.
2. For each externConstructorOrMethodPrototype in externConstructorOrMethodPrototype*:
   - Let bool' be $is_externConstructorPrototype(externConstructorOrMethodPrototype).

   Let bool'* be the resulting list.
3. Let externConstructorOrMethodPrototype'* be externConstructorOrMethodPrototype* filtered by bool'*.
4. Check that externConstructorOrMethodPrototype'* has type externConstructorPrototype*.
5. Let externConstructorPrototype* be externConstructorOrMethodPrototype'*.
6. For each externConstructorOrMethodPrototype in externConstructorOrMethodPrototype*:
   - Let bool be $is_externMethodPrototype(externConstructorOrMethodPrototype).

   Let bool* be the resulting list.
7. Let externConstructorOrMethodPrototype''* be externConstructorOrMethodPrototype* filtered by bool*.
8. Check that externConstructorOrMethodPrototype''* has type externMethodPrototype*.
9. Let externMethodPrototype* be externConstructorOrMethodPrototype''*.
10. Return (externConstructorPrototype*, externMethodPrototype*).

See Section 9.5.1 for details on how extern methods are represented internally.

11.10.1.1. Type checking

Extern methods are type checked using the following relation:

ExternMethod_ok:

- Typing externMethodPrototype, under context TC for an extern object typeId:
- The result is externMethodPrototypeIR.

After type checking, in $P4_{IR}$, an extern method declaration is represented as:

```
externMethodPrototypeIR
    : annotationList functionPrototypeIR ;
    | annotationList ABSTRACT functionPrototypeIR ;
    ;
```

These are produced by:

Typing externMethodPrototype, under context TC_0 for an extern object $typeId_{extern}$:

1. If let annotationList functionPrototype ';' be externMethodPrototype:
    a. Let typeOrVoid name typeParameterListOpt `( parameterList `) be functionPrototype.
    b. Let nameIR be the name of name.
    c. Check that nameIR is not equal to $typeId_{extern}$.
    d. Let context TC_1 and type parameters $typeId_{expl}^*$ be the result of typing typeParameterListOpt, under context TC_0 at LOCAL.
    e. Let $typeIR_{ret}$ and fresh type variables $typeId^*$ be the result of typing typeOrVoid, under context TC_1 at LOCAL.
    f. Check that $typeId^*$ is an empty list.
    g. Let TC_2 be TC_1 with LOCAL.KIND set to EXTERN_METHOD ':' $typeIR_{ret}$.
    h. Let context TC_3, $parameterIR^*$, and fresh type variables $typeId_{impl}^*$ be the result of typing parameterList, under context TC_2 at LOCAL.
    i. Let externMethodTypeDefIR be EXTERN_METHOD nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ':' $typeIR_{ret}$.
    j. Let bound be bound type variables from the BLOCK layer of TC_0.
    k. If externMethodTypeDefIR is a well-formed callable type definition, with bound type variables bound:
        i. Let externMethodPrototypeIR be annotationList $typeIR_{ret}$ nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ';'.
        ii. The result is externMethodPrototypeIR.
2. Else:
    a. Let annotationList ABSTRACT functionPrototype ';' be externMethodPrototype.
    b. Let typeOrVoid name typeParameterListOpt `( parameterList `) be functionPrototype.
    c. Let nameIR be the name of name.
    d. Check that nameIR is not equal to $typeId_{extern}$.
    e. Let context TC_1 and type parameters $typeId_{expl}^*$ be the result of typing typeParameterListOpt, under context TC_0 at LOCAL.
    f. Let $typeIR_{ret}$ and fresh type variables $typeId^*$ be the result of typing typeOrVoid, under context TC_1 at LOCAL.
    g. Check that $typeId^*$ is an empty list.

h. Let TC_2 be TC_1 with LOCAL.KIND set to EXTERN_METHOD ABSTRACT ':' $typeIR_{ret}$.

i. Let context TC_3, parameterIR*, and fresh type variables $typeId_{impl}$* be the result of typing parameterList, under context TC_2 at LOCAL.

j. Let externMethodTypeDefIR be EXTERN_METHOD ABSTRACT nameIR `< $typeId_{expl}$* ',' $typeId_{impl}$* `> `( parameterIR* `) ':' $typeIR_{ret}$.

k. Let bound' be bound type variables from the BLOCK layer of TC_0.

l. If externMethodTypeDefIR is a well-formed callable type definition, with bound type variables bound':

i. Let externMethodPrototypeIR be annotationList ABSTRACT $typeIR_{ret}$ nameIR `< $typeId_{expl}$* ',' $typeId_{impl}$* `> `( parameterIR* `) ';'.

ii. The result is externMethodPrototypeIR.

11.10.1.2. Compile-time evaluation

Extern method declarations are loaded into the context at compile-time by:

ExternMethod_inst: IC_0 |- externMethodPrototypeIR : %

IC_0 |- externMethodPrototypeIR : %:

1. If let annotationList typeIR nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `( parameterListIR `) ';' be externMethodPrototypeIR:
   a. Let callableId be the callable identifier for nameIR `( parameterListIR `).
   b. Let externMethodDef be EXTERN_METHOD nameIR `< typeParameterListIR ++ $typeParameterListIR_{inferred}$ `> `( parameterListIR `) ·.
   c. Let IC_1 be IC_0 where callableId to externMethodDef is added as an overloaded callable to the BLOCK layer.
   d. The result is IC_1.
2. Else:
   a. Let annotationList ABSTRACT typeIR nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `( parameterListIR `) ';' be externMethodPrototypeIR.
   b. Let callableId be the callable identifier for nameIR `( parameterListIR `).
   c. Let externMethodDef be EXTERN_METHOD ABSTRACT nameIR `< typeParameterListIR ++ $typeParameterListIR_{inferred}$ `> `( parameterListIR `).
   d. Let IC_1 be IC_0 where callableId to externMethodDef is added as an overloaded callable to the BLOCK layer.
   e. The result is IC_1.

A list of extern method declarations is loaded into the context by:

ExternMethods_inst:

- Compile-time evaluation of externMethodPrototypeIR*, under context IC:
- The result is context IC.

Compile-time evaluation of externMethodPrototypeIR*, under context IC:

1. If externMethodPrototypeIR* matches pattern []: the result is context IC.
2. Else:
   a. Let $\text{externMethodPrototypeIR}_h$ :: $\text{externMethodPrototypeIR}_t^*$ be externMethodPrototypeIR*.
   b. Let IC |- $\text{externMethodPrototypeIR}_h$: IC_1.
   c. Let context IC_2 be the result of compile-time evaluation of $\text{externMethodPrototypeIR}_t^*$, under context IC_1.
   d. The result is context IC_2.

11.10.2. Extern constructors

Extern objects may have constructors, which are declared using the following syntax:

```
externConstructorPrototype
    : annotationList typeIdentifier ( parameterList ) ;
    ;
```

11.10.2.1. Type checking

Extern constructors are type checked using the following relation:

ExternConstructor_ok:

- Typing externConstructorPrototype, under context TC for an extern object typeId:
- The result is externConstructorPrototypeIR.

After type checking, in P4_{IR}, an extern constructor declaration is represented as:

```
externConstructorPrototypeIR
    : annotationList nameIR < , typeParameterListIR > ( parameterListIR ) ;
    ;
```

These are produced by:

Typing annotationList typeIdentifier `( parameterList `) ';', under context TC_0 for an extern object $\text{typeId}_{\text{extern}}$:

1. Let nameIR be the name of typeIdentifier.
2. Check that nameIR is equal to $\text{typeId}_{\text{extern}}$.
3. Let context TC_1, constructorParameterIR*, and fresh type variables $\text{typeId}_{\text{impl}}^*$ be the result of typing `( parameterList `) under context TC_0.
4. Let typeDefIR'? be the type definition of $_{\text{BARE}}$ $\text{typeId}_{\text{extern}}$ from the GLOBAL layer of TC_0.
5. Check that typeDefIR'? is defined.
6. Let typeDefIR be typeDefIR'?.
7. Check that typeDefIR has type externObjectTypeDefIR.
8. Let externObjectTypeDefIR be typeDefIR.
9. Let ($\text{typeId}_{\text{expl}}^*$, typeParameterIR*) be the type parameters of externObjectTypeDefIR.
10. Check that typeParameterIR* is an empty list.
11. Let $\text{typeIR}_{\text{extern}}$ be the underlying type of externObjectTypeDefIR.
12. Let constructorTypeDefIR be CONSTRUCTOR `< $\text{typeId}_{\text{expl}}^*$ ',' $\text{typeId}_{\text{impl}}^*$ `> `( constructorParameterIR* `) ':' $\text{typeIR}_{\text{extern}}$.
13. Let bound be bound type variables from the BLOCK layer of TC_0.
14. If constructorTypeDefIR is a well-formed constructor type definition, with bound type variables bound:
    a. Let externConstructorPrototypeIR be annotationList nameIR `< ',' $\text{typeId}_{\text{impl}}^*$ `> `( constructorParameterIR* `) ';'.
    b. The result is externConstructorPrototypeIR.

11.10.3. Type checking

After type checking, an extern object declaration is represented in P4_{IR} as:

```
externObjectDeclarationIR
    : annotationList EXTERN nameIR < typeParameterListIR , >
        { externConstructorPrototypeListIR externMethodPrototypeListIR }
    ;

externConstructorPrototypeIR
    : annotationList nameIR < , typeParameterListIR > ( parameterListIR ) ;
    ;

externMethodPrototypeIR
    : annotationList functionPrototypeIR ;
    | annotationList ABSTRACT functionPrototypeIR ;
    ;
```

Typing annotationList EXTERN nonTypeName typeParameterListOpt `{ externConstructorOrMethodPrototypeList `} under context TC_0:

1. Let (externConstructorPrototype*, externMethodPrototype*) be externConstructorOrMethodPrototypeList split into constructors and methods.
2. Let TC_1 be TC_0 with BLOCK.KIND set to EXTERN.
3. Let context TC_2 and type parameters $\text{typeId}_{\text{expl}}^*$ be the result of typing typeParameterListOpt, under context TC_1 at BLOCK.
4. Let nameIR be the name of nonTypeName.

5. For each externMethodPrototype in externMethodPrototype*:

   - Let externMethodPrototypeIR be the result of typing externMethodPrototype, under context TC_2 for an extern object nameIR.

   Let externMethodPrototypeIR* be the resulting list.

6. For each externMethodPrototype in externMethodPrototype*:

   - Let $callableId_{method}$ be callable id of externMethodPrototype.

   Let $callableId_{method}$* be the resulting list.

7. Check that the elements of $callableId_{method}$* are distinct.
8. For each externMethodPrototypeIR in externMethodPrototypeIR*:

   - Let externMethodTypeDefIR be callable type definition of externMethodPrototypeIR.

   Let externMethodTypeDefIR* be the resulting list.

9. Let externMethodTypeDefEnv be `{ ( $callableId_{method}$ ':' externMethodTypeDefIR )* `}.
10. Let $typeDefIR_{extern}$ be EXTERN nameIR `< $typeId_{expl}$* ',' · `> externMethodTypeDefEnv.
11. Let TC_3 be TC_0 where nameIR to $typeDefIR_{extern}$ is added to the GLOBAL layer.
12. Let TC_4 be TC_3 with BLOCK.KIND set to EXTERN.
13. Let TC_5 be type parameters $typeId_{expl}$* added to the BLOCK layer of TC_4.
14. For each externConstructorPrototype in externConstructorPrototype*:

   - Let externConstructorPrototypeIR be the result of typing externConstructorPrototype, under context TC_5 for an extern object nameIR.

   Let externConstructorPrototypeIR* be the resulting list.

15. For each externConstructorPrototype in externConstructorPrototype*:

   - Let constructorId be constructor id of externConstructorPrototype.

   Let constructorId* be the resulting list.

16. Let $typeIR_{extern}$ be the underlying type of $typeDefIR_{extern}$.
17. For each externConstructorPrototypeIR in externConstructorPrototypeIR*:

   - Let constructorTypeDefIR be the constructor type definition of externConstructorPrototypeIR with type parameters $typeId_{expl}$* and type $typeIR_{extern}$.

   Let constructorTypeDefIR* be the resulting list.

18. Let TC_6 be where each of constructorId* to each of constructorTypeDefIR* are added to TC_3.
19. Let externObjectDeclarationIR be annotationList EXTERN nameIR `< $typeId_{expl}$* ',' `> `{ externConstructorPrototypeIR* externMethodPrototypeIR* `}.
20. The result is context TC_6 and externObjectDeclarationIR.

The constructor type definition of a constructor prototype is produced by:

The constructor type definition of _ nameIR `< ',' $typeParameterListIR_{impl}$ `> `( parameterIR* `) ';' with type parameters

typeParameterListIR$_{expl}$ and type typeIR

1. Let constructorTypeDefIR be CONSTRUCTOR `< typeParameterListIR$_{expl}$ ',' typeParameterListIR$_{impl}$ `> `( parameterIR$^*$ `) ':' typeIR.
2. Return constructorTypeDefIR.

11.10.4. Compile-time evaluation

At comile-time, extern object declarations are loaded into the context by:

Loading annotationList EXTERN nameIR `< typeParameterListIR ',' `> `{ externConstructorPrototypeIR$^*$ externMethodPrototypeIR$^*$ `}, under context IC$_0$, and store STO:

1. If externConstructorPrototypeIR* matches pattern []:
   a. For each externMethodPrototypeIR in externMethodPrototypeIR*:
      - Let callableId$_{method}$ be the callable id of externMethodPrototypeIR.

      Let callableId$_{method}^*$ be the resulting list.
   b. For each externMethodPrototypeIR in externMethodPrototypeIR*:
      - Let externMethodTypeDefIR be callable type definition of externMethodPrototypeIR.

      Let externMethodTypeDefIR* be the resulting list.
   c. Let externMethodTypeDefEnv be `{ ( callableId$_{method}$ ':' externMethodTypeDefIR )$^*$ `}.
   d. Let externObjectTypeDefIR be EXTERN nameIR `< typeParameterListIR ',' · `> externMethodTypeDefEnv.
   e. Let IC$_1$ be IC$_0$ where nameIR to externObjectTypeDefIR is added to the GLOBAL layer.
   f. Let callableId$_{constructor}$ be the callable identifier for nameIR `( · `).
   g. Let constructorDef be EXTERN nameIR `< typeParameterListIR `> `( · `) `{ externMethodPrototypeIR$^*$ `}.
   h. Let IC$_2$ be where callableId$_{constructor}$ to constructorDef is added to IC$_1$.
   i. The result is context IC$_2$ and store STO.
2. Else:
   a. For each externMethodPrototypeIR in externMethodPrototypeIR*:
      - Let callableId$_{method}$ be the callable id of externMethodPrototypeIR.

      Let callableId$_{method}^*$ be the resulting list.
   b. For each externMethodPrototypeIR in externMethodPrototypeIR*:
      - Let externMethodTypeDefIR be callable type definition of externMethodPrototypeIR.

      Let externMethodTypeDefIR* be the resulting list.
   c. Let externMethodTypeDefEnv be `{ ( callableId$_{method}$ ':' externMethodTypeDefIR )$^*$ `}.
   d. Let externObjectTypeDefIR be EXTERN nameIR `< typeParameterListIR ',' · `> externMethodTypeDefEnv.
   e. Let IC$_1$ be IC$_0$ where nameIR to externObjectTypeDefIR is added to the GLOBAL layer.
   f. For each externConstructorPrototypeIR in externConstructorPrototypeIR*:

- Let $callableId_{constructor}$ be the constructor id of externConstructorPrototypeIR.

Let $callableId_{constructor}^{*}$ be the resulting list.

g. For each externConstructorPrototypeIR in externConstructorPrototypeIR*:

- Let constructorDef be the constructor definition of externConstructorPrototypeIR with type parameters typeParameterListIR and methods externMethodPrototypeIR*.

Let constructorDef* be the resulting list.

h. Let IC_2 be where each of $callableId_{constructor}^{*}$ to each of constructorDef* are added to IC_1.

i. The result is context IC_2 and store STO.

The constructor of a constructor prototype is produced by:

The constructor definition of _ nameIR `< ',' typeParameterListIR$_{impl}$ `> `( parameterIR* `) ';' with type parameters typeParameterListIR$_{expl}$ and methods externMethodPrototypeListIR

1. Let typeParameterListIR be typeParameterListIR$_{expl}$ concatenated with typeParameterListIR$_{impl}$.
2. Let constructorDef be EXTERN nameIR `< typeParameterListIR `> `( parameterIR* `) `{ externMethodPrototypeListIR `}.
3. Return constructorDef.

11.11. Parser declarations

A parser declaration introduces a programmable parser:

```
parserDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { parserLocalDeclarationList parserStateList }
    ;

parserLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | valueSetDeclaration
    ;

parserState
    : annotationList STATE name
        { parserBlockElementStatementList transitionStatement }
    ;
```

11.11.1. Type checking

After type checking, a parser declaration is represented in $P4_{IR}$ as:

```
<span id="parserDeclarationIR"></span>
<span id="parserLocalDeclarationIR"></span>
<span id="parserStateIR"></span>
```

```
parserDeclarationIR
    : annotationList PARSER nameIR < typeParameterListIR , typeParameterListIR >
        ( parameterListIR ) ( constructorParameterListIR )
```

```
        { parserLocalDeclarationListIR parserStateListIR }
    ;

parserLocalDeclarationIR
    : constantDeclarationIR | instantiationIR | variableDeclarationIR
    | valueSetDeclarationIR
    ;

parserStateIR
    : annotationList STATE nameIR
        { parserBlockElementStatementListIR transitionStatementIR }
    ;
```

Typing annotationList PARSER name typeParameterListOpt `( parameterList `) constructorParameterListOpt `{ parserLocalDeclarationList parserStateList `} under context TC_0:

1. Let TC_1 be TC_0 with BLOCK.KIND set to PARSER.
2. Check that typeParameterListOpt is $_{EMPTY}$.
3. Let context TC_2, constructorParameterIR*, and fresh type variables typeId* be the result of typing constructorParameterListOpt under context TC_1.
4. Check that typeId* is an empty list.
5. Let context TC_3, parameterIR*, and fresh type variables typeId'* be the result of typing parameterList, under context TC_2 at BLOCK.
6. Check that typeId'* is an empty list.
7. Let context TC_4 and parserLocalDeclarationIR* be the result of typing parserLocalDeclarationList, under context TC_3.
8. Let TC_5 be TC_4 with LOCAL.KIND set to PARSER_STATE.
9. Let parserStateIR* be the result of typing parserStateList, under context TC_5.
10. Let callableTypeDefIR be PARSER_APPLY `( parameterIR* `).
11. Let bound'' be bound type variables from the BLOCK layer of TC_1.
12. If callableTypeDefIR is a well-formed callable type definition, with bound type variables bound'':
    a. Let nameIR be the name of name.
    b. Let constructorId be the constructor identifier for name constructorParameterListOpt.
    c. Let $typeDefIR_{parser}$ be PARSER nameIR `< · ',' · `> `( parameterIR* `).
    d. Let $typeIR_{parser}$ be the underlying type of $typeDefIR_{parser}$.
    e. Let constructorTypeDefIR be CONSTRUCTOR `< · ',' · `> `( constructorParameterIR* `) ':' $typeIR_{parser}$.
    f. Let bound' be bound type variables from the GLOBAL layer of TC_0.
    g. If constructorTypeDefIR is a well-formed constructor type definition, with bound type variables bound':
        i. Let TC_6 be where constructorId to constructorTypeDefIR is added to TC_0.
        ii. Let parserDeclarationIR be annotationList PARSER nameIR `< · ',' · `> `( parameterIR* `) `( constructorParameterIR* `) `{ parserLocalDeclarationIR* parserStateIR* `}.
        iii. The result is context TC_6 and parserDeclarationIR.

11.11.2. Compile-time evaluation

At compile-time, parser declarations are loaded into the context as parser object constructors.

Loading annotationList PARSER nameIR `< typeParameterListIR$_{expl}$ ',' typeParameterListIR$_{impl}$ `> `( parameterListIR `) `( constructorParameterListIR `) `{ parserLocalDeclarationListIR parserStateListIR `}, under context IC_0, and store STO:

1. Let callableId be the callable identifier for nameIR `( constructorParameterListIR `).
2. Let typeParameterListIR be typeParameterListIR$_{expl}$ concatenated with typeParameterListIR$_{impl}$.
3. Let constructorDef be PARSER `< typeParameterListIR `> `( parameterListIR `) `( constructorParameterListIR `) `{ parserLocalDeclarationListIR parserStateListIR `}.
4. Let IC_1 be where callableId to constructorDef is added to IC_0.
5. The result is context IC_1 and store STO.

When instantiated, an instance of a parser declaration is created:

```
parserObject
    : PARSER < theta > ( parameterListIR )
        { frame parserLocalDeclarationListIR stateEnv }
    ;
```

See Section 10.4 for details about parser objects.

11.12. Control declarations

A control declaration introduces a control block:

```
controlDeclaration
    : annotationList CONTROL name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { controlLocalDeclarationList APPLY controlBody }
    ;

controlLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | actionDeclaration | tableDeclaration
    ;

controlBody = blockStatement
```

11.12.1. Type checking

After type checking, a control declaration is represented in $P4_{IR}$ as:

```
controlDeclarationIR
    : annotationList CONTROL nameIR
        < typeParameterListIR , typeParameterListIR > ( parameterListIR )
        ( constructorParameterListIR )
        { controlLocalDeclarationListIR APPLY controlBodyIR }
    ;

controlLocalDeclarationIR
    : constantDeclarationIR | instantiationIR | variableDeclarationIR
```

```
    | actionDeclarationIR | tableDeclarationIR
    ;

controlBodyIR = blockStatementIR
```

Typing annotationList CONTROL name typeParameterListOpt `( parameterList `) constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody `} under context TC_0:

1. Let TC_1 be TC_0 with BLOCK.KIND set to CONTROL.
2. Check that typeParameterListOpt is $_{EMPTY}$.
3. Let context TC_2, constructorParameterIR*, and fresh type variables typeId* be the result of typing constructorParameterListOpt under context TC_1.
4. Check that typeId* is an empty list.
5. Let context TC_3, parameterIR*, and fresh type variables typeId'* be the result of typing parameterList, under context TC_2 at BLOCK.
6. Check that typeId'* is an empty list.
7. Let context TC_4 and controlLocalDeclarationIR* be the result of typing controlLocalDeclarationList, under context TC_3.
8. Let TC_5 be TC_4 with LOCAL.KIND set to CONTROL_APPLY_METHOD.
9. Let TC_6 be TC_5[CONTROL = CONT] with LOOP set to OUT.
10. Let context _ and controlBodyIR be the result of typing controlBody under context TC_6.
11. Let callableTypeDefIR be CONTROL_APPLY `( parameterIR* `).
12. Let bound'''' be bound type variables from the BLOCK layer of TC_1.
13. If callableTypeDefIR is a well-formed callable type definition, with bound type variables bound'''':
    a. Let nameIR be the name of name.
    b. Let constructorId be the constructor identifier for name constructorParameterListOpt.
    c. Let $typeDefIR_{control}$ be CONTROL nameIR `< · ',' · `> `( parameterIR* `).
    d. Let $typeIR_{control}$ be the underlying type of $typeDefIR_{control}$.
    e. Let constructorTypeDefIR be CONSTRUCTOR `< · ',' · `> `( constructorParameterIR* `) ':' $typeIR_{control}$.
    f. Let bound''' be bound type variables from the GLOBAL layer of TC_0.
    g. If constructorTypeDefIR is a well-formed constructor type definition, with bound type variables bound''':
        i. Let TC_7 be where constructorId to constructorTypeDefIR is added to TC_0.
        ii. Let controlDeclarationIR be annotationList CONTROL nameIR `< · ',' · `> `( parameterIR* `) `( constructorParameterIR* `) `{ controlLocalDeclarationIR* APPLY controlBodyIR `}.
        iii. The result is context TC_7 and controlDeclarationIR.

11.12.2. Compile-time evaluation

At compile-time, control declarations are loaded into the context as control object constructors.

Loading annotationList CONTROL nameIR `< $typeParameterListIR_{expl}$ ',' $typeParameterListIR_{impl}$ `> `( parameterListIR `) `( constructorParameterListIR `) `{ controlLocalDeclarationListIR APPLY controlBodyIR `}, under context IC_0, and store STO:

1. Let callableId be the callable identifier for nameIR `( constructorParameterListIR `).
2. Let typeParameterListIR be $\text{typeParameterListIR}_{expl}$ concatenated with $\text{typeParameterListIR}_{impl}$.
3. Let constructorDef be CONTROL `< typeParameterListIR `> `( parameterListIR `) `( constructorParameterListIR `) `{ controlLocalDeclarationListIR APPLY controlBodyIR `}.
4. Let IC_1 be where callableId to constructorDef is added to IC_0.
5. The result is context IC_1 and store STO.

When instantiated, an instance of a control declaration is created:

```
controlObject
    : CONTROL < theta > ( parameterListIR )
        { frame controlLocalDeclarationListIR actionDefEnv controlBodyIR }
    ;
```

See Section 10.5 for details about control objects.

11.13. Enum type declaration

An enumeration type is defined using the following syntax:

```
enumTypeDeclaration
    : annotationList ENUM name { nameList trailingCommaOpt }
    | annotationList ENUM type name { namedExpressionList trailingCommaOpt }
    ;

namedExpressionList
    : namedExpression | namedExpressionList , namedExpression
    ;

namedExpression
    : name = expression
    ;
```

After type checking, enum type declarations are represented as follows:

```
<span id="enumTypeDeclarationIR"></span>
<span id="namedValueListIR"></span>
<span id="namedValueIR"></span>
```

```
enumTypeDeclarationIR
    : annotationList ENUM nameIR { nameListIR }
    | annotationList ENUM typeIR nameIR { namedValueListIR }
    ;

namedValueListIR = namedValueIR*

namedValueIR
    : nameIR = value
    ;
```

See Section 8.4.8 for more details on enums.

11.13.1. Enum type declarations without an underyling type

11.13.1.1. Type checking

Typing annotationList ENUM name `{ $nameList_{field}$ _ `} under context TC_0:

1. Let nameIR be the name of name.
2. Let $name_{field}^*$ be $nameList_{field}$ flattened.
3. For each $name_{field}$ in $name_{field}^*$:

   - Let $nameIR_{field}$ be the name of $name_{field}$.

   Let $nameIR_{field}^*$ be the resulting list.

4. Let enumTypeDefIR be ENUM nameIR `{ $nameIR_{field}^*$ `}.
5. Let bound'''' be bound type variables from the GLOBAL layer of TC_0.
6. If enumTypeDefIR is a well-formed type definition, with bound type variables bound'''':
   a. Let TC_1 be TC_0 where nameIR to enumTypeDefIR is added to the GLOBAL layer.
   b. For each $nameIR_{field}$ in $nameIR_{field}^*$:

      - Let id_{field} be nameIR concatenated with "." concatenated with $nameIR_{field}$.

      Let id_{field}^* be the resulting list.

   c. For each $nameIR_{field}$ in $nameIR_{field}^*$:

      - Let $value_{field}$ be nameIR '.' $nameIR_{field}$.

      Let $value_{field}^*$ be the resulting list.

   d. For each $value_{field}$ in $value_{field}^*$:

      - Let varTypeIR be $_{EMPTY}$ enumTypeDefIR LCTK $value_{field}$.

      Let $varTypeIR^*$ be the resulting list.

   e. Let TC_2 be TC_1 where each of id_{field}^* to each of $varTypeIR^*$ are added to the GLOBAL layer.
   f. Let enumTypeDeclarationIR be annotationList ENUM nameIR `{ $nameIR_{field}^*$ `}.
   g. The result is context TC_2 and enumTypeDeclarationIR.

11.13.1.2. Compile-time evaluation

At compile-time, enum type declarations are loaded into the context by:

Loading annotationList ENUM nameIR `{ $nameIR_{field}^*$ `}, under context IC_0, and store STO:

1. For each $nameIR_{field}$ in $nameIR_{field}^*$:

   - Let $nameIR_{enum_field}$ be nameIR concatenated with "." concatenated with $nameIR_{field}$.

Let nameIR$_{enum_field}$* be the resulting list.

2. For each nameIR$_{field}$ in nameIR$_{field}$*:

   - Let value$_{enum_field}$ be nameIR '.' nameIR$_{field}$.

   Let value$_{enum_field}$* be the resulting list.

3. Let IC$_1$ be IC$_0$ where each of nameIR$_{enum_field}$* to each of value$_{enum_field}$* are added to the GLOBAL layer.
4. Let enumTypeDefIR be ENUM nameIR `{ nameIR$_{field}$* `}.
5. Let IC$_2$ be IC$_1$ where nameIR to enumTypeDefIR is added to the GLOBAL layer.
6. The result is context IC$_2$ and store STO.

11.13.2. Enum type declarations with an underlying type

11.13.2.1. Type checking


```
<span id="Decl_ok-enumTypeDeclaration-serializable"></span>
```


Typing annotationList ENUM type name `{ namedExpressionList$_{field}$ _ `} under context TC$_0$:

1. Let B be bound type variables from the GLOBAL layer of TC$_0$.
2. Let typeIR and fresh type variables typeId* be the result of typing type, under context TC$_0$ at GLOBAL.
3. Check that typeId* is an empty list.
4. If typeIR is a well-formed type, with bound type variables B:
   a. Let nameIR be the name of name.
   b. Let context TC$_1$ and field values namedValueIR$_{field}$* be the result of typing namedExpressionList$_{field}$ under typing context TC$_0$ with enum name nameIR and underlying typeIR.
   c. For each namedValueIR$_{field}$ in namedValueIR$_{field}$*:

      - Let nameIR$_{field}$ '=' value$_{field}$ be namedValueIR$_{field}$.

      Let nameIR$_{field}$* and value$_{field}$* be the resulting lists.

   d. For each nameIR$_{field}$ in nameIR$_{field}$*:

      - Let id$_{field}$ be nameIR concatenated with "." concatenated with nameIR$_{field}$.

      Let id$_{field}$* be the resulting list.

   e. Let enumTypeDefIR be ENUM nameIR `< typeIR `> `{ ( nameIR$_{field}$ '=' value$_{field}$ ';' )* `}.
   f. For each value$_{field}$ in value$_{field}$*:

      - Let varTypeIR be $_{EMPTY}$ enumTypeDefIR LCTK value$_{field}$.

      Let varTypeIR* be the resulting list.

   g. Let TC$_2$ be TC$_0$ where each of id$_{field}$* to each of varTypeIR* are added to the GLOBAL layer.
   h. If enumTypeDefIR is a well-formed type definition, with bound type variables B:
      i. Let TC$_3$ be TC$_2$ where nameIR to enumTypeDefIR is added to the GLOBAL layer.
      ii. Let enumTypeDeclarationIR be annotationList ENUM typeIR nameIR `{ namedValueIR$_{field}$* `}.

iii. The result is context TC_3 and enumTypeDeclarationIR.

An enum field is type-checked as follows:

Enum_serializable_field_ok:

- Typing namedExpression under typing context TC with enum name nameIR and underlying typeIR:
- The result is context TC and field value namedValueIR.

Typing name '=' expression under typing context TC_0 with enum name $nameIR_{enum}$ and underlying typeIR:

1. Let typedExpressionIR be the result of typing expression, under context TC_0 at BLOCK.
2. Let $typedExpressionIR'^?$ be typedExpressionIR implicitly cast to typeIR.
3. Check that $typedExpressionIR'^?$ is defined.
4. Let $typedExpressionIR_{cast}$ be $typedExpressionIR'^?$.
5. Check that the compile-time known-ness of $typedExpressionIR_{cast}$ is LCTK.
6. Let value be the result of local compile-time evaluation of $typedExpressionIR_{cast}$, under context TC_0 at BLOCK.
7. Let nameIR be the name of name.
8. Let varTypeIR be $_{EMPTY}$ typeIR LCTK value.
9. Let TC_1 be TC_0 where nameIR to varTypeIR is added to the BLOCK layer.
10. The result is context TC_1 and field value nameIR '=' value.

A list of enum fields is type-checked with the following relation:

Enum_serializable_fieldList_ok:

- Typing namedExpressionList under typing context TC with enum name nameIR and underlying typeIR:
- The result is context TC and field values $namedValueIR^*$.

Typing namedExpressionList under typing context TC_0 with enum name $nameIR_{enum}$ and underlying typeIR:

1. Let $namedExpression^*$ be namedExpressionList flattened.
2. Let TC_0 $nameIR_{enum}$ typeIR |- $namedExpression^*$: TC_1 $namedValueIR^*$.
3. The result is context TC_1 and field values $namedValueIR^*$.

Enum_serializable_fields_ok: TC nameIR$_{enum}$ typeIR |- namedExpression* : % %

TC nameIR$_{enum}$ typeIR |- namedExpression* : % %:

1. If namedExpression* matches pattern []: the result is TC and ·.
2. Else:
   a. Let namedExpression$_{h}$ :: namedExpression$_{t}^{*}$ be namedExpression*.
   b. Let context TC$_{1}$ and field value namedValueIR$_{h}$ be the result of typing namedExpression$_{h}$ under typing context TC with enum name nameIR$_{enum}$ and underlying typeIR.
   c. Let TC$_{1}$ nameIR$_{enum}$ typeIR |- namedExpression$_{t}^{*}$: TC$_{2}$ namedValueIR$_{t}^{*}$.
   d. Let namedValueIR* be namedValueIR$_{h}$ :: namedValueIR$_{t}^{*}$.
   e. The result is TC$_{2}$ and namedValueIR*.

11.13.2.2. Compile-time evaluation

At compile-time, enum type declarations are loaded into the context by:

Loading annotationList ENUM typeIR nameIR `{ ( nameIR$_{field}$ '=' value$_{field}$ )$^{*}$ `}, under context IC$_{0}$, and store STO:

1. For each nameIR$_{field}$ in nameIR$_{field}^{*}$:
   - Let nameIR$_{serenum_field}$ be nameIR concatenated with "." concatenated with nameIR$_{field}$.

   Let nameIR$_{serenum_field}^{*}$ be the resulting list.
2. For each nameIR$_{field}$ in nameIR$_{field}^{*}$ and value$_{field}$ in value$_{field}^{*}$:
   - Let value$_{serenum_field}$ be nameIR '.' nameIR$_{field}$ '.' value$_{field}$.

   Let value$_{serenum_field}^{*}$ be the resulting list.
3. Let IC$_{1}$ be IC$_{0}$ where each of nameIR$_{serenum_field}^{*}$ to each of value$_{serenum_field}^{*}$ are added to the GLOBAL layer.
4. Let enumTypeDefIR be ENUM nameIR `< typeIR `> `{ ( nameIR$_{field}$ '=' value$_{field}$ ';' )$^{*}$ `}.
5. Let IC$_{2}$ be IC$_{1}$ where nameIR to enumTypeDefIR is added to the GLOBAL layer.
6. The result is context IC$_{2}$ and store STO.

11.14. Struct type declaration

A struct type is defined with the following syntax:

```
structTypeDeclaration
    : annotationList STRUCT name typeParameterListOpt { typeFieldList }
    ;

typeFieldList
    : /* empty */ | typeFieldList typeField
    ;

typeField
    : annotationList type name ;
    ;
```

See Section 8.4.5 for more information about struct types.

11.14.1. Type checking

After type checking, a struct type declaration is represented in $P4_{IR}$ as:

```
structTypeDeclarationIR
    : annotationList STRUCT nameIR < typeParameterListIR > { typeFieldListIR }
    ;

typeFieldListIR = typeFieldIR*

typeFieldIR
    : annotationList typeIR nameIR ;
    ;
```

Typing annotationList STRUCT name typeParameterListOpt `{ typeFieldList `} under context TC_0:

1. Let context TC_1 and type parameters $typeId^*$ be the result of typing typeParameterListOpt, under context TC_0 at BLOCK.
2. Let $(annotationList_{field}\ type_{field}\ name_{field}\ ';')^*$ be typeFieldList flattened.
3. For each $type_{field}$ in $type_{field}^*$:
   - Let $typeIR_{field}$ and fresh type variables $typeId'^*$ be the result of typing $type_{field}$, under context TC_1 at BLOCK.

   Let $typeIR_{field}^*$ and $typeId'^{**}$ be the resulting lists.
4. Check that $typeId'^*$ is an empty list, for all $typeId'^*$ in $typeId'^{**}$.
5. Let nameIR be the name of name.
6. For each $name_{field}$ in $name_{field}^*$:
   - Let $nameIR_{field}$ be the name of $name_{field}$.

   Let $nameIR_{field}^*$ be the resulting list.
7. Let structTypeDefIR be $STRUCT\ nameIR\ `<\ typeId^*\ `>\ `\{\ ( annotationList_{field}\ typeIR_{field}\ nameIR_{field}\ ';' )^*\ `\}$.
8. Let $bound''''''$ be bound type variables from the GLOBAL layer of TC_0.
9. If structTypeDefIR is a well-formed type definition, with bound type variables $bound''''''$:

a. Let TC_2 be TC_0 where nameIR to structTypeDefIR is added to the GLOBAL layer.

b. Let structTypeDeclarationIR be annotationList STRUCT nameIR `< typeId* `> `{ ( $annotationList_{field}$ $typeIR_{field}$ $nameIR_{field}$ ';' )* `}.

c. The result is context TC_2 and structTypeDeclarationIR.

11.14.2. Compile-time evaluation

At compile-time, struct type declarations are loaded into the context by:

Loading annotationList STRUCT nameIR `< typeParameterListIR `> `{ typeFieldIR* `}, under context IC_0, and store STO:

1. For each typeFieldIR in typeFieldIR*:
   - Let $annotationList_{field}$ $typeIR_{field}$ $nameIR_{field}$ ';' be typeFieldIR.

   Let $annotationList_{field}$*, $nameIR_{field}$*, and $typeIR_{field}$* be the resulting lists.
2. Let structTypeDefIR be STRUCT nameIR `< typeParameterListIR `> `{ ( $annotationList_{field}$ $typeIR_{field}$ $nameIR_{field}$ ';' )* `}.
3. Let IC_1 be IC_0 where nameIR to structTypeDefIR is added to the GLOBAL layer.
4. The result is context IC_1 and store STO.

11.15. Header type declaration

A header type is defined with the following syntax:

```
headerTypeDeclaration
    : annotationList HEADER name typeParameterListOpt { typeFieldList }
    ;

typeFieldList
    : /* empty */ | typeFieldList typeField
    ;

typeField
    : annotationList type name ;
    ;
```

See Section 8.4.6 for more information about header types.

11.15.1. Type checking

After type checking, a header type declaration is represented in $P4_{IR}$ as:

```
headerTypeDeclarationIR
    : annotationList HEADER nameIR < typeParameterListIR > { typeFieldListIR }
    ;

typeFieldListIR = typeFieldIR*

typeFieldIR
```

```
    : annotationList typeIR nameIR ;
    ;
```

Typing annotationList HEADER name typeParameterListOpt `{ typeFieldList `} under context TC_0:

1. Let context TC_1 and type parameters $typeId^*$ be the result of typing typeParameterListOpt, under context TC_0 at BLOCK.
2. Let $(annotationList_f\ type_f\ name_f\ ';')^*$ be typeFieldList flattened.
3. For each $type_f$ in $type_f^*$:

   - Let $typeIR_f$ and fresh type variables $typeId'^*$ be the result of typing $type_f$, under context TC_1 at BLOCK.

   Let $typeIR_f^*$ and $typeId'^{**}$ be the resulting lists.
4. Check that $typeId'^*$ is an empty list, for all $typeId'^*$ in $typeId'^{**}$.
5. Let nameIR be the name of name.
6. For each $name_f$ in $name_f^*$:

   - Let $nameIR_f$ be the name of $name_f$.

   Let $nameIR_f^*$ be the resulting list.
7. Let headerTypeDefIR be HEADER nameIR `< $typeId^*$ `> `{ $( annotationList_f\ typeIR_f\ nameIR_f\ ';' )^*$ `}.
8. Let bound'''''' be bound type variables from the GLOBAL layer of TC_0.
9. If headerTypeDefIR is a well-formed type definition, with bound type variables bound'''''':
   a. Let TC_2 be TC_0 where nameIR to headerTypeDefIR is added to the GLOBAL layer.
   b. Let headerTypeDeclarationIR be annotationList HEADER nameIR `< $typeId^*$ `> `{ $( annotationList_f\ typeIR_f\ nameIR_f\ ';' )^*$ `}.
   c. The result is context TC_2 and headerTypeDeclarationIR.

11.15.2. Compile-time evaluation

At compile-time, header type declarations are loaded into the context by:

Loading annotationList HEADER nameIR `< typeParameterListIR `> `{ $typeFieldIR^*$ `}, under context IC_0, and store STO:

1. For each typeFieldIR in $typeFieldIR^*$:

   - Let $annotationList_{field}\ typeIR_{field}\ nameIR_{field}$ ';' be typeFieldIR.

   Let $annotationList_{field}^*$, $nameIR_{field}^*$, and $typeIR_{field}^*$ be the resulting lists.
2. Let headerTypeDefIR be HEADER nameIR `< typeParameterListIR `> `{ $( annotationList_{field}\ typeIR_{field}\ nameIR_{field}\ ';' )^*$ `}.
3. Let IC_1 be IC_0 where nameIR to headerTypeDefIR is added to the GLOBAL layer.
4. The result is context IC_1 and store STO.

11.16. Header union type declaration

A header union type is defined with the following syntax:

```
headerUnionTypeDeclaration
    : annotationList HEADER_UNION name typeParameterListOpt { typeFieldList }
    ;

typeFieldList
    : /* empty */ | typeFieldList typeField
    ;

typeField
    : annotationList type name ;
    ;
```

See Section 8.4.7 for more information about header union types.

11.16.1. Type checking

After type checking, a header union type declaration is represented in $P4_{IR}$ as:

```
headerUnionTypeDeclarationIR
    : annotationList HEADER_UNION nameIR < typeParameterListIR >
        { typeFieldListIR }
    ;

typeFieldListIR = typeFieldIR*

typeFieldIR
    : annotationList typeIR nameIR ;
    ;
```

Typing annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList `} under context TC_0:

1. Let context TC_1 and type parameters $typeId^*$ be the result of typing typeParameterListOpt, under context TC_0 at BLOCK.
2. Let $(annotationList_f\ type_f\ name_f\ ';')^*$ be typeFieldList flattened.
3. For each $type_f$ in $type_f^*$:

   - Let $typeIR_f$ and fresh type variables $typeId'^*$ be the result of typing $type_f$, under context TC_1 at BLOCK.

   Let $typeIR_f^*$ and $typeId'^{**}$ be the resulting lists.
4. Check that $typeId'^*$ is an empty list, for all $typeId'^*$ in $typeId'^{**}$.
5. Let nameIR be the name of name.
6. For each $name_f$ in $name_f^*$:

   - Let $nameIR_f$ be the name of $name_f$.

   Let $nameIR_f^*$ be the resulting list.

7. Let headerUnionTypeDefIR be HEADER_UNION nameIR `< typeId* `> `{ ( annotationList$_f$ typeIR$_f$ nameIR$_f$ ';' )* `}.
8. Let bound""""" be bound type variables from the GLOBAL layer of TC_0.
9. If headerUnionTypeDefIR is a well-formed type definition, with bound type variables bound""""":
   a. Let TC_2 be TC_0 where nameIR to headerUnionTypeDefIR is added to the GLOBAL layer.
   b. Let headerUnionTypeDeclarationIR be annotationList HEADER_UNION nameIR `< typeId* `> `{ ( annotationList$_f$ typeIR$_f$ nameIR$_f$ ';' )* `}.
   c. The result is context TC_2 and headerUnionTypeDeclarationIR.

11.16.2. Compile-time evaluation

At compile-time, header union type declarations are loaded into the context by:

Loading annotationList HEADER_UNION nameIR `< typeParameterListIR `> `{ typeFieldIR* `}, under context IC_0, and store STO:

1. For each typeFieldIR in typeFieldIR*:
   - Let annotationList$_{field}$ typeIR$_{field}$ nameIR$_{field}$ ';' be typeFieldIR.

   Let annotationList$_{field}$*, nameIR$_{field}$*, and typeIR$_{field}$* be the resulting lists.
2. Let headerUnionTypeDefIR be HEADER_UNION nameIR `< typeParameterListIR `> `{ ( annotationList$_{field}$ typeIR$_{field}$ nameIR$_{field}$ ';' )* `}.
3. Let IC_1 be IC_0 where nameIR to headerUnionTypeDefIR is added to the GLOBAL layer.
4. The result is context IC_1 and store STO.

11.17. Typedef declaration

A typedef or type declaration introduces an alias for an existing type.

```
typedef
    : type | derivedTypeDeclaration
    ;

typedefDeclaration
    : annotationList TYPEDEF typedef name ; | annotationList TYPE type name ;
    ;
```

In $P4_{IR}$, typedef and type declarations are represented as:

```
typedefIR
    : typeIR | derivedTypeDeclarationIR
    ;

typedefDeclarationIR
    : annotationList TYPEDEF typedefIR nameIR ;
    | annotationList TYPE typeIR nameIR ;
```

```
    ;
```

11.17.1. `typedef` declaration

See Section 8.5.7 for more details about typedef aliases.

11.17.1.1. Type checking

Typing annotationList TYPEDEF typedef name ';' under context TC_0:

1. If let type of type type be typedef:
    a. Let B be bound type variables from the GLOBAL layer of TC_0.
    b. Let typeIR and fresh type variables $typeId^*$ be the result of typing type, under context TC_0 at GLOBAL.
    c. Check that $typeId^*$ is an empty list.
    d. If typeIR is a well-formed type, with bound type variables B:
        i. Let nameIR be the name of name.
        ii. Let $typeDefIR_{typedef}$ be TYPEDEF nameIR typeIR.
        iii. If $typeDefIR_{typedef}$ is a well-formed type definition, with bound type variables B:
            A. Let TC_1 be TC_0 where nameIR to $typeDefIR_{typedef}$ is added to the GLOBAL layer.
            B. Let typedefDeclarationIR be annotationList TYPEDEF typeIR nameIR ';'.
            C. The result is context TC_1 and typedefDeclarationIR.
2. Else:
    a. Let derivedTypeDeclaration be typedef.
    b. Let context TC_1 and declarationIR be the result of typing derivedTypeDeclaration under context TC_0.
    c. Check that declarationIR has type derivedTypeDeclarationIR.
    d. Let derivedTypeDeclarationIR be declarationIR.
    e. Let set<typeId> be the domain of the map TC_1.GLOBAL.TYPE.
    f. Let set<typeId>' be the domain of the map TC_0.GLOBAL.TYPE.
    g. Let `{ $typeId'^*$ `} be the difference of the sets set<typeId> and set<typeId>'.
    h. Check that $typeId'^*$ is a list of length 1.
    i. Let typeId be $typeId'^*$.
    j. Let $typeDefIR'^?$ be the type definition of $_{BARE}$ typeId from the GLOBAL layer of TC_1.
    k. Check that $typeDefIR'^?$ is defined.
    l. Let typeDefIR be $typeDefIR'^?$.
    m. If typeDefIR is monomorphic:
        i. Let typeIR be the underlying type of typeDefIR.
        ii. Let nameIR be the name of name.
        iii. Let typedefTypeIR be TYPEDEF nameIR typeIR.
        iv. Let bound''''''''' be bound type variables from the GLOBAL layer of TC_0.
        v. If typedefTypeIR is a well-formed type definition, with bound type variables bound''''''':
            A. Let TC_2 be TC_1 where nameIR to typedefTypeIR is added to the GLOBAL layer.

B. Let typedefDeclarationIR be annotationList TYPEDEF derivedTypeDeclarationIR nameIR ';'.

C. The result is context TC_2 and typedefDeclarationIR.

n. Else:

i. Check that (·, ·) is equal to the type parameters of typeDefIR.

ii. Let nameIR be the name of name.

iii. Let typeIR be typeDefIR specialized by ·.

iv. Let typedefTypeIR be TYPEDEF nameIR typeIR.

v. Let bound""""""" be bound type variables from the GLOBAL layer of TC_0.

vi. If typedefTypeIR is a well-formed type definition, with bound type variables bound""""""":

A. Let TC_2 be TC_1 where nameIR to typedefTypeIR is added to the GLOBAL layer.

B. Let typedefDeclarationIR be annotationList TYPEDEF derivedTypeDeclarationIR nameIR ';'.

C. The result is context TC_2 and typedefDeclarationIR.

11.17.1.2. Compile-time evaluation

At compile-time, typedefs are loaded into the context with:

Loading annotationList TYPEDEF typedefIR nameIR ';', under context IC_0, and store STO:

1. If let typeIR of type typeIR be typedefIR:
   a. Let aliasTypeDefIR be TYPEDEF nameIR typeIR.
   b. Let IC_1 be IC_0 where nameIR to aliasTypeDefIR is added to the GLOBAL layer.
   c. The result is context IC_1 and store STO.
2. Else:
   a. Let derivedTypeDeclarationIR be typedefIR.
   b. Let context IC_1 and store _ be the result of loading derivedTypeDeclarationIR, under context IC_0, and store an empty store.
   c. Let set<typeId> be the domain of the map IC_1.GLOBAL.TYPE.
   d. Let set<typeId>' be the domain of the map IC_0.GLOBAL.TYPE.
   e. Let `{ typeId* `} be the difference of the sets set<typeId> and set<typeId>'.
   f. Check that $typeId^*$ is a list of length 1.
   g. Let $typeId_{newtype}$ be $typeId^*$.
   h. Let $typeDefIR'^?$ be the type definition of '.' $typeId_{newtype}$ from the GLOBAL layer of IC_1.
   i. Check that $typeDefIR'^?$ is defined.
   j. Let typeDefIR be $typeDefIR'^?$.
   k. If typeDefIR is monomorphic:
      i. Let typeIR be the underlying type of typeDefIR.
      ii. Let aliasTypeDefIR be TYPEDEF nameIR typeIR.
      iii. Let IC_2 be IC_1 where nameIR to aliasTypeDefIR is added to the GLOBAL layer.
      iv. The result is context IC_2 and store STO.
   l. Else:

i. Check that (·, ·) is equal to the type parameters of typeDefIR.

ii. Let typeIR be typeDefIR specialized by ·.

iii. Let typedefTypeIR be TYPEDEF nameIR typeIR.

iv. Let IC_2 be IC_1 where nameIR to typedefTypeIR is added to the GLOBAL layer.

v. The result is context IC_2 and store STO.

11.17.2. New type declaration

See Section 8.5.8 for more details about new type aliases.

11.17.2.1. Type checking

Typing annotationList TYPE type name ';' under context TC_0:

1. Let B be bound type variables from the GLOBAL layer of TC_0.
2. Let typeIR and fresh type variables typeId* be the result of typing type, under context TC_0 at GLOBAL.
3. Check that typeId* is an empty list.
4. If typeIR is a well-formed type, with bound type variables B:
   a. Let nameIR be the name of name.
   b. Let newTypeIR be TYPE nameIR typeIR.
   c. If newTypeIR is a well-formed type definition, with bound type variables B:
      i. Let TC_1 be TC_0 where nameIR to newTypeIR is added to the GLOBAL layer.
      ii. Let typedefDeclarationIR be annotationList TYPE typeIR nameIR ';'.
      iii. The result is context TC_1 and typedefDeclarationIR.

11.17.2.2. Compile-time evaluation

At compile-time, types are loaded into the context with:

Loading annotationList TYPE typeIR nameIR ';', under context IC_0, and store STO:

1. Let aliasTypeDefIR be TYPE nameIR typeIR.
2. Let IC_1 be IC_0 where nameIR to aliasTypeDefIR is added to the GLOBAL layer.
3. The result is context IC_1 and store STO.

11.18. Parser type declaration

A parser type declaration introduces a new parser type.

```
parserTypeDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList ) ;
```

```
    ;
```

See Section 8.5.2 for more details on parser object types.

11.18.1. Type checking

After type checking, a parser type declaration has the following form:

```
parserTypeDeclarationIR
    : annotationList PARSER nameIR < typeParameterListIR , typeParameterListIR >
        ( parameterListIR ) ;
    ;
```

Typing annotationList PARSER name typeParameterListOpt `( parameterList `) ';' under context TC_0:

1. Let TC_1 be TC_0 with BLOCK.KIND set to PARSER.
2. Let context TC_2 and type parameters $typeId_{expl}^*$ be the result of typing typeParameterListOpt, under context TC_1 at BLOCK.
3. Let context TC_3, $parameterIR^*$, and fresh type variables $typeId_{impl}^*$ be the result of typing parameterList, under context TC_2 at BLOCK.
4. Let nameIR be the name of name.
5. Let parserObjectTypeDefIR be PARSER nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `).
6. Let bound'''''''''' be bound type variables from the GLOBAL layer of TC_0.
7. If parserObjectTypeDefIR is a well-formed type definition, with bound type variables bound'''''''''':
   a. Let TC_4 be TC_0 where nameIR to parserObjectTypeDefIR is added to the GLOBAL layer.
   b. Let parserTypeDeclarationIR be annotationList PARSER nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ';'.
   c. The result is context TC_4 and parserTypeDeclarationIR.

11.18.2. Compile-time evaluation

At compile-time, a parser type declaration is loaded into the context with:

Loading annotationList PARSER nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `( $parameterIR^*$ `) ';', under context IC_0, and store STO:

1. Let parserObjectTypeDefIR be PARSER nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `( $parameterIR^*$ `).
2. Let IC_1 be IC_0 where nameIR to parserObjectTypeDefIR is added to the GLOBAL layer.
3. The result is context IC_1 and store STO.

11.19. Control type declaration

A control type declaration introduces a new control type.

```
controlTypeDeclaration
    : annotationList CONTROL name typeParameterListOpt ( parameterList ) ;
    ;
```

See Section 8.5.3 for more details on control object types.

11.19.1. Type checking

After type checking, a control type declaration has the following form:

```
controlTypeDeclarationIR
    : annotationList CONTROL nameIR
        < typeParameterListIR , typeParameterListIR > ( parameterListIR ) ;
    ;
```

Typing annotationList CONTROL name typeParameterListOpt `( parameterList `) ';' under context TC_0:

1. Let TC_1 be TC_0 with BLOCK.KIND set to CONTROL.
2. Let context TC_2 and type parameters $typeId_{expl}^*$ be the result of typing typeParameterListOpt, under context TC_1 at BLOCK.
3. Let context TC_3, $parameterIR^*$, and fresh type variables $typeId_{impl}^*$ be the result of typing parameterList, under context TC_2 at BLOCK.
4. Let nameIR be the name of name.
5. Let controlObjectTypeDefIR be CONTROL nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `).
6. Let bound'''''''''' be bound type variables from the GLOBAL layer of TC_0.
7. If controlObjectTypeDefIR is a well-formed type definition, with bound type variables bound'''''''''':
   a. Let TC_4 be TC_0 where nameIR to controlObjectTypeDefIR is added to the GLOBAL layer.
   b. Let controlTypeDeclarationIR be annotationList CONTROL nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( $parameterIR^*$ `) ';'.
   c. The result is context TC_4 and controlTypeDeclarationIR.

11.19.2. Compile-time evaluation

At compile-time, a control type declaration is loaded into the context with:

Loading annotationList CONTROL nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `( $parameterIR^*$ `) ';', under context IC_0, and store STO:

1. Let controlObjectTypeDefIR be CONTROL nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `( $parameterIR^*$ `).

2. Let IC_1 be IC_0 where nameIR to controlObjectTypeDefIR is added to the GLOBAL layer.
3. The result is context IC_1 and store STO.

11.20. Package type declaration

A package type declaration introduces a new package type.

```
packageTypeDeclaration
    : annotationList PACKAGE name typeParameterListOpt ( parameterList ) ;
    ;
```

See Section 8.5.4 for more details on package object types.

11.20.1. Type checking

After type checking, a package type declaration has the following form:

```
packageTypeDeclarationIR
    : annotationList PACKAGE nameIR
        < typeParameterListIR , typeParameterListIR > ( parameterListIR ) ;
    ;
```

Typing annotationList PACKAGE name typeParameterListOpt `( parameterList `) ';' under context TC_0:

1. Let B be bound type variables from the GLOBAL layer of TC_0.
2. Let TC_1 be TC_0 with BLOCK.KIND set to PACKAGE.
3. Let context TC_2 and type parameters $typeId_{expl}^*$ be the result of typing typeParameterListOpt, under context TC_1 at BLOCK.
4. Let context TC_3, constructorParameterIR*, and fresh type variables $typeId_{impl}^*$ be the result of typing `( parameterList `) under context TC_2.
5. Let nameIR be the name of name.
6. For each constructorParameterIR in constructorParameterIR*:
   - Let _ _ $typeIR_{package_inner}$ _ _ be constructorParameterIR.

   Let $typeIR_{package_inner}^*$ be the resulting list.
7. Let packageObjectTypeDefIR be PACKAGE nameIR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `{ $typeIR_{package\_inner}^*$ `}.
8. If packageObjectTypeDefIR is a well-formed type definition, with bound type variables B:
   a. Let TC_4 be TC_0 where nameIR to packageObjectTypeDefIR is added to the GLOBAL layer.
   b. Let constructorId be the constructor identifier for name `( parameterList `).
   c. Let $typeIR_{package}$ be the underlying type of packageObjectTypeDefIR.
   d. Let constructorTypeDefIR be CONSTRUCTOR `< $typeId_{expl}^*$ ',' $typeId_{impl}^*$ `> `( constructorParameterIR* `) ':' $typeIR_{package}$.
   e. If constructorTypeDefIR is a well-formed constructor type definition, with bound type variables B:

i. Let TC_5 be where constructorId to constructorTypeDefIR is added to TC_4.

ii. Let packageTypeDeclarationIR be annotationList PACKAGE nameIR `< $typeId_{expl}^{*}$ ',' $typeId_{impl}^{*}$ `> `( constructorParameterIR$^{*}$ `) ';'.

iii. The result is context TC_5 and packageTypeDeclarationIR.

11.20.2. Compile-time evaluation

At compile-time, a package type declaration is loaded into the context with:

Loading annotationList PACKAGE nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `( parameterIR$^{*}$ `) ';', under context IC_0, and store STO:

1. For each parameterIR in parameterIR*:

   - Let _ _ typeIR _ _ be parameterIR.

   Let typeIR* be the resulting list.

2. Let packageObjectTypeDefIR be PACKAGE nameIR `< typeParameterListIR ',' $typeParameterListIR_{inferred}$ `> `{ typeIR$^{*}$ `}.
3. Let IC_1 be IC_0 where nameIR to packageObjectTypeDefIR is added to the GLOBAL layer.
4. Let callableId be the callable identifier for nameIR `( parameterIR$^{*}$ `).
5. Let constructorDef be PACKAGE `< typeParameterListIR ++ $typeParameterListIR_{inferred}$ `> `( parameterIR$^{*}$ `).
6. Let IC_2 be where callableId to constructorDef is added to IC_1.
7. The result is context IC_2 and store STO.

12. L-values

L-values are expressions that may appear on the left side of an assignment operation or as arguments corresponding to out and inout function parameters. An l-value represents a storage reference. The following expressions are legal l-values:

```
lvalue
    : referenceExpression | lvalue . member | lvalue [ expression ]
    | lvalue [ expression sliceop expression ] | ( lvalue )
    ;
```

- Identifiers of a base or derived type.
- Structure, header, and header union field member access operations (using the dot notation).
- References to elements within header stacks (see Section 8.4.4): indexing, and references to last and next.
- The result of a bit-slice operator [m:l] and [m+:l].

The following is a legal l-value: headers.stack[4].field. Note that method and function calls cannot return l-values.

An l-value is a restricted form of expression. The surface forms are converted between each other with:

The expression corresponding to lvalue'

1. If let referenceExpression of type referenceExpression be lvalue': return referenceExpression.
2. Else if let $\text{lvalue}_{\text{base}}$ '.' member be lvalue':
   a. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{lvalue}_{\text{base}}$.
   b. Return $\text{expression}_{\text{base}}$ '.' member.
3. Else if let $\text{lvalue}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `] be lvalue':
   a. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{lvalue}_{\text{base}}$.
   b. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `].
4. Else if let $\text{lvalue}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ sliceop $\text{expression}_{\text{lo}}$ `] be lvalue':
   a. If sliceop matches pattern ':':
      i. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{lvalue}_{\text{base}}$.
      ii. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ ':' $\text{expression}_{\text{lo}}$ `].
   b. Else:
      i. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{lvalue}_{\text{base}}$.
      ii. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ '+:' $\text{expression}_{\text{lo}}$ `].
5. Else:
   a. Let `( lvalue `) be lvalue'.
   b. Let expression be the expression corresponding to lvalue.
   c. Return `( expression `).

The l-value corresponding to expression'

1. If let referenceExpression of type referenceExpression be expression': return referenceExpression.
2. Else if let memberAccessBase '.' member of type memberAccessExpression be expression':
   a. Check that memberAccessBase has type expression.
   b. Let expression be memberAccessBase.
   c. Let $\text{lvalue}'^{?}$ be the l-value corresponding to expression.
   d. Check that $\text{lvalue}'^{?}$ is defined.
   e. Let lvalue be $\text{lvalue}'^{?}$.
   f. Return lvalue '.' member.
3. Else if let $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `] of type indexAccessExpression be expression':
   a. Let $\text{lvalue}^{?}$ be the l-value corresponding to $\text{expression}_{\text{base}}$.
   b. Check that $\text{lvalue}^{?}$ is defined.
   c. Let $\text{lvalue}_{\text{base}}$ be $\text{lvalue}^{?}$.
   d. Return $\text{lvalue}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `].
4. Else if let $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ sliceop $\text{expression}_{\text{lo}}$ `] of type sliceAccessExpression be expression':
   a. If sliceop matches pattern ':':
      i. Let $\text{lvalue}^{?}$ be the l-value corresponding to $\text{expression}_{\text{base}}$.

ii. Check that lvalue? is defined.

iii. Let $lvalue_{base}$ be lvalue?.

iv. Return $lvalue_{base}$ `[ $expression_{hi}$ ':' $expression_{lo}$ `].

b. Else:

i. Let lvalue? be the l-value corresponding to $expression_{base}$.

ii. Check that lvalue? is defined.

iii. Let $lvalue_{base}$ be lvalue?.

iv. Return $lvalue_{base}$ `[ $expression_{hi}$ '+:' $expression_{lo}$ `].

5. Else if let `( expression `) of type parenthesizedExpression be expression':

a. Let lvalue'? be the l-value corresponding to expression.

b. Check that lvalue'? is defined.

c. Let lvalue be lvalue'?.

d. Return `( lvalue `).

12.1. Semantics of l-values

12.1.1. Type checking

After type checking, l-values are represented in $P4_{IR}$ as:

```
typedLvalueIR
    : lvalueIR # lvalueNoteIR
    ;

lvalueIR
    : referenceExpressionIR | typedLvalueIR . nameIR
    | typedLvalueIR [ typedExpressionIR ]
    | typedLvalueIR [ typedExpressionIR sliceop typedExpressionIR ]
    | ( typedLvalueIR )
    ;

lvalueNoteIR
    : ( typeIR )
    ;
```

Notice that l-values in $P4_{IR}$ carry type information. L-values are type checked according to the relation:

Lvalue_ok:

- Typing lvalue, under context TC at p:
- The result is typedLvalueIR.

Lvalue_ok dispatch:

1. If let prefixedNonTypeName of type prefixedNonTypeName be lvalue: goto referenceExpression
2. Else if let $\text{lvalue}_{\text{base}}$ '.' member be lvalue: goto memberAccess
3. Else if let $\text{lvalue}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `] be lvalue: goto indexAccess
4. Else if let $\text{lvalue}_{\text{base}}$ `[ expression sliceop expression' `] be lvalue: goto sliceAccess
5. Else if let `( $\text{lvalue}_{\text{base}}$ `) be lvalue: goto parenthesized

The following helper functions are used to fetch the type of a typed l-value:

The type of lvalue typedLvalueIR

1. Let `( typeIR `) be the note of typedLvalueIR.
2. Return typeIR.

A typed l-value and a typed expression are converted between each other with:

The expression form of lvalueIR with note `( typeIR `)

1. If let referenceExpressionIR of type referenceExpressionIR be lvalueIR:
   a. Return referenceExpressionIR annotated with a pair of type typeIR and compile-time known-ness DYN.
2. Else if let typedLvalueIR '.' nameIR be lvalueIR:
   a. Let $\text{typedExpressionIR}'^{?}$ be the expression form of typedLvalueIR.
   b. Check that $\text{typedExpressionIR}'^{?}$ is defined.
   c. Let typedExpressionIR be $\text{typedExpressionIR}'^{?}$.
   d. Return typedExpressionIR '.' nameIR annotated with a pair of type typeIR and compile-time known-ness DYN.
3. Else if let $\text{typedLvalueIR}_{\text{base}}$ `[ $\text{typedExpressionIR}_{\text{index}}$ `] be lvalueIR:
   a. Let $\text{typedExpressionIR}^{?}$ be the expression form of $\text{typedLvalueIR}_{\text{base}}$.
   b. Check that $\text{typedExpressionIR}^{?}$ is defined.
   c. Let $\text{typedExpressionIR}_{\text{base}}$ be $\text{typedExpressionIR}^{?}$.
   d. Return $\text{typedExpressionIR}_{\text{base}}$ `[ $\text{typedExpressionIR}_{\text{index}}$ `] annotated with a pair of type typeIR and compile-time known-ness DYN.
4. Else if let $\text{typedLvalueIR}_{\text{base}}$ `[ $\text{typedExpressionIR}_{\text{hi}}$ sliceop $\text{typedExpressionIR}_{\text{lo}}$ `] be lvalueIR:
   a. If sliceop matches pattern ':':
      i. Let $\text{typedExpressionIR}^{?}$ be the expression form of $\text{typedLvalueIR}_{\text{base}}$.
      ii. Check that $\text{typedExpressionIR}^{?}$ is defined.
      iii. Let $\text{typedExpressionIR}_{\text{base}}$ be $\text{typedExpressionIR}^{?}$.
      iv. Return $\text{typedExpressionIR}_{\text{base}}$ `[ $\text{typedExpressionIR}_{\text{hi}}$ ':' $\text{typedExpressionIR}_{\text{lo}}$ `] annotated with a pair of type typeIR and compile-time known-ness DYN.
   b. Else:
      i. Let $\text{typedExpressionIR}^{?}$ be the expression form of $\text{typedLvalueIR}_{\text{base}}$.
      ii. Check that $\text{typedExpressionIR}^{?}$ is defined.

iii. Let typedExpressionIR$_{base}$ be typedExpressionIR$^{?}$.

iv. Return typedExpressionIR$_{base}$ `[ typedExpressionIR$_{hi}$ '+:' typedExpressionIR$_{lo}$ `] annotated with a pair of type typeIR and compile-time known-ness DYN.

The l-value form of expressionIR annotated with a pair of type typeIR and compile-time known-ness _

1. If let referenceExpressionIR of type referenceExpressionIR be expressionIR:
    a. Return referenceExpressionIR with note `( typeIR `).
2. Else if let memberAccessBaseIR '.' nameIR of type memberAccessExpressionIR be expressionIR:
    a. Check that memberAccessBaseIR has type typedExpressionIR.
    b. Let typedExpressionIR be memberAccessBaseIR.
    c. Let typedLvalueIR$'^{?}$ be the l-value form of typedExpressionIR.
    d. Check that typedLvalueIR$'^{?}$ is defined.
    e. Let typedLvalueIR be typedLvalueIR$'^{?}$.
    f. Return typedLvalueIR '.' nameIR with note `( typeIR `).
3. Else if let typedExpressionIR$_{base}$ `[ typedExpressionIR$_{index}$ `] of type indexAccessExpressionIR be expressionIR:
    a. Let typedLvalueIR$^{?}$ be the l-value form of typedExpressionIR$_{base}$.
    b. Check that typedLvalueIR$^{?}$ is defined.
    c. Let typedLvalueIR$_{base}$ be typedLvalueIR$^{?}$.
    d. Return typedLvalueIR$_{base}$ `[ typedExpressionIR$_{index}$ `] with note `( typeIR `).
4. Else if let typedExpressionIR$_{base}$ `[ typedExpressionIR$_{hi}$ sliceop typedExpressionIR$_{lo}$ `] of type sliceAccessExpressionIR be expressionIR:
    a. If sliceop matches pattern ':':
        i. Let typedLvalueIR$^{?}$ be the l-value form of typedExpressionIR$_{base}$.
        ii. Check that typedLvalueIR$^{?}$ is defined.
        iii. Let typedLvalueIR$_{base}$ be typedLvalueIR$^{?}$.
        iv. Return typedLvalueIR$_{base}$ `[ typedExpressionIR$_{hi}$ ':' typedExpressionIR$_{lo}$ `] with note `( typeIR `).
    b. Else:
        i. Let typedLvalueIR$^{?}$ be the l-value form of typedExpressionIR$_{base}$.
        ii. Check that typedLvalueIR$^{?}$ is defined.
        iii. Let typedLvalueIR$_{base}$ be typedLvalueIR$^{?}$.
        iv. Return typedLvalueIR$_{base}$ `[ typedExpressionIR$_{hi}$ '+:' typedExpressionIR$_{lo}$ `] with note `( typeIR `).
5. Else if let `( typedExpressionIR `) of type parenthesizedExpressionIR be expressionIR:
    a. Let typedLvalueIR$'^{?}$ be the l-value form of typedExpressionIR.
    b. Check that typedLvalueIR$'^{?}$ is defined.
    c. Let typedLvalueIR be typedLvalueIR$'^{?}$.
    d. Return `( typedLvalueIR `) with note `( typeIR `).

12.1.2. Runtime evaluation

At runtime, l-values are represented as storage references:

```
storageReference
    : referenceExpressionIR | storageReference . nameIR
    | storageReference [ value ] | storageReference [ value sliceop value ]
    ;
```

12.1.2.1. Creation

The following relation evaluates an l-value to a storage reference:

Lvalue_eval:

- Evaluating typedLvalueIR, under context EC and state ARCH at p:
- The result is context EC, state ARCH, and result storageReferenceResult.

Lvalue_eval dispatch:

1. If let referenceExpressionIR of type referenceExpressionIR be lvalueIR: goto referenceExpressionIR
2. Else if let typedLvalueIR '.' nameIR be lvalueIR: goto memberAccess
3. Else if let typedLvalueIR `[ typedExpressionIR `] be lvalueIR: goto indexAccess
4. Else if let typedLvalueIR `[ $\text{typedExpressionIR}_l$ sliceop $\text{typedExpressionIR}_r$ `] be lvalueIR: goto sliceAccess
5. Else:
    a. Let `( typedLvalueIR `) be lvalueIR.
    b. Goto parenthesized

The result of evaluating an l-value is:

```
storageReferenceResult
    : continueResult<storageReference?> | abortResult
    ;

continueResult<X>
    : _CONT X
    ;

abortResult
    : exitResult | rejectTransitionResult
    ;
```

12.1.2.2. Read

The following relation reads a value from a storage reference:

Lvalue_read:

- Reading from storageReference, under context EC and state ARCH at p:
- The result is value value.

Lvalue_read dispatch:

1. If let prefixedNameIR of type prefixedNameIR be storageReference': goto referenceExpressionIR
2. Else if let storageReference '.' nameIR be storageReference': goto memberAccess
3. Else if let storageReference `[ value' `] be storageReference':
   a. Check that value' has type integerValue.
   b. Let $\text{integerValue}_{\text{index}}$ be value'.
   c. Goto indexAccess
4. Else:
   a. Let storageReference `[ value' sliceop value'' `] be storageReference'.
   b. Goto sliceAccess

12.1.2.3. Write

The following relation writes a value to a storage reference:

Lvalue_write:

- Writing value to storageReference, under context EC and state ARCH at p:
- The result is context EC.

Lvalue_write dispatch:

1. If let prefixedNameIR of type prefixedNameIR be storageReference': goto referenceExpressionIR
2. Else if let storageReference '.' nameIR be storageReference': goto memberAccess
3. Else if let storageReference `[ value' `] be storageReference':
   a. Check that value' has type integerValue.
   b. Let $\text{integerValue}_{\text{index}}$ be value'.
   c. Goto indexAccess
4. Else:
   a. Let storageReference `[ value' sliceop value'' `] be storageReference'.
   b. Goto sliceAccess

The subsequent sections describe each kind of l-value in detail.

12.2. Identifiers

12.2.1. Type checking

Typing prefixedNonTypeName, under context TC at p:

1. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
2. Let varTypeIR? be the type of variable prefixedNameIR from the p layer of TC.
3. Check that varTypeIR? is defined.
4. Let direction typeIR ctk value? be varTypeIR?.
5. Check that ctk is DYN.
6. Check that value? is none.
7. Check that direction is equal to OUT or direction is equal to INOUT.
8. The result is prefixedNameIR with note `( typeIR `).

12.2.2. Creation

Evaluating referenceExpressionIR with note lvalueNoteIR, under context EC and state ARCH at p:

1. The result is context EC, state ARCH, and result $_{\text{CONT}}$ referenceExpressionIR.

12.2.3. Read

Reading from prefixedNameIR, under context EC and state ARCH at p:

1. Let value be the value of prefixedNameIR from the p layer of EC.
2. The result is value value.

12.2.4. Write

Writing value to prefixedNameIR, under context EC_0 and state ARCH at p:

1. Let EC_1 be EC_0 where prefixedNameIR is updated to value from the p layer.
2. The result is context EC_1.

12.3. Member accesses

12.3.1. Type checking

Typing $\text{lvalue}_{\text{base}}$ '.' member, under context TC at p:

1. Let $\text{typedLvalueIR}_{\text{base}}$ be the result of typing $\text{lvalue}_{\text{base}}$, under context TC at p.
2. Let `( $\text{typeIR}_{\text{base}}$ `) be the note of $\text{typedLvalueIR}_{\text{base}}$.
3. Let typeIR' be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
4. If let HEADER_STACK typeIR `[ _ `] of type headerStackTypeIR be typeIR':
   a. Let nameIR be the name of member.
   b. Check that nameIR is equal to "next" or nameIR is equal to "last".
   c. Check that p is equal to BLOCK and TC.BLOCK.KIND is equal to PARSER or p is equal to LOCAL and TC.LOCAL.KIND is equal to PARSER_STATE.
   d. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}}$ '.' nameIR with note `( typeIR `).
   e. The result is typedLvalueIR.
5. Else if let STRUCT _ `< $\_^{*}$ `> `{ ( _ $\text{typeIR}_{\text{field}}$ $\text{nameIR}_{\text{field}}$ ';' )$^{*}$ `} of type structTypeIR be typeIR':
   a. Let nameIR be the name of member.
   b. Let $\text{typeIR''}^{?}$ be the value associated to nameIR in ($\text{nameIR}_{\text{field}}$, $\text{typeIR}_{\text{field}}$)*.
   c. Check that $\text{typeIR''}^{?}$ is defined.
   d. Let typeIR be $\text{typeIR''}^{?}$.
   e. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}}$ '.' nameIR with note `( typeIR `).
   f. The result is typedLvalueIR.
6. Else if let HEADER _ `< $\_^{*}$ `> `{ ( _ $\text{typeIR}_{\text{field}}$ $\text{nameIR}_{\text{field}}$ ';' )$^{*}$ `} of type headerTypeIR be typeIR':
   a. Let nameIR be the name of member.
   b. Let $\text{typeIR''}^{?}$ be the value associated to nameIR in ($\text{nameIR}_{\text{field}}$, $\text{typeIR}_{\text{field}}$)*.
   c. Check that $\text{typeIR''}^{?}$ is defined.
   d. Let typeIR be $\text{typeIR''}^{?}$.
   e. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}}$ '.' nameIR with note `( typeIR `).
   f. The result is typedLvalueIR.
7. Else if let HEADER_UNION _ `< $\_^{*}$ `> `{ ( _ $\text{typeIR}_{\text{field}}$ $\text{nameIR}_{\text{field}}$ ';' )$^{*}$ `} of type headerUnionTypeIR be typeIR':
   a. Let nameIR be the name of member.
   b. Let $\text{typeIR''}^{?}$ be the value associated to nameIR in ($\text{nameIR}_{\text{field}}$, $\text{typeIR}_{\text{field}}$)*.
   c. Check that $\text{typeIR''}^{?}$ is defined.
   d. Let typeIR be $\text{typeIR''}^{?}$.
   e. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}}$ '.' nameIR with note `( typeIR `).
   f. The result is typedLvalueIR.

12.3.2. Creation

Evaluating typedLvalueIR '.' nameIR with note lvalueNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$, and result storageReferenceResult' be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
2. If let abortResult of type abortResult be storageReferenceResult':
    a. The result is context EC_1, state $ARCH_1$, and result abortResult.
3. Else:
    a. Let $_{CONT}$ storageReference'$^?$ be storageReferenceResult'.
    b. Check that storageReference'$^?$ is defined.
    c. Let storageReference be storageReference'$^?$.
    d. Let value value be the result of reading from storageReference, under context EC_1 and state $ARCH_1$ at p.
    e. If value has type headerStackValue:
        i. Let headerStackValue be value.
        ii. Let HEADER_STACK `[ $value_{element}^*$ `(n_{idx} `) `] be headerStackValue.
        iii. Try:
            A. If nameIR is equal to "next":
                I. Let n_{size} be the length of $value_{element}^*$.
                II. If n_{idx} is greater than or equal to n_{size}:
                    1. Let rejectTransitionResult be REJECT ERROR '.' "StackOutOfBounds".
                    2. The result is context EC_1, state $ARCH_1$, and result rejectTransitionResult.
        iv. Then, try:
            A. Let n_{size} be the length of $value_{element}^*$.
            B. If nameIR is equal to "next":
                I. If n_{idx} is less than n_{size}:
                    1. Let storageReferenceResult be $_{CONT}$ storageReference '.' nameIR.
                    2. The result is context EC_1, state $ARCH_1$, and result storageReferenceResult.
        v. Then, try:
            A. If nameIR is equal to "last":
                I. Let n_{size} be the length of $value_{element}^*$.
                II. If n_{idx} is less than 1 or n_{idx} is greater than n_{size}:
                    1. Let rejectTransitionResult be REJECT ERROR '.' "StackOutOfBounds".
                    2. The result is context EC_1, state $ARCH_1$, and result rejectTransitionResult.
        vi. Then, try:
            A. Let n_{size} be the length of $value_{element}^*$.
            B. Check that nameIR is equal to "last".
            C. Check that $_n$idx~ < 1 \/ n_{idx} > n_{size}.
            D. Let storageReferenceResult be $_{CONT}$ storageReference '.' nameIR.
            E. The result is context EC_1, state $ARCH_1$, and result storageReferenceResult.
    f. Else:

i. Let storageReferenceResult be $_{CONT}$ storageReference '.' nameIR.

ii. The result is context EC_1, state $ARCH_1$, and result storageReferenceResult.

12.3.3. Read

Reading from storageReference '.' nameIR, under context EC and state ARCH at p:

1. Let value $value_{base}$ be the result of reading from storageReference, under context EC and state ARCH at p.
2. If let REF objectId of type objectReferenceValue be $value_{base}$:
   a. Let $object'^{?}$ be the object objectId in ARCH.
   b. Check that $object'^{?}$ is defined.
   c. Let object be $object'^{?}$.
   d. Check that object has type packageObject.
   e. Let PACKAGE `< _ `> `{ frame `} be object.
   f. Let $value'^{?}$ be the value of nameIR in map frame.
   g. Check that $value'^{?}$ is defined.
   h. Let value be $value'^{?}$.
   i. The result is value value.
3. Else if let $headerStackValue_{base}$ of type headerStackValue be $value_{base}$:
   a. Let HEADER_STACK `[ $value_{element}^{*}$ `(n_{idx} `) `] be $headerStackValue_{base}$.
   b. If nameIR is equal to "next":
      i. Let n_{size} be the length of $value_{element}^{*}$.
      ii. Check that n_{idx} is less than n_{size}.
      iii. Check that n_{idx} is less than the length of $value_{element}^{*}$.
      iv. Let value be $value_{element}^{*}[n_{idx}]$.
      v. The result is value value.
   c. Else if nameIR is equal to "last":
      i. Let n_{size} be the length of $value_{element}^{*}$.
      ii. Check that $_{n}idx\sim < 1 \setminus/ n_{idx} > n_{size}$.
      iii. Let i be $n_{idx} - 1$.
      iv. Check that i has type nat.
      v. Let n_{idx_last} be i.
      vi. Check that n_{idx_last} is less than the length of $value_{element}^{*}$.
      vii. Let value be $value_{element}^{*}[n_{idx_last}]$.
      viii. The result is value value.
4. Else if let $structValue_{base}$ of type structValue be $value_{base}$:
   a. Let STRUCT _ `{ ( $value_{field}$ $nameIR_{field}$ ';' )* `} be $structValue_{base}$.
   b. Let $value'^{?}$ be the value associated to nameIR in ($nameIR_{field}$, $value_{field}$)*.
   c. Check that $value'^{?}$ is defined.
   d.

Let value be value'?.

e. The result is value value.

5. Else if let $headerValue_{base}$ of type headerValue be $value_{base}$:
   a. Let HEADER _ `{ _ ';' ( $value_{field}$ $nameIR_{field}$ ';' )* `} be $headerValue_{base}$.
   b. Let value'? be the value associated to nameIR in ($nameIR_{field}$, $value_{field}$)*.
   c. Check that value'? is defined.
   d. Let value be value'?.
   e. The result is value value.
6. Else if let $headerUnionValue_{base}$ of type headerUnionValue be $value_{base}$:
   a. Let HEADER_UNION _ `{ ( $value_{field}$ $nameIR_{field}$ ';' )* `} be $headerUnionValue_{base}$.
   b. Let value'? be the value associated to nameIR in ($nameIR_{field}$, $value_{field}$)*.
   c. Check that value'? is defined.
   d. Let value be value'?.
   e. The result is value value.

12.3.4. Write

Writing value to storageReference '.' nameIR, under context EC_0 and state ARCH at p:

1. Let value value' be the result of reading from storageReference, under context EC_0 and state ARCH at p.
2. If let $headerStackValue_{base}$ of type headerStackValue be value':
   a. Let HEADER_STACK `[ $value_{element}$* `(n_{idx} `) `] be $headerStackValue_{base}$.
   b. If nameIR is equal to "next":
      i. Let $value_{element_update}$* be $value_{element}$* with [n_{idx}] set to value.
      ii. Let $headerStackValue_{base_update}$ be HEADER_STACK `[ $value_{element\_update}$* `(n_{idx} + 1 `) `].
      iii. Let context EC_1 be the result of writing $headerStackValue_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.
      iv. The result is context EC_1.
   c. Else if nameIR is equal to "last":
      i. Let i be n_{idx} - 1.
      ii. Check that i has type nat.
      iii. Let n_{idx_last} be i.
      iv. Let $value_{element_update}$* be $value_{element}$* with [n_{idx_last}] set to value.
      v. Let $headerStackValue_{base_update}$ be HEADER_STACK `[ $value_{element\_update}$* `(n_{idx} `) `].
      vi. Let context EC_1 be the result of writing $headerStackValue_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.
      vii. The result is context EC_1.
3. Else if let $structValue_{base}$ of type structValue be value':
   a. Let STRUCT typeId `{ fieldValue* `} be $structValue_{base}$.
   b. Let $fieldValue_{update}$* be $update_fieldValue(fieldValue*, nameIR, value).

c. Let structValue$_{base_update}$ be STRUCT typeId `{ fieldValue$_{update}^{*}$ `}.

d. Let context EC_1 be the result of writing structValue$_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.

e. The result is context EC_1.

4. Else if let headerValue$_{base}$ of type headerValue be value':

a. Let HEADER typeId `{ $b_{valid}$ ';' fieldValue$^{*}$ `} be headerValue$_{base}$.

b. Let fieldValue$_{update}^{*}$ be $update_fieldValue(fieldValue*, nameIR, value).

c. Let headerValue$_{base_update}$ be HEADER typeId `{ $b_{valid}$ ';' fieldValue$_{update}^{*}$ `}.

d. Let context EC_1 be the result of writing headerValue$_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.

e. The result is context EC_1.

5. Else if let headerUnionValue$_{base}$ of type headerUnionValue be value':

a. Let HEADER_UNION typeId `{ fieldValue$^{*}$ `} be headerUnionValue$_{base}$.

b. Let (value$_{field}$ nameIR$_{field}$ ';')* be fieldValue*.

c. Let value''$^{?}$ be the value associated to nameIR in (nameIR$_{field}$, value$_{field}$)*.

d. Check that value''$^{?}$ is defined.

e. Let value$_{old}$ be value''$^{?}$.

f. Check that value$_{old}$ has type headerValue.

g. Let HEADER _ `{ $b_{valid\_old}$ ';' _$^{*}$ `} be value$_{old}$.

h. Let fieldValue$_{update}^{*}$ be $update_fieldValue(fieldValue*, nameIR, value).

i. Check that value has type headerValue.

j. Let HEADER _ `{ $b_{valid\_new}$ ';' _$^{*}$ `} be value.

k. Let $b_{toggled}$ be $_b$valid_old~ and b_{valid_new} or b_{valid_old} and $_b$valid_new~.

l. If $_b$toggled~:

i. Let headerUnionValue$_{base_update}$ be HEADER_UNION typeId `{ fieldValue$_{update}^{*}$ `}.

ii. Let context EC_1 be the result of writing headerUnionValue$_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.

iii. The result is context EC_1.

m. Else:

i. Let fieldValue$_{update'}^{*}$ be $update_headerUnion(fieldValue$_{update}^{*}$, nameIR).

ii. Let headerUnionValue$_{base_update}$ be HEADER_UNION typeId `{ fieldValue$_{update'}^{*}$ `}.

iii. Let context EC_1 be the result of writing headerUnionValue$_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.

iv. The result is context EC_1.

12.4. Index accesses

12.4.1. Type checking

Typing lvalue$_{base}$ `[ expression$_{index}$ `], under context TC at p:

1. Let typedLvalueIR$_{base}$ be the result of typing lvalue$_{base}$, under context TC at p.
2. Let `( typeIR$_{base}$ `) be the note of typedLvalueIR$_{base}$.
3. Let typeIR' be typeIR$_{base}$ with typedefs unrolled.
4. If let ARRAY typeIR `[ n$_{size}$ `] of type arrayTypeIR be typeIR':
    a. Let typedExpressionIR$_{index}$ be the result of typing expression$_{index}$, under context TC at p.
    b. Let a pair of type typeIR$_{index}$ and compile-time known-ness ctk$_{index}$ be the note of typedExpressionIR$_{index}$.
    c. Let typedExpressionIR$^{?}$ be the result of reducing serializable enums in typedExpressionIR$_{index}$ until $compat_array_index is satisfied.
    d. Check that typedExpressionIR$^{?}$ is defined.
    e. Let typedExpressionIR$_{index_reduced}$ be typedExpressionIR$^{?}$.
    f. If ctk$_{index}$ is LCTK:
        i. Let value be the result of local compile-time evaluation of typedExpressionIR$_{index_reduced}$, under context TC at p.
        ii. Check that value has type integerValue.
        iii. Let integerValue$_{index}$ be value.
        iv. Let nat$^{?}$ be the natural number representation of integerValue$_{index}$.
        v. Check that nat$^{?}$ is defined.
        vi. Let n$_{index}$ be nat$^{?}$.
        vii. Check that n$_{index}$ is less than n$_{size}$.
        viii. Let typedLvalueIR be typedLvalueIR$_{base}$ `[ typedExpressionIR$_{index\_reduced}$ `] with note `( typeIR `).
        ix. The result is typedLvalueIR.
    g. Else:
        i. Let typedLvalueIR be typedLvalueIR$_{base}$ `[ typedExpressionIR$_{index\_reduced}$ `] with note `( typeIR `).
        ii. The result is typedLvalueIR.
5. Else if let HEADER_STACK typeIR `[ n$_{size}$ `] of type headerStackTypeIR be typeIR':
    a. Let typedExpressionIR$_{index}$ be the result of typing expression$_{index}$, under context TC at p.
    b. Let a pair of type typeIR$_{index}$ and compile-time known-ness ctk$_{index}$ be the note of typedExpressionIR$_{index}$.
    c. Let typedExpressionIR$^{?}$ be the result of reducing serializable enums in typedExpressionIR$_{index}$ until $compat_array_index is satisfied.
    d. Check that typedExpressionIR$^{?}$ is defined.
    e. Let typedExpressionIR$_{index_reduced}$ be typedExpressionIR$^{?}$.
    f. If ctk$_{index}$ is LCTK:
        i. Let value be the result of local compile-time evaluation of typedExpressionIR$_{index_reduced}$, under context TC at p.
        ii. Check that value has type integerValue.
        iii. Let integerValue$_{index}$ be value.
        iv. Let nat$^{?}$ be the natural number representation of integerValue$_{index}$.
        v. Check that nat$^{?}$ is defined.
        vi. Let n$_{index}$ be nat$^{?}$.
        vii. Check that n$_{index}$ is less than n$_{size}$.
        viii. Let typedLvalueIR be typedLvalueIR$_{base}$ `[ typedExpressionIR$_{index\_reduced}$ `] with note `( typeIR `).
        ix. The result is typedLvalueIR.

g. Else:

i. Let typedLvalueIR be typedLvalueIR$_{base}$ `[ typedExpressionIR$_{index\_reduced}$ `] with note `( typeIR `).

ii. The result is typedLvalueIR.

12.4.2. Creation

Evaluating typedLvalueIR `[ typedExpressionIR `] with note lvalueNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$, and result storageReferenceResult' be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
2. If let abortResult of type abortResult be storageReferenceResult':
   a. The result is context EC_1, state $ARCH_1$, and result abortResult.
3. Else:
   a. Let $_{CONT}$ storageReference'$^{?}$ be storageReferenceResult'.
   b. Check that storageReference'$^{?}$ is defined.
   c. Let storageReference be storageReference'$^{?}$.
   d. Let context EC_2, state $ARCH_2$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_1 and state $ARCH_1$ at p.
   e. If let abortResult of type abortResult be expressionResult:
      i. The result is context EC_2, state $ARCH_2$, and result abortResult.
   f. Else:
      i. Let $_{CONT}$ value be expressionResult.
      ii. Check that value has type integerValue.
      iii. Let integerValue$_{index}$ be value.
      iv. Let storageReferenceResult be $_{CONT}$ storageReference `[ integerValue$_{index}$ `].
      v. The result is context EC_2, state $ARCH_2$, and result storageReferenceResult.

12.4.3. Read

Reading from storageReference `[ integerValue$_{index}$ `], under context EC and state ARCH at p:

1. Let value value'' be the result of reading from storageReference, under context EC and state ARCH at p.
2. If let arrayValue$_{base}$ of type arrayValue be value'':
   a. Let ARRAY `[ value$_{element}^{*}$ `] be arrayValue$_{base}$.
   b. Let n_{size} be the length of value$_{element}^{*}$.
   c. Let i_{idx} be the integer representation of integerValue$_{index}$.
   d. Let b_{in_bounds} be i_{idx} is less than n_{size} and i_{idx} is greater than or equal to 0.
   e. If b_{in_bounds}:
      i. Check that i_{idx} has type nat.
      ii. Let n_{idx} be i_{idx}.

iii. Check that n_{idx} is less than the length of $value_{element}^{*}$.

iv. Let value be $value_{element}^{*}[n_{idx}]$.

v. The result is value value.

f. Else:

i. Check that 0 is less than the length of $value_{element}^{*}$.

ii. Let value''' be $value_{element}^{*}[0]$.

iii. Check that value''' has type headerValue.

iv. Let HEADER typeId `{ $b_{valid}$ ';' $fieldValue^{*}$ `} be value'''.

v. Let value be HEADER typeId `{ false ';' $fieldValue^{*}$ `}.

vi. The result is value value.

3. Else if let $headerStackValue_{base}$ of type headerStackValue be value'':

a. Let HEADER_STACK `[ $value_{element}^{*}$ `(_ `) `] be $headerStackValue_{base}$.

b. Let n_{size} be the length of $value_{element}^{*}$.

c. Let i_{idx} be the integer representation of $integerValue_{index}$.

d. Let b_{in_bounds} be i_{idx} is less than n_{size} and i_{idx} is greater than or equal to 0.

e. If b_{in_bounds}:

i. Check that i_{idx} has type nat.

ii. Let n_{idx} be i_{idx}.

iii. Check that n_{idx} is less than the length of $value_{element}^{*}$.

iv. Let value be $value_{element}^{*}[n_{idx}]$.

v. The result is value value.

f. Else:

i. Check that 0 is less than the length of $value_{element}^{*}$.

ii. Let value''' be $value_{element}^{*}[0]$.

iii. Check that value''' has type headerValue.

iv. Let HEADER typeId `{ $b_{valid}$ ';' $fieldValue^{*}$ `} be value'''.

v. Let value be HEADER typeId `{ false ';' $fieldValue^{*}$ `}.

vi. The result is value value.

12.4.4. Write

`<span id="Lvalue_write-indexAccess"></span>`

Writing value to storageReference `[ $integerValue_{index}$ `], under context EC_0 and state ARCH at p:

1. Let value value'' be the result of reading from storageReference, under context EC_0 and state ARCH at p.
2. If let $arrayValue_{base}$ of type arrayValue be value'':

a. Let ARRAY `[ $value_{element}^{*}$ `] be $arrayValue_{base}$.

b. Let n_{size} be the length of $value_{element}^{*}$.

c. Let i_{idx} be the integer representation of $integerValue_{index}$.

d. Let b_{in_bounds} be i_{idx} is less than n_{size} and i_{idx} is greater than or equal to 0.

e. If b_{in_bounds}:

i. Check that i_{idx} has type nat.

ii. Let n_{idx} be i_{idx}.

iii. Let $value_{element_update}^{*}$ be $value_{element}^{*}$ with $[n_{idx}]$ set to value.

iv. Let $arrayValue_{base_update}$ be ARRAY `[ $value_{element\_update}^{*}$ `].

v. Let context EC_1 be the result of writing $arrayValue_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.

vi. The result is context EC_1.

f. Else: the result is context EC_0.

3. Else if let $headerStackValue_{base}$ of type headerStackValue be value'':

a. Let HEADER_STACK `[ $value_{element}^{*}$ `(n_{base_idx} `) `] be $headerStackValue_{base}$.

b. Let n_{size} be the length of $value_{element}^{*}$.

c. Let i_{idx} be the integer representation of $integerValue_{index}$.

d. Let b_{in_bounds} be i_{idx} is less than n_{size} and i_{idx} is greater than or equal to 0.

e. If b_{in_bounds}:

i. Check that i_{idx} has type nat.

ii. Let n_{idx} be i_{idx}.

iii. Let $value_{element_update}^{*}$ be $value_{element}^{*}$ with $[n_{idx}]$ set to value.

iv. Let $headerStackValue_{base_update}$ be HEADER_STACK `[ $value_{element\_update}^{*}$ `(n_{base_idx} `) `].

v. Let context EC_1 be the result of writing $headerStackValue_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.

vi. The result is context EC_1.

f. Else: the result is context EC_0.

12.5. Bit-slice accesses

12.5.1. Type checking

Typing $lvalue_{base}$ `[ expression sliceop expression' `], under context TC at p:

1. If sliceop matches pattern ':':

a. Let $typedLvalueIR_{base}$ be the result of typing $lvalue_{base}$, under context TC at p.

b. Let `( $typeIR_{base}$ `) be the note of $typedLvalueIR_{base}$.

c. Check that $typeIR_{base}$ can be used as a bitslice base.

d. Let $typedExpressionIR_{hi}$ be the result of typing expression, under context TC at p.

e. Let $typedExpressionIR_{lo}$ be the result of typing expression', under context TC at p.

f. Let $typedExpressionIR^{?}$ be the result of reducing serializable enums in $typedExpressionIR_{hi}$ until $compat_bitslice_range_index is satisfied.

g. Check that $typedExpressionIR^{?}$ is defined.

h. Let $typedExpressionIR_{hi_reduced}$ be $typedExpressionIR^{?}$.

i. Let $typedExpressionIR'^{?}$ be the result of reducing serializable enums in $typedExpressionIR_{lo}$ until $compat_bitslice_range_index is satisfied.

j. Check that typedExpressionIR'? is defined.

k. Let typedExpressionIR$_{lo_reduced}$ be typedExpressionIR'?.

l. Let a pair of type typeIR$_{hi_reduced}$ and compile-time known-ness ctk$_{hi_reduced}$ be the note of typedExpressionIR$_{hi_reduced}$.

m. Let a pair of type typeIR$_{lo_reduced}$ and compile-time known-ness ctk$_{lo_reduced}$ be the note of typedExpressionIR$_{lo_reduced}$.

n. Check that ctk$_{hi_reduced}$ is LCTK.

o. Let value be the result of local compile-time evaluation of typedExpressionIR$_{hi_reduced}$, under context TC at p.

p. Check that value has type integerValue.

q. Let integerValue$_{hi}$ be value.

r. Let nat? be the natural number representation of integerValue$_{hi}$.

s. Check that nat? is defined.

t. Let n$_{hi}$ be nat?.

u. Check that ctk$_{lo_reduced}$ is LCTK.

v. Let value' be the result of local compile-time evaluation of typedExpressionIR$_{lo_reduced}$, under context TC at p.

w. Check that value' has type integerValue.

x. Let integerValue$_{lo}$ be value'.

y. Let nat'? be the natural number representation of integerValue$_{lo}$.

z. Check that nat'? is defined.

aa. Let n$_{lo}$ be nat'?.

ab. Check that typeIR$_{base}$ `[ n$_{lo}$ ':' n$_{hi}$ `] is a valid bitslice.

ac. Let i be n$_{hi}$ - n$_{lo}$ + 1.

ad. Check that i has type nat.

ae. Let n$_{slice}$ be i.

af. Let typeIR be BIT `< n$_{slice}$ `>.

ag. Let typedLvalueIR be typedLvalueIR$_{base}$ `[ typedExpressionIR$_{hi\_reduced}$ ':' typedExpressionIR$_{lo\_reduced}$ `] with note `( typeIR `).

ah. The result is typedLvalueIR.

2. Else:

a. Let typedLvalueIR$_{base}$ be the result of typing lvalue$_{base}$, under context TC at p.

b. Let `( typeIR$_{base}$ `) be the note of typedLvalueIR$_{base}$.

c. Check that typeIR$_{base}$ can be used as a bitslice base.

d. Let typedExpressionIR$_{l}$ be the result of typing expression, under context TC at p.

e. Let typedExpressionIR$_{w}$ be the result of typing expression', under context TC at p.

f. Let typedExpressionIR? be the result of reducing serializable enums in typedExpressionIR$_{l}$ until $compat_bitslice_offset_index is satisfied.

g. Check that typedExpressionIR? is defined.

h. Let typedExpressionIR$_{l_reduced}$ be typedExpressionIR?.

i. Let typedExpressionIR'? be the result of reducing serializable enums in typedExpressionIR$_{w}$ until $compat_bitslice_offset_width is satisfied.

j. Check that typedExpressionIR'? is defined.

k. Let typedExpressionIR$_{w_reduced}$ be typedExpressionIR'?.

l. Let a pair of type $typeIR_{w_reduced}$ and compile-time known-ness $ctk_{w_reduced}$ be the note of $typedExpressionIR_{w_reduced}$.

m. Check that $ctk_{w_reduced}$ is LCTK.

n. Let value be the result of local compile-time evaluation of $typedExpressionIR_{w_reduced}$, under context TC at p.

o. Check that value has type integerValue.

p. Let $integerValue_w$ be value.

q. Let $nat^?$ be the natural number representation of $integerValue_w$.

r. Check that $nat^?$ is defined.

s. Let n_w be $nat^?$.

t. Let typeIR be BIT `< $n_w$ `>.

u. Let typedLvalueIR be $typedLvalueIR_{base}$ `[ $typedExpressionIR_{l\_reduced}$ '+:' $typedExpressionIR_{w\_reduced}$ `] with note `( typeIR `).

v. The result is typedLvalueIR.

12.5.2. Creation

Evaluating typedLvalueIR `[ $typedExpressionIR_l$ sliceop $typedExpressionIR_r$ `] with note lvalueNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$, and result storageReferenceResult' be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
2. If let abortResult of type abortResult be storageReferenceResult':
   a. The result is context EC_1, state $ARCH_1$, and result abortResult.
3. Else:
   a. Let $_{CONT}$ $storageReference'^?$ be storageReferenceResult'.
   b. Check that $storageReference'^?$ is defined.
   c. Let storageReference be $storageReference'^?$.
   d. Let context EC_2, state $ARCH_2$ and expressionListResult be the result of evaluating [$typedExpressionIR_l$, $typedExpressionIR_r$], under context EC_1 and state $ARCH_1$ at p.
   e. If let abortResult of type abortResult be expressionListResult:
      i. The result is context EC_2, state $ARCH_2$, and result abortResult.
   f. Else:
      i. Let $_{CONT}$ $value^*$ be expressionListResult.
      ii. Check that $value^*$ is a list of length 2.
      iii. Let [$value_l$, $value_r$] be $value^*$.
      iv. Let storageReferenceResult be $_{CONT}$ storageReference `[ $value_l$ sliceop $value_r$ `].
      v. The result is context EC_1, state $ARCH_1$, and result storageReferenceResult.

12.5.3. Read

Reading from storageReference `[ value' sliceop value'' `], under context EC and state ARCH at p:

1. If sliceop matches pattern ':':
    a. Let value $value_{base}$ be the result of reading from storageReference, under context EC and state ARCH at p.
    b. Let value be $value_{base}$ `[ value' ':' value'' `].
    c. The result is value value.
2. Else:
    a. Let value $value_{base}$ be the result of reading from storageReference, under context EC and state ARCH at p.
    b. Let value be $value_{base}$ `[ value' '+:' value'' `].
    c. The result is value value.

12.5.4. Write

Writing value to storageReference `[ value' sliceop value'' `], under context EC_0 and state ARCH at p:

1. If sliceop matches pattern ':':
    a. Let value $value_{base}$ be the result of reading from storageReference, under context EC_0 and state ARCH at p.
    b. Let $value_{base_update}$ be $value_{base}$ `[ value' ':' value'' `] replaced with value.
    c. Let context EC_1 be the result of writing $value_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.
    d. The result is context EC_1.
2. Else:
    a. Let value $value_{base}$ be the result of reading from storageReference, under context EC_0 and state ARCH at p.
    b. Let $value_{base_update}$ be $value_{base}$ `[ value' '+:' value'' `] replaced with value.
    c. Let context EC_1 be the result of writing $value_{base_update}$ to storageReference, under context EC_0 and state ARCH at p.
    d. The result is context EC_1.

12.6. Parenthesized l-values

12.6.1. Type checking

Typing `( $lvalue_{base}$ `), under context TC at p:

1. Let $typedLvalueIR_{base}$ be the result of typing $lvalue_{base}$, under context TC at p.
2. Let `( $typeIR_{base}$ `) be the note of $typedLvalueIR_{base}$.
3. Let typedLvalueIR be `( $typedLvalueIR_{base}$ `) with note `( $typeIR_{base}$ `).
4. The result is typedLvalueIR.

12.6.2. Creation

Evaluating `( typedLvalueIR `) with note lvalueNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
2. The result is context EC_1, state $ARCH_1$, and result storageReferenceResult.

13. Statements

The syntax of statements is defined as follows:

```
statement
    : emptyStatement | assignmentStatement | callStatement
    | directApplicationStatement | returnStatement | exitStatement
    | blockStatement | conditionalStatement | forStatement | breakStatement
    | continueStatement | switchStatement
    ;
```

13.1. Semantics of statements

13.1.1. Type checking

Stmt_ok:

- Typing statement, under context TC at p:
- The result is context TC and statementIR.

Stmt_ok dispatch:

1. Try:
    a. If let emptyStatement of type emptyStatement be statement': goto emptyStatement
    b. Else if let lvalue assignop expression ';' of type assignmentStatement be statement': goto assignmentStatement
    c. Else if let callStatement of type callStatement be statement': goto callStatement
    d. Else if let namedType '.' APPLY `( argumentList `) ';' of type directApplicationStatement be statement': goto directApplicationStatement
    e. Else if let exitStatement of type exitStatement be statement': goto exitStatement
    f. Else if let conditionalStatement of type conditionalStatement be statement': goto conditionalStatement
2. Then, try:

a. Check that p is LOCAL.

b. If let returnStatement of type returnStatement be statement': goto returnStatement

c. Else if let blockStatement of type blockStatement be statement': goto blockStatement

d. Else if let forStatement of type forStatement be statement':

   i. Try:

      A. If let annotationList FOR `( forInitStatementList ';' expression ';' forUpdateStatementList `) statement be forStatement: goto forStatement-three-clause

   ii. Then, try:

      A. Goto forStatement-in

e. Else if let breakStatement of type breakStatement be statement': goto breakStatement

f. Else if let continueStatement of type continueStatement be statement': goto continueStatement

g. Else if let SWITCH `( $expression_{switch}$ `) `{ switchCaseList `} of type switchStatement be statement': goto switchStatement

After type checking, statements are represented in $P4_{IR}$ as follows:

```
statementIR
    : emptyStatementIR | assignmentStatementIR | callStatementIR
    | directApplicationStatementIR | returnStatementIR | exitStatementIR
    | blockStatementIR | conditionalStatementIR | forStatementIR
    | breakStatementIR | continueStatementIR | switchStatementIR
    ;
```

Within the typing context, in order to statically determine the control flow of a P4 program (e.g., whether a function body always returns), the following indicator for abstract control flow is used:

```
flow
    : CONT | RET
    ;
```

When two control-flow branches merge, their flows are joined:

The merged control flow of f and f'

1. If f is RET:

   a. If f' is RET: return RET.

2. Otherwise: return CONT.

The control flow of two branches of the typing context is joined, and the control flow recorded in the context is updated, with:

The join of control flows in TC_a and TC_b

1. Return the merged control flow of TC_a.CONTROL and TC_b.CONTROL.

TC with control flow set to flow

1. Return TC with CONTROL set to flow.

As a marker of whether a statement is nested inside a loop, the following indicator is used:

```
loop
    : IN | OUT
    ;
```

Entering a loop and updating the loop marker of the context are handled with:

TC within a loop

1. Return TC with LOOP set to IN.

TC with loop status set to loop

1. Return TC with LOOP set to loop.

13.1.2. Compile-time evaluation

Stmt_inst:

- Compile-time evaluation of statementIR, under context IC and store STO at p:
- The result is context IC, store STO, and statementIR.

Stmt_inst dispatch:

1. Try:
    a. If let emptyStatementIR of type emptyStatementIR be statementIR': goto emptyStatementIR
    b. Else if let assignmentStatementIR of type assignmentStatementIR be statementIR': goto assignmentStatementIR
    c. Else if let callStatementIR of type callStatementIR be statementIR': goto callStatementIR
    d. Else if let directApplicationStatementIR of type directApplicationStatementIR be statementIR': goto directApplicationStatementIR
    e. Else if let exitStatementIR of type exitStatementIR be statementIR': goto exitStatementIR
    f. Else if let conditionalStatementIR of type conditionalStatementIR be statementIR': goto conditionalStatementIR
    g. Else if let forStatementIR' of type forStatementIR be statementIR':
        i. Try:

A. If let annotationList FOR `( forInitStatementListIR ';' typedExpressionIR ';' forUpdateStatementListIR `) statementIR be forStatementIR': goto forStatementIR-three-clause

ii. Then, try:

A. Goto forStatementIR-in

h. Else if let breakStatementIR of type breakStatementIR be statementIR': goto breakStatementIR

i. Else if let continueStatementIR of type continueStatementIR be statementIR': goto continueStatementIR

j. Else if let SWITCH `( typedExpressionIR `) `{ switchCaseListIR `} of type switchStatementIR be statementIR': goto switchStatementIR

2. Then, try:

a. Check that p is LOCAL.

b. If let returnStatementIR of type returnStatementIR be statementIR': goto returnStatementIR

c. Else if let annotationList `{ blockElementStatementListIR `} of type blockStatementIR be statementIR': goto blockStatementIR

Instantiations nested within statements are processed according to the above relation.

13.1.3. Runtime evaluation

Stmt_eval:

- Runtime evaluation of statementIR, under context EC and state ARCH at p:
- The result is context EC, state ARCH, and statementResult.

Stmt_eval dispatch:

1. Try:

a. If let emptyStatement of type emptyStatement be statementIR': goto emptyStatementIR

b. Else if let typedLvalueIR assignop typedExpressionIR ';' of type assignmentStatementIR be statementIR': goto assignmentStatementIR

c. Else if let callableTargetIR `< typeArgumentIR* `> `( argumentIR* `) ';' of type callStatementIR be statementIR': goto callStatementIR

d. Else if let exitStatement of type exitStatement be statementIR': goto exitStatementIR

e. Else if let blockStatementIR of type blockStatementIR be statementIR': goto blockStatementIR

f. Else if let conditionalStatementIR of type conditionalStatementIR be statementIR': goto conditionalStatementIR

2. Then, try:

a. Check that p is LOCAL.

b. If let returnStatementIR of type returnStatementIR be statementIR': goto returnStatementIR

c. Else if let forStatementIR of type forStatementIR be statementIR':

i. Try:

A. If let annotationList FOR `( forInitStatementListIR ';' typedExpressionIR ';' forUpdateStatementListIR `) statementIR be forStatementIR: goto forStatementIR-three-clause

ii. Then, try:

A. Goto forStatementIR-in

d. Else if let breakStatement of type breakStatement be statementIR': goto breakStatementIR

e. Else if let continueStatement of type continueStatement be statementIR': goto continueStatementIR

f. Else if let SWITCH `( typedExpressionIR `) `{ switchCaseIR* `} of type switchStatementIR be statementIR': goto switchStatementIR

The result of evaluating a statement is represented as follows:

```
statementResult
    : continueEmptyResult | callResult | forResult
    ;

continueEmptyResult
    : /* empty */
    ;
```

The following is the result of evaluating a call statement:

```
callResult
    : abortResult | returnResult
    ;

abortResult
    : exitResult | rejectTransitionResult
    ;

returnResult
    : RETURN value?
    ;

exitResult
    : EXIT
    ;

rejectTransitionResult
    : REJECT errorValue
    ;
```

The following is the result of evaluating a for statement:

```
forResult
    : forBreakResult | forContinueResult
    ;

forBreakResult
    : BREAK
    ;

forContinueResult
    : CONTINUE
```

```
    ;
```

The subsequent sections describe each kind of statement in detail.

13.2. Empty statements

The empty statement, written ; is a no-op.

```
emptyStatement
    : ;
    ;
```

13.2.1. Type checking

After type checking, an empty statement is represented as:

```
emptyStatementIR = emptyStatement
```

Typing ';', under context TC at p:

1. The result is context TC and ';'.

13.2.2. Compile-time evaluation

Compile-time evaluation of emptyStatementIR, under context IC and store STO at p:

1. The result is context IC, store STO, and emptyStatementIR.

13.2.3. Runtime evaluation

Runtime evaluation of ';', under context EC and state ARCH at p:

1. The result is context EC, state ARCH, and $_{\text{EMPTY}}$.

13.3. Assignment statements

```
assignop
    : = | += | -= | |+|= | |-|= | *= | /= | %= | <<= | >>= | &= | ^= | |=
    ;

assignmentStatement
    : lvalue assignop expression ;
    ;
```

An assignment, written with the = sign, first evaluates its left sub-expression to an l-value, then evaluates its right sub-expression to a value, and finally copies the value into the l-value. Derived types (e.g. structs) are copied recursively, and all components of headers are copied, including validity bits. Assignment is not defined for extern values.

An assignment may also be written with a binary arithmetic or bit manipulation operator immediately before the = sign. This performs the binary operator on the old value of the left sub-expression and the right sub-expression and assigns the result to the l-value. Thus an assignment like A += B is equivalent to A = A + B, except that A is only evaluated once. This means that any side-effects within this operand (eg, a function call inside an array index or slice expression) only occur once. This is not valid for comparison operators, logical operators, concat, range, or mask operators.

The binary operator underlying a compound assignment operator is recovered with:

The binary operator corresponding to assignop

1. If assignop matches pattern '+=': return '+'.
2. Else if assignop matches pattern '-=': return '-'.
3. Else if assignop matches pattern '|+|=': return '|+|'.
4. Else if assignop matches pattern '|-|=': return '|-|'.
5. Else if assignop matches pattern '*=': return '*'.
6. Else if assignop matches pattern '/=': return '/'.
7. Else if assignop matches pattern '%=': return '%'.
8. Else if assignop matches pattern '<<=': return '<<'.
9. Else if assignop matches pattern '>>=': return '>>'.
10. Else if assignop matches pattern '&=': return '&'.
11. Else if assignop matches pattern '^=': return '^'.
12. Else if assignop matches pattern '|=': return '|'.
13. Otherwise: return ·.

13.3.1. Type checking

After type checking, an assignment statement has the form:

```
assignmentStatementIR
    : typedLvalueIR assignop typedExpressionIR ;
    ;
```

Typing lvalue assignop expression ';', under context TC at p:

1. If assignop is '=':
    a. Let typedLvalueIR be the result of typing lvalue, under context TC at p.
    b. Let typedExpressionIR be the result of typing expression, under context TC at p.
    c. Let $typeIR_l$ be the type of lvalue typedLvalueIR.
    d. Let $typeIR_r$ be the type of typedExpressionIR.
    e. Let $typedExpressionIR'^?$ be typedExpressionIR implicitly cast to $typeIR_l$.
    f. If $typedExpressionIR'^?$ is defined:
        i. Let $typedExpressionIR_{cast}$ be $typedExpressionIR'^?$.
        ii. The result is context TC and typedLvalueIR '=' $typedExpressionIR_{cast}$ ';'.
2. Else:
    a. Let $binop'^?$ be the binary operator corresponding to assignop.
    b. If $binop'^?$ is defined:
        i. Let binop be $binop'^?$.
        ii. Let typedLvalueIR be the result of typing lvalue, under context TC at p.
        iii. Let $expression_{lvalue}$ be the expression corresponding to lvalue.
        iv. Let $expression_{binary}$ be $expression_{lvalue}$ binop expression.
        v. Let $typedExpressionIR_{binary}$ be the result of typing $expression_{binary}$, under context TC at p.
        vi. Let `( $typeIR_{lvalue}$ `) be the note of typedLvalueIR.
        vii. Let $typedExpressionIR^?$ be $typedExpressionIR_{binary}$ implicitly cast to $typeIR_{lvalue}$.
        viii. If $typedExpressionIR^?$ is defined:
            A. Let expressionIR '#' _ be $typedExpressionIR^?$.
            B. If let _ binop'' $typedExpressionIR_{cast}$ of type binaryExpressionIR be expressionIR:
                I. If binop'' is equal to binop:
                    1. The result is context TC and typedLvalueIR assignop $typedExpressionIR_{cast}$ ';'.

13.3.2. Compile-time evaluation

Compile-time evaluation of assignmentStatementIR, under context IC and store STO at p:

1. The result is context IC, store STO, and assignmentStatementIR.

13.3.3. Runtime evaluation

Runtime evaluation of typedLvalueIR assignop typedExpressionIR ';', under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

2. If let abortResult of type abortResult be storageReferenceResult:
    a. The result is context EC_1, state $ARCH_1$, and abortResult.
3. Else:
    a. Let $_{CONT}$ storageReference'? be storageReferenceResult.
    b. If storageReference'? is defined:
        i. Let storageReference be storageReference'?.
        ii. If assignop is '=':
            A. Let context EC_2, state $ARCH_2$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_1 and state $ARCH_1$ at p.
            B. If let abortResult of type abortResult be expressionResult:
                I. The result is context EC_2, state $ARCH_2$, and abortResult.
            C. Else:
                I. Let $_{CONT}$ value be expressionResult.
                II. Let context EC_3 be the result of writing value to storageReference, under context EC_2 and state $ARCH_2$ at p.
                III. The result is context EC_3, state $ARCH_2$, and $_{EMPTY}$.
        iii. Else:
            A. Let context EC_2, state $ARCH_2$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_1 and state $ARCH_1$ at p.
            B. If let abortResult of type abortResult be expressionResult:
                I. The result is context EC_2, state $ARCH_2$, and abortResult.
            C. Else:
                I. Let $_{CONT}$ value be expressionResult.
                II. Let value $value_{lvalue}$ be the result of reading from storageReference, under context EC_2 and state $ARCH_2$ at p.
                III. Let binop'? be the binary operator corresponding to assignop.
                IV. If binop'? is defined:
                    1. Let binop be binop'?.
                    2. Let $value_{new}$ be $value_{lvalue}$ binop value.
                    3. Let context EC_3 be the result of writing $value_{new}$ to storageReference, under context EC_2 and state $ARCH_2$ at p.
                    4. The result is context EC_3, state $ARCH_2$, and $_{EMPTY}$.

13.4. Call statements

Call statements are used to invoke actions, functions, and methods.

```
callStatement
    : lvalue ( argumentList ) ; | lvalue < typeArgumentList > ( argumentList ) ;
    ;
```

Details of how calls are resolved and evaluated are described in Chapter 19.

13.4.1. Type checking

After type checking, a call statement is represented as:

```
callStatementIR
    : callableTargetIR < typeArgumentListIR > ( argumentListIR ) ;
    ;

callableTargetIR
    : referenceExpressionIR | typedExpressionIR . nameIR
    | TYPE prefixedNameIR . nameIR | ( callableTargetIR )
    ;
```

Typing callStatement, under context TC at p:

1. If let $\text{lvalue}_{\text{callable}}$ `( argumentList `) ';' be callStatement:
    a. Let typed callable target callableTargetIR be the result of typing $\text{lvalue}_{\text{callable}}$ as a callable target, under cursor p and context TC.
    b. Let argumentIR^{*} be the result of typing argumentList, under context TC at p.
    c. Let callable type callableTypeIR with fresh type variables $\text{typeId}_{\text{impl}}{}^{*}$, defaults $\text{id}_{\text{default}}{}^{*}$ and optionals $\text{id}_{\text{optional}}{}^{*}$ be the result of finding type of callable call callableTargetIR `< · `> `( $\text{argumentIR}^{*}$ `), under cursor p and context TC.
    d. Let $\text{typeArgumentIR}_{\text{infer}}{}^{*}$ be $(\ _{\text{NAME}}\ \text{typeId}_{\text{impl}}\ )^{*}$.
    e. Let return type $\text{typeIR}_{\text{ret}}$ with inferred $\text{typeArgumentIR}_{\text{inferred}}{}^{*}$ and casted $\text{argumentIR}_{\text{cast}}{}^{*}$ be the result of typing invocation callableTypeIR `< $\text{typeArgumentIR}_{\text{infer}}{}^{*}$ `> `( $\text{argumentIR}^{*}$ `) with omitted type arguments $\text{typeId}_{\text{impl}}{}^{*}$, defaults $\text{id}_{\text{default}}{}^{*}$ and optionals $\text{id}_{\text{optional}}{}^{*}$, under cursor p and context TC.
    f. If callableTypeIR is a static_assert call:
        i. Let callExpressionIR be callableTargetIR `< $\text{typeArgumentIR}_{\text{inferred}}{}^{*}$ `> `( $\text{argumentIR}_{\text{cast}}{}^{*}$ `).
        ii. Let expressionNoteIR be a pair of type $\text{typeIR}_{\text{ret}}$ and compile-time known-ness LCTK.
        iii. Let value be the result of local compile-time evaluation of callExpressionIR annotated with expressionNoteIR, under context TC at p.
        iv. If value is equal to $_{\text{B}}$ true:
            A. Let emptyStatementIR be ';'.
            B. The result is context TC and emptyStatementIR.
    g. Else:
        i. Let callStatementIR be callableTargetIR `< $\text{typeArgumentIR}_{\text{inferred}}{}^{*}$ `> `( $\text{argumentIR}_{\text{cast}}{}^{*}$ `) ';'.
        ii. The result is context TC and callStatementIR.
2. Else:
    a. Let $\text{lvalue}_{\text{callable}}$ `< typeArgumentList `> `( argumentList `) ';' be callStatement.
    b. Let typed callable target callableTargetIR be the result of typing $\text{lvalue}_{\text{callable}}$ as a callable target, under cursor p and context TC.
    c. Let $\text{typeArgumentIR}^{*}$ and fresh type variables $\text{typeId}_{\text{impl}}{}^{*}$ be the result of typing typeArgumentList, under context TC at p.
    d. Let argumentIR^{*} be the result of typing argumentList, under context TC at p.
    e.

Let callable type callableTypeIR with fresh type variables $typeId_{inserted}^{*}$, defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$ be the result of finding type of callable call callableTargetIR `< typeArgumentIR* `> `( argumentIR* `), under cursor p and context TC.

f. Let $typeArgumentIR_{infer}^{*}$ be typeArgumentIR* concatenated with (NAME $typeId_{inserted}$)*.

g. Let $typeId_{infer}^{*}$ be $typeId_{impl}^{*}$ concatenated with $typeId_{inserted}^{*}$.

h. Let return type $typeIR_{ret}$ with inferred $typeArgumentIR_{inferred}^{*}$ and casted $argumentIR_{cast}^{*}$ be the result of typing invocation callableTypeIR `< $typeArgumentIR_{infer}^{*}$ `> `( argumentIR* `) with omitted type arguments $typeId_{infer}^{*}$, defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$, under cursor p and context TC.

i. Let callStatementIR be callableTargetIR `< $typeArgumentIR_{inferred}^{*}$ `> `( $argumentIR_{cast}^{*}$ `) ';'.

j. The result is context TC and callStatementIR.

13.4.2. Compile-time evaluation

Compile-time evaluation of callStatementIR, under context IC and store STO at p:

1. The result is context IC, store STO, and callStatementIR.

13.4.3. Runtime evaluation

Runtime evaluation of callableTargetIR `< typeArgumentIR* `> `( argumentIR* `) ';', under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$, and callee calleeResult be the result of finding callee callableTargetIR `( argumentIR* `), under context EC and state ARCH at p.
2. If let abortResult of type abortResult be calleeResult:
   a. The result is context EC_1, state $ARCH_1$, and abortResult.
3. Else:
   a. Let CONT callee be calleeResult.
   b. Let context EC_2, state $ARCH_2$, and call result callResult be the result of calling callee `< typeArgumentIR* `> `( argumentIR* `), under context EC_1 and state $ARCH_1$ at p.
   c. If let abortResult of type abortResult be callResult:
      i. The result is context EC_2, state $ARCH_2$, and abortResult.
   d. Else:
      i. The result is context EC_2, state $ARCH_2$, and EMPTY.

13.5. Direct type invocation

Controls and parsers are often instantiated exactly once. As a light syntactic sugar, control and parser declarations with no constructor parameters may be applied directly, as if they were an instance. This has the effect of creating and applying a local instance of that type.

```
control Callee(/* parameters omitted */) { /* body omitted */ }
```

```
control Caller(/* parameters omitted */)(/* parameters omitted */) {
    apply {
        Callee.apply(/* arguments omitted */); // Callee is treated as an instance
    }
}
```

The definition of Caller is equivalent to the following.

```
control Caller(/* parameters omitted */)(/* parameters omitted */) {
    @name("Callee") Callee() Callee_inst; // local instance of Callee
    apply {
        Callee_inst.apply(/* arguments omitted */);         // Callee_inst is applied
    }
}
```

```
directApplicationStatement
    : namedType . APPLY ( argumentList ) ;
    ;
```

This feature is intended to streamline the common case where a type is instantiated exactly once. The grammar allows direct calls for generic controls or parsers:

```
control Callee<T>(/* parameters omitted */) { /* body omitted */ }

control Caller(/* parameters omitted */)(/* parameters omitted */) {
    apply {
        // Callee<bit<32>> is treated as an instance
        Callee<bit<32>>.apply(/* arguments omitted */);
    }
}
```

For completeness, the behavior of directly invoking the same type more than once is defined as follows.

- Direct type invocation in different scopes will result in different local instances with different fully-qualified control names.
- In the same scope, direct type invocation will result in a different local instance per invocation—however, instances of the same type will share the same global name, via the @name annotation. If the type contains controllable entities, then invoking it directly more than once in the same scope is illegal, because it will produce multiple controllable entities with the same fully-qualified control name.

See [sec-name-annotations] for details of @name annotations.

No direct invocation is possible for controls or parsers that require constructor arguments. These need to be instantiated before they are invoked.

13.5.1. Type checking

After type checking, a direct application statement is represented as:

```
directApplicationStatementIR
    : constructorTargetIR . APPLY ( argumentListIR ) ;
```

```
    ;
```

Typing namedType '.' APPLY `( argumentList `) ';', under context TC at p:

1. Let expressionIR annotated with a pair of type $typeIR_{object}$ and compile-time known-ness _ be the result of typing namedType `( $_{EMPTY}$ `), under context TC at p.
2. If let callExpressionIR of type callExpressionIR be expressionIR:
    a. If let constructorTargetIR `( $argumentIR^{*}$ `) be callExpressionIR:
        i. If $argumentIR^{*}$ is an empty list:
            A. Let prefixedNameIR `< $typeArgumentIR^{*}$ `> be constructorTargetIR.
            B. Let $typeIR_{object_unroll}$ be $typeIR_{object}$ with typedefs unrolled.
            C. If $typeIR_{object_unroll}$ has type parserObjectTypeIR or $typeIR_{object_unroll}$ has type controlObjectTypeIR:
                I. Let $nameIR_{object}$ be "__direct_application".
                II. Let TC_1 be TC where $nameIR_{object}$ to $_{EMPTY}$ $typeIR_{object}$ CTK · is added to the p layer.
                III. Let lvalue be $_{ID}$ $nameIR_{object}$ '.' $_{ID}$ "apply".
                IV. Let context _ and statementIR be the result of typing lvalue `( argumentList `) ';', under context TC_1 at p.
                V. If let callStatementIR of type callStatementIR be statementIR:
                    1. Let callableTargetIR `< $typeArgumentIR'^{*}$ `> `( $argumentIR_{cast}^{*}$ `) ';' be callStatementIR.
                    2. If let $typedExpressionIR_{base}$ '.' t be callableTargetIR:
                        1. If t is equal to "apply":
                            1. If $typeArgumentIR'^{*}$ is an empty list:
                                1. If $_{BARE}$ $nameIR_{object}$ annotated with a pair of type $typeIR_{object}$ and compile-time known-ness CTK is equal to $typedExpressionIR_{base}$:
                                    1. If constructorTargetIR is equal to prefixedNameIR `< $typeArgumentIR^{*}$ `>:
                                        1. Let directApplicationStatementIR be constructorTargetIR '.' APPLY `( $argumentIR_{cast}^{*}$ `) ';'.
                                        2. The result is context TC and directApplicationStatementIR.

13.5.2. Compile-time evaluation

At compile-time, a direct application is instantiated, and then desugared into a method call on that instance with the relation:

DirectApplicationStmt_inst:

- Compile-time evaluation of directApplicationStatementIR, under context IC and store STO at p:
- The result is store STO and desugared constantDeclarationIR, callStatementIR.

It is implemented as:

Compile-time evaluation of constructorTargetIR '.' APPLY `( argumentListIR `) ';', under context IC and store STO_0 at p:

1. Let prefixedNameIR `< typeArgumentListIR `> be constructorTargetIR.
2. Let constructor constructorDef `< $typeArgumentListIR_{inst}$ `> with defaults $id_{default}^*$ and optionals $id_{optional}^*$ be the result of finding constructor for prefixedNameIR `< typeArgumentListIR `> `( · `) under context IC at p.
3. Let typeId be the string form of prefixedNameIR.
4. Let IC_{callee} be IC with typeId added to the path.
5. Let store STO_1 and instantiated object be the result of instantiating constructorDef `< $typeArgumentListIR_{inst}$ `> `( · `) with defaults $id_{default}^*$ and optionals $id_{optional}^*$, under context IC_{callee} and store STO_0 at p.
6. Let $typeId_{fresh}$ be a fresh type variable.
7. Let typeIR be $_{NAME}$ typeId.
8. Let nameIR be the concatenation of [typeId, "_", $typeId_{fresh}$].
9. Let objectId be IC.PATH concatenated with typeId.
10. Let STO_2 be STO_1 where objectId to object is added.
11. Let constantDeclarationIR be $_{EMPTY}$ CONST typeIR nameIR '=' $_{VALUE}$ REF objectId ';'.
12. Let $typedExpressionIR_{base}$ be $_{BARE}$ nameIR annotated with a pair of type typeIR and compile-time known-ness CTK.
13. Let callableTargetIR be $typedExpressionIR_{base}$ '.' "apply".
14. Let callStatementIR be callableTargetIR `< $typeArgumentListIR_{inst}$ `> `( argumentListIR `) ';'.
15. The result is store STO_2 and desugared constantDeclarationIR, callStatementIR.

A direct application statement is instantiated as:

Compile-time evaluation of directApplicationStatementIR, under context IC and store STO at p:

1. Let store STO_1 and desugared constantDeclarationIR, callStatementIR be the result of compile-time evaluation of directApplicationStatementIR, under context IC and store STO at p.
2. Let blockStatementIR be $_{EMPTY}$ `{ [ constantDeclarationIR, callStatementIR ] `}.
3. The result is context IC, store STO_1, and blockStatementIR.

13.6. Return statements

The return statement immediately terminates the execution of the action, function or control containing it. return statements are not allowed within parsers. return statements followed by an expression are only allowed within functions that return values; in this case the type of the expression must match the return type of the function. Any copy-out behavior due to direction out or inout parameters of the enclosing action, function, or control are still performed after the execution of the return statement. See Section 19.6 for details on copy-out behavior.

```
returnStatement
    : RETURN ; | RETURN expression ;
    ;
```

13.6.1. Type checking

After type checking, a return statement has the form:

```
returnStatementIR
    : RETURN ; | RETURN typedExpressionIR ;
    ;
```

Typing returnStatement, under context TC at LOCAL:

1. If returnStatement matches pattern RETURN ';':
    a. Check that VOID is equal to the expected return type in TC.
    b. Let TC_1 be TC with control flow set to RET.
    c. The result is context TC_1 and RETURN ';'.
2. Else:
    a. Let RETURN expression ';' be returnStatement.
    b. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
    c. Let typeIR be the type of typedExpressionIR.
    d. Let $typeIR^{?}$ be the expected return type in TC.
    e. Check that $typeIR^{?}$ is defined.
    f. Let $typeIR_{ret}$ be $typeIR^{?}$.
    g. Let $typedExpressionIR^{?}$ be typedExpressionIR implicitly cast to $typeIR_{ret}$.
    h. Check that $typedExpressionIR^{?}$ is defined.
    i. Let $typedExpressionIR_{cast}$ be $typedExpressionIR^{?}$.
    j. Let TC_1 be TC with control flow set to RET.
    k. The result is context TC_1 and RETURN $typedExpressionIR_{cast}$ ';'.

13.6.2. Compile-time evaluation

Compile-time evaluation of returnStatementIR, under context IC and store STO at LOCAL:

1. The result is context IC, store STO, and returnStatementIR.

13.6.3. Runtime evaluation

Runtime evaluation of returnStatementIR, under context EC and state ARCH at LOCAL:

1. If returnStatementIR matches pattern RETURN ';':

a. The result is context EC, state ARCH, and RETURN ·.

2. Else:

a. Let RETURN typedExpressionIR ';' be returnStatementIR.

b. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC and state ARCH at LOCAL.

c. If let abortResult of type abortResult be expressionResult:

i. The result is context EC_1, state $ARCH_1$, and abortResult.

d. Else:

i. Let $_{CONT}$ value be expressionResult.

ii. The result is context EC_1, state $ARCH_1$, and RETURN value.

13.7. Exit statements

The exit statement immediately terminates the execution of all the blocks currently executing: the current action (if invoked within an action), the current control, and all its callers. exit statements are not allowed within parsers or functions.

Any copy-out behavior due to direction out or inout parameters of the enclosing action or control, and all of its callers, are still performed after the execution of the exit statement. See Section 19.6 for details on copy-out behavior.

```
exitStatement
    : EXIT ;
    ;
```

There are some expressions whose evaluation might cause an exit statement to be executed. Examples include:

- table.apply().action_run, which can only appear as the expression of a switch statement (see Section 13.13), and when it appears there, it must be the entire expression.
- Any expression containing table.apply().hit or table.apply().miss (see Section 17.3.7), which can be part of arbitrarily complex expressions in many places of a P4 program, such as:
  - the expression in an if statement.
  - the expression e1 in a conditional expression e1 ? e2 : e3.
  - in an assignment statement, in the left and/or right hand sides.
  - an argument passed to a function or method call.
  - an expression to calculate a table key (see [sec-mau-semantics]).

This list is not intended to be exhaustive.

If applying the table causes an action to be executed, which in turn causes an exit statement to be executed, then evaluation of the expression ends immediately, and the rest of the current expression or statement does not complete its execution. See Section 14.1 for the order of evaluation of the parts of an expression. For the examples listed above, it also means the following behavior after the expression evaluation is interrupted.

- For a switch statement, if table.apply() exits, then none of the blocks in the switch statement are executed.

- If table.apply().hit or table.apply().miss cause an exit during the evaluation of an expression:
  - If it is the expression of an if statement, then neither the 'then' nor 'else' branches of the if statement are executed.
  - If it is the expression e1 in a conditional expression e1 ? e2 : e3, then neither expression e2 nor e3 are evaluated.
  - If the expression is the right hand side of an assignment statement, or part of the calculation of the L-value on the left hand side (e.g. the index expression of a header stack reference), then no assignment occurs.
  - If the expression is an argument passed to a function or method call, then the function/method call does not occur.
  - If the expression is a table key, then the table is not applied.

13.7.1. Type checking

After type checking, an exit statement is represented in $P4_{IR}$ as:

```
exitStatementIR = exitStatement
```

Typing EXIT ';', under context TC at p:

1. The result is context TC and EXIT ';'.

13.7.2. Compile-time evaluation

Compile-time evaluation of exitStatementIR, under context IC and store STO at p:

1. The result is context IC, store STO, and exitStatementIR.

13.7.3. Runtime evaluation

Runtime evaluation of EXIT ';', under context EC and state ARCH at p:

1. The result is context EC, state ARCH, and EXIT.

13.8. Block statements

A block statement is denoted by curly braces. It contains a sequence of statements and declarations, which are executed sequentially. The declarations (e.g., variables and constants) within a block statement are only visible within the block.

```
blockStatement
    : annotationList { blockElementStatementList }
    ;

blockElementStatementList
    : /* empty */ | blockElementStatementList blockElementStatement
    ;

blockElementStatement
    : constantDeclaration | variableDeclaration | statement
    ;
```

13.8.1. Type checking

After type checking, block statements are represented in $P4_{IR}$ as:

```
blockStatementIR
    : annotationList { blockElementStatementListIR }
    ;

blockElementStatementListIR = blockElementStatementIR*

blockElementStatementIR
    : constantDeclarationIR | variableDeclarationIR | statementIR
    ;
```

Typing blockStatement, under context TC at LOCAL:

1. Let TC_1 be TC with a new local frame.
2. Let context TC_2 and blockStatementIR be the result of typing blockStatement under context TC_1.
3. Let TC_3 be TC_2 with the innermost frame removed.
4. The result is context TC_3 and blockStatementIR.

A block is type checked by the relation:

Block_ok:

- Typing blockStatement under context TC:
- The result is context TC and blockStatementIR.

Typing annotationList `{ blockElementStatementList `} under context TC_0:

1. Let context TC_1 and blockElementStatementIR* be the result of typing blockElementStatementList under context TC_0.
2. Let blockStatementIR be annotationList `{ blockElementStatementIR* `}.
3. The result is context TC_1 and blockStatementIR.

A statement or declaration within a block statement is type checked by the relation:

BlockElementStmt_ok:

- Typing blockElementStatement under context TC:
- The result is context TC and blockElementStatementIR.

Typing blockElementStatement under context TC_0:

1. If let constantDeclaration of type constantDeclaration be blockElementStatement:
   a. Let context TC_1 and constantDeclarationIR be the result of typing constantDeclaration, under context TC_0 at LOCAL.
   b. The result is context TC_1 and constantDeclarationIR.
2. Else if let variableDeclaration of type variableDeclaration be blockElementStatement:
   a. Let context TC_1 and variableDeclarationIR be the result of typing variableDeclaration, under context TC_0 at LOCAL.
   b. The result is context TC_1 and variableDeclarationIR.
3. Else:
   a. Let statement be blockElementStatement.
   b. Let context TC_1 and statementIR be the result of typing statement, under context TC_0 at LOCAL.
   c. The result is context TC_1 and statementIR.

A list of block elements is type checked by the relation:

BlockElementStmtList_ok:

- Typing blockElementStatementList under context TC:
- The result is context TC and blockElementStatementIR*.

Typing blockElementStatementList under context TC_0:

1. Let blockElementStatement* be blockElementStatementList flattened.
2. Let context TC_1 and blockElementStatementIR* be the result of typing blockElementStatement* under context TC_0.
3. The result is context TC_1 and blockElementStatementIR*.

BlockElementStmts_ok:

- Typing blockElementStatement* under context TC:
- The result is context TC and blockElementStatementIR*.

Typing blockElementStatement* under context TC:

1. If blockElementStatement* matches pattern []:
   a. The result is context TC and ·.
2. Else:
   a. Let $blockElementStatement_h$ :: $blockElementStatement_t^*$ be blockElementStatement*.
   b. Let context TC_1 and $blockElementStatementIR_h$ be the result of typing $blockElementStatement_h$ under context TC.
   c. Let context TC_2 and $blockElementStatementIR_t^*$ be the result of typing $blockElementStatement_t^*$ under context TC_1.
   d. The result is context TC_2 and $blockElementStatementIR_h$ :: $blockElementStatementIR_t^*$.

13.8.2. Compile-time evaluation

Compile-time evaluation of annotationList `{ blockElementStatementListIR `}, under context IC and store STO at LOCAL:

1. Let IC_1 be IC with a new local frame.
2. Let context IC_2, store STO_1, and _ `{ $blockElementStatementListIR_{inst}$ `} be the result of compile-time evaluation of annotationList `{ blockElementStatementListIR `}, under context IC_1, and store STO.
3. Let IC_3 be IC_2 with the innermost frame removed.
4. The result is context IC_3, store STO_1, and annotationList `{ $blockElementStatementListIR_{inst}$ `}.

A block is compile-time evaluated by the relation:

Block_inst:

- Compile-time evaluation of blockStatementIR, under context IC, and store STO:
- The result is context IC, store STO, and blockStatementIR.

Compile-time evaluation of annotationList `{ blockElementStatementListIR `}, under context IC_0, and store STO_0:

1. Let context IC_1, store STO_1, and $blockElementStatementListIR_{inst}$ be the result of compile-time evaluation of blockElementStatementListIR, under context IC_0, and store STO_0.
2. The result is context IC_1, store STO_1, and annotationList `{ $blockElementStatementListIR_{inst}$ `}.

A block element is compile-time evaluated by the relation:

BlockElementStmt_inst:

- Compile-time evaluation of blockElementStatementIR, under context IC, and store STO:
- The result is context IC, store STO, and blockElementStatementIR.

Compile-time evaluation of blockElementStatementIR, under context IC, and store STO:

1. If let constantDeclarationIR of type constantDeclarationIR be blockElementStatementIR:
   a. Let context IC_1 be the result of loading constantDeclarationIR, under context IC at LOCAL.
   b. The result is context IC_1, store STO, and constantDeclarationIR.
2. Else if let variableDeclarationIR of type variableDeclarationIR be blockElementStatementIR:
   a. The result is context IC, store STO, and variableDeclarationIR.
3. Else:
   a. Let statementIR be blockElementStatementIR.
   b. Let context IC_1, store STO_1, and $statementIR_{inst}$ be the result of compile-time evaluation of statementIR, under context IC and store STO at LOCAL.
   c. The result is context IC_1, store STO_1, and $statementIR_{inst}$.

A list of block elements is compile-time evaluated by the relation:

BlockElementStmtList_inst:

- Compile-time evaluation of blockElementStatementListIR, under context IC, and store STO:

- The result is context IC, store STO, and blockElementStatementIR*.

Compile-time evaluation of blockElementStatementIR*, under context IC, and store STO:

1. If blockElementStatementIR* matches pattern []:
   a. The result is context IC, store STO, and ·.
2. Else:
   a. Let $\text{blockElementStatementIR}_h$ :: $\text{blockElementStatementIR}_t^*$ be blockElementStatementIR*.
   b. Let context IC_1, store STO_1, and $\text{blockElementStatementIR}_{h_inst}$ be the result of compile-time evaluation of $\text{blockElementStatementIR}_h$, under context IC, and store STO.
   c. Let context IC_2, store STO_2, and $\text{blockElementStatementIR}_{t_inst}^*$ be the result of compile-time evaluation of $\text{blockElementStatementIR}_t^*$, under context IC_1, and store STO_1.
   d. The result is context IC_2, store STO_2, and $\text{blockElementStatementIR}_{h_inst}$ :: $\text{blockElementStatementIR}_{t_inst}^*$.

13.8.3. Runtime evaluation

Runtime evaluation of blockStatementIR, under context EC and state ARCH at p:

1. Let EC_1 be EC with a new local frame.
2. Let context EC_2, state $ARCH_1$, and statementResult be the result of runtime evaluation of blockStatementIR, under context EC_1, and state ARCH.
3. Let EC_3 be EC_2 with the innermost frame removed.
4. The result is context EC_3, state $ARCH_1$, and statementResult.

A block is runtime evaluated by the relation:

Block_eval:

- Runtime evaluation of blockStatementIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and statementResult.

Runtime evaluation of annotationList `{ blockElementStatementListIR `}, under context EC_0, and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of blockElementStatementListIR, under context EC_0, and state $ARCH_0$.
2. The result is context EC_1, state $ARCH_1$, and statementResult.

A block element is runtime evaluated by the relation:

BlockElementStmt_eval:

- Runtime evaluation of blockElementStatementIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and statementResult.

Runtime evaluation of blockElementStatementIR, under context EC_0, and state ARCH:

1. If let constantDeclarationIR of type constantDeclarationIR be blockElementStatementIR:
   a. Let context EC_1 be the result of evaluating constantDeclarationIR, under context EC_0 at LOCAL.
   b. The result is context EC_1, state ARCH, and $_{\text{EMPTY}}$.
2. Else if let variableDeclarationIR of type variableDeclarationIR be blockElementStatementIR:
   a. Let context EC_1, state $ARCH_1$, and result declarationResult be the result of evaluating variableDeclarationIR, under context EC_0 and state ARCH at LOCAL.
   b. The result is context EC_1, state $ARCH_1$, and declarationResult.
3. Else:
   a. Let statementIR be blockElementStatementIR.
   b. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of statementIR, under context EC_0 and state ARCH at LOCAL.
   c. The result is context EC_1, state $ARCH_1$, and statementResult.

A list of block elements is runtime evaluated by the relation:

BlockElementStmtList_eval:

- Runtime evaluation of blockElementStatementListIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and statementResult.

Runtime evaluation of $blockElementStatementIR^*$, under context EC, and state ARCH:

1. If $blockElementStatementIR^*$ matches pattern []:
   a. The result is context EC, state ARCH, and $_{\text{EMPTY}}$.
2. Else:
   a. Let $blockElementStatementIR_h :: blockElementStatementIR_t^*$ be $blockElementStatementIR^*$.
   b. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of

blockElementStatementIR$_h$, under context EC, and state ARCH.

c. Try:

i. If statementResult is not equal to $_{EMPTY}$:

A. The result is context EC$_1$, state ARCH$_1$, and statementResult.

d. Then, try:

i. Check that statementResult has type continueEmptyResult.

ii. Let continueEmptyResult be statementResult.

iii. Let context EC$_2$, state ARCH$_2$, and statementResult' be the result of runtime evaluation of blockElementStatementIR$_t^*$, under context EC$_1$, and state ARCH$_1$.

iv. The result is context EC$_2$, state ARCH$_2$, and statementResult'.

13.9. Conditional statements

The conditional statement uses standard syntax and semantics familiar from many programming languages.

However, the condition expression in P4 is required to be a Boolean (and not an integer).

```
conditionalStatement
    : IF ( expression ) statement | IF ( expression ) statement ELSE statement
    ;
```

When several if statements are nested, the else applies to the innermost if statement that does not have an else statement.

13.9.1. Type checking

After type checking, a conditional statement has the form:

```
conditionalStatementIR
    : IF ( typedExpressionIR ) statementIR
    | IF ( typedExpressionIR ) statementIR ELSE statementIR
    ;
```

Typing conditionalStatement, under context TC at p:

1. If let IF `( expression$_{cond}$ `) statement$_{then}$ be conditionalStatement:

a. Let typedExpressionIR$_{cond}$ be the result of typing expression$_{cond}$, under context TC at p.

b. Let typeIR$_{cond}$ be the type of typedExpressionIR$_{cond}$.

c. If BOOL is equal to typeIR$_{cond}$ with typedefs unrolled:

i. Let context TC$_{then}$ and statementIR$_{then}$ be the result of typing statement$_{then}$, under context TC at p.

ii. Let TC$_{post}$ be TC$_{then}$ with control flow set to TC.CONTROL.

iii. Let conditionalStatementIR be IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then}$.

iv. The result is context TC$_{post}$ and conditionalStatementIR.

2. Else:

a. Let IF `( expression$_{cond}$ `) statement$_{then}$ ELSE statement$_{else}$ be conditionalStatement.

b. Let typedExpressionIR$_{cond}$ be the result of typing expression$_{cond}$, under context TC at p.

c. Let typeIR$_{cond}$ be the type of typedExpressionIR$_{cond}$.

d. If BOOL is equal to typeIR$_{cond}$ with typedefs unrolled:

i. Let context TC$_{then}$ and statementIR$_{then}$ be the result of typing statement$_{then}$, under context TC at p.

ii. Let context TC$_{else}$ and statementIR$_{else}$ be the result of typing statement$_{else}$, under context TC at p.

iii. Let f$_{post}$ be the join of control flows in TC$_{then}$ and TC$_{else}$.

iv. Let TC$_{post}$ be TC with control flow set to f$_{post}$.

v. Let conditionalStatementIR be IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then}$ ELSE statementIR$_{else}$.

vi. The result is context TC$_{post}$ and conditionalStatementIR.

13.9.2. Compile-time evaluation

Compile-time evaluation of conditionalStatementIR, under context IC and store STO at p:

1. If let IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then}$ be conditionalStatementIR:

a. Let context IC$_{then}$, store STO$_1$, and statementIR$_{then_inst}$ be the result of compile-time evaluation of statementIR$_{then}$, under context IC and store STO at p.

b. The result is context IC, store STO$_1$, and IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then_inst}$.

2. Else:

a. Let IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then}$ ELSE statementIR$_{else}$ be conditionalStatementIR.

b. Let context IC$_{then}$, store STO$_1$, and statementIR$_{then_inst}$ be the result of compile-time evaluation of statementIR$_{then}$, under context IC and store STO at p.

c. Let context IC$_{else}$, store STO$_2$, and statementIR$_{else_inst}$ be the result of compile-time evaluation of statementIR$_{else}$, under context IC and store STO$_1$ at p.

d. The result is context IC, store STO$_2$, and IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then_inst}$ ELSE statementIR$_{else_inst}$.

13.9.3. Runtime evaluation

Runtime evaluation of conditionalStatementIR, under context EC and state ARCH at p:

1. If let IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then}$ be conditionalStatementIR:

a. Let context EC$_{cond}$, state ARCH$_{cond}$ and expressionResult be the result of evaluating typedExpressionIR$_{cond}$, under context EC and state ARCH at p.

b. Try:

i. If let abortResult of type abortResult be expressionResult:

A. The result is context EC$_{cond}$, state ARCH$_{cond}$, and abortResult.

c. Then, try:

i. If expressionResult is equal to $_{CONT\ B}$ true:

A. Let context EC_{then}, state $ARCH_{then}$, and statementResult be the result of runtime evaluation of $statementIR_{then}$, under context EC_{cond} and state $ARCH_{cond}$ at p.

B. The result is context EC_{then}, state $ARCH_{then}$, and statementResult.

ii. Else if expressionResult is equal to $_{CONT\ B}$ false:

A. The result is context EC_{cond}, state $ARCH_{cond}$, and $_{EMPTY}$.

2. Else:

a. Let IF `( $typedExpressionIR_{cond}$ `) $statementIR_{then}$ ELSE $statementIR_{else}$ be conditionalStatementIR.

b. Let context EC_{cond}, state $ARCH_{cond}$ and expressionResult be the result of evaluating $typedExpressionIR_{cond}$, under context EC and state ARCH at p.

c. Try:

i. If let abortResult of type abortResult be expressionResult:

A. The result is context EC_{cond}, state $ARCH_{cond}$, and abortResult.

d. Then, try:

i. If expressionResult is equal to $_{CONT\ B}$ true:

A. Let context EC_{then}, state $ARCH_{then}$, and statementResult be the result of runtime evaluation of $statementIR_{then}$, under context EC_{cond} and state $ARCH_{cond}$ at p.

B. The result is context EC_{then}, state $ARCH_{then}$, and statementResult.

ii. Else if expressionResult is equal to $_{CONT\ B}$ false:

A. Let context EC_{else}, state $ARCH_{else}$, and statementResult be the result of runtime evaluation of $statementIR_{else}$, under context EC_{cond} and state $ARCH_{cond}$ at p.

B. The result is context EC_{else}, state $ARCH_{else}$, and statementResult.

13.10. For statements

A for statement executes a statement zero or more times, based on a condition or a range or collection.

```
<span id="forStatement"></span>
<span id="forInitStatementList"></span>
<span id="forInitStatementListNonEmpty"></span>
<span id="forInitStatement"></span>
<span id="forUpdateStatementList"></span>
<span id="forUpdateStatementListNonEmpty"></span>
<span id="forUpdateStatement"></span>
<span id="forCollectionExpression"></span>
```

```
forStatement
    : annotationList FOR
        ( forInitStatementList ; expression ; forUpdateStatementList ) statement
    | annotationList FOR ( type name IN forCollectionExpression ) statement
    | annotationList FOR
        ( annotationListNonEmpty type name IN forCollectionExpression )
        statement
    ;

forInitStatementList
    : /* empty */ | forInitStatementListNonEmpty
    ;

forInitStatementListNonEmpty
    : forInitStatement | forInitStatementListNonEmpty , forInitStatement
```

```
    ;

forInitStatement
    : annotationList type name initializerOpt | forUpdateStatement
    ;

forUpdateStatementList
    : /* empty */ | forUpdateStatementListNonEmpty
    ;

forUpdateStatementListNonEmpty
    : forUpdateStatement | forUpdateStatementListNonEmpty , forUpdateStatement
    ;

forUpdateStatement
    : lvalue ( argumentList ) | lvalue < typeArgumentList > ( argumentList )
    | lvalue assignop expression
    ;

forCollectionExpression
    : expression | expression .. expression
    ;
```

There are two forms of for statements:

- Basic 3-clause for statements with initialization, condition, and update.
- for-in statements that iterate over a range, list, or header stack.

A break; statement may be executed in a loop body to immediately exit the loop, without executing the rest of the body or the update statements. break; statements are explained in detail in Section 13.11.

A continue; statement may be executed in a loop body to skip the rest of the current loop body, executing the update statements and then evaluating the condition to either execute the loop body again, or terminate the loop. continue; statements are explained in detail in Section 13.12.

The scope of any declaration in a for statement is limited to the for statement and its body.

After type checking, a for statement has the form:

```
forStatementIR
    : annotationList FOR
        (
          forInitStatementListIR ; typedExpressionIR ; forUpdateStatementListIR
        ) statementIR
    | annotationList FOR ( typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    | annotationList FOR
        ( annotationList typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    ;
```

13.10.1. 3-clause for statements

The basic 3-clause for statement is similar to a C for statement. The init statements will be executed prior to executing the loop. The condition will be evaluated before each iteration, to determine if the loop should exit. If the condition is false, the loop will exit without executing any statements from the loop body or update. The update statements will be executed after the loop body and before evaluating the condition again for the next iteration.

13.10.1.1. Type checking

After type checking, a 3-clause for statement has the form:

```
forStatementIR
    : annotationList FOR
        (
          forInitStatementListIR ; typedExpressionIR ; forUpdateStatementListIR
        ) statementIR
    | annotationList FOR ( typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    | annotationList FOR
        ( annotationList typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    ;

forInitStatementListIR = forInitStatementIR*

forInitStatementIR
    : annotationList typeIR nameIR initializerOptIR | forUpdateStatementIR
    ;

forUpdateStatementListIR = forUpdateStatementIR*

forUpdateStatementIR
    : callableTargetIR < typeArgumentListIR > ( argumentListIR )
    | typedLvalueIR assignop typedExpressionIR
    ;
```

Typing annotationList FOR `( forInitStatementList ';' expression ';' forUpdateStatementList `) statement, under context TC at LOCAL:

1. Let TC_1 be TC with a new local frame.
2. Let context TC_2 and forInitStatementIR* be the result of typing forInitStatementList under context TC_1.
3. Let typedExpressionIR be the result of typing expression, under context TC_2 at LOCAL.
4. Let typeIR be the type of typedExpressionIR.
5. If BOOL is equal to typeIR with typedefs unrolled:
    a. Let forUpdateStatementIR* be the result of typing forUpdateStatementList under context TC_2.
    b. Let TC_3 be TC_2 within a loop.
    c. Let context TC_4 and statementIR be the result of typing statement, under context TC_3 at LOCAL.
    d. Let TC_5 be TC_4 with the innermost frame removed.
    e. Let TC_6 be TC_5 with control flow set to TC_2.CONTROL.
    f. Let TC_7 be TC_6 with loop status set to TC_2.LOOP.
    g. Let forStatementIR be annotationList FOR `( forInitStatementIR* ';' typedExpressionIR ';' forUpdateStatementIR* `) statementIR.
    h. The result is context TC_7 and forStatementIR.

A list of initialization statements are type checked with the relation:

ForInitStmtList_ok:

- Typing forInitStatementList under context TC:
- The result is context TC and forInitStatementIR*.

Typing forInitStatementList under context TC_0:

1. Let forInitStatement* be $flatten_forInitStatementList(forInitStatementList).
2. Let context TC_1 and forInitStatementIR* be the result of typing forInitStatement* under context TC_0.
3. The result is context TC_1 and forInitStatementIR*.

ForInitStmts_ok:

- Typing forInitStatement* under context TC:
- The result is context TC and forInitStatementIR*.

Typing forInitStatement* under context TC:

1. If forInitStatement* matches pattern []:
   a. The result is context TC and ·.
2. Else:
   a. Let $forInitStatement_h$ :: $forInitStatement_t^*$ be forInitStatement*.
   b. Let context TC_1 and $forInitStatementIR_h$ be the result of typing $forInitStatement_h$ under context TC.
   c. Let context TC_2 and $forInitStatementIR_t^*$ be the result of typing $forInitStatement_t^*$ under context TC_1.
   d. The result is context TC_2 and $forInitStatementIR_h$ :: $forInitStatementIR_t^*$.

This relation invokes the following relation for each initialization statement:

ForInitStmt_ok:

- Typing forInitStatement under context TC:
- The result is context TC and forInitStatementIR.

Typing forInitStatement under context TC:

1. If let annotationList type name initializerOpt be forInitStatement:
    a. If initializerOpt matches pattern $_{\text{EMPTY}}$:
        i. Let typeIR and fresh type variables typeId* be the result of typing type, under context TC at LOCAL.
        ii. Check that typeId* is an empty list.
        iii. Let bound be bound type variables from the LOCAL layer of TC.
        iv. If typeIR is a well-formed type, with bound type variables bound:
            A. Check that typeIR supports assignment.
            B. Let nameIR be the name of name.
            C. Let TC_1 be TC where nameIR to INOUT typeIR DYN · is added to the LOCAL layer.
            D. The result is context TC_1 and annotationList typeIR nameIR ·.
    b. Else:
        i. Let '=' $\text{expression}_{\text{init}}$ be initializerOpt.
        ii. Let typeIR and fresh type variables typeId* be the result of typing type, under context TC at LOCAL.
        iii. Check that typeId* is an empty list.
        iv. Let bound' be bound type variables from the LOCAL layer of TC.
        v. If typeIR is a well-formed type, with bound type variables bound':
            A. Check that typeIR supports assignment.
            B. Let $\text{typedExpressionIR}_{\text{init}}$ be the result of typing $\text{expression}_{\text{init}}$, under context TC at LOCAL.
            C. Let typedExpressionIR? be $\text{typedExpressionIR}_{\text{init}}$ implicitly cast to typeIR.
            D. Check that typedExpressionIR? is defined.
            E. Let $\text{typedExpressionIR}_{\text{init_cast}}$ be typedExpressionIR?.
            F. Let nameIR be the name of name.
            G. Let TC_1 be TC where nameIR to INOUT typeIR DYN · is added to the LOCAL layer.
            H. The result is context TC_1 and annotationList typeIR nameIR '=' $\text{typedExpressionIR}_{\text{init_cast}}$.
2. Else:
    a. Let forUpdateStatement be forInitStatement.
    b. Let forUpdateStatementIR be the result of typing forUpdateStatement under context TC.
    c. The result is context TC and forUpdateStatementIR.

A list of update statements are type checked with the relation:

ForUpdateStmtList_ok:

- Typing forUpdateStatementList under context TC:
- The result is forUpdateStatementIR*.

Typing forUpdateStatementList under context TC:

1. Let forUpdateStatement* be $flatten_forUpdateStatementList(forUpdateStatementList).
2. For each forUpdateStatement in forUpdateStatement*:
   - Let forUpdateStatementIR be the result of typing forUpdateStatement under context TC.

   Let forUpdateStatementIR* be the resulting list.
3. The result is forUpdateStatementIR*.

This relation invokes the following relation for each update statement:

ForUpdateStmt_ok:

- Typing forUpdateStatement under context TC:
- The result is forUpdateStatementIR.

Typing forUpdateStatement under context TC:

1. If let $\text{lvalue}_{\text{callable}}$ `( argumentList `) be forUpdateStatement:
   a. Let context _ and statementIR be the result of typing $\text{lvalue}_{\text{callable}}$ `( argumentList `) ';', under context TC at LOCAL.
   b. Check that statementIR has type callStatementIR.
   c. Let callableTargetIR `< typeArgumentIR* `> `( argumentIR* `) ';' be statementIR.
   d. The result is callableTargetIR `< typeArgumentIR* `> `( argumentIR* `).
2. Else if let $\text{lvalue}_{\text{callable}}$ `< typeArgumentList `> `( argumentList `) be forUpdateStatement:
   a. Let context _ and statementIR be the result of typing $\text{lvalue}_{\text{callable}}$ `< typeArgumentList `> `( argumentList `) ';', under context TC at LOCAL.
   b. Check that statementIR has type callStatementIR.
   c. Let callableTargetIR `< typeArgumentIR* `> `( argumentIR* `) ';' be statementIR.
   d. The result is callableTargetIR `< typeArgumentIR* `> `( argumentIR* `).
3. Else:
   a. Let lvalue assignop expression be forUpdateStatement.
   b. Let context _ and statementIR be the result of typing lvalue assignop expression ';', under context TC at LOCAL.
   c. Check that statementIR has type assignmentStatementIR.
   d. Let typedLvalueIR assignop' typedExpressionIR ';' be statementIR.
   e. Check that assignop' is equal to assignop.
   f. The result is typedLvalueIR assignop typedExpressionIR.

13.10.1.2. Compile-time evaluation

Compile-time evaluation of annotationList FOR `( forInitStatementListIR ';' typedExpressionIR ';' forUpdateStatementListIR `) statementIR, under context IC and store STO at p:

1. Let context IC_{body}, store STO_1, and $statementIR_{body}$ be the result of compile-time evaluation of statementIR, under context IC and store STO at p.
2. Let forStatementIR be annotationList FOR `( forInitStatementListIR ';' typedExpressionIR ';' forUpdateStatementListIR `) $statementIR_{body}$.
3. The result is context IC, store STO_1, and forStatementIR.

13.10.1.3. Runtime evaluation

Runtime evaluation of annotationList FOR `( forInitStatementListIR ';' typedExpressionIR ';' forUpdateStatementListIR `) statementIR, under context EC and state ARCH at LOCAL:

1. Let EC_1 be EC with a new local frame.
2. Let the updated evaluation context EC_2, state $ARCH_1$, and statementResult' be the result of runtime evaluation of forInitStatementListIR, under evaluation context EC_1 and state ARCH.
3. If let abortResult of type abortResult be statementResult':
   a. Let EC_3 be EC_2 with the innermost frame removed.
   b. The result is context EC_3, state $ARCH_1$, and abortResult.
4. Else if let continueEmptyResult of type continueEmptyResult be statementResult':
   a. Let the updated evaluation context EC_3, state $ARCH_2$, and statementResult be the result of looping statementIR with condition typedExpressionIR and updates forUpdateStatementListIR, under evaluation context EC_2 and state $ARCH_1$.
   b. Let EC_4 be EC_3 with the innermost frame removed.
   c. The result is context EC_4, state $ARCH_2$, and statementResult.

A list of initialization statements is evaluated with the relation:

ForInitStmts_eval:

- Runtime evaluation of forInitStatementListIR, under evaluation context EC and state ARCH:
- The result is
  - the updated evaluation context EC, state ARCH, and statementResult.

Runtime evaluation of $forInitStatementIR^*$, under evaluation context EC and state ARCH:

1. If forInitStatementIR* matches pattern []:
    a. The result is
        - the updated evaluation context EC, state ARCH, and $_{\text{EMPTY}}$.
2. Else:
    a. Let $\text{forInitStatementIR}_h$ :: $\text{forInitStatementIR}_t^*$ be forInitStatementIR*.
    b. Let the updated evaluation context EC_1, state ARCH_1, and statementResult' be the result of runtime evaluation of $\text{forInitStatementIR}_h$, under evaluation context EC and state ARCH.
    c. If let abortResult of type abortResult be statementResult':
        i. The result is
            - the updated evaluation context EC_1, state ARCH_1, and abortResult.
    d. Else if let continueEmptyResult of type continueEmptyResult be statementResult':
        i. Let the updated evaluation context EC_2, state ARCH_2, and statementResult be the result of runtime evaluation of $\text{forInitStatementIR}_t^*$, under evaluation context EC_1 and state ARCH_1.
        ii. The result is
            - the updated evaluation context EC_2, state ARCH_2, and statementResult.

An initialization statement is evaluated with the relation:

ForInitStmt_eval:

- Runtime evaluation of forInitStatementIR, under evaluation context EC and state ARCH:
- The result is
    - the updated evaluation context EC, state ARCH, and statementResult.

Runtime evaluation of forInitStatementIR, under evaluation context EC_0 and state ARCH:

1. If forInitStatementIR matches pattern % % % %:
    a. Try:
        i. Let annotationList typeIR nameIR $\text{initializerIR}^?$ be forInitStatementIR.
        ii. If $\text{initializerIR}^?$ is none:
            A. Let $\text{typeIR}_{\text{subst}}$ be typeIR substituted by bound type variables in EC_0 from the LOCAL layer.
            B. Let $\text{value}_{\text{init}}$ be the default value for type $\text{typeIR}_{\text{subst}}$.
            C. Let EC_1 be EC_0 where $_{\text{BARE}}$ nameIR to $\text{value}_{\text{init}}$ is added to the LOCAL layer.
            D. The result is
                - the updated evaluation context EC_1, state ARCH, and $_{\text{EMPTY}}$.
    b. Then, try:
        i. Let annotationList typeIR nameIR initializerOptIR be forInitStatementIR.
        ii. Check that initializerOptIR is defined.
        iii. Let '=' $\text{typedExpressionIR}_{\text{init}}$ be initializerOptIR.

iv. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating $typedExpressionIR_{init}$, under context EC_0 and state ARCH at LOCAL.

v. If let abortResult of type abortResult be expressionResult:

A. The result is

- the updated evaluation context EC_1, state $ARCH_1$, and abortResult.

vi. Else:

A. Let $_{CONT}\ value_{init}$ be expressionResult.

B. Let EC_2 be EC_1 where $_{BARE}\ nameIR$ to $value_{init}$ is added to the LOCAL layer.

C. The result is

- the updated evaluation context EC_2, state $ARCH_1$, and $_{EMPTY}$.

2. Else:

a. Let forUpdateStatementIR be forInitStatementIR.

b. Let the updated evaluation context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of forUpdateStatementIR, under evaluation context EC_0 and state ARCH.

c. The result is

- the updated evaluation context EC_1, state $ARCH_1$, and statementResult.

A list of update statements is evaluated with the relation:

ForUpdateStmts_eval:

- Runtime evaluation of forUpdateStatementListIR, under evaluation context EC and state ARCH:
- The result is
  - the updated evaluation context EC, state ARCH, and statementResult.

Runtime evaluation of $forUpdateStatementIR^*$, under evaluation context EC and state ARCH:

1. If $forUpdateStatementIR^*$ matches pattern []:

a. The result is

- the updated evaluation context EC, state ARCH, and $_{EMPTY}$.

2. Else:

a. Let $forUpdateStatementIR_h :: forUpdateStatementIR_t^*$ be $forUpdateStatementIR^*$.

b. Let the updated evaluation context EC_1, state $ARCH_1$, and statementResult' be the result of runtime evaluation of $forUpdateStatementIR_h$, under evaluation context EC and state ARCH.

c. If let exitResult of type exitResult be statementResult':

i. The result is

- the updated evaluation context EC_1, state $ARCH_1$, and EXIT.

d. Else if let continueEmptyResult of type continueEmptyResult be statementResult':

i. Let the updated evaluation context EC_2, state $ARCH_2$, and statementResult be the result of runtime evaluation of $forUpdateStatementIR_t^*$, under evaluation context EC_1 and state $ARCH_1$.

ii. The result is

- the updated evaluation context EC_2, state $ARCH_2$, and statementResult.

An update statement is evaluated with the relation:

ForUpdateStmt_eval:

- Runtime evaluation of forUpdateStatementIR, under evaluation context EC and state ARCH:
- The result is
  - the updated evaluation context EC, state ARCH, and statementResult.

Runtime evaluation of forUpdateStatementIR, under evaluation context EC_0 and state $ARCH_0$:

1. If let callableTargetIR `< typeArgumentListIR `> `( argumentListIR `) be forUpdateStatementIR:
   a. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of callableTargetIR `< typeArgumentListIR `> `( argumentListIR `) ';', under context EC_0 and state $ARCH_0$ at LOCAL.
   b. The result is
      - the updated evaluation context EC_1, state $ARCH_1$, and statementResult.
2. Else:
   a. Let typedLvalueIR assignop typedExpressionIR be forUpdateStatementIR.
   b. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of typedLvalueIR assignop typedExpressionIR ';', under context EC_0 and state $ARCH_0$ at LOCAL.
   c. The result is
      - the updated evaluation context EC_1, state $ARCH_1$, and statementResult.

With these, the 3-clause for statement is evaluated with the relation:

ForThreeClauseStmt_eval:

- Looping statementIR with condition typedExpressionIR and updates forUpdateStatementListIR, under evaluation context EC and state ARCH:
- The result is
  - the updated evaluation context EC, state ARCH, and statementResult.

Looping statementIR with condition typedExpressionIR and updates forUpdateStatementListIR, under evaluation context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$ at LOCAL.
2. Try:
    a. If let abortResult of type abortResult be expressionResult:
        i. The result is
            - the updated evaluation context EC_1, state $ARCH_1$, and abortResult.
3. Then, try:
    a. If expressionResult is equal to $_{CONT\ B}$ false:
        i. The result is
            - the updated evaluation context EC_1, state $ARCH_1$, and $_{EMPTY}$.
    b. Else if expressionResult is equal to $_{CONT\ B}$ true:
        i. Let context EC_2, state $ARCH_2$, and statementResult' be the result of runtime evaluation of statementIR, under context EC_1 and state $ARCH_1$ at LOCAL.
        ii. Try:
            A. If let abortResult of type abortResult be statementResult':
                I. The result is
                    - the updated evaluation context EC_2, state $ARCH_2$, and abortResult.
            B. Else if let returnResult of type returnResult be statementResult':
                I. The result is
                    - the updated evaluation context EC_2, state $ARCH_2$, and returnResult.
            C. Else if let forBreakResult of type forBreakResult be statementResult':
                I. The result is
                    - the updated evaluation context EC_2, state $ARCH_2$, and $_{EMPTY}$.
        iii. Then, try:
            A. Check that statementResult' is equal to $_{EMPTY}$ or statementResult' is equal to CONTINUE.
            B. Let the updated evaluation context EC_3, state $ARCH_3$, and statementResult'' be the result of runtime evaluation of forUpdateStatementListIR, under evaluation context EC_2 and state $ARCH_2$.
            C. If let exitResult of type exitResult be statementResult'':
                I. The result is
                    - the updated evaluation context EC_3, state $ARCH_3$, and EXIT.
            D. Else if let continueEmptyResult of type continueEmptyResult be statementResult'':
                I. Let the updated evaluation context EC_4, state $ARCH_4$, and statementResult be the result of looping statementIR with condition typedExpressionIR and updates forUpdateStatementListIR, under evaluation context EC_3 and state $ARCH_3$.
                II. The result is
                    - the updated evaluation context EC_4, state $ARCH_4$, and statementResult.

13.10.2. for-in statements

The for-in statement executes the body once for each value in a range or each element in a list expression or header stack. The list or range expression itself will only be evaluated once, before the first iteration of the loop. All side effects in the list or range expression will occur before the first iteration of the loop body.

13.10.2.1. Type checking

After type checking, a for-in statement has the form:

```
forStatementIR
    : annotationList FOR
        (
          forInitStatementListIR ; typedExpressionIR ; forUpdateStatementListIR
        ) statementIR
    | annotationList FOR ( typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    | annotationList FOR
        ( annotationList typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    ;

forCollectionExpressionIR
    : typedExpressionIR | typedExpressionIR .. typedExpressionIR
    ;
```

Typing forStatement, under context TC at LOCAL:

1. If let annotationList FOR `( type name IN forCollectionExpression `) statement be forStatement:
    a. Let typeIR and fresh type variables typeId* be the result of typing type, under context TC at LOCAL.
    b. Check that typeId* is an empty list.
    c. Let bound be bound type variables from the LOCAL layer of TC.
    d. If typeIR is a well-formed type, with bound type variables bound:
        i. Check that typeIR supports assignment.
        ii. Let forCollectionExpressionIR be the result of typing forCollectionExpression under context TC with iterator type typeIR.
        iii. Let TC_1 be TC with a new local frame.
        iv. Let nameIR be the name of name.
        v. Let TC_2 be TC_1 where nameIR to INOUT typeIR DYN · is added to the LOCAL layer.
        vi. Let TC_3 be TC_2 within a loop.
        vii. Let context TC_4 and statementIR be the result of typing statement, under context TC_3 at LOCAL.
        viii. Let TC_5 be TC_4 with the innermost frame removed.
        ix. Let TC_6 be TC_5 with control flow set to TC_2.CONTROL.
        x. Let TC_7 be TC_6 with loop status set to TC_2.LOOP.
        xi. Let forStatementIR be annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR `) statementIR.
        xii. The result is context TC_7 and forStatementIR.
2. Else if let annotationList FOR `( annotationListNonEmpty type name IN forCollectionExpression `) statement be forStatement:
    a. Let context TC_1 and statementIR be the result of typing annotationList FOR `( type name IN forCollectionExpression `) statement, under context TC at LOCAL.
    b. Check that statementIR has type forStatementIR.
    c. Let forStatementIR be statementIR.

d. The result is context TC_1 and forStatementIR.

Collection expressions are type checked with the relation:

ForCollectionExpr_ok:

- Typing forCollectionExpression under context TC with iterator type typeIR:
- The result is forCollectionExpressionIR.

ForCollectionExpr_ok dispatch:

1. If let expression of type expression be forCollectionExpression: goto expression
2. Else:
   a. Let $expression_l$ '..' $expression_r$ be forCollectionExpression.
   b. Goto range

Typing expression under context TC with iterator type $typeIR_{var}$:

1. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
2. Let $typeIR_{expression}$ be the type of typedExpressionIR.
3. Let typeIR be $typeIR_{expression}$ with typedefs unrolled.
4. If let LIST `< $typeIR_e$ `> of type listTypeIR be typeIR:
   a. Let casts castResult be the result of checking if $typeIR_e$ can be implicitly cast to $typeIR_{var}$.
   b. Check that castResult matches pattern OK %.
   c. Let OK $typeIR_{cast}^*$ be castResult.
   d. For each $typeIR_{cast}$ in $typeIR_{cast}^*$:

      - Let $typeIR_{cast_list}$ be LIST `< $typeIR_{cast}$ `>.

      Let $typeIR_{cast_list}^*$ be the resulting list.

   e. Let $typedExpressionIR_{cast}$ be typedExpressionIR cast through $typeIR_{cast_list}^*$.
   f. The result is $typedExpressionIR_{cast}$.
5. Else if let ARRAY $typeIR_e$ `[ _ `] of type arrayTypeIR be typeIR:
   a. If $typeIR_e$ and $typeIR_{var}$ are the same type:
      i. The result is typedExpressionIR.
6. Else if let HEADER_STACK $typeIR_e$ `[ _ `] of type headerStackTypeIR be typeIR:
   a. If $typeIR_e$ and $typeIR_{var}$ are the same type:
      i. The result is typedExpressionIR.

Typing expression$_l$ '..' expression$_r$ under context TC with iterator type typeIR$_{var}$:

1. Let typedExpressionIR$_l$ be the result of typing expression$_l$, under context TC at LOCAL.
2. Let typedExpressionIR$_r$ be the result of typing expression$_r$, under context TC at LOCAL.
3. Let typedExpressionIR$^?$ be typedExpressionIR$_l$ implicitly cast to typeIR$_{var}$.
4. Check that typedExpressionIR$^?$ is defined.
5. Let typedExpressionIR$_{l_casted}$ be typedExpressionIR$^?$.
6. Let typedExpressionIR'$^?$ be typedExpressionIR$_r$ implicitly cast to typeIR$_{var}$.
7. Check that typedExpressionIR'$^?$ is defined.
8. Let typedExpressionIR$_{r_casted}$ be typedExpressionIR'$^?$.
9. Let typeIR$_l$ be the type of typedExpressionIR$_{l_casted}$.
10. Let typeIR$_r$ be the type of typedExpressionIR$_{r_casted}$.
11. Check that typeIR$_l$ and typeIR$_r$ can be used as range bounds.
12. The result is typedExpressionIR$_{l_casted}$ '..' typedExpressionIR$_{r_casted}$.

13.10.2.2. Compile-time evaluation

Compile-time evaluation of forStatementIR', under context IC and store STO at p:

1. If let annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR `) statementIR be forStatementIR':
   a. Let context IC$_{body}$, store STO$_1$, and statementIR$_{body}$ be the result of compile-time evaluation of statementIR, under context IC and store STO at p.
   b. Let forStatementIR be annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR `) statementIR$_{body}$.
   c. The result is context IC, store STO$_1$, and forStatementIR.
2. Else if let annotationList FOR `( annotationList' typeIR nameIR IN forCollectionExpressionIR `) statementIR be forStatementIR':
   a. If annotationList is equal to annotationList':
      i. Let context IC$_{body}$, store STO$_1$, and statementIR$_{body}$ be the result of compile-time evaluation of statementIR, under context IC and store STO at p.
      ii. Let forStatementIR be annotationList FOR `( annotationList typeIR nameIR IN forCollectionExpressionIR `) statementIR$_{body}$.
      iii. The result is context IC, store STO$_1$, and forStatementIR.

13.10.2.3. Runtime evaluation

Runtime evaluation of forStatementIR, under context EC and state ARCH at LOCAL:

1. If let annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR `) statementIR be forStatementIR:
   a. Let EC$_1$ be EC with a new local frame.

b. Let the updated evaluation context EC_2, state $ARCH_1$, and expressionListResult be the result of runtime evaluation of forCollectionExpressionIR, under evaluation context EC_1 and state ARCH.

c. If let abortResult of type abortResult be expressionListResult:

i. Let EC_3 be EC_2 with the innermost frame removed.

ii. The result is context EC_3, state $ARCH_1$, and abortResult.

d. Else:

i. Let $_{CONT}$ value* be expressionListResult.

ii. Let $typeIR_{subst}$ be typeIR substituted by bound type variables in EC from the LOCAL layer.

iii. Let $value_{init}$ be the default value for type $typeIR_{subst}$.

iv. Let EC_3 be EC_2 where $_{BARE}$ nameIR to $value_{init}$ is added to the LOCAL layer.

v. Let the updated evaluation context EC_4, state $ARCH_2$, and statementResult be the result of looping over value* with loop variable nameIR and body statementIR, under evaluation context EC_3 and state $ARCH_1$.

vi. Let EC_5 be EC_4 with the innermost frame removed.

vii. The result is context EC_5, state $ARCH_2$, and statementResult.

2. Else if let annotationList FOR `( $annotationList_{in}$ typeIR nameIR IN forCollectionExpressionIR `) statementIR be forStatementIR:

a. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR `) statementIR, under context EC and state ARCH at LOCAL.

b. The result is context EC_1, state $ARCH_1$, and statementResult.

Collection expressions are evaluated with the relation:

ForCollectionExpr_eval:

- Runtime evaluation of forCollectionExpressionIR, under evaluation context EC and state ARCH:
- The result is
  - the updated evaluation context EC, state ARCH, and expressionListResult.

ForCollectionExpr_eval dispatch:

1. If let typedExpressionIR of type typedExpressionIR be forCollectionExpressionIR: goto typedExpressionIR
2. Else:

a. Let $typedExpressionIR_l$ '..' $typedExpressionIR_r$ be forCollectionExpressionIR.

b. Goto range

Runtime evaluation of typedExpressionIR, under evaluation context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let abortResult of type abortResult be expressionResult:

a. The result is

- the updated evaluation context EC_1, state $ARCH_1$, and abortResult.

3. Else:
   a. Let $_{CONT}$ value' be expressionResult.
   b. If let listValue of type listValue be value':
      i. Let LIST `[ value* `] be listValue.
      ii. The result is
      - the updated evaluation context EC_1, state $ARCH_1$, and $_{CONT}$ value*.

   c. Else if let arrayValue of type arrayValue be value':
      i. Let ARRAY `[ value* `] be arrayValue.
      ii. The result is
      - the updated evaluation context EC_1, state $ARCH_1$, and $_{CONT}$ value*.

   d. Else if let headerStackValue of type headerStackValue be value':
      i. Let HEADER_STACK `[ value* `(_ `) `] be headerStackValue.
      ii. The result is
      - the updated evaluation context EC_1, state $ARCH_1$, and $_{CONT}$ value*.

Runtime evaluation of typedExpressionIR$_l$ '..' typedExpressionIR$_r$, under evaluation context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR$_l$, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let abortResult of type abortResult be expressionResult:
   a. The result is
   - the updated evaluation context EC_1, state $ARCH_1$, and abortResult.

3. Else:
   a. Let $_{CONT}$ value$_l$ be expressionResult.
   b. Let context EC_2, state $ARCH_2$ and expressionResult' be the result of evaluating typedExpressionIR$_r$, under context EC_1 and state $ARCH_1$ at LOCAL.
   c. If let abortResult of type abortResult be expressionResult':
      i. The result is
      - the updated evaluation context EC_2, state $ARCH_2$, and abortResult.

   d. Else:
      i. Let $_{CONT}$ value$_r$ be expressionResult'.
      ii. Let typeIR be the type of typedExpressionIR$_l$.
      iii. Let value* be the values in SET `{ value$_l$ '..' value$_r$ `} of type SET `< typeIR `>.
      iv. The result is
      - the updated evaluation context EC_2, state $ARCH_2$, and $_{CONT}$ value*.

A range collection is enumerated into the sequence of values to iterate over with:

The values in setValue of type SET `< typeIR `>

1. Check that setValue matches pattern SET `{ % '..' % `}.
2. Let SET `{ $value_l$ '..' $value_r$ `} be setValue.
3. Try:
    a. If $value_l$ '<' $value_r$:
        i. Let $value_{one}$ be D 1 casted to type typeIR.
        ii. Let $value_{l_inc}$ be $value_l$ '+' $value_{one}$.
        iii. Let $value^*$ be the values in SET `{ $value_{l\_inc}$ '..' $value_r$ `} of type SET `< typeIR `>.
        iv. Return $value_l$ :: $value^*$.
4. Then, try:
    a. If $value_l$ '==' $value_r$: return $value_l$.
5. Then, try:
    a. Check that $value_l$ '>' $value_r$.
    b. Return ·.

With these, the for-in statement is evaluated with the relation:

ForInStmt_eval:

- Looping over $value^*$ with loop variable nameIR and body statementIR, under evaluation context EC and state ARCH:
- The result is
    - the updated evaluation context EC, state ARCH, and statementResult.

Looping over $value^*$ with loop variable nameIR and body statementIR, under evaluation context EC and state ARCH:

1. If $value^*$ matches pattern []:
    a. The result is
        - the updated evaluation context EC, state ARCH, and $_{EMPTY}$.
2. Else:
    a. Let $value_h$ :: $value_t^*$ be $value^*$.
    b. Let EC_1 be EC where $_{BARE}$ nameIR is updated to $value_h$ from the LOCAL layer.
    c. Let context EC_2, state $ARCH_1$, and statementResult' be the result of runtime evaluation of statementIR, under context EC_1 and state ARCH at LOCAL.
    d. Try:
        i. If let abortResult of type abortResult be statementResult':
            A. The result is
                - the updated evaluation context EC_2, state $ARCH_1$, and abortResult.
        ii. Else if let returnResult of type returnResult be statementResult':
            A. The result is
                -

the updated evaluation context EC_2, state $ARCH_1$, and returnResult.

iii. Else if let forBreakResult of type forBreakResult be statementResult':

A. The result is

- the updated evaluation context EC_2, state $ARCH_1$, and $_{EMPTY}$.

e. Then, try:

i. Check that statementResult' is equal to $_{EMPTY}$ or statementResult' is equal to CONTINUE.

ii. Let the updated evaluation context EC_3, state $ARCH_2$, and statementResult be the result of looping over $value_t^*$ with loop variable nameIR and body statementIR, under evaluation context EC_2 and state $ARCH_1$.

iii. The result is

- the updated evaluation context EC_3, state $ARCH_2$, and statementResult.

13.11. Break statements

A break; statement exists the enclosing for loop.

```
breakStatement
    : BREAK ;
    ;
```

13.11.1. Type checking

After type checking, a break; statement has the form:

```
breakStatementIR = breakStatement
```

Typing breakStatement, under context TC at LOCAL:

1. Check that TC.LOOP is IN.
2. The result is context TC and breakStatement.

13.11.2. Compile-time evaluation

Compile-time evaluation of breakStatementIR, under context IC and store STO at p:

1. The result is context IC, store STO, and breakStatementIR.

13.11.3. Runtime evaluation

A break; statement yields the result:

```
forBreakResult
    : BREAK
    ;
```

Runtime evaluation of BREAK ';', under context EC and state ARCH at LOCAL:

1. The result is context EC, state ARCH, and BREAK.

13.12. Continue statements

A continue; statement skips the current iteration of the enclosing for loop.

```
continueStatement
    : CONTINUE ;
    ;
```

13.12.1. Type checking

After type checking, a continue; statement has the form:

```
continueStatementIR = continueStatement
```

Typing continueStatement, under context TC at LOCAL:

1. Check that TC.LOOP is IN.
2. The result is context TC and continueStatement.

13.12.2. Compile-time evaluation

Compile-time evaluation of continueStatementIR, under context IC and store STO at p:

1. The result is context IC, store STO, and continueStatementIR.

13.12.3. Runtime evaluation

A continue; statement yields the result:

```
forContinueResult
```

```
    : CONTINUE
    ;
```

Runtime evaluation of CONTINUE ';', under context EC and state ARCH at LOCAL:

1. The result is context EC, state ARCH, and CONTINUE.

13.13. Switch statements

The switch statement can only be used within control blocks, action bodies, or function bodies.

```
switchStatement
    : SWITCH ( expression ) { switchCaseList }
    ;

switchCaseList
    : /* empty */ | switchCaseList switchCase
    ;

switchCase
    : switchLabel : blockStatement | switchLabel :
    ;

switchLabel
    : DEFAULT | expressionNonBrace
    ;
```

The expressionNonBrace is the same as expression as defined in Chapter 14, except it does not include any cases that can begin with a left brace { character, to avoid syntactic ambiguity with a block statement.

expressionNonBrace and expression can be converted to each other by:

The expression corresponding to expressionNonBrace'

1. If let literalExpression of type literalExpression be expressionNonBrace': return literalExpression.
2. Else if let referenceExpression of type referenceExpression be expressionNonBrace': return referenceExpression.
3. Else if let unaryExpression of type unaryExpression be expressionNonBrace': return unaryExpression.
4. Else if let $\text{expressionNonBrace}_l$ binop expression_r of type binaryExpressionNonBrace be expressionNonBrace':
    a. Let expression_l be the expression corresponding to $\text{expressionNonBrace}_l$.
    b. Return expression_l binop expression_r.
5. Else if let $\text{expressionNonBrace}_{cond}$ '?' expression_{true} ':' $\text{expression}_{false}$ of type ternaryExpressionNonBrace be expressionNonBrace':
    a. Let expression_{cond} be the expression corresponding to $\text{expressionNonBrace}_{cond}$.
    b. Return expression_{cond} '?' expression_{true} ':' $\text{expression}_{false}$.
6. Else if let castExpression of type castExpression be expressionNonBrace': return castExpression.

7. Else if let errorAccessExpression of type errorAccessExpression be expressionNonBrace': return errorAccessExpression.
8. Else if let memberAccessBaseNonBrace '.' member of type memberAccessExpressionNonBrace be expressionNonBrace':
    a. If let prefixedTypeName of type prefixedTypeName be memberAccessBaseNonBrace: return prefixedTypeName '.' member.
    b. Else:
        i. Let $\text{expressionNonBrace}_{\text{base}}$ be memberAccessBaseNonBrace.
        ii. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{expressionNonBrace}_{\text{base}}$.
        iii. Return $\text{expression}_{\text{base}}$ '.' member.
9. Else if let $\text{expressionNonBrace}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `] of type indexAccessExpressionNonBrace be expressionNonBrace':
    a. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{expressionNonBrace}_{\text{base}}$.
    b. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `].
10. Else if let $\text{expressionNonBrace}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ sliceop $\text{expression}_{\text{lo}}$ `] of type sliceAccessExpressionNonBrace be expressionNonBrace':
    a. If sliceop matches pattern ':':
        i. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{expressionNonBrace}_{\text{base}}$.
        ii. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ ':' $\text{expression}_{\text{lo}}$ `].
    b. Else:
        i. Let $\text{expression}_{\text{base}}$ be the expression corresponding to $\text{expressionNonBrace}_{\text{base}}$.
        ii. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ '+:' $\text{expression}_{\text{lo}}$ `].
11. Else if let callExpressionNonBrace of type callExpressionNonBrace be expressionNonBrace':
    a. If let callTargetNonBrace `( argumentList `) be callExpressionNonBrace:
        i. If let expressionNonBrace of type expressionNonBrace be callTargetNonBrace:
            A. Let expression be the expression corresponding to expressionNonBrace.
            B. Return expression `( argumentList `).
        ii. Else:
            A. Let constructorTarget be callTargetNonBrace.
            B. Return constructorTarget `( argumentList `).
    b. Else:
        i. Let expressionNonBrace `< realTypeArgumentList `> `( argumentList `) be callExpressionNonBrace.
        ii. Let expression be the expression corresponding to expressionNonBrace.
        iii. Return expression `< realTypeArgumentList `> `( argumentList `).
12. Else:
    a. Let parenthesizedExpression be expressionNonBrace'.
    b. Return parenthesizedExpression.

The non-brace expression corresponding to expression'

1. If let literalExpression of type literalExpression be expression': return literalExpression.
2. Else if let referenceExpression of type referenceExpression be expression': return referenceExpression.
3. Else if let unaryExpression of type unaryExpression be expression': return unaryExpression.

4. Else if let expression$_l$ binop expression$_r$ of type binaryExpression be expression':
    a. Let expressionNonBrace$_l$ be the non-brace expression corresponding to expression$_l$.
    b. Return expressionNonBrace$_l$ binop expression$_r$.
5. Else if let expression$_{cond}$ '?' expression$_{true}$ ':' expression$_{false}$ of type ternaryExpression be expression':
    a. Let expressionNonBrace$_{cond}$ be the non-brace expression corresponding to expression$_{cond}$.
    b. Return expressionNonBrace$_{cond}$ '?' expression$_{true}$ ':' expression$_{false}$.
6. Else if let castExpression of type castExpression be expression': return castExpression.
7. Else if let errorAccessExpression of type errorAccessExpression be expression': return errorAccessExpression.
8. Else if let memberAccessBase '.' member of type memberAccessExpression be expression':
    a. If let prefixedTypeName of type prefixedTypeName be memberAccessBase: return prefixedTypeName '.' member.
    b. Else:
        i. Let expression$_{base}$ be memberAccessBase.
        ii. Let expressionNonBrace$_{base}$ be the non-brace expression corresponding to expression$_{base}$.
        iii. Return expressionNonBrace$_{base}$ '.' member.
9. Else if let expression$_{base}$ `[ expression$_{index}$ `] of type indexAccessExpression be expression':
    a. Let expressionNonBrace$_{base}$ be the non-brace expression corresponding to expression$_{base}$.
    b. Return expressionNonBrace$_{base}$ `[ expression$_{index}$ `].
10. Else if let expression$_{base}$ `[ expression$_{hi}$ sliceop expression$_{lo}$ `] of type sliceAccessExpression be expression':
    a. Check that sliceop is ':'.
    b. Let expressionNonBrace$_{base}$ be the non-brace expression corresponding to expression$_{base}$.
    c. Return expressionNonBrace$_{base}$ `[ expression$_{hi}$ ':' expression$_{lo}$ `].
11. Else if let callExpression of type callExpression be expression':
    a. If let callTarget `( argumentList `) be callExpression:
        i. If let expression of type expression be callTarget:
            A. Let expressionNonBrace be the non-brace expression corresponding to expression.
            B. Return expressionNonBrace `( argumentList `).
        ii. Else:
            A. Let constructorTarget be callTarget.
            B. Return constructorTarget `( argumentList `).
    b. Else:
        i. Let expression `< realTypeArgumentList `> `( argumentList `) be callExpression.
        ii. Let expressionNonBrace be the non-brace expression corresponding to expression.
        iii. Return expressionNonBrace `< realTypeArgumentList `> `( argumentList `).
12. Else if let parenthesizedExpression of type parenthesizedExpression be expression': return parenthesizedExpression.

There are two kinds of switch expressions allowed, described separately in the following two subsections.

13.13.1. Notes common to all switch statements

It is a compile-time error if two labels of a switch statement equal each other. The switch label values need not include all possible values of the switch expression. It is optional to have a switch case with the default

label, but if one is present, it must be the last one in the switch statement.

The requirement that any default label appear last is checked with:

Default label is only in the last position of switchLabelIR*

1. If switchLabelIR* matches pattern []: return true.
2. Else:
   a. Let switchLabelIR$_h$ :: switchLabelIR$_t^*$ be switchLabelIR*.
   b. If switchLabelIR$_h$ is DEFAULT:
      i. Return switchLabelIR$_t^*$ is equal to ·.
   c. Else:
      i. Return default label is only in the last position of switchLabelIR$_t^*$.

If a switch label is not followed by a block statement, it falls through to the next label. However, if a block statement is present, it does not fall through. Note that this is different from C-style switch statements, where a break is needed to prevent fall-through. If the last switch label is not followed by a block statement, the behavior is the same as if the last switch label were followed by an empty block statement { }.

The label of a switch case, and whether a case is non-fall-through (that is, has a block statement), are obtained with:

The label of switchCaseIR

1. If let switchLabelIR ':' _ be switchCaseIR: return switchLabelIR.
2. Else:
   a. Let switchLabelIR ':' be switchCaseIR.
   b. Return switchLabelIR.

switchCaseIR is not a fall-through case

1. If switchCaseIR matches pattern % ':' %: return true.
2. Else: return false.

When a switch statement is executed, first the switch expression is evaluated, and any side effects from evaluating this expression are visible to any switch case that is executed. Among switch labels that are not default, at most one of them can equal the value of the switch expression. If one is equal, that switch case is executed.

If no labels are equal to the switch expression, then:

- if there is a default label, the case with the default label is executed.
- if there is no default label, then no switch case is executed, and execution continues after the end of the switch statement, with no side effects (except any that were caused by evaluating the switch expression).

See "Implementing generalized P4_16 switch statements" for possible techniques that one might use to implement generalized switch statements.[1]

13.13.2. Switch statement on match-action table result

This type of switch statement can only be used within control apply blocks. For this variant of switch statement, the expression must be of the form t.apply().action_run, where t is the name of a table (see Section 17.3.7). All switch labels must be names of actions of the table t, or default.

```
switch (t.apply().action_run) {
   action1:                          // fall-through to action2:
   action2: { /* body omitted */ }
   action3: { /* body omitted */ }  // no fall-through from action2 to action3 labels
   default: { /* body omitted */ }
}
```

Note that the default label of the switch statement is used to match on the kind of action executed, no matter whether there was a table hit or miss. The default label does not indicate that the table missed and the default_action was executed.

13.13.2.1. Type checking

Switch_table_ok:

- Typing SWITCH `( id `) `{ switchCaseList `} under context TC:
- The result is context TC and switchCaseIR*.

Typing SWITCH `( typeId$_{table\_enum}$ `) `{ switchCaseList `} under context TC_0:

1. Let text be the text typeId$_{table_enum}$ with the suffix ")" removed.
2. Let typeId$_{table}$ be the text text with the prefix "action_list(" removed.
3. Let context TC_1 and switchCaseIR* be the result of typing switchCaseList for table typeId$_{table}$ under context TC_0.
4. The result is context TC_1 and switchCaseIR*.

Switch cases are type checked with the relation:

SwitchCaseList_table_ok:

- Typing switchCaseList for table typeId under context TC:
- The result is context TC and switchCaseListIR.

Typing switchCaseList for table typeId$_{table}$ under context TC_0:

1. Let switchCase* be switchCaseList flattened.
2. Let context TC_1 and switchCaseIR* be the result of typing switchCase* for table $typeId_{table}$ under context TC_0.
3. The result is context TC_1 and switchCaseIR*.

SwitchCases_table_ok:

- Typing switchCase* for table typeId under context TC:
- The result is context TC and switchCaseListIR.

Typing switchCase* for table $typeId_{table}$ under context TC:

1. If switchCase* matches pattern []:
   a. The result is context TC and ·.
2. Else:
   a. Let $switchCase_h$ :: $switchCase_t^*$ be switchCase*.
   b. Let context TC_1 and $switchCaseIR_h$ be the result of typing $switchCase_h$ for table $typeId_{table}$ under context TC.
   c. Let context TC_2 and $switchCaseIR_t^*$ be the result of typing $switchCase_t^*$ for table $typeId_{table}$ under context TC_1.
   d. The result is context TC_2 and $switchCaseIR_h$ :: $switchCaseIR_t^*$.

A switch case is type checked with:

SwitchCase_table_ok:

- Typing switchCase for table typeId under context TC:
- The result is context TC and switchCaseIR.

Typing switchCase for table $typeId_{table}$ under context TC_0:

1. If let switchLabel ':' blockStatement be switchCase:
   a. Let switchLabelIR be the result of typing switchLabel for table $typeId_{table}$, under context TC_0.
   b. Let context TC_1 and blockStatementIR be the result of typing blockStatement under context TC_0.
   c. Let switchCaseIR be switchLabelIR ':' blockStatementIR.
   d. The result is context TC_1 and switchCaseIR.
2. Else:
   a. Let switchLabel ':' be switchCase.

b. Let switchLabelIR be the result of typing switchLabel for table $typeId_{table}$, under context TC_0.

c. Let switchCaseIR be switchLabelIR ':'.

d. The result is context TC_0 and switchCaseIR.

A switch label is type checked with:

SwitchLabel_table_ok:

- Typing switchLabel for table typeId, under context TC:
- The result is switchLabelIR.

Typing switchLabel for table $typeId_{table}$, under context TC:

1. If switchLabel matches pattern DEFAULT: the result is DEFAULT.
2. Else if let prefixedNonTypeName of type prefixedNonTypeName be switchLabel:
   a. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
   b. Check that prefixedNameIR matches pattern $_{BARE}$ %.
   c. Let $_{BARE}$ $nameIR_{label}$ be prefixedNameIR.
   d. Let $typeId_{table_enum}$ be the concatenation of ["action_list(", $typeId_{table}$, ")"].
   e. Let id_{label} be the concatenation of [$typeId_{table_enum}$, ".", $nameIR_{label}$].
   f. Let $varTypeIR^?$ be the type of variable $_{BARE}$ id_{label} from the LOCAL layer of TC.
   g. Check that $varTypeIR^?$ is defined.
   h. Let _ $typeIR_{label}$ ctk_{label} $value^?$ be $varTypeIR^?$.
   i. Check that $value^?$ is defined.
   j. Let $value_{label}$ be $value^?$.
   k. Check that $value_{label}$ is equal to TABLE_ENUM $typeId_{table_enum}$ '.' $nameIR_{label}$.
   l. Let $typedExpressionIR_{label}$ be $_{BARE}$ $nameIR_{label}$ annotated with a pair of type $typeIR_{label}$ and compile-time known-ness ctk_{label}.
   m. The result is $typedExpressionIR_{label}$.

13.13.2.2. Runtime evaluation

Switch_table_eval: EC_0 $ARCH_0$ |- SWITCH `( $id_{enum}$ `) `{ $switchCaseIR^*$ `} : % % %

EC_0 $ARCH_0$ |- SWITCH `( $id_{enum}$ `) `{ $switchCaseIR^*$ `} : % % %:

1. Let the matched block $blockStatementIR'^?$ be the result of runtime evaluation of $switchCaseIR^*$ for invocation of table id_{enum}.
2. If $blockStatementIR'^?$ matches pattern ():
   a. The result is EC_0, $ARCH_0$, and $_{EMPTY}$.
3. Else:
   a. Let blockStatementIR be $blockStatementIR'^?$.
   b. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of blockStatementIR, under context EC_0 and state $ARCH_0$ at LOCAL.
   c. The result is EC_1, $ARCH_1$, and statementResult.

Switch cases are runtime evaluated with the relation:

SwitchCases_table_eval:

- Runtime evaluation of switchCaseListIR for invocation of table id:
- The result is the matched block $blockStatementIR^?$.

Runtime evaluation of $switchCaseIR^*$ for invocation of table id:

1. If $switchCaseIR^*$ matches pattern []:
   a. The result is the matched block ·.
2. Else:
   a. Let $switchCaseIR_h$ :: $switchCaseIR_t^*$ be $switchCaseIR^*$.
   b. If let $switchLabelIR_h$ ':' $blockStatementIR_h$ be $switchCaseIR_h$:
      i. If id matches $switchLabelIR_h$:
         A. The result is the matched block $blockStatementIR_h$.
      ii. Else:
         A. Let the matched block $blockStatementIR^?$ be the result of runtime evaluation of $switchCaseIR_t^*$ for invocation of table id.
         B. The result is the matched block $blockStatementIR^?$.
   c. Else:
      i. Let $switchLabelIR_h$ ':' be $switchCaseIR_h$.
      ii. If id matches $switchLabelIR_h$:
         A. Try:
            I. For each $switchCaseIR_t$ in $switchCaseIR_t^*$:
               - Let bool be $switchCaseIR_t$ is not a fall-through case.

            Let $bool^*$ be the resulting list.

II. Let switchCaseIR'* be switchCaseIR$_t$* filtered by bool*.

III. If switchCaseIR'* is a non-empty list:

1. Let switchCaseIR'' :: _* be switchCaseIR'*.

2. If let _ ':' blockStatementIR be switchCaseIR'':

1. The result is the matched block blockStatementIR.

B. Then, try:

I. Check that switchCaseIR$_t$* filtered by $is_non_fallthrough_switchCaseIR(switchCaseIR$_t$)* is an empty list.

II. The result is the matched block ·.

iii. Else:

A. Let the matched block blockStatementIR? be the result of runtime evaluation of switchCaseIR$_t$* for invocation of table id.

B. The result is the matched block blockStatementIR?.

A table switch label is matched against the executed action with:

id matches switchLabelIR

1. If switchLabelIR matches pattern DEFAULT: return true.
2. Else:
   a. Let expressionIR be the expression of switchLabelIR.
   b. If let prefixedNameIR of type prefixedNameIR be expressionIR:
      i. If let $_{BARE}$ id' be prefixedNameIR:
         A. If id is equal to id': return true.
3. Otherwise: return false.

13.13.3. Switch statement on numeric or enumerated type

For this variant of switch statement, the expression must evaluate to a result with one of these types:

- bit<W>
- int<W>
- enum, either with or without an underlying representation specified
- error

All switch labels must be expressions with compile-time known values, and must have a type that can be implicitly cast to the type of the switch expression (see [sec-implicit-casts]). Switch labels must not begin with a left brace character {, to avoid ambiguity with a block statement.

```
// Assume the expression hdr.ethernet.etherType has type bit<16>.
switch (hdr.ethernet.etherType) {
   0x86dd: { /* body omitted */ }
   0x0800: // fall-through to the next body
   0x0802: { /* body omitted */ }
   0xcafe: { /* body omitted */ }
   default: { /* body omitted */ }
}
```

13.13.3.1. Type checking

Switch_general_ok:

- Typing SWITCH `( typedExpressionIR `) `{ switchCaseList `} under context TC:
- The result is context TC and switchCaseIR*.

Typing SWITCH `( typedExpressionIR$_{switch}$ `) `{ switchCaseList `} under context TC$_0$:

1. Let typeIR$_{switch}$ be the type of typedExpressionIR$_{switch}$.
2. Check that typeIR$_{switch}$ is a valid switch key type.
3. Let context TC$_1$ and switchCaseIR* be the result of typing switchCaseList under context TC$_0$ with switch key type typeIR$_{switch}$.
4. The result is context TC$_1$ and switchCaseIR*.

Switch cases are type checked with the relation:

SwitchCaseList_general_ok:

- Typing switchCaseList under context TC with switch key type typeIR:
- The result is context TC and switchCaseListIR.

Typing switchCaseList under context TC$_0$ with switch key type typeIR$_{switch}$:

1. Let switchCase* be switchCaseList flattened.
2. Let context TC$_1$ and switchCaseIR* be the result of typing switchCase* under context TC$_0$ with switch key type typeIR$_{switch}$.
3. The result is context TC$_1$ and switchCaseIR*.

SwitchCases_general_ok:

- Typing switchCase* under context TC with switch key type typeIR:
- The result is context TC and switchCaseListIR.

Typing switchCase* under context TC_0 with switch key type $typeIR_{switch}$:

1. If switchCase* matches pattern []:
   a. The result is context TC_0 and ·.
2. Else:
   a. Let $switchCase_h$ :: $switchCase_t^*$ be switchCase*.
   b. Let context TC_1 and $switchCaseIR_h$ be the result of typing $switchCase_h$ under context TC_0 with switch key type $typeIR_{switch}$.
   c. Let context TC_2 and $switchCaseIR_t^*$ be the result of typing $switchCase_t^*$ under context TC_1 with switch key type $typeIR_{switch}$.
   d. The result is context TC_2 and $switchCaseIR_h$ :: $switchCaseIR_t^*$.

A switch case is type checked with:

SwitchCase_general_ok:

- Typing switchCase under context TC with switch key type typeIR:
- The result is context TC and switchCaseIR.

Typing switchCase under context TC_0 with switch key type $typeIR_{switch}$:

1. If let switchLabel ':' blockStatement be switchCase:
   a. Let switchLabelIR be the result of typing switchLabel under context TC_0 with switch key type $typeIR_{switch}$.
   b. Let context TC_1 and blockStatementIR be the result of typing blockStatement under context TC_0.
   c. Let switchCaseIR be switchLabelIR ':' blockStatementIR.
   d. The result is context TC_1 and switchCaseIR.
2. Else:
   a. Let switchLabel ':' be switchCase.
   b. Let switchLabelIR be the result of typing switchLabel under context TC_0 with switch key type $typeIR_{switch}$.
   c. Let switchCaseIR be switchLabelIR ':'.
   d. The result is context TC_0 and switchCaseIR.

A switch label is type checked with:

SwitchLabel_general_ok:

- Typing switchLabel under context TC with switch key type typeIR:
- The result is switchLabelIR.

Typing switchLabel under context TC with switch key type typeIR:

1. If switchLabel matches pattern DEFAULT: the result is DEFAULT.
2. Else:
   a. Let $\text{expressionNonBrace}_{\text{label}}$ be switchLabel.
   b. Let $\text{expression}_{\text{label}}$ be the expression corresponding to $\text{expressionNonBrace}_{\text{label}}$.
   c. Let $\text{typedExpressionIR}_{\text{label}}$ be the result of typing $\text{expression}_{\text{label}}$, under context TC at LOCAL.
   d. Let $\text{typedExpressionIR}^{?}$ be $\text{typedExpressionIR}_{\text{label}}$ implicitly cast to typeIR.
   e. Check that $\text{typedExpressionIR}^{?}$ is defined.
   f. Let $\text{typedExpressionIR}_{\text{label_cast}}$ be $\text{typedExpressionIR}^{?}$.
   g. Let a pair of type _ and compile-time known-ness ctk be the note of $\text{typedExpressionIR}_{\text{label_cast}}$.
   h. Check that ctk is LCTK.
   i. The result is $\text{typedExpressionIR}_{\text{label_cast}}$.

13.13.3.2. Runtime evaluation

Switch_general_eval: EC_0 ARCH_0 |- SWITCH `( value `) `{ $\text{switchCaseIR}^{*}$ `} : % % %

EC_0 ARCH_0 |- SWITCH `( value `) `{ $\text{switchCaseIR}^{*}$ `} : % % %:

1. Let the updated evaluation context EC_1, state ARCH_1, and $\text{blockStatementIR}'^{?}$ be the result of runtime evaluation of switchCaseIR^{*}, under evaluation context LOCAL and state EC_0 with switch key value.
2. If $\text{blockStatementIR}'^{?}$ matches pattern ():
   a. The result is EC_1, ARCH_1, and $_{\text{EMPTY}}$.
3. Else:
   a. Let blockStatementIR be $\text{blockStatementIR}'^{?}$.
   b. Let context EC_2, state ARCH_2, and statementResult be the result of runtime evaluation of blockStatementIR, under context EC_1 and state ARCH_1 at LOCAL.
   c. The result is EC_2, ARCH_2, and statementResult.

Switch cases are runtime evaluated with the relation:

SwitchCases_general_eval:

- Runtime evaluation of switchCaseListIR, under evaluation context p and state EC with switch key value:
- The result is
  - the updated evaluation context EC, state ARCH, and blockStatementIR?.

Runtime evaluation of switchCaseIR*, under evaluation context LOCAL and state EC_0 with switch key value:

1. If switchCaseIR* matches pattern []:
   a. The result is
      - the updated evaluation context EC_0, state $ARCH_0$, and ·.
2. Else:
   a. Let $switchCaseIR_h$ :: $switchCaseIR_t$* be switchCaseIR*.
   b. If let $switchLabelIR_h$ ':' $blockStatementIR_h$ be $switchCaseIR_h$:
      i. Let the updated evaluation context EC_1, state $ARCH_1$, and expressionResult be the result of runtime evaluation of $ARCH_0$, under evaluation context LOCAL and state EC_0.
      ii. Check that expressionResult has type continueResult<value>.
      iii. Let $_{CONT}\ value_{label}$ be expressionResult.
      iv. If value matches $value_{label}$:
         A. The result is
            - the updated evaluation context EC_1, state $ARCH_1$, and $blockStatementIR_h$.
      v. Else:
         A. Let the updated evaluation context EC_2, state $ARCH_2$, and blockStatementIR? be the result of runtime evaluation of $switchCaseIR_t$*, under evaluation context LOCAL and state EC_1 with switch key value.
         B. The result is
            - the updated evaluation context EC_2, state $ARCH_2$, and blockStatementIR?.
   c. Else:
      i. Let $switchLabelIR_h$ ':' be $switchCaseIR_h$.
      ii. Let the updated evaluation context EC_1, state $ARCH_1$, and expressionResult be the result of runtime evaluation of $ARCH_0$, under evaluation context LOCAL and state EC_0.
      iii. Check that expressionResult has type continueResult<value>.
      iv. Let $_{CONT}\ value_{label}$ be expressionResult.
      v. If value matches $value_{label}$:
         A. Try:
            I. For each $switchCaseIR_t$ in $switchCaseIR_t$*:
               - Let bool be $switchCaseIR_t$ is not a fall-through case.

               Let bool* be the resulting list.
            II. Let switchCaseIR'* be $switchCaseIR_t$* filtered by bool*.
            III. If switchCaseIR'* is a non-empty list:
               1. Let switchCaseIR'' :: _* be switchCaseIR'*.

2. If let _ ':' blockStatementIR be switchCaseIR'':
    1. The result is
        - the updated evaluation context EC_1, state $ARCH_1$, and blockStatementIR.

B. Then, try:

I. Check that $switchCaseIR_t^*$ filtered by $is_non_fallthrough_switchCaseIR(switchCaseIR_t)^*$ is an empty list.

II. The result is

- the updated evaluation context EC_1, state $ARCH_1$, and ·.

vi. Else:

A. Let the updated evaluation context EC_2, state $ARCH_2$, and $blockStatementIR^?$ be the result of runtime evaluation of $switchCaseIR_t^*$, under evaluation context LOCAL and state EC_1 with switch key value.

B. The result is

- the updated evaluation context EC_2, state $ARCH_2$, and $blockStatementIR^?$.

A switch label is runtime evaluated with:

SwitchLabel_general_eval:

- Runtime evaluation of ARCH, under evaluation context p and state EC:
- The result is
    - the updated evaluation context EC, state ARCH, and expressionResult.

Runtime evaluation of $ARCH_0$, under evaluation context LOCAL and state EC_0:

1. If switchLabelIR matches pattern DEFAULT:
    a. The result is
        - the updated evaluation context EC_0, state $ARCH_0$, and $_{CONT}$ DEFAULT.
2. Else:
    a. Let typedExpressionIR be switchLabelIR.
    b. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$ at LOCAL.
    c. The result is
        - the updated evaluation context EC_1, state $ARCH_1$, and expressionResult.

A general switch label is matched against the switch value with:

value matches value'

1. If let defaultValue of type defaultValue be value': return true.
2. Otherwise: return value '==' value'.

13.13.4. Type checking

Typing SWITCH `( expression$_{switch}$ `) `{ switchCaseList `}, under context TC at LOCAL:

1. Check that TC.LOCAL.KIND is equal to CONTROL_APPLY_METHOD.
2. Let typedExpressionIR$_{switch}$ be the result of typing expression$_{switch}$, under context TC at LOCAL.
3. Let typeIR$_{switch}$ be the type of typedExpressionIR$_{switch}$.
4. Try:
   a. Let typeIR be typeIR$_{switch}$ with typedefs unrolled.
   b. If let TABLE_ENUM typeId$_{table_enum}$ `{ _$^{*}$ `} of type tableMetadataEnumTypeIR be typeIR:
      i. Let context TC$_1$ and switchCaseIR* be the result of typing SWITCH `( typeId$_{table\_enum}$ `) `{ switchCaseList `} under context TC.
      ii. For each switchCaseIR in switchCaseIR*:

          - Let switchLabelIR be the label of switchCaseIR.

          Let switchLabelIR* be the resulting list.

      iii. If default label is only in the last position of switchLabelIR*:
         A. If the elements of switchLabelIR* are distinct:
            I. Let switchStatementIR be SWITCH `( typedExpressionIR$_{switch}$ `) `{ switchCaseIR$^{*}$ `}.
            II. The result is context TC$_1$ and switchStatementIR.
5. Then, try:
   a. Check that $unroll_typeIR(typeIR$_{switch}$) does not have type tableMetadataEnumTypeIR.
   b. Let context TC$_1$ and switchCaseIR* be the result of typing SWITCH `( typedExpressionIR$_{switch}$ `) `{ switchCaseList `} under context TC.
   c. For each switchCaseIR in switchCaseIR*:

      - Let switchLabelIR be the label of switchCaseIR.

      Let switchLabelIR* be the resulting list.

   d. Check that default label is only in the last position of switchLabelIR*.
   e. Check that the elements of switchLabelIR* are distinct.
   f. Let switchStatementIR be SWITCH `( typedExpressionIR$_{switch}$ `) `{ switchCaseIR$^{*}$ `}.
   g. The result is context TC$_1$ and switchStatementIR.

13.13.5. Compile-time evaluation

Compile-time evaluation of SWITCH `( typedExpressionIR `) `{ switchCaseListIR `}, under context IC and store STO at p:

1. Let store STO$_1$ and switchCaseListIR$_{inst}$ be the result of compile-time evaluation of switchCaseListIR, under context IC and store STO at p.
2. The result is context IC, store STO$_1$, and SWITCH `( typedExpressionIR `) `{ switchCaseListIR$_{inst}$ `}.

The compile-time evaluation of switch cases is defined with the relation:

SwitchCases_inst:

- Compile-time evaluation of switchCaseIR*, under context IC and store STO at p:
- The result is store STO and switchCaseIR*.

Compile-time evaluation of switchCaseIR*, under context IC and store STO at p:

1. If switchCaseIR* matches pattern []:
   a. The result is store STO and ·.
2. Else:
   a. Let switchCaseIR_h :: switchCaseIR_t^* be switchCaseIR*.
   b. Let store STO_1 and $\text{switchCaseIR}_{h'}$ be the result of compile-time evaluation of switchCaseIR_h, under context IC and store STO at p.
   c. Let store STO_2 and $\text{switchCaseIR}_{t'}^*$ be the result of compile-time evaluation of switchCaseIR_t^*, under context IC and store STO_1 at p.
   d. The result is store STO_2 and $\text{switchCaseIR}_{h'}$ :: $\text{switchCaseIR}_{t'}^*$.

A switch case is compile-time evaluated with:

SwitchCase_inst:

- Compile-time evaluation of switchCaseIR, under context IC and store STO at p:
- The result is store STO and switchCaseIR.

Compile-time evaluation of switchCaseIR, under context IC and store STO at p:

1. If let switchLabelIR ':' blockStatementIR be switchCaseIR:
   a. Let context $\text{IC}_{\text{switch}}$, store STO_1, and statementIR be the result of compile-time evaluation of blockStatementIR, under context IC and store STO at p.
   b. Check that statementIR has type blockStatementIR.
   c. Let $\text{blockStatementIR}_{\text{inst}}$ be statementIR.
   d. The result is store STO_1 and switchLabelIR ':' $\text{blockStatementIR}_{\text{inst}}$.
2. Else:
   a. Let switchLabelIR ':' be switchCaseIR.

b. The result is store STO and switchLabelIR ':'.

13.13.6. Runtime evaluation

Runtime evaluation of SWITCH `( typedExpressionIR `) `{ switchCaseIR* `}, under context EC and state ARCH at LOCAL:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC and state ARCH at LOCAL.
2. If let abortResult of type abortResult be expressionResult:
   a. The result is context EC_1, state $ARCH_1$, and abortResult.
3. Else:
   a. Let $_{CONT}$ value be expressionResult.
   b. If value has type tableMetadataEnumValue:
      i. Let TABLE_ENUM _ '.' id_{enum} be value.
      ii. Let EC_1 $ARCH_1$ |- SWITCH `( $id_{enum}$ `) `{ switchCaseIR* `} : EC_2 $ARCH_2$ statementResult.
      iii. The result is context EC_2, state $ARCH_2$, and statementResult.
   c. Else:
      i. Let EC_1 $ARCH_1$ |- SWITCH `( value `) `{ switchCaseIR* `} : EC_2 $ARCH_2$ statementResult.
      ii. The result is context EC_2, state $ARCH_2$, and statementResult.

[1] https://github.com/p4lang/p4-spec/blob/master/p4-16/spec/docs/implementing-generalized-switch-statements.md

14. Expressions

The syntax of expressions is defined as follows:

```
expression
    : literalExpression | referenceExpression | defaultExpression
    | unaryExpression | binaryExpression | ternaryExpression | castExpression
    | dataExpression | accessExpression | callExpression
    | parenthesizedExpression
    ;
```

14.1. Semantics of expressions

Given a compound expression, the order in which sub-expressions are evaluated is important when the sub-expressions have side-effects. P4 expressions are evaluated as follows:

- Boolean operators `&&` and `||` use short-circuit evaluation—i.e., the second operand is only evaluated if necessary.
- The conditional operator `e1 ? e2 : e3` evaluates `e1`, and then either evaluates `e2` or `e3`.
- All other expressions are evaluated left-to-right as they appear in the source program.
- Method and function calls are evaluated as described in Section 19.6.

14.1.1. Type checking

Expr_ok:

- Typing expression, under context TC at p:
- The result is typedExpressionIR.

Expr_ok dispatch:

1. Try:
   a. Goto literalExpression
2. Then, try:
   a. Goto referenceExpression
3. Then, try:
   a. If let defaultExpression of type defaultExpression be expression'': goto defaultExpression
   b. Else if let unop expression of type unaryExpression be expression'':
      i. Try:
         A. If unop matches pattern '!': goto unaryExpression-lnot
         B. Else if unop matches pattern '~': goto unaryExpression-bnot
      ii. Then, try:
         A. Goto unaryExpression-uplusminus
   c. Else if let $expression_l$ binop $expression_r$ of type binaryExpression be expression'':
      i. Try:
         A. Goto binaryExpression-plusminusmult

ii. Then, try:

A. Goto binaryExpression-satplusminus

iii. Then, try:

A. Goto binaryExpression-divmod

iv. Then, try:

A. Goto binaryExpression-shift

v. Then, try:

A. Goto binaryExpression-equality

vi. Then, try:

A. Goto binaryExpression-comparison

vii. Then, try:

A. Goto binaryExpression-bitwise

viii. Then, try:

A. If binop is '++': goto binaryExpression-concat

ix. Then, try:

A. Goto binaryExpression-logical

d. Else if let $expression_{cond}$ '?' $expression_{true}$ ':' $expression_{false}$ of type ternaryExpression be expression'': goto ternaryExpression

e. Else if let `( $type_t$ `) expression of type castExpression be expression'': goto castExpression

f. Else if let invalidHeaderExpression of type invalidHeaderExpression be expression'': goto invalidHeaderExpression

g. Else if let `{ sequenceOrRecordElementExpression trailingCommaOpt `} of type sequenceOrRecordExpression be expression'':

i. If let expressionList of type expressionList be sequenceOrRecordElementExpression: goto sequenceExpression

ii. Else:

A. Let recordElementExpression be sequenceOrRecordElementExpression.

B. Goto recordExpression

h. Else if let ERROR '.' member of type errorAccessExpression be expression'': goto errorAccessExpression

i. Else if let memberAccessBase '.' member of type memberAccessExpression be expression'':

i. If let $prefixedTypeName_{base}$ of type prefixedTypeName be memberAccessBase: goto memberAccessExpression-type

ii. Else:

A. Let $expression_{base}$ be memberAccessBase.

B. Goto memberAccessExpression-expression

j. Else if let $expression_{base}$ `[ $expression_{index}$ `] of type indexAccessExpression be expression'': goto indexAccessExpression

k. Else if let $expression_{base}$ `[ expression sliceop expression' `] of type sliceAccessExpression be expression'': goto sliceAccessExpression

l. Else if let callExpression of type callExpression be expression'':

i. Try:

A. Goto callExpression-callable

ii. Then, try:

A. Check that callExpression matches pattern % `( % `).

B. Let callTarget `( argumentList `) be callExpression.

C. Goto callExpression-constructor

m. Else if let `( expression `) of type parenthesizedExpression be expression'': goto parenthesizedExpression

After type checking, expressions are represented in $P4_{IR}$ as follows:

```
typedExpressionIR
    : expressionIR # expressionNoteIR
    ;

expressionIR
    : literalExpressionIR | referenceExpressionIR | defaultExpressionIR
    | unaryExpressionIR | binaryExpressionIR | ternaryExpressionIR
    | castExpressionIR | dataExpressionIR | accessExpressionIR
    | callExpressionIR | parenthesizedExpressionIR
    ;

expressionNoteIR
    : ( typeIR ctk )
    ;
```

Notice that after type checking, expressions in P4 are annotated with their types and their compile-time known-ness. See Section 7.5 for more details of compile-time known and local compile-time known values.

The following helper functions are used to fetch the type and compile-time known-ness of an expression:

The type of typedExpressionIR

1. Let a pair of type typeIR and compile-time known-ness _ be the note of typedExpressionIR.
2. Return typeIR.

The compile-time known-ness of typedExpressionIR

1. Let a pair of type _ and compile-time known-ness ctk be the note of typedExpressionIR.
2. Return ctk.

14.1.2. Local compile-time evaluation

Expr_eval_lctk:

- Local compile-time evaluation of typedExpressionIR, under context TC at p:
- The result is value.

Expr_eval_lctk dispatch:

1. Try:
   a. Goto literalExpressionIR
2. Then, try:
   a. If let prefixedNameIR of type prefixedNameIR be expressionIR: goto referenceExpressionIR
   b. Else if let defaultExpression of type defaultExpression be expressionIR: goto defaultExpressionIR

c. Else if let unop typedExpressionIR of type unaryExpressionIR be expressionIR: goto unaryExpressionIR

d. Else if let typedExpressionIR$_l$ binop typedExpressionIR$_r$ of type binaryExpressionIR be expressionIR: goto binaryExpressionIR

e. Else if let typedExpressionIR$_{cond}$ '?' typedExpressionIR$_{true}$ ':' typedExpressionIR$_{false}$ of type ternaryExpressionIR be expressionIR: goto ternaryExpressionIR

f. Else if let `( typeIR `) typedExpressionIR of type castExpressionIR be expressionIR: goto castExpressionIR

g. Else if let invalidHeaderExpression of type invalidHeaderExpression be expressionIR: goto invalidHeaderExpressionIR

h. Else if let sequenceExpressionIR of type sequenceExpressionIR be expressionIR: goto sequenceExpressionIR

i. Else if let recordExpressionIR of type recordExpressionIR be expressionIR: goto recordExpressionIR

3. Then, try:

a. Let typeIR and ctk be the type and the compile-time known-ness of expressionNoteIR.

b. If let ERROR '.' nameIR of type errorAccessExpressionIR be expressionIR: goto errorAccessExpressionIR

4. Then, try:

a. If let memberAccessBaseIR '.' nameIR of type memberAccessExpressionIR be expressionIR:

i. If let TYPE prefixedNameIR$_{base}$ be memberAccessBaseIR: goto memberAccessExpressionIR-type

ii. Else:

A. Let typedExpressionIR$_{base}$ be memberAccessBaseIR.

B. Goto memberAccessExpressionIR-typedExpressionIR

b. Else if let typedExpressionIR$_{base}$ `[ typedExpressionIR$_{index}$ `] of type indexAccessExpressionIR be expressionIR: goto indexAccessExpressionIR

c. Else if let typedExpressionIR$_{base}$ `[ typedExpressionIR sliceop typedExpressionIR' `] of type sliceAccessExpressionIR be expressionIR: goto sliceAccessExpressionIR

d. Else if let callExpressionIR of type callExpressionIR be expressionIR:

i. Check that callExpressionIR matches pattern % `< % `> `( % `).

ii. Let callableTargetIR' `< typeArgumentIR$^*$ `> `( argumentIR$^*$ `) be callExpressionIR.

iii. Goto callExpressionIR

e. Else if let `( typedExpressionIR `) of type parenthesizedExpressionIR be expressionIR: goto parenthesizedExpressionIR

Local compile-time known expressions are evaluated with the above relation.

14.1.3. Compile-time evaluation

```
<span id="relation_latex_Expr_inst"></span>
```

```
<span id="relation_prose_Expr_inst"></span>
```

Expr_inst:

- Compile-time evaluation of typedExpressionIR, under context IC and store STO at p:
- The result is store STO and value.

Expr_inst dispatch:

1. Try:

a. Goto literalExpressionIR

2. Then, try:
    a. If let prefixedNameIR of type prefixedNameIR be expressionIR: goto referenceExpressionIR
    b. Else if let defaultExpression of type defaultExpression be expressionIR: goto defaultExpressionIR
    c. Else if let unop typedExpressionIR of type unaryExpressionIR be expressionIR: goto unaryExpressionIR
    d. Else if let $\text{typedExpressionIR}_{l}$ binop $\text{typedExpressionIR}_{r}$ of type binaryExpressionIR be expressionIR: goto binaryExpressionIR
    e. Else if let $\text{typedExpressionIR}_{cond}$ '?' $\text{typedExpressionIR}_{true}$ ':' $\text{typedExpressionIR}_{false}$ of type ternaryExpressionIR be expressionIR: goto ternaryExpressionIR
    f. Else if let `( typeIR `) typedExpressionIR of type castExpressionIR be expressionIR: goto castExpressionIR
    g. Else if let invalidHeaderExpression of type invalidHeaderExpression be expressionIR: goto invalidHeaderExpressionIR
    h. Else if let sequenceExpressionIR of type sequenceExpressionIR be expressionIR: goto sequenceExpressionIR
    i. Else if let recordExpressionIR of type recordExpressionIR be expressionIR: goto recordExpressionIR
    j. Else if let ERROR '.' nameIR of type errorAccessExpressionIR be expressionIR: goto errorAccessExpressionIR
    k. Else if let memberAccessBaseIR '.' nameIR of type memberAccessExpressionIR be expressionIR:
        i. If let TYPE prefixedNameIR be memberAccessBaseIR: goto memberAccessExpressionIR-type
        ii. Else:
            A. Let $\text{typedExpressionIR}_{base}$ be memberAccessBaseIR.
            B. Goto memberAccessExpressionIR-typedExpressionIR
    l. Else if let $\text{typedExpressionIR}_{base}$ `[ $\text{typedExpressionIR}_{index}$ `] of type indexAccessExpressionIR be expressionIR: goto indexAccessExpressionIR
    m. Else if let $\text{typedExpressionIR}_{base}$ `[ typedExpressionIR sliceop typedExpressionIR' `] of type sliceAccessExpressionIR be expressionIR: goto sliceAccessExpressionIR
    n. Else if let callExpressionIR of type callExpressionIR be expressionIR:
        i. If let prefixedNameIR `< typeArgumentListIR `> `( argumentListIR `) be callExpressionIR: goto callExpressionIR-constructor
        ii. Else:
            A. Let callableTargetIR `< _ `> `( _ `) be callExpressionIR.
            B. Try:
                I. Goto callExpressionIR-callable
            C. Then, try:
                I. Check that callableTargetIR matches pattern `( % `).
                II. Let `( callableTargetIR' `) be callableTargetIR.
                III. Goto callExpressionIR-parenthesized
    o. Else if let `( typedExpressionIR `) of type parenthesizedExpressionIR be expressionIR: goto parenthesizedExpressionIR

Compile-time known expressions are evaluated with the above relation. A sequence of expressions can be evaluated with the following relation:

Exprs_inst:

- Compile-time evaluation of $\text{typedExpressionIR}^{*}$, under context IC and store STO at p:

- The result is store STO and value*.

A list of expressions is evaluated with the following relation:

Exprs_inst:

- Compile-time evaluation of typedExpressionIR*, under context IC and store STO at p:
- The result is store STO and value*.

Compile-time evaluation of typedExpressionIR*, under context IC and store STO at p:

1. If typedExpressionIR* matches pattern []:
    a. The result is store STO and ·.
2. Else:
    a. Let $typedExpressionIR_h$ :: $typedExpressionIR_t^*$ be typedExpressionIR*.
    b. Let store STO_1 and $value_h$ be the result of compile-time evaluation of $typedExpressionIR_h$, under context IC and store STO at p.
    c. Let store STO_2 and $value_t^*$ be the result of compile-time evaluation of $typedExpressionIR_t^*$, under context IC and store STO_1 at p.
    d. The result is store STO_2 and $value_h$ :: $value_t^*$.

14.1.4. Runtime evaluation

Expr_eval:

- Evaluating typedExpressionIR, under context EC and state ARCH at p:
- The result is context EC, state ARCH and expressionResult.

Expr_eval dispatch:

1. Try:
    a. Goto literalExpressionIR
2. Then, try:
    a. If let prefixedNameIR of type prefixedNameIR be expressionIR: goto referenceExpressionIR
    b. Else if let defaultExpression of type defaultExpression be expressionIR: goto defaultExpressionIR
    c. Else if let unop typedExpressionIR of type unaryExpressionIR be expressionIR: goto unaryExpressionIR
    d. Else if let $typedExpressionIR_l$ binop $typedExpressionIR_r$ of type binaryExpressionIR be expressionIR: goto binaryExpressionIR
    e. Else if let $typedExpressionIR_{cond}$ '?' $typedExpressionIR_{true}$ ':' $typedExpressionIR_{false}$ of type ternaryExpressionIR be expressionIR: goto ternaryExpressionIR

f. Else if let `( typeIR `) typedExpressionIR of type castExpressionIR be expressionIR: goto castExpressionIR

g. Else if let invalidHeaderExpression of type invalidHeaderExpression be expressionIR: goto invalidHeaderExpressionIR

h. Else if let sequenceExpressionIR of type sequenceExpressionIR be expressionIR: goto sequenceExpressionIR

i. Else if let recordExpressionIR of type recordExpressionIR be expressionIR: goto recordExpressionIR

j. Else if let ERROR '.' nameIR of type errorAccessExpressionIR be expressionIR: goto errorAccessExpressionIR

k. Else if let memberAccessBaseIR '.' nameIR of type memberAccessExpressionIR be expressionIR:

   i. If let TYPE prefixedNameIR be memberAccessBaseIR: goto memberAccessExpressionIR-type

   ii. Else:

      A. Let $\text{typedExpressionIR}_{\text{base}}$ be memberAccessBaseIR.

      B. Goto memberAccessExpressionIR-typedExpressionIR

l. Else if let $\text{typedExpressionIR}_{\text{base}}$ `[ $\text{typedExpressionIR}_{\text{index}}$ `] of type indexAccessExpressionIR be expressionIR: goto indexAccessExpressionIR

m. Else if let $\text{typedExpressionIR}_{\text{base}}$ `[ typedExpressionIR sliceop typedExpressionIR' `] of type sliceAccessExpressionIR be expressionIR: goto sliceAccessExpressionIR

n. Else if let callExpressionIR of type callExpressionIR be expressionIR:

   i. Check that callExpressionIR matches pattern % `< % `> `( % `).

   ii. Let callableTargetIR `< $\text{typeArgumentIR}^*$ `> `( $\text{argumentIR}^*$ `) be callExpressionIR.

   iii. Goto callExpressionIR

o. Else if let `( typedExpressionIR `) of type parenthesizedExpressionIR be expressionIR: goto parenthesizedExpressionIR

The result of evaluating an expression is represented as follows:

```
expressionResult
    : continueResult<value> | abortResult
    ;

continueResult<X>
    : _CONT X
    ;

abortResult
    : exitResult | rejectTransitionResult
    ;

exitResult
    : EXIT
    ;

rejectTransitionResult
    : REJECT errorValue
    ;
```

See Section 13.7 and [sec-packet] for more details on exit and reject results.

A list of expressions can be evaluated with the following relation:

Exprs_eval:

- Evaluating typedExpressionIR*, under context EC and state ARCH at p:
- The result is context EC, state ARCH and expressionListResult.

The result of evaluating a list of expressions is represented as follows:

```
expressionListResult
    : continueResult<value*> | abortResult
    ;
```

Evaluating typedExpressionIR*, under context EC and state ARCH at p:

1. If typedExpressionIR* matches pattern []:
    a. The result is context EC, state ARCH and $_{CONT}\cdot$.
2. Else:
    a. Let $typedExpressionIR_h :: typedExpressionIR_t^*$ be typedExpressionIR*.
    b. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating $typedExpressionIR_h$, under context EC and state ARCH at p.
    c. If let abortResult of type abortResult be expressionResult:
        i. The result is context EC_1, state $ARCH_1$ and abortResult.
    d. Else:
        i. Let $_{CONT}\,value_h$ be expressionResult.
        ii. Let context EC_2, state $ARCH_2$ and expressionListResult be the result of evaluating $typedExpressionIR_t^*$, under context EC_1 and state $ARCH_1$ at p.
        iii. If let abortResult of type abortResult be expressionListResult:
            A. The result is context EC_2, state $ARCH_2$ and abortResult.
        iv. Else:
            A. Let $_{CONT}\,value_t^*$ be expressionListResult.
            B. The result is context EC_2, state $ARCH_2$ and $_{CONT}\,value_h :: value_t^*$.

14.2. Literal expressions

Literal expressions denote fixed values of boolean, numeric, or string types:

```
literalExpression
    : booleanLiteral | integerLiteral | stringLiteral
    ;

booleanLiteral
    : TRUE | FALSE
    ;
```

```
integerLiteral
    : D int | nat W int | nat S int
    ;

stringLiteral
    : " text "
    ;
```

14.2.1. Type checking

After type checking, literal expressions have the form:

```
literalExpressionIR = literalExpression
```

Typing expression'', under context TC at p:

1. If let booleanLiteral of type booleanLiteral be expression'':
    a. Let expressionNoteIR be a pair of type BOOL and compile-time known-ness LCTK.
    b. The result is booleanLiteral annotated with expressionNoteIR.
2. Else if let integerLiteral of type integerLiteral be expression'':
    a. If let D i be integerLiteral:
        i. Let expressionNoteIR be a pair of type INT and compile-time known-ness LCTK.
        ii. The result is D i annotated with expressionNoteIR.
    b. Else if let n S i be integerLiteral:
        i. Let i_{norm} be the bitstring of width n representing the integer i.
        ii. Let expressionNoteIR be a pair of type INT `< n `> and compile-time known-ness LCTK.
        iii. If n is greater than 0:
            A. The result is n S i_{norm} annotated with expressionNoteIR.
    c. Else:
        i. Let n W i be integerLiteral.
        ii. Let i_{norm} be the bitstring of width n representing the integer i.
        iii. Let expressionNoteIR be a pair of type BIT `< n `> and compile-time known-ness LCTK.
        iv. The result is n W i_{norm} annotated with expressionNoteIR.
3. Else if let stringLiteral of type stringLiteral be expression'':
    a. Let expressionNoteIR be a pair of type STRING and compile-time known-ness LCTK.
    b. The result is stringLiteral annotated with expressionNoteIR.

14.2.2. Local compile-time evaluation

Local compile-time evaluation of expressionIR annotated with expressionNoteIR, under context TC at p:

1. If let booleanLiteral of type booleanLiteral be expressionIR:
    a. Let boolValue be $_{B}$ true if booleanLiteral is equal to TRUE otherwise $_{B}$ false.
    b. The result is boolValue.
2. Else if let integerLiteral of type integerLiteral be expressionIR: the result is integerLiteral.
3. Else if let stringLiteral of type stringLiteral be expressionIR: the result is stringLiteral.

14.2.3. Compile-time evaluation

Compile-time evaluation of expressionIR annotated with expressionNoteIR, under context IC and store STO at p:

1. If let booleanLiteral of type booleanLiteral be expressionIR:
    a. Let boolValue be $_{B}$ true if booleanLiteral is equal to TRUE otherwise $_{B}$ false.
    b. The result is store STO and boolValue.
2. Else if let integerLiteral of type integerLiteral be expressionIR:
    a. The result is store STO and integerLiteral.
3. Else if let stringLiteral of type stringLiteral be expressionIR:
    a. The result is store STO and stringLiteral.

14.2.4. Runtime evaluation

Evaluating expressionIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. If let booleanLiteral of type booleanLiteral be expressionIR:
    a. Let boolValue be $_{B}$ true if booleanLiteral is equal to TRUE otherwise $_{B}$ false.
    b. Let expressionResult be $_{CONT}$ boolValue.
    c. The result is context EC, state ARCH and expressionResult.
2. Else if let integerLiteral of type integerLiteral be expressionIR:
    a. Let expressionResult be $_{CONT}$ integerLiteral.
    b. The result is context EC, state ARCH and expressionResult.
3. Else if let stringLiteral of type stringLiteral be expressionIR:
    a. Let expressionResult be $_{CONT}$ stringLiteral.
    b. The result is context EC, state ARCH and expressionResult.

14.3. Reference expressions

A reference expression is an expression that refers to a named entity in a P4 program:

```
referenceExpression
    : prefixedNonTypeName | THIS
    ;

prefixedNonTypeName
    : nonTypeName | _ID . nonTypeName
    ;
```

References are resolved according to the scoping rules of P4 (see Section 6.3).

14.3.1. Type checking

After type checking, a reference expression is represented in $P4_{IR}$ as:

```
referenceExpressionIR = prefixedNameIR
```

Typing expression'', under context TC at p:

1. Try:
    a. If let prefixedNonTypeName of type prefixedNonTypeName be expression'':
        i. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
        ii. Let varTypeIR? be the type of variable prefixedNameIR from the p layer of TC.
        iii. If varTypeIR? is defined:
            A. Let _ typeIR ctk _? be varTypeIR?.
            B. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.
            C. The result is prefixedNameIR annotated with expressionNoteIR.
2. Then, try:
    a. If expression'' is equal to THIS:
        i. Let prefixedNameIR be $_{BARE}$ "this".
        ii. Let varTypeIR? be the type of variable prefixedNameIR from the p layer of TC.
        iii. If varTypeIR? is defined:
            A. Let _ typeIR ctk _? be varTypeIR?.
            B. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.
            C. The result is prefixedNameIR annotated with expressionNoteIR.

A variable is looked up in the typing context with:

The type of variable prefixedNameIR from the p layer of TC

1. If let '.' id be prefixedNameIR:
    a. Let typeFrame be TC.GLOBAL.FRAME.

b. Return the value of id in map typeFrame.

2. Else:
   a. Let $_{\text{BARE}}$ id be prefixedNameIR.
   b. If p matches pattern GLOBAL:
      i. Let typeFrame be TC.GLOBAL.FRAME.
      ii. Return the value of id in map typeFrame.
   c. Else if p matches pattern BLOCK:
      i. Let typeFrame be TC.BLOCK.FRAME.
      ii. Try:
         A. Let varTypeIR'? be the value of id in map typeFrame.
         B. If varTypeIR'? is defined:
            I. Let varTypeIR be varTypeIR'?.
            II. Return varTypeIR.
      iii. Then, try:
         A. Check that the value of id in map typeFrame is none.
         B. Return the type of variable $_{\text{BARE}}$ id from the GLOBAL layer of TC.
   d. Else:
      i. Let typeFrame* be TC.LOCAL.FRAMES.
      ii. Try:
         A. Let varTypeIR'? be the value of id in maps typeFrame*.
         B. If varTypeIR'? is defined:
            I. Let varTypeIR be varTypeIR'?.
            II. Return varTypeIR.
      iii. Then, try:
         A. Check that the value of id in maps typeFrame* is none.
         B. Return the type of variable $_{\text{BARE}}$ id from the BLOCK layer of TC.

14.3.2. Local compile-time evaluation

Local compile-time evaluation of prefixedNameIR annotated with expressionNoteIR, under context TC at p:

1. Let value be the value of variable prefixedNameIR from the p layer of TC.
2. The result is value.

14.3.3. Compile-time evaluation

Compile-time evaluation of prefixedNameIR annotated with expressionNoteIR, under context IC and store STO at p:

1. Let value be the value of prefixedNameIR from the p layer of IC.

2. The result is store STO and value.

A variable is looked up in the instantiation context with:

The value of prefixedNameIR from the p layer of IC

1. If let '.' id be prefixedNameIR:
    a. Let value'? be the value of id in map IC.GLOBAL.FRAME.
    b. Check that value'? is defined.
    c. Let value be value'?.
    d. Return value.
2. Else:
    a. Let BARE id be prefixedNameIR.
    b. If p matches pattern GLOBAL:
        i. Let value'? be the value of id in map IC.GLOBAL.FRAME.
        ii. Check that value'? is defined.
        iii. Let value be value'?.
        iv. Return value.
    c. Else if p matches pattern BLOCK:
        i. Try:
            A. Let value'? be the value of id in map IC.BLOCK.FRAME.
            B. If value'? is defined:
                I. Let value be value'?.
                II. Return value.
        ii. Then, try:
            A. Check that the value of id in map IC.BLOCK.FRAME is none.
            B. Return the value of BARE id from the GLOBAL layer of IC.
    d. Else:
        i. Try:
            A. Let value'? be the value of id in maps IC.LOCAL.FRAMES.
            B. If value'? is defined:
                I. Let value be value'?.
                II. Return value.
        ii. Then, try:
            A. Check that the value of id in maps IC.LOCAL.FRAMES is none.
            B. Return the value of BARE id from the BLOCK layer of IC.

14.3.4. Runtime evaluation

Evaluating prefixedNameIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let value be the value of prefixedNameIR from the p layer of EC.
2. Let expressionResult be $_{\text{CONT}}$ value.
3. The result is context EC, state ARCH and expressionResult.

A variable is looked up in the runtime context with:

The value of prefixedNameIR from the p layer of EC

1. If let '.' id be prefixedNameIR:
    a. Let value'? be the value of id in map EC.GLOBAL.FRAME.
    b. Check that value'? is defined.
    c. Let value be value'?.
    d. Return value.
2. Else:
    a. Let $_{\text{BARE}}$ id be prefixedNameIR.
    b. If p matches pattern GLOBAL:
        i. Let value'? be the value of id in map EC.GLOBAL.FRAME.
        ii. Check that value'? is defined.
        iii. Let value be value'?.
        iv. Return value.
    c. Else if p matches pattern BLOCK:
        i. Try:
            A. Let value'? be the value of id in map EC.BLOCK.FRAME.
            B. If value'? is defined:
                I. Let value be value'?.
                II. Return value.
        ii. Then, try:
            A. Check that the value of id in map EC.BLOCK.FRAME is none.
            B. Return the value of $_{\text{BARE}}$ id from the GLOBAL layer of EC.
    d. Else:
        i. Try:
            A. Let value'? be the value of id in maps EC.LOCAL.FRAMES.
            B. If value'? is defined:
                I. Let value be value'?.
                II. Return value.
        ii. Then, try:
            A. Check that the value of id in maps EC.LOCAL.FRAMES is none.
            B. Return the value of $_{\text{BARE}}$ id from the BLOCK layer of EC.

14.4. Default expressions

Default expressions provide a way to specify a default value for a type:

```
defaultExpression
    : ...
    ;
```

See Section 8.6.1 for more details on default types. When default expressions are cast to a specific type, they evaluate to the default value for that type. See Section 20.3 for more details on default values.

14.4.1. Type checking

After type checking, default expressions have the form:

```
defaultExpressionIR = defaultExpression
```

Typing '...', under context TC at p:

1. Let expressionNoteIR be a pair of type DEFAULT and compile-time known-ness LCTK.
2. The result is '...' annotated with expressionNoteIR.

14.4.2. Local compile-time evaluation

Local compile-time evaluation of '...' annotated with expressionNoteIR, under context TC at p: the result is DEFAULT.

14.4.3. Compile-time evaluation

Compile-time evaluation of '...' annotated with expressionNoteIR, under context IC and store STO at p:

1. The result is store STO and DEFAULT.

14.4.4. Runtime evaluation

Evaluating '...' annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let expressionResult be $_{\text{CONT}}$ DEFAULT.
2. The result is context EC, state ARCH and expressionResult.

14.5. Unary expressions

```
unaryExpression
    : unop expression
    ;

unop
    : ! | ~ | - | +
    ;
```

See Section 20.1 for details on how each unary operator is evaluated.

14.5.1. Type checking

After type checking, unary expressions have the form:

```
unaryExpressionIR
    : unop typedExpressionIR
    ;
```

14.5.1.1. Negation

Typing '!' expression, under context TC at p:

1. Let typedExpressionIR be the result of typing expression, under context TC at p.
2. Let $typedExpressionIR'^{?}$ be the result of reducing serializable enums in typedExpressionIR until $compat_lnot is satisfied.
3. If $typedExpressionIR'^{?}$ is defined:
    a. Let $typedExpressionIR_{reduced}$ be $typedExpressionIR'^{?}$.
    b. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{reduced}$.
    c. Let $ctk_{reduced}$ be the compile-time known-ness of $typedExpressionIR_{reduced}$.
    d. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
    e. The result is '!' $typedExpressionIR_{reduced}$ annotated with expressionNoteIR.

14.5.1.2. Bitwise complement

Typing '~' expression, under context TC at p:

1. Let typedExpressionIR be the result of typing expression, under context TC at p.
2. Let $typedExpressionIR'^{?}$ be the result of reducing serializable enums in typedExpressionIR until $compat_bnot is satisfied.
3. If $typedExpressionIR'^{?}$ is defined:
    a. Let $typedExpressionIR_{reduced}$ be $typedExpressionIR'^{?}$.
    b. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{reduced}$.

c. Let $ctk_{reduced}$ be the compile-time known-ness of $typedExpressionIR_{reduced}$.

d. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.

e. The result is '~' $typedExpressionIR_{reduced}$ annotated with expressionNoteIR.

14.5.1.3. Plus and minus

Typing unop expression, under context TC at p:

1. Check that unop is equal to '+' or unop is equal to '-'.
2. Let typedExpressionIR be the result of typing expression, under context TC at p.
3. Let $typedExpressionIR'^{?}$ be the result of reducing serializable enums in typedExpressionIR until $compat_uplusminus is satisfied.
4. Check that $typedExpressionIR'^{?}$ is defined.
5. Let $typedExpressionIR_{reduced}$ be $typedExpressionIR'^{?}$.
6. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{reduced}$.
7. Let $ctk_{reduced}$ be the compile-time known-ness of $typedExpressionIR_{reduced}$.
8. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
9. The result is unop $typedExpressionIR_{reduced}$ annotated with expressionNoteIR.

14.5.2. Local compile-time evaluation

Local compile-time evaluation of unop typedExpressionIR annotated with expressionNoteIR, under context TC at p:

1. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
2. The result is unop value.

14.5.3. Compile-time evaluation

Compile-time evaluation of unop typedExpressionIR annotated with expressionNoteIR, under context IC and store STO at p:

1. Let store STO_1 and value be the result of compile-time evaluation of typedExpressionIR, under context IC and store STO at p.
2. The result is store STO_1 and unop value.

14.5.4. Runtime evaluation

Evaluating unop typedExpressionIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$ and expressionResult' be the result of evaluating typedExpressionIR, under context EC and state ARCH at p.
2. If let abortResult of type abortResult be expressionResult':
    a. The result is context EC_1, state $ARCH_1$ and abortResult.
3. Else:
    a. Let $_{CONT}$ value be expressionResult'.
    b. Let value''' be unop value.
    c. Let expressionResult be $_{CONT}$ value'''.
    d. The result is context EC_1, state $ARCH_1$ and expressionResult.

14.6. Binary expressions

```
binaryExpression
    : expression binop expression
    ;

binop
    : * | / | % | + | - | |+| | |-| | << | >> | <= | >= | < | > | != | == | &
    | ^ | | | ++ | && | ||
    ;
```

See Section 20.2 for details on how each binary operator is evaluated. Note that logical operators (&&, ||) use short-circuit evaluation.

14.6.1. Type checking

14.6.1.1. Plus, minus, and multiplication

Typing $expression_l$ binop $expression_r$, under context TC at p:

1. If binop is in ['+', '-', '*']:
    a. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at p.
    b. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at p.
    c. Let $(typedExpressionIR, typedExpressionIR)^?$ be $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.
    d. If $(typedExpressionIR, typedExpressionIR)^?$ is defined:
        i. Let ($typedExpressionIR_{l_cast}$, $typedExpressionIR_{r_cast}$) be $(typedExpressionIR, typedExpressionIR)^?$.
        ii. Let $(typedExpressionIR, typedExpressionIR)'^?$ be the result of reducing serializable enums in $typedExpressionIR_{l_cast}$ and $typedExpressionIR_{r_cast}$ until $compat_plusminusmult is satisfied.
        iii. If $(typedExpressionIR, typedExpressionIR)'^?$ is defined:
            A. Let ($typedExpressionIR_{l_reduced}$, $typedExpressionIR_{r_reduced}$) be $(typedExpressionIR, typedExpressionIR)'^?$.
            B. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{l_reduced}$.

C. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.

D. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.

E. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.

F. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.

G. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

14.6.1.2. Saturating plus and minus

Typing $expression_l$ binop $expression_r$, under context TC at p:

1. If binop is in ['|+|', '|-|']:
   a. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at p.
   b. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at p.
   c. Let $(typedExpressionIR, typedExpressionIR)^?$ be $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.
   d. If $(typedExpressionIR, typedExpressionIR)^?$ is defined:
      i. Let $(typedExpressionIR_{l_cast}, typedExpressionIR_{r_cast})$ be $(typedExpressionIR, typedExpressionIR)^?$.
      ii. Let $(typedExpressionIR, typedExpressionIR)'^?$ be the result of reducing serializable enums in $typedExpressionIR_{l_cast}$ and $typedExpressionIR_{r_cast}$ until $compat_satplusminus is satisfied.
      iii. If $(typedExpressionIR, typedExpressionIR)'^?$ is defined:
         A. Let $(typedExpressionIR_{l_reduced}, typedExpressionIR_{r_reduced})$ be $(typedExpressionIR, typedExpressionIR)'^?$.
         B. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{l_reduced}$.
         C. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.
         D. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.
         E. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.
         F. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
         G. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

14.6.1.3. Division and modulo

Typing $expression_l$ binop $expression_r$, under context TC at p:

1. If binop is in ['/', '%']:
   a. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at p.
   b. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at p.
   c. Let $(typedExpressionIR, typedExpressionIR)^?$ be $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.
   d. If $(typedExpressionIR, typedExpressionIR)^?$ is defined:
      i. Let $(typedExpressionIR_{l_cast}, typedExpressionIR_{r_cast})$ be $(typedExpressionIR, typedExpressionIR)^?$.

ii. Let (typedExpressionIR, typedExpressionIR)'? be the result of reducing serializable enums in $typedExpressionIR_{l_cast}$ and $typedExpressionIR_{r_cast}$ until $compat_divmod is satisfied.

iii. If (typedExpressionIR, typedExpressionIR)'? is defined:

A. Let ($typedExpressionIR_{l_reduced}$, $typedExpressionIR_{r_reduced}$) be (typedExpressionIR, typedExpressionIR)'?.

B. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{l_reduced}$.

C. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.

D. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.

E. If $ctk_{r_reduced}$ is LCTK:

I. Let value be the result of local compile-time evaluation of $typedExpressionIR_{r_reduced}$, under context TC at p.

II. If let $integerValue_r$ of type integerValue be value:

1. Let nat? be the natural number representation of $integerValue_r$.

2. If nat? is defined:

1. Let n_r be nat?.

2. If n_r is greater than 0:

1. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.

2. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.

3. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

F. Else:

I. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.

II. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.

III. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

14.6.1.4. Shift left and right

Typing $expression_l$ binop $expression_r$, under context TC at p:

1. If binop is in ['<<', '>>']:

a. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at p.

b. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at p.

c. Let (typedExpressionIR, typedExpressionIR)? be the result of reducing serializable enums in $typedExpressionIR_l$ and $typedExpressionIR_r$ until $compat_shift is satisfied.

d. If (typedExpressionIR, typedExpressionIR)? is defined:

i. Let ($typedExpressionIR_{l_reduced}$, $typedExpressionIR_{r_reduced}$) be (typedExpressionIR, typedExpressionIR)?.

ii. Let $typeIR_{l_reduced}$ be the type of $typedExpressionIR_{l_reduced}$.

iii. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.

iv. Let $typeIR_{r_reduced}$ be the type of $typedExpressionIR_{r_reduced}$.

v. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.

vi. Try:

A. Let typeIR be $typeIR_{r_reduced}$ with typedefs unrolled.

B. If typeIR has type fixedBitTypeIR:

I. If if $typeIR_{l_reduced}$ is equal to INT, then $ctk_{r_reduced}$ is equal to LCTK:

1. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.

2. Let expressionNoteIR be a pair of type $typeIR_{l_reduced}$ and compile-time known-ness $ctk_{reduced}$.

3. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

vii. Then, try:

A. If $typeIR_{r_reduced}$ does not have type fixedBitTypeIR:

I. If $ctk_{r_reduced}$ is LCTK:

1. Let value be the result of local compile-time evaluation of $typedExpressionIR_{r_reduced}$, under context TC at p.

2. If let $integerValue_r$ of type integerValue be value:

1. Let i_r be the integer representation of $integerValue_r$.

2. If i_r is greater than or equal to 0:

1. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.

2. Let expressionNoteIR be a pair of type $typeIR_{l_reduced}$ and compile-time known-ness $ctk_{reduced}$.

3. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

14.6.1.5. Equality and inequality

`<span id="Expr_ok-binaryExpression-equality"></span>`

Typing $expression_l$ binop $expression_r$, under context TC at p:

1. If binop is in ['==', '!=']:

a. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at p.

b. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at p.

c. Let $(typedExpressionIR, typedExpressionIR)^?$ be $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.

d. If $(typedExpressionIR, typedExpressionIR)^?$ is defined:

i. Let ($typedExpressionIR_{l_cast}$, $typedExpressionIR_{r_cast}$) be $(typedExpressionIR, typedExpressionIR)^?$.

ii. Let $typeIR_{cast}$ be the type of $typedExpressionIR_{l_cast}$.

iii. Let ctk_{l_cast} be the compile-time known-ness of $typedExpressionIR_{l_cast}$.

iv. Let ctk_{r_cast} be the compile-time known-ness of $typedExpressionIR_{r_cast}$.

v. If $typeIR_{cast}$ supports equality comparison:

A. Let ctk_{cast} be the join of ctk_{l_cast} and ctk_{r_cast}.

B. Let expressionNoteIR be a pair of type BOOL and compile-time known-ness ctk_{cast}.

C. The result is $typedExpressionIR_{l_cast}$ binop $typedExpressionIR_{r_cast}$ annotated with expressionNoteIR.

14.6.1.6. Comparison

Typing expression_l binop expression_r, under context TC at p:

1. If binop is in ['<=', '>=', '<', '>']:
    a. Let $\text{typedExpressionIR}_l$ be the result of typing expression_l, under context TC at p.
    b. Let $\text{typedExpressionIR}_r$ be the result of typing expression_r, under context TC at p.
    c. Let $(\text{typedExpressionIR, typedExpressionIR})^?$ be $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.
    d. If $(\text{typedExpressionIR, typedExpressionIR})^?$ is defined:
        i. Let $(\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast})$ be $(\text{typedExpressionIR, typedExpressionIR})^?$.
        ii. Let $(\text{typedExpressionIR, typedExpressionIR})'^?$ be the result of reducing serializable enums in $\text{typedExpressionIR}_{l_cast}$ and $\text{typedExpressionIR}_{r_cast}$ until $compat_compare is satisfied.
        iii. If $(\text{typedExpressionIR, typedExpressionIR})'^?$ is defined:
            A. Let $(\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced})$ be $(\text{typedExpressionIR, typedExpressionIR})'^?$.
            B. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
            C. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
            D. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
            E. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
            F. Let expressionNoteIR be a pair of type BOOL and compile-time known-ness $\text{ctk}_{reduced}$.
            G. The result is $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR.

14.6.1.7. Bitwise and, xor, and or

Typing expression_l binop expression_r, under context TC at p:

1. If binop is in ['&', '^', '|']:
    a. Let $\text{typedExpressionIR}_l$ be the result of typing expression_l, under context TC at p.
    b. Let $\text{typedExpressionIR}_r$ be the result of typing expression_r, under context TC at p.
    c. Let $(\text{typedExpressionIR, typedExpressionIR})^?$ be $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.
    d. If $(\text{typedExpressionIR, typedExpressionIR})^?$ is defined:
        i. Let $(\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast})$ be $(\text{typedExpressionIR, typedExpressionIR})^?$.
        ii. Let $(\text{typedExpressionIR, typedExpressionIR})'^?$ be the result of reducing serializable enums in $\text{typedExpressionIR}_{l_cast}$ and $\text{typedExpressionIR}_{r_cast}$ until $compat_bitwise is satisfied.
        iii. If $(\text{typedExpressionIR, typedExpressionIR})'^?$ is defined:
            A. Let $(\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced})$ be $(\text{typedExpressionIR, typedExpressionIR})'^?$.
            B. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
            C. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
            D. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.

E. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.

F. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.

G. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

14.6.1.8. Concatenation

Typing $expression_l$ '++' $expression_r$, under context TC at p:

1. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at p.
2. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at p.
3. Let $(typedExpressionIR, typedExpressionIR)^?$ be the result of reducing serializable enums in $typedExpressionIR_l$ and $typedExpressionIR_r$ until $compat_concat is satisfied.
4. If $(typedExpressionIR, typedExpressionIR)^?$ is defined:
   a. Let $(typedExpressionIR_{l_reduced}, typedExpressionIR_{r_reduced})$ be $(typedExpressionIR, typedExpressionIR)^?$.
   b. Let $typeIR_{l_reduced}$ be the type of $typedExpressionIR_{l_reduced}$.
   c. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.
   d. Let $typeIR_{r_reduced}$ be the type of $typedExpressionIR_{r_reduced}$.
   e. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.
   f. If if $typeIR_{l_reduced}$ is equal to STRING and $typeIR_{r_reduced}$ is equal to STRING, then $ctk_{l_reduced}$ is equal to LCTK and $ctk_{r_reduced}$ is equal to LCTK:
      i. Let $typeIR_{reduced}$ be $result_concat($typeIR_{l_reduced}$, $typeIR_{r_reduced}$).
      ii. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.
      iii. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
      iv. The result is $typedExpressionIR_{l_reduced}$ '++' $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

14.6.1.9. Logical and, and or

Typing $expression_l$ binop $expression_r$, under context TC at p:

1. Check that binop is in ['&&', '||'].
2. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at p.
3. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at p.
4. Let $(typedExpressionIR, typedExpressionIR)^?$ be $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.
5. Check that $(typedExpressionIR, typedExpressionIR)^?$ is defined.
6. Let $(typedExpressionIR_{l_cast}, typedExpressionIR_{r_cast})$ be $(typedExpressionIR, typedExpressionIR)^?$.
7. Let $(typedExpressionIR, typedExpressionIR)'^?$ be the result of reducing serializable enums in $typedExpressionIR_{l_cast}$ and $typedExpressionIR_{r_cast}$ until $compat_logical is satisfied.
8. Check that $(typedExpressionIR, typedExpressionIR)'^?$ is defined.
9. Let $(typedExpressionIR_{l_reduced}, typedExpressionIR_{r_reduced})$ be $(typedExpressionIR, typedExpressionIR)'^?$.

10. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{l_reduced}$.
11. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.
12. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.
13. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.
14. Let expressionNoteIR be a pair of type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
15. The result is $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with expressionNoteIR.

14.6.2. Local compile-time evaluation

Local compile-time evaluation of $typedExpressionIR_l$ binop $typedExpressionIR_r$ annotated with expressionNoteIR, under context TC at p:

1. Try:
    a. If binop is not in ['&&', '||']:
        i. Let $value_l$ be the result of local compile-time evaluation of $typedExpressionIR_l$, under context TC at p.
        ii. Let $value_r$ be the result of local compile-time evaluation of $typedExpressionIR_r$, under context TC at p.
        iii. The result is $value_l$ binop $value_r$.
2. Then, try:
    a. If binop matches pattern '&&':
        i. Let value be the result of local compile-time evaluation of $typedExpressionIR_l$, under context TC at p.
        ii. If value is equal to $_B$ false: the result is $_B$ false.
        iii. Else if value is equal to $_B$ true:
            A. Let $value_r$ be the result of local compile-time evaluation of $typedExpressionIR_r$, under context TC at p.
            B. The result is $value_r$.
    b. Else if binop matches pattern '||':
        i. Let value be the result of local compile-time evaluation of $typedExpressionIR_l$, under context TC at p.
        ii. If value is equal to $_B$ true: the result is $_B$ true.
        iii. Else if value is equal to $_B$ false:
            A. Let $value_r$ be the result of local compile-time evaluation of $typedExpressionIR_r$, under context TC at p.
            B. The result is $value_r$.

14.6.3. Compile-time evaluation

Compile-time evaluation of $typedExpressionIR_l$ binop $typedExpressionIR_r$ annotated with expressionNoteIR, under context IC and store STO at p:

1. Try:
    a. If binop is not in ['&&', '||']:

i. Let store STO_1 and $value_l$ be the result of compile-time evaluation of $typedExpressionIR_l$, under context IC and store STO at p.

ii. Let store STO_2 and $value_r$ be the result of compile-time evaluation of $typedExpressionIR_r$, under context IC and store STO_1 at p.

iii. The result is store STO_2 and $value_l$ binop $value_r$.

2. Then, try:

a. If binop matches pattern '&&':

i. Let store STO_1 and value be the result of compile-time evaluation of $typedExpressionIR_l$, under context IC and store STO at p.

ii. If value is equal to $_B$ false:

A. The result is store STO_1 and $_B$ false.

iii. Else if value is equal to $_B$ true:

A. Let store STO_2 and $value_r$ be the result of compile-time evaluation of $typedExpressionIR_r$, under context IC and store STO_1 at p.

B. The result is store STO_2 and $value_r$.

b. Else if binop matches pattern '||':

i. Let store STO_1 and value be the result of compile-time evaluation of $typedExpressionIR_l$, under context IC and store STO at p.

ii. If value is equal to $_B$ true:

A. The result is store STO_1 and $_B$ true.

iii. Else if value is equal to $_B$ false:

A. Let store STO_2 and $value_r$ be the result of compile-time evaluation of $typedExpressionIR_r$, under context IC and store STO_1 at p.

B. The result is store STO_2 and $value_r$.

14.6.4. Runtime evaluation

Evaluating $typedExpressionIR_l$ binop $typedExpressionIR_r$ annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Try:

a. If binop is not in ['&&', '||']:

i. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating [$typedExpressionIR_l$, $typedExpressionIR_r$], under context EC and state ARCH at p.

ii. If let abortResult of type abortResult be expressionListResult:

A. The result is context EC_1, state $ARCH_1$ and abortResult.

iii. Else:

A. Let $_{CONT}$ $value^*$ be expressionListResult.

B. If $value^*$ is a list of length 2:

I. Let [$value_l$, $value_r$] be $value^*$.

II. Let value"" be $value_l$ binop $value_r$.

III. Let expressionResult be $_{CONT}$ value"".

IV. The result is context EC_1, state $ARCH_1$ and expressionResult.

2. Then, try:
    a. If binop matches pattern '&&':
        i. Let context EC_1, state $ARCH_1$ and expressionResult' be the result of evaluating typedExpressionIR$_l$, under context EC and state ARCH at p.
        ii. Try:
            A. If let abortResult of type abortResult be expressionResult':
                I. The result is context EC_1, state $ARCH_1$ and abortResult.
        iii. Then, try:
            A. If expressionResult' is equal to $_{CONT\ B}$ false:
                I. Let expressionResult be $_{CONT\ B}$ false.
                II. The result is context EC_1, state $ARCH_1$ and expressionResult.
            B. Else if expressionResult' is equal to $_{CONT\ B}$ true:
                I. Let context EC_2, state $ARCH_2$ and expressionResult$_r$ be the result of evaluating typedExpressionIR$_r$, under context EC_1 and state $ARCH_1$ at p.
                II. The result is context EC_2, state $ARCH_2$ and expressionResult$_r$.
    b. Else if binop matches pattern '||':
        i. Let context EC_1, state $ARCH_1$ and expressionResult' be the result of evaluating typedExpressionIR$_l$, under context EC and state ARCH at p.
        ii. Try:
            A. If let abortResult of type abortResult be expressionResult':
                I. The result is context EC_1, state $ARCH_1$ and abortResult.
        iii. Then, try:
            A. If expressionResult' is equal to $_{CONT\ B}$ true:
                I. Let expressionResult be $_{CONT\ B}$ true.
                II. The result is context EC_1, state $ARCH_1$ and expressionResult.
            B. Else if expressionResult' is equal to $_{CONT\ B}$ false:
                I. Let context EC_2, state $ARCH_2$ and expressionResult$_r$ be the result of evaluating typedExpressionIR$_r$, under context EC_1 and state $ARCH_1$ at p.
                II. The result is context EC_2, state $ARCH_2$ and expressionResult$_r$.

14.7. Ternary expressions

A conditional expression of the form e1 ? e2 : e3 behaves the same as in languages like C. As described above, the expression e1 is evaluated first, and second either e2 or e3 is evaluated depending on the result.

The first sub-expression e1 must have Boolean type and the second and third sub-expressions must have the same type, which cannot both be arbitrary-precision integers unless the condition itself can be evaluated at compilation time. This restriction is designed to ensure that the width of the result of the conditional expression can be inferred statically at compile time.

```
<span id="ternaryExpression"></span>
```

```
ternaryExpression
    : expression ? expression : expression
    ;
```

14.7.1. Type checking

After type checking, ternary expressions have the form:

```
ternaryExpressionIR
    : typedExpressionIR ? typedExpressionIR : typedExpressionIR
    ;
```

Typing expression$_{cond}$ '?' expression$_{true}$ ':' expression$_{false}$, under context TC at p:

1. Let typedExpressionIR$_{cond}$ be the result of typing expression$_{cond}$, under context TC at p.
2. Let typeIR$_{cond}$ be the type of typedExpressionIR$_{cond}$.
3. Let ctk$_{cond}$ be the compile-time known-ness of typedExpressionIR$_{cond}$.
4. Check that BOOL is equal to typeIR$_{cond}$ with typedefs unrolled.
5. Let typedExpressionIR$_{true}$ be the result of typing expression$_{true}$, under context TC at p.
6. Let typedExpressionIR$_{false}$ be the result of typing expression$_{false}$, under context TC at p.
7. Let (typedExpressionIR, typedExpressionIR)$^{?}$ be typedExpressionIR$_{true}$ and typedExpressionIR$_{false}$ implicitly cast to equal types.
8. Check that (typedExpressionIR, typedExpressionIR)$^{?}$ is defined.
9. Let (typedExpressionIR$_{true_cast}$, typedExpressionIR$_{false_cast}$) be (typedExpressionIR, typedExpressionIR)$^{?}$.
10. Let typeIR$_{cast}$ be the type of typedExpressionIR$_{true_cast}$.
11. Let ctk$_{true_cast}$ be the compile-time known-ness of typedExpressionIR$_{true_cast}$.
12. Let ctk$_{false_cast}$ be the compile-time known-ness of typedExpressionIR$_{false_cast}$.
13. Check that if typeIR$_{cast}$ is equal to INT, then ctk$_{cond}$ is not equal to DYN.
14. Let ctk be the join of [ctk$_{cond}$, ctk$_{true_cast}$, ctk$_{false_cast}$].
15. Let expressionNoteIR be a pair of type typeIR$_{cast}$ and compile-time known-ness ctk.
16. The result is typedExpressionIR$_{cond}$ '?' typedExpressionIR$_{true_cast}$ ':' typedExpressionIR$_{false_cast}$ annotated with expressionNoteIR.

14.7.2. Local compile-time evaluation

Local compile-time evaluation of typedExpressionIR$_{cond}$ '?' typedExpressionIR$_{true}$ ':' typedExpressionIR$_{false}$ annotated with expressionNoteIR, under context TC at p:

1. Let value be the result of local compile-time evaluation of typedExpressionIR$_{cond}$, under context TC at p.
2. If value is equal to $_{B}$ true:
    a. Let value$_{true}$ be the result of local compile-time evaluation of typedExpressionIR$_{true}$, under context TC at p.
    b. The result is value$_{true}$.
3. Else if value is equal to $_{B}$ false:
    a. Let value$_{false}$ be the result of local compile-time evaluation of typedExpressionIR$_{false}$, under context TC at p.

b. The result is $value_{false}$.

14.7.3. Compile-time evaluation

Compile-time evaluation of $typedExpressionIR_{cond}$ '?' $typedExpressionIR_{true}$ ':' $typedExpressionIR_{false}$ annotated with expressionNoteIR, under context IC and store STO at p:

1. Let store STO_1 and value be the result of compile-time evaluation of $typedExpressionIR_{cond}$, under context IC and store STO at p.
2. If value is equal to $_B$ true:
   a. Let store STO_2 and $value_{true}$ be the result of compile-time evaluation of $typedExpressionIR_{true}$, under context IC and store STO_1 at p.
   b. The result is store STO_2 and $value_{true}$.
3. Else if value is equal to $_B$ false:
   a. Let store STO_2 and $value_{false}$ be the result of compile-time evaluation of $typedExpressionIR_{false}$, under context IC and store STO_1 at p.
   b. The result is store STO_2 and $value_{false}$.

14.7.4. Runtime evaluation

Evaluating $typedExpressionIR_{cond}$ '?' $typedExpressionIR_{true}$ ':' $typedExpressionIR_{false}$ annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating $typedExpressionIR_{cond}$, under context EC and state ARCH at p.
2. Try:
   a. If let abortResult of type abortResult be expressionResult:
      i. The result is context EC_1, state $ARCH_1$ and abortResult.
3. Then, try:
   a. If expressionResult is equal to $_{CONT\ B}$ true:
      i. Let context EC_2, state $ARCH_2$ and $expressionResult_{true}$ be the result of evaluating $typedExpressionIR_{true}$, under context EC_1 and state $ARCH_1$ at p.
      ii. The result is context EC_2, state $ARCH_2$ and $expressionResult_{true}$.
   b. Else if expressionResult is equal to $_{CONT\ B}$ false:
      i. Let context EC_2, state $ARCH_2$ and $expressionResult_{false}$ be the result of evaluating $typedExpressionIR_{false}$, under context EC_1 and state $ARCH_1$ at p.
      ii. The result is context EC_2, state $ARCH_2$ and $expressionResult_{false}$.

14.8. Cast expressions

P4 provides a limited set of casts between types. A cast is written (t) e, where t is a type and e is an expression.

```
castExpression
    : ( type ) expression
    ;
```

14.8.1. Type checking

Typing `( $type_t$ `) expression, under context TC at p:

1. Let $typeIR_t$ and fresh type variables $typeId^*$ be the result of typing $type_t$, under context TC at p.
2. Check that $typeId^*$ is an empty list.
3. Let bound be bound type variables from the p layer of TC.
4. If $typeIR_t$ is a well-formed type, with bound type variables bound:
    a. Let typedExpressionIR be the result of typing expression, under context TC at p.
    b. Let typeIR be the type of typedExpressionIR.
    c. Let ctk be the compile-time known-ness of typedExpressionIR.
    d. Check that $typeIR_t$ can be the target type of an explicit cast.
    e. Let casts castResult be the result of checking if typeIR can be explicitly cast to $typeIR_t$.
    f. Check that castResult matches pattern OK %.
    g. Let OK $typeIR_{cast}{}^*$ be castResult.
    h. Let $expressionIR_{cast}$ be the expression of typedExpressionIR cast through $typeIR_{cast}{}^*$.
    i. Let expressionNoteIR be a pair of type $typeIR_t$ and compile-time known-ness ctk.
    j. The result is $expressionIR_{cast}$ annotated with expressionNoteIR.

14.8.2. Local compile-time evaluation

Local compile-time evaluation of `( typeIR `) typedExpressionIR annotated with expressionNoteIR, under context TC at p:

1. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
2. Let $value_{cast}$ be value casted to type typeIR.
3. The result is $value_{cast}$.

14.8.3. Compile-time evaluation

Compile-time evaluation of `( typeIR `) typedExpressionIR annotated with expressionNoteIR, under context IC and store STO at p:

1. Let store STO_1 and value be the result of compile-time evaluation of typedExpressionIR, under context IC and

store STO at p.

2. Let $value_{cast}$ be value casted to type typeIR.
3. The result is store STO_1 and $value_{cast}$.

14.8.4. Runtime evaluation

Evaluating `( typeIR `) typedExpressionIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC and state ARCH at p.
2. If let abortResult of type abortResult be expressionResult:
   a. The result is context EC_1, state $ARCH_1$ and abortResult.
3. Else:
   a. Let $_{CONT}$ value be expressionResult.
   b. Let $value_{cast}$ be value casted to type typeIR.
   c. The result is context EC_1, state $ARCH_1$ and $_{CONT}$ $value_{cast}$.

14.9. Invalid header expressions

{#} represents an invalid header expression. See Section 8.6.2 for more details.

```
invalidHeaderExpression
    : {#}
    ;
```

14.9.1. Type checking

After type checking, an invalid header expression is represented as:

```
invalidHeaderExpressionIR = invalidHeaderExpression
```

Typing '{#}', under context TC at p:

1. Let expressionNoteIR be a pair of type HEADER_INVALID and compile-time known-ness LCTK.
2. The result is '{#}' annotated with expressionNoteIR.

14.9.2. Local compile-time evaluation

Local compile-time evaluation of '{#}' annotated with expressionNoteIR, under context TC at p: the result is '{#}'.

14.9.3. Compile-time evaluation

Compile-time evaluation of '{#}' annotated with expressionNoteIR, under context IC and store STO at p:

1. The result is store STO and '{#}'.

14.9.4. Runtime evaluation

Evaluating '{#}' annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let expressionResult be $_{\text{CONT}}$ '{#}'.
2. The result is context EC, state ARCH and expressionResult.

14.10. Sequence expressions

A sequence expression is a comma-separated list of expressions enclosed in curly braces {}. See Section 8.6.3 for details.

```
sequenceOrRecordExpression
    : { sequenceOrRecordElementExpression trailingCommaOpt }
    ;

sequenceElementExpression = expressionList

sequenceOrRecordElementExpression
    : sequenceElementExpression | recordElementExpression
    ;
```

14.10.1. Type checking

After type checking, a sequence expression is represented as:

```
sequenceExpressionIR
    : SEQ { typedExpressionListIR } | SEQ { typedExpressionListIR , ... }
    ;
```

Typing `{ expressionList trailingCommaOpt `}, under context TC at p:

1. Let expression_e^* be expressionList flattened.

2. If '...' is not in $expression_e^*$:
    a. For each $expression_e$ in $expression_e^*$:
        - Let $typedExpressionIR_e$ be the result of typing $expression_e$, under context TC at p.

        Let $typedExpressionIR_e^*$ be the resulting list.
    b. For each $typedExpressionIR_e$ in $typedExpressionIR_e^*$:
        - Let $typeIR_e$ be the type of $typedExpressionIR_e$.

        Let $typeIR_e^*$ be the resulting list.
    c. For each $typedExpressionIR_e$ in $typedExpressionIR_e^*$:
        - Let ctk_e be the compile-time known-ness of $typedExpressionIR_e$.

        Let ctk_e^* be the resulting list.
    d. Let typeIR be SEQ `< $typeIR_e^*$ `>.
    e. Let ctk be the join of ctk_e^*.
    f. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.
    g. The result is SEQ `{ $typedExpressionIR_e^*$ `} annotated with expressionNoteIR.
3. Else:
    a. Let $expression^*$ be the reversal of $expression_e^*$.
    b. Check that $expression^*$ is a non-empty list.
    c. Let expression' :: $expression_{e_h_rev}^*$ be $expression^*$.
    d. Check that expression' has type defaultExpression.
    e. Let defaultExpression be expression'.
    f. Let $expression_{e_h}^*$ be the reversal of $expression_{e_h_rev}^*$.
    g. Check that '...' is not in $expression_{e_h}^*$.
    h. For each $expression_{e_h}$ in $expression_{e_h}^*$:
        - Let $typedExpressionIR_{e_h}$ be the result of typing $expression_{e_h}$, under context TC at p.

        Let $typedExpressionIR_{e_h}^*$ be the resulting list.
    i. For each $typedExpressionIR_{e_h}$ in $typedExpressionIR_{e_h}^*$:
        - Let $typeIR_{e_h}$ be the type of $typedExpressionIR_{e_h}$.

        Let $typeIR_{e_h}^*$ be the resulting list.
    j. For each $typedExpressionIR_{e_h}$ in $typedExpressionIR_{e_h}^*$:
        - Let ctk_{e_h} be the compile-time known-ness of $typedExpressionIR_{e_h}$.

        Let $ctk_{e_h}^*$ be the resulting list.
    k. Let typeIR be SEQ `< $typeIR_{e\_h}^*$ ',' '...' `>.
    l. Let ctk be the join of $ctk_{e_h}^*$.
    m. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.
    n. The result is SEQ `{ $typedExpressionIR_{e\_h}^*$ ',' '...' `} annotated with expressionNoteIR.

14.10.2. Local compile-time evaluation

Local compile-time evaluation of sequenceExpressionIR annotated with expressionNoteIR, under context TC at p:

1. If let SEQ `{ typedExpressionIR* `} be sequenceExpressionIR:
    a. For each typedExpressionIR in typedExpressionIR*:
        - Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.

        Let value* be the resulting list.
    b. The result is SEQ `( value* `).
2. Else:
    a. Let SEQ `{ typedExpressionIR* ',' '...' `} be sequenceExpressionIR.
    b. For each typedExpressionIR in typedExpressionIR*:
        - Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.

        Let value* be the resulting list.
    c. The result is SEQ `( value* ',' '...' `).

14.10.3. Compile-time evaluation

Compile-time evaluation of sequenceExpressionIR annotated with expressionNoteIR, under context IC and store STO at p:

1. If let SEQ `{ typedExpressionListIR `} be sequenceExpressionIR:
    a. Let store STO_1 and value* be the result of compile-time evaluation of typedExpressionListIR, under context IC and store STO at p.
    b. The result is store STO_1 and SEQ `( value* `).
2. Else:
    a. Let SEQ `{ typedExpressionListIR ',' '...' `} be sequenceExpressionIR.
    b. Let store STO_1 and value* be the result of compile-time evaluation of typedExpressionListIR, under context IC and store STO at p.
    c. The result is store STO_1 and SEQ `( value* ',' '...' `).

14.10.4. Runtime evaluation

Evaluating sequenceExpressionIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. If let SEQ `{ typedExpressionListIR `} be sequenceExpressionIR:
    a. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating typedExpressionListIR, under context EC and state ARCH at p.

b. If let abortResult of type abortResult be expressionListResult:

i. The result is context EC_1, state $ARCH_1$ and abortResult.

c. Else:

i. Let $_{CONT}$ value* be expressionListResult.

ii. Let expressionResult be $_{CONT}$ SEQ `( value* `).

iii. The result is context EC_1, state $ARCH_1$ and expressionResult.

2. Else:

a. Let SEQ `{ typedExpressionListIR ',' '...' `} be sequenceExpressionIR.

b. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating typedExpressionListIR, under context EC and state ARCH at p.

c. If let abortResult of type abortResult be expressionListResult:

i. The result is context EC_1, state $ARCH_1$ and abortResult.

d. Else:

i. Let $_{CONT}$ value* be expressionListResult.

ii. Let expressionResult be $_{CONT}$ SEQ `( value* ',' '...' `).

iii. The result is context EC_1, state $ARCH_1$ and expressionResult.

14.11. Record expressions

A record expression is a comma-separated list of named expressions enclosed in curly braces {}. See Section 8.6.4 for details.

```
sequenceOrRecordExpression
    : { sequenceOrRecordElementExpression trailingCommaOpt }
    ;

recordElementExpression
    : name = expression | name = expression , ...
    | name = expression , namedExpressionList
    | name = expression , namedExpressionList , ...
    ;

sequenceOrRecordElementExpression
    : sequenceElementExpression | recordElementExpression
    ;
```

14.11.1. Type checking

After type checking, a record expression is represented as:

```
recordExpressionIR
    : RECORD { namedExpressionListIR } | RECORD { namedExpressionListIR , ... }
    ;
```

Typing `{ recordElementExpression trailingCommaOpt `}, under context TC at p:

1. If let $name_f$ '=' $expression_f$ be recordElementExpression:
    a. Let $nameIR_f$ be the name of $name_f$.
    b. Let $typedExpressionIR_f$ be the result of typing $expression_f$, under context TC at p.
    c. Let $typeIR_f$ be the type of $typedExpressionIR_f$.
    d. Let ctk_f be the compile-time known-ness of $typedExpressionIR_f$.
    e. Let typeIR be RECORD `{ $_{EMPTY}$ $typeIR_f$ $nameIR_f$ ';' `}.
    f. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk_f.
    g. The result is RECORD `{ $nameIR_f$ '=' $typedExpressionIR_f$ `} annotated with expressionNoteIR.
2. Else if let $name_f$ '=' $expression_f$ ',' '...' be recordElementExpression:
    a. Let $nameIR_f$ be the name of $name_f$.
    b. Let $typedExpressionIR_f$ be the result of typing $expression_f$, under context TC at p.
    c. Let $typeIR_f$ be the type of $typedExpressionIR_f$.
    d. Let ctk_f be the compile-time known-ness of $typedExpressionIR_f$.
    e. Let typeIR be RECORD `{ $_{EMPTY}$ $typeIR_f$ $nameIR_f$ ';' ',' '...' `}.
    f. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk_f.
    g. The result is RECORD `{ $nameIR_f$ '=' $typedExpressionIR_f$ ',' '...' `} annotated with expressionNoteIR.
3. Else if let $name_{f_h}$ '=' $expression_{f_h}$ ',' $namedExpressionList_t$ be recordElementExpression:
    a. Let ($name_{f_t}$ '=' $expression_{f_t}$)* be $namedExpressionList_t$ flattened.
    b. Let $name_f^*$ be $name_{f_h}$ :: $name_{f_t}^*$.
    c. For each $name_f$ in $name_f^*$:
        - Let $nameIR_f$ be the name of $name_f$.

        Let $nameIR_f^*$ be the resulting list.
    d. Check that the elements of $nameIR_f^*$ are distinct.
    e. Let $expression_f^*$ be $expression_{f_h}$ :: $expression_{f_t}^*$.
    f. For each $expression_f$ in $expression_f^*$:
        - Let $typedExpressionIR_f$ be the result of typing $expression_f$, under context TC at p.

        Let $typedExpressionIR_f^*$ be the resulting list.
    g. For each $typedExpressionIR_f$ in $typedExpressionIR_f^*$:
        - Let $typeIR_f$ be the type of $typedExpressionIR_f$.

        Let $typeIR_f^*$ be the resulting list.
    h. For each $typedExpressionIR_f$ in $typedExpressionIR_f^*$:
        - Let ctk_f be the compile-time known-ness of $typedExpressionIR_f$.

        Let ctk_f^* be the resulting list.
    i. Check that the length of $nameIR_f^*$ is equal to the length of $typeIR_f^*$.
    j. Let typeIR be RECORD `{ ( $_{EMPTY}$ $typeIR_f$ $nameIR_f$ ';' )* `}.
    k. Let ctk be the join of ctk_f^*.
    l. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.

m. The result is RECORD `{ ( nameIR$_f$ '=' typedExpressionIR$_f$ )$^*$ `} annotated with expressionNoteIR.

4. Else:
    a. Let name$_{f_h}$ '=' expression$_{f_h}$ ',' namedExpressionList$_t$ ',' '...' be recordElementExpression.
    b. Let (name$_{f_t}$ '=' expression$_{f_t}$)* be namedExpressionList$_t$ flattened.
    c. Let name$_f^*$ be name$_{f_h}$ :: name$_{f_t}^*$.
    d. For each name$_f$ in name$_f^*$:

        - Let nameIR$_f$ be the name of name$_f$.

        Let nameIR$_f^*$ be the resulting list.

    e. Check that the elements of nameIR$_f^*$ are distinct.
    f. Let expression$_f^*$ be expression$_{f_h}$ :: expression$_{f_t}^*$.
    g. For each expression$_f$ in expression$_f^*$:

        - Let typedExpressionIR$_f$ be the result of typing expression$_f$, under context TC at p.

        Let typedExpressionIR$_f^*$ be the resulting list.

    h. For each typedExpressionIR$_f$ in typedExpressionIR$_f^*$:

        - Let typeIR$_f$ be the type of typedExpressionIR$_f$.

        Let typeIR$_f^*$ be the resulting list.

    i. For each typedExpressionIR$_f$ in typedExpressionIR$_f^*$:

        - Let ctk$_f$ be the compile-time known-ness of typedExpressionIR$_f$.

        Let ctk$_f^*$ be the resulting list.

    j. Check that the length of nameIR$_f^*$ is equal to the length of typeIR$_f^*$.
    k. Let typeIR be RECORD `{ ( $_{\text{EMPTY}}$ typeIR$_f$ nameIR$_f$ ';' )$^*$ ',' '...' `}.
    l. Let ctk be the join of ctk$_f^*$.
    m. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.
    n. The result is RECORD `{ ( nameIR$_f$ '=' typedExpressionIR$_f$ )$^*$ ',' '...' `} annotated with expressionNoteIR.

14.11.2. Local compile-time evaluation

Local compile-time evaluation of recordExpressionIR annotated with expressionNoteIR, under context TC at p:

1. If let RECORD `{ ( nameIR '=' typedExpressionIR )$^*$ `} be recordExpressionIR:
    a. For each typedExpressionIR in typedExpressionIR*:

        - Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.

        Let value* be the resulting list.

    b. The result is RECORD `{ ( value nameIR ';' )$^*$ `}.
2. Else:

a. Let RECORD `{ ( nameIR '=' typedExpressionIR )* ',' '...' `} be recordExpressionIR.

b. For each typedExpressionIR in typedExpressionIR*:

- Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.

Let value* be the resulting list.

c. The result is RECORD `{ ( value nameIR ';' )* ',' '...' `}.

14.11.3. Compile-time evaluation

Compile-time evaluation of recordExpressionIR annotated with expressionNoteIR, under context IC and store STO at p:

1. If let RECORD `{ ( nameIR '=' typedExpressionIR )* `} be recordExpressionIR:
   a. Let store STO_1 and value* be the result of compile-time evaluation of typedExpressionIR*, under context IC and store STO at p.
   b. Check that the length of nameIR* is equal to the length of value*.
   c. The result is store STO_1 and RECORD `{ ( value nameIR ';' )* `}.
2. Else:
   a. Let RECORD `{ ( nameIR '=' typedExpressionIR )* ',' '...' `} be recordExpressionIR.
   b. Let store STO_1 and value* be the result of compile-time evaluation of typedExpressionIR*, under context IC and store STO at p.
   c. Check that the length of nameIR* is equal to the length of value*.
   d. The result is store STO_1 and RECORD `{ ( value nameIR ';' )* ',' '...' `}.

14.11.4. Runtime evaluation

Evaluating recordExpressionIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. If let RECORD `{ ( nameIR '=' typedExpressionIR )* `} be recordExpressionIR:
   a. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating typedExpressionIR*, under context EC and state ARCH at p.
   b. If let abortResult of type abortResult be expressionListResult:
      i. The result is context EC_1, state $ARCH_1$ and abortResult.
   c. Else:
      i. Let $_{CONT}$ value* be expressionListResult.
      ii. Check that the length of nameIR* is equal to the length of value*.
      iii. Let expressionResult be $_{CONT}$ RECORD `{ ( value nameIR ';' )* `}.
      iv. The result is context EC_1, state $ARCH_1$ and expressionResult.
2. Else:
   a. Let RECORD `{ ( nameIR '=' typedExpressionIR )* ',' '...' `} be recordExpressionIR.

b. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating typedExpressionIR*, under context EC and state ARCH at p.

c. If let abortResult of type abortResult be expressionListResult:

   i. The result is context EC_1, state $ARCH_1$ and abortResult.

d. Else:

   i. Let $_{CONT}$ value* be expressionListResult.

   ii. Check that the length of nameIR* is equal to the length of value*.

   iii. Let expressionResult be $_{CONT}$ RECORD `{ ( value nameIR ';' )* ',' '...' `}.

   iv. The result is context EC_1, state $ARCH_1$ and expressionResult.

14.12. Error access expression

An error access expression accesses a symbolic name in the error namespace. See Section 8.2.4 for details about error types and values.

```
errorAccessExpression
    : ERROR . member
    ;
```

14.12.1. Type checking

After type checking, an error access expression is represented as:

```
errorAccessExpressionIR
    : ERROR . nameIR
    ;
```

Typing ERROR '.' member, under context TC at p:

1. Let nameIR be the name of member.
2. Let $nameIR_{error}$ be "error." concatenated with nameIR.
3. Check that ERROR '.' nameIR is equal to the value of variable $_{BARE}$ $nameIR_{error}$ from the p layer of TC.
4. Let expressionNoteIR be a pair of type ERROR and compile-time known-ness LCTK.
5. The result is ERROR '.' nameIR annotated with expressionNoteIR.

14.12.2. Local compile-time evaluation

Local compile-time evaluation of ERROR '.' nameIR annotated with a pair of type typeIR and compile-time known-ness ctk, under context TC at p:

1. Let nameIR$_{error}$ be "error." concatenated with nameIR.
2. Let value$_{error}$ be the value of variable $_{BARE}$ nameIR$_{error}$ from the p layer of TC.
3. The result is value$_{error}$.

14.12.3. Compile-time evaluation

Compile-time evaluation of ERROR '.' nameIR annotated with expressionNoteIR, under context IC and store STO at p:

1. Let nameIR$_{error}$ be "error." concatenated with nameIR.
2. Let value$_{error}$ be the value of $_{BARE}$ nameIR$_{error}$ from the p layer of IC.
3. The result is store STO and value$_{error}$.

14.12.4. Runtime evaluation

Evaluating ERROR '.' nameIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let nameIR$_{error}$ be "error." concatenated with nameIR.
2. Let value$_{error}$ be the value of $_{BARE}$ nameIR$_{error}$ from the p layer of EC.
3. Let expressionResult be $_{CONT}$ value$_{error}$.
4. The result is context EC, state ARCH and expressionResult.

14.13. Member access expressions

A member access expression accesses a user-defined or built-in field in an aggregate value. Also, it may access an element defined in an enum declaration.

```
memberAccessExpression
    : memberAccessBase . member
    ;

memberAccessBase
    : prefixedTypeName | expression
    ;
```

14.13.1. Type checking

After type checking, a member access expression is represented in P4$_{IR}$ as:

```
memberAccessExpressionIR
```

```
    : memberAccessBaseIR . nameIR
    ;

memberAccessBaseIR
    : TYPE prefixedNameIR | typedExpressionIR
    ;
```

Accessing an enum member

`<span id="Expr_ok-memberAccessExpression-type"></span>`

Typing prefixedTypeName$_{base}$ '.' member, under context TC at p:

1. Let prefixedNameIR$_{base}$ be the prefixed name of prefixedTypeName$_{base}$.
2. Let typeDefIR$^{?}$ be the type definition of prefixedNameIR$_{base}$ from the p layer of TC.
3. Check that typeDefIR$^{?}$ is defined.
4. Let typeDefIR$_{base}$ be typeDefIR$^{?}$.
5. Check that typeDefIR$_{base}$ is monomorphic.
6. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.
7. Let typeIR be typeIR$_{base}$ with typedefs unrolled.
8. If let ENUM _ `{ nameIR$_{field}^{*}$ `} of type simpleEnumTypeIR be typeIR:
    a. Let nameIR be the name of member.
    b. Check that nameIR is in nameIR$_{field}^{*}$.
    c. Let expressionNoteIR be a pair of type typeIR$_{base}$ and compile-time known-ness LCTK.
    d. The result is TYPE prefixedNameIR$_{base}$ '.' nameIR annotated with expressionNoteIR.
9. Else if let ENUM _ `< _ `> `{ ( nameIR$_{field}$ '=' _ ';' )$^{*}$ `} of type serializableEnumTypeIR be typeIR:
    a. Let nameIR be the name of member.
    b. Check that nameIR is in nameIR$_{field}^{*}$.
    c. Let expressionNoteIR be a pair of type typeIR$_{base}$ and compile-time known-ness LCTK.
    d. The result is TYPE prefixedNameIR$_{base}$ '.' nameIR annotated with expressionNoteIR.

Accessing an aggregate member

`<span id="Expr_ok-memberAccessExpression-expression"></span>`

Typing expression$_{base}$ '.' member, under context TC at p:

1. Let typedExpressionIR$_{base}$ be the result of typing expression$_{base}$, under context TC at p.
2. Let typeIR$_{base}$ be the type of typedExpressionIR$_{base}$.
3. Try:
    a. Let typeIR' be typeIR$_{base}$ with typedefs unrolled.
    b. If let HEADER_STACK typeIR `[ n$_{size}$ `] of type headerStackTypeIR be typeIR':
        i. Try:
            A. If "size" is equal to the name of member:
                I. Let expressionNoteIR be a pair of type BIT `< 32 `> and compile-time known-ness LCTK.
                II.

The result is typedExpressionIR$_{base}$ '.' "size" annotated with expressionNoteIR.

ii. Then, try:

A. If "lastIndex" is equal to the name of member:

I. If p is equal to BLOCK and TC.BLOCK.KIND is equal to PARSER or p is equal to LOCAL and TC.LOCAL.KIND is equal to PARSER_STATE:

1. Let expressionNoteIR be a pair of type BIT `< 32 `> and compile-time known-ness DYN.

2. The result is typedExpressionIR$_{base}$ '.' "lastIndex" annotated with expressionNoteIR.

iii. Then, try:

A. If "last" is equal to the name of member:

I. If p is equal to BLOCK and TC.BLOCK.KIND is equal to PARSER or p is equal to LOCAL and TC.LOCAL.KIND is equal to PARSER_STATE:

1. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness DYN.

2. The result is typedExpressionIR$_{base}$ '.' "last" annotated with expressionNoteIR.

iv. Then, try:

A. If "next" is equal to the name of member:

I. If p is equal to BLOCK and TC.BLOCK.KIND is equal to PARSER or p is equal to LOCAL and TC.LOCAL.KIND is equal to PARSER_STATE:

1. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness DYN.

2. The result is typedExpressionIR$_{base}$ '.' "next" annotated with expressionNoteIR.

4. Then, try:

a. Let ctk$_{base}$ be the compile-time known-ness of typedExpressionIR$_{base}$.

b. Let typeIR' be typeIR$_{base}$ with typedefs unrolled.

c. If let STRUCT _ `< _$^*$ `> `{ ( _ typeIR$_f$ nameIR$_f$ ';' )$^*$ `} of type structTypeIR be typeIR':

i. Let nameIR be the name of member.

ii. Let typeIR"$^?$ be the value associated to nameIR in (nameIR$_f$, typeIR$_f$)*.

iii. If typeIR"$^?$ is defined:

A. Let typeIR be typeIR"$^?$.

B. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk$_{base}$.

C. The result is typedExpressionIR$_{base}$ '.' nameIR annotated with expressionNoteIR.

d. Else if let HEADER _ `< _$^*$ `> `{ ( _ typeIR$_f$ nameIR$_f$ ';' )$^*$ `} of type headerTypeIR be typeIR':

i. Let nameIR be the name of member.

ii. Let typeIR"$^?$ be the value associated to nameIR in (nameIR$_f$, typeIR$_f$)*.

iii. If typeIR"$^?$ is defined:

A. Let typeIR be typeIR"$^?$.

B. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk$_{base}$.

C. The result is typedExpressionIR$_{base}$ '.' nameIR annotated with expressionNoteIR.

e. Else if let HEADER_UNION _ `< _$^*$ `> `{ ( _ typeIR$_f$ nameIR$_f$ ';' )$^*$ `} of type headerUnionTypeIR be typeIR':

i. Let nameIR be the name of member.

ii. Let typeIR"$^?$ be the value associated to nameIR in (nameIR$_f$, typeIR$_f$)*.

iii. If typeIR"$^?$ is defined:

A. Let typeIR be typeIR"$^?$.

B. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk$_{base}$.

C. The result is typedExpressionIR$_{base}$ '.' nameIR annotated with expressionNoteIR.

5. Then, try:
    a. Let typeIR be typeIR$_{base}$ with typedefs unrolled.
    b. Check that typeIR has type tableMetadataStructTypeIR.
    c. Let TABLE_STRUCT _ `{ HIT boolTypeIR ';' MISS _ ';' ACTION_RUN _ ';' `} be typeIR.
    d. Let nameIR be the name of member.
    e. If nameIR is equal to "hit":
        i. Let expressionNoteIR be a pair of type boolTypeIR and compile-time known-ness DYN.
        ii. The result is typedExpressionIR$_{base}$ '.' nameIR annotated with expressionNoteIR.
    f. Else if nameIR is equal to "miss":
        i. Let expressionNoteIR be a pair of type _ and compile-time known-ness DYN.
        ii. The result is typedExpressionIR$_{base}$ '.' nameIR annotated with expressionNoteIR.
    g. Else if nameIR is equal to "action_run":
        i. Let expressionNoteIR be a pair of type _ and compile-time known-ness DYN.
        ii. The result is typedExpressionIR$_{base}$ '.' nameIR annotated with expressionNoteIR.

14.13.2. Local compile-time evaluation

Accessing an enum member

`<span id="Expr_eval_lctk-memberAccessExpressionIR-type"></span>`

Local compile-time evaluation of TYPE prefixedNameIR$_{base}$ '.' nameIR annotated with expressionNoteIR, under context TC at p:

1. Let typeDefIR$^{?}$ be the type definition of prefixedNameIR$_{base}$ from the p layer of TC.
2. Check that typeDefIR$^{?}$ is defined.
3. Let typeDefIR$_{base}$ be typeDefIR$^{?}$.
4. Check that typeDefIR$_{base}$ is monomorphic.
5. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.
6. Let typeIR' be typeIR$_{base}$ with typedefs unrolled.
7. If let ENUM typeId `{ nameIR$_{field}^{*}$ `} of type simpleEnumTypeIR be typeIR':
    a. Check that nameIR is in nameIR$_{field}^{*}$.
    b. The result is typeId '.' nameIR.
8. Else if let ENUM typeId `< typeIR `> `{ ( nameIR$_{field}$ '=' value$_{field}$ ';' )$^{*}$ `} of type serializableEnumTypeIR be typeIR':
    a. Let value'$^{?}$ be the value associated to nameIR in (nameIR$_{field}$, value$_{field}$)*.
    b. Check that value'$^{?}$ is defined.
    c. Let value be value'$^{?}$.
    d. The result is typeId '.' nameIR '.' value.

Accessing an aggregate member

`<span id="Expr_eval_lctk-memberAccessExpressionIR-typedExpressionIR"></span>`

Local compile-time evaluation of typedExpressionIR$_{base}$ '.' nameIR annotated with expressionNoteIR, under context TC at p:

1. Try:
    a. Let typeIR$_{base}$ be the type of typedExpressionIR$_{base}$.
    b. Let typeIR be typeIR$_{base}$ with typedefs unrolled.
    c. If let HEADER_STACK _ `[ n$_{size}$ `] of type headerStackTypeIR be typeIR:
        i. If nameIR is equal to "size": the result is 32 W n$_{size}$.
2. Then, try:
    a. Let value' be the result of local compile-time evaluation of typedExpressionIR$_{base}$, under context TC at p.
    b. If let structValue of type structValue be value':
        i. Let STRUCT _ `{ ( value$_{field}$ nameIR$_{field}$ ';' )$^{*}$ `} be structValue.
        ii. Let value''$^{?}$ be the value associated to nameIR in (nameIR$_{field}$, value$_{field}$)*.
        iii. Check that value''$^{?}$ is defined.
        iv. Let value be value''$^{?}$.
        v. The result is value.
    c. Else if let headerValue of type headerValue be value':
        i. Let HEADER _ `{ _ ';' ( value$_{field}$ nameIR$_{field}$ ';' )$^{*}$ `} be headerValue.
        ii. Let value''$^{?}$ be the value associated to nameIR in (nameIR$_{field}$, value$_{field}$)*.
        iii. Check that value''$^{?}$ is defined.
        iv. Let value be value''$^{?}$.
        v. The result is value.
    d. Else if let headerUnionValue of type headerUnionValue be value':
        i. Let HEADER_UNION _ `{ ( value$_{field}$ nameIR$_{field}$ ';' )$^{*}$ `} be headerUnionValue.
        ii. Let value''$^{?}$ be the value associated to nameIR in (nameIR$_{field}$, value$_{field}$)*.
        iii. Check that value''$^{?}$ is defined.
        iv. Let value be value''$^{?}$.
        v. The result is value.

14.13.3. Compile-time evaluation

Accessing an enum member

```
<span id="Expr_inst-memberAccessExpressionIR-type"></span>
```

Compile-time evaluation of TYPE prefixedNameIR '.' nameIR annotated with expressionNoteIR, under context IC and store STO at p:

1. Let typeDefIR$^{?}$ be the type definition of prefixedNameIR from the p layer of IC.
2. Check that typeDefIR$^{?}$ is defined.
3. Let typeDefIR$_{base}$ be typeDefIR$^{?}$.
4. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.
5. Let typeIR be typeIR$_{base}$ with typedefs unrolled.

6. If let ENUM typeId `{ _ *`} of type simpleEnumTypeIR be typeIR:
    a. The result is store STO and typeId '.' nameIR.
7. Else if let ENUM typeId `< _ `> `{ ( $id_{member}$ '=' $value_{member}$ ';' )* `} of type serializableEnumTypeIR be typeIR:
    a. Let value'? be the value associated to nameIR in (id_{member}, $value_{member}$)*.
    b. Check that value'? is defined.
    c. Let value be value'?.
    d. The result is store STO and typeId '.' nameIR '.' value.

Accessing an aggregate member

Compile-time evaluation of $typedExpressionIR_{base}$ '.' nameIR annotated with expressionNoteIR, under context IC and store STO at p:

1. Try:
    a. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
    b. Let typeIR be $typeIR_{base}$ with typedefs unrolled.
    c. If let HEADER_STACK _ `[ $n_{size}$ `] of type headerStackTypeIR be typeIR:
        i. If nameIR is equal to "size":
            A. The result is store STO and 32 W n_{size}.
2. Then, try:
    a. Let store STO_1 and value' be the result of compile-time evaluation of $typedExpressionIR_{base}$, under context IC and store STO at p.
    b. If let structValue of type structValue be value':
        i. Let STRUCT _ `{ ( $value_{field}$ $id_{field}$ ';' )* `} be structValue.
        ii. Let value''? be the value associated to nameIR in (id_{field}, $value_{field}$)*.
        iii. Check that value''? is defined.
        iv. Let value be value''?.
        v. The result is store STO_1 and value.
    c. Else if let headerValue of type headerValue be value':
        i. Let HEADER _ `{ _ ';' ( $value_{field}$ $id_{field}$ ';' )* `} be headerValue.
        ii. Let value''? be the value associated to nameIR in (id_{field}, $value_{field}$)*.
        iii. Check that value''? is defined.
        iv. Let value be value''?.
        v. The result is store STO_1 and value.
    d. Else if let headerUnionValue of type headerUnionValue be value':
        i. Let HEADER_UNION _ `{ ( $value_{field}$ $id_{field}$ ';' )* `} be headerUnionValue.
        ii. Let value''? be the value associated to nameIR in (id_{field}, $value_{field}$)*.
        iii. Check that value''? is defined.
        iv. Let value be value''?.
        v. The result is store STO_1 and value.

14.13.4. Runtime evaluation

Accessing an enum member

Evaluating TYPE prefixedNameIR '.' nameIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let typeDefIR? be $find_typeDef_e(p, EC, prefixedNameIR).
2. Check that typeDefIR? is defined.
3. Let $typeDefIR_{base}$ be typeDefIR?.
4. Let $typeIR_{base}$ be the underlying type of $typeDefIR_{base}$.
5. Let typeIR be $typeIR_{base}$ with typedefs unrolled.
6. If let ENUM typeId `{ _ *`} of type simpleEnumTypeIR be typeIR:
   a. Let expressionResult be $_{CONT}$ typeId '.' nameIR.
   b. The result is context EC, state ARCH and expressionResult.
7. Else if let ENUM typeId `< _ `> `{ ( $nameIR_{member}$ '=' $value_{member}$ ';' )* `} of type serializableEnumTypeIR be typeIR:
   a. Let value'? be the value associated to nameIR in ($nameIR_{member}$, $value_{member}$)*.
   b. Check that value'? is defined.
   c. Let value be value'?.
   d. Let expressionResult be $_{CONT}$ typeId '.' nameIR '.' value.
   e. The result is context EC, state ARCH and expressionResult.

Accessing an aggregate member

Evaluating $typedExpressionIR_{base}$ '.' nameIR annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Try:
   a. Let context EC_1, state $ARCH_1$ and expressionResult' be the result of evaluating $typedExpressionIR_{base}$, under context EC and state ARCH at p.
   b. If let abortResult of type abortResult be expressionResult':
      i. The result is context EC_1, state $ARCH_1$ and abortResult.
   c. Else:
      i. Let $_{CONT}$ value be expressionResult'.
      ii. If let $headerStackValue_{base}$ of type headerStackValue be value:
         A. Let HEADER_STACK `[ $value_{elem}$* `(_ `) `] be $headerStackValue_{base}$.
         B. If nameIR is equal to "size":
            I. Let n_{size} be the length of $value_{elem}$*.
            II. Let expressionResult be $_{CONT}$ 32 W n_{size}.
            III. The result is context EC_1, state $ARCH_1$ and expressionResult.
         C. Else if nameIR is equal to "last":
            I. Let n_{size} be the length of $value_{elem}$*.

II. Let $b_{out_of_bounds}$ be _ is less than 1 or _ is greater than n_{size}.

III. If $b_{out_of_bounds}$:

1. Let rejectTransitionResult be REJECT ERROR '.' "StackOutOfBounds".
2. The result is context EC_1, state $ARCH_1$ and rejectTransitionResult.

IV. Else:

1. Let i be _ - 1.
2. If let n_{idx_last} of type nat be i:
    1. If n_{idx_last} is less than the length of $value_{elem}^{*}$:
        1. Let $value_{last}$ be $value_{elem}^{*}[n_{idx_last}]$.
        2. Let expressionResult be CONT $value_{last}$.
        3. The result is context EC_1, state $ARCH_1$ and expressionResult.

D. Else if nameIR is equal to "lastIndex":

I. Let nat' be max`( [ _, 1 ] `).

II. Let i be nat' - 1.

III. If let n_{idx_last} of type nat be i:

1. Let $value_{idx_last}$ be 32 W n_{idx_last}.
2. Let expressionResult be CONT $value_{idx_last}$.
3. The result is context EC_1, state $ARCH_1$ and expressionResult.

iii. Else if let $structValue_{base}$ of type structValue be value:

A. Let STRUCT _ `{ ( $value_{field}$ $id_{field}$ ';' )* `} be $structValue_{base}$.

B. Let $value'^{?}$ be the value associated to nameIR in (id_{field}, $value_{field}$)*.

C. If $value'^{?}$ is defined:

I. Let $value_{member}$ be $value'^{?}$.

II. Let expressionResult be CONT $value_{member}$.

III. The result is context EC_1, state $ARCH_1$ and expressionResult.

iv. Else if let $headerValue_{base}$ of type headerValue be value:

A. Let HEADER _ `{ _ ';' ( $value_{field}$ $id_{field}$ ';' )* `} be $headerValue_{base}$.

B. Let $value'^{?}$ be the value associated to nameIR in (id_{field}, $value_{field}$)*.

C. If $value'^{?}$ is defined:

I. Let $value_{member}$ be $value'^{?}$.

II. Let expressionResult be CONT $value_{member}$.

III. The result is context EC_1, state $ARCH_1$ and expressionResult.

v. Else if let $headerUnionValue_{base}$ of type headerUnionValue be value:

A. Let HEADER_UNION _ `{ ( $value_{field}$ $id_{field}$ ';' )* `} be $headerUnionValue_{base}$.

B. Let $value'^{?}$ be the value associated to nameIR in (id_{field}, $value_{field}$)*.

C. If $value'^{?}$ is defined:

I. Let $value_{member}$ be $value'^{?}$.

II. Let expressionResult be CONT $value_{member}$.

III. The result is context EC_1, state $ARCH_1$ and expressionResult.

2. Then, try:

a. Let context EC_1, state $ARCH_1$ and expressionResult' be the result of evaluating $typedExpressionIR_{base}$, under

context EC and state ARCH at p.

b. Check that expressionResult' has type continueResult<value>.

c. Let $_{CONT}$ value be expressionResult'.

d. Check that value has type tableMetadataStructValue.

e. Let $tableMetadataStructValue_{base}$ be value.

f. Let TABLE_STRUCT _ `{ HIT $boolValue_{hit}$ ';' MISS _ ';' ACTION_RUN _ ';' `} be $tableMetadataStructValue_{base}$.

g. If nameIR is equal to "hit":

   i. Let expressionResult be $_{CONT}$ $boolValue_{hit}$.

   ii. The result is context EC_1, state $ARCH_1$ and expressionResult.

h. Else if nameIR is equal to "miss":

   i. Let expressionResult be $_{CONT}$ _.

   ii. The result is context EC_1, state $ARCH_1$ and expressionResult.

i. Else if nameIR is equal to "action_run":

   i. Let expressionResult be $_{CONT}$ _.

   ii. The result is context EC_1, state $ARCH_1$ and expressionResult.

14.14. Index access expressions

An index access expression accesses an element of a tuple or a header stack.

```
indexAccessExpression
    : expression [ expression ]
    ;
```

14.14.1. Type checking

After type checking, an index access expression is represented as:

```
indexAccessExpressionIR
    : typedExpressionIR [ typedExpressionIR ]
    ;
```

Typing $expression_{base}$ `[ $expression_{index}$ `], under context TC at p:

1. Let $typedExpressionIR_{base}$ be the result of typing $expression_{base}$, under context TC at p.
2. Let $typedExpressionIR_{index}$ be the result of typing $expression_{index}$, under context TC at p.
3. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
4. Let ctk_{base} be the compile-time known-ness of $typedExpressionIR_{base}$.
5. Let $typeIR_{index}$ be the type of $typedExpressionIR_{index}$.
6.

Let ctk_{index} be the compile-time known-ness of $typedExpressionIR_{index}$.

7. Let $typedExpressionIR^?$ be the result of reducing serializable enums in $typedExpressionIR_{index}$ until $compat_array_index is satisfied.
8. Check that $typedExpressionIR^?$ is defined.
9. Let $typedExpressionIR_{index_reduced}$ be $typedExpressionIR^?$.
10. Let typeIR' be $typeIR_{base}$ with typedefs unrolled.
11. If let TUPLE `< $typeIR_e^*$ `> of type tupleTypeIR be typeIR':
    a. Check that ctk_{index} is LCTK.
    b. Let value be the result of local compile-time evaluation of $typedExpressionIR_{index_reduced}$, under context TC at p.
    c. Check that value has type integerValue.
    d. Let $integerValue_{index}$ be value.
    e. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
    f. Check that $nat^?$ is defined.
    g. Let n_{index} be $nat^?$.
    h. Check that n_{index} is less than the length of $typeIR_e^*$.
    i. Check that n_{index} is less than the length of $typeIR_e^*$.
    j. Let expressionNoteIR be a pair of type $typeIR_e^*[n_{index}]$ and compile-time known-ness ctk_{base}.
    k. The result is $typedExpressionIR_{base}$ `[ $typedExpressionIR_{index\_reduced}$ `] annotated with expressionNoteIR.
12. Else if let ARRAY typeIR `[ $n_{size}$ `] of type arrayTypeIR be typeIR':
    a. If ctk_{index} is LCTK:
        i. Let value be the result of local compile-time evaluation of $typedExpressionIR_{index_reduced}$, under context TC at p.
        ii. Check that value has type integerValue.
        iii. Let $integerValue_{index}$ be value.
        iv. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
        v. Check that $nat^?$ is defined.
        vi. Let n_{index} be $nat^?$.
        vii. Check that n_{index} is less than n_{size}.
        viii. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk_{base}.
        ix. The result is $typedExpressionIR_{base}$ `[ $typedExpressionIR_{index\_reduced}$ `] annotated with expressionNoteIR.
    b. Else:
        i. Let ctk be the join of ctk_{base} and ctk_{index}.
        ii. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.
        iii. The result is $typedExpressionIR_{base}$ `[ $typedExpressionIR_{index\_reduced}$ `] annotated with expressionNoteIR.
13. Else if let HEADER_STACK typeIR `[ $n_{size}$ `] of type headerStackTypeIR be typeIR':
    a. If ctk_{index} is LCTK:
        i. Let value be the result of local compile-time evaluation of $typedExpressionIR_{index_reduced}$, under context TC at p.
        ii. Check that value has type integerValue.
        iii. Let $integerValue_{index}$ be value.
        iv. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
        v. Check that $nat^?$ is defined.

vi. Let n_{index} be $nat^?$.

vii. Check that n_{index} is less than n_{size}.

viii. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk_{base}.

ix. The result is $typedExpressionIR_{base}$ `[ $typedExpressionIR_{index\_reduced}$ `] annotated with expressionNoteIR.

b. Else:

i. Let ctk be the join of ctk_{base} and ctk_{index}.

ii. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.

iii. The result is $typedExpressionIR_{base}$ `[ $typedExpressionIR_{index\_reduced}$ `] annotated with expressionNoteIR.

14.14.2. Local compile-time evaluation

Local compile-time evaluation of $typedExpressionIR_{base}$ `[ $typedExpressionIR_{index}$ `] annotated with expressionNoteIR, under context TC at p:

1. Let $value_{base}$ be the result of local compile-time evaluation of $typedExpressionIR_{base}$, under context TC at p.
2. Let value' be the result of local compile-time evaluation of $typedExpressionIR_{index}$, under context TC at p.
3. Check that value' has type integerValue.
4. Let $integerValue_{index}$ be value'.
5. If let TUPLE `( $value_e^*$ `) of type tupleValue be $value_{base}$:
   a. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
   b. Check that $nat^?$ is defined.
   c. Let n_{index} be $nat^?$.
   d. Check that n_{index} is less than the length of $value_e^*$.
   e. Check that n_{index} is less than the length of $value_e^*$.
   f. Let value be $value_e^*[n_{index}]$.
   g. The result is value.
6. Else if let ARRAY `[ $value_e^*$ `] of type arrayValue be $value_{base}$:
   a. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
   b. Check that $nat^?$ is defined.
   c. Let n_{index} be $nat^?$.
   d. Check that n_{index} is less than the length of $value_e^*$.
   e. Check that n_{index} is less than the length of $value_e^*$.
   f. Let value be $value_e^*[n_{index}]$.
   g. The result is value.
7. Else if let HEADER_STACK `[ $value_e^*$ `(_ `) `] of type headerStackValue be $value_{base}$:
   a. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
   b. Check that $nat^?$ is defined.
   c. Let n_{index} be $nat^?$.
   d. Check that n_{index} is less than the length of $value_e^*$.
   e. Check that n_{index} is less than the length of $value_e^*$.

f. Let value be $value_e^*[n_{index}]$.

g. The result is value.

14.14.3. Compile-time evaluation

Compile-time evaluation of $typedExpressionIR_{base}$ `[ $typedExpressionIR_{index}$ `] annotated with expressionNoteIR, under context IC and store STO at p:

1. Let store STO_1 and $value_{base}$ be the result of compile-time evaluation of $typedExpressionIR_{base}$, under context IC and store STO at p.
2. Let store STO_2 and value' be the result of compile-time evaluation of $typedExpressionIR_{index}$, under context IC and store STO_1 at p.
3. Check that value' has type integerValue.
4. Let $integerValue_{index}$ be value'.
5. If let TUPLE `( $value_e^*$ `) of type tupleValue be $value_{base}$:
   a. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
   b. Check that $nat^?$ is defined.
   c. Let n_{index} be $nat^?$.
   d. Check that n_{index} is less than the length of $value_e^*$.
   e. Check that n_{index} is less than the length of $value_e^*$.
   f. Let value be $value_e^*[n_{index}]$.
   g. The result is store STO_2 and value.
6. Else if let ARRAY `[ $value_e^*$ `] of type arrayValue be $value_{base}$:
   a. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
   b. Check that $nat^?$ is defined.
   c. Let n_{index} be $nat^?$.
   d. Check that n_{index} is less than the length of $value_e^*$.
   e. Check that n_{index} is less than the length of $value_e^*$.
   f. Let value be $value_e^*[n_{index}]$.
   g. The result is store STO_2 and value.
7. Else if let HEADER_STACK `[ $value_e^*$ `(_ `) `] of type headerStackValue be $value_{base}$:
   a. Let $nat^?$ be the natural number representation of $integerValue_{index}$.
   b. Check that $nat^?$ is defined.
   c. Let n_{index} be $nat^?$.
   d. Check that n_{index} is less than the length of $value_e^*$.
   e. Check that n_{index} is less than the length of $value_e^*$.
   f. Let value be $value_e^*[n_{index}]$.
   g. The result is store STO_2 and value.

14.14.4. Runtime evaluation

Evaluating typedExpressionIR$_{base}$ `[ typedExpressionIR$_{index}$ `] annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let context EC$_1$, state ARCH$_1$ and expressionListResult be the result of evaluating [typedExpressionIR$_{base}$, typedExpressionIR$_{index}$], under context EC and state ARCH at p.
2. If let abortResult of type abortResult be expressionListResult:
   a. The result is context EC$_1$, state ARCH$_1$ and abortResult.
3. Else:
   a. Let $_{CONT}$ value''* be expressionListResult.
   b. Check that value''* is a list of length 2.
   c. Let [value, value'] be value''*.
   d. If let tupleValue$_{base}$ of type tupleValue be value:
      i. Check that value' has type integerValue.
      ii. Let integerValue$_{index}$ be value'.
      iii. Let TUPLE `( value$_{elem}^*$ `) be tupleValue$_{base}$.
      iv. Let nat$^?$ be the natural number representation of integerValue$_{index}$.
      v. Check that nat$^?$ is defined.
      vi. Let n$_{index}$ be nat$^?$.
      vii. Check that n$_{index}$ is less than the length of value$_{elem}^*$.
      viii. Let expressionResult be $_{CONT}$ value$_{elem}^*$[n$_{index}$].
      ix. The result is context EC$_1$, state ARCH$_1$ and expressionResult.
   e. Else if let arrayValue$_{base}$ of type arrayValue be value:
      i. Check that value' has type integerValue.
      ii. Let integerValue$_{index}$ be value'.
      iii. Let ARRAY `[ value$_{elem}^*$ `] be arrayValue$_{base}$.
      iv. Let n$_{size}$ be the length of value$_{elem}^*$.
      v. Let i$_{index}$ be the integer representation of integerValue$_{index}$.
      vi. Let b$_{in_bounds}$ be i$_{index}$ is less than n$_{size}$ and i$_{index}$ is greater than or equal to 0.
      vii. Let i be i$_{index}$ if b$_{in_bounds}$ otherwise n$_{size}$ - 1.
      viii. Check that i has type nat.
      ix. Let n$_{idx}$ be i.
      x. Check that n$_{idx}$ is less than the length of value$_{elem}^*$.
      xi. Let expressionResult be $_{CONT}$ value$_{elem}^*$[n$_{idx}$].
      xii. The result is context EC$_1$, state ARCH$_1$ and expressionResult.
   f. Else if let headerStackValue$_{base}$ of type headerStackValue be value:
      i. Check that value' has type integerValue.
      ii. Let integerValue$_{index}$ be value'.
      iii. Let HEADER_STACK `[ value$_{elem}^*$ `(_ `) `] be headerStackValue$_{base}$.
      iv. Let n$_{size}$ be the length of value$_{elem}^*$.

v. Let i_{index} be the integer representation of $integerValue_{index}$.

vi. Let b_{in_bounds} be i_{index} is less than n_{size} and i_{index} is greater than or equal to 0.

vii. Let i be i_{index} if b_{in_bounds} otherwise n_{size} - 1.

viii. Check that i has type nat.

ix. Let n_{idx} be i.

x. Check that n_{idx} is less than the length of $value_{elem}^{*}$.

xi. Let expressionResult be $_{CONT}\ value_{elem}^{*}[n_{idx}]$.

xii. The result is context EC_1, state $ARCH_1$ and expressionResult.

14.15. Bitslice access expressions

A bitslice access expression is used to access a contiguous range of bits within a numeric value. The syntax for a bitslice access expression is as follows:

```
sliceAccessExpression
    : expression [ expression sliceop expression ]
    ;
```

14.15.1. Type checking

After type checking, a bitslice access expression is represented as:

```
sliceAccessExpressionIR
    : typedExpressionIR [ typedExpressionIR sliceop typedExpressionIR ]
    ;
```

Typing $expression_{base}$ `[ expression sliceop expression' `], under context TC at p:

1. If sliceop matches pattern ':':
   a. Let $typedExpressionIR_{base}$ be the result of typing $expression_{base}$, under context TC at p.
   b. Let $typedExpressionIR_{hi}$ be the result of typing expression, under context TC at p.
   c. Let $typedExpressionIR_{lo}$ be the result of typing expression', under context TC at p.
   d. Let $typedExpressionIR^{?}$ be the result of reducing serializable enums in $typedExpressionIR_{base}$ until $compat_bitslice_base is satisfied.
   e. Check that $typedExpressionIR^{?}$ is defined.
   f. Let $typedExpressionIR_{base_reduced}$ be $typedExpressionIR^{?}$.
   g. Let $typeIR_{base_reduced}$ be the type of $typedExpressionIR_{base_reduced}$.
   h. Let $ctk_{base_reduced}$ be the compile-time known-ness of $typedExpressionIR_{base_reduced}$.
   i. Let $typedExpressionIR'^{?}$ be the result of reducing serializable enums in $typedExpressionIR_{hi}$ until $compat_bitslice_range_index is satisfied.
   j. Check that $typedExpressionIR'^{?}$ is defined.

k. Let typedExpressionIR$_{hi_reduced}$ be typedExpressionIR$^{?}$.

l. Let typedExpressionIR''$^{?}$ be the result of reducing serializable enums in typedExpressionIR$_{lo}$ until $compat_bitslice_range_index is satisfied.

m. Check that typedExpressionIR''$^{?}$ is defined.

n. Let typedExpressionIR$_{lo_reduced}$ be typedExpressionIR''$^{?}$.

o. Let typeIR$_{hi_reduced}$ be the type of typedExpressionIR$_{hi_reduced}$.

p. Let ctk$_{hi_reduced}$ be the compile-time known-ness of typedExpressionIR$_{hi_reduced}$.

q. Let typeIR$_{lo_reduced}$ be the type of typedExpressionIR$_{lo_reduced}$.

r. Let ctk$_{lo_reduced}$ be the compile-time known-ness of typedExpressionIR$_{lo_reduced}$.

s. Check that ctk$_{hi_reduced}$ is LCTK.

t. Let value be the result of local compile-time evaluation of typedExpressionIR$_{hi_reduced}$, under context TC at p.

u. Check that value has type integerValue.

v. Let integerValue$_{hi}$ be value.

w. Let nat$^{?}$ be the natural number representation of integerValue$_{hi}$.

x. Check that nat$^{?}$ is defined.

y. Let n$_{hi}$ be nat$^{?}$.

z. Check that ctk$_{lo_reduced}$ is LCTK.

aa. Let value' be the result of local compile-time evaluation of typedExpressionIR$_{lo_reduced}$, under context TC at p.

ab. Check that value' has type integerValue.

ac. Let integerValue$_{lo}$ be value'.

ad. Let nat'$^{?}$ be the natural number representation of integerValue$_{lo}$.

ae. Check that nat'$^{?}$ is defined.

af. Let n$_{lo}$ be nat'$^{?}$.

ag. Check that typeIR$_{base_reduced}$ `[ n$_{lo}$ ':' n$_{hi}$ `] is a valid bitslice.

ah. Let i be n$_{hi}$ - n$_{lo}$ + 1.

ai. Check that i has type nat.

aj. Let n$_{slice}$ be i.

ak. Let typeIR be BIT `< n$_{slice}$ `>.

al. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk$_{base_reduced}$.

am. The result is typedExpressionIR$_{base}$ `[ typedExpressionIR$_{hi\_reduced}$ ':' typedExpressionIR$_{lo\_reduced}$ `] annotated with expressionNoteIR.

2. Else:

a. Let typedExpressionIR$_{base}$ be the result of typing expression$_{base}$, under context TC at p.

b. Let typedExpressionIR$_{i}$ be the result of typing expression, under context TC at p.

c. Let typedExpressionIR$_{w}$ be the result of typing expression', under context TC at p.

d. Let typedExpressionIR$^{?}$ be the result of reducing serializable enums in typedExpressionIR$_{base}$ until $compat_bitslice_base is satisfied.

e. Check that typedExpressionIR$^{?}$ is defined.

f. Let typedExpressionIR$_{base_reduced}$ be typedExpressionIR$^{?}$.

g. Let typeIR$_{base_reduced}$ be the type of typedExpressionIR$_{base_reduced}$.

h. Let ctk$_{base_reduced}$ be the compile-time known-ness of typedExpressionIR$_{base_reduced}$.

i. Let typedExpressionIR'$^{?}$ be the result of reducing serializable enums in typedExpressionIR$_{i}$ until

$compat_bitslice_offset_index is satisfied.

j. Check that typedExpressionIR'? is defined.

k. Let typedExpressionIR$_{l_reduced}$ be typedExpressionIR'?.

l. Let typedExpressionIR''? be the result of reducing serializable enums in typedExpressionIR$_{w}$ until $compat_bitslice_offset_width is satisfied.

m. Check that typedExpressionIR''? is defined.

n. Let typedExpressionIR$_{w_reduced}$ be typedExpressionIR''?.

o. Let typeIR$_{l_reduced}$ be the type of typedExpressionIR$_{l_reduced}$.

p. Let ctk$_{l_reduced}$ be the compile-time known-ness of typedExpressionIR$_{l_reduced}$.

q. Let typeIR$_{w_reduced}$ be the type of typedExpressionIR$_{w_reduced}$.

r. Let ctk$_{w_reduced}$ be the compile-time known-ness of typedExpressionIR$_{w_reduced}$.

s. If ctk$_{l_reduced}$ is LCTK:

   i. Let value be the result of local compile-time evaluation of typedExpressionIR$_{l_reduced}$, under context TC at p.

   ii. Check that value has type integerValue.

   iii. Let integerValue$_{l}$ be value.

   iv. Let nat? be the natural number representation of integerValue$_{l}$.

   v. Check that nat? is defined.

   vi. Let n$_{l}$ be nat?.

   vii. Check that ctk$_{w_reduced}$ is LCTK.

   viii. Let value' be the result of local compile-time evaluation of typedExpressionIR$_{w_reduced}$, under context TC at p.

   ix. Check that value' has type integerValue.

   x. Let integerValue$_{w}$ be value'.

   xi. Let nat'? be the natural number representation of integerValue$_{w}$.

   xii. Check that nat'? is defined.

   xiii. Let n$_{w}$ be nat'?.

   xiv. Let i be n$_{l}$ if n$_{w}$ is equal to 0 otherwise n$_{w}$ + n$_{l}$ - 1.

   xv. Check that i has type nat.

   xvi. Let n$_{h}$ be i.

   xvii. Check that typeIR$_{base_reduced}$ `[ n$_{l}$ ':' n$_{h}$ `] is a valid bitslice.

   xviii. Let typeIR be BIT `< n$_{w}$ `>.

   xix. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk$_{base_reduced}$.

   xx. The result is typedExpressionIR$_{base}$ `[ typedExpressionIR$_{l\_reduced}$ '+:' typedExpressionIR$_{w\_reduced}$ `] annotated with expressionNoteIR.

t. Else:

   i. Let typeIR' be typeIR$_{l_reduced}$ with typedefs unrolled.

   ii. Check that typeIR' has type fixedBitTypeIR.

   iii. Check that ctk$_{w_reduced}$ is LCTK.

   iv. Let value be the result of local compile-time evaluation of typedExpressionIR$_{w_reduced}$, under context TC at p.

   v. Check that value has type integerValue.

vi. Let $integerValue_w$ be value.

vii. Let $nat^?$ be the natural number representation of $integerValue_w$.

viii. Check that $nat^?$ is defined.

ix. Let n_w be $nat^?$.

x. Let typeIR be BIT `< $n_w$ `>.

xi. Let ctk be the join of [$ctk_{base_reduced}$, $ctk_{l_reduced}$, $ctk_{w_reduced}$].

xii. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.

xiii. The result is $typedExpressionIR_{base}$ `[ $typedExpressionIR_{l\_reduced}$ '+:' $typedExpressionIR_{w\_reduced}$ `] annotated with expressionNoteIR.

A slice is valid only when its bounds lie within the width of the sliced value:

typeIR' `[ $n_{lo}$ ':' $n_{hi}$ `] is a valid bitslice

1. If let intTypeIR of type intTypeIR be typeIR':
   a. Return n_{lo} is less than or equal to n_{hi}.
2. Else if let INT `< w `> of type fixedIntTypeIR be typeIR':
   a. Let i be n_{hi} - n_{lo} + 1.
   b. If let w_{slice} of type nat be i:
      i. Return n_{lo} is less than or equal to n_{hi} and n_{hi} is less than or equal to w and w_{slice} is less than or equal to w.
3. Else if let BIT `< w `> of type fixedBitTypeIR be typeIR':
   a. Let i be n_{hi} - n_{lo} + 1.
   b. If let w_{slice} of type nat be i:
      i. Return n_{lo} is less than or equal to n_{hi} and n_{hi} is less than or equal to w and w_{slice} is less than or equal to w.
4. Else if let TYPEDEF _ typeIR of type typedefTypeIR be typeIR':
   a. Return typeIR `[ $n_{lo}$ ':' $n_{hi}$ `] is a valid bitslice.
5. Otherwise: return false.

14.15.2. Local compile-time evaluation

`<span id="Expr_eval_lctk-sliceAccessExpressionIR"></span>`

Local compile-time evaluation of $typedExpressionIR_{base}$ `[ typedExpressionIR sliceop typedExpressionIR' `] annotated with expressionNoteIR, under context TC at p:

1. If sliceop matches pattern ':':
   a. Let $value_{base}$ be the result of local compile-time evaluation of $typedExpressionIR_{base}$, under context TC at p.
   b. Let $value_{hi}$ be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
   c. Let $value_{lo}$ be the result of local compile-time evaluation of typedExpressionIR', under context TC at p.
   d. The result is $value_{base}$ `[ $value_{hi}$ ':' $value_{lo}$ `].
2. Else:
   a. Let $value_{base}$ be the result of local compile-time evaluation of $typedExpressionIR_{base}$, under context TC at p.
   b. Let $value_{i}$ be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.

c. Let $value_w$ be the result of local compile-time evaluation of typedExpressionIR', under context TC at p.

d. The result is $value_{base}$ `[ $value_l$ '+:' $value_w$ `].

14.15.3. Compile-time evaluation

Compile-time evaluation of $typedExpressionIR_{base}$ `[ typedExpressionIR sliceop typedExpressionIR' `] annotated with expressionNoteIR, under context IC and store STO at p:

1. If sliceop matches pattern ':':
   a. Let store STO_1 and $value_{base}$ be the result of compile-time evaluation of $typedExpressionIR_{base}$, under context IC and store STO at p.
   b. Let store STO_2 and $value_{hi}$ be the result of compile-time evaluation of typedExpressionIR, under context IC and store STO_1 at p.
   c. Let store STO_3 and $value_{lo}$ be the result of compile-time evaluation of typedExpressionIR', under context IC and store STO_2 at p.
   d. The result is store STO_3 and $value_{base}$ `[ $value_{hi}$ ':' $value_{lo}$ `].
2. Else:
   a. Let store STO_1 and $value_{base}$ be the result of compile-time evaluation of $typedExpressionIR_{base}$, under context IC and store STO at p.
   b. Let store STO_2 and $value_l$ be the result of compile-time evaluation of typedExpressionIR, under context IC and store STO_1 at p.
   c. Let store STO_3 and $value_w$ be the result of compile-time evaluation of typedExpressionIR', under context IC and store STO_2 at p.
   d. The result is store STO_3 and $value_{base}$ `[ $value_l$ '+:' $value_w$ `].

14.15.4. Runtime evaluation

Evaluating $typedExpressionIR_{base}$ `[ typedExpressionIR sliceop typedExpressionIR' `] annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Try:
   a. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating [$typedExpressionIR_{base}$, typedExpressionIR, typedExpressionIR'], under context EC and state ARCH at p.
   b. If let abortResult of type abortResult be expressionListResult:
      i. The result is context EC_1, state $ARCH_1$ and abortResult.
2. Then, try:
   a. If sliceop matches pattern ':':
      i. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating [$typedExpressionIR_{base}$, typedExpressionIR, typedExpressionIR'], under context EC and state ARCH at p.
      ii. Check that expressionListResult has type continueResult<value*>.
      iii. Let $_{CONT}$ $value^*$ be expressionListResult.
      iv. Check that $value^*$ is a list of length 3.
      v. Let [$value_{base}$, $value_{hi}$, $value_{lo}$] be $value^*$.

vi. Let value'''' be $value_{base}$ `[ $value_{hi}$ ':' $value_{lo}$ `].

vii. Let expressionResult be $_{CONT}$ value''''.

viii. The result is context EC_1, state $ARCH_1$ and expressionResult.

b. Else:

i. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating [$typedExpressionIR_{base}$, typedExpressionIR, typedExpressionIR'], under context EC and state ARCH at p.

ii. Check that expressionListResult has type continueResult<value*>.

iii. Let $_{CONT}$ $value^*$ be expressionListResult.

iv. Check that $value^*$ is a list of length 3.

v. Let [$value_{base}$, $value_l$, $value_w$] be $value^*$.

vi. Let value''''' be $value_{base}$ `[ $value_l$ '+:' $value_w$ `].

vii. Let expressionResult be $_{CONT}$ value'''''.

viii. The result is context EC_1, state $ARCH_1$ and expressionResult.

14.16. Call expressions

Call expressions are used to invoke callables or constructors.

```
<span id="callExpression"></span>
<span id="callTarget"></span>
<span id="callableTarget"></span>
<span id="constructorTarget"></span>
```

```
callExpression
    : callTarget ( argumentList )
    | callableTarget < realTypeArgumentList > ( argumentList )
    ;

callTarget
    : callableTarget | constructorTarget
    ;

callableTarget = expression

constructorTarget = namedType
```

Details of how calls are resolved and evaluated are described in Chapter 19.

14.16.1. Type checking

After type checking, call expressions are represented as:

```
<span id="callExpressionIR"></span>
<span id="constructorTargetIR"></span>
```

```
callExpressionIR
    : constructorTargetIR ( argumentListIR )
    | callableTargetIR < typeArgumentListIR > ( argumentListIR )
    ;

callableTargetIR
    : referenceExpressionIR | typedExpressionIR . nameIR
    | TYPE prefixedNameIR . nameIR | ( callableTargetIR )
    ;
```

```
constructorTargetIR
    : prefixedNameIR < typeArgumentListIR >
    ;
```

Calling a callable

Typing callExpression, under context TC at p:

1. If let callTarget `( argumentList `) be callExpression:
    a. If let callableTarget of type callableTarget be callTarget:
        i. Let typed callable target callableTargetIR be the result of typing callableTarget as a callable target, under cursor p and context TC.
        ii. Let argumentIR^* be the result of typing argumentList, under context TC at p.
        iii. Let callable type callableTypeIR with fresh type variables $\text{typeId}_{\text{infer}}{}^*$, defaults $\text{id}_{\text{default}}{}^*$ and optionals $\text{id}_{\text{optional}}{}^*$ be the result of finding type of callable call callableTargetIR `< · `> `( $\text{argumentIR}^*$ `), under cursor p and context TC.
        iv. Let $\text{typeArgumentIR}_{\text{infer}}{}^*$ be $(\ _{\text{NAME}}\ \text{typeId}_{\text{infer}}\ )^*$.
        v. Let return type $\text{typeIR}_{\text{ret}}$ with inferred $\text{typeArgumentIR}_{\text{inferred}}{}^*$ and casted $\text{argumentIR}_{\text{cast}}{}^*$ be the result of typing invocation callableTypeIR `< $\text{typeArgumentIR}_{\text{infer}}{}^*$ `> `( $\text{argumentIR}^*$ `) with omitted type arguments $\text{typeId}_{\text{infer}}{}^*$, defaults $\text{id}_{\text{default}}{}^*$ and optionals $\text{id}_{\text{optional}}{}^*$, under cursor p and context TC.
        vi. If $\text{typeIR}_{\text{ret}}$ is not equal to VOID:
            A. Let ctk be compile-time known-ness of callableTypeIR.
            B. If callableTypeIR is a static_assert call:
                I. Let callExpressionIR be callableTargetIR `< $\text{typeArgumentIR}_{\text{inferred}}{}^*$ `> `( $\text{argumentIR}_{\text{cast}}{}^*$ `).
                II. Let expressionNoteIR be a pair of type $\text{typeIR}_{\text{ret}}$ and compile-time known-ness ctk.
                III. Let value be the result of local compile-time evaluation of callExpressionIR annotated with expressionNoteIR, under context TC at p.
                IV. If value is equal to $_{\text{B}}$ true:
                    1. Let literalExpressionIR be TRUE.
                    2. The result is literalExpressionIR annotated with expressionNoteIR.
            C. Else:
                I. Let callExpressionIR be callableTargetIR `< $\text{typeArgumentIR}_{\text{inferred}}{}^*$ `> `( $\text{argumentIR}_{\text{cast}}{}^*$ `).
                II. Let expressionNoteIR be a pair of type $\text{typeIR}_{\text{ret}}$ and compile-time known-ness ctk.
                III. The result is callExpressionIR annotated with expressionNoteIR.
2. Else:
    a. Let callableTarget `< realTypeArgumentList `> `( argumentList `) be callExpression.
    b. Let typed callable target callableTargetIR be the result of typing callableTarget as a callable target, under cursor p and context TC.
    c. Let $\text{realTypeArgument}^*$ be realTypeArgumentList flattened.
    d. Let typeArgumentIR^* and fresh type variables $\text{typeId}_{\text{impl}}{}^*$ be the result of typing $\text{realTypeArgument}^*$, under context TC at p.
    e. Let argumentIR^* be the result of typing argumentList, under context TC at p.
    f. Let callable type callableTypeIR with fresh type variables $\text{typeId}_{\text{inserted}}{}^*$, defaults $\text{id}_{\text{default}}{}^*$ and optionals $\text{id}_{\text{optional}}{}^*$ be the result of finding type of callable call callableTargetIR `< $\text{typeArgumentIR}^*$ `> `( $\text{argumentIR}^*$ `), under

cursor p and context TC.

g. Let $\text{typeArgumentIR}_{\text{infer}}^{*}$ be $\text{typeArgumentIR}^{*}$ concatenated with (NAME $\text{typeId}_{\text{inserted}}$)*.

h. Let $\text{typeId}_{\text{infer}}^{*}$ be $\text{typeId}_{\text{impl}}^{*}$ concatenated with $\text{typeId}_{\text{inserted}}^{*}$.

i. Let return type $\text{typeIR}_{\text{ret}}$ with inferred $\text{typeArgumentIR}_{\text{inferred}}^{*}$ and casted $\text{argumentIR}_{\text{cast}}^{*}$ be the result of typing invocation callableTypeIR `< $\text{typeArgumentIR}_{\text{infer}}^{*}$ `> `( $\text{argumentIR}^{*}$ `) with omitted type arguments $\text{typeId}_{\text{infer}}^{*}$, defaults $\text{id}_{\text{default}}^{*}$ and optionals $\text{id}_{\text{optional}}^{*}$, under cursor p and context TC.

j. If $\text{typeIR}_{\text{ret}}$ is not equal to VOID:

i. Let ctk be compile-time known-ness of callableTypeIR.

ii. Let callExpressionIR be callableTargetIR `< $\text{typeArgumentIR}_{\text{inferred}}^{*}$ `> `( $\text{argumentIR}_{\text{cast}}^{*}$ `).

iii. Let expressionNoteIR be a pair of type $\text{typeIR}_{\text{ret}}$ and compile-time known-ness ctk.

iv. The result is callExpressionIR annotated with expressionNoteIR.

Whether a call is compile-time known, and whether it is a static_assert call, are determined from the callable type with:

Compile-time known-ness of callableTypeIR

1. If let EXTERN_FUNCTION t `( _* `) ':' _ of type externFunctionTypeIR be callableTypeIR:
   a. If t is equal to "static_assert": return LCTK.
2. Else if callableTypeIR has type builtinSizeMethodTypeIR: return LCTK.
3. Otherwise: return DYN.

callableTypeIR is a static_assert call

1. If let EXTERN_FUNCTION t `( _* `) ':' _ of type externFunctionTypeIR be callableTypeIR:
   a. If t is equal to "static_assert": return true.
2. Otherwise: return false.

Calling a constructor

Typing callTarget `( argumentList `), under context TC at p:

1. If let prefixedTypeName of type prefixedTypeName be callTarget:
   a. Let argumentIR^{*} be the result of typing argumentList, under context TC at p.
   b. Let prefixedNameIR be the prefixed name of prefixedTypeName.
   c. Let constructorTargetIR be prefixedNameIR `< · `>.
   d. Let constructorTypeIR with fresh type variables $\text{typeId}_{\text{impl}}^{*}$, defaults $\text{id}_{\text{default}}^{*}$ and optionals $\text{id}_{\text{optional}}^{*}$ be the result of finding type of constructor call constructorTargetIR `( $\text{argumentIR}^{*}$ `), under cursor p and context TC.
   e. Let $\text{typeArgumentIR}_{\text{infer}}^{*}$ be (NAME $\text{typeId}_{\text{impl}}$)*.
   f. Let constructed type $\text{typeIR}_{\text{object}}$ with inferred $\text{typeArgumentIR}_{\text{inferred}}^{*}$ and casted $\text{argumentIR}_{\text{cast}}^{*}$ be the result of typing instantiation constructorTypeIR `< $\text{typeArgumentIR}_{\text{infer}}^{*}$ `> `( $\text{argumentIR}^{*}$ `) with omitted type arguments $\text{typeId}_{\text{impl}}^{*}$, defaults $\text{id}_{\text{default}}^{*}$ and optionals $\text{id}_{\text{optional}}^{*}$, under cursor p, context TC, and ANON instantiation site context.

g. Check that typeIR$_{object}$ is an extern object type without abstract methods.

h. Let callExpressionIR be prefixedNameIR `< typeArgumentIR$_{inferred}^{*}$ `> `( argumentIR$_{cast}^{*}$ `).

i. Let expressionNoteIR be a pair of type typeIR$_{object}$ and compile-time known-ness CTK.

j. The result is callExpressionIR annotated with expressionNoteIR.

2. Else if let prefixedTypeName `< typeArgumentList `> of type specializedType be callTarget:

a. Let typeArgumentIR* and fresh type variables typeId$_{impl}^{*}$ be the result of typing typeArgumentList, under context TC at p.

b. Let argumentIR* be the result of typing argumentList, under context TC at p.

c. Let prefixedNameIR be the prefixed name of prefixedTypeName.

d. Let constructorTargetIR be prefixedNameIR `< typeArgumentIR$^{*}$ `>.

e. Let constructorTypeIR with fresh type variables typeId$_{inserted}^{*}$, defaults id$_{default}^{*}$ and optionals id$_{optional}^{*}$ be the result of finding type of constructor call constructorTargetIR `( argumentIR$^{*}$ `), under cursor p and context TC.

f. Let typeArgumentIR$_{infer}^{*}$ be typeArgumentIR* concatenated with (NAME typeId$_{inserted}$)*.

g. Let typeId$_{infer}^{*}$ be typeId$_{impl}^{*}$ concatenated with typeId$_{inserted}^{*}$.

h. Let constructed type typeIR$_{object}$ with inferred typeArgumentIR$_{inferred}^{*}$ and casted argumentIR$_{cast}^{*}$ be the result of typing instantiation constructorTypeIR `< typeArgumentIR$_{infer}^{*}$ `> `( argumentIR$^{*}$ `) with omitted type arguments typeId$_{infer}^{*}$, defaults id$_{default}^{*}$ and optionals id$_{optional}^{*}$, under cursor p, context TC, and ANON instantiation site context.

i. Check that typeIR$_{object}$ is an extern object type without abstract methods.

j. Let callExpressionIR be prefixedNameIR `< typeArgumentIR$_{inferred}^{*}$ `> `( argumentIR$_{cast}^{*}$ `).

k. Let expressionNoteIR be a pair of type typeIR$_{object}$ and compile-time known-ness CTK.

l. The result is callExpressionIR annotated with expressionNoteIR.

14.16.2. Local compile-time evaluation

Certain call expressions can be evaluated during type checking:

Local compile-time evaluation of callableTargetIR' `< typeArgumentIR$^{*}$ `> `( argumentIR$^{*}$ `) annotated with expressionNoteIR, under context TC at p:

1. If let prefixedNameIR of type prefixedNameIR be callableTargetIR':

a. Let callable type callableTypeIR with fresh type variables _*, defaults id$_{default}^{*}$ and optionals id$_{optional}^{*}$ be the result of finding type of callable call prefixedNameIR `< typeArgumentIR$^{*}$ `> `( argumentIR$^{*}$ `), under cursor p and context TC.

b. Check that callableTypeIR has type externFunctionTypeIR.

c. Let externFunctionTypeIR be callableTypeIR.

d. Let EXTERN_FUNCTION t `( parameterIR$^{*}$ `) ':' typeIR be externFunctionTypeIR.

e. Check that t is equal to "static_assert".

f. Check that typeIR has type boolTypeIR.

g. Let boolTypeIR be typeIR.

h. For each parameterIR in parameterIR*:

- Let _ _ _ nameIR$_{param}$ _ be parameterIR.

Let nameIR$_{param}$* be the resulting list.

i. Let TC$_{callee_0}$ be an empty typing context.

j. Let GIVEN parameterIR$_{aligned}$* DEFAULT parameterIR'* OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are id$_{default}$* and optionals are id$_{optional}$*.

k. Check that parameterIR'* is an empty list.

l. For each argumentIR in argumentIR*:

- Let value$_{argument}$ be the result of local compile-time evaluation of argumentIR, under context TC at p.

Let value$_{argument}$* be the resulting list.

m. For each parameterIR$_{aligned}$ in parameterIR$_{aligned}$*:

- Let _ direction$_{param_aligned}$ typeIR$_{param_aligned}$ nameIR$_{param_aligned}$ _ be parameterIR$_{aligned}$.

Let direction$_{param_aligned}$*, nameIR$_{param_aligned}$*, and typeIR$_{param_aligned}$* be the resulting lists.

n. Check that the length of direction$_{param_aligned}$* is equal to the length of typeIR$_{param_aligned}$* and the length of typeIR$_{param_aligned}$* is equal to the length of value$_{argument}$*.

o. For each direction$_{param_aligned}$ in direction$_{param_aligned}$*, typeIR$_{param_aligned}$ in typeIR$_{param_aligned}$*, and value$_{argument}$ in value$_{argument}$*:

- Let varTypeIR be direction$_{param_aligned}$ typeIR$_{param_aligned}$ LCTK value$_{argument}$.

Let varTypeIR* be the resulting list.

p. Let TC$_{callee_1}$ be TC$_{callee_0}$ where each of nameIR$_{param_aligned}$* to each of varTypeIR* are added to the LOCAL layer.

q. Let value be the result of calling an extern function "static_assert"(nameIR$_{param}$*) under context TC$_{callee_1}$.

r. Check that value has type boolValue.

s. Let boolValue be value.

t. Check that $_{B}$ true is equal to boolValue.

u. The result is boolValue.

2. Else if let typedExpressionIR$_{base}$ '.' nameIR be callableTargetIR':

a. Let a pair of type typeIR$_{base}$ and compile-time known-ness _ be the note of typedExpressionIR$_{base}$.

b. Check that nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].

c. The result is $sizeof(typeIR$_{base}$, nameIR).

3. Else if let TYPE prefixedNameIR '.' nameIR be callableTargetIR':

a. Let typeDefIR? be the type definition of prefixedNameIR from the p layer of TC.

b. Check that typeDefIR? is defined.

c. Let typeDefIR$_{base}$ be typeDefIR?.

d. If typeDefIR$_{base}$ is monomorphic:

i. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.

ii. Check that nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].

iii. The result is $sizeof(typeIR$_{base}$, nameIR).

e. Else:

i. Check that (·, ·) is equal to the type parameters of typeDefIR$_{base}$.

ii. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.

iii. Check that nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].

iv. The result is $sizeof(typeIR$_{base}$, nameIR).

4. Else:

a. Let `( callableTargetIR `) be callableTargetIR'.

b. Let typeIR and ctk be the type and the compile-time known-ness of expressionNoteIR.

c. Let value be the result of local compile-time evaluation of callableTargetIR `< typeArgumentIR$^{*}$ `> `( argumentIR$^{*}$ `) annotated with a pair of type typeIR and compile-time known-ness ctk, under context TC at p.

d. The result is value.

14.16.3. Compile-time evaluation

Call expressions may also be evaluated at compile time when instantiation phase takes place. A constructor call evaluates to an instance of the constructed type, while a callable call evaluates to the result of the call.

Calling a callable

Compile-time evaluation of callableTargetIR `< _ `> `( _ `) annotated with expressionNoteIR, under context IC and store STO at p:

1. If let typedExpressionIR$_{base}$ '.' nameIR be callableTargetIR:

a. Let a pair of type typeIR$_{base}$ and compile-time known-ness _ be the note of typedExpressionIR$_{base}$.

b. If nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"]:

i. The result is store STO and $sizeof(typeIR$_{base}$, nameIR).

2. Else if let TYPE prefixedNameIR '.' nameIR be callableTargetIR:

a. Let typeDefIR$^{?}$ be the type definition of prefixedNameIR from the p layer of IC.

b. If typeDefIR$^{?}$ is defined:

i. Let typeDefIR$_{base}$ be typeDefIR$^{?}$.

ii. If typeDefIR$_{base}$ is monomorphic:

A. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.

B. If nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"]:

I. The result is store STO and $sizeof(typeIR$_{base}$, nameIR).

iii. Else:

A. If (·, ·) is equal to the type parameters of typeDefIR$_{base}$:

I. Let typeIR$_{base}$ be the underlying type of typeDefIR$_{base}$.

II. If nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"]:

1. The result is store STO and $sizeof(typeIR$_{base}$, nameIR).

Calling a constructor

Compile-time evaluation of prefixedNameIR `< typeArgumentListIR `> `( argumentListIR `) annotated with expressionNoteIR, under context IC and store STO at p:

1. Let constructor constructorDef `< $\text{typeArgumentListIR}_{\text{inst}}$ `> with defaults $\text{id}_{\text{default}}^{*}$ and optionals $\text{id}_{\text{optional}}^{*}$ be the result of finding constructor for prefixedNameIR `< typeArgumentListIR `> `( argumentListIR `) under context IC at p.
2. Let store STO_1 and instantiated object be the result of instantiating constructorDef `< $\text{typeArgumentListIR}_{\text{inst}}$ `> `( argumentListIR `) with defaults $\text{id}_{\text{default}}^{*}$ and optionals $\text{id}_{\text{optional}}^{*}$, under context IC and store STO at p.
3. Let STO_2 be STO_1 where IC.PATH to object is added.
4. Let $\text{value}_{\text{ref}}$ be REF IC.PATH.
5. The result is store STO_2 and $\text{value}_{\text{ref}}$.

Parenthesized call

`<span id="Expr_inst-callExpressionIR-parenthesized"></span>`

Compile-time evaluation of `( callableTargetIR' `) `< _ `> `( _ `) annotated with expressionNoteIR, under context IC and store STO at p:

1. Let store STO_1 and value be the result of compile-time evaluation of callableTargetIR' `< _ `> `( _ `) annotated with expressionNoteIR, under context IC and store STO at p.
2. The result is store STO_1 and value.

14.16.4. Runtime evaluation

`<span id="Expr_eval-callExpressionIR"></span>`

Evaluating callableTargetIR `< $\text{typeArgumentIR}^{*}$ `> `( $\text{argumentIR}^{*}$ `) annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state ARCH_1, and callee calleeResult be the result of finding callee callableTargetIR `( $\text{argumentIR}^{*}$ `), under context EC and state ARCH at p.
2. If let abortResult of type abortResult be calleeResult:
    a. The result is context EC_1, state ARCH_1 and abortResult.
3. Else:
    a. Let $_{\text{CONT}}$ callee be calleeResult.
    b. Let context EC_2, state ARCH_2, and call result callResult be the result of calling callee `< $\text{typeArgumentIR}^{*}$ `> `( $\text{argumentIR}^{*}$ `), under context EC_1 and state ARCH_1 at p.
    c. If let abortResult of type abortResult be callResult:
        i. The result is context EC_2, state ARCH_2 and abortResult.
    d. Else:
        i. Let RETURN $\text{value}'^{?}$ be callResult.
        ii. Check that $\text{value}'^{?}$ is defined.
        iii. Let value be $\text{value}'^{?}$.
        iv. The result is context EC_2, state ARCH_2 and $_{\text{CONT}}$ value.

14.17. Parenthesized expressions

Expressions may be enclosed in parentheses to explicitly specify the order of evaluation.

```
parenthesizedExpression
    : ( expression )
    ;
```

14.17.1. Type checking

After type checking, parenthesized expressions have the form:

```
parenthesizedExpressionIR
    : ( typedExpressionIR )
    ;
```

Typing `( expression `), under context TC at p:

1. Let typedExpressionIR be the result of typing expression, under context TC at p.
2. Let typeIR be the type of typedExpressionIR.
3. Let ctk be the compile-time known-ness of typedExpressionIR.
4. Let expressionNoteIR be a pair of type typeIR and compile-time known-ness ctk.
5. The result is `( typedExpressionIR `) annotated with expressionNoteIR.

14.17.2. Local compile-time evaluation

Local compile-time evaluation of `( typedExpressionIR `) annotated with expressionNoteIR, under context TC at p:

1. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
2. The result is value.

14.17.3. Compile-time evaluation

Compile-time evaluation of `( typedExpressionIR `) annotated with expressionNoteIR, under context IC and store STO at p:

1. Let store STO_1 and value be the result of compile-time evaluation of typedExpressionIR, under context IC and store STO at p.
2. The result is store STO_1 and value.

14.17.4. Runtime evaluation

Evaluating `( typedExpressionIR `) annotated with expressionNoteIR, under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC and state ARCH at p.
2. The result is context EC_1, state $ARCH_1$ and expressionResult.

14.18. Keyset expressions

Keyset expressions are used to represent a set of values to be matched in switch statements (Section 13.13) and parser select expressions (Section 16.6).

The following relation defines how keysets are evaluated:

Eval_keyset:

- Evaluating keysetExpressionIR, under context EC and state ARCH:
- The result is context EC, state ARCH, and result keysetExpressionResult.

The result of evaluating a keyset expression is:

```
keysetExpressionResult
    : continueResult<setValue*> | abortResult
    ;
```

The relation works as follows:

Evaluating keysetExpressionIR, under context EC_0 and state $ARCH_0$:

1. Let simpleKeysetExpressionListIR be the flattened keyset expression list of keysetExpressionIR.
2. Let EC_0 $ARCH_0$ |- simpleKeysetExpressionListIR : EC_1 $ARCH_1$ keysetExpressionResult.
3. The result is context EC_1, state $ARCH_1$, and result keysetExpressionResult.

It decomposes the keysetExpressionIR into simpleKeysetExpressionIRs, evaluates each of those, and then combines the results into a single keysetExpressionResult.

The simpleKeysetExpressionIRs are evaluated as follows:

Eval_keysets_simple: EC ARCH |- simpleKeysetExpressionIR* : % % %

EC ARCH |- simpleKeysetExpressionIR* : % % %:

1. If simpleKeysetExpressionIR* matches pattern []:
    a. The result is EC, ARCH, and $_{CONT}$ ·.
2. Else:
    a. Let simpleKeysetExpressionIR$_h$ :: simpleKeysetExpressionIR$_t$* be simpleKeysetExpressionIR*.
    b. Let context EC$_1$, state ARCH$_1$, and result simpleKeysetExpressionResult be the result of evaluating simpleKeysetExpressionIR$_h$, under context EC and state ARCH.
    c. If let abortResult of type abortResult be simpleKeysetExpressionResult:
        i. The result is EC$_1$, ARCH$_1$, and abortResult.
    d. Else:
        i. Let $_{CONT}$ setValue$_h$ be simpleKeysetExpressionResult.
        ii. Let EC$_1$ ARCH$_1$ |- simpleKeysetExpressionIR$_t$* : EC$_2$ ARCH$_2$ keysetExpressionResult'.
        iii. If let abortResult of type abortResult be keysetExpressionResult':
            A. The result is EC$_2$, ARCH$_2$, and abortResult.
        iv. Else:
            A. Let $_{CONT}$ setValue$_t$* be keysetExpressionResult'.
            B. Let keysetExpressionResult be $_{CONT}$ setValue$_h$ :: setValue$_t$*.
            C. The result is EC$_2$, ARCH$_2$, and keysetExpressionResult.

A simpleKeysetExpressionIR is evaluated as follows:

Eval_keyset_simple:

- Evaluating simpleKeysetExpressionIR, under context EC and state ARCH:
- The result is context EC, state ARCH, and result simpleKeysetExpressionResult.

Eval_keyset_simple dispatch:

1. Try:
    a. If let typedExpressionIR of type typedExpressionIR be simpleKeysetExpressionIR: goto typedExpressionIR
    b. Else if let typedExpressionIR$_l$ '&&&' typedExpressionIR$_r$ be simpleKeysetExpressionIR: goto mask
    c. Else if let typedExpressionIR$_l$ '..' typedExpressionIR$_r$ be simpleKeysetExpressionIR: goto range
2. Then, try:
    a. Goto default-any

The result of evaluating a simpleKeysetExpressionIR is:

```
simpleKeysetExpressionResult
    : continueResult<setValue> | abortResult
    ;
```

Expression keys

Evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let abortResult of type abortResult be expressionResult:
    a. The result is context EC_1, state $ARCH_1$, and result abortResult.
3. Else:
    a. Let $_{CONT}$ value' be expressionResult.
    b. If let setValue of type setValue be value':
        i. The result is context EC_1, state $ARCH_1$, and result $_{CONT}$ setValue.
    c. Else if let REF objectId of type objectReferenceValue be value':
        i. Let object'? be the object objectId in $ARCH_1$.
        ii. If object'? is defined:
            A. Let object be object'?.
            B. If let VALUE_SET `{ value* `(n_{size} `) `} of type valueSetObject be object:
                I. Let setValue be SET `{ value* `(n_{size} `) `}.
                II. The result is context EC_1, state $ARCH_1$, and result $_{CONT}$ setValue.

Mask keys

Evaluating typedExpressionIR$_l$ '&&&' typedExpressionIR$_r$, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR$_l$, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let abortResult of type abortResult be expressionResult:
    a. The result is context EC_1, state $ARCH_1$, and result abortResult.
3. Else:
    a. Let $_{CONT}$ value$_l$ be expressionResult.
    b. Let context EC_2, state $ARCH_2$ and expressionResult' be the result of evaluating typedExpressionIR$_r$, under context EC_1 and state $ARCH_1$ at LOCAL.
    c. If let abortResult of type abortResult be expressionResult':
        i. The result is context EC_2, state $ARCH_2$, and result abortResult.
    d. Else:

i. Let $_{CONT}$ value$_r$ be expressionResult'.

ii. Let setValue be SET `{ value$_l$ '&&&' value$_r$ `}.

iii. The result is context EC_2, state $ARCH_2$, and result $_{CONT}$ setValue.

Range keys

Evaluating typedExpressionIR$_l$ '..' typedExpressionIR$_r$, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR$_l$, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let abortResult of type abortResult be expressionResult:
   a. The result is context EC_1, state $ARCH_1$, and result abortResult.
3. Else:
   a. Let $_{CONT}$ value$_l$ be expressionResult.
   b. Let context EC_2, state $ARCH_2$ and expressionResult' be the result of evaluating typedExpressionIR$_r$, under context EC_1 and state $ARCH_1$ at LOCAL.
   c. If let abortResult of type abortResult be expressionResult':
      i. The result is context EC_2, state $ARCH_2$, and result abortResult.
   d. Else:
      i. Let $_{CONT}$ value$_r$ be expressionResult'.
      ii. Let setValue be SET `{ value$_l$ '..' value$_r$ `}.
      iii. The result is context EC_2, state $ARCH_2$, and result $_{CONT}$ setValue.

Default and don't care keys

Evaluating simpleKeysetExpressionIR, under context EC_0 and state $ARCH_0$:

1. Check that simpleKeysetExpressionIR is equal to DEFAULT or simpleKeysetExpressionIR is equal to '_'.
2. Let setValue be SET `{ '...' `}.
3. The result is context EC_0, state $ARCH_0$, and result $_{CONT}$ setValue.

15. Arguments and type arguments

The syntax of arguments and type arguments is defined as follows:

```
argument
    : expression | name = expression | name = _ | _
    ;
```

```
argumentListNonEmpty
    : argument | argumentListNonEmpty , argument
    ;

argumentList
    : /* empty */ | argumentListNonEmpty
    ;
```

```
realTypeArgument
    : type | VOID | _
    ;

nonTypeName
    : identifier | APPLY | KEY | ACTIONS | STATE | ENTRIES | TYPE | PRIORITY
    ;

typeArgument
    : realTypeArgument | nonTypeName
    ;

typeArgumentList
    : /* empty */ | typeArgument | typeArgumentList , typeArgument
    ;
```

15.1. Semantics of arguments

An invocation can optionally specify for each argument the corresponding parameter name. It is illegal to use names only for some arguments: either all or no arguments must specify the parameter name. Arguments are evaluated in the order they appear, left to right, before the invocation takes place.

The "don't care" identifier (_) can only be used for an out argument, when the value of returned in that argument is ignored by subsequent computations. When used in generic functions or methods, the compiler may reject the program if it is unable to infer a type for the don't care argument.

15.1.1. Type checking

Argument_ok:

- Typing argument, under context TC at p:
- The result is argumentIR.

After type checking, arguments are represented in $P4_{IR}$ as follows:

```
argumentIR
    : typedExpressionIR | nameIR = typedExpressionIR | nameIR = _ | _
    ;
```

Typing argument, under context TC at p:

1. If let expression of type expression be argument:
    a. Let typedExpressionIR be the result of typing expression, under context TC at p.
    b. The result is typedExpressionIR.
2. Else if let name '=' expression be argument:
    a. Let nameIR be the name of name.
    b. Let typedExpressionIR be the result of typing expression, under context TC at p.
    c. The result is nameIR '=' typedExpressionIR.
3. Else if let name '=' '_' be argument:
    a. Let nameIR be the name of name.
    b. The result is nameIR '=' '_'.
4. Else: the result is '_'.

A list of arguments is type checked with:

ArgumentList_ok:

- Typing argumentList, under context TC at p:
- The result is argumentListIR.

Typing argumentList, under context TC at p:

1. Let argument* be argumentList flattened.
2. For each argument in argument*:
    - Let argumentIR be the result of typing argument, under context TC at p.

    Let argumentIR* be the resulting list.
3. The result is argumentIR*.

15.1.2. Local Compile-time evaluation

Argument_eval_lctk:

-

Local compile-time evaluation of argumentIR, under context TC at p:

- The result is value.

Local compile-time evaluation of argumentIR, under context TC at p:

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:
    a. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
    b. The result is value.
2. Else if let nameIR '=' typedExpressionIR be argumentIR:
    a. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at p.
    b. The result is value.

Local compile-time known arguments are evaluated with the above relation.

15.1.3. Compile-time evaluation

Argument_inst:

- Compile-time evaluation of argumentIR, under context IC and store STO at p:
- The result is store STO and value.

Compile-time evaluation of argumentIR, under context IC and store STO_0 at p:

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:
    a. Let store STO_1 and value be the result of compile-time evaluation of typedExpressionIR, under context IC and store STO_0 at p.
    b. The result is store STO_1 and value.
2. Else if let nameIR '=' typedExpressionIR be argumentIR:
    a. Let store STO_1 and value be the result of compile-time evaluation of typedExpressionIR, under context IC and store STO_0 at p.
    b. The result is store STO_1 and value.

Compile-time known arguments are evaluated with the above relation.

15.1.4. Runtime evaluation

At runtime, an argument may be evaluated to a value, or a storage reference, depending on the direction of the corresponding parameter.

15.1.4.1. `in` and directionless arguments

Arguments for `in` and directionless parameters are evaluated to values at runtime.

Argument_eval: p EC_0 $ARCH_0$ |- argumentIR : % % %

p EC_0 $ARCH_0$ |- argumentIR : % % %:

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:
    a. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$ at p.
    b. The result is EC_1, $ARCH_1$, and expressionResult.
2. Else if let nameIR '=' typedExpressionIR be argumentIR:
    a. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$ at p.
    b. The result is EC_1, $ARCH_1$, and expressionResult.

15.1.4.2. `out` and `inout` arguments

Arguments for `out` and `inout` parameters are evaluated to storage references at runtime.

Argument_eval_lvalue: p EC ARCH |- argumentIR : % % %

p EC ARCH |- argumentIR : % % %:

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:
    a. Let typedLvalueIR$^{!?}$ be the l-value form of typedExpressionIR.
    b. Check that typedLvalueIR$^{!?}$ is defined.
    c. Let typedLvalueIR be typedLvalueIR$^{!?}$.
    d. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
    e. The result is EC_1, $ARCH_1$, and storageReferenceResult.
2. Else if let nameIR '=' typedExpressionIR be argumentIR:
    a. Let typedLvalueIR$^{!?}$ be the l-value form of typedExpressionIR.
    b. Check that typedLvalueIR$^{!?}$ is defined.

c. Let typedLvalueIR be typedLvalueIR'?.

d. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

e. The result is EC_1, $ARCH_1$, and storageReferenceResult.

3. Else if argumentIR matches pattern % '=' '_':

a. The result is EC, ARCH, and $_{\text{CONT}}$ ·.

4. Else:

a. The result is EC, ARCH, and $_{\text{CONT}}$ ·.

15.2. Semantics of type arguments

For generics the type arguments can be explicitly specified.

15.2.1. Type checking

TypeArgument_ok:

- Typing typeArgument, under context TC at p:
- The result is typeArgumentIR and fresh type variables typeId*.

After type checking, arguments are represented in $P4_{IR}$ as follows:

```
typeArgumentIR = typeIR
```

Typing typeArgument, under context TC at p:

1. Try:

a. If let type of type type be typeArgument:

i. Let typeIR and fresh type variables $\text{typeId}_{\text{impl}}^*$ be the result of typing type, under context TC at p.

ii. The result is typeIR and fresh type variables $\text{typeId}_{\text{impl}}^*$.

b. Else if let nonTypeName of type nonTypeName be typeArgument:

i. Let typeId be the name of nonTypeName.

ii. Let bound be bound type variables from the p layer of TC.

iii. If $_{\text{NAME}}$ typeId is a well-formed type, with bound type variables bound:

A. The result is $_{\text{NAME}}$ typeId and fresh type variables ·.

2. Then, try:

a. If typeArgument is equal to VOID:

i. The result is VOID and fresh type variables ·.

b. Else if typeArgument is equal to '_':

   i. Let $\text{typeId}_{\text{impl}}$ be a fresh type variable.

   ii. The result is $_{\text{NAME}}$ $\text{typeId}_{\text{impl}}$ and fresh type variables $\text{typeId}_{\text{impl}}$.

A list of arguments is type checked with:

TypeArgumentList_ok:

- Typing typeArgumentList, under context TC at p:
- The result is typeArgumentListIR and fresh type variables typeId^*.

Typing typeArgumentList, under context TC at p:

1. Let typeArgument^* be typeArgumentList flattened.
2. Let typeArgumentIR^* and fresh type variables $\text{typeId}_{\text{impl}}{}^*$ be the result of typing typeArgument^*, under context TC at p.
3. The result is typeArgumentIR^* and fresh type variables $\text{typeId}_{\text{impl}}{}^*$.

TypeArguments_ok:

- Typing typeArgument^*, under context TC at p:
- The result is typeArgumentListIR and fresh type variables typeId^*.

Typing typeArgument^*, under context TC at p:

1. If typeArgument^* matches pattern []:
   a. The result is · and fresh type variables ·.
2. Else:
   a. Let $\text{typeArgument}_h :: \text{typeArgument}_t{}^*$ be typeArgument^*.
   b. Let typeArgumentIR_h and fresh type variables $\text{typeId}_{\text{impl_h}}{}^*$ be the result of typing typeArgument_h, under context TC at p.
   c. Let $\text{typeArgumentIR}_t{}^*$ and fresh type variables $\text{typeId}_{\text{impl_t}}{}^*$ be the result of typing $\text{typeArgument}_t{}^*$, under context TC at p.
   d. Let $\text{typeId}_{\text{impl}}{}^*$ be $\text{typeId}_{\text{impl_h}}{}^*$ concatenated with $\text{typeId}_{\text{impl_t}}{}^*$.
   e. The result is $\text{typeArgumentIR}_h :: \text{typeArgumentIR}_t{}^*$ and fresh type variables $\text{typeId}_{\text{impl}}{}^*$.

16. Abstract parser machine

A parser declaration introduces a constructor for a parser object. See Section 10.4 and Section 11.11 for details.

```
parserDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { parserLocalDeclarationList parserStateList }
    ;

parserLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | valueSetDeclaration
    ;

parserState
    : annotationList STATE name
        { parserBlockElementStatementList transitionStatement }
    ;
```

This section describes the P4 constructs specific to parsing network packets.

16.1. Parser state machine

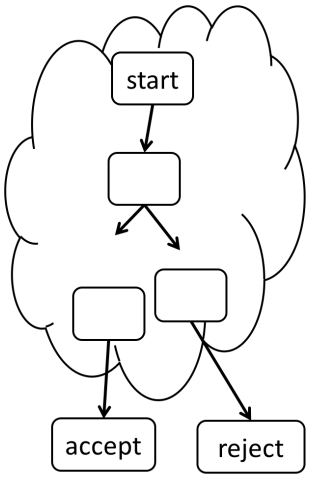

Figure 12. Parser FSM structure.

A P4 parser describes a state machine with one start state and two final states. The start state is always named start. The two final states are named accept (indicating successful parsing) and reject (indicating a parsing failure). The start state is part of the parser, while the accept and reject states are distinct from the states provided by the programmer and are logically outside of the parser. Figure 12 illustrates the general structure of a parser state machine.

The semantics of a P4 parser can be formulated in terms of an abstract machine that manipulates a ParserModel data structure. This section describes this abstract machine in pseudo-code.

A parser starts execution in the start state and ends execution when one of the reject or accept states has been reached.

```
ParserModel {
    error       parseError;
    onPacketArrival(packet p) {
        ParserModel.parseError = error.NoError;
```

```
        goto start;
    }
}
```

An architecture must specify the behavior when the accept and reject states are reached. For example, an architecture may specify that all packets reaching the reject state are dropped without further processing. Alternatively, it may specify that such packets are passed to the next block after the parser, with intrinsic metadata indicating that the parser reached the reject state, along with the error recorded.

16.1.1. Sub-parsers

P4 also allows parsers to invoke the services of other parsers, similar to subroutines. To invoke the services of another parser, the sub-parser must be first instantiated; the services of an instance are invoked by calling it using its apply method.

The following example shows a sub-parser invocation:

```
parser callee(packet_in packet, out IPv4 ipv4) { /* body omitted */ }
parser caller(packet_in packet, out Headers h) {
     callee() subparser;  // instance of callee
     state subroutine {
          subparser.apply(packet, h.ipv4);  // invoke sub-parser
          transition accept;  // accept if sub-parser ends in accept state
     }
}
```

The semantics of a sub-parser invocation can be described as follows:

- The state invoking the sub-parser is split into two half-states at the parser invocation statement.
- The top half includes a transition to the sub-parser start state.
- The sub-parser's accept state is identified with the bottom half of the current state
- The sub-parser's reject state is identified with the reject state of the current parser.

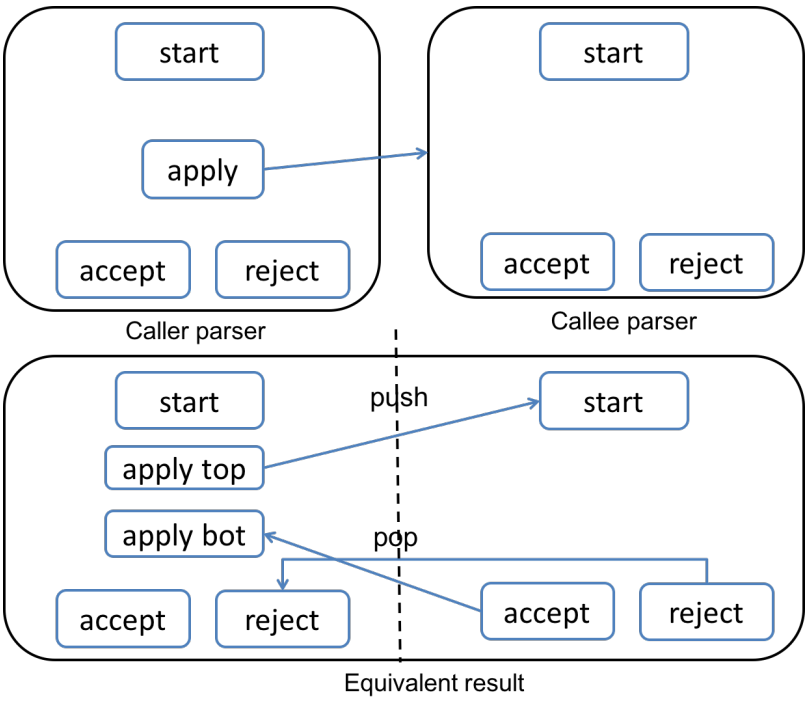

Figure 13. Semantics of invoking a sub-parser: top: original program, bottom: equivalent program.

Figure 13 shows a diagram of this process.

Note that since P4 requires definitions to precede uses, it is impossible to create recursive (or mutually recursive) parsers.

When a parser is instantiated, local instantiations of stateful objects are evaluated recursively. That is, each instantiation of a parser has a unique set of local parser value sets, extern objects, inner parser instances, etc. Thus, in general, invoking a parser instance twice is not the same as invoking two copies of the same parser instance. Note however that local variables do not persist across invocations of the parser. This semantics also applies to direct invocation (see Section 13.5).

Architectures may impose (static or dynamic) constraints on the number of parser states that can be traversed for processing each packet. For example, a compiler for a specific target may reject parsers containing loops that cannot be unrolled at compilation time or that may contain cycles that do not advance the cursor. If a parser aborts execution dynamically because it exceeded the time budget allocated for parsing, the parser should transition to reject and set the standard error error.ParserTimeout.

16.1.2. Runtime evaluation of a parser

At runtime, a parser is evaluated by invoking its apply method. The following algorithm describes the evaluation of a parser apply method invocation:

Calling parserApplyMethodCallee `< typeArgumentListIR `> `( argumentListIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let PARSER _ '.' APPLY `( parameterListIR '#' $id_{default}^{*}$ `) `{ theta ';' frame ';' parserLocalDeclarationListIR ';' stateEnv `} be parserApplyMethodCallee.
2. Let EC_{callee_0} be copy of EC_0 from the GLOBAL layer.
3. Let EC_{callee_1} be EC_{callee_0}[BLOCK.TYPE = theta][BLOCK.FRAME = frame] with BLOCK.STATE set to stateEnv.
4. Let GIVEN $parameterIR_{aligned}^{*}$ DEFAULT $parameterIR_{default}^{*}$ OPTIONAL $_^{*}$ be aligning parameterListIR with argumentListIR where defaults are $id_{default}^{*}$ and optionals are ·.
5. Let p_{shared} be GLOBAL.
6. Let state $ARCH_1$, caller's context EC_1, callee's context EC_{callee_2}, and copy-in result copyInResult be the result of copy-in from argumentListIR at BLOCK of EC_{callee_1} to $parameterIR_{aligned}^{*}$ in p at EC_0, under state $ARCH_0$ at shared p_{shared}.
7. If let rejectTransitionResult of type rejectTransitionResult be copyInResult:
   a. The result is context EC_1, state $ARCH_1$, and call result rejectTransitionResult.
8. Else if let $_{CONT}$ $storageReference^{?*}$ of type continueResult<storageReference?*> be copyInResult:
   a. Let EC_{callee_3} be the result of copying defaults for $parameterIR_{default}^{*}$, under context BLOCK and context EC_{callee_2}.
   b. Let context EC_{callee_4}, state $ARCH_2$, and result parserDeclarationResult be the result of evaluating parser local declarations parserLocalDeclarationListIR, under context EC_{callee_3}, and state $ARCH_1$.
   c. If let rejectTransitionResult of type rejectTransitionResult be parserDeclarationResult:
      i. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_4}, under state $ARCH_2$ at shared p_{shared}.
      ii. The result is context EC_2, state $ARCH_2$, and call result rejectTransitionResult.
   d. Else:
      i. Let context EC_{callee_5}, state $ARCH_3$, and transitionResult be the result of runtime evaluation of parser transition "start", under context EC_{callee_4} and state $ARCH_2$.
      ii. If let rejectTransitionResult of type rejectTransitionResult be transitionResult:
          A.

Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_5}, under state $ARCH_3$ at shared p_{shared}.

B. The result is context EC_2, state $ARCH_3$, and call result rejectTransitionResult.

iii. Else if let acceptTransitionResult of type acceptTransitionResult be transitionResult:

A. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_5}, under state $ARCH_3$ at shared p_{shared}.

B. The result is context EC_2, state $ARCH_3$, and call result RETURN ·.

16.1.3. Parser state transitions

At runtime, parser state transitions are defined as:

ParserState_trans:

- Runtime evaluation of parser transition nameIR, under context EC and state ARCH:
- The result is context EC, state ARCH, and transitionResult.

The result of a transition is:

```
transitionResult
    : acceptTransitionResult | rejectTransitionResult | stateTransitionResult
    ;

acceptTransitionResult
    : ACCEPT
    ;

rejectTransitionResult
    : REJECT errorValue
    ;

stateTransitionResult
    : STATE id
    ;
```

Runtime evaluation of parser transition nameIR, under context EC_0 and state $ARCH_0$:

1. Let $parserStateIR'^{?}$ be the parser state of nameIR in EC_0.
2. Check that $parserStateIR'^{?}$ is defined.
3. Let parserStateIR be $parserStateIR'^{?}$.
4. Let context EC_1, state $ARCH_1$, and transitionResult' be the result of evaluating parser state parserStateIR, under context EC_0, and state $ARCH_0$.
5. If let acceptTransitionResult of type acceptTransitionResult be transitionResult':

a. The result is context EC_1, state $ARCH_1$, and ACCEPT.

6. Else if let rejectTransitionResult of type rejectTransitionResult be transitionResult':
   a. The result is context EC_1, state $ARCH_1$, and rejectTransitionResult.
7. Else:
   a. Let STATE $nameIR_{next}$ be transitionResult'.
   b. Let context EC_2, state $ARCH_2$, and transitionResult be the result of runtime evaluation of parser transition $nameIR_{next}$, under context EC_1 and state $ARCH_1$.
   c. The result is context EC_2, state $ARCH_2$, and transitionResult.

The following sections describe the components of a parser state transition in more detail.

16.2. Parser local declarations

Parser local declarations are defined as follows:

```
parserLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | valueSetDeclaration
    ;

parserLocalDeclarationList
    : /* empty */ | parserLocalDeclarationList parserLocalDeclaration
    ;
```

16.2.1. Semantics of parser local declarations

16.2.1.1. Type checking

ParserLocalDecl_ok:

- Typing parserLocalDeclaration, under context TC:
- The result is context TC and parserLocalDeclarationIR.

ParserLocalDecl_ok dispatch:

1. If let constantDeclaration of type constantDeclaration be parserLocalDeclaration: goto constantDeclaration
2. Else if let instantiation of type instantiation be parserLocalDeclaration: goto instantiation
3. Else if let variableDeclaration of type variableDeclaration be parserLocalDeclaration: goto variableDeclaration
4. Else:
   a. Let annotationList VALUE_SET `< valueSetType `> `( expression `) name ';' be parserLocalDeclaration.
   b. Goto valueSetDeclaration

After type checking, parser local declarations are represented in $P4_{IR}$ as follows:

```
parserLocalDeclarationIR
    : constantDeclarationIR | instantiationIR | variableDeclarationIR
    | valueSetDeclarationIR
    ;
```

A list of parser local declarations is type checked by:

ParserLocalDeclList_ok:

- Typing parserLocalDeclarationList, under context TC:
- The result is context TC and parserLocalDeclarationIR*.

Typing parserLocalDeclarationList, under context TC_0:

1. Let parserLocalDeclaration* be parserLocalDeclarationList flattened.
2. Let the updated context TC_1 and the typed parser local declarations parserLocalDeclarationIR* be the result of typing parserLocalDeclaration*, under context TC_0.
3. The result is context TC_1 and parserLocalDeclarationIR*.

ParserLocalDecls_ok:

- Typing parserLocalDeclaration*, under context TC:
- The result is the updated context TC and the typed parser local declarations parserLocalDeclarationIR*.

Typing parserLocalDeclaration*, under context TC:

1. If parserLocalDeclaration* matches pattern []:
   a. The result is the updated context TC and the typed parser local declarations ·.
2. Else:
   a. Let $parserLocalDeclaration_h :: parserLocalDeclaration_t^*$ be parserLocalDeclaration*.
   b. Let context TC_1 and $parserLocalDeclarationIR_h$ be the result of typing $parserLocalDeclaration_h$, under context TC.
   c. Let the updated context TC_2 and the typed parser local declarations $parserLocalDeclarationIR_t^*$ be the result of typing $parserLocalDeclaration_t^*$, under context TC_1.
   d. The result is the updated context TC_2 and the typed parser local declarations $parserLocalDeclarationIR_h :: parserLocalDeclarationIR_t^*$.

16.2.1.2. Compile-time evaluation

ParserLocalDecl_inst:

- Compile-time evaluation of parserLocalDeclarationIR, under context IC and store STO:
- The result is context IC, store STO, and parserLocalDeclarationIR.

ParserLocalDecl_inst dispatch:

1. If let constantDeclarationIR of type constantDeclarationIR be parserLocalDeclarationIR: goto constantDeclarationIR
2. Else if let instantiationIR of type instantiationIR be parserLocalDeclarationIR: goto instantiationIR
3. Else if let variableDeclarationIR of type variableDeclarationIR be parserLocalDeclarationIR: goto variableDeclarationIR
4. Else:
    a. Let annotationList VALUE_SET `< typeIR `> `( typedExpressionIR$_{size}$ `) nameIR ';' be parserLocalDeclarationIR.
    b. Goto valueSetDeclarationIR

A list of parser local declarations is compile-time evaluated by:

ParserLocalDecls_inst:

- Compile-time evaluation of parserLocalDeclarationIR*, under context IC and store STO:
- The result is context IC, store STO, and parserLocalDeclarationIR*.

Compile-time evaluation of parserLocalDeclarationIR*, under context IC and store STO:

1. If parserLocalDeclarationIR* matches pattern []:
    a. The result is context IC, store STO, and ·.
2. Else:
    a. Let parserLocalDeclarationIR$_h$ :: parserLocalDeclarationIR$_t^*$ be parserLocalDeclarationIR*.
    b. Let context IC$_1$, store STO$_1$, and parserLocalDeclarationIR$_{h_inst}$ be the result of compile-time evaluation of parserLocalDeclarationIR$_h$, under context IC and store STO.
    c. Let context IC$_2$, store STO$_2$, and parserLocalDeclarationIR$_{t_inst}^*$ be the result of compile-time evaluation of parserLocalDeclarationIR$_t^*$, under context IC$_1$ and store STO$_1$.
    d. The result is context IC$_2$, store STO$_2$, and parserLocalDeclarationIR$_{h_inst}$ :: parserLocalDeclarationIR$_{t_inst}^*$.

16.2.1.3. Runtime evaluation

ParserLocalDecl_eval:

- Evaluating parser local declaration parserLocalDeclarationIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and result parserDeclarationResult.

ParserLocalDecl_eval dispatch:

1. If let constantDeclarationIR of type constantDeclarationIR be parserLocalDeclarationIR: goto constantDeclarationIR
2. Else if let variableDeclarationIR of type variableDeclarationIR be parserLocalDeclarationIR: goto variableDeclarationIR

Evaluation of a parser local declaration yields:

```
parserDeclarationResult = parserStatementResult

parserStatementResult
    : continueEmptyResult | rejectTransitionResult
    ;

continueEmptyResult
    : /* empty */
    ;

rejectTransitionResult
    : REJECT errorValue
    ;
```

A list of parser local declarations is runtime evaluated by:

ParserLocalDecls_eval:

- Evaluating parser local declarations parserLocalDeclarationIR*, under context EC, and state ARCH:
- The result is context EC, state ARCH, and result parserDeclarationResult.

Evaluating parser local declarations parserLocalDeclarationIR*, under context EC, and state ARCH:

1. If parserLocalDeclarationIR* matches pattern []:
    a. The result is context EC, state ARCH, and result $_{\text{EMPTY}}$.

2. Else:
    a. Let parserLocalDeclarationIR$_h$ :: parserLocalDeclarationListIR$_t$ be parserLocalDeclarationIR*.
    b. Let context EC$_1$, state ARCH$_1$, and result parserDeclarationResult be the result of evaluating parser local declaration parserLocalDeclarationIR$_h$, under context EC, and state ARCH.
    c. If let rejectTransitionResult of type rejectTransitionResult be parserDeclarationResult:
        i. The result is context EC$_1$, state ARCH$_1$, and result rejectTransitionResult.
    d. Else:
        i. Let context EC$_2$, state ARCH$_2$, and result parserStatementResult be the result of evaluating parser local declarations parserLocalDeclarationListIR$_t$, under context EC$_1$, and state ARCH$_1$.
        ii. The result is context EC$_2$, state ARCH$_2$, and result parserStatementResult.

The subsequent sections describe each kind of declaration in detail.

16.2.2. Constant declarations

See Section 11.3 for general information about constant declarations.

16.2.2.1. Type checking

Typing constantDeclaration, under context TC$_0$:

1. Let context TC$_1$ and constantDeclarationIR be the result of typing constantDeclaration, under context TC$_0$ at BLOCK.
2. The result is context TC$_1$ and constantDeclarationIR.

16.2.2.2. Compile-time evaluation

Compile-time evaluation of constantDeclarationIR, under context IC$_0$ and store STO:

1. Let context IC$_1$ be the result of loading constantDeclarationIR, under context IC$_0$ at BLOCK.
2. The result is context IC$_1$, store STO, and constantDeclarationIR.

16.2.2.3. Runtime evaluation

Evaluating parser local declaration constantDeclarationIR, under context EC$_0$, and state ARCH:

1. Let context EC$_1$ be the result of evaluating constantDeclarationIR, under context EC$_0$ at BLOCK.
2. The result is context EC$_1$, state ARCH, and result $_{\text{EMPTY}}$.

16.2.3. Instantiations

See Section 11.4 for general information about instantiations.

16.2.3.1. Type checking

Typing instantiation, under context TC_0:

1. Let context TC_1 and instantiationIR be the result of typing instantiation, under context TC_0 at BLOCK.
2. The result is context TC_1 and instantiationIR.

16.2.3.2. Compile-time evaluation

Compile-time evaluation of instantiationIR, under context IC_0 and store STO:

1. Let context IC_1, store STO_1, and reference to object $constantDeclarationIR_{inst}$ be the result of loading instantiationIR, under context IC_0 and store STO at BLOCK.
2. The result is context IC_1, store STO_1, and $constantDeclarationIR_{inst}$.

16.2.4. Variable declarations

See Section 11.2 for general information about variable declarations.

16.2.4.1. Type checking

Typing variableDeclaration, under context TC_0:

1. Let context TC_1 and variableDeclarationIR be the result of typing variableDeclaration, under context TC_0 at BLOCK.
2. The result is context TC_1 and variableDeclarationIR.

16.2.4.2. Compile-time evaluation

Compile-time evaluation of variableDeclarationIR, under context IC_0 and store STO:

1. The result is context IC_0, store STO, and variableDeclarationIR.

16.2.4.3. Runtime evaluation

Evaluating parser local declaration variableDeclarationIR, under context EC_0, and state ARCH:

1. Let context EC_1, state $ARCH_1$, and result declarationResult be the result of evaluating variableDeclarationIR, under context EC_0 and state ARCH at BLOCK.
2. If let rejectTransitionResult of type rejectTransitionResult be declarationResult:
   a. The result is context EC_1, state $ARCH_1$, and result rejectTransitionResult.
3. Else if let continueEmptyResult of type continueEmptyResult be declarationResult:
   a. The result is context EC_1, state $ARCH_1$, and result $_{\text{EMPTY}}$.

16.2.5. Parser value set declarations

Parser value sets are declared with the syntax:

```
valueSetDeclaration
    : annotationList VALUE_SET < valueSetType > ( expression ) name ;
    ;

valueSetType
    : baseType | tupleType | prefixedTypeName
    ;
```

See Section 10.7 for general information about value sets.

16.2.5.1. Type checking

Typing annotationList VALUE_SET `< valueSetType `> `( expression `) name ';', under context TC_0:

1. Let typeIR and fresh type variables typeId* be the result of typing valueSetType, under context TC_0 at BLOCK.
2. Check that typeId* is an empty list.
3. Let bound be bound type variables from the BLOCK layer of TC_0.
4. If SET `< typeIR `> is a well-formed type, with bound type variables bound:
   a. Let typedExpressionIR be the result of typing expression, under context TC_0 at BLOCK.
   b. Check that the compile-time known-ness of typedExpressionIR does not match pattern DYN.
   c. Let nameIR be the name of name.
   d. Let varTypeIR be $_{\text{EMPTY}}$ SET `< typeIR `> CTK ·.
   e. Let TC_1 be TC_0 where nameIR to varTypeIR is added to the BLOCK layer.
   f. Let valueSetDeclarationIR be annotationList VALUE_SET `< typeIR `> `( typedExpressionIR `) nameIR ';'.
   g. The result is context TC_1 and valueSetDeclarationIR.

16.2.5.2. Compile-time evaluation

Compile-time evaluation of annotationList VALUE_SET `< typeIR `> `( typedExpressionIR$_{\text{size}}$ `) nameIR ';', under context IC_0 and store STO:

1. Let store STO_1 and value be the result of compile-time evaluation of $typedExpressionIR_{size}$, under context IC_0 and store STO at BLOCK.
2. Check that value has type integerValue.
3. Let $integerValue_{size}$ be value.
4. Let $nat^?$ be the natural number representation of $integerValue_{size}$.
5. Check that $nat^?$ is defined.
6. Let n_{size} be $nat^?$.
7. Let $typeIR_{subst}$ be typeIR substituted by bound type variables in IC_0 from the BLOCK layer.
8. Let valueSetObject be VALUE_SET `{ · `(n_{size} `) `}.
9. Let objectId be IC_0.PATH concatenated with nameIR.
10. Let STO_2 be STO_1 where objectId to valueSetObject is added.
11. Let $typeIR_{set}$ be SET `< typeIR `>.
12. Let constantDeclarationIR be $_{EMPTY}$ CONST $typeIR_{set}$ nameIR '=' $_{VALUE}$ REF objectId ';'.
13. The result is context IC_0, store STO_2, and constantDeclarationIR.

After compile-time evaluation, parser value set declarations are represented in $P4_{IR}$ as follows:

```
valueSetDeclarationIR
    : annotationList VALUE_SET < typeIR > ( typedExpressionIR ) nameIR ;
    ;
```

16.3. Parser states

A parser state is declared with the following syntax:

```
parserState
    : annotationList STATE name
        { parserBlockElementStatementList transitionStatement }
    ;
```

Each state has a name and a body. The body consists of a sequence of statements that describe the processing performed when the parser transitions to that state, including:

- Local variable declarations,
- Assignment statements,
- Method calls, which serve several purposes:
  - Invoking functions (e.g., using verify to check the validity of data already parsed), and
  - Invoking methods (e.g., extracting data out of packets or computing checksums) and other parsers (see Section 16.1.1), and
- Conditional statements,
- Transitions to other states (discussed in Section 16.5).

Architectures may place restrictions on the expressions and statements that can be used in a parser—e.g., they may forbid the use of operations such as multiplication or place restrictions on the number of local variables that may be used.

16.3.1. Type checking

A list of parser states is type checked with the following relation:

ParserStateList_ok:

- Typing parserStateList, under context TC:
- The result is parserStateIR*.

Typing parserStateList, under context TC:

1. Let parserState* be parserStateList flattened.
2. For each parserState in parserState*:
   - Let _ STATE name_state `{ _ _ `} be parserState.

   Let name_state* be the resulting list.
3. For each name_state in name_state*:
   - Let nameIR_state be the name of name_state.

   Let nameIR_state* be the resulting list.
4. Check that the elements of nameIR_state* are distinct.
5. Check that "start" is in nameIR_state*.
6. Check that "accept" is not in nameIR_state* and "reject" is not in nameIR_state*.
7. Let nameIR_state_impl* be "accept" :: "reject" :: nameIR_state*.
8. For each parserState in parserState*:
   - Let parserStateIR be the result of typing parserState, under context TC and parser state names nameIR_state_impl*.

   Let parserStateIR* be the resulting list.
9. The result is parserStateIR*.

A parser state is type checked with the following relation:

ParserState_ok:

- Typing parserState, under context TC and parser state names nameIR*:
- The result is parserStateIR.

Typing parserState, under context TC_0 and parser state names $nameIR_{state}^*$:

1. Let annotationList STATE name `{ parserBlockElementStatementList transitionStatement `} be parserState.
2. Let nameIR be the name of name.
3. Let TC_1 be TC_0 with a new local frame.
4. Let context TC_2 and $parserBlockElementStatementIR^*$ be the result of typing parserBlockElementStatementList under context TC_1.
5. Let typed transition statement transitionStatementIR be the result of typing transitionStatement, under context TC_2 and parser state names $nameIR_{state}^*$.
6. Let TC_3 be TC_2 with the innermost frame removed.
7. Let parserStateIR be annotationList STATE nameIR `{ $parserBlockElementStatementIR^*$ transitionStatementIR `}.
8. The result is parserStateIR.

16.3.2. Compile-time evaluation

A list parser states is compile-time evaluated with the following relation:

ParserStates_inst:

- Compile-time evaluation of $parserStateIR^*$, under context IC and store STO:
- The result is context IC and store STO.

Compile-time evaluation of $parserStateIR^*$, under context IC and store STO:

1. If $parserStateIR^*$ matches pattern []:
   a. The result is context IC and store STO.
2. Else:
   a. Let $parserStateIR_h :: parserStateIR_t^*$ be $parserStateIR^*$.
   b. Let context IC_1 and store STO_1 be the result of compile-time evaluation of $parserStateIR_h$, under context IC and store STO.
   c. Let context IC_2 and store STO_2 be the result of compile-time evaluation of $parserStateIR_t^*$, under context IC_1 and store STO_1.
   d. The result is context IC_2 and store STO_2.

A parser state is compile-time evaluated with the following relation:

ParserState_inst:

- Compile-time evaluation of parserStateIR, under context IC and store STO:
- The result is context IC and store STO.

Compile-time evaluation of parserStateIR, under context IC_0 and store STO_0:

1. Let annotationList STATE nameIR `{ parserBlockElementStatementListIR transitionStatementIR `} be parserStateIR.
2. Let IC_1 be IC_0 with a new local frame.
3. Let context IC_2, store STO_1, and $parserBlockElementStatementListIR_{inst}$ be the result of compile-time evaluation of parserBlockElementStatementListIR, under context IC_1, and store STO_0.
4. Let IC_3 be IC_2 with the innermost frame removed.
5. Let $parserStateIR_{inst}$ be annotationList STATE nameIR `{ $parserBlockElementStatementListIR_{inst}$ transitionStatementIR `}.
6. Let IC_4 be IC_3 where nameIR to $parserStateIR_{inst}$ is added.
7. The result is context IC_4 and store STO_1.

16.3.3. Runtime evaluation

A parser state is runtime evaluated with the following relation:

ParserState_eval:

- Evaluating parser state parserStateIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and transitionResult.

The result of evaluation is a state transition,

```
transitionResult
    : acceptTransitionResult | rejectTransitionResult | stateTransitionResult
    ;
```

Evaluating parser state parserStateIR, under context EC_0, and state $ARCH_0$:

1. Let annotationList STATE nameIR `{ parserBlockElementStatementListIR transitionStatementIR `} be parserStateIR.
2. Let EC_1 be EC_0 with a new local frame.
3. Let context EC_2, state $ARCH_1$, and parserStatementResult be the result of runtime evaluation of parserBlockElementStatementListIR, under context EC_1, and state $ARCH_0$.
4. If let rejectTransitionResult of type rejectTransitionResult be parserStatementResult:

a. Let EC_3 be EC_2 with the innermost frame removed.

b. The result is context EC_3, state $ARCH_1$, and rejectTransitionResult.

5. Else:

a. Let continueEmptyResult be parserStatementResult.

b. Let context EC_3, state $ARCH_2$, and transitionResult be the result of runtime evaluation of transitionStatementIR, under context EC_2 and state $ARCH_1$.

c. Let EC_4 be EC_3 with the innermost frame removed.

d. The result is context EC_4, state $ARCH_2$, and transitionResult.

16.4. Parser statements

The syntax for parser statements is given by the following grammar rules:

```
<span id="parserStatement"></span>
<span id="parserBlockElementStatement"></span>
<span id="parserBlockElementStatementList"></span>
<span id="parserBlockStatement"></span>
<span id="parserConditionalStatement"></span>
```

```
parserStatement
    : emptyStatement | assignmentStatement | callStatement
    | directApplicationStatement | parserBlockStatement
    | parserConditionalStatement
    ;

parserBlockElementStatement
    : constantDeclaration | variableDeclaration | parserStatement
    ;

parserBlockElementStatementList
    : /* empty */ | parserBlockElementStatementList parserBlockElementStatement
    ;

parserBlockStatement
    : annotationList { parserBlockElementStatementList }
    ;

parserConditionalStatement
    : IF ( expression ) parserStatement
    | IF ( expression ) parserStatement ELSE parserStatement
    ;
```

16.4.1. Semantics of parser statements

16.4.1.1. Type checking

```
<span id="relation_latex_ParserStmt_ok"></span>

<span id="relation_prose_ParserStmt_ok"></span>
```

ParserStmt_ok:

- Typing parserStatement, under context TC:
- The result is context TC and typed parser statement parserStatementIR.

ParserStmt_ok dispatch:

1. If let emptyStatement of type emptyStatement be parserStatement: goto emptyStatement
2. Else if let assignmentStatement of type assignmentStatement be parserStatement: goto assignmentStatement
3. Else if let callStatement of type callStatement be parserStatement: goto callStatement
4. Else if let directApplicationStatement of type directApplicationStatement be parserStatement: goto directApplicationStatement
5. Else if let parserBlockStatement of type parserBlockStatement be parserStatement: goto parserBlockStatement
6. Else:
    a. Let parserConditionalStatement be parserStatement.
    b. Goto parserConditionalStatement

After type checking, parser statements are represented in $P4_{IR}$ as follows:

```
<span id="parserStatementIR"></span>
<span id="parserBlockElementStatementIR"></span>
<span id="parserBlockElementStatementListIR"></span>
<span id="parserBlockStatementIR"></span>
<span id="parserConditionalStatementIR"></span>
```

```
parserStatementIR
    : emptyStatementIR | assignmentStatementIR | callStatementIR
    | directApplicationStatementIR | parserBlockStatementIR
    | parserConditionalStatementIR
    ;

parserBlockElementStatementIR
    : constantDeclarationIR | variableDeclarationIR | parserStatementIR
    ;

parserBlockElementStatementListIR = parserBlockElementStatementIR*

parserBlockStatementIR
    : annotationList { parserBlockElementStatementListIR }
    ;

parserConditionalStatementIR
    : IF ( typedExpressionIR ) parserStatementIR
    | IF ( typedExpressionIR ) parserStatementIR ELSE parserStatementIR
    ;
```

16.4.1.2. Compile-time evaluation

```
<span id="relation_latex_ParserStmt_inst"></span>

<span id="relation_prose_ParserStmt_inst"></span>
```

ParserStmt_inst:

- Compile-time evaluation of parserStatementIR, under context IC and store STO:
- The result is context IC and store STO.

ParserStmt_inst dispatch:

1.

If let emptyStatementIR of type emptyStatementIR be parserStatementIR: goto emptyStatementIR

2. Else if let assignmentStatementIR of type assignmentStatementIR be parserStatementIR: goto assignmentStatementIR
3. Else if let callStatementIR of type callStatementIR be parserStatementIR: goto callStatementIR
4. Else if let directApplicationStatementIR of type directApplicationStatementIR be parserStatementIR: goto directApplicationStatementIR
5. Else if let parserBlockStatementIR of type parserBlockStatementIR be parserStatementIR: goto parserBlockStatementIR
6. Else:
    a. Let parserConditionalStatementIR be parserStatementIR.
    b. Goto parserConditionalStatementIR

16.4.1.3. Runtime evaluation

ParserStmt_eval:

- Runtime evaluation of parserStatementIR, under context EC and state ARCH:
- The result is context EC, state ARCH, and parserStatementResult.

ParserStmt_eval dispatch:

1. If let emptyStatementIR of type emptyStatementIR be parserStatementIR: goto emptyStatementIR
2. Else if let assignmentStatementIR of type assignmentStatementIR be parserStatementIR: goto assignmentStatementIR
3. Else if let callStatementIR of type callStatementIR be parserStatementIR: goto callStatementIR
4. Else if let parserBlockStatementIR of type parserBlockStatementIR be parserStatementIR: goto parserBlockStatementIR
5. Else if let parserConditionalStatementIR of type parserConditionalStatementIR be parserStatementIR: goto parserConditionalStatementIR

Evaluation of a parser statement yields:

```
parserStatementResult
    : continueEmptyResult | rejectTransitionResult
    ;

continueEmptyResult
    : /* empty */
    ;

rejectTransitionResult
    : REJECT errorValue
    ;
```

The subsequent sections describe each kind of statement in detail.

16.4.2. Empty statement

See Section 13.2 for general information about empty statements.

16.4.2.1. Type checking

Typing emptyStatement, under context TC:

1. Let context TC' and statementIR be the result of typing emptyStatement, under context TC at LOCAL.
2. Check that TC' is equal to TC.
3. Check that statementIR has type emptyStatementIR.
4. Let emptyStatementIR be statementIR.
5. The result is context TC and typed parser statement emptyStatementIR.

16.4.2.2. Compile-time evaluation

Compile-time evaluation of emptyStatementIR, under context IC_0 and store STO_0:

1. Let context IC_1, store STO_1, and statementIR be the result of compile-time evaluation of emptyStatementIR, under context IC_0 and store STO_0 at LOCAL.
2. Check that statementIR has type emptyStatementIR.
3. Let $emptyStatementIR_{inst}$ be statementIR.
4. The result is context IC_1 and store STO_1.

16.4.2.3. Runtime evaluation

Runtime evaluation of emptyStatementIR, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of emptyStatementIR, under context EC_0 and state $ARCH_0$ at LOCAL.
2. Check that statementResult has type continueEmptyResult.
3. Let continueEmptyResult be statementResult.
4. The result is context EC_1, state $ARCH_1$, and $_{EMPTY}$.

16.4.3. Assignment statement

See Section 13.3 for general information about assignment statements.

16.4.3.1. Type checking

Typing assignmentStatement, under context TC:

1. Let context TC_1 and statementIR be the result of typing assignmentStatement, under context TC at LOCAL.
2. Check that statementIR has type assignmentStatementIR.
3. Let assignmentStatementIR be statementIR.
4. The result is context TC_1 and typed parser statement assignmentStatementIR.

16.4.3.2. Compile-time evaluation

Compile-time evaluation of assignmentStatementIR, under context IC_0 and store STO_0:

1. Let context IC_1, store STO_1, and statementIR be the result of compile-time evaluation of assignmentStatementIR, under context IC_0 and store STO_0 at LOCAL.
2. Check that statementIR has type assignmentStatementIR.
3. Let $assignmentStatementIR_{inst}$ be statementIR.
4. The result is context IC_1 and store STO_1.

16.4.3.3. Runtime evaluation

Runtime evaluation of assignmentStatementIR, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of assignmentStatementIR, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let rejectTransitionResult of type rejectTransitionResult be statementResult:
   a. The result is context EC_1, state $ARCH_1$, and rejectTransitionResult.
3. Else if let continueEmptyResult of type continueEmptyResult be statementResult:
   a. The result is context EC_1, state $ARCH_1$, and $_{EMPTY}$.

16.4.4. Call statement

See Section 13.4 for general information about call statements.

16.4.4.1. Type checking

Typing callStatement, under context TC:

1. Let context TC_1 and statementIR be the result of typing callStatement, under context TC at LOCAL.
2. Check that statementIR has type callStatementIR.
3. Let callStatementIR be statementIR.
4. The result is context TC_1 and typed parser statement callStatementIR.

16.4.4.2. Compile-time evaluation

Compile-time evaluation of callStatementIR, under context IC_0 and store STO_0:

1. Let context IC_1, store STO_1, and statementIR be the result of compile-time evaluation of callStatementIR, under context IC_0 and store STO_0 at LOCAL.
2. Check that statementIR has type callStatementIR.
3. Let $callStatementIR_{inst}$ be statementIR.
4. The result is context IC_1 and store STO_1.

16.4.4.3. Runtime evaluation

Runtime evaluation of callStatementIR, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of callStatementIR, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let rejectTransitionResult of type rejectTransitionResult be statementResult:
   a. The result is context EC_1, state $ARCH_1$, and rejectTransitionResult.
3. Else if let continueEmptyResult of type continueEmptyResult be statementResult:
   a. The result is context EC_1, state $ARCH_1$, and $_{EMPTY}$.

16.4.5. Direct type invocation

See Section 13.5 for general information about direct type invocations.

16.4.5.1. Type checking

Typing directApplicationStatement, under context TC:

1. Let context TC_1 and statementIR be the result of typing directApplicationStatement, under context TC at LOCAL.
2. Check that statementIR has type directApplicationStatementIR.
3. Let directApplicationStatementIR be statementIR.
4. The result is context TC_1 and typed parser statement directApplicationStatementIR.

16.4.5.2. Compile-time evaluation

Compile-time evaluation of directApplicationStatementIR, under context IC_0 and store STO_0:

1. Let store STO_1 and desugared constantDeclarationIR, callStatementIR be the result of compile-time evaluation of directApplicationStatementIR, under context IC_0 and store STO_0 at LOCAL.

2. Let parserBlockStatementIR be $_{\text{EMPTY}}$ `{ [ constantDeclarationIR, callStatementIR ] `}.
3. The result is context IC_0 and store STO_1.

16.4.6. Block statement

A parser block statement is a sequence of parser statements or declarations enclosed in braces. It is similar to a block statement explained in Section 13.8, but it can only contain parser statements, constant declarations, and variable declarations.

```
parserBlockStatement
    : annotationList { parserBlockElementStatementList }
    ;

parserBlockElementStatementList
    : /* empty */ | parserBlockElementStatementList parserBlockElementStatement
    ;

parserBlockElementStatement
    : constantDeclaration | variableDeclaration | parserStatement
    ;
```

16.4.6.1. Type checking

After type checking, a parser block statement is represented in $P4_{IR}$ as:

```
parserBlockStatement
    : annotationList { parserBlockElementStatementList }
    ;

parserBlockElementStatementListIR = parserBlockElementStatementIR*

parserBlockElementStatementIR
    : constantDeclarationIR | variableDeclarationIR | parserStatementIR
    ;
```

Typing parserBlockStatement, under context TC:

1. Let TC_1 be TC with a new local frame.
2. Let context TC_2 and parserBlockStatementIR be the result of typing parserBlockStatement under context TC_1.
3. Let TC_3 be TC_2 with the innermost frame removed.
4. The result is context TC_3 and typed parser statement parserBlockStatementIR.

A parser block is type checked by the relation:

ParserBlock_ok:

- Typing parserBlockStatement under context TC:

- The result is context TC and parserBlockStatementIR.

Typing annotationList `{ parserBlockElementStatementList `} under context TC_0:

1. Let context TC_1 and parserBlockElementStatementIR* be the result of typing parserBlockElementStatementList under context TC_0.
2. Let parserBlockStatementIR be annotationList `{ parserBlockElementStatementIR* `}.
3. The result is context TC_1 and parserBlockStatementIR.

A statement or declaration within a parser block statement is type checked by the relation:

ParserBlockElementStmt_ok:

- Typing parserBlockElementStatement under context TC:
- The result is context TC and parserBlockElementStatementIR.

Typing parserBlockElementStatement under context TC_0:

1. If let constantDeclaration of type constantDeclaration be parserBlockElementStatement:
   a. Let context TC_1 and constantDeclarationIR be the result of typing constantDeclaration, under context TC_0 at LOCAL.
   b. The result is context TC_1 and constantDeclarationIR.
2. Else if let variableDeclaration of type variableDeclaration be parserBlockElementStatement:
   a. Let context TC_1 and variableDeclarationIR be the result of typing variableDeclaration, under context TC_0 at LOCAL.
   b. The result is context TC_1 and variableDeclarationIR.
3. Else:
   a. Let parserStatement be parserBlockElementStatement.
   b. Let context TC_1 and typed parser statement parserStatementIR be the result of typing parserStatement, under context TC_0.
   c. The result is context TC_1 and parserStatementIR.

A list of block elements is type checked by the relation:

ParserBlockElementStmtList_ok:

-

Typing parserBlockElementStatementList under context TC:

- The result is context TC and parserBlockElementStatementIR*.

Typing parserBlockElementStatementList under context TC_0:

1. Let parserBlockElementStatement* be parserBlockElementStatementList flattened.
2. Let context TC_1 and parserBlockElementStatementIR* be the result of typing parserBlockElementStatement* under context TC_0.
3. The result is context TC_1 and parserBlockElementStatementIR*.

ParserBlockElementStmts_ok:

- Typing parserBlockElementStatement* under context TC:
- The result is context TC and parserBlockElementStatementIR*.

Typing parserBlockElementStatement* under context TC:

1. If parserBlockElementStatement* matches pattern []:
   a. The result is context TC and ·.
2. Else:
   a. Let $\text{parserBlockElementStatement}_h :: \text{parserBlockElementStatement}_t^*$ be parserBlockElementStatement*.
   b. Let context TC_1 and $\text{parserBlockElementStatementIR}_h$ be the result of typing $\text{parserBlockElementStatement}_h$ under context TC.
   c. Let context TC_2 and $\text{parserBlockElementStatementIR}_t^*$ be the result of typing $\text{parserBlockElementStatement}_t^*$ under context TC_1.
   d. The result is context TC_2 and $\text{parserBlockElementStatementIR}_h :: \text{parserBlockElementStatementIR}_t^*$.

16.4.6.2. Compile-time evaluation

Compile-time evaluation of parserBlockStatementIR, under context IC_0 and store STO_0:

1. Let IC_1 be IC_0 with a new local frame.
2. Let context IC_2, store STO_1, and $\text{parserBlockStatementIR}_{inst}$ be the result of compile-time evaluation of parserBlockStatementIR, under context IC_1, and store STO_0.
3. Let IC_3 be IC_2 with the innermost frame removed.
4. The result is context IC_3 and store STO_1.

A parser block is compile-time evaluated by the relation:

ParserBlock_inst:

- Compile-time evaluation of parserBlockStatementIR, under context IC, and store STO:
- The result is context IC, store STO, and parserBlockStatementIR.

Compile-time evaluation of annotationList `{ parserBlockElementStatementListIR `}, under context IC_0, and store STO_0:

1. Let context IC_1, store STO_1, and $parserBlockElementStatementListIR_{inst}$ be the result of compile-time evaluation of parserBlockElementStatementListIR, under context IC_0, and store STO_0.
2. The result is context IC_1, store STO_1, and annotationList `{ $parserBlockElementStatementListIR_{inst}$ `}.

A parser block element is compile-time evaluated by the relation:

ParserBlockElementStmt_inst:

- Compile-time evaluation of parserBlockElementStatementIR, under context IC, and store STO:
- The result is context IC, store STO, and parserBlockElementStatementIR.

Compile-time evaluation of parserBlockElementStatementIR, under context IC, and store STO:

1. If let constantDeclarationIR of type constantDeclarationIR be parserBlockElementStatementIR:
   a. Let context IC_1 be the result of loading constantDeclarationIR, under context IC at LOCAL.
   b. The result is context IC_1, store STO, and constantDeclarationIR.
2. Else if let variableDeclarationIR of type variableDeclarationIR be parserBlockElementStatementIR:
   a. The result is context IC, store STO, and variableDeclarationIR.
3. Else:
   a. Let parserStatementIR be parserBlockElementStatementIR.
   b. Let context IC_1 and store STO_1 be the result of compile-time evaluation of parserStatementIR, under context IC and store STO.
   c. The result is context IC_1, store STO_1, and $parserStatementIR_{inst}$.

A list of parser block elements is compile-time evaluated by the relation:

ParserBlockElementStmtList_inst:

- Compile-time evaluation of parserBlockElementStatementListIR, under context IC, and store STO:
- The result is context IC, store STO, and $\text{parserBlockElementStatementIR}^*$.

Compile-time evaluation of $\text{parserBlockElementStatementIR}^*$, under context IC, and store STO:

1. If $\text{parserBlockElementStatementIR}^*$ matches pattern []:
   a. The result is context IC, store STO, and ·.
2. Else:
   a. Let $\text{parserBlockElementStatementIR}_h$ :: $\text{parserBlockElementStatementIR}_t^*$ be $\text{parserBlockElementStatementIR}^*$.
   b. Let context IC_1, store STO_1, and $\text{parserBlockElementStatementIR}_{h_inst}$ be the result of compile-time evaluation of $\text{parserBlockElementStatementIR}_h$, under context IC, and store STO.
   c. Let context IC_2, store STO_2, and $\text{parserBlockElementStatementIR}_{t_inst}^*$ be the result of compile-time evaluation of $\text{parserBlockElementStatementIR}_t^*$, under context IC_1, and store STO_1.
   d. The result is context IC_2, store STO_2, and $\text{parserBlockElementStatementIR}_{h_inst}$ :: $\text{parserBlockElementStatementIR}_{t_inst}^*$.

16.4.6.3. Runtime evaluation

Runtime evaluation of parserBlockStatementIR, under context EC_0 and state ARCH_0:

1. Let EC_1 be EC_0 with a new local frame.
2. Let context EC_2, state ARCH_1, and parserStatementResult be the result of runtime evaluation of parserBlockStatementIR, under context EC_1, and state ARCH_0.
3. Let EC_3 be EC_2 with the innermost frame removed.
4. The result is context EC_3, state ARCH_1, and parserStatementResult.

A parser block is runtime evaluated by the relation:

ParserBlock_eval:

- Runtime evaluation of parserBlockStatementIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and parserStatementResult.

Runtime evaluation of annotationList `{ parserBlockElementStatementListIR `}, under context EC_0, and state ARCH_0:

1. Let context EC_1, state $ARCH_1$, and parserStatementResult be the result of runtime evaluation of parserBlockElementStatementListIR, under context EC_0, and state $ARCH_0$.
2. The result is context EC_1, state $ARCH_1$, and parserStatementResult.

A parser block element is runtime evaluated by the relation:

ParserBlockElementStmt_eval:

- Runtime evaluation of parserBlockElementStatementIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and parserStatementResult.

Runtime evaluation of parserBlockElementStatementIR, under context EC_0, and state ARCH:

1. If let constantDeclarationIR of type constantDeclarationIR be parserBlockElementStatementIR:
    a. Let context EC_1 be the result of evaluating constantDeclarationIR, under context EC_0 at LOCAL.
    b. The result is context EC_1, state ARCH, and $_{\text{EMPTY}}$.
2. Else if let variableDeclarationIR of type variableDeclarationIR be parserBlockElementStatementIR:
    a. Let context EC_1, state $ARCH_1$, and result declarationResult be the result of evaluating variableDeclarationIR, under context EC_0 and state ARCH at LOCAL.
    b. If let rejectTransitionResult of type rejectTransitionResult be declarationResult:
        i. The result is context EC_1, state $ARCH_1$, and rejectTransitionResult.
    c. Else if let continueEmptyResult of type continueEmptyResult be declarationResult:
        i. The result is context EC_1, state $ARCH_1$, and $_{\text{EMPTY}}$.
3. Else:
    a. Let parserStatementIR be parserBlockElementStatementIR.
    b. Let context EC_1, state $ARCH_1$, and parserStatementResult be the result of runtime evaluation of parserStatementIR, under context EC_0 and state ARCH.
    c. The result is context EC_1, state $ARCH_1$, and parserStatementResult.

A list of block elements is runtime evaluated by the relation:

ParserBlockElementStmtList_eval:

- Runtime evaluation of parserBlockElementStatementListIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and parserStatementResult.

Runtime evaluation of parserBlockElementStatementIR*, under context EC, and state ARCH:

1. If parserBlockElementStatementIR* matches pattern []:
    a. The result is context EC, state ARCH, and $_{\text{EMPTY}}$.
2. Else:
    a. Let $\text{parserBlockElementStatementIR}_h$ :: $\text{parserBlockElementStatementIR}_t^*$ be parserBlockElementStatementIR*.
    b. Let context EC_1, state ARCH_1, and parserStatementResult be the result of runtime evaluation of $\text{parserBlockElementStatementIR}_h$, under context EC, and state ARCH.
    c. Try:
        i. If parserStatementResult is not equal to $_{\text{EMPTY}}$:
            A. The result is context EC_1, state ARCH_1, and parserStatementResult.
    d. Then, try:
        i. Check that parserStatementResult has type continueEmptyResult.
        ii. Let continueEmptyResult be parserStatementResult.
        iii. Let context EC_2, state ARCH_2, and parserStatementResult' be the result of runtime evaluation of $\text{parserBlockElementStatementIR}_t^*$, under context EC_1, and state ARCH_1.
        iv. The result is context EC_2, state ARCH_2, and parserStatementResult'.

16.4.7. Conditional statement

A parser conditional statement is defined as:

```
parserConditionalStatement
    : IF ( expression ) parserStatement
    | IF ( expression ) parserStatement ELSE parserStatement
    ;
```

It is similar to a conditional statement explained in Section 13.9, but it can only contain parser statements in its branches.

16.4.7.1. Type checking

After type checking, a parser conditional statement is represented in P4_{IR} as:

```
parserConditionalStatementIR
    : IF ( typedExpressionIR ) parserStatementIR
    | IF ( typedExpressionIR ) parserStatementIR ELSE parserStatementIR
    ;
```

```
<span id="ParserStmt_ok-parserConditionalStatement"></span>
```

Typing parserConditionalStatement, under context TC:

1. If let IF `( $\text{expression}_{\text{cond}}$ `) $\text{parserStatement}_{\text{then}}$ be parserConditionalStatement:
    a. Let $\text{typedExpressionIR}_{\text{cond}}$ be the result of typing $\text{expression}_{\text{cond}}$, under context TC at LOCAL.
    b. Let $\text{typeIR}_{\text{cond}}$ be the type of $\text{typedExpressionIR}_{\text{cond}}$.
    c. Check that BOOL is equal to $\text{typeIR}_{\text{cond}}$ with typedefs unrolled.
    d.

Let context TC_{then} and typed parser statement $parserStatementIR_{then}$ be the result of typing $parserStatement_{then}$, under context TC.

e. The result is context TC and typed parser statement IF `( $typedExpressionIR_{cond}$ `) $parserStatementIR_{then}$.

2. Else:

a. Let IF `( $expression_{cond}$ `) $parserStatement_{then}$ ELSE $parserStatement_{else}$ be parserConditionalStatement.

b. Let $typedExpressionIR_{cond}$ be the result of typing $expression_{cond}$, under context TC at LOCAL.

c. Let $typeIR_{cond}$ be the type of $typedExpressionIR_{cond}$.

d. Check that BOOL is equal to $typeIR_{cond}$ with typedefs unrolled.

e. Let context TC_{then} and typed parser statement $parserStatementIR_{then}$ be the result of typing $parserStatement_{then}$, under context TC.

f. Let context TC_{else} and typed parser statement $parserStatementIR_{else}$ be the result of typing $parserStatement_{else}$, under context TC.

g. The result is context TC and typed parser statement IF `( $typedExpressionIR_{cond}$ `) $parserStatementIR_{then}$ ELSE $parserStatementIR_{else}$.

16.4.7.2. Compile-time evaluation

Compile-time evaluation of parserConditionalStatementIR, under context IC_0 and store STO_0:

1. If let IF `( $typedExpressionIR_{cond}$ `) $parserStatementIR_{then}$ be parserConditionalStatementIR:

a. Let context IC_{then} and store STO_1 be the result of compile-time evaluation of $parserStatementIR_{then}$, under context IC_0 and store STO_0.

b. The result is context IC_0 and store STO_1.

2. Else:

a. Let IF `( $typedExpressionIR_{cond}$ `) $parserStatementIR_{then}$ ELSE $parserStatementIR_{else}$ be parserConditionalStatementIR.

b. Let context IC_{then} and store STO_1 be the result of compile-time evaluation of $parserStatementIR_{then}$, under context IC_0 and store STO_0.

c. Let context IC_{else} and store STO_2 be the result of compile-time evaluation of $parserStatementIR_{else}$, under context IC_0 and store STO_1.

d. The result is context IC_0 and store STO_2.

16.4.7.3. Runtime evaluation

Runtime evaluation of parserConditionalStatementIR, under context EC_0 and state $ARCH_0$:

1. If let IF `( $typedExpressionIR_{cond}$ `) $parserStatementIR_{then}$ be parserConditionalStatementIR:

a. Let context EC_{cond}, state $ARCH_{cond}$ and expressionResult be the result of evaluating $typedExpressionIR_{cond}$, under context EC_0 and state $ARCH_0$ at LOCAL.

b. Try:

i. If let rejectTransitionResult of type rejectTransitionResult be expressionResult:

A. The result is context EC_{cond}, state $ARCH_{cond}$, and rejectTransitionResult.

c. Then, try:

i. If expressionResult is equal to $_{CONT\ B}$ true:

A. Let context EC_{then}, state $ARCH_{then}$, and parserStatementResult be the result of runtime evaluation of $parserStatementIR_{then}$, under context EC_{cond} and state $ARCH_{cond}$.

B. The result is context EC_{then}, state $ARCH_{then}$, and parserStatementResult.

ii. Else if expressionResult is equal to $_{CONT\ B}$ false:

A. The result is context EC_{cond}, state $ARCH_{cond}$, and $_{EMPTY}$.

2. Else:

a. Let IF `( $typedExpressionIR_{cond}$ `) $parserStatementIR_{then}$ ELSE $parserStatementIR_{else}$ be parserConditionalStatementIR.

b. Let context EC_{cond}, state $ARCH_{cond}$ and expressionResult be the result of evaluating $typedExpressionIR_{cond}$, under context EC_0 and state $ARCH_0$ at LOCAL.

c. Try:

i. If let rejectTransitionResult of type rejectTransitionResult be expressionResult:

A. The result is context EC_{cond}, state $ARCH_{cond}$, and rejectTransitionResult.

d. Then, try:

i. If expressionResult is equal to $_{CONT\ B}$ true:

A. Let context EC_{then}, state $ARCH_{then}$, and parserStatementResult be the result of runtime evaluation of $parserStatementIR_{then}$, under context EC_{cond} and state $ARCH_{cond}$.

B. The result is context EC_{then}, state $ARCH_{then}$, and parserStatementResult.

ii. Else if expressionResult is equal to $_{CONT\ B}$ false:

A. Let context EC_{else}, state $ARCH_{else}$, and parserStatementResult be the result of runtime evaluation of $parserStatementIR_{else}$, under context EC_{cond} and state $ARCH_{cond}$.

B. The result is context EC_{else}, state $ARCH_{else}$, and parserStatementResult.

16.5. Parser transition statements

The last statement in a parser state is an optional transition statement, which transfers control to another state, possibly accept or reject. A transition statements is written using the following syntax:

```
transitionStatement
    : /* empty */ | TRANSITION stateExpression
    ;

stateExpression
    : name ; | selectExpression
    ;
```

The execution of the transition statement causes stateExpression to be evaluated, and transfers control to the resulting state.

In terms of the ParserModel, the semantics of a transition statement can be formalized as follows:

```
goto eval(stateExpression)
```

For example, this statement:

```
transition accept;
```

terminates execution of the current parser and transitions immediately to the accept state.

If the body of a state block does not end with a transition statement, the implied statement is:

```
transition reject;
```

See Section 16.6 for description of the select expression that can be used as the stateExpression in a transition statement.

16.5.1. Type checking

Parser transition statements are type-checked as follows:

ParserTransition_ok:

- Typing transitionStatement, under context TC and parser state names nameIR*:
- The result is typed transition statement transitionStatementIR.

Typing transitionStatement, under context TC_0 and parser state names $nameIR_{state}$*:

1. If transitionStatement matches pattern $_{EMPTY}$:
   a. The result is typed transition statement TRANSITION "reject" ';'.
2. Else:
   a. Let TRANSITION stateExpression be transitionStatement.
   b. If let name ';' be stateExpression:
      i. Let nameIR be the name of name.
      ii. Check that nameIR is in $nameIR_{state}$*.
      iii. Let transitionStatementIR be TRANSITION nameIR ';'.
      iv. The result is typed transition statement transitionStatementIR.
   c. Else:
      i. Let selectExpression be stateExpression.
      ii. Let typed transition statement selectExpressionIR be the result of typing selectExpression, under context TC_0 and parser state names $nameIR_{state}$*.
      iii. The result is typed transition statement TRANSITION selectExpressionIR.

After type checking, transition statements are represented in $P4_{IR}$ as:

```
transitionStatementIR
    : TRANSITION stateExpressionIR
    ;

stateExpressionIR
    : nameIR ; | selectExpressionIR
    ;
```

16.5.2. Runtime evaluation

ParserTransition_eval:

- Runtime evaluation of transitionStatementIR, under context EC and state ARCH:
- The result is context EC, state ARCH, and transitionResult.

Runtime evaluation of TRANSITION stateExpressionIR, under context EC and state ARCH:

1. If let nameIR ';' be stateExpressionIR:
    a. Try:
        i. If nameIR is equal to "accept":
            A. The result is context EC, state ARCH, and ACCEPT.
        ii. Else if nameIR is equal to "reject":
            A. Let errorValue be ERROR '.' "NoError".
            B. The result is context EC, state ARCH, and REJECT errorValue.
    b. Then, try:
        i. Check that nameIR is not equal to "accept" and nameIR is not equal to "reject".
        ii. The result is context EC, state ARCH, and STATE nameIR.
2. Else:
    a. Let selectExpressionIR be stateExpressionIR.
    b. Let context EC_1, state $ARCH_1$, and transitionResult be the result of runtime evaluation of selectExpressionIR, under context EC and state ARCH.
    c. The result is context EC_1, state $ARCH_1$, and transitionResult.

16.6. Parser select expression

A select expression evaluates to a state. The syntax for a select expression is as follows:

```
selectExpression
```

```
    : SELECT ( expressionList ) { selectCaseList }
    ;

selectCaseList
    : /* empty */ | selectCaseList selectCase
    ;

selectCase
    : keysetExpression : name ;
    ;
```

Each expression in the expressionList must have a set type, or a type that can be nested in a set type.

In terms of the ParserModel, the meaning of a select expression:

```
select(e) {
    ks[0]: s[0];
    ks[1]: s[1];
    /* more labels omitted */
    ks[n-2]: s[n-1];
    _ : sd;  // ks[n-1] is default
}
```

is defined in pseudo-code as:

```
key = eval(e);
for (int i=0; i < n; i++) {
    keyset = eval(ks[i]);
    if (keyset.contains(key)) return s[i];
}
verify(false, error.NoMatch);
```

Some targets may require that all keyset expressions in a select expression be compile-time known values. Keysets are evaluated in order, from top to bottom as implied by the pseudo-code above; the first keyset that includes the value in the select argument provides the result state. If no label matches, the execution triggers a runtime error with the standard error code error.NoMatch.

Note that this implies that all cases after a default or _ label are unreachable; the compiler should emit a warning if it detects unreachable cases. This constitutes an important difference between select expressions and the switch statements found in many programming languages since the keysets of a select expression may "overlap".

The typical way to use a select expression is to compare the value of a recently-extracted header field against a set of values, as in the following example:

```
header IPv4_h { bit<8> protocol; /* more fields omitted */ }
struct P { IPv4_h ipv4; /* more fields omitted */ }
P headers;
select (headers.ipv4.protocol) {
    8w6  : parse_tcp;
    8w17 : parse_udp;
    _    : accept;
}
```

For example, to detect TCP reserved ports (< 1024) one could write:

```
select (p.tcp.port) {
    16w0 &&& 16w0xFC00: well_known_port;
```

```
    _: other_port;
}
```

The expression 16w0 &&& 16w0xFC00 describes the set of 16-bit values whose most significant six bits are zero.

Some targets may support parser value sets; see Section 10.7. Given a type T for the type parameter of the value set, the type of the value set is set<T>. The type of the value set must match to the type of all other keysetExpressions in the same select expression. If there is a mismatch, the compiler must raise an error. The type of the values in the set must be either bit<>, int<>, tuple, struct, or serializable enum.

For example, to allow the control plane API to specify TCP reserved ports at runtime, one could write:

```
struct vsk_t {
    @match(ternary)
    bit<16> port;
}
value_set<vsk_t>(4) pvs;
select (p.tcp.port) {
    pvs: runtime_defined_port;
    _: other_port;
}
```

The above example allows the runtime API to populate up to 4 different keysetExpressions in the value_set. If the value_set takes a struct as type parameter, the runtime API can use the struct field names to name the objects in the value set. The match type of the struct field is specified with the @match annotation. If the @match annotation is not specified on a struct field, by default it is assumed to be @match(exact). A single non-exact field must be placed into a struct by itself, with the desired @match annotation.

16.6.1. Type checking

A select expression is type checked as follows:

ParserSelect_ok:

- Typing selectExpression, under context TC and parser state names nameIR*:
- The result is typed transition statement selectExpressionIR.

Typing SELECT `( expressionList$_{key}$ `) `{ selectCaseList `}, under context TC_0 and parser state names nameIR$_{state}$*:

1. Let expression$_{key}$* be expressionList$_{key}$ flattened.
2. For each expression$_{key}$ in expression$_{key}$*:

   - Let typedExpressionIR$_{key}$ be the result of typing expression$_{key}$, under context TC_0 at LOCAL.

   Let typedExpressionIR$_{key}$* be the resulting list.

3. For each typedExpressionIR$_{key}$ in typedExpressionIR$_{key}$*:

- Let typedExpressionIR$_{key_reduced}$ be typedExpressionIR$_{key}$ with surrounding serializable enums reduced.

Let typedExpressionIR$_{key_reduced}^{*}$ be the resulting list.

4. For each typedExpressionIR$_{key_reduced}$ in typedExpressionIR$_{key_reduced}^{*}$:

   - Let typeIR$_{key_reduced}$ be the type of typedExpressionIR$_{key_reduced}$.

   Let typeIR$_{key_reduced}^{*}$ be the resulting list.

5. Let bound be bound type variables from the LOCAL layer of TC$_0$.
6. If SET `< typeIR$_{key\_reduced}$ `> is a well-formed type, with bound type variables bound, for all typeIR$_{key_reduced}$ in typeIR$_{key_reduced}^{*}$:

   a. Let selectCase* be selectCaseList flattened.

   b. For each selectCase in selectCase*:

      - Let selectCaseIR be the result of typing selectCase under context TC$_0$ with states nameIR$_{state}^{*}$ and key types typeIR$_{key_reduced}^{*}$.

      Let selectCaseIR* be the resulting list.

   c. Let selectExpressionIR be SELECT `( typedExpressionIR$_{key\_reduced}^{*}$ `) `{ selectCaseIR$^{*}$ `}.

   d. The result is typed transition statement selectExpressionIR.

The type checking of a select case visits its keyset expressions, and each simple keyset expression is visited in turn.

SelectCase_ok:

- Typing selectCase under context TC with states nameIR* and key types typeIR*:
- The result is selectCaseIR.

Typing keysetExpression ':' name ';' under context TC with states nameIR$_{state}^{*}$ and key types typeIR$_{key}^{*}$:

1. Let keysetExpressionIR be the result of typing keysetExpression under context TC with key types typeIR$_{key}^{*}$.
2. Let nameIR be the name of name.
3. Check that nameIR is in nameIR$_{state}^{*}$.
4. The result is keysetExpressionIR ':' nameIR ';'.

A keyset expression is type checked with the relation:

```
keysetExpression
    : simpleKeysetExpression | tupleKeysetExpression
    ;
```

SelectCase_keyset_ok:

- Typing keysetExpression under context TC with key types typeIR*:
- The result is keysetExpressionIR.

SelectCase_keyset_ok dispatch:

1. Try:
    a. Goto simpleKeysetExpression
2. Then, try:
    a. Check that keysetExpression has type tupleKeysetExpression.
    b. Let tupleKeysetExpression be keysetExpression.
    c. Goto tupleKeysetExpression

16.6.1.1. Simple keyset expressions

Simple keyset expressions are type checked as:

Typing keysetExpression under context TC with key types typeIR*:

1. Try:
    a. If typeIR* matches pattern [_/1]:
        i. Let $typeIR_{key}$ be typeIR*.
        ii. Try:
            A. If let expression of type expression be keysetExpression:
                I. Let simpleKeysetExpressionIR be the result of typing expression under context TC with key type $typeIR_{key}$.
                II. The result is simpleKeysetExpressionIR.
        iii. Then, try:
            A. If let simpleKeysetExpression of type simpleKeysetExpression be keysetExpression:
                I. If let $expression_l$ '&&&' $expression_r$ be simpleKeysetExpression:
                    1. Let simpleKeysetExpressionIR be the result of typing $expression_l$ '&&&' $expression_r$ under context TC with key type $typeIR_{key}$.
                    2. The result is simpleKeysetExpressionIR.
                II. Else if let $expression_l$ '..' $expression_r$ be simpleKeysetExpression:
                    1. Let simpleKeysetExpressionIR be the result of typing $expression_l$ '..' $expression_r$ under context TC with key type $typeIR_{key}$.
                    2. The result is simpleKeysetExpressionIR.
        iv. Then, try:
            A. If keysetExpression is equal to DEFAULT:
                I. Let simpleKeysetExpressionIR be the result of typing DEFAULT under context TC with key type $typeIR_{key}$.

II. The result is simpleKeysetExpressionIR.

B. Else if keysetExpression is equal to '_':

I. Let simpleKeysetExpressionIR be the result of typing '_' under context TC with key type typeIR$_{key}$.

II. The result is simpleKeysetExpressionIR.

b. Else if typeIR* matches pattern []:

i. If keysetExpression is equal to DEFAULT: the result is DEFAULT.

2. Then, try:

a. If let expression of type expression be keysetExpression:

i. If the length of typeIR* is greater than 1:

A. Let typeIR$_{key_tuple}$ be TUPLE `< typeIR* `>.

B. Let simpleKeysetExpressionIR be the result of typing expression under context TC with key type typeIR$_{key_tuple}$.

C. The result is simpleKeysetExpressionIR.

3. Then, try:

a. If keysetExpression is equal to DEFAULT:

i. If the length of typeIR* is greater than 1:

A. Let simpleKeysetExpressionIR be the result of typing DEFAULT under context TC with key type SEQ `< typeIR* `>.

B. The result is simpleKeysetExpressionIR.

b. Else if keysetExpression is equal to '_':

i. If the length of typeIR* is greater than 1:

A. Let simpleKeysetExpressionIR be the result of typing '_' under context TC with key type SEQ `< typeIR* `>.

B. The result is simpleKeysetExpressionIR.

A simple keyset expression is type checked as follows:

```
simpleKeysetExpression
    : expression | expression &&& expression | expression .. expression
    | DEFAULT | _
    ;
```

SelectCase_keyset_simple_ok:

- Typing simpleKeysetExpression under context TC with key type typeIR:
- The result is simpleKeysetExpressionIR.

SelectCase_keyset_simple_ok dispatch:

1. If let expression of type expression be simpleKeysetExpression: goto expression
2. Else if let expression$_l$ '&&&' expression$_r$ be simpleKeysetExpression: goto mask

3. Else if let expression$_l$ '..' expression$_r$ be simpleKeysetExpression: goto range
4. Else if simpleKeysetExpression matches pattern DEFAULT: goto default
5. Else: goto dontcare

Expression keys

Typing expression under context TC with key type typeIR$_{key}$:

1. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
2. Let typeIR be the type of typedExpressionIR.
3. Let typeIR' be typeIR with typedefs unrolled.
4. If typeIR' has type setTypeIR:
   a. Let SET `< typeIR$_{base}$ `> be typeIR'.
   b. If typeIR$_{base}$ and typeIR$_{key}$ are the same type:
      i. The result is typedExpressionIR.
   c. Else:
      i. Let casts castResult be the result of checking if typeIR$_{base}$ can be implicitly cast to typeIR$_{key}$.
      ii. Check that castResult matches pattern OK %.
      iii. Let typeIR$_{cast}$ be SET `< typeIR$_{key}$ `>.
      iv. Let typedExpressionIR$_{cast}$ be `( typeIR$_{cast}$ `) typedExpressionIR annotated with a pair of type typeIR$_{cast}$ and compile-time known-ness DYN.
      v. The result is typedExpressionIR$_{cast}$.
5. Else:
   a. Let casts castResult be the result of checking if typeIR can be implicitly cast to typeIR$_{key}$.
   b. Check that castResult matches pattern OK %.
   c. Let typeIR$_{cast}$ be SET `< typeIR$_{key}$ `>.
   d. Let typedExpressionIR$_{cast}$ be `( typeIR$_{cast}$ `) typedExpressionIR annotated with a pair of type typeIR$_{cast}$ and compile-time known-ness DYN.
   e. The result is typedExpressionIR$_{cast}$.

Mask keys

Typing expression$_l$ '&&&' expression$_r$ under context TC with key type typeIR$_{key}$:

1. Let typedExpressionIR$_l$ be the result of typing expression$_l$, under context TC at LOCAL.
2. Let typedExpressionIR$_r$ be the result of typing expression$_r$, under context TC at LOCAL.
3. Let (typedExpressionIR, typedExpressionIR)$^?$ be typedExpressionIR$_l$ and typedExpressionIR$_r$ implicitly cast to equal types.
4. Check that (typedExpressionIR, typedExpressionIR)$^?$ is defined.
5. Let (typedExpressionIR$_{l_cast}$, typedExpressionIR$_{r_cast}$) be (typedExpressionIR, typedExpressionIR)$^?$.
6. Let (typedExpressionIR, typedExpressionIR)'$^?$ be the result of reducing serializable enums in typedExpressionIR$_{l_cast}$

and typedExpressionIR$_{r_cast}$ until $compat_mask is satisfied.

7. Check that (typedExpressionIR, typedExpressionIR)'? is defined.
8. Let (typedExpressionIR$_{l_reduced}$, typedExpressionIR$_{r_reduced}$) be (typedExpressionIR, typedExpressionIR)'?.
9. Let typeIR$_{reduced}$ be the type of typedExpressionIR$_{l_reduced}$.
10. Let typedExpressionIR? be typedExpressionIR$_{l_reduced}$ implicitly cast to typeIR$_{key}$.
11. Check that typedExpressionIR? is defined.
12. Let typedExpressionIR$_{l_casted}$ be typedExpressionIR?.
13. Let typedExpressionIR'? be typedExpressionIR$_{r_reduced}$ implicitly cast to typeIR$_{key}$.
14. Check that typedExpressionIR'? is defined.
15. Let typedExpressionIR$_{r_casted}$ be typedExpressionIR'?.
16. The result is typedExpressionIR$_{l_casted}$ '&&&' typedExpressionIR$_{r_casted}$.

Range keys

Typing expression$_l$ '..' expression$_r$ under context TC with key type typeIR$_{key}$:

1. Let typedExpressionIR$_l$ be the result of typing expression$_l$, under context TC at LOCAL.
2. Let typedExpressionIR$_r$ be the result of typing expression$_r$, under context TC at LOCAL.
3. Let (typedExpressionIR, typedExpressionIR)? be typedExpressionIR$_l$ and typedExpressionIR$_r$ implicitly cast to equal types.
4. Check that (typedExpressionIR, typedExpressionIR)? is defined.
5. Let (typedExpressionIR$_{l_cast}$, typedExpressionIR$_{r_cast}$) be (typedExpressionIR, typedExpressionIR)?.
6. Let (typedExpressionIR, typedExpressionIR)'? be the result of reducing serializable enums in typedExpressionIR$_{l_cast}$ and typedExpressionIR$_{r_cast}$ until $compat_range is satisfied.
7. Check that (typedExpressionIR, typedExpressionIR)'? is defined.
8. Let (typedExpressionIR$_{l_reduced}$, typedExpressionIR$_{r_reduced}$) be (typedExpressionIR, typedExpressionIR)'?.
9. Let typeIR$_{reduced}$ be the type of typedExpressionIR$_{l_reduced}$.
10. Check that typeIR$_{reduced}$ is equal to the type of typedExpressionIR$_{r_reduced}$.
11. Let typedExpressionIR? be typedExpressionIR$_{l_reduced}$ implicitly cast to typeIR$_{key}$.
12. Check that typedExpressionIR? is defined.
13. Let typedExpressionIR$_{l_casted}$ be typedExpressionIR?.
14. Let typedExpressionIR'? be typedExpressionIR$_{r_reduced}$ implicitly cast to typeIR$_{key}$.
15. Check that typedExpressionIR'? is defined.
16. Let typedExpressionIR$_{r_casted}$ be typedExpressionIR'?.
17. The result is typedExpressionIR$_{l_casted}$ '..' typedExpressionIR$_{r_casted}$.

Default and don't care keys

Typing DEFAULT under context TC with key type typeIR$_{key}$: the result is DEFAULT.

Typing '_' under context TC with key type $typeIR_{key}$: the result is '_'.

16.6.1.2. Tuple keyset expressions

Tuple keyset expressions are type checked as:

Typing tupleKeysetExpression under context TC with key types $typeIR^*$:

1. Try:
   a. If $typeIR^*$ is a list of length 1:
      i. Let $typeIR_{key}$ be $typeIR^*$.
      ii. If let `( $expression_l$ '&&&' $expression_r$ `) be tupleKeysetExpression:
         A. Let simpleKeysetExpressionIR be the result of typing $expression_l$ '&&&' $expression_r$ under context TC with key type $typeIR_{key}$.
         B. The result is `( simpleKeysetExpressionIR `).
      iii. Else if let `( $expression_l$ '..' $expression_r$ `) be tupleKeysetExpression:
         A. Let simpleKeysetExpressionIR be the result of typing $expression_l$ '..' $expression_r$ under context TC with key type $typeIR_{key}$.
         B. The result is `( simpleKeysetExpressionIR `).
      iv. Else if tupleKeysetExpression matches pattern `( DEFAULT `):
         A. Let simpleKeysetExpressionIR be the result of typing DEFAULT under context TC with key type $typeIR_{key}$.
         B. The result is `( simpleKeysetExpressionIR `).
      v. Else if tupleKeysetExpression matches pattern `( '_' `):
         A. Let simpleKeysetExpressionIR be the result of typing '_' under context TC with key type $typeIR_{key}$.
         B. The result is `( simpleKeysetExpressionIR `).
2. Then, try:
   a. If tupleKeysetExpression matches pattern `( DEFAULT `):
      i. Check that the length of $typeIR^*$ is greater than 1.
      ii. Let simpleKeysetExpressionIR be the result of typing DEFAULT under context TC with key type SEQ `< $typeIR^*$ `>.
      iii. The result is `( simpleKeysetExpressionIR `).
   b. Else if tupleKeysetExpression matches pattern `( '_' `):
      i. Check that the length of $typeIR^*$ is greater than 1.
      ii. Let simpleKeysetExpressionIR be the result of typing '_' under context TC with key type SEQ `< $typeIR^*$ `>.
      iii. The result is `( simpleKeysetExpressionIR `).
   c. Else if let `( $simpleKeysetExpression_h$ ',' $simpleKeysetExpressionList_t$ `) be tupleKeysetExpression:
      i. Let $simpleKeysetExpression_t^*$ be $simpleKeysetExpressionList_t$ flattened.
      ii. Let $simpleKeysetExpression^*$ be $simpleKeysetExpression_h$ :: $simpleKeysetExpression_t^*$.
      iii. Check that the length of $typeIR^*$ is equal to the length of $simpleKeysetExpression^*$.

iv. For each simpleKeysetExpression in simpleKeysetExpression* and typeIR in typeIR*:

- Let simpleKeysetExpressionIR be the result of typing simpleKeysetExpression under context TC with key type typeIR.

Let simpleKeysetExpressionIR* be the resulting list.

v. The result is `( simpleKeysetExpressionIR* `).

16.6.2. Runtime evaluation

The runtime evaluation of a select expression is defined as follows:

ParserSelect_eval:

- Runtime evaluation of selectExpressionIR, under context EC and state ARCH:
- The result is context EC, state ARCH, and transitionResult.

Runtime evaluation of selectExpressionIR, under context EC_0 and state $ARCH_0$:

1. Let SELECT `( $typedExpressionIR_{key}$* `) `{ selectCaseListIR `} be selectExpressionIR.
2. Let context EC_1, state $ARCH_1$ and expressionListResult be the result of evaluating $typedExpressionIR_{key}$*, under context EC_0 and state $ARCH_0$ at LOCAL.
3. If let rejectTransitionResult of type rejectTransitionResult be expressionListResult:
   a. The result is context EC_1, state $ARCH_1$, and rejectTransitionResult.
4. Else if let CONT $value_{key}$* of type continueResult<value*> be expressionListResult:
   a. Let the updated context EC_2, state $ARCH_2$, and selectCaseMatchResult be the result of matching $value_{key}$* against selectCaseListIR under context EC_1 and state $ARCH_1$.
   b. If let rejectTransitionResult of type rejectTransitionResult be selectCaseMatchResult:
      i. The result is context EC_2, state $ARCH_2$, and rejectTransitionResult.
   c. Else:
      i. Let CONT $nameIR_{state}$? be selectCaseMatchResult.
      ii. If $nameIR_{state}$? matches pattern ():
         A. Let rejectTransitionResult be REJECT ERROR '.' "NoMatch".
         B. The result is context EC_2, state $ARCH_2$, and rejectTransitionResult.
      iii. Else:
         A. Let $nameIR_{state'}$ be $nameIR_{state}$?.
         B. Let context EC_3, state $ARCH_3$, and transitionResult be the result of runtime evaluation of TRANSITION $nameIR_{state'}$ ';', under context EC_2 and state $ARCH_2$.
         C. The result is context EC_3, state $ARCH_3$, and transitionResult.

Matching against select cases is defined as follows:

SelectCases_match:

- Matching value* against selectCaseListIR under context EC and state ARCH:
- The result is the updated context EC, state ARCH, and selectCaseMatchResult.

The result of matching a select case is defined as follows:

```
selectCaseMatchResult
    : continueResult<nameIR?> | rejectTransitionResult
    ;
```

Matching $value_{key}^{*}$ against $selectCaseIR^{*}$ under context EC and state ARCH:

1. If $selectCaseIR^{*}$ matches pattern []:
    a. The result is the updated context EC, state ARCH, and $_{CONT}\cdot$.
2. Else:
    a. Let $selectCaseIR_{h} :: selectCaseIR_{t}^{*}$ be $selectCaseIR^{*}$.
    b. Let the updated context EC_1, state $ARCH_1$, and selectCaseMatchResult' be the result of matching $value_{key}^{*}$ against $selectCaseIR_{h}$ under context EC and state ARCH.
    c. Try:
        i. If let rejectTransitionResult of type rejectTransitionResult be selectCaseMatchResult':
            A. The result is the updated context EC_1, state $ARCH_1$, and rejectTransitionResult.
        ii. Else:
            A. Let $_{CONT}\,nameIR^{?}$ be selectCaseMatchResult'.
            B. If $nameIR^{?}$ is defined:
                I. Let $nameIR_{state}$ be $nameIR^{?}$.
                II. The result is the updated context EC_1, state $ARCH_1$, and $_{CONT}\,nameIR_{state}$.
    d. Then, try:
        i. Check that selectCaseMatchResult' is equal to $_{CONT}\cdot$.
        ii. Let the updated context EC_2, state $ARCH_2$, and selectCaseMatchResult be the result of matching $value_{key}^{*}$ against $selectCaseIR_{t}^{*}$ under context EC_1 and state $ARCH_1$.
        iii. The result is the updated context EC_2, state $ARCH_2$, and selectCaseMatchResult.

The matching of a select case is defined as follows:

SelectCase_match:

- Matching value* against selectCaseIR under context EC and state ARCH:
- The result is the updated context EC, state ARCH, and selectCaseMatchResult.

Matching $value_{key}^{*}$ against keysetExpressionIR ':' nameIR ';' under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and result keysetExpressionResult be the result of evaluating keysetExpressionIR, under context EC_0 and state $ARCH_0$.
2. If let rejectTransitionResult of type rejectTransitionResult be keysetExpressionResult:
   a. The result is the updated context EC_1, state $ARCH_1$, and rejectTransitionResult.
3. Else if let $_{CONT}$ setValue* of type continueResult<setValue*> be keysetExpressionResult:
   a. If $value_{key}^{*}$ does not match setValue*:
      i. The result is the updated context EC_1, state $ARCH_1$, and $_{CONT}$ ·.
   b. Else:
      i. The result is the updated context EC_1, state $ARCH_1$, and $_{CONT}$ nameIR.

16.7. verify

The verify statement provides a simple form of error handling. verify can only be invoked within a parser; it is used syntactically as if it were a function with the following signature:

```
extern void verify(in bool condition, in error err);
```

If the first argument is true, then executing the statement has no side-effect. However, if the first argument is false, it causes an immediate transition to reject, which causes immediate parsing termination; at the same time, the parserError associated with the parser is set to the value of the second argument.

In terms of the ParserModel the semantics of a verify statement is given by:

```
ParserModel.verify(bool condition, error err) {
    if (condition == false) {
        ParserModel.parserError = err;
        goto reject;
    }
}
```

16.8. Packet data extraction

The P4 core library contains the following declaration of a built-in extern type called packet_in that represents incoming network packets. The packet_in extern is special: it cannot be instantiated by the user explicitly. Instead, the architecture supplies a separate instance for each packet_in argument to a parser instantiation.

```
extern packet_in {
    void extract<T>(out T headerLvalue);
    void extract<T>(out T variableSizeHeader, in bit<32> varFieldSizeBits);
    T lookahead<T>();
```

```
    bit<32> length();  // This method may be unavailable in some architectures
    void advance(bit<32> bits);
}
```

To extract data from a packet represented by an argument b with type packet_in, a parser invokes the extract methods of b. There are two variants of the extract method: a one-argument variant for extracting fixed-size headers, and a two-argument variant for extracting variable-sized headers. Because these operations can cause runtime verification failures (see below), these methods can only be executed within parsers.

When extracting data into a bit-string or integer, the first packet bit is extracted to the most significant bit of the integer.

Some targets may perform cut-through packet processing, i.e., they may start processing a packet before its length is known (i.e., before all bytes have been received). On such a target calls to the packet_in.length() method cannot be implemented. Attempts to call this method should be flagged as errors (either at compilation time by the compiler back-end, or when attempting to load the compiled P4 program onto a target that does not support this method).

In terms of the ParserModel, the semantics of packet_in can be captured using the following abstract model of packets:

```
packet_in {
    unsigned nextBitIndex;
    byte[] data;
    unsigned lengthInBits;
    void initialize(byte[] data) {
        this.data = data;
        this.nextBitIndex = 0;
        this.lengthInBits = data.sizeInBytes * 8;
    }
    bit<32> length() { return this.lengthInBits / 8; }
}
```

16.8.1. Fixed-width extraction

The single-argument extract method handles fixed-width headers, and is declared in P4 as follows:

```
void extract<T>(out T headerLvalue);
```

The expression headerLvalue must evaluate to an l-value (see Chapter 12) of type header with a fixed width. If this method executes successfully, on completion the headerLvalue is filled with data from the packet and its validity bit is set to true. This method may fail in various ways—e.g., if there are not enough bits left in the packet to fill the specified header.

For example, the following program fragment extracts an Ethernet header:

```
struct Result { Ethernet_h ethernet;  /* more fields omitted */ }
parser P(packet_in b, out Result r) {
    state start {
        b.extract(r.ethernet);
    }
}
```

In terms of the ParserModel, the semantics of the single-argument extract is given in terms of the following pseudo-code method, using data from the packet class defined above. We use the special valid$ identifier to indicate the hidden valid bit of a header, isNext$ to indicate that the l-value was obtained using next, and

nextIndex$ to indicate the corresponding header or header union stack properties.

```
void packet_in.extract<T>(out T headerLValue) {
   bitsToExtract = sizeofInBits(headerLValue);
   lastBitNeeded = this.nextBitIndex + bitsToExtract;
   ParserModel.verify(this.lengthInBits >= lastBitNeeded, error.PacketTooShort);
   headerLValue = this.data.extractBits(this.nextBitIndex, bitsToExtract);
   headerLValue.valid$ = true;
   if headerLValue.isNext$ {
     verify(headerLValue.nextIndex$ < headerLValue.size, error.StackOutOfBounds);
     headerLValue.nextIndex$ = headerLValue.nextIndex$ + 1;
   }
   this.nextBitIndex += bitsToExtract;
}
```

16.8.2. Variable-width extraction

The two-argument extract handles variable-width headers, and is declared in P4 as follows:

```
void extract<T>(out T headerLvalue, in bit<32> variableFieldSize);
```

The expression headerLvalue must be an l-value representing a header that contains **exactly one** varbit field. The expression variableFieldSize must evaluate to a bit<32> value that indicates the number of bits to be extracted into the unique varbit field of the header (i.e., this size is not the size of the complete header, just the varbit field).

In terms of the ParserModel, the semantics of the two-argument extract is captured by the following pseudo-code:

```
void packet_in.extract<T>(out T headerLvalue,
                          in bit<32> variableFieldSize) {
   // targets are allowed to include the following line, but need not
   // verify(variableFieldSize[2:0] == 0, error.ParserInvalidArgument);
   bitsToExtract = sizeOfFixedPart(headerLvalue) + variableFieldSize;
   lastBitNeeded = this.nextBitIndex + bitsToExtract;
   ParserModel.verify(this.lengthInBits >= lastBitNeeded, error.PacketTooShort);
   ParserModel.verify(bitsToExtract <= headerLvalue.maxSize, error.HeaderTooShort);
   headerLvalue = this.data.extractBits(this.nextBitIndex, bitsToExtract);
   headerLvalue.varbitField.size = variableFieldSize;
   headerLvalue.valid$ = true;
   if headerLValue.isNext$ {
     verify(headerLValue.nextIndex$ < headerLValue.size, error.StackOutOfBounds);
     headerLValue.nextIndex$ = headerLValue.nextIndex$ + 1;
   }
   this.nextBitIndex += bitsToExtract;
}
```

The following example shows one way to parse IPv4 options—by splitting the IPv4 header into two separate headers:

```
// IPv4 header without options
header IPv4_no_options_h {
   bit<4>   version;
   bit<4>   ihl;
   bit<8>   diffserv;
   bit<16>  totalLen;
   bit<16>  identification;
   bit<3>   flags;
   bit<13>  fragOffset;
```

```
    bit<8>   ttl;
    bit<8>   protocol;
    bit<16>  hdrChecksum;
    bit<32>  srcAddr;
    bit<32>  dstAddr;
}
header IPv4_options_h {
    varbit<320> options;
}

struct Parsed_headers {
     // Some fields omitted
     IPv4_no_options_h ipv4;
     IPv4_options_h    ipv4options;
}

error { InvalidIPv4Header }

parser Top(packet_in b, out Parsed_headers headers) {
    // Some states omitted

    state parse_ipv4 {
        b.extract(headers.ipv4);
        verify(headers.ipv4.ihl >= 5, error.InvalidIPv4Header);
        transition select (headers.ipv4.ihl) {
            5: dispatch_on_protocol;
            _: parse_ipv4_options;
        }
    }

    state parse_ipv4_options {
        // use information in the ipv4 header to compute the number of bits to extract
        b.extract(headers.ipv4options,
                  (bit<32>)(((bit<16>)headers.ipv4.ihl - 5) * 32));
        transition dispatch_on_protocol;
    }
}
```

16.8.3. Lookahead

The lookahead method provided by the packet_in packet abstraction evaluates to a set of bits from the input packet without advancing the nextBitIndex pointer. Similar to extract, it will transition to reject and set the error if there are not enough bits in the packet. When lookahead returns a value that contains headers (e.g., a header type, or a struct containing headers), the headers values in the returned result are always valid (otherwise lookahead must have transitioned to the reject state).

The lookahead method can be invoked as follows:

```
b.lookahead<T>()
```

where T must be a type with fixed width. In case of success the result of the evaluation of lookahead returns a value of type T.

In terms of the ParserModel, the semantics of lookahead is given by the following pseudocode:

```
T packet_in.lookahead<T>() {
    bitsToExtract = sizeof(T);
    lastBitNeeded = this.nextBitIndex + bitsToExtract;
    ParserModel.verify(this.lengthInBits >= lastBitNeeded, error.PacketTooShort);
    T tmp = this.data.extractBits(this.nextBitIndex, bitsToExtract);
    return tmp;
```

```
}
```

The TCP options example from Section 8.4.7 also illustrates how lookahead can be used:

```
state start {
    transition select(b.lookahead<bit<8>>()) {
        0: parse_tcp_option_end;
        1: parse_tcp_option_nop;
        2: parse_tcp_option_ss;
        3: parse_tcp_option_s;
        5: parse_tcp_option_sack;
    }
}

// Some states omitted

state parse_tcp_option_sack {
    bit<8> n = b.lookahead<Tcp_option_sack_top>().length;
    b.extract(vec.next.sack, (bit<32>) (8 * n - 16));
    transition start;
}
```

16.8.4. Skipping bits

P4 provides two ways to skip over bits in an input packet without assigning them to a header:

One way is to extract to the underscore identifier, explicitly specifying the type of the data:

```
b.extract<T>(_)
```

Another way is to use the advance method of the packet when the number of bits to skip is known.

In terms of the ParserModel, the meaning of advance is given in pseudocode as follows:

```
void packet_in.advance(bit<32> bits) {
   // targets are allowed to include the following line, but need not
   // verify(bits[2:0] == 0, error.ParserInvalidArgument);
   lastBitNeeded = this.nextBitIndex + bits;
   ParserModel.verify(this.lengthInBits >= lastBitNeeded, error.PacketTooShort);
   this.nextBitIndex += bits;
}
```

17. Abstract control machine

A control declaration introduces a constructor for a control object. See Section 10.5 and Section 11.12 for details.

```
controlDeclaration
    : annotationList CONTROL name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { controlLocalDeclarationList APPLY controlBody }
    ;

controlLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | actionDeclaration | tableDeclaration
    ;
```

```
controlBody = blockStatement
```

The rest of this section describes the core components of a control block.

17.1. Match-action pipeline machine

We can describe the computational model of a match-action pipeline, embodied by a control block: the body of the control block is executed, similarly to the execution of a traditional imperative program:

- At runtime, statements within a block are executed in the order they appear in the control block.
- Execution of the return statement causes immediate termination of the execution of the current control block, and a return to the caller.
- Execution of the exit statement causes the immediate termination of the execution of the current control block and of all the enclosing caller control blocks.
- Applying a table executes the corresponding match-action unit.

17.1.1. Sub-controls

P4 allows controls to invoke the services of other controls, similar to subroutines. To invoke the services of another control, it must be first instantiated; the services of an instance are invoked by calling it using its apply method.

The following example shows a control invocation:

```
control Callee(inout IPv4 ipv4) { /* body omitted */ }
control Caller(inout Headers h) {
     Callee() instance;  // instance of callee
     apply {
          instance.apply(h.ipv4);  // invoke control
     }
}
```

As with parsers, when a control is instantiated, local instantiations of stateful objects are evaluated recursively. That is, each instantiation of a control has a unique set of local tables, extern objects, inner control instances, etc. Thus, in general, invoking a control instance twice is not the same as invoking two copies of the same control instance. Note however, that local variables do not persist across invocations of the control. This semantics also applies to direct invocation (see Section 13.5).

When a control is instantiated, all its local declarations of stateful instantiations are evaluated recursively. Each instantiation of a control will have a unique set of local tables, extern objects, and inner control instances. Thus, invoking a control instance twice is different from invoking two control instances each once, where the former accesses the same local stateful constructs while the latter access two different copies.

The exactly-once evaluation only applies to local stateful instantiations. For local variable declarations, whether in the apply block or out, and whether with initializers or not, they are always evaluated when a control instance is invoked. That is, local variables in a control never persist across invocations. For variables declared outside the apply block, they are evaluated at the beginning of execution.

All the behavior above also applies to direct invocation (see Section 13.5).

17.1.2. Runtime evaluation of a control

At runtime, a control is evaluated by invoking its apply method. The following algorithm describes the evaluation of a control apply method invocation:

Calling controlApplyMethodCallee `< typeArgumentListIR `> `( argumentListIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let CONTROL _ '.' APPLY `( parameterListIR '#' $id_{default}^{*}$ `) `{ theta ';' frame ';' controlLocalDeclarationListIR ';' actionDefEnv ';' controlBodyIR `} be controlApplyMethodCallee.
2. Let EC_{callee_0} be copy of EC_0 from the GLOBAL layer.
3. Let `{ ( callableId ':' actionDef )* `} be actionDefEnv.
4. Let EC_{callee_1} be EC_{callee_0}[BLOCK.TYPE = theta][BLOCK.FRAME = frame] with BLOCK.CALLABLE set to `{ ( callableId ':' actionDef )* `}.
5. Let GIVEN $parameterIR_{aligned}^{*}$ DEFAULT $parameterIR_{default}^{*}$ OPTIONAL $_{}^{*}$ be aligning parameterListIR with argumentListIR where defaults are $id_{default}^{*}$ and optionals are ·.
6. Let p_{shared} be GLOBAL.
7. Let state $ARCH_1$, caller's context EC_1, callee's context EC_{callee_2}, and copy-in result copyInResult be the result of copy-in from argumentListIR at BLOCK of EC_{callee_1} to $parameterIR_{aligned}^{*}$ in p at EC_0, under state $ARCH_0$ at shared p_{shared}.
8. If let exitResult of type exitResult be copyInResult:
   a. The result is context EC_1, state $ARCH_1$, and call result EXIT.
9. Else if let $_{CONT}$ $storageReference^{?*}$ of type continueResult<storageReference?*> be copyInResult:
   a. Let EC_{callee_3} be the result of copying defaults for $parameterIR_{default}^{*}$, under context BLOCK and context EC_{callee_2}.
   b. Let context EC_{callee_4}, state $ARCH_2$, and result controlLocalDeclarationResult be the result of evaluating controlLocalDeclarationListIR, under context EC_{callee_3}, and state $ARCH_1$.
   c. If let exitResult of type exitResult be controlLocalDeclarationResult:
      i. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_4}, under state $ARCH_2$ at shared p_{shared}.
      ii. The result is context EC_2, state $ARCH_2$, and call result EXIT.
   d. Else:
      i. Let continueEmptyResult be controlLocalDeclarationResult.
      ii. Let context EC_{callee_5}, state $ARCH_3$, and result controlBodyResult be the result of evaluating controlBodyIR, under context EC_{callee_4}, and state $ARCH_2$.
      iii. Try:
         A. If let exitResult of type exitResult be controlBodyResult:
            I. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_5}, under state $ARCH_3$ at shared p_{shared}.
            II. The result is context EC_2, state $ARCH_3$, and call result EXIT.
      iv. Then, try:
         A. Check that controlBodyResult is equal to RETURN ·.
         B. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_5}, under state $ARCH_3$ at shared p_{shared}.
         C. The result is context EC_2, state $ARCH_3$, and call result RETURN ·.

17.2. Control local declarations

Control local declarations are defined as follows:

```
controlLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | actionDeclaration | tableDeclaration
    ;

controlLocalDeclarationList
    : /* empty */ | controlLocalDeclarationList controlLocalDeclaration
    ;
```

17.2.1. Semantics of parser local declarations

17.2.1.1. Type checking

ControlLocalDecl_ok:

- Typing controlLocalDeclaration, under context TC:
- The result is context TC and controlLocalDeclarationIR.

ControlLocalDecl_ok dispatch:

1. If let constantDeclaration of type constantDeclaration be controlLocalDeclaration: goto constantDeclaration
2. Else if let instantiation of type instantiation be controlLocalDeclaration: goto instantiation
3. Else if let variableDeclaration of type variableDeclaration be controlLocalDeclaration: goto variableDeclaration
4. Else if let actionDeclaration of type actionDeclaration be controlLocalDeclaration: goto actionDeclaration
5. Else:
    a. Let annotationList TABLE name `{ tablePropertyList `} be controlLocalDeclaration.
    b. Goto tableDeclaration

After type checking, control local declarations are represented in $P4_{IR}$ as follows:

```
controlLocalDeclarationIR
    : constantDeclarationIR | instantiationIR | variableDeclarationIR
    | actionDeclarationIR | tableDeclarationIR
    ;
```

A list of control local declarations is type checked by:

ControlLocalDeclList_ok:

- Typing controlLocalDeclarationList, under context TC:
- The result is context TC and controlLocalDeclarationIR*.

Typing controlLocalDeclarationList, under context TC_0:

1. Let controlLocalDeclaration* be controlLocalDeclarationList flattened.
2. Let the updated context TC_1 and the typed control local declarations controlLocalDeclarationIR* be the result of typing controlLocalDeclaration*, under context TC_0.
3. The result is context TC_1 and controlLocalDeclarationIR*.

ControlLocalDecls_ok:

- Typing controlLocalDeclaration*, under context TC:
- The result is the updated context TC and the typed control local declarations controlLocalDeclarationIR*.

Typing controlLocalDeclaration*, under context TC:

1. If controlLocalDeclaration* matches pattern []:
   a. The result is the updated context TC and the typed control local declarations ·.
2. Else:
   a. Let $controlLocalDeclaration_h :: controlLocalDeclaration_t^*$ be controlLocalDeclaration*.
   b. Let context TC_1 and $controlLocalDeclarationIR_h$ be the result of typing $controlLocalDeclaration_h$, under context TC.
   c. Let the updated context TC_2 and the typed control local declarations $controlLocalDeclarationIR_t^*$ be the result of typing $controlLocalDeclaration_t^*$, under context TC_1.
   d. The result is the updated context TC_2 and the typed control local declarations $controlLocalDeclarationIR_h :: controlLocalDeclarationIR_t^*$.

17.2.1.2. Compile-time evaluation

ControlLocalDecl_inst:

- Compile-time evaluation of controlLocalDeclarationIR, under context ICand store STO:
- The result is context IC, store STO, and controlLocalDeclarationIR?.

ControlLocalDecl_inst dispatch:

1. If let constantDeclarationIR of type constantDeclarationIR be controlLocalDeclarationIR: goto constantDeclarationIR
2. Else if let instantiationIR of type instantiationIR be controlLocalDeclarationIR: goto instantiationIR
3. Else if let variableDeclarationIR of type variableDeclarationIR be controlLocalDeclarationIR: goto variableDeclarationIR
4. Else if let actionDeclarationIR of type actionDeclarationIR be controlLocalDeclarationIR: goto actionDeclarationIR
5. Else:
    a. Let annotationList TABLE typeIR nameIR `{ tablePropertyListIR `} be controlLocalDeclarationIR.
    b. Goto tableDeclarationIR

A list of control local declarations is compile-time evaluated by:

ControlLocalDecls_inst:

- Compile-time evaluation of controlLocalDeclarationIR*, under context ICand store STO:
- The result is context IC, store STO, and controlLocalDeclarationIR*.

Compile-time evaluation of controlLocalDeclarationIR*, under context ICand store STO:

1. If controlLocalDeclarationIR* matches pattern []:
    a. The result is context IC, store STO, and ·.
2. Else:
    a. Let $controlLocalDeclarationIR_h$ :: $controlLocalDeclarationIR_t^*$ be controlLocalDeclarationIR*.
    b. Let context IC_1, store STO_1, and controlLocalDeclarationIR'? be the result of compile-time evaluation of $controlLocalDeclarationIR_h$, under context ICand store STO.
    c. If controlLocalDeclarationIR'? matches pattern ():
        i. Let context IC_2, store STO_2, and $controlLocalDeclarationIR_{t_inst}^*$ be the result of compile-time evaluation of $controlLocalDeclarationIR_t^*$, under context $IC_1$and store STO_1.
        ii. The result is context IC_2, store STO_2, and $controlLocalDeclarationIR_{t_inst}^*$.
    d. Else:
        i. Let $controlLocalDeclarationIR_{h_inst}$ be controlLocalDeclarationIR'?.
        ii. Let context IC_2, store STO_2, and $controlLocalDeclarationIR_{t_inst}^*$ be the result of compile-time evaluation of $controlLocalDeclarationIR_t^*$, under context $IC_1$and store STO_1.
        iii. The result is context IC_2, store STO_2, and $controlLocalDeclarationIR_{h_inst}$ :: $controlLocalDeclarationIR_{t_inst}^*$.

17.2.1.3. Runtime evaluation

ControlLocalDecl_eval:

- Evaluating controlLocalDeclarationIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and result controlLocalDeclarationResult.

ControlLocalDecl_eval dispatch:

1. If let constantDeclarationIR of type constantDeclarationIR be controlLocalDeclarationIR: goto constantDeclarationIR
2. Else if let variableDeclarationIR of type variableDeclarationIR be controlLocalDeclarationIR: goto variableDeclarationIR

Evaluation of a control local declaration yields:

```
controlLocalDeclarationResult
    : continueEmptyResult | exitResult
    ;

continueEmptyResult
    : /* empty */
    ;

exitResult
    : EXIT
    ;
```

A list of control local declarations is runtime evaluated by:

ControlLocalDecls_eval:

- Evaluating controlLocalDeclarationIR*, under context EC, and state ARCH:
- The result is context EC, state ARCH, and result controlLocalDeclarationResult.

Evaluating controlLocalDeclarationIR*, under context EC, and state ARCH:

1. If controlLocalDeclarationIR* matches pattern []:
   a. The result is context EC, state ARCH, and result EMPTY.
2. Else:
   a. Let $controlLocalDeclarationIR_h$ :: $controlLocalDeclarationListIR_t$ be controlLocalDeclarationIR*.
   b. Let context EC_1, state $ARCH_1$, and result controlLocalDeclarationResult' be the result of evaluating $controlLocalDeclarationIR_h$, under context EC, and state ARCH.
   c. If let exitResult of type exitResult be controlLocalDeclarationResult':
      i. The result is context EC_1, state $ARCH_1$, and result EXIT.

d. Else:

   i. Let continueEmptyResult be controlLocalDeclarationResult'.

   ii. Let context EC_2, state $ARCH_2$, and result controlLocalDeclarationResult be the result of evaluating controlLocalDeclarationListIR$_t$, under context EC_1, and state $ARCH_1$.

   iii. The result is context EC_2, state $ARCH_2$, and result controlLocalDeclarationResult.

The subsequent sections describe each kind of declaration in detail. table declarations are special and are separately described in Section 17.3.

17.2.2. Constant declarations

See Section 11.3 for general information about constant declarations.

17.2.2.1. Type checking

Typing constantDeclaration, under context TC_0:

1. Let context TC_1 and constantDeclarationIR be the result of typing constantDeclaration, under context TC_0 at BLOCK.
2. The result is context TC_1 and constantDeclarationIR.

17.2.2.2. Compile-time evaluation

Compile-time evaluation of constantDeclarationIR, under context $IC_0$and store STO:

1. Let context IC_1 be the result of loading constantDeclarationIR, under context IC_0 at BLOCK.
2. The result is context IC_1, store STO, and constantDeclarationIR.

17.2.2.3. Runtime evaluation

Evaluating constantDeclarationIR, under context EC_0, and state ARCH:

1. Let context EC_1 be the result of evaluating constantDeclarationIR, under context EC_0 at BLOCK.
2. The result is context EC_1, state ARCH, and result $_{\text{EMPTY}}$.

17.2.3. Instantiations

See Section 11.4 for general information about instantiations.

17.2.3.1. Type checking

Typing instantiation, under context TC_0:

1. Let context TC_1 and instantiationIR be the result of typing instantiation, under context TC_0 at BLOCK.
2. The result is context TC_1 and instantiationIR.

17.2.3.2. Compile-time evaluation

Compile-time evaluation of instantiationIR, under context $IC_0$and store STO:

1. Let context IC_1, store STO_1, and reference to object constantDeclarationIR be the result of loading instantiationIR, under context IC_0 and store STO at BLOCK.
2. The result is context IC_1, store STO_1, and constantDeclarationIR.

17.2.4. Variable declarations

See Section 11.2 for general information about variable declarations.

17.2.4.1. Type checking

Typing variableDeclaration, under context TC_0:

1. Let context TC_1 and variableDeclarationIR be the result of typing variableDeclaration, under context TC_0 at BLOCK.
2. The result is context TC_1 and variableDeclarationIR.

17.2.4.2. Compile-time evaluation

Compile-time evaluation of variableDeclarationIR, under context $IC_0$and store STO:

1. The result is context IC_0, store STO, and variableDeclarationIR.

17.2.4.3. Runtime evaluation

Evaluating variableDeclarationIR, under context EC_0, and state ARCH:

1. Let context EC_1, state $ARCH_1$, and result declarationResult be the result of evaluating variableDeclarationIR, under context EC_0 and state ARCH at BLOCK.
2. If let exitResult of type exitResult be declarationResult:
    a. The result is context EC_1, state $ARCH_1$, and result EXIT.
3. Else if let continueEmptyResult of type continueEmptyResult be declarationResult:

a. The result is context EC_1, state $ARCH_1$, and result $_{EMPTY}$.

17.2.5. Action declarations

See Section 11.6 for general information about action declarations.

17.2.5.1. Type checking

Typing actionDeclaration, under context TC_0:

1. Let context TC_1 and actionDeclarationIR be the result of typing actionDeclaration, under context TC_0 at BLOCK.
2. The result is context TC_1 and actionDeclarationIR.

17.2.5.2. Compile-time evaluation

Compile-time evaluation of actionDeclarationIR, under context $IC_0$and store STO:

1. Let context IC_1 be the result of loading actionDeclarationIR, under context IC_0 at BLOCK.
2. The result is context IC_1, store STO, and ·.

17.2.6. Table declarations

17.2.6.1. Type checking

Typing annotationList TABLE name `{ tablePropertyList `}, under context TC_0:

1. Check that annotationList has at most one name annotation.
2. Let TC_1 be TC_0 with LOCAL.KIND set to TABLE_APPLY_METHOD.
3. Let $tableProperty^*$ be tablePropertyList flattened.
4. Let context TBLC and $tablePropertyIR^*$ be the result of typing $tableProperty^*$, under context TC_1.
5. Let nameIR be the name of name.
6. Let context TC_2 and type of table $typeIR_{table}$ be the result of typing name, under context TC_1 and table context TBLC.
7. Let varTypeIR be $_{EMPTY}$ $typeIR_{table}$ DYN ·.
8. Let TC_3 be TC_2 where nameIR to varTypeIR is added to the BLOCK layer.
9. Let tableDeclarationIR be annotationList TABLE $typeIR_{table}$ nameIR `{ $tablePropertyIR^*$ `}.
10. The result is context TC_3 and tableDeclarationIR.

A table is type checked by:

Table_ok:

- Typing tableProperty*, under context TC:
- The result is context TBLC and tablePropertyListIR.

Typing tableProperty*, under context TC:

1. Check that the length of tableProperty* filtered by $is_tableKeysProperty(tableProperty)* is less than or equal to 1.
2. Check that the length of tableProperty* filtered by $is_tableActionsProperty(tableProperty)* is equal to 1.
3. If the length of tableProperty* filtered by $is_tableDefaultActionProperty(tableProperty)* is equal to 0:
    a. Check that the length of tableProperty* filtered by $is_tableLargestPriorityWinsProperty(tableProperty)* is less than or equal to 1.
    b. Let $TBLC_0$ be $empty_tableContext.
    c. Let table context $TBLC_1$ and tablePropertyIR* be the result of typing tableProperty*, under context TC and table context $TBLC_0$.
    d. Let (callableId, callableTypeDefIR)? be the non-overloaded callable definition of '.' "NoAction" from the LOCAL layer of TC.
    e. Check that (callableId, callableTypeDefIR)? is defined.
    f. Let (_, callableTypeDefIR) be (callableId, callableTypeDefIR)?.
    g. Check that callableTypeDefIR has type actionTypeIR.
    h. Let _ ACTION _ `( parameterIR* `) be callableTypeDefIR.
    i. Check that parameterIR* is an empty list.
    j. Let $tablePropertyIR_{default_action}$ be $_{EMPTY}$ · DEFAULT_ACTION '=' $_{BARE}$ "NoAction" · `( · `) ';'.
    k. Let ($TBLC_2$, $tablePropertyIR_{updated}$*) be $insert_NoAction_tablePropertyIR($TBLC_1$, tablePropertyIR*).
    l. The result is context $TBLC_2$ and $tablePropertyIR_{updated}$* concatenated with $tablePropertyIR_{default_action}$.
4. Else if the length of tableProperty* filtered by $is_tableDefaultActionProperty(tableProperty)* is equal to 1:
    a. Check that the length of tableProperty* filtered by $is_tableLargestPriorityWinsProperty(tableProperty)* is less than or equal to 1.
    b. Let $TBLC_0$ be $empty_tableContext.
    c. Let table context $TBLC_1$ and tablePropertyIR* be the result of typing tableProperty*, under context TC and table context $TBLC_0$.
    d. The result is context $TBLC_1$ and tablePropertyIR*.

As explained in Section 8.6.6, a table declaration introduces a synthesized table metadata type.

TableType_ok:

- Typing name, under context TC and table context TBLC:
- The result is context TC and type of table typeIR.

Typing name, under context TC_0 and table context TBLC:

1. Let nameIR be the name of name.
2. Let $typeId_{enum}$ be "action_list(“` concatenated with nameIR concatenated with `”)".
3. Let $(prefixedNameIR_{action} `( _ '@' _ `))^*$ be TBLC.ACTIONS.
4. For each $prefixedNameIR_{action}$ in $prefixedNameIR_{action}^*$:

   - Let $nameIR_{enum_field}$ be the string form of $prefixedNameIR_{action}$.

   Let $nameIR_{enum_field}^*$ be the resulting list.
5. Let tableMetadataEnumTypeIR be TABLE_ENUM $typeId_{enum}$ `{ $nameIR_{enum\_field}^*$ `}.
6. For each $nameIR_{enum_field}$ in $nameIR_{enum_field}^*$:

   - Let $value_{enum_field}$ be TABLE_ENUM $typeId_{enum}$ '.' $nameIR_{enum_field}$.

   Let $value_{enum_field}^*$ be the resulting list.
7. For each $value_{enum_field}$ in $value_{enum_field}^*$:

   - Let $varTypeIR_{enum_field}$ be $_{EMPTY}$ tableMetadataEnumTypeIR LCTK $value_{enum_field}$.

   Let $varTypeIR_{enum_field}^*$ be the resulting list.
8. Let TC_1 be TC_0 where each of $(typeId_{enum} ++ "." ++ nameIR_{enum_field})^*$ to each of $varTypeIR_{enum_field}^*$ are added to the BLOCK layer.
9. Let $typeId_{struct}$ be "apply_result(“` concatenated with nameIR concatenated with `”)".
10. Let tableMetadataStructTypeIR be TABLE_STRUCT $typeId_{struct}$ `{ HIT BOOL ';' MISS BOOL ';' ACTION_RUN tableMetadataEnumTypeIR ';' `}.
11. Let $typeIR_{table}$ be TABLE nameIR `{ tableMetadataStructTypeIR `}.
12. The result is context TC_1 and type of table $typeIR_{table}$.

17.2.6.2. Compile-time evaluation

Compile-time evaluation of annotationList TABLE typeIR nameIR `{ tablePropertyListIR `}, under context $IC_0$and store STO:

1. Let IC_{callee_0} be an instantiation context same as IC_0 up to the BLOCK layer.
2. Let IC_{frame} be IC_{callee_0} with a new local frame.
3. Let store STO_1 and $tablePropertyListIR_{inst}$ be the result of compile-time evaluation of tablePropertyListIR, under context $IC_{frame}$and store STO.
4. Let TBL be $init_table($tablePropertyListIR_{inst}$).
5. Let frame'* be IC_{frame}.LOCAL.FRAMES.

6. Check that frame'* is a non-empty list.
7. Let frame :: _* be frame'*.
8. Let $object_{table}$ be TABLE nameIR `{ frame TBL `}.
9. Let $nameIR_{object}$ be $name_annotation_default(annotationList, nameIR).
10. Let objectId be IC_0.PATH concatenated with $nameIR_{object}$.
11. Let STO_2 be STO_1 where objectId to $object_{table}$ is added.
12. Try:
    a. If $name_annotation(annotationList) is an empty list:
        i. Let IC_1 be IC_0 where nameIR to REF objectId is added to the BLOCK layer.
        ii. Let constantDeclarationIR be $_{EMPTY}$ CONST typeIR nameIR '=' $_{VALUE}$ REF objectId ';'.
        iii. The result is context IC_1, store STO_2, and constantDeclarationIR.
13. Then, try:
    a. Let nameIR'* be $name_annotation(annotationList).
    b. Check that nameIR'* is a list of length 1.
    c. Let $nameIR_{annotation}$ be nameIR'*.
    d. Let $objectId_{annotation}$ be IC_0.PATH concatenated with $nameIR_{annotation}$.
    e. Let STO_3 be STO_2 where $objectId_{annotation}$ to $object_{table}$ is added.
    f. Let IC_1 be IC_0 where nameIR to REF objectId is added to the BLOCK layer.
    g. Let constantDeclarationIR be $_{EMPTY}$ CONST typeIR nameIR '=' $_{VALUE}$ REF objectId ';'.
    h. The result is context IC_1, store STO_3, and constantDeclarationIR.

17.3. Match-action tables

tables are the primary mechanism for implementing match-action behavior in P4 programs. A table performs a lookup based on a key computed from the packet header fields and other metadata, and applies an action associated with the matching entry in the table.

A table is declared with standard properties and target-specific custom properties.

```
tableDeclaration
    : annotationList TABLE name { tablePropertyList }
    ;

tablePropertyList
    : /* empty */ | tablePropertyList tableProperty
    ;

tableProperty
    : KEY = { tableKeyList } | ACTIONS = { tableActionList }
    | annotationList constOpt ENTRIES = { tableEntryList }
    | annotationList constOpt tableCustomName initializer ;
    ;
```

The standard table properties include:

- key: An expression that describes how the key used for look-up is computed.
- actions: A list of all actions that may be found in the table.

In addition, the tables may optionally define the following properties,

- default_action: an action to execute when the lookup in the lookup table fails to find a match for the key used.
- size: an integer specifying the desired size of the table.
- entries: entries that are initially added to a table when the P4 program is loaded, some or all of which may be unchangeable by the control plane software.
- largest_priority_wins: Only useful for some tables with the entries property. See [sec-table-entries] for details.
- priority_delta: Only useful for some tables with the entries property. See [sec-table-entries] for details.

The compiler must set the default_action to NoAction (and also insert it into the list of actions) for tables that do not define the default_action property. Hence, all tables can be thought of as having a default_action property, either implicitly or explicitly.

In addition, tables may contain architecture-specific properties (see [sec-additional-table-properties]).

A property marked as const cannot be changed dynamically by the control plane. The key, actions, and size properties cannot be modified so the const keyword is not needed for these.

17.3.1. Semantics of match-action table properties

17.3.1.1. Type checking

Table properties are type checked with the relation:

TableProperty_ok:

- Typing tableProperty, under context TC and table context TBLC:
- The result is table context TBLC and tablePropertyIR.

TableProperty_ok dispatch:

1. If let KEY '=' `{ tableKeyList `} be tableProperty: goto key
2. Else if let ACTIONS '=' `{ tableActionList `} be tableProperty: goto actions
3. Else if let annotationList constOpt ENTRIES '=' `{ tableEntryList `} be tableProperty: goto entries
4. Else:
    a. Try:
        i. Let annotationList constOpt tableCustomName initializer ';' be tableProperty.
        ii. Goto default-action
    b. Then, try:
        i. Let annotationList constOpt tableCustomName '=' expression ';' be tableProperty.
        ii. Try:
            A. Goto priority
        iii. Then, try:

A. Goto size-and-others

A list of table properties are type checked as:

TableProperties_ok:

- Typing tableProperty*, under context TC and table context TBLC:
- The result is table context TBLC and tablePropertyListIR.

Typing tableProperty*, under context TC and table context TBLC:

1. If tableProperty* matches pattern []:
    a. The result is table context TBLC and ·.
2. Else:
    a. Let tableProperty_h :: tableProperty_t^* be tableProperty*.
    b. Let table context TBLC_1 and tablePropertyIR_h be the result of typing tableProperty_h, under context TC and table context TBLC.
    c. Let table context TBLC_2 and $\text{tablePropertyIR}_t^*$ be the result of typing tableProperty_t^*, under context TC and table context TBLC_1.
    d. The result is table context TBLC_2 and tablePropertyIR_h :: $\text{tablePropertyIR}_t^*$.

17.3.1.2. Compile-time evaluation

Table properties are compile-time evaluated with the relation:

TableProperty_inst:

- Compile-time evaluation of tablePropertyIR, under context ICand store STO:
- The result is store STO and tablePropertyIR.

TableProperty_inst dispatch:

1. If let tableKeysPropertyIR of type tableKeysPropertyIR be tablePropertyIR: goto tableKeysPropertyIR
2. Else if let ACTIONS '=' `{ tableActionListIR `} of type tableActionsPropertyIR be tablePropertyIR: goto tableActionsPropertyIR
3. Else if let tableDefaultActionPropertyIR of type tableDefaultActionPropertyIR be tablePropertyIR: goto tableDefaultActionPropertyIR
4. Else if let tableEntriesPropertyIR of type tableEntriesPropertyIR be tablePropertyIR: goto tableEntriesPropertyIR
5. Else:

a. Let tableCustomPropertyIR be tablePropertyIR.

b. Goto tableCustomPropertyIR

After type checking, table properties are represented in $P4_{IR}$ as:

```
tablePropertyListIR = tablePropertyIR*

tablePropertyIR
    : tableKeysPropertyIR | tableActionsPropertyIR
    | tableDefaultActionPropertyIR | tableEntriesPropertyIR
    | tableCustomPropertyIR
    ;
```

A list of table properties are compile-time evaluated as:

TableProperties_inst:

- Compile-time evaluation of tablePropertyIR*, under context ICand store STO:
- The result is store STO and tablePropertyIR*.

Compile-time evaluation of tablePropertyIR*, under context ICand store STO:

1. If tablePropertyIR* matches pattern []:
   a. The result is store STO and ·.
2. Else:
   a. Let $\text{tablePropertyIR}_h :: \text{tablePropertyIR}_t^*$ be tablePropertyIR*.
   b. Let store STO_1 and $\text{tablePropertyIR}_{h_inst}$ be the result of compile-time evaluation of tablePropertyIR_h, under context ICand store STO.
   c. Let store STO_2 and $\text{tablePropertyIR}_{t_inst}^*$ be the result of compile-time evaluation of $\text{tablePropertyIR}_t^*$, under context ICand store STO_1.
   d. The result is store STO_2 and $\text{tablePropertyIR}_{h_inst} :: \text{tablePropertyIR}_{t_inst}^*$.

During compile-time evaluation, local instances within table properties are instantiated.

The following subsections describe the standard properties of tables in more detail.

17.3.2. Table keys

The key is a table property which specifies the data-plane values that should be used to look up an entry. A key is a list of pairs of the form (e : m), where e is an expression that describes the data to be matched in the table, and m is a match_kind that describes the algorithm used to perform the lookup (see Section Section 8.2.5).

```
<span id="tableKey"></span>
```

```
tableKeyList
    : /* empty */ | tableKeyList tableKey
    ;

tableKey
    : expression : name annotationList ;
    ;
```

For example, consider the following program fragment:

```
table Fwd {
    key = {
       ipv4header.dstAddress : ternary;
       ipv4header.version    : exact;
    }
    // more fields omitted
}
```

Here the key comprises two fields from the ipv4header header: dstAddress and version. The match_kind elements serve three purposes:

- They specify the algorithm used to match data-plane values against the entries in the table at runtime.
- They are used to synthesize the control-plane API that is used to populate the table.
- They are used by the compiler back-end to allocate resources for the implementation of the table.

The P4 core library contains three predefined match_kind identifiers:

```
match_kind {
   exact,
   ternary,
   lpm
}
```

These identifiers correspond to the $P4_{14}$ match kinds with the same names. The semantics of these match kinds is actually not needed to describe the behavior of the P4 abstract machine; how they are used influences only the control-plane API and the implementation of the look-up table. From the point of view of the P4 program, a look-up table is an abstract finite map that is given a key and produces as a result either an action or a "miss" indication, as described in Section 17.3.7.

The expected meaning of these values is as follows:

- an exact match kind on a key field means that the value of the field in the table specifies exactly the value the lookup key field must have in order to match. This is applicable for all legal key fields whose types support equality comparisons.
- a ternary match kind on a key field means that the field in the table specifies a set of values for the key field using a value and a mask. The meaning of the (value, mask) pair is similar to the P4 mask expressions, as described in Section 8.6.5.1.3: a key field k matches the table entry when k & mask == value & mask.
- a lpm (longest prefix match) match kind on a key field is a specific type of ternary match where the mask is required to have a form in binary that is a contiguous set of 1 bits followed by a contiguous set of 0 bits. Masks with more 1 bits have automatically higher priorities. A mask with all bits 0 is legal.

Some table entries, in particular the ones with at least one ternary field, also require a priority value. A

priority is a numeric value which is used to break ties when a particular key belongs to multiple sets. When table entries are specified in the P4 program the priorities are generated by the compiler; when entries are specified by the control-plane, the priority may need to be explicitly specified. Entries with higher priority are matched first. This specification does not mandate whether "higher" priorities are represented by higher or lower numeric values; this choice is left to the target implementation.

An example specifying entries for a table is given in [sec-table-entries].

If a table has no key property, or if the value of its key property is the empty tuple, i.e. key = {}, then it contains no look-up table, just a default action—i.e., the associated lookup table is always the empty map.

Each key element can have an optional @name annotation which is used to synthesize the control-plane-visible name for the key field.

Note some implementations might only support a limited number of keys or a limited combinations of match_kind for the keys. The implementation should reject those cases with an error message in this case.

17.3.2.1. Type checking

A table key property is type checked as follows:

Typing KEY '=' `{ tableKeyList `}, under context TC and table context $TBLC_0$:

1. Let tableKey* be tableKeyList flattened.
2. Let table context $TBLC_1$ and tableKeyIR* be the result of typing tableKey*, under context TC and table context $TBLC_0$.
3. Let $TBLC_2$ be $TBLC_1$ with match mode updated based on keys tableKeyIR*.
4. The result is table context $TBLC_2$ and KEY '=' `{ tableKeyIR* `}.

After type checking, a table key property is represented in $P4_{IR}$ as:

```
tableKeysPropertyIR
    : KEY = { tableKeyListIR }
    ;

tableKeyListIR = tableKeyIR*

tableKeyIR
    : typedExpressionIR # nameIR : nameIR annotationList ;
    ;
```

Table keys are type checked with the following relation:

TableKeys_ok:

- Typing tableKey*, under context TC and table context TBLC:
- The result is table context TBLC and tableKeyListIR.

Typing tableKey*, under context TC and table context TBLC:

1. If tableKey* matches pattern []:
    a. The result is table context TBLC and ·.
2. Else:
    a. Let $tableKey_h$ :: $tableKey_t^*$ be tableKey*.
    b. Let table context $TBLC_1$ and $tableKeyIR_h$ be the result of typing $tableKey_h$, under context TC and table context TBLC.
    c. Let table context $TBLC_2$ and $tableKeyIR_t^*$ be the result of typing $tableKey_t^*$, under context TC and table context $TBLC_1$.
    d. The result is table context $TBLC_2$ and $tableKeyIR_h$ :: $tableKeyIR_t^*$.

The relation invokes the following relation for each key:

TableKey_ok:

- Typing tableKey, under context TC and table context TBLC:
- The result is table context TBLC and tableKeyIR.

Typing expression ':' $name_{matchkind}$ annotationList ';', under context TC and table context $TBLC_0$:

1. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
2. Let typeIR be the type of typedExpressionIR.
3. Let bound be bound type variables from the LOCAL layer of TC.
4. If SET `< typeIR `> is a well-formed type, with bound type variables bound:
    a. Let text be expression as text.
    b. Let $nameIR_{default}$ be the text text with all whitespace removed.
    c. Let $nameIR_{key}$ be $name_annotation_default(annotationList, $nameIR_{default}$).
    d. Let $nameIR_{matchkind}$ be the name of $name_{matchkind}$.
    e. Check that MATCH_KIND '.' $nameIR_{matchkind}$ is equal to the value of variable $_{BARE}$ $nameIR_{matchkind}$ from the LOCAL layer of TC.
    f. Check that typeIR can be used with match kind $nameIR_{matchkind}$.
    g. Let $TBLC_1$ be $TBLC_0$ with match key $nameIR_{matchkind}$ of type typeIR added.
    h. Let tableKeyIR be typedExpressionIR '#' $nameIR_{key}$ ':' $nameIR_{matchkind}$ annotationList ';'.
    i. The result is table context $TBLC_1$ and tableKeyIR.

17.3.2.2. Compile-time evaluation

Compile-time evaluation of tableKeysPropertyIR, under context ICand store STO:

1. The result is store STO and tableKeysPropertyIR.

17.3.2.3. Runtime evaluation

A table key is evaluated at runtime as follows:

TableKey_eval:

- Evaluating tableKeyIR under context EC and state ARCH:
- The result is context EC, state ARCH , and result tableKeyResult.

The result of evaluating a table key is:

```
tableKeyResult
    : continueResult<tableKeyValue> | exitResult
    ;

tableKeyValue
    : value : nameIR
    ;

exitResult
    : EXIT
    ;
```

Evaluating typedExpressionIR '#' nameIR' ':' nameIR annotationList ';' under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$ and expressionResult be the result of evaluating typedExpressionIR, under context EC_0 and state $ARCH_0$ at LOCAL.
2. If let exitResult of type exitResult be expressionResult:
    a. The result is context EC_1, state $ARCH_1$, and result EXIT.
3. Else if let $_{CONT}$ value of type continueResult<value> be expressionResult:
    a. Let tableKeyValue be value ':' nameIR.
    b. The result is context EC_1, state $ARCH_1$, and result $_{CONT}$ tableKeyValue.

A list of table keys is evaluated at runtime as follows:

TableKeys_eval:

- Evaluating tableKeyListIR under context EC and state ARCH:
- The result is context EC, state ARCH , and result tableKeysResult.

The result of evaluating table keys is:

```
tableKeysResult
    : continueResult<tableKeyValue*> | exitResult
    ;
```

Evaluating tableKeyIR* under context EC and state ARCH:

1. If tableKeyIR* matches pattern []:
    a. The result is context EC, state ARCH , and result $_{\text{CONT}}\cdot$.
2. Else:
    a. Let $\text{tableKeyIR}_h :: \text{tableKeyIR}_t^*$ be tableKeyIR*.
    b. Let context EC_1, state ARCH_1 , and result tableKeyResult be the result of evaluating tableKeyIR_h under context EC and state ARCH.
    c. If let exitResult of type exitResult be tableKeyResult:
        i. The result is context EC_1, state ARCH_1 , and result EXIT.
    d. Else:
        i. Let $_{\text{CONT}}\,\text{tableKeyValue}_h$ be tableKeyResult.
        ii. Let context EC_2, state ARCH_2 , and result tableKeysResult be the result of evaluating tableKeyIR_t^* under context EC_1 and state ARCH_1.
        iii. If let exitResult of type exitResult be tableKeysResult:
            A. The result is context EC_2, state ARCH_2 , and result EXIT.
        iv. Else:
            A. Let $_{\text{CONT}}\,\text{tableKeyValue}_t^*$ be tableKeysResult.
            B. Let tableKeyValue^* be $\text{tableKeyValue}_h :: \text{tableKeyValue}_t^*$.
            C. The result is context EC_2, state ARCH_2 , and result $_{\text{CONT}}\,\text{tableKeyValue}^*$.

17.3.3. Table actions

A table must declare all possible actions that may appear within the associated lookup table or in the default action. This is done with the actions property; the value of this property is always an actionList:

```
tableActionList
```

```
    : /* empty */ | tableActionList tableAction
    ;

tableAction
    : annotationList tableActionReference ;
    ;

tableActionReference
    : prefixedNonTypeName | prefixedNonTypeName ( argumentList )
    ;
```

To illustrate, recall the example Very Simple Switch program in Section 5.3:

```
action Drop_action() {
  outCtrl.outputPort = DROP_PORT;
}

action Rewrite_smac(EthernetAddress sourceMac) {
  headers.ethernet.srcAddr = sourceMac;
}

table smac {
    key = { outCtrl.outputPort : exact; }
    actions = {
        Drop_action;
        Rewrite_smac;
    }
}
```

- The entries in the smac table may contain two different actions: Drop_action and Rewrite_mac.
- The Rewrite_smac action has one parameter, sourceMac, which in this case will be provided by the control plane.

Each action in the list of actions for a table must have a distinct name—e.g., the following program fragment is illegal:

```
action a() {}
control c() {
    action a() {}
    // Illegal table: two actions with the same name
    table t { actions = { a; .a; } }
}
```

Each action parameter that has a direction (in, inout, or out) must be bound in the actions list specification; conversely, no directionless parameters may be bound in the list. The expressions supplied as arguments to an action are not evaluated until the action is invoked. Applying tables, whether directly via an expression like table1.apply().hit, or indirectly, are forbidden in the expressions supplied as action arguments.

```
action a(in bit<32> x) { /* body omitted */ }
bit<32> z;
action b(inout bit<32> x, bit<8> data) { /* body omitted */ }
table t {
    actions = {
       // a; -- illegal, x parameter must be bound
       a(5);  // binding a's parameter x to 5
       b(z);  // binding b's parameter x to z
       // b(z, 3); -- illegal, cannot bind directionless data parameter
       // b(); -- illegal, x parameter must be bound
       // a(table2.apply().hit ? 5 : 3); -- illegal, cannot apply a table here
    }
```

```
}
```

17.3.3.1. Type checking

A table action property is type checked as follows:

Typing ACTIONS '=' `{ tableActionList `}, under context TC and table context $TBLC_0$:

1. Let tableAction* be tableActionList flattened.
2. Let table context $TBLC_1$ and tableActionIR* be the result of typing tableAction*, under context TC and table context $TBLC_0$.
3. The result is table context $TBLC_1$ and ACTIONS '=' `{ tableActionIR* `}.

After type checking, a table action property is represented in $P4_{IR}$ as:

```
tableActionsPropertyIR
    : ACTIONS = { tableActionListIR }
    ;

tableActionListIR = tableActionIR*

tableActionIR
    : annotationList tableActionReferenceIR #
        ( parameterListIR , parameterListIR ) ;
    ;

tableActionReferenceIR
    : prefixedNameIR controlPlaneNameIR ( argumentListIR )
    ;
```

Table actions are type checked with the following relation:

TableActions_ok:

- Typing tableAction*, under context TC and table context TBLC:
- The result is table context TBLC and tableActionListIR.

Typing tableAction*, under context TC and table context TBLC:

1. If tableAction* matches pattern []:
   a. The result is table context TBLC and ·.

2. Else:
    a. Let $tableAction_h :: tableAction_t^*$ be $tableAction^*$.
    b. Let table context $TBLC_1$ and $tableActionIR_h$ be the result of typing $tableAction_h$, under context TC and table context TBLC.
    c. Let table context $TBLC_2$ and $tableActionIR_t^*$ be the result of typing $tableAction_t^*$, under context TC and table context $TBLC_1$.
    d. The result is table context $TBLC_2$ and $tableActionIR_h :: tableActionIR_t^*$.

The relation invokes the following relation for each action:

TableAction_ok:

- Typing tableAction, under context TC and table context TBLC:
- The result is table context TBLC and tableActionIR.

Typing annotationList tableActionReference ';', under context TC and table context $TBLC_0$:

1. If let prefixedNonTypeName of type prefixedNonTypeName be tableActionReference:
    a. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
    b. Let $(callableId, callableTypeDefIR)^?$ be the non-overloaded callable definition of prefixedNameIR from the LOCAL layer of TC.
    c. Check that $(callableId, callableTypeDefIR)^?$ is defined.
    d. Let $(_, callableTypeDefIR)$ be $(callableId, callableTypeDefIR)^?$.
    e. Check that callableTypeDefIR has type actionTypeIR.
    f. Let $annotationList_{action}$ ACTION _ `( $parameterIR^*$ `) be callableTypeDefIR.
    g. Let $annotation_{name}^?$ be $find_name_annotation_opt(annotationList_{action})$.
    h. Let $annotationList_{update}$ be $add_annotationList(annotationList, annotation_{name}^?)$.
    i. Let parameters $parameterIR_{data}^*$ for data-plane, $parameterIR_{control}^*$ for control-plane, and casted $argumentIR^*$ be the result of typing partial action call on $parameterIR^*$ with ·, under context TC.
    j. Check that $argumentIR^*$ is an empty list.
    k. Let $TBLC_1$ be $TBLC_0$ with match action prefixedNameIR `( $parameterIR^*$ `) applied using · added.
    l. Let tableActionIR be $annotationList_{update}$ prefixedNameIR · `( · `) '#' `( $parameterIR_{data}^*$ ',' $parameterIR_{control}^*$ `) ';'.
    m. The result is table context $TBLC_1$ and tableActionIR.
2. Else:
    a. Let prefixedNonTypeName `( argumentList `) be tableActionReference.
    b. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
    c. Let $(callableId, callableTypeDefIR)^?$ be the non-overloaded callable definition of prefixedNameIR from the LOCAL layer of TC.
    d. Check that $(callableId, callableTypeDefIR)^?$ is defined.
    e. Let $(_, callableTypeDefIR)$ be $(callableId, callableTypeDefIR)^?$.

f. Check that callableTypeDefIR has type actionTypeIR.

g. Let $annotationList_{action}$ ACTION _ `( parameterIR* `) be callableTypeDefIR.

h. Let $annotation_{name}^{?}$ be $find_name_annotation_opt(annotationList_{action}).

i. Let $annotationList_{update}$ be $add_annotationList(annotationList, $annotation_{name}^{?}$).

j. Let argumentIR* be the result of typing argumentList, under context TC at LOCAL.

k. Let parameters $parameterIR_{data}^{*}$ for data-plane, $parameterIR_{control}^{*}$ for control-plane, and casted $argumentIR_{cast}^{*}$ be the result of typing partial action call on parameterIR* with argumentIR*, under context TC.

l. Let $TBLC_1$ be $TBLC_0$ with match action prefixedNameIR `( parameterIR* `) applied using $argumentIR_{cast}^{*}$ added.

m. Let tableActionIR be $annotationList_{update}$ prefixedNameIR · `( $argumentIR_{cast}^{*}$ `) '#' `( $parameterIR_{data}^{*}$ ',' $parameterIR_{control}^{*}$ `) ';'.

n. The result is table context $TBLC_1$ and tableActionIR.

To check that each action conforms to the calling convention, the following relation is used:

Call_action_partial_ok:

- Typing partial action call on parameterIR* with argumentListIR, under context TC:
- The result is parameters parameterIR* for data-plane, parameterIR'* for control-plane, and casted argumentListIR.

Typing partial action call on parameterIR* with argumentIR*, under context TC:

1. Let ($parameterIR_{data}^{*}$, $parameterIR_{control}^{*}$) be parameterIR* split into data-plane and control-plane parameters.
2. Check that the length of $parameterIR_{data}^{*}$ is equal to the length of argumentIR*.
3. Let $typedExpressionIR_{sub}^{*}$ be the sub-expressions of argumentIR*.
4. Check that all $_{$is_table_application(typedExpressionIR}sub~)^{*}$ are true.
5. Let casted $argumentIR_{cast}^{*}$ be the result of checking calling convention of argumentIR* applied on $parameterIR_{data}^{*}$, under context TC and action context ACTION at LOCAL.
6. The result is parameters $parameterIR_{data}^{*}$ for data-plane, $parameterIR_{control}^{*}$ for control-plane, and casted $argumentIR_{cast}^{*}$.

17.3.3.2. Compile-time evaluation

Compile-time evaluation of ACTIONS '=' `{ tableActionListIR `}, under context ICand store STO:

1. Let store STO_1 and $tableActionListIR_{inst}$ be the result of compile-time evaluation of tableActionListIR, under context ICand store STO.
2. Let tableActionsPropertyIR be ACTIONS '=' `{ $tableActionListIR_{inst}$ `}.
3.

The result is store STO_1 and tableActionsPropertyIR.

Table actions are evaluated at compile time as follows:

TableActions_inst:

- Compile-time evaluation of $tableActionIR^*$, under context ICand store STO:
- The result is store STO and $tableActionIR^*$.

Compile-time evaluation of $tableActionIR^*$, under context ICand store STO:

1. If $tableActionIR^*$ matches pattern []:
   a. The result is store STO and ·.
2. Else:
   a. Let $tableActionIR_h$ :: $tableActionIR_t^*$ be $tableActionIR^*$.
   b. Let store STO_1 and $tableActionIR_{h_inst}$ be the result of compile-time evaluation of $tableActionIR_h$, under context ICand store STO.
   c. Let store STO_2 and $tableActionIR_{t_inst}^*$ be the result of compile-time evaluation of $tableActionIR_t^*$, under context ICand store STO_1.
   d. The result is store STO_1 and $tableActionIR_{h_inst}$ :: $tableActionIR_{t_inst}^*$.

The relation invokes the following relation for each action:

TableAction_inst:

- Compile-time evaluation of tableActionIR, under context ICand store STO:
- The result is store STO and tableActionIR.

Compile-time evaluation of annotationList tableActionReferenceIR '#' `( parameterListIR ',' $parameterListIR_{control}$ `) ';', under context ICand store STO:

1. Let prefixedNameIR _ `( argumentListIR `) be tableActionReferenceIR.
2. Let $(cursor, callableId, callableDef)^?$ be the non-overloaded callable definition of prefixedNameIR from the LOCAL layer of IC.
3. Check that $(cursor, callableId, callableDef)^?$ is defined.
4. Let (cursor, _, callableDef) be $(cursor, callableId, callableDef)^?$.
5.

Check that callableDef has type actionDef.

6. Let actionDef be callableDef.
7. Let annotationList$_{\text{actionDef}}$ ACTION nameIR `( _ `) _ be actionDef.
8. Try:
   a. Let nameIR'* be $name_annotation(annotationList$_{\text{actionDef}}$).
   b. If nameIR'* is a list of length 1:
      i. Let nameIR$_{\text{anno}}$ be nameIR'*.
      ii. If the text nameIR$_{\text{anno}}$ starts with ".":
         A. Let text be the text nameIR$_{\text{anno}}$ with the prefix "." removed.
         B. Let controlPlaneNameIR be text.
         C. Let tableActionReferenceIR' be prefixedNameIR controlPlaneNameIR `( argumentListIR `).
         D. Let tableActionIR be annotationList tableActionReferenceIR' '#' `( parameterListIR ',' parameterListIR$_{\text{control}}$ `) ';'.
         E. The result is store STO and tableActionIR.
      iii. Else:
         A. Let text' be the result of joining IC.PATH concatenated with nameIR$_{\text{anno}}$ with separator ".".
         B. Let controlPlaneNameIR be nameIR$_{\text{anno}}$ if cursor is equal to GLOBAL otherwise text'.
         C. Let tableActionReferenceIR' be prefixedNameIR controlPlaneNameIR `( argumentListIR `).
         D. Let tableActionIR be annotationList tableActionReferenceIR' '#' `( parameterListIR ',' parameterListIR$_{\text{control}}$ `) ';'.
         E. The result is store STO and tableActionIR.
9. Then, try:
   a. Check that $name_annotation(annotationList$_{\text{actionDef}}$) is an empty list.
   b. Let text'' be the result of joining IC.PATH concatenated with nameIR with separator ".".
   c. Let controlPlaneNameIR be nameIR if cursor is equal to GLOBAL otherwise text''.
   d. Let tableActionReferenceIR' be prefixedNameIR controlPlaneNameIR `( argumentListIR `).
   e. Let tableActionIR be annotationList tableActionReferenceIR' '#' `( parameterListIR ',' parameterListIR$_{\text{control}}$ `) ';'.
   f. The result is store STO and tableActionIR.

17.3.4. Table default actions

The default action for a table is an action that is invoked automatically by the match-action unit whenever the lookup table does not find a match for the supplied key.

If present, the default_action property *must* appear after the action property. It may be declared as const, indicating that it cannot be changed dynamically by the control-plane. The default action *must* be one of the actions that appear in the actions list. In particular, the expressions passed as in, out, or inout parameters must be syntactically identical to the expressions used in one of the elements of the actions list.

For example, in the above table we could set the default action as follows (marking it also as constant):

```
const default_action = Rewrite_smac(48w0xAA_BB_CC_DD_EE_FF);
```

Note that the specified default action must supply arguments for the control-plane-bound parameters

(i.e., the directionless parameters), since the action is synthesized at compilation time. The expressions supplied as arguments for parameters with a direction (in, inout, or out) are evaluated when the action is invoked while the expressions supplied as arguments for directionless parameters are evaluated at compile time.

Continuing the example from the previous section, the following are several legal and illegal specifications of default actions for the table t:

```
default_action = a(5); // OK - no control-plane parameters
// default_action = a(z); -- illegal, a's x parameter is already bound to 5
default_action = b(z,8w8); // OK - bind b's data parameter to 8w8
// default_action = b(z); -- illegal, b's data parameter is not bound
// default_action = b(x, 3); -- illegal: x parameter of b bound to x instead of z
```

17.3.4.1. Type checking

A table default action property is type checked as follows:

Typing annotationList constOpt tableCustomName initializer ';', under context TC and table context $TBLC_0$:

1. If "default_action" is equal to the name of tableCustomName:
   a. Let tableActionReferenceIR be the result of typing initializer, under context TC and table context $TBLC_0$.
   b. Let constOptIR be the optional const qualifier in constOpt.
   c. Let tablePropertyIR be annotationList constOptIR DEFAULT_ACTION '=' tableActionReferenceIR ';'.
   d. The result is table context $TBLC_0$ and tablePropertyIR.

After type checking, a table default action property is represented in $P4_{IR}$ as:

```
tableDefaultActionPropertyIR
    : annotationList constOptIR DEFAULT_ACTION = tableActionReferenceIR ;
    ;

tableActionReferenceIR
    : prefixedNameIR controlPlaneNameIR ( argumentListIR )
    ;
```

Table default actions are type checked with the following relation:

TableDefaultAction_ok:

- Typing initializer, under context TC and table context TBLC:
- The result is tableActionReferenceIR.

Typing '=' expression, under context TC and table context TBLC:

1. If let prefixedNonTypeName of type prefixedNonTypeName be expression:
   a. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
   b. Check that (·, ·) is equal to $find_action(TBLC, prefixedNameIR).
   c. The result is prefixedNameIR · `( · `).
2. Else if let callExpression of type callExpression be expression:
   a. Check that callExpression matches pattern % `( % `).
   b. Let callTarget `( argumentList `) be callExpression.
   c. Check that callTarget has type prefixedNonTypeName.
   d. Let prefixedNonTypeName be callTarget.
   e. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
   f. Let (parameterIR*, argumentListIR)$^?$ be $find_action(TBLC, prefixedNameIR).
   g. Check that (parameterIR*, argumentListIR)$^?$ is defined.
   h. Let ($parameterIR_{action}{}^*$, $argumentIR_{action}{}^*$) be (parameterIR*, argumentListIR)$^?$.
   i. Let $argumentIR^*$ be the result of typing argumentList, under context TC at LOCAL.
   j. Let parameters $parameterIR_{action_data}{}^*$ for data-plane, $parameterIR_{action_control}{}^*$ for control-plane,and casted $argumentIR_{cast}{}^*$ be the result of typing action call on $parameterIR_{action}{}^*$ with $argumentIR^*$, under context TC.
   k. Let $argumentIR_{action_data}{}^*$ be $argumentIR_{action}{}^*$[0 : the length of $parameterIR_{action_data}{}^*$].
   l. Let $argumentIR_{cast_data}{}^*$ be $argumentIR_{cast}{}^*$[0 : the length of $parameterIR_{action_data}{}^*$].
   m. Check that the length of $argumentIR_{action_data}{}^*$ is equal to the length of $argumentIR_{cast_data}{}^*$.
   n. Check that $argumentIR_{action_data}$ is equal to $argumentIR_{cast_data}$, for all $argumentIR_{action_data}$ in $argumentIR_{action_data}{}^*$ and $argumentIR_{cast_data}$ in $argumentIR_{cast_data}{}^*$.
   o. The result is prefixedNameIR · `( $argumentIR_{cast}{}^*$ `).

To check that a default action conforms to the calling convention, the following relation is used:

Call_action_default_ok:

- Typing action call on $parameterIR^*$ with argumentListIR, under context TC:
- The result is parameters $parameterIR^*$ for data-plane, $parameterIR'^*$ for control-plane,and casted argumentListIR.

Typing action call on $parameterIR^*$ with $argumentIR^*$, under context TC:

1. Let ($parameterIR_{data}{}^*$, $parameterIR_{control}{}^*$) be $parameterIR^*$ split into data-plane and control-plane parameters.
2. Let $typedExpressionIR_{sub}{}^*$ be the sub-expressions of $argumentIR^*$.
3. Check that all $_{\$is_table_application(typedExpressionIR}sub\sim)^*$ are true.
4. Let casted $argumentIR_{cast}{}^*$ be the result of checking calling convention of $argumentIR^*$ applied on $parameterIR^*$,

under context TC and action context ACTION at LOCAL.

5. The result is parameters $\text{parameterIR}_{\text{data}}^{*}$ for data-plane, $\text{parameterIR}_{\text{control}}^{*}$ for control-plane,and casted $\text{argumentIR}_{\text{cast}}^{*}$.

17.3.4.2. Compile-time evaluation

Compile-time evaluation of tableDefaultActionPropertyIR, under context ICand store STO:

1. The result is store STO and tableDefaultActionPropertyIR.

17.3.5. Table entries

While table entries are typically installed by the control plane, tables may also be initialized at compile time with a set of entries.

Declaring these entries with const entries is useful in situations where tables are used to implement fixed algorithms—defining table entries statically enables expressing these algorithms directly in P4, which allows the compiler to infer how the table is actually used and potentially make better allocation decisions for targets with limited resources.

Declaring entries with entries (without the const qualifier) enables one to specify a mix of some immutable entries that are always in the table, and some mutable entries that the control plane is allowed to later change or remove.

Entries declared in the P4 source are installed in the table when the program is loaded onto the target. Entries cannot be specified for a table with no key (see Section 17.3.2).

Table entries are defined using the following syntax:

```
tableEntryList
    : /* empty */ | tableEntryList tableEntry
    ;

tableEntry
    : constOpt tableEntryPriority keysetExpression : tableActionReference
        annotationList ;
    | constOpt keysetExpression : tableActionReference annotationList ;
    ;

tableEntryPriority
    : PRIORITY = integerLiteral : | PRIORITY = ( expression ) :
    ;
```

Table entries defined using const entries are immutable—i.e., they can only be read by the control plane. The control plane is not allowed to remove or modify any entries defined within const entries, nor is it allowed to add entries to such a table. It is allowed for individual entries to have the const keyword before them, but this is redundant when the entries are declared using const entries.

Table entries defined using entries (without a const qualifier before it) may have const before them, or not, independently for each entry. Entries with const before them may not be modified or removed by the

control plane. Entries without const may be modified or removed by the control plane. It is permitted for the control plane to add entries to such a table (subject to table capacity limitations), unlike tables declared with const entries.

Whether the control plane is allowed to modify a table's default action at run time is determined by the table's default_action table property (see Section 17.3.4), independently of whether the control plane is allowed to modify the entries of the table.

The keysetExpression component of an entry is a tuple that must provide a field for each key in the table keys (see [sec-table-props]). The table key type must match the type of the element of the set. The tableActionReference component must be an action which appears in the table actions list (and must not have the @defaultonly annotation), with all its arguments bound.

If no entry priorities are specified in the source code, and if the runtime API requires a priority for the entries of a table—e.g. when using the P4 Runtime API, tables with at least one ternary search key field—then the entries are matched in program order, stopping at the first matching entry. Architectures should define the significance of entry order (if any) for other kinds of tables.

Because control-plane APIs cannot insert or remove entries of a table that is declared with const entries, the relative priorities of such a table's entries are determined solely by the program order of the entries. Therefore assigning numeric priorities to entries of a table that has const entries is not allowed.

Depending on the match_kind of the keys, key set expressions may define one or multiple entries. The compiler will synthesize the correct number of entries to be installed in the table. Target constraints may further restrict the ability of synthesizing entries. For example, if the number of synthesized entries exceeds the table size, the compiler implementation may choose to issue a warning or an error, depending on target capabilities.

To illustrate, consider the following example:

```
header hdr {
    bit<8>  e;
    bit<16> t;
    bit<8>  l;
    bit<8>  r;
    bit<1>  v;
}

struct Header_t {
    hdr h;
}
struct Meta_t {}

control ingress(inout Header_t h, inout Meta_t m,
                inout standard_metadata_t standard_meta) {

    action a() { standard_meta.egress_spec = 0; }
    action a_params(bit<9> x) { standard_meta.egress_spec = x; }

    table t_exact_ternary {

        key = {
            h.h.e : exact;
            h.h.t : ternary;
        }

    actions = {
            a;
            a_params;
        }

    default_action = a;
```

```
        const entries = {
            (0x01, 0x1111 &&& 0xF   ) : a_params(1);
            (0x02, 0x1181           ) : a_params(2);
            (0x03, 0x1111 &&& 0xF000) : a_params(3);
            (0x04, 0x1211 &&& 0x02F0) : a_params(4);
            (0x04, 0x1311 &&& 0x02F0) : a_params(5);
            (0x06, _                ) : a_params(6);
            _                         : a;
        }
    }

}
```

In this example we define a set of 7 entries, all of which invoke action a_params except for the final entry which invokes action a. Once the program is loaded, these entries are installed in the table in the order they are enumerated in the program.

17.3.5.1. Entry priorities

If a table has fields where their match_kinds are all exact or lpm, there is no reason to assign numeric priorities to its entries. If they are all exact, duplicate keys are not allowed, and thus every lookup key can match at most one entry, so there is no need for a tiebreaker. If there is an lpm field, the priority of the entry corresponds to the length of the prefix, i.e. if a lookup key matches multiple prefixes, the longest prefix is always the winner.

For tables with other match_kind values, e.g. at least one ternary field, in general it is possible to install multiple entries such that the same lookup key can match the key of multiple entries installed into the table at the same time. Control plane APIs such as P4Runtime API [2 - Chapter 10] and TDI [1] require control plane software to provide a numeric priority with each entry added to such a table. This enables the data plane to determine which of several matching entries is the "winner", i.e. the one entry whose action is invoked.

Unfortunately there are two commonly used, but different, ways of interpreting numeric priority values.

The P4Runtime API requires numeric priorities to be positive integers, i.e. 1 or larger, and defines that entries with larger priorities must win over entries with smaller priorities. We will call this convention largest_priority_wins.

TDI requires numeric priorities to be non-negative integers, i.e. 0 or larger, and defines that entries with smaller priorities must win over entries with larger priorities. We will call this convention smallest_priority_wins.

We wish to support either of these conventions when developers specify priorities for initial table entries in the program. Thus there is a table property largest_priority_wins. If explicitly specified for a table, its value must be boolean. If true, then the priority values use the largest_priority_wins convention. If false, then the priority values use the smallest_priority_wins convention. If the table property is not present at all, then the default convention is true, corresponding to largest_priority_wins.

We also wish to support developers that want the convenience of predictable entry priority values automatically selected by the compiler, without having to write them in the program, plus the ability to specify entry priorities explicitly, if they wish.

In some cases, developers may wish the initial priority values to have "gaps" between their values, to leave room for possible later insertion of new entries between two initial entries. They can achieve this by explicitly specifying all priority values, of course, but as a convenience we define the table property priority_delta to be a positive integer value, with a default value of 1 if not specified for a table, to use as a default difference between the priorities of consecutive entries.

There are two steps that occur at compile time for a table with the entries property involving entry

priorities:

- Determine the value of the priority of every entry in the entries list.
- Issue any errors or warnings that are appropriate for these priority values. Warnings may be suppressed via an appropriate @noWarn annotation.

These steps are performed independently for each table with the entries property, and each is described in more detail below.

In general, if the developer specifies a priority value for an entry, that is the value that will be used.

If the developer does not specify priority values for any entry, then the compiler calculates priority values for every entry as follows:

```
// For this pseudocode, table entries in the `entries` list are
// numbered 0 through n-1, 0 being the first to appear in order in the
// source code.  Their priority values are named prio[0] through
// prio[n-1].
int p = 1;
if (largest_priority_wins == true) {
    for (int j = n-1; j >= 0; j -= 1) {
        prio[j] = p;
        p += priority_delta;
    }
} else {
    for (int j = 0; j < n; j += 1) {
        prio[j] = p;
        p += priority_delta;
    }
}
```

If the developer specifies priority values for at least one entry, then in order to simplify the rules for determining priorities of entries without one in the source code, the first entry *must* have a priority value explicitly provided. The priorities of entries that do not have one in the source code (if any) are determined as follows:

```
// Same conventions here as in the previous block of pseudocode above.
// If entry j has a priority value specified in the source code,
// prio_specified[j] is true, otherwise it is false.
assert(prio_specified[0]);  // compile time error if prio_specified[0] is false
p = prio[0];
for (int j = 1; j < n; j += 1) {
    if (prio_specified[j]) {
        p = prio[j];
    } else {
        if (largest_priority_wins == true) {
            p -= priority_delta;
        } else {
            p += priority_delta;
        }
        prio[j] = p;
    }
}
```

This is the end of the first step: determining entry priorities.

The priorities determined in this way are the values used when the P4 program is first loaded into a device. Afterwards, the priorities may only change by means provided by the control plane API in use.

In the second step, the compiler issues errors for out of range priority values, and/or warnings for certain combinations of entry priorities that might be unintended by the developer, unless the developer

explicitly disables those warnings.

If any priority values are negative, or larger than the maximum supported value, that is a compile time error.

If the annotation @noWarn("duplicate_priorities") is *not* used on the entries table property, then the compiler issues a warning if any two entries for the same table have equal priority values. Both P4Runtime and TDI leave it unspecified which entry is the winner if a lookup key matches multiple keys that all have the same priority, hence a warning is useful to less experienced developers that are unfamiliar with this unspecified behavior.

If the annotation @noWarn("duplicate_priorities") is used on the entries table property, then no warnings of this type are ever issued by the compiler. Using equal priority values for multiple entries in the same table is sometimes useful in reducing the number of hardware updates required when adding entries to such a table.

If the annotation @noWarn("entries_out_of_priority_order") is *not* used on the entries table property, then the compiler issues a warning if:

- If largest_priority_wins is true for the table, and there is any pair of consecutive entries where prio[j] < prio[j+1], then a warning is issued for that pair of entries.
- If largest_priority_wins is false for the table, and there is any pair of consecutive entries where prio[j] > prio[j+1], then a warning is issued for that pair of entries.

This warning is useful to developers that want the order that entries appear in the source code to match the relative priority of entries in the target device.

If the annotation @noWarn("entries_out_of_priority_order") is used on the entries table property, then no warnings of this type are ever issued by the compiler for this table. his option is provided for developers who explicitly choose to specify entries in an order that does not match their relative priority order.

The following example is the same as the first example in [sec-entries], except for the definition of table t_exact_ternary shown below.

```
table t_exact_ternary {
    key = {
        h.h.e : exact;
        h.h.t : ternary;
    }

    actions = {
        a;
        a_params;
    }

    default_action = a;

    largest_priority_wins = false;
    priority_delta = 10;
    @noWarn("duplicate_priorities")
    entries = {
        const priority=10: (0x01, 0x1111 &&& 0xF   ) : a_params(1);
                           (0x02, 0x1181           ) : a_params(2); // priority=20
                           (0x03, 0x1000 &&& 0xF000) : a_params(3); // priority=30
        const              (0x04, 0x0210 &&& 0x02F0) : a_params(4); // priority=40
              priority=40: (0x04, 0x0010 &&& 0x02F0) : a_params(5);
                           (0x06, _                ) : a_params(6); // priority=50
    }
}
```

The entries that do not have an explicit priority specified will be assigned the priority values shown in the

comments, because priority_delta is 10, and because of those entries that do have priority values specified.

Normally this program would cause a warning about multiple entries with the same priority of 40, but those warnings will be suppressed because of the @noWarn("duplicate_priorities") annotation.

17.3.5.1.1. Type checking

After type checking, a table entry priority is represented in $P4_{IR}$ as:

```
tableEntryPriorityIR
    : PRIORITY = integerLiteral :
    ;
```

A table entry priority is type checked as follows:

TableEntry_priority_ok:

- Typing tableEntryPriority, under context TC:
- The result is tableEntryPriorityIR.

Typing tableEntryPriority, under context TC:

1. If let PRIORITY '=' integerLiteral ':' be tableEntryPriority:
    a. The result is PRIORITY '=' integerLiteral ':'.
2. Else:
    a. Let PRIORITY '=' `( expression `) ':' be tableEntryPriority.
    b. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
    c. Check that the compile-time known-ness of typedExpressionIR is LCTK.
    d. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at LOCAL.
    e. Check that value has type integerValue.
    f. Let integerValue be value.
    g. Let nat? be the natural number representation of integerValue.
    h. Check that nat? is defined.
    i. Let n be nat?.
    j. The result is PRIORITY '=' D n ':'.

Priority-related table properties (largest_priority_wins and priority_delta) are type checked as follows:

Typing annotationList constOpt tableCustomName '=' expression ';', under context TC and table context $TBLC_0$:

1. Try:
    a. If "largest_priority_wins" is equal to the name of tableCustomName:
        i. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
        ii. Let typeIR be the type of typedExpressionIR.
        iii. If the compile-time known-ness of typedExpressionIR is LCTK:
            A. If BOOL is equal to typeIR with typedefs unrolled:
                I. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at LOCAL.
                II. If let $_{B}$ b_{lw} of type boolValue be value:
                    1. Let $TBLC_1$ be $TBLC_0$ with PRIORITIES.LARGEST_WINS set to b_{lw}.
                    2. Let constOptIR be the optional const qualifier in constOpt.
                    3. Let tablePropertyIR be annotationList constOptIR CUSTOM_CONST "largest_priority_wins" '=' $_{VALUE\ B}$ b_{lw} ';'.
                    4. The result is table context $TBLC_1$ and tablePropertyIR.
2. Then, try:
    a. If "priority_delta" is equal to the name of tableCustomName:
        i. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
        ii. Let typeIR be the type of typedExpressionIR.
        iii. If the compile-time known-ness of typedExpressionIR is LCTK:
            A. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
            B. If $typeIR_{unroll}$ has type integerTypeIR and $typeIR_{unroll}$ does not have type varBitTypeIR:
                I. Let value be the result of local compile-time evaluation of typedExpressionIR, under context TC at LOCAL.
                II. If let integerValue of type integerValue be value:
                    1. Let $nat^?$ be the natural number representation of integerValue.
                    2. If $nat^?$ is defined:
                        1. Let n_{delta} be $nat^?$.
                        2. If n_{delta} is greater than 0:
                            1. Let $TBLC_1$ be $TBLC_0$ with PRIORITIES.DELTA set to n_{delta}.
                            2. Let constOptIR be the optional const qualifier in constOpt.
                            3. Let tablePropertyIR be annotationList constOptIR CUSTOM_CONST "priority_delta" '=' $_{VALUE}$ D n_{delta} ';'.
                            4. The result is table context $TBLC_1$ and tablePropertyIR.

17.3.5.2. Type checking

A table entry property is type checked as follows:

Typing annotationList constOpt ENTRIES '=' `{ tableEntryList `}, under context TC and table context $TBLC_0$:

1. Let $tableEntry^*$ be tableEntryList flattened.
2. Check that if the length of $TBLC_0$.KEYS is equal to 0, then the length of $tableEntry^*$ is equal to 0.

3. Let $TBLC_1$ be $TBLC_0$ with ENTRIES.SIZE set to the length of tableEntry*.
4. Let constOptIR be the optional const qualifier in constOpt.
5. Let $TBLC_2$ be $TBLC_1$ with ENTRIES.CONST set to constOptIR = CONST.
6. For each tableEntry in tableEntry*:
   - Let table entry state TBLS and tableEntryIR be the result of typing tableEntry, under context TC and table context $TBLC_2$.

   Let TBLS* and tableEntryIR* be the resulting lists.
7. Let $tableEntryIR_{pri}$* be tableEntryIR* with priorities set based on table context $TBLC_2$ and entry states TBLS*.
8. The result is table context $TBLC_2$ and annotationList constOptIR ENTRIES '=' `{ $tableEntryIR_{pri}$* `}.

After type checking, a table key property is represented in $P4_{IR}$ as:

```
tableEntriesPropertyIR
    : annotationList constOptIR ENTRIES = { tableEntryListIR }
    ;

tableEntryListIR = tableEntryIR*

tableEntryIR
    : constOptIR tableEntryPriorityOptIR keysetExpressionIR :
        tableActionReferenceIR annotationList ;
    ;
```

A table entry is type checked with the following relation:

TableEntry_ok:

- Typing tableEntry, under context TC and table context TBLC:
- The result is table entry state TBLS and tableEntryIR.

Typing tableEntry, under context TC and table context TBLC:

1. If let constOpt tableEntryPriority keysetExpression ':' tableActionReference annotationList ';' be tableEntry:
   a. Let table entry state TBLS and keysetExpressionIR be the result of typing keysetExpression, under context TC and table context TBLC.
   b. Let tableActionReferenceIR be the result of typing tableActionReference, under context TC and table context TBLC.
   c. Let tableEntryPriorityIR be the result of typing tableEntryPriority, under context TC.
   d. Let constOptIR be the optional const qualifier in constOpt.
   e. Let tableEntryIR be constOptIR tableEntryPriorityIR keysetExpressionIR ':' tableActionReferenceIR

annotationList ';'.

f. The result is table entry state TBLS and tableEntryIR.

2. Else:

a. Let constOpt keysetExpression ':' tableActionReference annotationList ';' be tableEntry.

b. Let table entry state TBLS and keysetExpressionIR be the result of typing keysetExpression, under context TC and table context TBLC.

c. Let tableActionReferenceIR be the result of typing tableActionReference, under context TC and table context TBLC.

d. Let constOptIR be the optional const qualifier in constOpt.

e. Let tableEntryPriorityOptIR be ·.

f. Let tableEntryIR be constOptIR tableEntryPriorityOptIR keysetExpressionIR ':' tableActionReferenceIR annotationList ';'.

g. The result is table entry state TBLS and tableEntryIR.

17.3.5.2.1. Type checking table entry keyset

A table entry keyset may be a simple keyset or a tuple keyset. During typing, the keyset elements of tuple keysets are decomposed into a sequence of simple keysets.

TableEntry_keyset_ok:

- Typing keysetExpression, under context TC and table context TBLC:
- The result is table entry state TBLS and keysetExpressionIR.

TableEntry_keyset_ok dispatch:

1. Try:

a. If let expression of type expression be keysetExpression: goto expression

2. Then, try:

a. Goto mask

3. Then, try:

a. Goto range

4. Then, try:

a. Goto default

5. Then, try:

a. Goto dontcare

6. Then, try:

a. Check that keysetExpression has type tupleKeysetExpression.

b. Let tupleKeysetExpression be keysetExpression.

c. Check that tupleKeysetExpression matches pattern `( % ',' % `).

d. Let `( $\text{simpleKeysetExpression}_h$ ',' $\text{simpleKeysetExpressionList}_t$ `) be tupleKeysetExpression.

e. Goto tuple

TableEntry_keyset_simple_ok:

- Typing matchKey against simpleKeysetExpression under context TC and table context TBLC:
- The result is table entry state TBLS and simpleKeysetExpressionIR.

TableEntry_keyset_simple_ok dispatch:

1. Try:
    a. If let expression of type expression be simpleKeysetExpression: goto expression
    b. Else if let $expression_l$ '&&&' $expression_r$ be simpleKeysetExpression: goto mask
    c. Else if simpleKeysetExpression matches pattern DEFAULT: goto default
    d. Else if simpleKeysetExpression matches pattern '_': goto dontcare
2. Then, try:
    a. Check that t is equal to "range".
    b. Check that simpleKeysetExpression matches pattern % '..' %.
    c. Let $expression_l$ '..' $expression_r$ be simpleKeysetExpression.
    d. Goto range

Expressions

Typing expression, under context TC and table context TBLC:

1. If the length of TBLC.KEYS is equal to 1:
    a. Let table entry state TBLS and simpleKeysetExpressionIR'* be the result of typing TBLC.KEYS against expression under context TC, table context TBLC, and table entry state NON_LPM.
    b. If simpleKeysetExpressionIR'* is a list of length 1:
        i. Let simpleKeysetExpressionIR be simpleKeysetExpressionIR'*.
        ii. The result is table entry state TBLS and simpleKeysetExpressionIR.

Typing $typeIR_{key}$ ':' t against expression under context TC and table context TBLC:

1. If t is equal to "lpm":
    a. Let matchMode be TBLC.MODE.
    b. If let LPM n be matchMode:
        i. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
        ii. Let typeIR be the type of typedExpressionIR.
        iii. Let ctk be the compile-time known-ness of typedExpressionIR.

iv. Let casts castResult be the result of checking if typeIR can be implicitly cast to $typeIR_{key}$.

v. If castResult matches pattern OK %:

A. Let $typeIR_{set}$ be SET `< $typeIR_{key}$ `>.

B. Let $typedExpressionIR_{set}$ be `( $typeIR_{set}$ `) typedExpressionIR annotated with a pair of type $typeIR_{set}$ and compile-time known-ness ctk.

C. The result is table entry state LPM n and $typedExpressionIR_{set}$.

2. Else:

a. If expression is a singleton list expression:

i. If let `{ sequenceOrRecordElementExpression trailingCommaOpt `} of type sequenceOrRecordExpression be expression:

A. If let $expression_{elem}$ of type expression be sequenceOrRecordElementExpression:

I. If trailingCommaOpt is $_{EMPTY}$:

1. Let typedExpressionIR be the result of typing $expression_{elem}$, under context TC at LOCAL.

2. Let typeIR be the type of typedExpressionIR.

3. Let ctk be the compile-time known-ness of typedExpressionIR.

4. Let casts castResult be the result of checking if typeIR can be implicitly cast to $typeIR_{key}$.

5. If castResult matches pattern OK %:

1. Let $typeIR_{set}$ be SET `< $typeIR_{key}$ `>.

2. Let $typedExpressionIR_{set}$ be `( $typeIR_{set}$ `) typedExpressionIR annotated with a pair of type $typeIR_{set}$ and compile-time known-ness ctk.

3. The result is table entry state NON_LPM and $typedExpressionIR_{set}$.

b. Else:

i. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.

ii. Let typeIR be the type of typedExpressionIR.

iii. Let ctk be the compile-time known-ness of typedExpressionIR.

iv. Let casts castResult be the result of checking if typeIR can be implicitly cast to $typeIR_{key}$.

v. If castResult matches pattern OK %:

A. Let $typeIR_{set}$ be SET `< $typeIR_{key}$ `>.

B. Let $typedExpressionIR_{set}$ be `( $typeIR_{set}$ `) typedExpressionIR annotated with a pair of type $typeIR_{set}$ and compile-time known-ness ctk.

C. The result is table entry state NON_LPM and $typedExpressionIR_{set}$.

Masks

```
<span id="TableEntry_keyset_ok-mask"></span>
```

Typing keysetExpression, under context TC and table context TBLC:

1. If let simpleKeysetExpression of type simpleKeysetExpression be keysetExpression:

a. If let $expression_l$ '&&&' $expression_r$ be simpleKeysetExpression:

i. If the length of TBLC.KEYS is equal to 1:

A. Let table entry state TBLS and $simpleKeysetExpressionIR'^{*}$ be the result of typing TBLC.KEYS against $expression_l$ '&&&' $expression_r$ under context TC, table context TBLC, and table entry state NON_LPM.

B. If $simpleKeysetExpressionIR'^{*}$ is a list of length 1:

I. Let simpleKeysetExpressionIR be simpleKeysetExpressionIR'*.

II. The result is table entry state TBLS and simpleKeysetExpressionIR.

2. Else:

a. Let tupleKeysetExpression be keysetExpression.

b. If let `( expression$_l$ '&&&' expression$_r$ `) be tupleKeysetExpression:

i. If the length of TBLC.KEYS is equal to 1:

A. Let table entry state TBLS and simpleKeysetExpressionIR'* be the result of typing TBLC.KEYS against expression$_l$ '&&&' expression$_r$ under context TC, table context TBLC, and table entry state NON_LPM.

B. If simpleKeysetExpressionIR'* is a list of length 1:

I. Let simpleKeysetExpressionIR be simpleKeysetExpressionIR'*.

II. The result is table entry state TBLS and `( simpleKeysetExpressionIR `).

Typing typeIR$_{key}$ ':' t against expression$_l$ '&&&' expression$_r$ under context TC and table context TBLC:

1. If t is equal to "lpm":

a. Let matchMode be TBLC.MODE.

b. If let LPM n be matchMode:

i. Let typedExpressionIR$_l$ be the result of typing expression$_l$, under context TC at LOCAL.

ii. Let typedExpressionIR$_r$ be the result of typing expression$_r$, under context TC at LOCAL.

iii. Let typedExpressionIR? be typedExpressionIR$_l$ implicitly cast to typeIR$_{key}$.

iv. If typedExpressionIR? is defined:

A. Let typedExpressionIR$_{l_cast}$ be typedExpressionIR?.

B. Let typedExpressionIR'? be typedExpressionIR$_r$ implicitly cast to typeIR$_{key}$.

C. If typedExpressionIR'? is defined:

I. Let typedExpressionIR$_{r_cast}$ be typedExpressionIR'?.

II. Let (typedExpressionIR, typedExpressionIR)? be the result of reducing serializable enums in typedExpressionIR$_{l_cast}$ and typedExpressionIR$_{r_cast}$ until $compat_mask is satisfied.

III. If (typedExpressionIR, typedExpressionIR)? is defined:

1. Let (typedExpressionIR$_{l_reduced}$, typedExpressionIR$_{r_reduced}$) be (typedExpressionIR, typedExpressionIR)?.

2. Let ctk$_{r_reduced}$ be the compile-time known-ness of typedExpressionIR$_{r_reduced}$.

3. If ctk$_{r_reduced}$ is LCTK:

1. Let value be the result of local compile-time evaluation of typedExpressionIR$_{r_reduced}$, under context TC at LOCAL.

2. If let integerValue$_r$ of type integerValue be value:

1. Let nat? be the natural number representation of integerValue$_r$.

2. If nat? is defined:

1. Let n$_r$ be nat?.

2. Let int be the bitstring of width n representing the integer n$_r$.

3. Let n$_{prefix}$ be $tableEntry_lpm_prefix(n W int).

4. The result is table entry state LPM n$_{prefix}$ and typedExpressionIR$_{l_reduced}$ '&&&'

typedExpressionIR$_{r_reduced}$.

2. Else if t is equal to "ternary":
    a. Let typedExpressionIR$_l$ be the result of typing expression$_l$, under context TC at LOCAL.
    b. Let typedExpressionIR$_r$ be the result of typing expression$_r$, under context TC at LOCAL.
    c. Let typedExpressionIR$^?$ be typedExpressionIR$_l$ implicitly cast to typeIR$_{key}$.
    d. If typedExpressionIR$^?$ is defined:
        i. Let typedExpressionIR$_{l_cast}$ be typedExpressionIR$^?$.
        ii. Let typedExpressionIR'$^?$ be typedExpressionIR$_r$ implicitly cast to typeIR$_{key}$.
        iii. If typedExpressionIR'$^?$ is defined:
            A. Let typedExpressionIR$_{r_cast}$ be typedExpressionIR'$^?$.
            B. Let (typedExpressionIR, typedExpressionIR)$^?$ be the result of reducing serializable enums in typedExpressionIR$_{l_cast}$ and typedExpressionIR$_{r_cast}$ until $compat_mask is satisfied.
            C. If (typedExpressionIR, typedExpressionIR)$^?$ is defined:
                I. Let (typedExpressionIR$_{l_reduced}$, typedExpressionIR$_{r_reduced}$) be (typedExpressionIR, typedExpressionIR)$^?$.
                II. The result is table entry state NON_LPM and typedExpressionIR$_{l_reduced}$ '&&' typedExpressionIR$_{r_reduced}$.

Ranges

Typing keysetExpression, under context TC and table context TBLC:

1. If let simpleKeysetExpression of type simpleKeysetExpression be keysetExpression:
    a. If let expression$_l$ '..' expression$_r$ be simpleKeysetExpression:
        i. If the length of TBLC.KEYS is equal to 1:
            A. Let table entry state TBLS and simpleKeysetExpressionIR'* be the result of typing TBLC.KEYS against expression$_l$ '..' expression$_r$ under context TC, table context TBLC, and table entry state NON_LPM.
            B. If simpleKeysetExpressionIR'* is a list of length 1:
                I. Let simpleKeysetExpressionIR be simpleKeysetExpressionIR'*.
                II. The result is table entry state TBLS and simpleKeysetExpressionIR.
2. Else:
    a. Let tupleKeysetExpression be keysetExpression.
    b. If let `( expression$_l$ '..' expression$_r$ `) be tupleKeysetExpression:
        i. If the length of TBLC.KEYS is equal to 1:
            A. Let table entry state TBLS and simpleKeysetExpressionIR'* be the result of typing TBLC.KEYS against expression$_l$ '..' expression$_r$ under context TC, table context TBLC, and table entry state NON_LPM.
            B. If simpleKeysetExpressionIR'* is a list of length 1:
                I. Let simpleKeysetExpressionIR be simpleKeysetExpressionIR'*.
                II. The result is table entry state TBLS and `( simpleKeysetExpressionIR `).

Typing $typeIR_{key}$ ':' "range" against $expression_l$ '..' $expression_r$ under context TC and table context TBLC:

1. Let $typedExpressionIR_l$ be the result of typing $expression_l$, under context TC at LOCAL.
2. Let $typedExpressionIR_r$ be the result of typing $expression_r$, under context TC at LOCAL.
3. Let $typedExpressionIR^?$ be $typedExpressionIR_l$ implicitly cast to $typeIR_{key}$.
4. Check that $typedExpressionIR^?$ is defined.
5. Let $typedExpressionIR_{l_cast}$ be $typedExpressionIR^?$.
6. Let $typedExpressionIR'^?$ be $typedExpressionIR_r$ implicitly cast to $typeIR_{key}$.
7. Check that $typedExpressionIR'^?$ is defined.
8. Let $typedExpressionIR_{r_cast}$ be $typedExpressionIR'^?$.
9. Let $(typedExpressionIR, typedExpressionIR)^?$ be the result of reducing serializable enums in $typedExpressionIR_{l_cast}$ and $typedExpressionIR_{r_cast}$ until $compat_range is satisfied.
10. Check that $(typedExpressionIR, typedExpressionIR)^?$ is defined.
11. Let ($typedExpressionIR_{l_reduced}$, $typedExpressionIR_{r_reduced}$) be $(typedExpressionIR, typedExpressionIR)^?$.
12. The result is table entry state NON_LPM and $typedExpressionIR_{l_reduced}$ '..' $typedExpressionIR_{r_reduced}$.

Default

`<span id="TableEntry_keyset_ok-default"></span>`

Typing keysetExpression, under context TC and table context TBLC:

1. If keysetExpression is equal to DEFAULT:
   a. Try:
      i. Let matchMode be TBLC.MODE.
      ii. If let LPM n be matchMode:
         A. Let TBLS be LPM n.
         B. The result is table entry state TBLS and DEFAULT.
   b. Then, try:
      i. If TBLC.MODE is equal to NONE or TBLC.MODE is equal to PRI:
         A. Let TBLS be NON_LPM.
         B. The result is table entry state TBLS and DEFAULT.
2. Else if keysetExpression is equal to `( DEFAULT `):
   a. Try:
      i. Let matchMode be TBLC.MODE.
      ii. If let LPM n be matchMode:
         A. Let TBLS be LPM n.
         B. The result is table entry state TBLS and `( DEFAULT `).
   b. Then, try:
      i. If TBLC.MODE is equal to NONE or TBLC.MODE is equal to PRI:
         A. Let TBLS be NON_LPM.
         B. The result is table entry state TBLS and `( DEFAULT `).

Typing $\text{typeIR}_{\text{key}}$ ':' t against DEFAULT under context TC and table context TBLC:

1. If t is equal to "lpm":
    a. Let matchMode be TBLC.MODE.
    b. If let LPM n be matchMode:
        i. The result is table entry state LPM n and DEFAULT.
2. Else:
    a. If t is not equal to "exact":
        i. The result is table entry state NON_LPM and DEFAULT.

Don't care

Typing keysetExpression, under context TC and table context TBLC:

1. If keysetExpression is equal to '_':
    a. Try:
        i. Let matchMode be TBLC.MODE.
        ii. If matchMode matches pattern LPM %:
            A. Let TBLS be LPM 0.
            B. The result is table entry state TBLS and '_'.
    b. Then, try:
        i. If TBLC.MODE is equal to NONE or TBLC.MODE is equal to PRI:
            A. Let TBLS be NON_LPM.
            B. The result is table entry state TBLS and '_'.
2. Else if keysetExpression is equal to `( '_' )`:
    a. Try:
        i. Let matchMode be TBLC.MODE.
        ii. If matchMode matches pattern LPM %:
            A. Let TBLS be LPM 0.
            B. The result is table entry state TBLS and `( '_' )`.
    b. Then, try:
        i. If TBLC.MODE is equal to NONE or TBLC.MODE is equal to PRI:
            A. Let TBLS be NON_LPM.
            B. The result is table entry state TBLS and `( '_' )`.

Typing $\text{typeIR}_{\text{key}}$ ':' t against '_' under context TC and table context TBLC:

1. If t is equal to "lpm":

a. Let matchMode be TBLC.MODE.

b. If matchMode matches pattern LPM %:

i. The result is table entry state LPM 0 and '_'.

2. Else:

a. If t is not equal to "exact":

i. The result is table entry state NON_LPM and '_'.

Tuples

Typing `( simpleKeysetExpression$_h$ ',' simpleKeysetExpressionList$_t$ `), under context TC and table context TBLC:

1. Let simpleKeysetExpression$_t^*$ be simpleKeysetExpressionList$_t$ flattened.
2. Let simpleKeysetExpression* be simpleKeysetExpression$_h$ :: simpleKeysetExpression$_t^*$.
3. Check that the length of TBLC.KEYS is equal to the length of simpleKeysetExpression*.
4. Let table entry state TBLS and simpleKeysetExpressionIR* be the result of typing TBLC.KEYS against simpleKeysetExpression* under context TC, table context TBLC, and table entry state NON_LPM.
5. The result is table entry state TBLS and `( simpleKeysetExpressionIR$^*$ `).

TableEntry_keysets_simple_ok:

- Typing matchKey* against simpleKeysetExpression* under context TC, table context TBLC, and table entry state TBLS:
- The result is table entry state TBLS and simpleKeysetExpressionIR*.

Typing matchKey* against simpleKeysetExpression* under context TC, table context TBLC, and table entry state TBLS:

1. If matchKey* matches pattern []:

a. Check that simpleKeysetExpression* is an empty list.

b. The result is table entry state TBLS and ·.

2. Else:

a. Let matchKey$_h$ :: matchKey$_t^*$ be matchKey*.

b. Check that simpleKeysetExpression* is a non-empty list.

c. Let simpleKeysetExpression$_h$ :: simpleKeysetExpression$_t^*$ be simpleKeysetExpression*.

d. Let table entry state TBLS$_1$ and simpleKeysetExpressionIR$_h$ be the result of typing matchKey$_h$ against simpleKeysetExpression$_h$ under context TC and table context TBLC.

e. Let TBLS$_2$ be $join_tableEntryState(TBLS, TBLS$_1$).

f. Let table entry state TBLS$_3$ and simpleKeysetExpressionIR$_t^*$ be the result of typing matchKey$_t^*$ against simpleKeysetExpression$_t^*$ under context TC, table context TBLC, and table entry state TBLS$_2$.

g. The result is table entry state $TBLS_3$ and $simpleKeysetExpressionIR_h :: simpleKeysetExpressionIR_t^*$.

17.3.5.2.2. Type checking table entry action reference

TableEntry_action_ok:

- Typing tableActionReference, under context TC and table context TBLC:
- The result is tableActionReferenceIR.

Typing tableActionReference, under context TC and table context TBLC:

1. If let prefixedNonTypeName of type prefixedNonTypeName be tableActionReference:
   a. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
   b. Check that (·, ·) is equal to $find_action(TBLC, prefixedNameIR).
   c. The result is prefixedNameIR · `( · `).
2. Else:
   a. Let prefixedNonTypeName `( argumentList `) be tableActionReference.
   b. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
   c. Let $(parameterIR*, argumentListIR)^?$ be $find_action(TBLC, prefixedNameIR).
   d. Check that $(parameterIR*, argumentListIR)^?$ is defined.
   e. Let $(parameterIR_{action}^*, argumentIR_{action}^*)$ be $(parameterIR*, argumentListIR)^?$.
   f. Let $argumentIR^*$ be the result of typing argumentList, under context TC at LOCAL.
   g. Let parameters $parameterIR_{action_data}^*$ for data-plane, $parameterIR_{action_control}^*$ for control-plane,and casted $argumentIR_{cast}^*$ be the result of typing action call on $parameterIR_{action}^*$ with $argumentIR^*$, under context TC.
   h. Let $argumentIR_{action_data}^*$ be $argumentIR_{action}^*[0 : the length of parameterIR_{action_data}^*]$.
   i. Let $argumentIR_{cast_data}^*$ be $argumentIR_{cast}^*[0 : the length of parameterIR_{action_data}^*]$.
   j. Check that the length of $argumentIR_{action_data}^*$ is equal to the length of $argumentIR_{cast_data}^*$.
   k. Check that $argumentIR_{action_data}$ is equal to $argumentIR_{cast_data}$, for all $argumentIR_{action_data}$ in $argumentIR_{action_data}^*$ and $argumentIR_{cast_data}$ in $argumentIR_{cast_data}^*$.
   l. The result is prefixedNameIR · `( $argumentIR_{cast}^*$ `).

17.3.5.3. Compile-time evaluation

Compile-time evaluation of tableEntriesPropertyIR, under context ICand store STO:

1. The result is store STO and tableEntriesPropertyIR.

17.3.6. Additional properties

A table declaration defines its essential control and data plane interfaces—i.e., keys and actions. However, the best way to implement a table may actually depend on the nature of the entries that will be installed at runtime (for example, tables could be dense or sparse, could be implemented as hash-tables, associative memories, tries, etc.) In addition, some architectures may support extra table properties whose semantics lies outside the scope of this specification. For example, in architectures where table resources are statically allocated, programmers may be required to define a size table property, which can be used by the compiler back-end to allocate storage resources. However, these architecture-specific properties may not change the semantics of table lookups, which always produce either a hit and an action or a miss--they can only change how those results are interpreted on the state of the data plane. This restriction is needed to ensure that it is possible to reason about the behavior of tables during compilation.

As another example, an implementation property could be used to pass additional information to the compiler back-end. The value of this property could be an instance of an extern block chosen from a suitable library of components. For example, the core functionality of the $P4_{14}$ table action_profile constructs could be implemented on architectures that support this feature using a construct such as the following:

```
extern ActionProfile {
   ActionProfile(bit<32> size); // number of distinct actions expected
}
table t {
    key = { /* body omitted */ }
    size = 1024;
    implementation = ActionProfile(32);  // constructor invocation
}
```

Here the action profile might be used to optimize for the case where the table has a large number of entries, but the actions associated with those entries are expected to range over a small number of distinct values. Introducing a layer of indirection enables sharing identical entries, which can significantly reduce the table's storage requirements.

17.3.6.1. Type checking

Additional properties, including size, is type checked as follows:

Typing annotationList constOpt tableCustomName '=' expression ';', under context TC and table context $TBLC_0$:

1. Try:
    a. If "size" is equal to the name of tableCustomName:
        i. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.
        ii. Let typeIR be the type of typedExpressionIR.
        iii. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
        iv. If $typeIR_{unroll}$ has type integerTypeIR and $typeIR_{unroll}$ does not have type varBitTypeIR:
            A. If the compile-time known-ness of typedExpressionIR does not match pattern DYN:
                I. Let constOptIR be the optional const qualifier in constOpt.
                II. Let tablePropertyIR be annotationList constOptIR CUSTOM "size" '=' typedExpressionIR ';'.
                III. The result is table context $TBLC_0$ and tablePropertyIR.
2. Then, try:

a. Let nameIR be the name of tableCustomName.

b. Check that nameIR is not equal to "default_action" and nameIR is not equal to "size" and nameIR is not equal to "largest_priority_wins" and nameIR is not equal to "priority_delta".

c. Let typedExpressionIR be the result of typing expression, under context TC at LOCAL.

d. Let constOptIR be the optional const qualifier in constOpt.

e. Let tablePropertyIR be annotationList constOptIR CUSTOM nameIR '=' typedExpressionIR ';'.

f. The result is table context $TBLC_0$ and tablePropertyIR.

After type checking, these are represented in $P4_{IR}$ as:

```
tableCustomPropertyIR
    : annotationList constOptIR CUSTOM nameIR initializerIR ;
    | annotationList constOptIR CUSTOM_CONST nameIR constantInitializerIR ;
    ;
```

17.3.6.2. Compile-time evaluation

Compile-time evaluation of tableCustomPropertyIR, under context ICand store STO:

1. If let annotationList constOptIR CUSTOM nameIR '=' typedExpressionIR ';' be tableCustomPropertyIR:
   a. Let IC_{inner} be IC with nameIR added to the path.
   b. Let store STO_1 and value be the result of compile-time evaluation of typedExpressionIR, under context IC_{inner} and store STO at LOCAL.
   c. Let $tableCustomPropertyIR_{inst}$ be annotationList constOptIR CUSTOM_CONST nameIR '=' $_{VALUE}$ value ';'.
   d. The result is store STO_1 and $tableCustomPropertyIR_{inst}$.
2. Else:
   a. Let annotationList constOptIR CUSTOM_CONST nameIR constantInitializerIR ';' be tableCustomPropertyIR.
   b. Let $tableCustomPropertyIR_{inst}$ be annotationList constOptIR CUSTOM_CONST nameIR constantInitializerIR ';'.
   c. The result is store STO and $tableCustomPropertyIR_{inst}$.

17.3.7. Match-action table invocation

The semantics of a table invocation statement:

```
t.apply();
```

is given by the following pseudocode (see also Figure 11):

```
apply_result(m) m.apply() {
    apply_result(m) result;

    var lookupKey = m.buildKey(m.key); // using key block
    action RA = m.table.lookup(lookupKey);
    if (RA == null) {      // miss in lookup table
       result.hit = false;
```

```
        RA = m.default_action;  // use default action
    }
    else {
        result.hit = true;
    }
    result.miss = !result.hit;
    result.action_run = action_type(RA);
    evaluate_and_copy_in_RA_args(RA);
    execute(RA);
    copy_out_RA_args(RA);
    return result;
}
```

The behavior of the buildKey call in the pseudocode above is to evaluate each key expression in the order they appear in the table key definition. The behavior must be the same as if the result of evaluating each key expression is assigned to a fresh temporary variable, before starting the evaluation of the following key expression. For example, this P4 table definition and apply call:

```
bit<8> f1 (in bit<8> a, inout bit<8> b) {
    b = a + 5;
    return a >> 1;
}
bit<8> x;
bit<8> y;
table t1 {
    key = {
        y & 0x7  : exact @name("masked_y");
        f1(x, y) : exact @name("f1");
        y        : exact;
    }
    // ... rest of table properties defined here, not relevant to example
}
apply {
    // assign values to x and y here, not relevant to example
    t1.apply();
}
```

is equivalent in behavior to the following table definition and apply call:

```
// same definition of f1, x, and y as before, so they are not repeated here
bit<8> tmp_1;
bit<8> tmp_2;
bit<8> tmp_3;
table t1 {
    key = {
        tmp_1 : exact @name("masked_y");
        tmp_2 : exact @name("f1");
        tmp_3 : exact @name("y");
    }
    // ... rest of table properties defined here, not relevant to example
}
apply {
    // assign values to x and y here, not relevant to example
    tmp_1 = y & 0x7;
    tmp_2 = f1(x, y);
    tmp_3 = y;
    t1.apply();
}
```

Note that the second code example above is given in order to specify the behavior of the first one. An implementation is free to choose any technique that achieves this behavior.[2]

The following algorithm describes the evaluation of a table apply method invocation:

Calling tableApplyMethodCallee `< typeArgumentListIR `> `( argumentListIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let TABLE objectId '.' APPLY `{ frame ';' TBL `} be tableApplyMethodCallee.
2. Let EC_{callee_0} be copy of EC_0 from the BLOCK layer.
3. Let EC_{callee_1} be EC_{callee_0} with LOCAL.FRAMES set to frame.
4. Let typeId'* be the reversal of objectId.
5. Check that typeId'* is a non-empty list.
6. Let typeId :: _* be typeId'*.
7. Let EC_{callee_1} $ARCH_0$ |- typeId TBL : EC_{callee_2} $ARCH_1$ tableResult.
8. If let exitResult of type exitResult be tableResult:
    a. Let EC_1 be $copy_e(BLOCK, EC_{callee_2}, EC_0).
    b. The result is context EC_1, state $ARCH_1$, and call result EXIT.
9. Else:
    a. Let returnResult be tableResult.
    b. Let EC_1 be $copy_e(BLOCK, EC_{callee_2}, EC_0).
    c. The result is context EC_1, state $ARCH_1$, and call result returnResult.

The table match-action is implemented as:

Table_eval: EC_0 $ARCH_0$ |- typeId TBL : % % %

EC_0 $ARCH_0$ |- typeId TBL : % % %:

1. Let KEY '=' `{ tableKeyListIR `} be TBL.KEYS.
2. Let context EC_1, state $ARCH_1$, and result tableKeysResult be the result of evaluating tableKeyListIR under context EC_0 and state $ARCH_0$.
3. If let exitResult of type exitResult be tableKeysResult:
    a. The result is EC_1, $ARCH_1$, and EXIT.
4. Else:
    a. Let $_{CONT}$ tableKeyValue* be tableKeysResult.
    b. Let _ _ ENTRIES '=' `{ tableEntryListIR `} be TBL.ENTRIES.
    c. Let context EC_2, state $ARCH_2$, and result tableMatchesResult be the result of evaluating table matches for keys tableKeyValue* against tableEntryListIR, under context EC_1 and state $ARCH_1$.
    d. If let exitResult of type exitResult be tableMatchesResult:
        i. The result is EC_2, $ARCH_2$, and EXIT.
    e. Else:
        i. Let $_{CONT}$ tableMatch* be tableMatchesResult.

ii. Let prefixedNameIR _ `( argumentListIR `) be tie-breaking among matches tableMatch* for table object TBL.

iii. Let context EC_3, state $ARCH_3$, and statementResult be the result of runtime evaluation of prefixedNameIR `< · `> `( argumentListIR `) ';', under context EC_2 and state $ARCH_2$ at LOCAL.

iv. If let exitResult of type exitResult be statementResult:

A. The result is EC_3, $ARCH_3$, and EXIT.

v. Else if let continueEmptyResult of type continueEmptyResult be statementResult:

A. Let b_{hit} be ~tableMatch* = ·.

B. Let b_{miss} be $_{b}$hit~.

C. Let nameIR be the string form of prefixedNameIR.

D. Let tableMetadataEnumValue be TABLE_ENUM "action_list(" ++ typeId ++ ")" '.' nameIR.

E. Let tableMetadataStructValue be TABLE_STRUCT "apply_result(" ++ typeId ++ ")" `{ HIT $_{B}$ $b_{hit}$ ';' MISS $_{B}$ $b_{miss}$ ';' ACTION_RUN tableMetadataEnumValue ';' `}.

F. The result is EC_3, $ARCH_3$, and RETURN tableMetadataStructValue.

The result of evaluating a table is either an exit or a return.

```
tableResult
    : exitResult | returnResult
    ;
```

After evaluating the table keys, the table lookup is performed as:

TableMatches_eval:

- Evaluating table matches for keys tableKeyValue* against tableEntryListIR, under context EC and state ARCH:
- The result is context EC, state ARCH , and result tableMatchesResult.

Evaluating table matches for keys tableKeyValue* against tableEntryIR*, under context EC and state ARCH:

1. If tableEntryIR* matches pattern []:
   a. The result is context EC, state ARCH , and result $_{CONT}$ ·.
2. Else:
   a. Let $tableEntryIR_h$ :: $tableEntryIR_t$* be tableEntryIR*.
   b. Let context EC_1, state $ARCH_1$, and result tableMatchResult be the result of evaluating table match for keys tableKeyValue* against $tableEntryIR_h$, under context EC and state ARCH.
   c. If let exitResult of type exitResult be tableMatchResult:
      i. The result is context EC_1, state $ARCH_1$, and result EXIT.
   d. Else:

i. Let $_{CONT}$ tableMatch$_h^?$ be tableMatchResult.

ii. Let context EC_2, state $ARCH_2$, and result tableMatchesResult be the result of evaluating table matches for keys tableKeyValue* against tableEntryIR$_t^*$, under context EC_1 and state $ARCH_1$.

iii. If let exitResult of type exitResult be tableMatchesResult:

A. The result is context EC_2, state $ARCH_2$, and result EXIT.

iv. Else:

A. Let $_{CONT}$ tableMatch$_t^*$ be tableMatchesResult.

B. Let tableMatch'* be $opt_as_seq_<tableMatch>(tableMatch$_h^?$).

C. Let tableMatch* be tableMatch'* concatenated with tableMatch$_t^*$.

D. The result is context EC_2, state $ARCH_2$, and result $_{CONT}$ tableMatch*.

TableMatch_eval:

- Evaluating table match for keys tableKeyValue* against tableEntryIR, under context EC and state ARCH:
- The result is context EC, state ARCH , and result tableMatchResult.

Evaluating table match for keys tableKeyValue* against tableEntryIR, under context EC_0 and state $ARCH_0$:

1. Let _ tableEntryPriorityIR$^?$ keysetExpressionIR ':' tableActionReferenceIR _ ';' be tableEntryIR.
2. Let $n_{priority}^?$ be the priority of tableEntryPriorityIR$^?$.
3. Let context EC_1, state $ARCH_1$, and result keysetExpressionResult be the result of evaluating keysetExpressionIR, under context EC_0 and state $ARCH_0$.
4. If let exitResult of type exitResult be keysetExpressionResult:
   a. The result is context EC_1, state $ARCH_1$, and result EXIT.
5. Else if let $_{CONT}$ setValue* of type continueResult<setValue*> be keysetExpressionResult:
   a. For each tableKeyValue in tableKeyValue*:
      - Let value$_{key}$ ':' _ be tableKeyValue.

      Let value$_{key}^*$ be the resulting list.
   b. If value$_{key}^*$ does not match setValue*:
      i. The result is context EC_1, state $ARCH_1$, and result $_{CONT}$ ·.
   c. Else:
      i. Let tableMatch be $n_{priority}^?$ ':' tableActionReferenceIR.
      ii. The result is context EC_1, state $ARCH_1$, and result $_{CONT}$ tableMatch.

[1] TDI is the Table Driven Interface. More information can be found here: https://github.com/p4lang/tdi

[2] Most existing $P4_{16}$ programs today do not use function or method calls in table key expressions, and the order of evaluation of these key expressions makes no difference in the resulting lookup key value. In this overwhelmingly common case, if an implementation chooses to insert extra assignment statements to implement side-effecting key expressions, but does not insert them when there are no side-effecting key expressions, then in typical programs they will almost never be inserted.

18. Casting

P4 provides a limited set of casts between types. While this design is arguably more onerous for programmers, it has several benefits:

- It makes user intent unambiguous.
- It makes the costs associated with converting numeric values explicit. Implementing certain casts involves sign extensions, and thus can require significant computational resources on some targets.
- It reduces the number of cases that have to be considered in the P4 specification. Some targets may not support all casts.

18.1. Cast chains

A cast is not always a single step. Casting a sequence to a struct, for example, casts each element of the sequence before casting the sequence as a whole, and casting between two enums with an underlying type goes through that underlying type. Checking a cast therefore yields the chain of types to cast through, in order, or reports that no such cast exists.

```
castResult
    : OK typeIR* | FAIL
    ;
```

The casts of the elements of an aggregate type are chained in lockstep, one cast of the whole aggregate per step. An element whose chain is exhausted holds its current type, so that all elements reach their target together.

$chain_casts(typeIR$_{cur}^{*}$, typeIR$_{chain}^{**}$)

1. If $concat_<typeIR>(typeIR$_{chain}^{**}$) matches pattern []: return ·.
2. Else:
   a. Check that the length of typeIR$_{chain}^{**}$ is equal to the length of typeIR$_{cur}^{*}$.
   b. For each typeIR$_{chain}^{*}$ in typeIR$_{chain}^{**}$ and typeIR$_{cur}$ in typeIR$_{cur}^{*}$:
      - Let (typeIR$_{next}$, typeIR$_{chain_rest}^{*}$) be $chain_casts'(typeIR$_{cur}$, typeIR$_{chain}^{*}$).

      Let typeIR$_{chain_rest}^{**}$ and typeIR$_{next}^{*}$ be the resulting lists.
   c. Let typeIR** be $chain_casts(typeIR$_{next}^{*}$, typeIR$_{chain_rest}^{**}$).
   d. Return typeIR$_{next}^{*}$ :: typeIR**.

$chain_casts'(typeIR$_{cur}$, typeIR*)

1. If typeIR* matches pattern []: return (typeIR$_{cur}$, ·).
2. Else:
   a. Let typeIR$_{h}$ :: typeIR$_{t}^{*}$ be typeIR*.
   b. Return (typeIR$_{h}$, typeIR$_{t}^{*}$).

Each step of the chain is then wrapped back into the shape of the aggregate type being cast, a sequence type,

$chain_casts_sequenceTypeIR(typeIR$_a^*$, typeIR$_{cast_elem}^{**}$, typeIR$_b$)

1. Let typeIR$_{cast_step}^{**}$ be $chain_casts(typeIR$_a^*$, typeIR$_{cast_elem}^{**}$).
2. For each typeIR$_{cast_step}^{*}$ in typeIR$_{cast_step}^{**}$:
   - Let typeIR$_{cast_agg}$ be SEQ `< typeIR$_{cast\_step}^{*}$ `>.

   Let typeIR$_{cast_agg}^{*}$ be the resulting list.
3. Return typeIR$_{cast_agg}^{*}$ concatenated with typeIR$_b$.

$chain_casts_sequenceTypeIR_default(typeIR$_a^*$, typeIR$_{cast_elem}^{**}$, typeIR$_b$)

1. Let typeIR$_{cast_step}^{**}$ be $chain_casts(typeIR$_a^*$, typeIR$_{cast_elem}^{**}$).
2. For each typeIR$_{cast_step}^{*}$ in typeIR$_{cast_step}^{**}$:
   - Let typeIR$_{cast_agg}$ be SEQ `< typeIR$_{cast\_step}^{*}$ ',' '...' `>.

   Let typeIR$_{cast_agg}^{*}$ be the resulting list.
3. Return typeIR$_{cast_agg}^{*}$ concatenated with typeIR$_b$.

or a record type, whose field names are kept across the chain.

$chain_casts_recordTypeIR((annotationList$_a$ typeIR$_a$ id$_a$ ';')*, typeIR$_{cast_elem}^{**}$, typeIR$_b$)

1. Let typeIR$_{cast_step}^{**}$ be $chain_casts(typeIR$_a^*$, typeIR$_{cast_elem}^{**}$).
2. Check that the length of annotationList$_a^*$ is equal to the length of id$_a^*$ and the length of id$_a^*$ is equal to the length of typeIR$_{cast_step}^{*}$, for all typeIR$_{cast_step}^{*}$ in typeIR$_{cast_step}^{**}$.
3. For each typeIR$_{cast_step}^{*}$ in typeIR$_{cast_step}^{**}$:
   - Let typeIR$_{cast_agg}$ be RECORD `{ ( annotationList$_a$ typeIR$_{cast\_step}$ id$_a$ ';' )$^*$ `}.

   Let typeIR$_{cast_agg}^{*}$ be the resulting list.
4. Return typeIR$_{cast_agg}^{*}$ concatenated with typeIR$_b$.

$chain_casts_recordTypeIR_default((annotationList$_a$ typeIR$_a$ id$_a$ ';')*, typeIR$_{cast_elem}^{**}$, typeIR$_b$)

1. Let typeIR$_{cast_step}^{**}$ be $chain_casts(typeIR$_a^*$, typeIR$_{cast_elem}^{**}$).
2. Check that the length of annotationList$_a^*$ is equal to the length of id$_a^*$ and the length of id$_a^*$ is equal to the length of typeIR$_{cast_step}^{*}$, for all typeIR$_{cast_step}^{*}$ in typeIR$_{cast_step}^{**}$.
3. For each typeIR$_{cast_step}^{*}$ in typeIR$_{cast_step}^{**}$:
   - Let typeIR$_{cast_agg}$ be RECORD `{ ( annotationList$_a$ typeIR$_{cast\_step}$ id$_a$ ';' )$^*$ ',' '...' `}.

   Let typeIR$_{cast_agg}^{*}$ be the resulting list.
4. Return typeIR$_{cast_agg}^{*}$ concatenated with typeIR$_b$.

18.2. Explicit casts

The below relation checks whether a type can be explicitly cast to another type.

Cast_expl:

- Checking if typeIR can be explicitly cast to typeIR':
- The result is casts castResult.

First, the two types are unrolled and checked to see if they are the same type. If so, no cast is needed.

Checking if typeIR_a can be explicitly cast to typeIR_b:

1. Let typeIR_{a_unroll} be typeIR_a with typedefs unrolled.
2. Let typeIR_{b_unroll} be typeIR_b with typedefs unrolled.
3. If typeIR_{a_unroll} and typeIR_{b_unroll} are the same type:
    a. The result is casts OK typeIR_b.
4. Else:
    a. Let casts castResult be the result of checking if typeIR_{a_unroll} can be explicitly cast to typeIR_{b_unroll} (when unequal).
    b. The result is casts castResult.

If the two types are not equal, the below relation is applied.

Cast_expl_neq:

- Checking if typeIR can be explicitly cast to typeIR' (when unequal):
- The result is casts castResult.

Cast_expl_neq dispatch:

1. Try:
    a. If let boolTypeIR of type boolTypeIR be typeIR:
        i. If typeIR' is equal to BIT `< 1 `>: goto boolean-fixBit
    b. Else if let intTypeIR of type intTypeIR be typeIR: goto arbitraryInt
    c. Else if let INT `< nat `> of type fixedIntTypeIR be typeIR: goto fixInt
    d. Else if let BIT `< nat `> of type fixedBitTypeIR be typeIR: goto fixBit
2. Then, try:

a. Goto newTypeIR

3. Then, try:
    a. Goto enumTypeIR-serializable
4. Then, try:
    a. If let defaultTypeIR of type defaultTypeIR be typeIR: goto defaultTypeIR
    b. Else if let invalidHeaderTypeIR of type invalidHeaderTypeIR be typeIR: goto invalidHeaderTypeIR
    c. Else if let sequenceTypeIR of type sequenceTypeIR be typeIR:
        i. If let SEQ `< $typeIR_a^*$ `> be sequenceTypeIR: goto sequenceTypeIR-non-default
        ii. Else:
            A. Let SEQ `< $typeIR_a^*$ ',' '...' `> be sequenceTypeIR.
            B. Goto sequenceTypeIR-default
    d. Else if let recordTypeIR of type recordTypeIR be typeIR:
        i. If let RECORD `{ $fieldTypeIR_a^*$ `} be recordTypeIR: goto recordTypeIR-non-default
        ii. Else:
            A. Let RECORD `{ $fieldTypeIR_a^*$ ',' '...' `} be recordTypeIR.
            B. Goto recordTypeIR-default

Cast from bool to bit<1>

Checking if BOOL can be explicitly cast to BIT `< 1 `> (when unequal):

1. The result is casts OK BIT `< 1 `>.

Cast from int to bool, int<S>, or bit<W>

Checking if INT can be explicitly cast to typeIR' (when unequal):

1. Try:
    a. If typeIR' is equal to BOOL:
        i. The result is casts OK BOOL.
2. Then, try:
    a. If let INT `< w `> of type fixedIntTypeIR be typeIR':
        i. The result is casts OK INT `< w `>.
    b. Else if let BIT `< w `> of type fixedBitTypeIR be typeIR':
        i. The result is casts OK BIT `< w `>.

Cast from int<S> to int, int<T>, or bit<W>

Checking if INT `< nat `> can be explicitly cast to typeIR' (when unequal):

1. Try:
    a. If typeIR' is equal to INT:
        i. The result is casts OK INT.
    b. Else if typeIR' is equal to BIT `< nat `>:
        i. The result is casts OK BIT `< nat `>.
2. Then, try:
    a. If let INT `< $w_b$ `> of type fixedIntTypeIR be typeIR':
        i. The result is casts OK INT `< $w_b$ `>.

Cast from bit<W> to bool, int, int<S>, or bit<X>

Checking if BIT `< nat `> can be explicitly cast to typeIR' (when unequal):

1. Try:
    a. If nat is equal to 1:
        i. If typeIR' is equal to BOOL:
            A. The result is casts OK BOOL.
2. Then, try:
    a. If typeIR' is equal to INT:
        i. The result is casts OK INT.
    b. Else if typeIR' is equal to INT `< nat `>:
        i. The result is casts OK INT `< nat `>.
3. Then, try:
    a. If let BIT `< $w_b$ `> of type fixedBitTypeIR be typeIR':
        i. The result is casts OK BIT `< $w_b$ `>.

Cast from a new type to its underlying type, and vice versa

Checking if typeIR can be explicitly cast to typeIR' (when unequal):

1. Try:
    a. If let TYPE _ $typeIR_a$ of type newTypeIR be typeIR:
        i. Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to typeIR'.
        ii. If let OK $typeIR_{cast}^*$ be castResult:
            A. The result is casts OK $typeIR_a$ :: $typeIR_{cast}^*$.
2. Then, try:
    a. If let newTypeIR of type newTypeIR be typeIR':
        i. Let TYPE _ $typeIR_b$ be newTypeIR.
        ii. Let casts castResult be the result of checking if typeIR can be implicitly cast to $typeIR_b$.
        iii. If let OK $typeIR_{cast}^*$ be castResult:

A. The result is casts OK $typeIR_{cast}^{*}$ ++ newTypeIR.

Cast from an enum with an underlying type to the underlying type, and vice versa

Checking if typeIR can be explicitly cast to typeIR' (when unequal):

1. Try:
    a. If let ENUM _ `< $typeIR_a$ `> `{ _ $^{*}$ `} of type serializableEnumTypeIR be typeIR:
        i. Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to typeIR'.
        ii. If let OK $typeIR_{cast}^{*}$ be castResult:
            A. The result is casts OK $typeIR_a$ :: $typeIR_{cast}^{*}$.
2. Then, try:
    a. If let serializableEnumTypeIR of type serializableEnumTypeIR be typeIR':
        i. Let ENUM _ `< $typeIR_b$ `> `{ _ $^{*}$ `} be serializableEnumTypeIR.
        ii. Let casts castResult be the result of checking if typeIR can be implicitly cast to $typeIR_b$.
        iii. If let OK $typeIR_{cast}^{*}$ be castResult:
            A. The result is casts OK $typeIR_{cast}^{*}$ ++ serializableEnumTypeIR.

Cast from a default type to the target type

Checking if DEFAULT can be explicitly cast to typeIR' (when unequal):

1. If typeIR' can be default-initialized:
    a. The result is casts OK typeIR'.

Cast from an invalid header type to a header or header union type

Checking if HEADER_INVALID can be explicitly cast to typeIR' (when unequal):

1. If let headerTypeIR of type headerTypeIR be typeIR':
    a. The result is casts OK headerTypeIR.
2. Else if let headerUnionTypeIR of type headerUnionTypeIR be typeIR':
    a. The result is casts OK headerUnionTypeIR.

Cast from a sequence type to a list, tuple, header stack, struct, header, or another sequence type

Checking if SEQ `< $typeIR_a^{*}$ `> can be explicitly cast to typeIR' (when unequal):

1. If let listTypeIR of type listTypeIR be typeIR':
    a. Let LIST `< $typeIR_b$ `> be listTypeIR.
    b. For each $typeIR_a$ in $typeIR_a^*$:
        - Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_b$.

        Let $castResult^*$ be the resulting list.
    c. If castResult matches pattern OK %, for all castResult in $castResult^*$:
        i. For each castResult in $castResult^*$:
            - Let OK $typeIR_{cast}^*$ be castResult.

            Let $typeIR_{cast}^{**}$ be the resulting list.
        ii. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR($typeIR_a^*$, $typeIR_{cast}^{**}$, listTypeIR).
        iii. The result is casts OK $typeIR_{cast_agg}^*$.
2. Else if let tupleTypeIR of type tupleTypeIR be typeIR':
    a. Let TUPLE `< $typeIR_b^*$ `> be tupleTypeIR.
    b. If the length of $typeIR_a^*$ is equal to the length of $typeIR_b^*$:
        i. For each $typeIR_a$ in $typeIR_a^*$ and $typeIR_b$ in $typeIR_b^*$:
            - Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_b$.

            Let $castResult^*$ be the resulting list.
        ii. If castResult matches pattern OK %, for all castResult in $castResult^*$:
            A. For each castResult in $castResult^*$:
                - Let OK $typeIR_{cast}^*$ be castResult.

                Let $typeIR_{cast}^{**}$ be the resulting list.
            B. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR($typeIR_a^*$, $typeIR_{cast}^{**}$, tupleTypeIR).
            C. The result is casts OK $typeIR_{cast_agg}^*$.
3. Else if let arrayTypeIR of type arrayTypeIR be typeIR':
    a. Let ARRAY $typeIR_b$ `[ $n_{size}$ `] be arrayTypeIR.
    b. If the length of $typeIR_a^*$ is equal to n_{size}:
        i. For each $typeIR_a$ in $typeIR_a^*$:
            - Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_b$.

            Let $castResult^*$ be the resulting list.
        ii. If castResult matches pattern OK %, for all castResult in $castResult^*$:
            A. For each castResult in $castResult^*$:
                - Let OK $typeIR_{cast}^*$ be castResult.

                Let $typeIR_{cast}^{**}$ be the resulting list.
            B. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR($typeIR_a^*$, $typeIR_{cast}^{**}$, arrayTypeIR).
            C. The result is casts OK $typeIR_{cast_agg}^*$.

4. Else if let headerStackTypeIR of type headerStackTypeIR be typeIR':
    a. Let HEADER_STACK typeIR$_b$ `[ $n_{size}$ `] be headerStackTypeIR.
    b. If the length of typeIR$_a^*$ is equal to n_{size}:
        i. For each typeIR$_a$ in typeIR$_a^*$:
            - Let casts castResult be the result of checking if typeIR$_a$ can be explicitly cast to typeIR$_b$.

        Let castResult* be the resulting list.

        ii. If castResult matches pattern OK %, for all castResult in castResult*:
            A. For each castResult in castResult*:
                - Let OK typeIR$_{cast}^*$ be castResult.

            Let typeIR$_{cast}^{**}$ be the resulting list.

            B. Let typeIR$_{cast_agg}^*$ be $chain_casts_sequenceTypeIR(typeIR$_a^*$, typeIR$_{cast}^{**}$, headerStackTypeIR).
            C. The result is casts OK typeIR$_{cast_agg}^*$.
5. Else if let structTypeIR of type structTypeIR be typeIR':
    a. Let STRUCT _ `< _$^*$ `> `{ ( _ typeIR$_{field\_b}$ _ ';' )$^*$ `} be structTypeIR.
    b. If the length of typeIR$_a^*$ is equal to the length of typeIR$_{field_b}^*$:
        i. For each typeIR$_a$ in typeIR$_a^*$ and typeIR$_{field_b}$ in typeIR$_{field_b}^*$:
            - Let casts castResult be the result of checking if typeIR$_a$ can be explicitly cast to typeIR$_{field_b}$.

        Let castResult* be the resulting list.

        ii. If castResult matches pattern OK %, for all castResult in castResult*:
            A. For each castResult in castResult*:
                - Let OK typeIR$_{cast}^*$ be castResult.

            Let typeIR$_{cast}^{**}$ be the resulting list.

            B. Let typeIR$_{cast_agg}^*$ be $chain_casts_sequenceTypeIR(typeIR$_a^*$, typeIR$_{cast}^{**}$, structTypeIR).
            C. The result is casts OK typeIR$_{cast_agg}^*$.
6. Else if let headerTypeIR of type headerTypeIR be typeIR':
    a. Let HEADER _ `< _$^*$ `> `{ ( _ typeIR$_{field\_b}$ _ ';' )$^*$ `} be headerTypeIR.
    b. If the length of typeIR$_a^*$ is equal to the length of typeIR$_{field_b}^*$:
        i. For each typeIR$_a$ in typeIR$_a^*$ and typeIR$_{field_b}$ in typeIR$_{field_b}^*$:
            - Let casts castResult be the result of checking if typeIR$_a$ can be explicitly cast to typeIR$_{field_b}$.

        Let castResult* be the resulting list.

        ii. If castResult matches pattern OK %, for all castResult in castResult*:
            A. For each castResult in castResult*:
                - Let OK typeIR$_{cast}^*$ be castResult.

            Let typeIR$_{cast}^{**}$ be the resulting list.

            B. Let typeIR$_{cast_agg}^*$ be $chain_casts_sequenceTypeIR(typeIR$_a^*$, typeIR$_{cast}^{**}$, headerTypeIR).

C. The result is casts OK $typeIR_{cast_agg}^{*}$.

Cast from a sequence type with default initializer to a tuple, header stack, struct, or header type

Checking if SEQ `< $typeIR_a^{*}$ ',' '...' `> can be explicitly cast to typeIR' (when unequal):

1. If let tupleTypeIR of type tupleTypeIR be typeIR':
   a. Let TUPLE `< $typeIR_b^{*}$ `> be tupleTypeIR.
   b. If the length of $typeIR_a^{*}$ is less than the length of $typeIR_b^{*}$:
      i. Let ($typeIR_{b_non_default}^{*}$, $typeIR_{b_default}^{*}$) be $partition_<typeIR>($typeIR_b^{*}$, the length of $typeIR_a^{*}$).
      ii. If the length of $typeIR_a^{*}$ is equal to the length of $typeIR_{b_non_default}^{*}$:
         A. For each $typeIR_a$ in $typeIR_a^{*}$ and $typeIR_{b_non_default}$ in $typeIR_{b_non_default}^{*}$:
            - Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_{b_non_default}$.

            Let castResult* be the resulting list.
         B. If castResult matches pattern OK %, for all castResult in castResult*:
            I. For each castResult in castResult*:
               - Let OK $typeIR_{cast}^{*}$ be castResult.

               Let $typeIR_{cast}^{**}$ be the resulting list.
            II. If $typeIR_{b_default}$ can be default-initialized, for all $typeIR_{b_default}$ in $typeIR_{b_default}^{*}$:
               1. Let $typeIR_{cast_agg}^{*}$ be $chain_casts_sequenceTypeIR_default($typeIR_a^{*}$, $typeIR_{cast}^{**}$, tupleTypeIR).
               2. The result is casts OK $typeIR_{cast_agg}^{*}$.
2. Else if let arrayTypeIR of type arrayTypeIR be typeIR':
   a. Let ARRAY $typeIR_b$ `[ $n_{size}$ `] be arrayTypeIR.
   b. If the length of $typeIR_a^{*}$ is less than n_{size}:
      i. For each $typeIR_a$ in $typeIR_a^{*}$:
         - Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_b$.

         Let castResult* be the resulting list.
      ii. If castResult matches pattern OK %, for all castResult in castResult*:
         A. For each castResult in castResult*:
            - Let OK $typeIR_{cast}^{*}$ be castResult.

            Let $typeIR_{cast}^{**}$ be the resulting list.
         B. If $typeIR_b$ can be default-initialized:
            I. Let $typeIR_{cast_agg}^{*}$ be $chain_casts_sequenceTypeIR_default($typeIR_a^{*}$, $typeIR_{cast}^{**}$, arrayTypeIR).
            II. The result is casts OK $typeIR_{cast_agg}^{*}$.
3. Else if let headerStackTypeIR of type headerStackTypeIR be typeIR':
   a. Let HEADER_STACK $typeIR_b$ `[ $n_{size}$ `] be headerStackTypeIR.

b. If the length of $typeIR_a^*$ is less than n_{size}:

i. For each $typeIR_a$ in $typeIR_a^*$:

- Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_b$.

Let $castResult^*$ be the resulting list.

ii. If castResult matches pattern OK %, for all castResult in $castResult^*$:

A. For each castResult in $castResult^*$:

- Let OK $typeIR_{cast}^*$ be castResult.

Let $typeIR_{cast}^{**}$ be the resulting list.

B. If $typeIR_b$ can be default-initialized:

I. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR_default($typeIR_a^*$, $typeIR_{cast}^{**}$, headerStackTypeIR).

II. The result is casts OK $typeIR_{cast_agg}^*$.

4. Else if let structTypeIR of type structTypeIR be typeIR':

a. Let STRUCT _ `< _ $^*$ `> `{ ( _ $typeIR_{field\_b}$ _ ';' )$^*$ `} be structTypeIR.

b. If the length of $typeIR_a^*$ is less than the length of $typeIR_{field_b}^*$:

i. Let ($typeIR_{field_b_non_default}^*$, $typeIR_{field_b_default}^*$) be $partition_<typeIR>($typeIR_{field_b}^*$, the length of $typeIR_a^*$).

ii. If the length of $typeIR_a^*$ is equal to the length of $typeIR_{field_b_non_default}^*$:

A. For each $typeIR_a$ in $typeIR_a^*$ and $typeIR_{field_b_non_default}$ in $typeIR_{field_b_non_default}^*$:

- Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_{field_b_non_default}$.

Let $castResult^*$ be the resulting list.

B. If castResult matches pattern OK %, for all castResult in $castResult^*$:

I. For each castResult in $castResult^*$:

- Let OK $typeIR_{cast}^*$ be castResult.

Let $typeIR_{cast}^{**}$ be the resulting list.

II. If $typeIR_{field_b_default}$ can be default-initialized, for all $typeIR_{field_b_default}$ in $typeIR_{field_b_default}^*$:

1. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR_default($typeIR_a^*$, $typeIR_{cast}^{**}$, structTypeIR).

2. The result is casts OK $typeIR_{cast_agg}^*$.

5. Else if let headerTypeIR of type headerTypeIR be typeIR':

a. Let HEADER _ `< _ $^*$ `> `{ ( _ $typeIR_{field\_b}$ _ ';' )$^*$ `} be headerTypeIR.

b. If the length of $typeIR_a^*$ is less than the length of $typeIR_{field_b}^*$:

i. Let ($typeIR_{field_b_non_default}^*$, $typeIR_{field_b_default}^*$) be $partition_<typeIR>($typeIR_{field_b}^*$, the length of $typeIR_a^*$).

ii. If the length of $typeIR_a^*$ is equal to the length of $typeIR_{field_b_non_default}^*$:

A. For each $typeIR_a$ in $typeIR_a^*$ and $typeIR_{field_b_non_default}$ in $typeIR_{field_b_non_default}^*$:

- Let casts castResult be the result of checking if $typeIR_a$ can be explicitly cast to $typeIR_{field_b_non_default}$.

Let $castResult^*$ be the resulting list.

B. If castResult matches pattern OK %, for all castResult in castResult*:

I. For each castResult in castResult*:

- Let OK typeIR$_{cast}$* be castResult.

Let typeIR$_{cast}$** be the resulting list.

II. If typeIR$_{field_b_default}$ can be default-initialized, for all typeIR$_{field_b_default}$ in typeIR$_{field_b_default}$*:

1. Let typeIR$_{cast_agg}$* be $chain_casts_sequenceTypeIR_default(typeIR$_a$*, typeIR$_{cast}$**, headerTypeIR).
2. The result is casts OK typeIR$_{cast_agg}$*.

Cast from a record type to a struct or header type

Checking if RECORD `{ fieldTypeIR$_a$* `} can be explicitly cast to typeIR' (when unequal):

1. If let structTypeIR of type structTypeIR be typeIR':
   a. Let (_ typeIR$_{field_a}$ id$_{field_a}$ ';')* be fieldTypeIR$_a$*.
   b. Let STRUCT _ `< _* `> `{ ( _ typeIR$_{field\_b}$ id$_{field\_b}$ ';' )* `} be structTypeIR.
   c. If `{ id$_{field\_a}$* `} is equal to `{ id$_{field\_b}$* `}:
      i. If the length of id$_{field_a}$* is equal to the length of typeIR$_{field_a}$*, for all id$_{field_a}$ in id$_{field_a}$*:
         A. For each id$_{field_a}$ in id$_{field_a}$*:
            - Let typeIR''$^?$ be the value of id$_{field_a}$ in map `{ ( id$_{field\_a}$ ':' typeIR$_{field\_a}$ )* `}.

            Let typeIR''$^{?*}$ be the resulting list.

         B. If typeIR''$^?$ is defined, for all typeIR''$^?$ in typeIR''$^{?*}$:
            I. For each typeIR''$^?$ in typeIR''$^{?*}$:
               - Let typeIR$_{field_a_aligned}$ be typeIR''$^?$.

               Let typeIR$_{field_a_aligned}$* be the resulting list.

            II. For each id$_{field_a}$ in id$_{field_a}$*:
               - Let typeIR'''$^?$ be the value of id$_{field_a}$ in map `{ ( id$_{field\_b}$ ':' typeIR$_{field\_b}$ )* `}.

               Let typeIR'''$^{?*}$ be the resulting list.

            III. If typeIR'''$^?$ is defined, for all typeIR'''$^?$ in typeIR'''$^{?*}$:
               1. For each typeIR'''$^?$ in typeIR'''$^{?*}$:
                  - Let typeIR$_{field_b_aligned}$ be typeIR'''$^?$.

                  Let typeIR$_{field_b_aligned}$* be the resulting list.

               2. For each typeIR$_{field_a_aligned}$ in typeIR$_{field_a_aligned}$* and typeIR$_{field_b_aligned}$ in typeIR$_{field_b_aligned}$*:
                  - Let casts castResult be the result of checking if typeIR$_{field_a_aligned}$ can be explicitly cast to typeIR$_{field_b_aligned}$.

Let castResult* be the resulting list.

3. If castResult matches pattern OK %, for all castResult in castResult*:
    1. For each castResult in castResult*:
        - Let OK typeIR$_{cast}$* be castResult.

        Let typeIR$_{cast}$** be the resulting list.
    2. Let typeIR$_{cast_agg}$* be $chain_casts_recordTypeIR(fieldTypeIR$_a$*, typeIR$_{cast}$**, structTypeIR).
    3. The result is casts OK typeIR$_{cast_agg}$*.

2. Else if let headerTypeIR of type headerTypeIR be typeIR':
    a. Let (_ typeIR$_{field_a}$ id$_{field_a}$ ';')* be fieldTypeIR$_a$*.
    b. Let HEADER _ `< _* `> `{ ( _ typeIR$_{field\_b}$ id$_{field\_b}$ ';' )* `} be headerTypeIR.
    c. If `{ id$_{field\_a}$* `} is equal to `{ id$_{field\_b}$* `}:
        i. If the length of id$_{field_a}$* is equal to the length of typeIR$_{field_a}$*, for all id$_{field_a}$ in id$_{field_a}$*:
            A. For each id$_{field_a}$ in id$_{field_a}$*:
                - Let typeIR''? be the value of id$_{field_a}$ in map `{ ( id$_{field\_a}$ ':' typeIR$_{field\_a}$ )* `}.

            Let typeIR''?* be the resulting list.

            B. If typeIR''? is defined, for all typeIR''? in typeIR''?*:
                I. For each typeIR''? in typeIR''?*:
                    - Let typeIR$_{field_a_aligned}$ be typeIR''?.

                Let typeIR$_{field_a_aligned}$* be the resulting list.

                II. For each id$_{field_a}$ in id$_{field_a}$*:
                    - Let typeIR'''? be the value of id$_{field_a}$ in map `{ ( id$_{field\_b}$ ':' typeIR$_{field\_b}$ )* `}.

                Let typeIR'''?* be the resulting list.

                III. If typeIR'''? is defined, for all typeIR'''? in typeIR'''?*:
                    1. For each typeIR'''? in typeIR'''?*:
                        - Let typeIR$_{field_b_aligned}$ be typeIR'''?.

                    Let typeIR$_{field_b_aligned}$* be the resulting list.
                    2. For each typeIR$_{field_a_aligned}$ in typeIR$_{field_a_aligned}$* and typeIR$_{field_b_aligned}$ in typeIR$_{field_b_aligned}$*:
                        - Let casts castResult be the result of checking if typeIR$_{field_a_aligned}$ can be explicitly cast to typeIR$_{field_b_aligned}$.

                    Let castResult* be the resulting list.
                    3. If castResult matches pattern OK %, for all castResult in castResult*:
                        1. For each castResult in castResult*:
                            - Let OK typeIR$_{cast}$* be castResult.

                        Let typeIR$_{cast}$** be the resulting list.

2. Let $typeIR_{cast_agg}^{*}$ be $chain_casts_recordTypeIR(fieldTypeIR_a^{*}, typeIR_{cast}^{**}, headerTypeIR).
3. The result is casts OK $typeIR_{cast_agg}^{*}$.

Cast from a record type with default initializer to a struct or header type

Checking if RECORD `{ $fieldTypeIR_a^{*}$ ',' '...' `} can be explicitly cast to typeIR' (when unequal):

1. If let structTypeIR of type structTypeIR be typeIR':
   a. Let (_ $typeIR_{field_a}$ id_{field_a} ';')* be $fieldTypeIR_a^{*}$.
   b. Let STRUCT _ `< _* `> `{ ( _ $typeIR_{field\_b}$ $id_{field\_b}$ ';' )* `} be structTypeIR.
   c. If `{ $id_{field\_a}^{*}$ `} is a subset of `{ $id_{field\_b}^{*}$ `}:
      i. If the length of $id_{field_a}^{*}$ is equal to the length of $typeIR_{field_a}^{*}$, for all id_{field_a} in $id_{field_a}^{*}$:
         A. For each id_{field_a} in $id_{field_a}^{*}$:
            - Let $typeIR''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_a}$ ':' $typeIR_{field\_a}$ )* `}.

            Let $typeIR''^{?*}$ be the resulting list.
         B. If $typeIR''^{?}$ is defined, for all $typeIR''^{?}$ in $typeIR''^{?*}$:
            I. For each $typeIR''^{?}$ in $typeIR''^{?*}$:
               - Let $typeIR_{field_a_aligned}$ be $typeIR''^{?}$.

               Let $typeIR_{field_a_aligned}^{*}$ be the resulting list.
            II. For each id_{field_a} in $id_{field_a}^{*}$:
               - Let $typeIR'''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_b}$ ':' $typeIR_{field\_b}$ )* `}.

               Let $typeIR'''^{?*}$ be the resulting list.
            III. If $typeIR'''^{?}$ is defined, for all $typeIR'''^{?}$ in $typeIR'''^{?*}$:
               1. For each $typeIR'''^{?}$ in $typeIR'''^{?*}$:
                  - Let $typeIR_{field_b_non_default_aligned}$ be $typeIR'''^{?}$.

                  Let $typeIR_{field_b_non_default_aligned}^{*}$ be the resulting list.
               2. For each $typeIR_{field_a_aligned}$ in $typeIR_{field_a_aligned}^{*}$ and $typeIR_{field_b_non_default_aligned}$ in $typeIR_{field_b_non_default_aligned}^{*}$:
                  - Let casts castResult be the result of checking if $typeIR_{field_a_aligned}$ can be explicitly cast to $typeIR_{field_b_non_default_aligned}$.

                  Let castResult* be the resulting list.
               3. If castResult matches pattern OK %, for all castResult in castResult*:
                  1. For each castResult in castResult*:
                     - Let OK $typeIR_{cast}^{*}$ be castResult.

                     Let $typeIR_{cast}^{**}$ be the resulting list.

2. Let `{ $id_{default}$* `} be the difference of the sets `{ $id_{field\_b}$* `} and `{ $id_{field\_a}$* `}.
3. For each $id_{default}$ in $id_{default}$*:
    - Let $typeIR''''^{?}$ be the value of $id_{default}$ in map `{ ( $id_{field\_b}$ ':' $typeIR_{field\_b}$ )* `}.

    Let $typeIR''''^{?*}$ be the resulting list.
4. If $typeIR''''^{?}$ is defined, for all $typeIR''''^{?}$ in $typeIR''''^{?*}$:
    1. For each $typeIR''''^{?}$ in $typeIR''''^{?*}$:
        - Let $typeIR_{field_b_default}$ be $typeIR''''^{?}$.

        Let $typeIR_{field_b_default}$* be the resulting list.
    2. If $typeIR_{field_b_default}$ can be default-initialized, for all $typeIR_{field_b_default}$ in $typeIR_{field_b_default}$*:
        1. Let $typeIR_{cast_agg}$* be $chain_casts_recordTypeIR_default(fieldTypeIR_a*, typeIR_cast**, structTypeIR).
        2. The result is casts OK $typeIR_{cast_agg}$*.

2. Else if let headerTypeIR of type headerTypeIR be typeIR':
    a. Let (_ $typeIR_{field_a}$ id_{field_a} ';')* be $fieldTypeIR_a$*.
    b. Let HEADER _ `< _* `> `{ ( _ $typeIR_{field\_b}$ $id_{field\_b}$ ';' )* `} be headerTypeIR.
    c. If `{ $id_{field\_a}$* `} is a subset of `{ $id_{field\_b}$* `}:
        i. If the length of id_{field_a}* is equal to the length of $typeIR_{field_a}$*, for all id_{field_a} in id_{field_a}*:
            A. For each id_{field_a} in id_{field_a}*:
                - Let $typeIR''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_a}$ ':' $typeIR_{field\_a}$ )* `}.

            Let $typeIR''^{?*}$ be the resulting list.
            B. If $typeIR''^{?}$ is defined, for all $typeIR''^{?}$ in $typeIR''^{?*}$:
                I. For each $typeIR''^{?}$ in $typeIR''^{?*}$:
                    - Let $typeIR_{field_a_aligned}$ be $typeIR''^{?}$.

                Let $typeIR_{field_a_aligned}$* be the resulting list.
                II. For each id_{field_a} in id_{field_a}*:
                    - Let $typeIR'''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_b}$ ':' $typeIR_{field\_b}$ )* `}.

                Let $typeIR'''^{?*}$ be the resulting list.
                III. If $typeIR'''^{?}$ is defined, for all $typeIR'''^{?}$ in $typeIR'''^{?*}$:
                    1. For each $typeIR'''^{?}$ in $typeIR'''^{?*}$:
                        - Let $typeIR_{field_b_non_default_aligned}$ be $typeIR'''^{?}$.

                    Let $typeIR_{field_b_non_default_aligned}$* be the resulting list.
                    2. For each $typeIR_{field_a_aligned}$ in $typeIR_{field_a_aligned}$* and $typeIR_{field_b_non_default_aligned}$ in $typeIR_{field_b_non_default_aligned}$*:
                        - Let casts castResult be the result of checking if $typeIR_{field_a_aligned}$ can be explicitly cast to $typeIR_{field_b_non_default_aligned}$.

Let castResult* be the resulting list.

3. If castResult matches pattern OK %, for all castResult in castResult*:
   1. For each castResult in castResult*:
      - Let OK typeIR$_{cast}$* be castResult.

      Let typeIR$_{cast}$** be the resulting list.
   2. Let `{ id$_{default}$* `} be the difference of the sets `{ id$_{field\_b}$* `} and `{ id$_{field\_a}$* `}.
   3. For each id$_{default}$ in id$_{default}$*:
      - Let typeIR''''? be the value of id$_{default}$ in map `{ ( id$_{field\_b}$ ':' typeIR$_{field\_b}$ )* `}.

      Let typeIR''''?* be the resulting list.
   4. If typeIR''''? is defined, for all typeIR''''? in typeIR''''?*:
      1. For each typeIR''''? in typeIR''''?*:
         - Let typeIR$_{field_b_default}$ be typeIR''''?.

         Let typeIR$_{field_b_default}$* be the resulting list.
      2. If typeIR$_{field_b_default}$ can be default-initialized, for all typeIR$_{field_b_default}$ in typeIR$_{field_b_default}$*:
         1. Let typeIR$_{cast_agg}$* be $chain_casts_recordTypeIR_default(fieldTypeIR$_a$*, typeIR$_{cast}$**, headerTypeIR).
         2. The result is casts OK typeIR$_{cast_agg}$*.

When no explicit cast exists

If none of the above applies, the two types cannot be explicitly cast.

1. Otherwise:
   a. Checking if typeIR can be explicitly cast to typeIR' (when unequal): the result is casts FAIL.

18.3. Implicit casts

To keep the language simple and avoid introducing hidden costs, the main implicit casts in P4 are casts from int to fixed-width types and from enums with an underlying type to the underlying type. In particular, applying a binary operation (except shifts and concatenation) to an expression of type int and an expression with a fixed-width type will implicitly cast the int expression to the type of the other expression. For enums with an underlying type, it can be implicitly cast to its underlying type whenever appropriate, including but not limited to in shifts, concatenation, bit slicing indexes, header stack indexes as well as other unary and binary operations.

For example, given the following declarations,

```
enum bit<8> E {
    a = 5
}

bit<8>  x;
```

```
bit<16> y;
int<8>  z;
```

the compiler will add implicit casts as follows:

- x + 1 becomes x + (bit<8>)1
- z < 0 becomes z < (int<8>)0
- x | 0xFFF becomes x | (bit<8>)0xFFF; overflow warning
- x + E.a becomes x + (bit<8>)E.a
- x &&& 8 becomes x &&& (bit<8>)8
- x << 256 remains unchanged; 256 not implicitly cast to 8w0 in a shift; overflow warning
- 16w11 << E.a becomes 16w11 << (bit<8>)E.a
- x[E.a:0] becomes x[(bit<8>)E.a:0]
- E.a ++ 8w0 becomes (bit<8>)E.a ++ 8w0

The compiler also adds implicit casts when types of different expressions need to match; for example, as described in Section [sec-select], since select labels are compared against the selected expression, the compiler will insert implicit casts for the select labels when they have int types. Similarly, when assigning a structure-valued expression to a structure or header, the compiler will add implicit casts for int fields.

The below relation checks whether a type can be implicitly cast to another type.

Cast_impl:

- Checking if typeIR can be implicitly cast to typeIR':
- The result is casts castResult.

First, the two types are unrolled and checked to see if they are the same type. If so, no cast is needed.

Checking if typeIR_a can be implicitly cast to typeIR_b:

1. Let $\mathit{typeIR}_{a_unroll}$ be typeIR_a with typedefs unrolled.
2. Let $\mathit{typeIR}_{b_unroll}$ be typeIR_b with typedefs unrolled.
3. If $\mathit{typeIR}_{a_unroll}$ and $\mathit{typeIR}_{b_unroll}$ are the same type: the result is casts OK ·.
4. Else:
   a. Let casts castResult be the result of checking if $\mathit{typeIR}_{a_unroll}$ can be implicitly cast to $\mathit{typeIR}_{b_unroll}$ (when unequal).
   b. The result is casts castResult.

If the two types are not equal, the below relation is applied.

Cast_impl_neq:

- Checking if typeIR can be implicitly cast to typeIR' (when unequal):
- The result is casts castResult.

Cast_impl_neq dispatch:

1. If let intTypeIR of type intTypeIR be typeIR: goto arbitraryInt
2. Else if let ENUM typeId `< $typeIR_a$ `> `{ $valueFieldIR^*$ `} of type serializableEnumTypeIR be typeIR: goto enumTypeIR-serializable
3. Else if let defaultTypeIR of type defaultTypeIR be typeIR: goto defaultTypeIR
4. Else if let invalidHeaderTypeIR of type invalidHeaderTypeIR be typeIR: goto invalidHeaderTypeIR
5. Else if let sequenceTypeIR of type sequenceTypeIR be typeIR:
   a. If let SEQ `< $typeIR_a^*$ `> be sequenceTypeIR: goto sequenceTypeIR-non-default
   b. Else:
      i. Let SEQ `< $typeIR_a^*$ ',' '...' `> be sequenceTypeIR.
      ii. Goto sequenceTypeIR-default
6. Else if let recordTypeIR of type recordTypeIR be typeIR:
   a. If let RECORD `{ $fieldTypeIR_a^*$ `} be recordTypeIR: goto recordTypeIR-non-default
   b. Else:
      i. Let RECORD `{ $fieldTypeIR_a^*$ ',' '...' `} be recordTypeIR.
      ii. Goto recordTypeIR-default

Cast from int to int<S> or bit<W>

`<span id="Cast_impl_neq-arbitraryInt"></span>`

Checking if INT can be implicitly cast to typeIR' (when unequal):

1. If let INT `< w `> of type fixedIntTypeIR be typeIR':
   a. The result is casts OK INT `< w `>.
2. Else if let BIT `< w `> of type fixedBitTypeIR be typeIR':
   a. The result is casts OK BIT `< w `>.

Cast from an enum with an underlying type to the underlying type

`<span id="Cast_impl_neq-enumTypeIR-serializable"></span>`

Checking if ENUM typeId `< $typeIR_a$ `> `{ $valueFieldIR^*$ `} can be implicitly cast to typeIR' (when unequal):

1. Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to typeIR'.
2. If let OK $typeIR_{cast}^*$ be castResult:
   a. The result is casts OK $typeIR_a$ :: $typeIR_{cast}^*$.

Cast from a default type to the target type

Checking if DEFAULT can be implicitly cast to typeIR' (when unequal):

1. If typeIR' can be default-initialized:
    a. The result is casts OK typeIR'.

Cast from an invalid header type to a header or header union type

Checking if HEADER_INVALID can be implicitly cast to typeIR' (when unequal):

1. If let headerTypeIR of type headerTypeIR be typeIR':
    a. The result is casts OK headerTypeIR.
2. Else if let headerUnionTypeIR of type headerUnionTypeIR be typeIR':
    a. The result is casts OK headerUnionTypeIR.

Cast from a sequence type to a list, tuple, header stack, struct, header, or another sequence type

Checking if SEQ `< $typeIR_a^*` > can be implicitly cast to typeIR' (when unequal):

1. If let listTypeIR of type listTypeIR be typeIR':
    a. Let LIST `< $typeIR_b` > be listTypeIR.
    b. For each $typeIR_a$ in $typeIR_a^*$:

        - Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_b$.

    Let $castResult^*$ be the resulting list.

    c. If castResult matches pattern OK %, for all castResult in $castResult^*$:
        i. For each castResult in $castResult^*$:

            - Let OK $typeIR_{cast}^*$ be castResult.

        Let $typeIR_{cast}^{**}$ be the resulting list.

        ii. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR($typeIR_a^*$, $typeIR_{cast}^{**}$, listTypeIR).
        iii. The result is casts OK $typeIR_{cast_agg}^*$.
2. Else if let tupleTypeIR of type tupleTypeIR be typeIR':
    a. Let TUPLE `< $typeIR_b^*$ `> be tupleTypeIR.
    b. If the length of $typeIR_a^*$ is equal to the length of $typeIR_b^*$:
        i. For each $typeIR_a$ in $typeIR_a^*$ and $typeIR_b$ in $typeIR_b^*$:

            - Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_b$.

Let castResult* be the resulting list.

ii. If castResult matches pattern OK %, for all castResult in castResult*:

A. For each castResult in castResult*:

- Let OK typeIR$_{cast}$* be castResult.

Let typeIR$_{cast}$** be the resulting list.

B. Let typeIR$_{cast_agg}$* be $chain_casts_sequenceTypeIR(typeIR$_a$*, typeIR$_{cast}$**, tupleTypeIR).

C. The result is casts OK typeIR$_{cast_agg}$*.

3. Else if let arrayTypeIR of type arrayTypeIR be typeIR':

a. Let ARRAY typeIR$_b$ `[ n$_{size}$ `] be arrayTypeIR.

b. If the length of typeIR$_a$* is equal to n$_{size}$:

i. For each typeIR$_a$ in typeIR$_a$*:

- Let casts castResult be the result of checking if typeIR$_a$ can be implicitly cast to typeIR$_b$.

Let castResult* be the resulting list.

ii. If castResult matches pattern OK %, for all castResult in castResult*:

A. For each castResult in castResult*:

- Let OK typeIR$_{cast}$* be castResult.

Let typeIR$_{cast}$** be the resulting list.

B. Let typeIR$_{cast_agg}$* be $chain_casts_sequenceTypeIR(typeIR$_a$*, typeIR$_{cast}$**, arrayTypeIR).

C. The result is casts OK typeIR$_{cast_agg}$*.

4. Else if let headerStackTypeIR of type headerStackTypeIR be typeIR':

a. Let HEADER_STACK typeIR$_b$ `[ n$_{size}$ `] be headerStackTypeIR.

b. If the length of typeIR$_a$* is equal to n$_{size}$:

i. For each typeIR$_a$ in typeIR$_a$*:

- Let casts castResult be the result of checking if typeIR$_a$ can be implicitly cast to typeIR$_b$.

Let castResult* be the resulting list.

ii. If castResult matches pattern OK %, for all castResult in castResult*:

A. For each castResult in castResult*:

- Let OK typeIR$_{cast}$* be castResult.

Let typeIR$_{cast}$** be the resulting list.

B. Let typeIR$_{cast_agg}$* be $chain_casts_sequenceTypeIR(typeIR$_a$*, typeIR$_{cast}$**, headerStackTypeIR).

C. The result is casts OK typeIR$_{cast_agg}$*.

5. Else if let structTypeIR of type structTypeIR be typeIR':

a. Let STRUCT _ `< _* `> `{ ( _ typeIR$_{field\_b}$ _ ';' )* `} be structTypeIR.

b. If the length of typeIR$_a$* is equal to the length of typeIR$_{field_b}$*:

i. For each typeIR$_a$ in typeIR$_a$* and typeIR$_{field_b}$ in typeIR$_{field_b}$*:

- Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_{field_b}$.

Let $castResult^*$ be the resulting list.

ii. If castResult matches pattern OK %, for all castResult in $castResult^*$:

A. For each castResult in $castResult^*$:

- Let OK $typeIR_{cast}{}^*$ be castResult.

Let $typeIR_{cast}{}^{**}$ be the resulting list.

B. Let $typeIR_{cast_agg}{}^*$ be $chain_casts_sequenceTypeIR($typeIR_a^*$, $typeIR_{cast}{}^{**}$, structTypeIR).

C. The result is casts OK $typeIR_{cast_agg}{}^*$.

6. Else if let headerTypeIR of type headerTypeIR be typeIR':

a. Let HEADER _ `< _* `> `{ ( _ $typeIR_{field\_b}$ _ ';' )* `} be headerTypeIR.

b. If the length of $typeIR_a{}^*$ is equal to the length of $typeIR_{field_b}{}^*$:

i. For each $typeIR_a$ in $typeIR_a{}^*$ and $typeIR_{field_b}$ in $typeIR_{field_b}{}^*$:

- Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_{field_b}$.

Let $castResult^*$ be the resulting list.

ii. If castResult matches pattern OK %, for all castResult in $castResult^*$:

A. For each castResult in $castResult^*$:

- Let OK $typeIR_{cast}{}^*$ be castResult.

Let $typeIR_{cast}{}^{**}$ be the resulting list.

B. Let $typeIR_{cast_agg}{}^*$ be $chain_casts_sequenceTypeIR($typeIR_a^*$, $typeIR_{cast}{}^{**}$, headerTypeIR).

C. The result is casts OK $typeIR_{cast_agg}{}^*$.

7. Else if let $sequenceTypeIR_b$ of type sequenceTypeIR be typeIR':

a. If let SEQ `< $typeIR_b{}^*$ `> be $sequenceTypeIR_b$:

i. If the length of $typeIR_a{}^*$ is equal to the length of $typeIR_b{}^*$:

A. For each $typeIR_a$ in $typeIR_a{}^*$ and $typeIR_b$ in $typeIR_b{}^*$:

- Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_b$.

Let $castResult^*$ be the resulting list.

B. If castResult matches pattern OK %, for all castResult in $castResult^*$:

I. For each castResult in $castResult^*$:

- Let OK $typeIR_{cast}{}^*$ be castResult.

Let $typeIR_{cast}{}^{**}$ be the resulting list.

II. Let $typeIR_{cast_agg}{}^*$ be $chain_casts_sequenceTypeIR($typeIR_a^*$, $typeIR_{cast}{}^{**}$, $sequenceTypeIR_b$).

III. The result is casts OK $typeIR_{cast_agg}{}^*$.

Cast from a sequence type with default initializer to a tuple, header stack, struct, or header type

`<span id="Cast_impl_neq-sequenceTypeIR-default"></span>`

Checking if SEQ `< $typeIR_a^*$ ',' '...' `> can be implicitly cast to typeIR' (when unequal):

1. If let tupleTypeIR of type tupleTypeIR be typeIR':
   a. Let TUPLE `< $typeIR_b^*$ `> be tupleTypeIR.
   b. If the length of $typeIR_a^*$ is less than the length of $typeIR_b^*$:
      i. Let ($typeIR_{b_non_default}^*$, $typeIR_{b_default}^*$) be $partition_<typeIR>($typeIR_b^*$, the length of $typeIR_a^*$).
      ii. If the length of $typeIR_a^*$ is equal to the length of $typeIR_{b_non_default}^*$:
         A. For each $typeIR_a$ in $typeIR_a^*$ and $typeIR_{b_non_default}$ in $typeIR_{b_non_default}^*$:
            - Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_{b_non_default}$.

            Let $castResult^*$ be the resulting list.
         B. If castResult matches pattern OK %, for all castResult in $castResult^*$:
            I. For each castResult in $castResult^*$:
               - Let OK $typeIR_{cast}^*$ be castResult.

               Let $typeIR_{cast}^{**}$ be the resulting list.
            II. If $typeIR_{b_default}$ can be default-initialized, for all $typeIR_{b_default}$ in $typeIR_{b_default}^*$:
               1. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR_default($typeIR_a^*$, $typeIR_{cast}^{**}$, tupleTypeIR).
               2. The result is casts OK $typeIR_{cast_agg}^*$.
2. Else if let arrayTypeIR of type arrayTypeIR be typeIR':
   a. Let ARRAY $typeIR_b$ `[ $n_{size}$ `] be arrayTypeIR.
   b. If the length of $typeIR_a^*$ is less than n_{size}:
      i. For each $typeIR_a$ in $typeIR_a^*$:
         - Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_b$.

         Let $castResult^*$ be the resulting list.
      ii. If castResult matches pattern OK %, for all castResult in $castResult^*$:
         A. For each castResult in $castResult^*$:
            - Let OK $typeIR_{cast}^*$ be castResult.

            Let $typeIR_{cast}^{**}$ be the resulting list.
         B. If $typeIR_b$ can be default-initialized:
            I. Let $typeIR_{cast_agg}^*$ be $chain_casts_sequenceTypeIR_default($typeIR_a^*$, $typeIR_{cast}^{**}$, arrayTypeIR).
            II. The result is casts OK $typeIR_{cast_agg}^*$.
3. Else if let headerStackTypeIR of type headerStackTypeIR be typeIR':
   a. Let HEADER_STACK $typeIR_b$ `[ $n_{size}$ `] be headerStackTypeIR.
   b. If the length of $typeIR_a^*$ is less than n_{size}:
      i. For each $typeIR_a$ in $typeIR_a^*$:
         - Let casts castResult be the result of checking if $typeIR_a$ can be implicitly cast to $typeIR_b$.

         Let $castResult^*$ be the resulting list.

ii. If castResult matches pattern OK %, for all castResult in castResult*:

A. For each castResult in castResult*:

- Let OK typeIR$_{cast}$* be castResult.

Let typeIR$_{cast}$** be the resulting list.

B. If typeIR$_{b}$ can be default-initialized:

I. Let typeIR$_{cast_agg}$* be $chain_casts_sequenceTypeIR_default(typeIR$_{a}$*, typeIR$_{cast}$**, headerStackTypeIR).

II. The result is casts OK typeIR$_{cast_agg}$*.

4. Else if let structTypeIR of type structTypeIR be typeIR':

a. Let STRUCT _ `< _* `> `{ ( _ typeIR$_{field\_b}$ _ ';' )* `} be structTypeIR.

b. If the length of typeIR$_{a}$* is less than the length of typeIR$_{field_b}$*:

i. Let (typeIR$_{field_b_non_default}$*, typeIR$_{field_b_default}$*) be $partition_<typeIR>(typeIR$_{field_b}$*, the length of typeIR$_{a}$*).

ii. If the length of typeIR$_{a}$* is equal to the length of typeIR$_{field_b_non_default}$*:

A. For each typeIR$_{a}$ in typeIR$_{a}$* and typeIR$_{field_b_non_default}$ in typeIR$_{field_b_non_default}$*:

- Let casts castResult be the result of checking if typeIR$_{a}$ can be implicitly cast to typeIR$_{field_b_non_default}$.

Let castResult* be the resulting list.

B. If castResult matches pattern OK %, for all castResult in castResult*:

I. For each castResult in castResult*:

- Let OK typeIR$_{cast}$* be castResult.

Let typeIR$_{cast}$** be the resulting list.

II. If typeIR$_{field_b_default}$ can be default-initialized, for all typeIR$_{field_b_default}$ in typeIR$_{field_b_default}$*:

1. Let typeIR$_{cast_agg}$* be $chain_casts_sequenceTypeIR_default(typeIR$_{a}$*, typeIR$_{cast}$**, structTypeIR).

2. The result is casts OK typeIR$_{cast_agg}$*.

5. Else if let headerTypeIR of type headerTypeIR be typeIR':

a. Let HEADER _ `< _* `> `{ ( _ typeIR$_{field\_b}$ _ ';' )* `} be headerTypeIR.

b. If the length of typeIR$_{a}$* is less than the length of typeIR$_{field_b}$*:

i. Let (typeIR$_{field_b_non_default}$*, typeIR$_{field_b_default}$*) be $partition_<typeIR>(typeIR$_{field_b}$*, the length of typeIR$_{a}$*).

ii. If the length of typeIR$_{a}$* is equal to the length of typeIR$_{field_b_non_default}$*:

A. For each typeIR$_{a}$ in typeIR$_{a}$* and typeIR$_{field_b_non_default}$ in typeIR$_{field_b_non_default}$*:

- Let casts castResult be the result of checking if typeIR$_{a}$ can be implicitly cast to typeIR$_{field_b_non_default}$.

Let castResult* be the resulting list.

B. If castResult matches pattern OK %, for all castResult in castResult*:

I. For each castResult in castResult*:

- Let OK typeIR$_{cast}$* be castResult.

Let typeIR$_{cast}$** be the resulting list.

II. If $typeIR_{field_b_default}$ can be default-initialized, for all $typeIR_{field_b_default}$ in $typeIR_{field_b_default}^{*}$:

1. Let $typeIR_{cast_agg}^{*}$ be $chain_casts_sequenceTypeIR_default(typeIR_{a}^{*}, typeIR_{cast}^{**}, headerTypeIR)$.
2. The result is casts OK $typeIR_{cast_agg}^{*}$.

Cast from a record type to a struct or header type

Checking if RECORD `{ $fieldTypeIR_{a}^{*}$ `} can be implicitly cast to typeIR' (when unequal):

1. If let structTypeIR of type structTypeIR be typeIR':
   a. Let ($annotationList_{a}$ $typeIR_{field_a}$ id_{field_a} ';')* be $fieldTypeIR_{a}^{*}$.
   b. Let STRUCT _ `< _* `> `{ ( _ $typeIR_{field\_b}$ $id_{field\_b}$ ';' )* `} be structTypeIR.
   c. If `{ $id_{field\_a}^{*}$ `} is equal to `{ $id_{field\_b}^{*}$ `}:
      i. If the length of $id_{field_a}^{*}$ is equal to the length of $typeIR_{field_a}^{*}$, for all id_{field_a} in $id_{field_a}^{*}$:
         A. For each id_{field_a} in $id_{field_a}^{*}$:
            - Let $typeIR''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_a}$ ':' $typeIR_{field\_a}$ )* `}.

            Let $typeIR''^{?*}$ be the resulting list.
         B. If $typeIR''^{?}$ is defined, for all $typeIR''^{?}$ in $typeIR''^{?*}$:
            I. For each $typeIR''^{?}$ in $typeIR''^{?*}$:
               - Let $typeIR_{field_a_aligned}$ be $typeIR''^{?}$.

               Let $typeIR_{field_a_aligned}^{*}$ be the resulting list.
            II. For each id_{field_a} in $id_{field_a}^{*}$:
               - Let $typeIR'''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_b}$ ':' $typeIR_{field\_b}$ )* `}.

               Let $typeIR'''^{?*}$ be the resulting list.
            III. If $typeIR'''^{?}$ is defined, for all $typeIR'''^{?}$ in $typeIR'''^{?*}$:
               1. For each $typeIR'''^{?}$ in $typeIR'''^{?*}$:
                  - Let $typeIR_{field_b_aligned}$ be $typeIR'''^{?}$.

                  Let $typeIR_{field_b_aligned}^{*}$ be the resulting list.
               2. For each $typeIR_{field_a_aligned}$ in $typeIR_{field_a_aligned}^{*}$ and $typeIR_{field_b_aligned}$ in $typeIR_{field_b_aligned}^{*}$:
                  - Let casts castResult be the result of checking if $typeIR_{field_a_aligned}$ can be implicitly cast to $typeIR_{field_b_aligned}$.

                  Let $castResult^{*}$ be the resulting list.
               3. If castResult matches pattern OK %, for all castResult in $castResult^{*}$:
                  1. For each castResult in $castResult^{*}$:
                     - Let OK $typeIR_{cast}^{*}$ be castResult.

Let $typeIR_{cast}^{**}$ be the resulting list.

2. Let $typeIR_{cast_agg}^{*}$ be $chain_casts_recordTypeIR(fieldTypeIR_a^{*}, typeIR_{cast}^{**}, structTypeIR).
3. The result is casts OK $typeIR_{cast_agg}^{*}$.

2. Else if let headerTypeIR of type headerTypeIR be typeIR':
   a. Let ($annotationList_a$ $typeIR_{field_a}$ id_{field_a} ';')* be $fieldTypeIR_a^{*}$.
   b. Let HEADER _ `< _* `> `{ ( _ $typeIR_{field\_b}$ $id_{field\_b}$ ';' )* `} be headerTypeIR.
   c. If `{ $id_{field\_a}^{*}$ `} is equal to `{ $id_{field\_b}^{*}$ `}:
      i. If the length of $id_{field_a}^{*}$ is equal to the length of $typeIR_{field_a}^{*}$, for all id_{field_a} in $id_{field_a}^{*}$:
         A. For each id_{field_a} in $id_{field_a}^{*}$:
            - Let $typeIR''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_a}$ ':' $typeIR_{field\_a}$ )* `}.

            Let $typeIR''^{?*}$ be the resulting list.

         B. If $typeIR''^{?}$ is defined, for all $typeIR''^{?}$ in $typeIR''^{?*}$:
            I. For each $typeIR''^{?}$ in $typeIR''^{?*}$:
               - Let $typeIR_{field_a_aligned}$ be $typeIR''^{?}$.

               Let $typeIR_{field_a_aligned}^{*}$ be the resulting list.

            II. For each id_{field_a} in $id_{field_a}^{*}$:
               - Let $typeIR'''^{?}$ be the value of id_{field_a} in map `{ ( $id_{field\_b}$ ':' $typeIR_{field\_b}$ )* `}.

               Let $typeIR'''^{?*}$ be the resulting list.

            III. If $typeIR'''^{?}$ is defined, for all $typeIR'''^{?}$ in $typeIR'''^{?*}$:
               1. For each $typeIR'''^{?}$ in $typeIR'''^{?*}$:
                  - Let $typeIR_{field_b_aligned}$ be $typeIR'''^{?}$.

                  Let $typeIR_{field_b_aligned}^{*}$ be the resulting list.

               2. For each $typeIR_{field_a_aligned}$ in $typeIR_{field_a_aligned}^{*}$ and $typeIR_{field_b_aligned}$ in $typeIR_{field_b_aligned}^{*}$:
                  - Let casts castResult be the result of checking if $typeIR_{field_a_aligned}$ can be implicitly cast to $typeIR_{field_b_aligned}$.

                  Let castResult* be the resulting list.

               3. If castResult matches pattern OK %, for all castResult in castResult*:
                  1. For each castResult in castResult*:
                     - Let OK $typeIR_{cast}^{*}$ be castResult.

                     Let $typeIR_{cast}^{**}$ be the resulting list.

                  2. Let $typeIR_{cast_agg}^{*}$ be $chain_casts_recordTypeIR(fieldTypeIR_a^{*}, typeIR_{cast}^{**}, headerTypeIR).
                  3. The result is casts OK $typeIR_{cast_agg}^{*}$.

Cast from a record type with default initializer to a struct or header type

Checking if RECORD `{ fieldTypeIR$_a^*$ ',' '...' `} can be implicitly cast to typeIR' (when unequal):

1. If let structTypeIR of type structTypeIR be typeIR':
   a. Let (annotationList$_a$ typeIR$_{field_a}$ id$_{field_a}$ ';')* be fieldTypeIR$_a^*$.
   b. Let STRUCT _ `< _$^*$ `> `{ ( _ typeIR$_{field\_b}$ id$_{field\_b}$ ';' )$^*$ `} be structTypeIR.
   c. If `{ id$_{field\_a}^*$ `} is a subset of `{ id$_{field\_b}^*$ `}:
      i. If the length of id$_{field_a}^*$ is equal to the length of typeIR$_{field_a}^*$, for all id$_{field_a}$ in id$_{field_a}^*$:
         A. For each id$_{field_a}$ in id$_{field_a}^*$:
            - Let typeIR''$^?$ be the value of id$_{field_a}$ in map `{ ( id$_{field\_a}$ ':' typeIR$_{field\_a}$ )$^*$ `}.

            Let typeIR''$^{?*}$ be the resulting list.
         B. If typeIR''$^?$ is defined, for all typeIR''$^?$ in typeIR''$^{?*}$:
            I. For each typeIR''$^?$ in typeIR''$^{?*}$:
               - Let typeIR$_{field_a_aligned}$ be typeIR''$^?$.

               Let typeIR$_{field_a_aligned}^*$ be the resulting list.
            II. For each id$_{field_a}$ in id$_{field_a}^*$:
               - Let typeIR'''$^?$ be the value of id$_{field_a}$ in map `{ ( id$_{field\_b}$ ':' typeIR$_{field\_b}$ )$^*$ `}.

               Let typeIR'''$^{?*}$ be the resulting list.
            III. If typeIR'''$^?$ is defined, for all typeIR'''$^?$ in typeIR'''$^{?*}$:
               1. For each typeIR'''$^?$ in typeIR'''$^{?*}$:
                  - Let typeIR$_{field_b_non_default_aligned}$ be typeIR'''$^?$.

                  Let typeIR$_{field_b_non_default_aligned}^*$ be the resulting list.
               2. For each typeIR$_{field_a_aligned}$ in typeIR$_{field_a_aligned}^*$ and typeIR$_{field_b_non_default_aligned}$ in typeIR$_{field_b_non_default_aligned}^*$:
                  - Let casts castResult be the result of checking if typeIR$_{field_a_aligned}$ can be implicitly cast to typeIR$_{field_b_non_default_aligned}$.

                  Let castResult* be the resulting list.
               3. If castResult matches pattern OK %, for all castResult in castResult*:
                  1. For each castResult in castResult*:
                     - Let OK typeIR$_{cast}^*$ be castResult.

                     Let typeIR$_{cast}^{**}$ be the resulting list.
                  2. Let `{ id$_{default}^*$ `} be the difference of the sets `{ id$_{field\_b}^*$ `} and `{ id$_{field\_a}^*$ `}.
                  3. For each id$_{default}$ in id$_{default}^*$:
                     - Let typeIR''''$^?$ be the value of id$_{default}$ in map `{ ( id$_{field\_b}$ ':' typeIR$_{field\_b}$ )$^*$ `}.

Let typeIR''''?* be the resulting list.

4. If typeIR''''? is defined, for all typeIR''''? in typeIR''''?*:
    1. For each typeIR''''? in typeIR''''?*:
        - Let $typeIR_{field_b_default}$ be typeIR''''?.

        Let $typeIR_{field_b_default}$* be the resulting list.

    2. If $typeIR_{field_b_default}$ can be default-initialized, for all $typeIR_{field_b_default}$ in $typeIR_{field_b_default}$*:
        1. Let $typeIR_{cast_agg}$* be $chain_casts_recordTypeIR_default(fieldTypeIR$a*, $typeIR_{cast}$*, structTypeIR).
        2. The result is casts OK $typeIR_{cast_agg}$*.

2. Else if let headerTypeIR of type headerTypeIR be typeIR':
    a. Let ($annotationList_a$ $typeIR_{field_a}$ id_{field_a} ';')* be $fieldTypeIR_a$*.
    b. Let HEADER _ `< _* `> `{ ( _ $typeIR_{field\_b}$ $id_{field\_b}$ ';' )* `} be headerTypeIR.
    c. If `{ $id_{field\_a}$* `} is a subset of `{ $id_{field\_b}$* `}:
        i. If the length of id_{field_a}* is equal to the length of $typeIR_{field_a}$*, for all id_{field_a} in id_{field_a}*:
            A. For each id_{field_a} in id_{field_a}*:
                - Let typeIR''? be the value of id_{field_a} in map `{ ( $id_{field\_a}$ ':' $typeIR_{field\_a}$ )* `}.

            Let typeIR''?* be the resulting list.

            B. If typeIR''? is defined, for all typeIR''? in typeIR''?*:
                I. For each typeIR''? in typeIR''?*:
                    - Let $typeIR_{field_a_aligned}$ be typeIR''?.

                Let $typeIR_{field_a_aligned}$* be the resulting list.

                II. For each id_{field_a} in id_{field_a}*:
                    - Let typeIR''? be the value of id_{field_a} in map `{ ( $id_{field\_b}$ ':' $typeIR_{field\_b}$ )* `}.

                Let typeIR''?* be the resulting list.

                III. If typeIR''? is defined, for all typeIR''? in typeIR''?*:
                    1. For each typeIR''? in typeIR''?*:
                        - Let $typeIR_{field_b_non_default_aligned}$ be typeIR''?.

                    Let $typeIR_{field_b_non_default_aligned}$* be the resulting list.

                    2. For each $typeIR_{field_a_aligned}$ in $typeIR_{field_a_aligned}$* and $typeIR_{field_b_non_default_aligned}$ in $typeIR_{field_b_non_default_aligned}$*:
                        - Let casts castResult be the result of checking if $typeIR_{field_a_aligned}$ can be implicitly cast to $typeIR_{field_b_non_default_aligned}$.

                    Let castResult* be the resulting list.

                    3. If castResult matches pattern OK %, for all castResult in castResult*:
                        1. For each castResult in castResult*:

- Let OK $typeIR_{cast}^{*}$ be castResult.

Let $typeIR_{cast}^{**}$ be the resulting list.

2. Let `{ $id_{default}^{*}$ `} be the difference of the sets `{ $id_{field\_b}^{*}$ `} and `{ $id_{field\_a}^{*}$ `}.
3. For each $id_{default}$ in $id_{default}^{*}$:

   - Let $typeIR''''^{?}$ be the value of $id_{default}$ in map `{ ( $id_{field\_b}$ ':' $typeIR_{field\_b}$ )* `}.

   Let $typeIR''''^{?*}$ be the resulting list.

4. If $typeIR''''^{?}$ is defined, for all $typeIR''''^{?}$ in $typeIR''''^{?*}$:
   1. For each $typeIR''''^{?}$ in $typeIR''''^{?*}$:

      - Let $typeIR_{field_b_default}$ be $typeIR''''^{?}$.

      Let $typeIR_{field_b_default}^{*}$ be the resulting list.

   2. If $typeIR_{field_b_default}$ can be default-initialized, for all $typeIR_{field_b_default}$ in $typeIR_{field_b_default}^{*}$:
      1. Let $typeIR_{cast_agg}^{*}$ be $chain_casts_recordTypeIR_default(fieldTypeIR_a^{*}, typeIR_{cast}^{*}, headerTypeIR).
      2. The result is casts OK $typeIR_{cast_agg}^{*}$.

When no implicit cast exists

If none of the above applies, the two types cannot be implicitly cast.

1. Otherwise:
   a. Checking if typeIR can be implicitly cast to typeIR' (when unequal): the result is casts FAIL.

18.4. Cast insertion

When typing a $P4_{16}$ program to a $P4_{IR}$ program, all implicit casts are made explicit. This process is called *cast insertion*. A cast may be inserted such that an expression matches the expected type, and it may also be inserted to ensure that two expressions have the same type.

Since a cast may take several steps, as described in Section Section 18.1, one cast is inserted per step of the chain.

typedExpressionIR cast through $typeIR^{*}$

1. If $typeIR^{*}$ matches pattern []: return typedExpressionIR.
2. Else:
   a. Let $typeIR_{h} :: typeIR_{t}^{*}$ be $typeIR^{*}$.
   b. Let a pair of type _ and compile-time known-ness ctk be the note of typedExpressionIR.
   c. Let $typedExpressionIR_{cast}$ be `( $typeIR_{h}$ `) typedExpressionIR annotated with a pair of type $typeIR_{h}$ and compile-time known-ness ctk.
   d. Return $typedExpressionIR_{cast}$ cast through $typeIR_{t}^{*}$.

18.4.1. Casts to match expected type

The below function inserts casts to match expected types.

typedExpressionIR implicitly cast to $\text{typeIR}_{\text{to}}$

typedExpressionIR implicitly cast to $\text{typeIR}_{\text{to}}$

1. Let a pair of type typeIR and compile-time known-ness _ be the note of typedExpressionIR.
2. Let casts castResult be the result of checking if typeIR can be implicitly cast to $\text{typeIR}_{\text{to}}$.
3. If let OK $\text{typeIR}_{\text{cast}}{}^{*}$ be castResult:
    a. Let $\text{typedExpressionIR}_{\text{cast}}$ be typedExpressionIR cast through $\text{typeIR}_{\text{cast}}{}^{*}$.
    b. Return $\text{typedExpressionIR}_{\text{cast}}$.
4. Otherwise: return ·.

18.4.2. Casts to ensure type equality

The below function inserts casts to ensure that two expressions have the same type.

$\text{typedExpressionIR}_{\text{l}}$ and $\text{typedExpressionIR}_{\text{r}}$ implicitly cast to equal types

$\text{typedExpressionIR}_{\text{l}}$ and $\text{typedExpressionIR}_{\text{r}}$ implicitly cast to equal types

$\text{typedExpressionIR}_{\text{l}}$ and $\text{typedExpressionIR}_{\text{r}}$ implicitly cast to equal types

1. Let a pair of type typeIR_{l} and compile-time known-ness _ be the note of $\text{typedExpressionIR}_{\text{l}}$.
2. Let a pair of type typeIR_{r} and compile-time known-ness _ be the note of $\text{typedExpressionIR}_{\text{r}}$.
3. Let casts castResult be the result of checking if typeIR_{l} can be implicitly cast to typeIR_{r}.
4. If let OK $\text{typeIR}_{\text{cast}}{}^{*}$ be castResult:
    a. Let $\text{typedExpressionIR}_{\text{l_cast}}$ be $\text{typedExpressionIR}_{\text{l}}$ cast through $\text{typeIR}_{\text{cast}}{}^{*}$.
    b. Return ($\text{typedExpressionIR}_{\text{l_cast}}$, $\text{typedExpressionIR}_{\text{r}}$).
5. Else:
    a. Let casts castResult' be the result of checking if typeIR_{r} can be implicitly cast to typeIR_{l}.
    b. If let OK $\text{typeIR}_{\text{cast}}{}^{*}$ be castResult':
        i. Let $\text{typedExpressionIR}_{\text{r_cast}}$ be $\text{typedExpressionIR}_{\text{r}}$ cast through $\text{typeIR}_{\text{cast}}{}^{*}$.
        ii. Return ($\text{typedExpressionIR}_{\text{l}}$, $\text{typedExpressionIR}_{\text{r_cast}}$).

c. Else:
    i. Let typedExpressionIR$_{l_cast}$ be typedExpressionIR$_{l}$ with surrounding serializable enums reduced.
    ii. Try:
        A. If typedExpressionIR$_{l}$ is not equal to typedExpressionIR$_{l_cast}$:
            I. Return typedExpressionIR$_{l_cast}$ and typedExpressionIR$_{r}$ implicitly cast to equal types.
    iii. Then, try:
        A. If typedExpressionIR$_{l}$ is equal to typedExpressionIR$_{l_cast}$:
            I. Let typedExpressionIR$_{r_cast}$ be typedExpressionIR$_{r}$ with surrounding serializable enums reduced.
            II. If typedExpressionIR$_{r}$ is not equal to typedExpressionIR$_{r_cast}$:
                1. Return typedExpressionIR$_{l}$ and typedExpressionIR$_{r_cast}$ implicitly cast to equal types.
6. Otherwise: return ·.

18.4.3. Serializable enum reduction

Casting and operator typing may require reducing serializable enums to their underlying types. A single expression is reduced with:

typedExpressionIR with surrounding serializable enums reduced

1. Let typeIR be the type of typedExpressionIR.
2. Let ctk be the compile-time known-ness of typedExpressionIR.
3. Let typeIR' be typeIR with typedefs unrolled.
4. If let ENUM _ `< typeIR$_{underlying}$ `> `{ _ $^{*}$ `} of type serializableEnumTypeIR be typeIR':
    a. Let typedExpressionIR$_{reduced}$ be `( typeIR$_{underlying}$ `) typedExpressionIR annotated with a pair of type typeIR$_{underlying}$ and compile-time known-ness ctk.
    b. Return typedExpressionIR$_{reduced}$.
5. Otherwise: return typedExpressionIR.

A single operand is reduced until a condition on its type holds with:

The result of reducing serializable enums in typedExpressionIR until $check is satisfied

1. Let a pair of type typeIR and compile-time known-ness ctk be the note of typedExpressionIR.
2. If $check(typeIR): return typedExpressionIR.
3. Else:
    a. Let typeIR' be typeIR with typedefs unrolled.
    b. If let ENUM _ `< typeIR$_{underlying}$ `> `{ _ $^{*}$ `} of type serializableEnumTypeIR be typeIR':
        i. Let typedExpressionIR$_{cast}$ be `( typeIR$_{underlying}$ `) typedExpressionIR annotated with a pair of type typeIR$_{underlying}$ and compile-time known-ness ctk.
        ii. Return the result of reducing serializable enums in typedExpressionIR$_{cast}$ until $check is satisfied.
4. Otherwise: return ·.

A pair of operands is reduced until a condition on both types holds with:

The result of reducing serializable enums in typedExpressionIR$_l$ and typedExpressionIR$_r$ until $check is satisfied

1. Let a pair of type typeIR$_l$ and compile-time known-ness ctk be the note of typedExpressionIR$_l$.
2. Let a pair of type typeIR$_r$ and compile-time known-ness _ be the note of typedExpressionIR$_r$.
3. If $check(typeIR$_l$, typeIR$_r$):
    a. Return (typedExpressionIR$_l$, typedExpressionIR$_r$).
4. Else:
    a. Try:
        i. Let typeIR be typeIR$_l$ with typedefs unrolled.
        ii. If let ENUM _ `< typeIR$_{l\_underlying}$ `> `{ _* `} of type serializableEnumTypeIR be typeIR:
            A. Let typedExpressionIR$_{l_cast}$ be `( typeIR$_{l\_underlying}$ `) typedExpressionIR$_l$ annotated with a pair of type typeIR$_{l_underlying}$ and compile-time known-ness ctk.
            B. Return the result of reducing serializable enums in typedExpressionIR$_{l_cast}$ and typedExpressionIR$_r$ until $check is satisfied.
    b. Then, try:
        i. If $unroll_typeIR(typeIR$_l$) does not have type serializableEnumTypeIR:
            A. Let typeIR be typeIR$_r$ with typedefs unrolled.
            B. If let ENUM _ `< typeIR$_{r\_underlying}$ `> `{ _* `} of type serializableEnumTypeIR be typeIR:
                I. Let typedExpressionIR$_{r_cast}$ be `( typeIR$_{r\_underlying}$ `) typedExpressionIR$_r$ annotated with a pair of type typeIR$_{r_underlying}$ and compile-time known-ness _.
                II. Return the result of reducing serializable enums in typedExpressionIR$_l$ and typedExpressionIR$_{r_cast}$ until $check is satisfied.
5. Otherwise: return ·.

18.5. Performing casts

Cast operations are evaluated with:

value casted to type typeIR

value casted to type typeIR

1. If let boolValue of type boolValue be value:
    a. Let typeIR$_{unroll}$ be typeIR with typedefs unrolled.
    b. Return boolValue casted to type typeIR$_{unroll}$.
2. Else if let integerValue of type integerValue be value:
    a. Let typeIR$_{unroll}$ be typeIR with typedefs unrolled.
    b. Return integerValue casted to type typeIR$_{unroll}$.
3. Else if let errorValue of type errorValue be value:
    a. Let typeIR$_{unroll}$ be typeIR with typedefs unrolled.

    b. Return errorValue casted to type $typeIR_{unroll}$.

4. Else if let MATCH_KIND '.' nameIR of type matchKindValue be value:
    a. Check that MATCH_KIND is equal to typeIR with typedefs unrolled.
    b. Return MATCH_KIND '.' nameIR.
5. Else if let stringValue of type stringValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return stringValue casted to type $typeIR_{unroll}$.
6. Else if let listValue of type listValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return listValue casted to type $typeIR_{unroll}$.
7. Else if let tupleValue of type tupleValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return tupleValue casted to type $typeIR_{unroll}$.
8. Else if let arrayValue of type arrayValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return arrayValue casted to type $typeIR_{unroll}$.
9. Else if let headerStackValue of type headerStackValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return headerStackValue casted to type $typeIR_{unroll}$.
10. Else if let structValue of type structValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return structValue casted to type $typeIR_{unroll}$.
11. Else if let headerValue of type headerValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return headerValue casted to type $typeIR_{unroll}$.
12. Else if let headerUnionValue of type headerUnionValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return headerUnionValue casted to type $typeIR_{unroll}$.
13. Else if let enumValue of type enumValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return enumValue casted to type $typeIR_{unroll}$.
14. Else if let sequenceValue of type sequenceValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return sequenceValue casted to type $typeIR_{unroll}$.
15. Else if let recordValue of type recordValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return recordValue casted to type $typeIR_{unroll}$.
16. Else if let defaultValue of type defaultValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return DEFAULT casted to type $typeIR_{unroll}$.

17. Else if value has type invalidHeaderValue:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return '{#}' casted to type $typeIR_{unroll}$.
18. Else if let setValue of type setValue be value:
    a. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
    b. Return setValue casted to type $typeIR_{unroll}$.

From a bool value

boolValue casted to type typeIR'

1. Try:
    a. Let $_{B}$ b be boolValue.
    b. If let boolTypeIR of type boolTypeIR be typeIR': return $_{B}$ b.
2. Then, try:
    a. If let BIT `< w `> of type fixedBitTypeIR be typeIR':
        i. If boolValue is equal to $_{B}$ true: return w W 1.
        ii. Else if boolValue is equal to $_{B}$ false: return w W 0.
    b. Else if let TYPE _ typeIR of type newTypeIR be typeIR':
        i. Return boolValue casted to type typeIR.
    c. Else if let SET `< typeIR `> of type setTypeIR be typeIR':
        i. Let value be boolValue casted to type typeIR.
        ii. Return SET `{ value `}.

From an integer value

integerValue casted to type typeIR'

1. If let boolTypeIR of type boolTypeIR be typeIR':
    a. Check that integerValue has type integerLiteral.
    b. Let integerLiteral be integerValue.
    c. If let D i be integerLiteral: return $_{B}$ i =/= 0.
    d. Else if let w W i be integerLiteral: return $_{B}$ i =/= 0.
2. Else if let intTypeIR of type intTypeIR be typeIR':
    a. Check that integerValue has type integerLiteral.
    b. Let integerLiteral be integerValue.
    c. If let D i be integerLiteral: return D i.
    d. Else if let _ W i be integerLiteral: return D i.
    e. Else:
        i. Let w S i be integerLiteral.
        ii. Let int be the two's complement representation of bitstring i of width w.
        iii. Return D int.

3. Else if let BIT `< $w_{to}$ `> of type fixedBitTypeIR be typeIR':
    a. Check that integerValue has type integerLiteral.
    b. Let integerLiteral be integerValue.
    c. If let D i be integerLiteral:
        i. Let i_{cast} be the bitstring of width w_{to} representing the integer i.
        ii. Return w_{to} W i_{cast}.
    d. Else if let _ W i be integerLiteral:
        i. Let i_{cast} be the bitstring of width w_{to} representing the integer i.
        ii. Return w_{to} W i_{cast}.
    e. Else:
        i. Let w_{from} S i be integerLiteral.
        ii. Let i_{from} be the two's complement representation of bitstring i of width w_{from}.
        iii. Let i_{cast} be the bitstring of width w_{to} representing the integer i_{from}.
        iv. Return w_{to} W i_{cast}.
4. Else if let INT `< $w_{to}$ `> of type fixedIntTypeIR be typeIR':
    a. Check that integerValue has type integerLiteral.
    b. Let integerLiteral be integerValue.
    c. If let D i be integerLiteral:
        i. Let i_{cast} be the bitstring of width w_{to} representing the integer i.
        ii. Return w_{to} S i_{cast}.
    d. Else if let _ W i be integerLiteral:
        i. Let i_{cast} be the bitstring of width w_{to} representing the integer i.
        ii. Return w_{to} S i_{cast}.
    e. Else:
        i. Let w_{from} S i be integerLiteral.
        ii. Let i_{from} be the two's complement representation of bitstring i of width w_{from}.
        iii. Let i_{cast} be the bitstring of width w_{to} representing the integer i_{from}.
        iv. Return w_{to} S i_{cast}.
5. Else if let VARBIT `< $w_{max}$ `> of type varBitTypeIR be typeIR':
    a. Check that integerValue matches pattern % '.' % V %.
    b. Let $w_{max'}$ '.' w V i be integerValue.
    c. Check that w_{max} is equal to $w_{max'}$.
    d. Return w_{max} '.' w V i.
6. Else if let TYPE _ typeIR of type newTypeIR be typeIR':
    a. Check that integerValue has type integerLiteral.
    b. Let integerLiteral be integerValue.
    c. If let D i be integerLiteral: return D i casted to type typeIR.
    d. Else if let w W i be integerLiteral: return w W i casted to type typeIR.
    e. Else:
        i. Let w S i be integerLiteral.
        ii. Return w S i casted to type typeIR.

7. Else if let enumTypeIR of type enumTypeIR be typeIR':
    a. Check that integerValue has type integerLiteral.
    b. Let integerLiteral be integerValue.
    c. If let w W i be integerLiteral:
        i. Return w W i casted to enum type enumTypeIR.
    d. Else if let w S i be integerLiteral:
        i. Return w S i casted to enum type enumTypeIR.
8. Else if let SET `< typeIR `> of type setTypeIR be typeIR':
    a. Check that integerValue has type integerLiteral.
    b. Let integerLiteral be integerValue.
    c. If let D i be integerLiteral:
        i. Let value be D i casted to type typeIR.
        ii. Return SET `{ value `}.
    d. Else if let w W i be integerLiteral:
        i. Let value' be w W i casted to type typeIR.
        ii. Return SET `{ value' `}.
    e. Else:
        i. Let w S i be integerLiteral.
        ii. Let value'' be w S i casted to type typeIR.
        iii. Return SET `{ value'' `}.

From an error value

errorValue casted to type typeIR'

1. Try:
    a. Let ERROR '.' nameIR be errorValue.
    b. If let errorTypeIR of type errorTypeIR be typeIR': return ERROR '.' nameIR.
2. Then, try:
    a. Check that typeIR' has type setTypeIR.
    b. Let SET `< typeIR `> be typeIR'.
    c. Let value be errorValue casted to type typeIR.
    d. Return SET `{ value `}.

From a string value

stringValue casted to type typeIR

1. Check that typeIR has type stringTypeIR.
2. Let stringTypeIR be typeIR.
3. Return stringValue.

From a list value

LIST `[ value* `] casted to type typeIR'

1. Check that typeIR' has type listTypeIR.
2. Let LIST `< typeIR `> be typeIR'.
3. For each value in value*:

   - Let value' be value casted to type typeIR.

   Let value'* be the resulting list.
4. Return LIST `[ value'* `].

From a tuple value

tupleValue casted to type typeIR'

1. Try:
   a. Let TUPLE `( value* `) be tupleValue.
   b. If let TUPLE `< typeIR* `> of type tupleTypeIR be typeIR':
      i. If the length of typeIR* is equal to the length of value*:
         A. For each typeIR in typeIR* and value in value*:

            - Let value' be value casted to type typeIR.

            Let value'* be the resulting list.

         B. Return TUPLE `( value'* `).
2. Then, try:
   a. Check that typeIR' has type setTypeIR.
   b. Let SET `< typeIR `> be typeIR'.
   c. Let value'' be tupleValue casted to type typeIR.
   d. Return SET `{ value'' `}.

From an array value

ARRAY `[ value* `] casted to type typeIR

1. Check that typeIR has type arrayTypeIR.
2. Return ARRAY `[ value* `].

From a header stack value

HEADER_STACK `[ value* `(n_{idx} `) `] casted to type typeIR

1. Check that typeIR has type headerStackTypeIR.

2. Return HEADER_STACK `[ value$^{*}$ `(n_{idx} `) `].

From a struct value

STRUCT typeId' `{ ( $value_{field}$ $nameIR_{field}$ ';' )$^{*}$ `} casted to type typeIR

1. Check that typeIR has type structTypeIR.
2. Let STRUCT typeId `< _$^{*}$ `> `{ _$^{*}$ `} be typeIR.
3. Check that typeId is equal to typeId'.
4. Return STRUCT typeId `{ ( $value_{field}$ $nameIR_{field}$ ';' )$^{*}$ `}.

From a header value

HEADER typeId' `{ b ';' ( $value_{field}$ $nameIR_{field}$ ';' )$^{*}$ `} casted to type typeIR

1. Check that typeIR has type headerTypeIR.
2. Let HEADER typeId `< _$^{*}$ `> `{ _$^{*}$ `} be typeIR.
3. Check that typeId is equal to typeId'.
4. Return HEADER typeId `{ b ';' ( $value_{field}$ $nameIR_{field}$ ';' )$^{*}$ `}.

From a header union value

HEADER_UNION typeId' `{ ( $value_{field}$ $nameIR_{field}$ ';' )$^{*}$ `} casted to type typeIR

1. Check that typeIR has type headerUnionTypeIR.
2. Let HEADER_UNION typeId `< _$^{*}$ `> `{ _$^{*}$ `} be typeIR.
3. Check that typeId is equal to typeId'.
4. Return HEADER_UNION typeId `{ ( $value_{field}$ $nameIR_{field}$ ';' )$^{*}$ `}.

From an enum value

enumValue casted to type typeIR'

1. Try:
    a. If let ENUM typeId `{ _$^{*}$ `} of type simpleEnumTypeIR be typeIR':
        i. If let typeId' '.' nameIR be enumValue:
            A. If typeId is equal to typeId': return typeId '.' nameIR.
    b. Else if let ENUM typeId `< _ `> `{ _$^{*}$ `} of type serializableEnumTypeIR be typeIR':
        i. If let typeId' '.' nameIR '.' value be enumValue:
            A. If typeId is equal to typeId':
                I. Return typeId '.' nameIR '.' value.
    c. Else if let SET `< typeIR `> of type setTypeIR be typeIR':
        i. Let value' be enumValue casted to type typeIR.

ii. Return SET `{ value' `}.

2. Then, try:
   a. Check that enumValue matches pattern % '.' % '.' %.
   b. Let _ '.' _ '.' value be enumValue.
   c. Check that typeIR' does not have type serializableEnumTypeIR and typeIR' does not have type setTypeIR.
   d. Return value casted to type typeIR'.

To an enum value

Casting an underlying value to a serializable enum:

value casted to enum type enumTypeIR

1. Check that enumTypeIR has type serializableEnumTypeIR.
2. Let ENUM typeId `< typeIR `> `{ valueFieldIR* `} be enumTypeIR.
3. Return $cast_to_enum'(typeId, typeIR, valueFieldIR*, value).

$cast_to_enum'(typeId, typeIR, valueFieldIR*, $value_{target}$)

1. If valueFieldIR* matches pattern []:
   a. Let $value_{idx}$ be $value_{target}$ casted to type typeIR.
   b. Let $id_{unnamed}$ be "__UNSPECIFIED".
   c. Return typeId '.' $id_{unnamed}$ '.' $value_{idx}$.
2. Else:
   a. Let $valueFieldIR_h$ :: $valueFieldIR_t$* be valueFieldIR*.
   b. Let nameIR '=' value ';' be $valueFieldIR_h$.
   c. If value '==' $value_{target}$:
      i. Return typeId '.' nameIR '.' value.
   d. Else:
      i. Return $cast_to_enum'(typeId, typeIR, valueFieldIR_t^*, value_{target})$.

From a sequence value

sequenceValue casted to type typeIR'

1. If let SET `< typeIR `> of type setTypeIR be typeIR':
   a. Let value' be sequenceValue casted to type typeIR.
   b. Return SET `{ value' `}.
2. Else if let LIST `< typeIR `> of type listTypeIR be typeIR':
   a. Check that sequenceValue matches pattern SEQ `( % `).
   b. Let SEQ `( value* `) be sequenceValue.
   c. For each value in value*:
      •

Let value'' be value casted to type typeIR.

Let value''* be the resulting list.

d. Return LIST `[ value''* `].

3. Else if let TUPLE `< typeIR* `> of type tupleTypeIR be typeIR':

a. If let SEQ `( value* `) be sequenceValue:

i. Check that the length of typeIR* is equal to the length of value*.

ii. For each typeIR in typeIR* and value in value*:

- Let value''' be value casted to type typeIR.

Let value'''* be the resulting list.

iii. Return TUPLE `( value'''* `).

b. Else:

i. Let SEQ `( value* ',' '...' `) be sequenceValue.

ii. Let ($typeIR_{given}$*, $typeIR_{rest}$*) be $partition_<typeIR>(typeIR*, the length of value*).

iii. Check that the length of $typeIR_{given}$* is equal to the length of value*.

iv. For each $typeIR_{given}$ in $typeIR_{given}$* and value in value*:

- Let $value_{given}$ be value casted to type $typeIR_{given}$.

Let $value_{given}$* be the resulting list.

v. For each $typeIR_{rest}$ in $typeIR_{rest}$*:

- Let $value_{rest}$ be the default value for type $typeIR_{rest}$.

Let $value_{rest}$* be the resulting list.

vi. Let $value_{cast}$* be $value_{given}$* concatenated with $value_{rest}$*.

vii. Return TUPLE `( $value_{cast}$* `).

4. Else if let sequenceTypeIR of type sequenceTypeIR be typeIR':

a. If let SEQ `< typeIR* `> be sequenceTypeIR:

i. Check that sequenceValue matches pattern SEQ `( % `).

ii. Let SEQ `( value* `) be sequenceValue.

iii. Check that the length of typeIR* is equal to the length of value*.

iv. For each typeIR in typeIR* and value in value*:

- Let value'''' be value casted to type typeIR.

Let value''''* be the resulting list.

v. Return SEQ `( value''''* `).

b. Else:

i. Let SEQ `< typeIR* ',' '...' `> be sequenceTypeIR.

ii. Check that sequenceValue matches pattern SEQ `( % ',' '...' `).

iii. Let SEQ `( value* ',' '...' `) be sequenceValue.

iv. Check that the length of typeIR* is equal to the length of value*.

v. For each typeIR in typeIR* and value in value*:

- Let value_cast be value casted to type typeIR.

Let value_cast* be the resulting list.

vi. Return SEQ `( value_cast* ',' '...' `).

5. Else if let ARRAY typeIR `[ n_size `] of type arrayTypeIR be typeIR':

a. If let SEQ `( value* `) be sequenceValue:

i. For each value in value*:

- Let value_cast be value casted to type typeIR.

Let value_cast* be the resulting list.

ii. Return ARRAY `[ value_cast* `].

b. Else:

i. Let SEQ `( value* ',' '...' `) be sequenceValue.

ii. For each value in value*:

- Let value_given be value casted to type typeIR.

Let value_given* be the resulting list.

iii. Let i be n_size - the length of value*.

iv. Check that i has type nat.

v. Let n_rest be i.

vi. Let value'''' be the default value for type typeIR.

vii. Let value_rest* be a list of n_rest copies of value''''.

viii. Let value_cast* be value_given* concatenated with value_rest*.

ix. Return ARRAY `[ value_cast* `].

6. Else if let HEADER_STACK typeIR `[ n_size `] of type headerStackTypeIR be typeIR':

a. If let SEQ `( value* `) be sequenceValue:

i. For each value in value*:

- Let value_cast be value casted to type typeIR.

Let value_cast* be the resulting list.

ii. Let n_idx be the length of value*.

iii. Return HEADER_STACK `[ value_cast* `(n_idx `) `].

b. Else:

i. Let SEQ `( value* ',' '...' `) be sequenceValue.

ii. For each value in value*:

- Let value_given be value casted to type typeIR.

Let value_given* be the resulting list.

iii. Let i be n_size - the length of value*.

iv. Check that i has type nat.

v. Let n_{rest} be i.

vi. Let value'''''' be the default value for type typeIR.

vii. Let $value_{rest}^*$ be a list of n_{rest} copies of value''''''.

viii. Let $value_{cast}^*$ be $value_{given}^*$ concatenated with $value_{rest}^*$.

ix. Let n_{idx} be the length of $value^*$.

x. Return HEADER_STACK `[ $value_{cast}^*$ `(n_{idx} `) `].

7. Else if let STRUCT typeId `< _* `> `{ ( _ $typeIR_{field}$ $nameIR_{field}$ ';' )* `} of type structTypeIR be typeIR':

a. If let SEQ `( $value^*$ `) be sequenceValue:

i. Check that the length of $typeIR_{field}^*$ is equal to the length of $value^*$.

ii. For each $typeIR_{field}$ in $typeIR_{field}^*$ and value in $value^*$:

- Let $value_{cast}$ be value casted to type $typeIR_{field}$.

Let $value_{cast}^*$ be the resulting list.

iii. Return STRUCT typeId `{ ( $value_{cast}$ $nameIR_{field}$ ';' )* `}.

b. Else:

i. Let SEQ `( $value^*$ ',' '...' `) be sequenceValue.

ii. Let ($typeIR_{field_given}^*$, $typeIR_{field_rest}^*$) be $partition_<typeIR>($typeIR_{field}^*$, the length of $value^*$).

iii. Check that the length of $typeIR_{field_given}^*$ is equal to the length of $value^*$.

iv. For each $typeIR_{field_given}$ in $typeIR_{field_given}^*$ and value in $value^*$:

- Let $value_{given}$ be value casted to type $typeIR_{field_given}$.

Let $value_{given}^*$ be the resulting list.

v. For each $typeIR_{field_rest}$ in $typeIR_{field_rest}^*$:

- Let $value_{rest}$ be the default value for type $typeIR_{field_rest}$.

Let $value_{rest}^*$ be the resulting list.

vi. Let $value_{cast}^*$ be $value_{given}^*$ concatenated with $value_{rest}^*$.

vii. Check that the length of $nameIR_{field}^*$ is equal to the length of $value_{cast}^*$.

viii. Return STRUCT typeId `{ ( $value_{cast}$ $nameIR_{field}$ ';' )* `}.

8. Else if let HEADER typeId `< _* `> `{ ( _ $typeIR_{field}$ $nameIR_{field}$ ';' )* `} of type headerTypeIR be typeIR':

a. If let SEQ `( $value^*$ `) be sequenceValue:

i. Check that the length of $typeIR_{field}^*$ is equal to the length of $value^*$.

ii. For each $typeIR_{field}$ in $typeIR_{field}^*$ and value in $value^*$:

- Let $value_{cast}$ be value casted to type $typeIR_{field}$.

Let $value_{cast}^*$ be the resulting list.

iii. Return HEADER typeId `{ true ';' ( $value_{cast}$ $nameIR_{field}$ ';' )* `}.

b. Else:

i. Let SEQ `( $value^*$ ',' '...' `) be sequenceValue.

ii. Let ($typeIR_{field_given}^*$, $typeIR_{field_rest}^*$) be $partition_<typeIR>($typeIR_{field}^*$, the length of $value^*$).

iii. Check that the length of $typeIR_{field_given}^*$ is equal to the length of $value^*$.

iv. For each $typeIR_{field_given}$ in $typeIR_{field_given}^{*}$ and value in $value^{*}$:

- Let $value_{given}$ be value casted to type $typeIR_{field_given}$.

Let $value_{given}^{*}$ be the resulting list.

v. For each $typeIR_{field_rest}$ in $typeIR_{field_rest}^{*}$:

- Let $value_{rest}$ be the default value for type $typeIR_{field_rest}$.

Let $value_{rest}^{*}$ be the resulting list.

vi. Let $value_{cast}^{*}$ be $value_{given}^{*}$ concatenated with $value_{rest}^{*}$.

vii. Check that the length of $nameIR_{field}^{*}$ is equal to the length of $value_{cast}^{*}$.

viii. Return HEADER typeId `{ true ';' ( $value_{cast}$ $nameIR_{field}$ ';' )* `}.

From a record value

recordValue casted to type typeIR

1. If let STRUCT typeId `< _* `> `{ ( _ $typeIR_{t\_field}$ $id_{t\_field}$ ';' )* `} of type structTypeIR be typeIR:
   a. If let RECORD `{ ( $value_{field}$ $nameIR_{field}$ ';' )* `} be recordValue:
      i. For each id_{t_field} in $id_{t_field}^{*}$:
         - Let $value^{?}$ be the value of id_{t_field} in map `{ ( $nameIR_{field}$ ':' $value_{field}$ )* `}.

      Let $value^{?*}$ be the resulting list.

      ii. Check that $value^{?}$ is defined, for all $value^{?}$ in $value^{?*}$.
      iii. For each $value^{?}$ in $value^{?*}$:
         - Let $value_{field'}$ be $value^{?}$.

      Let $value_{field'}^{*}$ be the resulting list.

      iv. For each $typeIR_{t_field}$ in $typeIR_{t_field}^{*}$ and $value_{field'}$ in $value_{field'}^{*}$:
         - Let $value_{field_cast}$ be $value_{field'}$ casted to type $typeIR_{t_field}$.

      Let $value_{field_cast}^{*}$ be the resulting list.

      v. Return STRUCT typeId `{ ( $value_{field\_cast}$ $id_{t\_field}$ ';' )* `}.
   b. Else:
      i. Let RECORD `{ ( $value_{field}$ $nameIR_{field}$ ';' )* ',' '...' `} be recordValue.
      ii. For each id_{t_field} in $id_{t_field}^{*}$ and $typeIR_{t_field}$ in $typeIR_{t_field}^{*}$:
         - Let $value_{field_cast}$ be $cast_record_field($typeIR_{t_field}$, id_{t_field}, `{ ( $nameIR_{field}$ ':' $value_{field}$ )* `}).

      Let $value_{field_cast}^{*}$ be the resulting list.

      iii. Return STRUCT typeId `{ ( $value_{field\_cast}$ $id_{t\_field}$ ';' )* `}.
2. Else if let HEADER typeId `< _* `> `{ ( _ $typeIR_{t\_field}$ $id_{t\_field}$ ';' )* `} of type headerTypeIR be typeIR:
   a. If let RECORD `{ ( $value_{field}$ $nameIR_{field}$ ';' )* `} be recordValue:

i. For each id_{t_field} in $id_{t_field}^{*}$:

- Let $value^{?}$ be the value of id_{t_field} in map `{ ( $nameIR_{field}$ ':' $value_{field}$ )* `}.

Let $value^{?*}$ be the resulting list.

ii. Check that $value^{?}$ is defined, for all $value^{?}$ in $value^{?*}$.

iii. For each $value^{?}$ in $value^{?*}$:

- Let $value_{field'}$ be $value^{?}$.

Let $value_{field'}^{*}$ be the resulting list.

iv. For each $typeIR_{t_field}$ in $typeIR_{t_field}^{*}$ and $value_{field'}$ in $value_{field'}^{*}$:

- Let $value_{field_cast}$ be $value_{field'}$ casted to type $typeIR_{t_field}$.

Let $value_{field_cast}^{*}$ be the resulting list.

v. Return HEADER typeId `{ true ';' ( $value_{field\_cast}$ $id_{t\_field}$ ';' )* `}.

b. Else:

i. Let RECORD `{ ( $value_{field}$ $nameIR_{field}$ ';' )* ',' '...' `} be recordValue.

ii. For each id_{t_field} in $id_{t_field}^{*}$ and $typeIR_{t_field}$ in $typeIR_{t_field}^{*}$:

- Let $value_{field_cast}$ be $cast_record_field($typeIR_{t_field}$, id_{t_field}, `{ ( $nameIR_{field}$ ':' $value_{field}$ )* `}).

Let $value_{field_cast}^{*}$ be the resulting list.

iii. Return HEADER typeId `{ true ';' ( $value_{field\_cast}$ $id_{t\_field}$ ';' )* `}.

3. Else if let recordTypeIR of type recordTypeIR be typeIR:

a. If let RECORD `{ ( _ $typeIR_{t\_field}$ $id_{t\_field}$ ';' )* `} be recordTypeIR:

i. Check that recordValue matches pattern RECORD `{ % `}.

ii. Let RECORD `{ ( $value_{field}$ $nameIR_{field}$ ';' )* `} be recordValue.

iii. For each id_{t_field} in $id_{t_field}^{*}$:

- Let $value^{?}$ be the value of id_{t_field} in map `{ ( $nameIR_{field}$ ':' $value_{field}$ )* `}.

Let $value^{?*}$ be the resulting list.

iv. Check that $value^{?}$ is defined, for all $value^{?}$ in $value^{?*}$.

v. For each $value^{?}$ in $value^{?*}$:

- Let $value_{field'}$ be $value^{?}$.

Let $value_{field'}^{*}$ be the resulting list.

vi. For each $typeIR_{t_field}$ in $typeIR_{t_field}^{*}$ and $value_{field'}$ in $value_{field'}^{*}$:

- Let $value_{field_cast}$ be $value_{field'}$ casted to type $typeIR_{t_field}$.

Let $value_{field_cast}^{*}$ be the resulting list.

vii. Return RECORD `{ ( $value_{field\_cast}$ $id_{t\_field}$ ';' )* `}.

b. Else:

i. Let RECORD `{ ( _ $typeIR_{t\_field}$ $id_{t\_field}$ ';' )* ',' '...' `} be recordTypeIR.

ii. Check that recordValue matches pattern RECORD `{ % ',' '...' `}.

iii. Let RECORD `{ ( $value_{field}$ $nameIR_{field}$ ':' )* ',' '...' `} be recordValue.

iv. For each id_{t_field} in id_{t_field}*:

- Let $value^{?}$ be the value of id_{t_field} in map `{ ( $nameIR_{field}$ ':' $value_{field}$ )* `}.

Let $value^{?*}$ be the resulting list.

v. Check that $value^{?}$ is defined, for all $value^{?}$ in $value^{?*}$.

vi. For each $value^{?}$ in $value^{?*}$:

- Let $value_{field'}$ be $value^{?}$.

Let $value_{field'}$* be the resulting list.

vii. For each $typeIR_{t_field}$ in $typeIR_{t_field}$* and $value_{field'}$ in $value_{field'}$*:

- Let $value_{field_cast}$ be $value_{field'}$ casted to type $typeIR_{t_field}$.

Let $value_{field_cast}$* be the resulting list.

viii. Return RECORD `{ ( $value_{field\_cast}$ $id_{t\_field}$ ':' )* ',' '...' `}.

Each field of the target record is casted individually with:

$cast_record_field(typeIR, id, `{ ( $id_{field}$ ':' $value_{field}$ )* `})

1. Try:
   a. Let $value^{?}$ be the value of id in map `{ ( $id_{field}$ ':' $value_{field}$ )* `}.
   b. If $value^{?}$ is defined:
      i. Let $value_{found}$ be $value^{?}$.
      ii. Return $value_{found}$ casted to type typeIR.
2. Then, try:
   a. Check that the value of id in map `{ ( $id_{field}$ ':' $value_{field}$ )* `} is none.
   b. Return the default value for type typeIR.

From a default value

defaultValue casted to type typeIR

1. Return the default value for type typeIR.

From an invalid header value

'{#}' casted to type typeIR

1. If let invalidHeaderTypeIR of type invalidHeaderTypeIR be typeIR: return '{#}'.
2. Else if let headerTypeIR of type headerTypeIR be typeIR:
   a. Return the default value for type headerTypeIR.

3. Else if let headerUnionTypeIR of type headerUnionTypeIR be typeIR:
    a. Return the default value for type headerUnionTypeIR.

From a set value

setValue casted to type typeIR'

1. Check that typeIR' has type setTypeIR.
2. Let SET `< typeIR `> be typeIR'.
3. If let SET `{ value `} be setValue:
    a. Let value' be value casted to type typeIR.
    b. Return SET `{ value' `}.
4. Else if let SET `{ $value_b$ '&&&' $value_m$ `} be setValue:
    a. Let $value_{b_cast}$ be $value_b$ casted to type typeIR.
    b. Let $value_{m_cast}$ be $value_m$ casted to type typeIR.
    c. Return SET `{ $value_{b\_cast}$ '&&&' $value_{m\_cast}$ `}.
5. Else if let SET `{ $value_l$ '..' $value_u$ `} be setValue:
    a. Let $value_{l_cast}$ be $value_l$ casted to type typeIR.
    b. Let $value_{u_cast}$ be $value_u$ casted to type typeIR.
    c. Return SET `{ $value_{l\_cast}$ '..' $value_{u\_cast}$ `}.

18.6. Type equivalence

P4 has generic types, thus to reason about equivalence of types, alpha-equivalence is used. Two types are alpha-equivalent if they are the same up to renaming of type variables.

The following relation defines alpha-equivalence of types:

Type_alpha:

- typeIR and typeIR' are the same type

Type_alpha dispatch:

1. Try:
    a. If let baseTypeIR of type baseTypeIR be typeIR:
        i. If let baseTypeIR' of type baseTypeIR be typeIR':
            A. If baseTypeIR is equal to baseTypeIR': goto baseTypeIR
    b. Else if let NAME typeId of type nameTypeIR be typeIR:
        i. If let NAME typeId' of type nameTypeIR be typeIR':
            A. If typeId is equal to typeId': goto nameTypeIR

2. Then, try:
    a. Goto typedefTypeIR
3. Then, try:
    a. If let TYPE typeId $typeIR_a$ of type newTypeIR be typeIR:
        i. If let TYPE typeId' $typeIR_b$ of type newTypeIR be typeIR':
            A. If typeId is equal to typeId': goto newTypeIR
    b. Else if let LIST `< $typeIR_a$ `> of type listTypeIR be typeIR:
        i. If let LIST `< $typeIR_b$ `> of type listTypeIR be typeIR': goto listTypeIR
    c. Else if let TUPLE `< $typeIR_a^*$ `> of type tupleTypeIR be typeIR:
        i. If let TUPLE `< $typeIR_b^*$ `> of type tupleTypeIR be typeIR': goto tupleTypeIR
    d. Else if let ARRAY $typeIR_a$ `[ $n_{size}$ `] of type arrayTypeIR be typeIR:
        i. If let ARRAY $typeIR_b$ `[ $n_{size'}$ `] of type arrayTypeIR be typeIR':
            A. If n_{size} is equal to $n_{size'}$: goto arrayTypeIR
    e. Else if let HEADER_STACK $typeIR_a$ `[ $n_{size}$ `] of type headerStackTypeIR be typeIR:
        i. If let HEADER_STACK $typeIR_b$ `[ $n_{size'}$ `] of type headerStackTypeIR be typeIR':
            A. If n_{size} is equal to $n_{size'}$: goto headerStackTypeIR
    f. Else if let STRUCT typeId `< $typeArgumentIR_a^*$ `> `{ $fieldTypeIR^*$ `} of type structTypeIR be typeIR:
        i. If let STRUCT typeId' `< $typeArgumentIR_b^*$ `> `{ _ $^*$ `} of type structTypeIR be typeIR':
            A. If typeId is equal to typeId': goto structTypeIR
    g. Else if let HEADER typeId `< $typeArgumentIR_a^*$ `> `{ $fieldTypeIR^*$ `} of type headerTypeIR be typeIR:
        i. If let HEADER typeId' `< $typeArgumentIR_b^*$ `> `{ _ $^*$ `} of type headerTypeIR be typeIR':
            A. If typeId is equal to typeId': goto headerTypeIR
    h. Else if let HEADER_UNION typeId `< $typeArgumentIR_a^*$ `> `{ $fieldTypeIR^*$ `} of type headerUnionTypeIR be typeIR:
        i. If let HEADER_UNION typeId' `< $typeArgumentIR_b^*$ `> `{ _ $^*$ `} of type headerUnionTypeIR be typeIR':
            A. If typeId is equal to typeId': goto headerUnionTypeIR
4. Then, try:
    a. Goto enumTypeIR
5. Then, try:
    a. If let EXTERN typeId `< $typeArgumentIR_a^*$ `> externMethodTypeDefEnv of type externObjectTypeIR be typeIR:
        i. Check that typeIR' has type externObjectTypeIR.
        ii. Let EXTERN typeId' `< $typeArgumentIR_b^*$ `> _ be typeIR'.
        iii. Check that typeId is equal to typeId'.
        iv. Goto externObjectTypeIR
    b. Else if let PARSER typeId `< typeArgumentIR$^*$ `> `( $parameterIR_a^*$ `) of type parserObjectTypeIR be typeIR:
        i. Check that typeIR' has type parserObjectTypeIR.
        ii. Let PARSER _ `< _ $^*$ `> `( $parameterIR_b^*$ `) be typeIR'.
        iii. Goto parserObjectTypeIR
    c. Else if let CONTROL typeId `< typeArgumentIR$^*$ `> `( $parameterIR_a^*$ `) of type controlObjectTypeIR be typeIR:
        i. Check that typeIR' has type controlObjectTypeIR.
        ii. Let CONTROL _ `< _ $^*$ `> `( $parameterIR_b^*$ `) be typeIR'.
        iii. Goto controlObjectTypeIR
    d. Else if let PACKAGE typeId `< typeArgumentIR$^*$ `> `{ $typeIR_a^*$ `} of type packageObjectTypeIR be typeIR:

i. Check that typeIR' has type packageObjectTypeIR.

ii. Let PACKAGE _ `< _* `> `{ $typeIR_b$* `} be typeIR'.

iii. Goto packageObjectTypeIR

e. Else if let tableObjectTypeIR of type tableObjectTypeIR be typeIR:

i. Check that typeIR' has type tableObjectTypeIR.

ii. Let tableObjectTypeIR' be typeIR'.

iii. Check that tableObjectTypeIR is equal to tableObjectTypeIR'.

iv. Goto tableObjectTypeIR

f. Else if let defaultTypeIR of type defaultTypeIR be typeIR:

i. Check that typeIR' has type defaultTypeIR.

ii. Let defaultTypeIR' be typeIR'.

iii. Goto defaultTypeIR

g. Else if let invalidHeaderTypeIR of type invalidHeaderTypeIR be typeIR:

i. Check that typeIR' has type invalidHeaderTypeIR.

ii. Let invalidHeaderTypeIR' be typeIR'.

iii. Goto invalidHeaderTypeIR

h. Else if let sequenceTypeIR of type sequenceTypeIR be typeIR:

i. Check that typeIR' has type sequenceTypeIR.

ii. Let sequenceTypeIR' be typeIR'.

iii. Goto sequenceTypeIR

i. Else if let recordTypeIR of type recordTypeIR be typeIR:

i. Check that typeIR' has type recordTypeIR.

ii. Let recordTypeIR' be typeIR'.

iii. Goto recordTypeIR

j. Else if let SET `< $typeIR_a$ `> of type setTypeIR be typeIR:

i. Check that typeIR' has type setTypeIR.

ii. Let SET `< $typeIR_b$ `> be typeIR'.

iii. Goto setTypeIR

k. Else if let tableMetadataTypeIR of type tableMetadataTypeIR be typeIR:

i. Check that typeIR' has type tableMetadataTypeIR.

ii. Let tableMetadataTypeIR' be typeIR'.

iii. Check that tableMetadataTypeIR is equal to tableMetadataTypeIR'.

iv. Goto tableMetadataTypeIR

The following relation defines alpha-equivalence of parameters:

ParameterType_alpha:

- annotationList direction $typeIR_a$ id parameterInitializerOptIR and annotationList' direction' $typeIR_b$ id' parameterInitializerOptIR' are the same type

annotationList direction typeIR$_a$ id parameterInitializerOptIR and annotationList' direction typeIR$_b$ id' parameterInitializerOptIR' are the same type:

1. If typeIR$_a$ and typeIR$_b$ are the same type: then, the relation holds.

18.6.1. Base types

baseTypeIR and baseTypeIR are the same type: then, the relation holds.

18.6.2. Named types

NAME typeId and NAME typeId are the same type: then, the relation holds.

18.6.3. Data types

18.6.3.1. List types

LIST `< typeIR$_a$ `> and LIST `< typeIR$_b$ `> are the same type:

1. If typeIR$_a$ and typeIR$_b$ are the same type: then, the relation holds.

18.6.3.2. Tuple types

TUPLE `< typeIR$_a^*$ `> and TUPLE `< typeIR$_b^*$ `> are the same type:

1. If the length of typeIR$_a^*$ is equal to the length of typeIR$_b^*$:
    a. If typeIR$_a$ and typeIR$_b$ are the same type, for all typeIR$_a$ in typeIR$_a^*$ and typeIR$_b$ in typeIR$_b^*$: then, the relation holds.

18.6.3.3. Array types

ARRAY typeIR$_a$ `[ n$_{size}$ `] and ARRAY typeIR$_b$ `[ n$_{size}$ `] are the same type:

1. If typeIR$_a$ and typeIR$_b$ are the same type: then, the relation holds.

18.6.3.4. Header stack types

HEADER_STACK typeIR$_a$ `[ n$_{size}$ `] and HEADER_STACK typeIR$_b$ `[ n$_{size}$ `] are the same type:

1. If typeIR$_a$ and typeIR$_b$ are the same type: then, the relation holds.

18.6.3.5. Struct types

STRUCT typeId `< typeArgumentIR$_a$$^*$ `> `{ fieldTypeIR$^*$ `} and STRUCT typeId `< typeArgumentIR$_b$$^*$ `> `{ _$^*$ `} are the same type:

1. If the length of typeArgumentIR$_a$* is equal to the length of typeArgumentIR$_b$*:
    a. If typeArgumentIR$_a$ and typeArgumentIR$_b$ are the same type, for all typeArgumentIR$_a$ in typeArgumentIR$_a$* and typeArgumentIR$_b$ in typeArgumentIR$_b$*: then, the relation holds.

18.6.3.6. Header types

HEADER typeId `< typeArgumentIR$_a$$^*$ `> `{ fieldTypeIR$^*$ `} and HEADER typeId `< typeArgumentIR$_b$$^*$ `> `{ _$^*$ `} are the same type:

1. If the length of typeArgumentIR$_a$* is equal to the length of typeArgumentIR$_b$*:
    a. If typeArgumentIR$_a$ and typeArgumentIR$_b$ are the same type, for all typeArgumentIR$_a$ in typeArgumentIR$_a$* and typeArgumentIR$_b$ in typeArgumentIR$_b$*: then, the relation holds.

18.6.3.7. Header union types

HEADER_UNION typeId `< typeArgumentIR$_a$$^*$ `> `{ fieldTypeIR$^*$ `} and HEADER_UNION typeId `< typeArgumentIR$_b$$^*$ `> `{ _$^*$ `} are the same type:

1. If the length of typeArgumentIR$_a$* is equal to the length of typeArgumentIR$_b$*:
    a. If typeArgumentIR$_a$ and typeArgumentIR$_b$ are the same type, for all typeArgumentIR$_a$ in typeArgumentIR$_a$* and typeArgumentIR$_b$ in typeArgumentIR$_b$*: then, the relation holds.

18.6.3.8. Enum types

typeIR and typeIR' are the same type:

1. If let ENUM typeId `{ nameIR$_{field}$$^*$ `} of type simpleEnumTypeIR be typeIR:
    a. If typeIR' is equal to ENUM typeId `{ nameIR$_{field}$$^*$ `}: then, the relation holds.

2. Else if let ENUM typeId `< typeIR$_a$ `> `{ ( nameIR$_{field}$ '=' value$_{field}$ ';' )$^*$ `} of type serializableEnumTypeIR be typeIR:
    a. If let ENUM typeId' `< typeIR$_b$ `> `{ valueFieldIR$^*$ `} of type serializableEnumTypeIR be typeIR':
        i. If typeId' is equal to typeId:
            A. If valueFieldIR* is equal to (nameIR$_{field}$ '=' value$_{field}$ ';')*:
                I. If typeIR$_a$ and typeIR$_b$ are the same type: then, the relation holds.

18.6.4. Object types

18.6.4.1. Extern object types

EXTERN typeId `< typeArgumentIR$_a$$^*$ `> externMethodTypeDefEnv and EXTERN typeId `< typeArgumentIR$_b$$^*$ `> _ are the same type:

1. Check that the length of typeArgumentIR$_a$* is equal to the length of typeArgumentIR$_b$*.
2. If typeArgumentIR$_a$ and typeArgumentIR$_b$ are the same type, for all typeArgumentIR$_a$ in typeArgumentIR$_a$* and typeArgumentIR$_b$ in typeArgumentIR$_b$*: then, the relation holds.

Additional equivalence rules for extern methods are defined as follows:

ExternMethodTypeDef_alpha:

- externMethodTypeDefIR$_a$ and externMethodTypeDefIR$_b$ are the same type

externMethodTypeDefIR$_a$ and externMethodTypeDefIR$_b$ are the same type:

1. Let (typeParameterIR$_{expl_a}$*, typeParameterIR$_{impl_a}$*) be the type parameters of externMethodTypeDefIR$_a$.
2. Let (typeParameterIR$_{expl_b}$*, typeParameterIR$_{impl_b}$*) be the type parameters of externMethodTypeDefIR$_b$.
3. Check that the length of typeParameterIR$_{expl_a}$* is equal to the length of typeParameterIR$_{expl_b}$*.
4. Check that the length of typeParameterIR$_{impl_a}$* is equal to the length of typeParameterIR$_{impl_b}$*.
5. Let typeParameterIR$_{fresh}$* be the length of typeParameterIR$_{expl_a}$* concatenated with typeParameterIR$_{impl_a}$* fresh type variables.
6. Let typeParameterIR$_a$* be typeParameterIR$_{expl_a}$* concatenated with typeParameterIR$_{impl_a}$*.
7. Check that the length of typeParameterIR$_a$* is equal to the length of typeParameterIR$_{fresh}$*.
8. Let theta$_a$ be `{ ( typeParameterIR$_a$ ':' $_{NAME}$ typeParameterIR$_{fresh}$ )$^*$ `}.
9. Let callableTypeIR be the underlying type of externMethodTypeDefIR$_a$.
10. Check that callableTypeIR has type externMethodTypeIR.
11. Let externMethodTypeIR$_a$ be callableTypeIR.
12. Let callableTypeIR' be externMethodTypeIR$_a$ substituted by theta$_a$.
13.

Check that callableTypeIR' has type externMethodTypeIR.

14. Let $externMethodTypeIR_{a_subst}$ be callableTypeIR'.
15. Let $typeParameterIR_b^*$ be $typeParameterIR_{expl_b}^*$ concatenated with $typeParameterIR_{impl_b}^*$.
16. Check that the length of $typeParameterIR_b^*$ is equal to the length of $typeParameterIR_{fresh}^*$.
17. Let $theta_b$ be `{ ( typeParameterIR$_b$ ':' $_{NAME}$ typeParameterIR$_{fresh}$ )* `}.
18. Let callableTypeIR'' be the underlying type of $externMethodTypeDefIR_b$.
19. Check that callableTypeIR'' has type externMethodTypeIR.
20. Let $externMethodTypeIR_b$ be callableTypeIR''.
21. Let callableTypeIR''' be $externMethodTypeIR_b$ substituted by $theta_b$.
22. Check that callableTypeIR''' has type externMethodTypeIR.
23. Let $externMethodTypeIR_{b_subst}$ be callableTypeIR'''.
24. If $externMethodTypeIR_{a_subst}$ and $externMethodTypeIR_{b_subst}$ are the same type: then, the relation holds.

ExternMethodType_alpha:

- externMethodTypeIR and externMethodTypeIR' are the same type

externMethodTypeIR and externMethodTypeIR' are the same type:

1. If let EXTERN_METHOD nameIR `( $parameterIR_a^*$ `) ':' $typeIR_a$ be externMethodTypeIR:
   a. Check that externMethodTypeIR' matches pattern EXTERN_METHOD % `( % `) ':' %.
   b. Let EXTERN_METHOD nameIR' `( $parameterIR_b^*$ `) ':' $typeIR_b$ be externMethodTypeIR'.
   c. Check that nameIR' is equal to nameIR.
   d. Check that the length of $parameterIR_a^*$ is equal to the length of $parameterIR_b^*$.
   e. If $parameterIR_a$ and $parameterIR_b$ are the same type, for all $parameterIR_a$ in $parameterIR_a^*$ and $parameterIR_b$ in $parameterIR_b^*$:
      i. If $typeIR_a$ and $typeIR_b$ are the same type: then, the relation holds.
2. Else:
   a. Let EXTERN_METHOD ABSTRACT nameIR `( $parameterIR_a^*$ `) ':' $typeIR_a$ be externMethodTypeIR.
   b. Check that externMethodTypeIR' matches pattern EXTERN_METHOD ABSTRACT % `( % `) ':' %.
   c. Let EXTERN_METHOD ABSTRACT nameIR' `( $parameterIR_b^*$ `) ':' $typeIR_b$ be externMethodTypeIR'.
   d. Check that nameIR' is equal to nameIR.
   e. Check that the length of $parameterIR_a^*$ is equal to the length of $parameterIR_b^*$.
   f. If $parameterIR_a$ and $parameterIR_b$ are the same type, for all $parameterIR_a$ in $parameterIR_a^*$ and $parameterIR_b$ in $parameterIR_b^*$:
      i. If $typeIR_a$ and $typeIR_b$ are the same type: then, the relation holds.

18.6.4.2. Parser object types

PARSER typeId `< typeArgumentIR* `> `( parameterIR$_a$* `) and PARSER _ `< _* `> `( parameterIR$_b$* `) are the same type:

1. Check that the length of parameterIR$_a$* is equal to the length of parameterIR$_b$*.
2. If parameterIR$_a$ and parameterIR$_b$ are the same type, for all parameterIR$_a$ in parameterIR$_a$* and parameterIR$_b$ in parameterIR$_b$*: then, the relation holds.

18.6.4.3. Control object types

CONTROL typeId `< typeArgumentIR* `> `( parameterIR$_a$* `) and CONTROL _ `< _* `> `( parameterIR$_b$* `) are the same type:

1. Check that the length of parameterIR$_a$* is equal to the length of parameterIR$_b$*.
2. If parameterIR$_a$ and parameterIR$_b$ are the same type, for all parameterIR$_a$ in parameterIR$_a$* and parameterIR$_b$ in parameterIR$_b$*: then, the relation holds.

18.6.4.4. Package object types

PACKAGE typeId `< typeArgumentIR* `> `{ typeIR$_a$* `} and PACKAGE _ `< _* `> `{ typeIR$_b$* `} are the same type:

1. Check that the length of typeIR$_a$* is equal to the length of typeIR$_b$*.
2. If typeIR$_a$ and typeIR$_b$ are the same type, for all typeIR$_a$ in typeIR$_a$* and typeIR$_b$ in typeIR$_b$*: then, the relation holds.

18.6.4.5. Table object types

tableObjectTypeIR and tableObjectTypeIR are the same type: then, the relation holds.

18.6.5. Alias types

18.6.5.1. typedef types

typeIR and typeIR' are the same type:

1. Try:
    a. If let TYPEDEF _ typeIR$_a$ of type typedefTypeIR be typeIR:
        i. If typeIR$_a$ and typeIR' are the same type: then, the relation holds.
2. Then, try:

a. If let TYPEDEF _ $typeIR_b$ of type typedefTypeIR be typeIR':

i. If typeIR does not have type typedefTypeIR:

A. If typeIR and $typeIR_b$ are the same type: then, the relation holds.

18.6.5.2. `type` types

TYPE typeId $typeIR_a$ and TYPE typeId $typeIR_b$ are the same type:

1. If $typeIR_a$ and $typeIR_b$ are the same type: then, the relation holds.

18.6.6. Synthesized types

18.6.6.1. Default types

DEFAULT and DEFAULT are the same type: then, the relation holds.

18.6.6.2. Invalid header types

HEADER_INVALID and HEADER_INVALID are the same type: then, the relation holds.

18.6.6.3. Sequence types

sequenceTypeIR and sequenceTypeIR' are the same type:

1. If let SEQ `< $typeIR_a^*$ `> be sequenceTypeIR:
    a. Check that sequenceTypeIR' matches pattern SEQ `< % `>.
    b. Let SEQ `< $typeIR_b^*$ `> be sequenceTypeIR'.
    c. Check that the length of $typeIR_a^*$ is equal to the length of $typeIR_b^*$.
    d. If $typeIR_a$ and $typeIR_b$ are the same type, for all $typeIR_a$ in $typeIR_a^*$ and $typeIR_b$ in $typeIR_b^*$: then, the relation holds.
2. Else:
    a. Let SEQ `< $typeIR_a^*$ ',' '...' `> be sequenceTypeIR.
    b. Check that sequenceTypeIR' matches pattern SEQ `< % ',' '...' `>.
    c. Let SEQ `< $typeIR_b^*$ ',' '...' `> be sequenceTypeIR'.
    d. Check that the length of $typeIR_a^*$ is equal to the length of $typeIR_b^*$.
    e. If $typeIR_a$ and $typeIR_b$ are the same type, for all $typeIR_a$ in $typeIR_a^*$ and $typeIR_b$ in $typeIR_b^*$: then, the relation holds.

18.6.6.4. Record types

recordTypeIR and recordTypeIR' are the same type:

1. If let RECORD `{ ( annotationList typeIR$_{field\_a}$ nameIR$_{field}$ ';' )$^{*}$ `} be recordTypeIR:
    a. Check that recordTypeIR' matches pattern RECORD `{ % `}.
    b. Let RECORD `{ ( annotationList' typeIR$_{field\_b}$ nameIR ';' )$^{*}$ `} be recordTypeIR'.
    c. Check that the length of annotationList* is equal to the length of annotationList'*.
    d. Check that annotationList' is equal to annotationList, for all annotationList in annotationList* and annotationList' in annotationList'*.
    e. Check that nameIR is equal to nameIR$_{field}$, for all nameIR$_{field}$ in nameIR$_{field}{}^{*}$ and nameIR in nameIR*.
    f. If typeIR$_{field_a}$ and typeIR$_{field_b}$ are the same type, for all typeIR$_{field_a}$ in typeIR$_{field_a}{}^{*}$ and typeIR$_{field_b}$ in typeIR$_{field_b}{}^{*}$: then, the relation holds.
2. Else:
    a. Let RECORD `{ ( annotationList typeIR$_{field\_a}$ nameIR$_{field}$ ';' )$^{*}$ ',' '...' `} be recordTypeIR.
    b. Check that recordTypeIR' matches pattern RECORD `{ % ',' '...' `}.
    c. Let RECORD `{ ( annotationList' typeIR$_{field\_b}$ nameIR ';' )$^{*}$ ',' '...' `} be recordTypeIR'.
    d. Check that the length of annotationList* is equal to the length of annotationList'*.
    e. Check that annotationList' is equal to annotationList, for all annotationList in annotationList* and annotationList' in annotationList'*.
    f. Check that nameIR is equal to nameIR$_{field}$, for all nameIR$_{field}$ in nameIR$_{field}{}^{*}$ and nameIR in nameIR*.
    g. If typeIR$_{field_a}$ and typeIR$_{field_b}$ are the same type, for all typeIR$_{field_a}$ in typeIR$_{field_a}{}^{*}$ and typeIR$_{field_b}$ in typeIR$_{field_b}{}^{*}$: then, the relation holds.

18.6.6.5. Set types

SET `< typeIR$_{a}$ `> and SET `< typeIR$_{b}$ `> are the same type:

1. If typeIR$_{a}$ and typeIR$_{b}$ are the same type: then, the relation holds.

18.6.6.6. Table metadata types

tableMetadataTypeIR and tableMetadataTypeIR are the same type: then, the relation holds.

19. Calls

Calls can be made in several different contexts:

- From call expressions (Section 14.16), to invoke a callable or a constructor.
- From call statements (Section 13.4), to invoke a callable.

- From a direct type invocation (Section 13.5), which is a syntactic sugar for invoking a constructor and then immediately calling an apply method on the resulting instance.
- From an instantiation declaration (Section 11.4), to invoke a constructor.

A callable invocation can optionally specify for each argument the corresponding parameter name. It is illegal to use names only for some arguments: either all or no arguments must specify the parameter name. Calalble arguments are evaluated in the order they appear, left to right, before the function invocation takes place.

```
extern void f(in bit<32> x, out bit<16> y);
bit<32> xa = 0;
bit<16> ya;
f(xa, ya);  // match arguments by position
f(x = xa, y = ya);  // match arguments by name
f(y = ya, x = xa);  // match arguments by name in any order
//f(x = xa);  -- error: enough arguments
//f(x = xa, x = ya);  -- error: argument specified twice
//f(x = xa, ya);  -- error: some arguments specified by name
//f(z = xa, w = yz);  -- error: no parameter named z or w
//f(x = xa, y = 0);  -- error: y must be a left-value
```

The calling convention is copy-in/copy-out (Section 6.5). For generic callables and constructors the type arguments can be explicitly specified in the call. The compiler only inserts implicit casts for direction in or directionless arguments to methods, functions, or constructors as described in [sec-casts]. The types for all other arguments must match the parameter types exactly.

The result returned by a call is discarded when the function call is used as a statement.

The "don't care" identifier (_) can only be used for an out function/method argument, when the value of returned in that argument is ignored by subsequent computations. When used in generic callables, the compiler may reject the program if it is unable to infer a type for the don't care argument.

This section describes the semantics of calls in detail. In particular, it explains how the call target is found (also with overload resolution), how the call convention is checked, and how the call is evaluated at runtime.

19.1. Call targets

Calls are composed of three components: the call target, type arguments, and arguments. Call target identifies what is being called. It may be a callable (actions, functions, and methods) or a constructor.

When calls are made from an expression, the call target is as follows:

```
callExpression
    : callTarget ( argumentList )
    | callableTarget < realTypeArgumentList > ( argumentList )
    ;

callTarget
    : callableTarget | constructorTarget
    ;

callableTarget = expression

constructorTarget = namedType
```

When calls are made from a statement, the call target is as follows:

```
lvalue
    : referenceExpression | lvalue . member | lvalue [ expression ]
    | lvalue [ expression sliceop expression ] | ( lvalue )
    ;

callStatement
    : lvalue ( argumentList ) ; | lvalue < typeArgumentList > ( argumentList ) ;
    ;

directApplicationStatement
    : namedType . APPLY ( argumentList ) ;
    ;
```

And when calls are made from instantiation declarations, the call target is as follows:

```
instantiation
    : annotationList type ( argumentList ) name ;
    | annotationList type ( argumentList ) name objectInitializer ;
    ;
```

After type checking, call targets are represented as follows:

```
callableTargetIR
    : referenceExpressionIR | typedExpressionIR . nameIR
    | TYPE prefixedNameIR . nameIR | ( callableTargetIR )
    ;

constructorTargetIR
    : prefixedNameIR < typeArgumentListIR >
    ;
```

Thus, calls are represented as:

```
callExpressionIR
    : constructorTargetIR ( argumentListIR )
    | callableTargetIR < typeArgumentListIR > ( argumentListIR )
    ;

callStatementIR
    : callableTargetIR < typeArgumentListIR > ( argumentListIR ) ;
    ;

directApplicationStatementIR
    : constructorTargetIR . APPLY ( argumentListIR ) ;
    ;

instantiationIR
    : annotationList typeIR constructorTargetIR ( argumentListIR ) nameIR
        objectInitializerOptIR ;
    ;
```

The following relation is used to check callable targets:

CallableTarget_ok:

- Typing callableTarget as a callable target, under cursor p and context TC:
- The result is typed callable target callableTargetIR.

Typing callableTarget as a callable target, under cursor p and context TC:

1. Try:
    a. If let prefixedNonTypeName of type prefixedNonTypeName be callableTarget:
        i. Let prefixedNameIR be the prefixed name of prefixedNonTypeName.
        ii. The result is typed callable target prefixedNameIR.
    b. Else if callableTarget has type memberAccessExpression:
        i. Let memberAccessBase '.' member be callableTarget.
        ii. If let prefixedTypeName of type prefixedTypeName be memberAccessBase:
            A. Let prefixedNameIR be the prefixed name of prefixedTypeName.
            B. Let nameIR be the name of member.
            C. The result is typed callable target TYPE prefixedNameIR '.' nameIR.
        iii. Else:
            A. Let $\text{expression}_{\text{base}}$ be memberAccessBase.
            B. Let nameIR be the name of member.
            C. Let $\text{typedExpressionIR}_{\text{base}}$ be the result of typing $\text{expression}_{\text{base}}$, under context TC at p.
            D. The result is typed callable target $\text{typedExpressionIR}_{\text{base}}$ '.' nameIR.
    c. Else if callableTarget has type parenthesizedExpression:
        i. Let `( expression `) be callableTarget.
        ii. Let typed callable target callableTargetIR be the result of typing expression as a callable target, under cursor p and context TC.
        iii. The result is typed callable target `( callableTargetIR `).
2. Then, try:
    a. Check that callableTarget is equal to THIS.
    b. The result is typed callable target $_{\text{BARE}}$ "this".

When a l-value is used as a callable target, the following relation is used to check it:

CallableTarget_lvalue_ok:

- Typing lvalue as a callable target, under cursor p and context TC:
- The result is typed callable target callableTargetIR.

Typing lvalue as a callable target, under cursor p and context TC:

1. Let expression be the expression corresponding to lvalue.
2. Let typed callable target callableTargetIR be the result of typing expression as a callable target, under cursor p and context TC.
3. The result is typed callable target callableTargetIR.

19.2. Argument alignment

The call semantics classify parameters according to whether an argument must be provided, a default value is available, or the parameter is optional.

19.2.1. Parameter classification

The following function partitions a parameter list using the names of the parameters with default values and the parameters annotated with @optional:

$partition_parameterListIR(parameterIR*, nameIR*, nameIR'*)

1. If parameterIR* matches pattern []: return (·, ·, ·).
2. Else:
   a. Let $parameterIR_h$ :: $parameterIR_t$* be parameterIR*.
   b. Let ($parameterIR_{given}$*, $parameterIR_{default}$*, $parameterIR_{optional}$*) be $partition_parameterListIR(parameterIR_t*, nameIR*, nameIR'*).
   c. Let $nameIR_h$ be the name of $parameterIR_h$.
   d. Try:
      i. If $nameIR_h$ is in nameIR*:
         A. Return ($parameterIR_{given}$*, $parameterIR_h$ :: $parameterIR_{default}$*, $parameterIR_{optional}$*).
      ii. Else:
         A. If $nameIR_h$ is not in nameIR'*:
            I. Return ($parameterIR_h$ :: $parameterIR_{given}$*, $parameterIR_{default}$*, $parameterIR_{optional}$*).
   e. Then, try:
      i. Check that $nameIR_h$ is in nameIR'*.
      ii. Return ($parameterIR_{given}$*, $parameterIR_{default}$*, $parameterIR_h$ :: $parameterIR_{optional}$*).

19.2.2. Aligning arguments

Positional arguments are aligned by position and named arguments are aligned by name. The result separates supplied parameters from default and optional parameters:

```
callAlign
    : GIVEN parameterIR* DEFAULT parameterIR* OPTIONAL parameterIR*
    ;
```

Aligning parameterIR* with argumentIR* where defaults are $nameIR_{default}$* and optionals are $nameIR_{optional}$*

1. Let ($parameterIR_{given}$*, $parameterIR_{default}$*, $parameterIR_{optional}$*) be $partition_parameterListIR(parameterIR*,

nameIR$_{default}^{*}$, nameIR$_{optional}^{*}$).

2. For each parameterIR$_{given}$ in parameterIR$_{given}^{*}$:

   - Let _ _ _ nameIR$_{given}$ _ be parameterIR$_{given}$.

   Let nameIR$_{given}^{*}$ be the resulting list.

3. Let parameterIR$_{given_aligned}^{*}$ be $align_parameterListIR_given(`{ ( nameIR$_{given}$ ':' parameterIR$_{given}$ )$^{*}$ `}, parameterIR$_{given}^{*}$, argumentIR*).
4. Return GIVEN parameterIR$_{given_aligned}^{*}$ DEFAULT parameterIR$_{default}^{*}$ OPTIONAL parameterIR$_{optional}^{*}$.

The following helper aligns the parameters that are supplied with arguments:

$align_parameterListIR_given(`{ ( id ':' parameterIR )$^{*}$ `}, parameterIR'*, argumentIR*)

1. If parameterIR'* matches pattern []:
   a. Check that argumentIR* is an empty list.
   b. Return ·.
2. Else:
   a. Let parameterIR$_{h}$ :: parameterIR$_{t}^{*}$ be parameterIR'*.
   b. Check that argumentIR* is a non-empty list.
   c. Let argumentIR$_{h}$ :: argumentIR$_{t}^{*}$ be argumentIR*.
   d. If argumentIR$_{h}$ has type typedExpressionIR:
      i. Let parameterIR$_{aligned}^{*}$ be $align_parameterListIR_given(`{ ( id ':' parameterIR )$^{*}$ `}, parameterIR$_{t}^{*}$, argumentIR$_{t}^{*}$).
      ii. Return parameterIR$_{h}$ :: parameterIR$_{aligned}^{*}$.
   e. Else if argumentIR$_{h}$ matches pattern '_':
      i. Let parameterIR$_{aligned}^{*}$ be $align_parameterListIR_given(`{ ( id ':' parameterIR )$^{*}$ `}, parameterIR$_{t}^{*}$, argumentIR$_{t}^{*}$).
      ii. Return parameterIR$_{h}$ :: parameterIR$_{aligned}^{*}$.
   f. Else if let nameIR '=' _ be argumentIR$_{h}$:
      i. Let parameterIR''$^{?}$ be the value of nameIR in map `{ ( id ':' parameterIR )$^{*}$ `}.
      ii. Check that parameterIR''$^{?}$ is defined.
      iii. Let parameterIR$_{match}$ be parameterIR''$^{?}$.
      iv. Let parameterIR$_{aligned}^{*}$ be $align_parameterListIR_given(`{ ( id ':' parameterIR )$^{*}$ `}, parameterIR$_{t}^{*}$, argumentIR$_{t}^{*}$).
      v. Return parameterIR$_{match}$ :: parameterIR$_{aligned}^{*}$.
   g. Else:
      i. Let nameIR '=' '_' be argumentIR$_{h}$.
      ii. Let parameterIR''$^{?}$ be the value of nameIR in map `{ ( id ':' parameterIR )$^{*}$ `}.
      iii. Check that parameterIR''$^{?}$ is defined.
      iv. Let parameterIR$_{match}$ be parameterIR''$^{?}$.
      v. Let parameterIR$_{aligned}^{*}$ be $align_parameterListIR_given(`{ ( id ':' parameterIR )$^{*}$ `}, parameterIR$_{t}^{*}$, argumentIR$_{t}^{*}$).
      vi. Return parameterIR$_{match}$ :: parameterIR$_{aligned}^{*}$.

19.3. Overload resolution

As explained in Section 6.6, functions, methods, and constructors can be overloaded. Overload should be resolved at compile time based on the number of arguments or the names of the arguments. Additionally, arguments may be omitted when the corresponding parameters have default values or are declared as optional.

The following algorithm is used to resolve overloads and omitted arguments:

Finding the item in `{ ( callableId ':' V )* `} matching nameIR$_f$ `( id$_{arg}^{?*}$ `) using parameter getter $get_parameterListIR

The result of this algorithm is a matched overload item with the default and optional parameter names, or an empty result if no match is found.

```
callResolution<V>
    : callableId : V # nameIR* # nameIR*
    ;
```

The algorithm takes as input the key for overload resolution, which is the call target name and the argument names (if specified).

```
callTargetKey
    : nameIR ( id?* )
    ;
```

The call target key for nameIR$_f$ `( argumentIR* `)

1. For each argumentIR in argumentIR*:
    - Let id$_{arg}^{?}$ be $callTargetKey'(argumentIR).

   Let id$_{arg}^{?*}$ be the resulting list.
2. Return nameIR$_f$ `( id$_{arg}^{?*}$ `).

The argument names are collected by inspecting each argument in turn:

$callTargetKey'(argumentIR)

1. If argumentIR has type typedExpressionIR: return ·.
2. Else if let id$_{arg}$ '=' _ be argumentIR: return id$_{arg}$.
3. Else if let id$_{arg}$ '=' '_' be argumentIR: return id$_{arg}$.
4. Else: return ·.

Based on whether the arguments are positional or named, the key is refined to either:

```
namedCallTargetKey
    : nameIR ( id* )
    ;

unnamedCallTargetKey
    : nameIR ( nat )
    ;
```

For each item in the overloaded set, the key is compared to determine if it is a match:

```
callTargetMatch
    : MATCH id* id* | NOMATCH
    ;
```

The algorithm is as follows:

Finding the item in `{ ( callableId ':' V )* `} matching nameIR$_f$ `( id$_{arg}^{?*}$ `) using parameter getter $get_parameterListIR

1. Try:
    a. If the length of id$_{arg}^{?*}$ is greater than 0:
        i. If id$_{arg}^{?}$ is defined, for all id$_{arg}^{?}$ in id$_{arg}^{?*}$:
            A. For each id$_{arg}^{?}$ in id$_{arg}^{?*}$:
                - Let id$_{arg_inner}$ be id$_{arg}^{?}$.

            Let id$_{arg_inner}^{*}$ be the resulting list.

            B. Try:
                I. If finding all items in `{ ( callableId ':' V )* `} matching named key nameIR$_f$ `( id$_{arg\_inner}^{*}$ `) using parameter getter $get_parameterListIR is an empty list: return ·.
            C. Then, try:
                I. Let callResolution<V>* be finding all items in `{ ( callableId ':' V )* `} matching named key nameIR$_f$ `( id$_{arg\_inner}^{*}$ `) using parameter getter $get_parameterListIR.
                II. If callResolution<V>* is a list of length 1:
                    1. Let callableId$_{found}$ ':' V$_{found}$ '#' id$_{default}^{*}$ '#' id$_{optional}^{*}$ be callResolution<V>*.
                    2. Return callableId$_{found}$ ':' V$_{found}$ '#' id$_{default}^{*}$ '#' id$_{optional}^{*}$.
2. Then, try:
    a. Check that id$_{arg}^{?}$ is none, for all id$_{arg}^{?}$ in id$_{arg}^{?*}$.
    b. Try:
        i. If finding all items in `{ ( callableId ':' V )* `} matching unnamed key nameIR$_f$ `( the length of id$_{arg}^{?*}$ `) using parameter getter $get_parameterListIR is an empty list: return ·.
    c. Then, try:
        i. Let callResolution<V>* be finding all items in `{ ( callableId ':' V )* `} matching unnamed key nameIR$_f$ `( the length of id$_{arg}^{?*}$ `) using parameter getter $get_parameterListIR.
        ii. Check that callResolution<V>* is a list of length 1.

iii. Let callableId$_{found}$ ':' V$_{found}$ '#' id$_{omitted}^{*}$ '#' id$_{optional}^{*}$ be callResolution<V>*.

iv. Return callableId$_{found}$ ':' V$_{found}$ '#' id$_{omitted}^{*}$ '#' id$_{optional}^{*}$.

19.3.1. Positional arguments

The following algorithm is used when arguments are positional:

Finding all items in set<pair<callableId, V>> matching unnamed key nameIR$_{f}$ `( nat `) using parameter getter $get_parameterListIR

Finding all items in set<pair<callableId, V>> matching unnamed key nameIR$_{f}$ `( nat `) using parameter getter $get_parameterListIR

1. Try:
    a. If set<pair<callableId, V>> is equal to `{ · `}: return ·.
2. Then, try:
    a. Let `{ pair<callableId, V>$^{*}$ `} be set<pair<callableId, V>>.
    b. Check that pair<callableId, V>* is a non-empty list.
    c. Let callTargetId$_{h}$ ':' V$_{h}$ :: (callTargetId$_{t}$ ':' V$_{t}$)* be pair<callableId, V>*.
    d. Try:
        i. If matching callTargetId$_{h}$ ':' V$_{h}$ against unnamed key nameIR$_{f}$ `( nat `) using parameter getter $get_parameterListIR is NOMATCH:
            A. Return finding all items in `{ ( callTargetId$_{t}$ ':' V$_{t}$ )$^{*}$ `} matching unnamed key nameIR$_{f}$ `( nat `) using parameter getter $get_parameterListIR.
    e. Then, try:
        i. Let callTargetMatch be matching callTargetId$_{h}$ ':' V$_{h}$ against unnamed key nameIR$_{f}$ `( nat `) using parameter getter $get_parameterListIR.
        ii. Check that callTargetMatch matches pattern MATCH % %.
        iii. Let MATCH id$_{default}^{*}$ id$_{optional}^{*}$ be callTargetMatch.
        iv. Let callResolution<V>* be finding all items in `{ ( callTargetId$_{t}$ ':' V$_{t}$ )$^{*}$ `} matching unnamed key nameIR$_{f}$ `( nat `) using parameter getter $get_parameterListIR.
        v. Return callTargetId$_{h}$ ':' V$_{h}$ '#' id$_{default}^{*}$ '#' id$_{optional}^{*}$ :: callResolution<V>*.

Each candidate in the overloaded set is matched against the unnamed key as follows:

Matching callTargetId ':' V against unnamed key nameIR$_{f}$ `( n$_{arg}$ `) using parameter getter $get_parameterListIR

1. Let nameIR `( id$_{param}^{*}$ `) be callTargetId.
2. If nameIR is equal to nameIR$_{f}$:
    a. Let parameterIR* be $get_parameterListIR(V).
    b. For each parameterIR in parameterIR*:

- Let $b_{default}$ be parameterIR is a parameter with a default value.

Let $b_{default}^{*}$ be the resulting list.

c. For each parameterIR in parameterIR*:

- Let $b_{optional}$ be parameterIR is an optional parameter.

Let $b_{optional}^{*}$ be the resulting list.

d. If $exists_(bdefault~*) /\ $exists_($b_{optional}^{*}$):

i. Try:

A. If the length of id_{param}^{*} is equal to n_{arg}:

I. Return MATCH with defaults · and optionals ·.

ii. Then, try:

A. If the length of id_{param}^{*} is greater than n_{arg}:

I. Let ($parameterIR_{given}^{*}$, $parameterIR_{omitted}^{*}$) be $partition_<parameterIR>(parameterIR*, n_{arg}).

II. Try:

1. If $parameterIR_{omitted}$ is not an optional parameter, for all $parameterIR_{omitted}$ in $parameterIR_{omitted}^{*}$:

1. If $parameterIR_{omitted}$ is a parameter with a default value, for all $parameterIR_{omitted}$ in $parameterIR_{omitted}^{*}$:

1. For each $parameterIR_{omitted}$ in $parameterIR_{omitted}^{*}$:

- Let $id_{default}$ be the identifier of $parameterIR_{omitted}$.

Let $id_{default}^{*}$ be the resulting list.

2. Return MATCH with defaults $id_{default}^{*}$ and optionals ·.

III. Then, try:

1. If $parameterIR_{given}$ is a parameter without a default value, for all $parameterIR_{given}$ in $parameterIR_{given}^{*}$:

1. If $parameterIR_{omitted}$ is an optional parameter, for all $parameterIR_{omitted}$ in $parameterIR_{omitted}^{*}$:

1. For each $parameterIR_{omitted}$ in $parameterIR_{omitted}^{*}$:

- Let $id_{optional}$ be the identifier of $parameterIR_{omitted}$.

Let $id_{optional}^{*}$ be the resulting list.

2. Return MATCH with defaults · and optionals $id_{optional}^{*}$.

3. Otherwise: return NOMATCH.

19.3.2. Named arguments

The following algorithm is used when arguments are named:

Finding all items in set<pair<callableId, V>> matching named key $nameIR_{f}$ `( $id_{arg}^{*}$ `) using parameter getter

$get_parameterListIR

Finding all items in set<pair<callableId, V>> matching named key nameIR$_f$ `( id$_{arg}^*$ `) using parameter getter $get_parameterListIR

1. Try:
    a. If set<pair<callableId, V>> is equal to `{ · `}: return ·.
2. Then, try:
    a. Let `{ pair<callableId, V>$^*$ `} be set<pair<callableId, V>>.
    b. Check that pair<callableId, V>* is a non-empty list.
    c. Let callTargetId$_h$ ':' V$_h$ :: (callTargetId$_t$ ':' V$_t$)* be pair<callableId, V>*.
    d. Try:
        i. If matching callTargetId$_h$ ':' V$_h$ against named key nameIR$_f$ `( id$_{arg}^*$ `) using parameter getter $get_parameterListIR is NOMATCH:
            A. Return finding all items in `{ ( callTargetId$_t$ ':' V$_t$ )$^*$ `} matching named key nameIR$_f$ `( id$_{arg}^*$ `) using parameter getter $get_parameterListIR.
    e. Then, try:
        i. Let callTargetMatch be matching callTargetId$_h$ ':' V$_h$ against named key nameIR$_f$ `( id$_{arg}^*$ `) using parameter getter $get_parameterListIR.
        ii. Check that callTargetMatch matches pattern MATCH % %.
        iii. Let MATCH id$_{default}^*$ id$_{optional}^*$ be callTargetMatch.
        iv. Let callResolution<V>* be finding all items in `{ ( callTargetId$_t$ ':' V$_t$ )$^*$ `} matching named key nameIR$_f$ `( id$_{arg}^*$ `) using parameter getter $get_parameterListIR.
        v. Return callTargetId$_h$ ':' V$_h$ '#' id$_{default}^*$ '#' id$_{optional}^*$ :: callResolution<V>*.

Each candidate in the overloaded set is matched against the named key as follows:

Matching callTargetId ':' V against named key nameIR$_f$ `( id$_{arg}^*$ `) using parameter getter $get_parameterListIR

1. Let nameIR `( id$_{param}^*$ `) be callTargetId.
2. If nameIR is equal to nameIR$_f$:
    a. Let parameterIR* be $get_parameterListIR(V).
    b. If the length of id$_{param}^*$ is equal to the length of parameterIR*:
        i. If id$_{param}$ is equal to the identifier of parameterIR, for all id$_{param}$ in id$_{param}^*$ and parameterIR in parameterIR*:
            A. Try:
                I. If the length of id$_{param}^*$ is equal to the length of id$_{arg}^*$:
                    1. Let bool be `{ id$_{param}^*$ `} is equal to `{ id$_{arg}^*$ `}.
                    2. Return MATCH with defaults · and optionals · if bool otherwise NOMATCH.
            B. Then, try:
                I. For each parameterIR in parameterIR*:

                - Let b$_{default}$ be parameterIR is a parameter with a default value.

                Let b$_{default}^*$ be the resulting list.

                II. For each parameterIR in parameterIR*:

- Let $b_{optional}$ be parameterIR is an optional parameter.

Let $b_{optional}^{*}$ be the resulting list.

III. If the length of id_{param}^{*} is greater than the length of id_{arg}^{*}:

1. Let `{ $id_{param\_omitted}^{*}$ `} be the difference of the sets `{ $id_{param}^{*}$ `} and `{ $id_{arg}^{*}$ `}.
2. For each parameterIR in parameterIR*:

   - Let bool'' be parameterIR is a parameter with a default value.

   Let bool''* be the resulting list.

3. Let $id_{default_param}^{*}$ be id_{param}^{*} filtered by bool''*.
4. Let $id_{default}^{*}$ be $id_{param_omitted}^{*}$ filtered by ($id_{param_omitted}$ is in $id_{default_param}^{*}$)*.
5. For each parameterIR in parameterIR*:

   - Let bool' be parameterIR is an optional parameter.

   Let bool'* be the resulting list.

6. Let $id_{optional_param}^{*}$ be id_{param}^{*} filtered by bool'*.
7. Let $id_{optional}^{*}$ be $id_{param_omitted}^{*}$ filtered by ($id_{param_omitted}$ is in $id_{optional_param}^{*}$)*.
8. If `{ $id_{default}^{*}$ ++ $id_{optional}^{*}$ `} is equal to `{ $id_{param\_omitted}^{*}$ `}:
   1. Return MATCH with defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$.

3. Otherwise: return NOMATCH.

19.3.3. Non-overloaded targets

When the call target is not overloaded, resolution instead looks up the callable directly by name:

$find_non_overloaded<V>(`{ ( callableId ':' V )* `}, $nameIR_f$)

1. Let (callableId, V)* be $find_non_overloadeds<V>(`{ ( callableId ':' V )* `}, $nameIR_f$).
2. If (callableId, V)* is a list of length 1:
   a. Let ($callableId_{found}$, V_{found}) be (callableId, V)*.
   b. Return ($callableId_{found}$, V_{found}).
3. Otherwise: return ·.

The candidates sharing the given name are collected as follows:

$find_non_overloadeds<V>(set<pair<callableId, V>>, $nameIR_f$)

1. Try:
   a. If set<pair<callableId, V>> is equal to `{ · `}: return ·.
2. Then, try:
   a. Let `{ pair<callableId, V>* `} be set<pair<callableId, V>>.
   b. Check that pair<callableId, V>* is a non-empty list.
   c. Let $callableId_h$ ':' V_h :: ($callableId_t$ ':' V_t)* be pair<callableId, V>*.

d. Let nameIR `( _* `) be callableId$_h$.

e. If nameIR is equal to nameIR$_f$:

i. Let (callableId$_{t_found}$, V_{t_found})* be $find_non_overloadeds<V>(`{ ( callableId$_t$ ':' $V_t$ )* `}, nameIR$_f$).

ii. Return (callableId$_h$, V_h) :: (callableId$_{t_found}$, V_{t_found})*.

f. Else:

i. Let (callableId$_{t_found}$, V_{t_found})* be $find_non_overloadeds<V>(`{ ( callableId$_t$ ':' $V_t$ )* `}, nameIR$_f$).

ii. Return (callableId$_{t_found}$, V_{t_found})*.

19.4. Callee resolution

Given a call target, type arguments, and arguments, the callee should be resolved. This is done both during type checking and runtime evaluation.

19.4.1. Resolving constructors

When type checking, constructors are resolved by:

ConstructorType_ok:

- Finding type of constructor call constructorTargetIR `( argumentIR* `), under cursor p and context TC:
- The result is constructorTypeIR with fresh type variables typeId*, defaults id* and optionals id'*.

Finding type of constructor call prefixedNameIR `< typeArgumentIR'* `> `( argumentIR* `), under cursor p and context TC:

1. Try:
   a. If the type definition of prefixedNameIR from the p layer of TC is none:
      i. Let callResolution<constructorTypeDefIR>? be the overloaded constructor definition of prefixedNameIR `( argumentIR* `) in TC.
      ii. If callResolution<constructorTypeDefIR>? is defined:
         A. Let callableId ':' constructorTypeDefIR '#' $id_{default}$* '#' $id_{optional}$* be callResolution<constructorTypeDefIR>?.
         B. Let (constructorTypeIR, $typeId_{impl}$*) be constructorTypeDefIR specialized by typeArgumentIR'*.
         C. The result is constructorTypeIR with fresh type variables $typeId_{impl}$*, defaults $id_{default}$* and optionals $id_{optional}$*.
2. Then, try:
   a. Let typeDefIR'? be the type definition of prefixedNameIR from the p layer of TC.
   b. If typeDefIR'? is defined:
      i. Let typeDefIR be typeDefIR'?.
      ii. If typeDefIR does not have type typedefTypeIR:
         A. Let callResolution<constructorTypeDefIR>? be the overloaded constructor definition of prefixedNameIR `( argumentIR* `) in TC.

B. If callResolution<constructorTypeDefIR>? is defined:

I. Let callableId ':' constructorTypeDefIR '#' $id_{default}$* '#' $id_{optional}$* be callResolution<constructorTypeDefIR>?.

II. Let (constructorTypeIR, $typeId_{impl}$*) be constructorTypeDefIR specialized by typeArgumentIR'*.

III. The result is constructorTypeIR with fresh type variables $typeId_{impl}$*, defaults $id_{default}$* and optionals $id_{optional}$*.

3. Then, try:

a. Check that typeArgumentIR'* is an empty list.

b. Let typeDefIR? be the type definition of prefixedNameIR from the p layer of TC.

c. Check that typeDefIR? is defined.

d. Let typeDefIR' be typeDefIR?.

e. Check that typeDefIR' has type typedefTypeIR.

f. Let TYPEDEF $nameIR_{alias}$ $typeIR_{alias}$ be typeDefIR'.

g. Let (nameIR, typeArgumentIR*) be type alias in $typeIR_{alias}$ resolved to aliased name and type arguments.

h. Let callResolution<constructorTypeDefIR>? be the overloaded constructor definition of BARE nameIR `( argumentIR* `) in TC.

i. Check that callResolution<constructorTypeDefIR>? is defined.

j. Let callableId ':' constructorTypeDefIR '#' $id_{default}$* '#' $id_{optional}$* be callResolution<constructorTypeDefIR>?.

k. Let (constructorTypeIR, $typeId_{impl}$*) be constructorTypeDefIR specialized by typeArgumentIR*.

l. The result is constructorTypeIR with fresh type variables $typeId_{impl}$*, defaults $id_{default}$* and optionals $id_{optional}$*.

A constructor may be referenced through a type alias. The underlying object name and its type arguments are recovered with:

Type alias in typeIR resolved to aliased name and type arguments

1. If let TYPEDEF $nameIR_{alias}$ $typeIR_{alias}$ of type typedefTypeIR be typeIR:

a. Return type alias in $typeIR_{alias}$ resolved to aliased name and type arguments.

2. Else if let EXTERN $nameIR_{alias}$ `< $typeIR_{arg}$* `> _ of type externObjectTypeIR be typeIR:

a. Return ($nameIR_{alias}$, $typeIR_{arg}$*).

3. Else if let PACKAGE $nameIR_{alias}$ `< $typeIR_{arg}$* `> `{ _* `} of type packageObjectTypeIR be typeIR:

a. Return ($nameIR_{alias}$, $typeIR_{arg}$*).

During compile-time evaluation, constructors are resolved by:

Constructor_inst:

- Finding constructor for prefixedNameIR `< typeArgumentListIR `> `( argumentListIR `) under context IC at p:
- The result is constructor constructorDef `< typeArgumentListIR `> with defaults id* and optionals id'*.

Finding constructor for prefixedNameIR `< typeArgumentIR'* `> `( argumentIR* `) under context IC at p:

1. Try:
    a. If the type definition of prefixedNameIR from the p layer of IC is none:
        i. Let callResolution<constructorDef>? be the overloaded constructor definition of prefixedNameIR `( argumentIR* `) in IC.
        ii. If callResolution<constructorDef>? is defined:
            A. Let _ ':' constructorDef '#' $id_{default}$* '#' $id_{optional}$* be callResolution<constructorDef>?.
            B. The result is constructor constructorDef `< typeArgumentIR'* `> with defaults $id_{default}$* and optionals $id_{optional}$*.
2. Then, try:
    a. Let typeDefIR'? be the type definition of prefixedNameIR from the p layer of IC.
    b. If typeDefIR'? is defined:
        i. Let typeDefIR be typeDefIR'?.
        ii. If typeDefIR does not have type typedefTypeIR:
            A. Let callResolution<constructorDef>? be the overloaded constructor definition of prefixedNameIR `( argumentIR* `) in IC.
            B. If callResolution<constructorDef>? is defined:
                I. Let _ ':' constructorDef '#' $id_{default}$* '#' $id_{optional}$* be callResolution<constructorDef>?.
                II. The result is constructor constructorDef `< typeArgumentIR'* `> with defaults $id_{default}$* and optionals $id_{optional}$*.
3. Then, try:
    a. Check that typeArgumentIR'* is an empty list.
    b. Let typeDefIR? be the type definition of prefixedNameIR from the p layer of IC.
    c. Check that typeDefIR? is defined.
    d. Let typeDefIR' be typeDefIR?.
    e. Check that typeDefIR' has type typedefTypeIR.
    f. Let TYPEDEF $nameIR_{alias}$ $typeIR_{alias}$ be typeDefIR'.
    g. Let (nameIR, typeArgumentIR*) be type alias in $typeIR_{alias}$ resolved to aliased name and type arguments.
    h. Let callResolution<constructorDef>? be the overloaded constructor definition of $_{BARE}$ nameIR `( argumentIR* `) in IC.
    i. Check that callResolution<constructorDef>? is defined.
    j. Let _ ':' constructorDef '#' $id_{default}$* '#' $id_{optional}$* be callResolution<constructorDef>?.
    k. The result is constructor constructorDef `< typeArgumentIR* `> with defaults $id_{default}$* and optionals $id_{optional}$*.

19.4.2. Resolving callables

When type checking, callables are resolved by:

CallableType_ok:

- Finding type of callable call callableTargetIR `< typeArgumentListIR `> `( argumentIR* `), under cursor p and context TC:
- The result is callable type callableTypeIR with fresh type variables typeId*, defaults id* and optionals id'*.

CallableType_ok dispatch:

1. If let prefixedNameIR of type prefixedNameIR be callableTargetIR':
   a. Try:
      i. If typeArgumentIR* is an empty list: goto actionTypeIR
   b. Then, try:
      i. Goto functionTypeIR
2. Else if let typedExpressionIR$_{base}$ '.' nameIR be callableTargetIR':
   a. Try:
      i. If typeArgumentIR* is an empty list:
         A. If argumentIR'* is an empty list:
            I. Try:
               1. Goto builtinMethodTypeIR-size
            II. Then, try:
               1. Goto builtinMethodTypeIR-header
   b. Then, try:
      i. If nameIR is equal to "push_front":
         A. If typeArgumentIR* is an empty list:
            I. If argumentIR'* is a list of length 1:
               1. Let argumentIR be argumentIR'*.
               2. Goto builtinMethodTypeIR-header-stack-push-front
      ii. Else if nameIR is equal to "pop_front":
         A. If typeArgumentIR* is an empty list:
            I. If argumentIR'* is a list of length 1:
               1. Let argumentIR be argumentIR'*.
               2. Goto builtinMethodTypeIR-header-stack-pop-front
      iii. Else if nameIR is equal to "isValid":
         A. If typeArgumentIR* is an empty list:
            I. If argumentIR'* is an empty list: goto builtinMethodTypeIR-header-union
   c. Then, try:
      i. Goto externMethodTypeIR
   d. Then, try:
      i. Check that nameIR is equal to "apply".
      ii. Check that typeArgumentIR* is an empty list.
      iii. Try:
         A. Goto parserApplyMethodTypeIR
      iv. Then, try:
         A. Goto controlApplyMethodTypeIR
      v. Then, try:

A. Check that argumentIR'* is an empty list.

B. Goto tableApplyMethodTypeIR

3. Else if let `( callableTargetIR `) be callableTargetIR': goto parenthesized

At runtime, callables are resolved to callees by:

Callee_eval:

- Finding callee callableTargetIR `( argumentListIR `), under context EC and state ARCH at p:
- The result is context EC, state ARCH, and callee calleeResult.

Callee_eval dispatch:

1. Try:
    a. If let referenceExpressionIR of type referenceExpressionIR be callableTargetIR':
        i. Try:
            A. Goto actionCallee
        ii. Then, try:
            A. Goto externFunctionCallee
        iii. Then, try:
            A. Goto definedFunctionCallee
    b. Else if let typedExpressionIR '.' t be callableTargetIR':
        i. If t is equal to "push_front": goto builtinPushFrontMethodCallee
        ii. Else if t is equal to "pop_front": goto builtinPopFrontMethodCallee
        iii. Else if t is equal to "isValid": goto builtinIsValidMethodCallee
        iv. Else if t is equal to "setValid": goto builtinSetValidMethodCallee
        v. Else if t is equal to "setInvalid": goto builtinSetInvalidMethodCallee
2. Then, try:
    a. Let argumentIR* be argumentListIR.
    b. If let typedExpressionIR$_{base}$ '.' nameIR be callableTargetIR':
        i. If argumentIR* is an empty list: goto builtinSizeMethodCallee
3. Then, try:
    a. If let typedExpressionIR '.' nameIR be callableTargetIR':
        i. Try:
            A. Goto externMethodCallee
        ii. Then, try:
            A. If nameIR is equal to "apply":
                I. Try:
                    1. Goto parserApplyMethodCallee
                II. Then, try:
                    1. Goto controlApplyMethodCallee

4. Then, try:
    a. Let argumentIR* be argumentListIR.
    b. If let typedExpressionIR '.' t be callableTargetIR':
        i. If t is equal to "apply":
            A. If argumentIR* is an empty list: goto tableApplyMethodCallee
5. Then, try:
    a. Check that callableTargetIR' matches pattern `( % `).
    b. Let `( callableTargetIR `) be callableTargetIR'.
    c. Goto parenthesized

While resolving callees, exit; statements may be encountered. Thus, the result of callee resolution is:

```
<span id="calleeResult"></span>
<span id="callee"></span>
<span id="actionCallee"></span>
<span id="functionCallee"></span>
<span id="methodCallee"></span>
```

```
calleeResult
    : continueResult<callee> | abortResult
    ;

callee
    : actionCallee | functionCallee | methodCallee
    ;

actionCallee
    : ACTION cursor . nameIR ( parameterListIR # id* ) blockStatementIR
    ;

functionCallee
    : definedFunctionCallee | externFunctionCallee
    ;

methodCallee
    : builtinMethodCallee | externMethodCallee | parserApplyMethodCallee
    | controlApplyMethodCallee | tableApplyMethodCallee
    ;
```

The following subsections describe the two phases, for each kind of callable.

19.4.2.1. Actions

During type checking, actions are resolved by:

```
actionTypeIR
    : annotationList ACTION nameIR ( parameterIR* )
    ;
```

```
<span id="CallableType_ok-actionTypeIR"></span>
```

Finding type of callable call prefixedNameIR `< · `> `( argumentIR'* `), under cursor p and context TC:

1. Let callResolution<callableTypeDefIR>? be the overloaded callable definition of prefixedNameIR `( argumentIR'* `) from the p layer of TC.

2. If callResolution<callableTypeDefIR>? is defined:
    a. Let callableId ':' callableTypeDefIR '#' $id_{default}$* '#' $id_{optional}$* be callResolution<callableTypeDefIR>?.
    b. If let actionTypeIR of type actionTypeIR be callableTypeDefIR:
        i. The result is callable type actionTypeIR with fresh type variables ·, defaults $id_{default}$* and optionals $id_{optional}$*.

During runtime evaluation, actions are resolved by:

```
actionCallee
    : ACTION cursor . nameIR ( parameterListIR # id* ) blockStatementIR
    ;
```

Finding callee referenceExpressionIR `( argumentListIR `), under context EC and state ARCH at p:

1. Let callResolution<(cursor, callableDef)>? be the overloaded callable definition of referenceExpressionIR(argumentListIR) in the p layer of EC.
2. If callResolution<(cursor, callableDef)>? is defined:
    a. Let _ ':' (cursor, callableDef) '#' $id_{default}$* '#' _* be callResolution<(cursor, callableDef)>?.
    b. If let (p_{ref}, actionDef) of type (cursor, actionDef) be (cursor, callableDef):
        i. Let _ ACTION nameIR `( parameterListIR `) blockStatementIR be actionDef.
        ii. Let actionCallee be ACTION p_{ref} '.' nameIR `( parameterListIR '#' $id_{default}$* `) blockStatementIR.
        iii. The result is context EC, state ARCH, and callee $_{CONT}$ actionCallee.

19.4.2.2. Functions

During type checking, functions are resolved by:

```
functionTypeIR
    : definedFunctionTypeIR | externFunctionTypeIR
    ;

definedFunctionTypeIR
    : FUNCTION nameIR ( parameterIR* ) : typeIR
    ;

externFunctionTypeIR
    : EXTERN_FUNCTION nameIR ( parameterIR* ) : typeIR
    ;
```

Finding type of callable call prefixedNameIR `< typeArgumentIR* `> `( argumentIR'* `), under cursor p and context TC:

1. Let callResolution<callableTypeDefIR>? be the overloaded callable definition of prefixedNameIR `( argumentIR'* `) from the p layer of TC.
2. Check that callResolution<callableTypeDefIR>? is defined.
3. Let callableId ':' callableTypeDefIR '#' $id_{default}$* '#' $id_{optional}$* be callResolution<callableTypeDefIR>?.
4. Check that callableTypeDefIR has type functionTypeDefIR.

5. Let functionTypeDefIR be callableTypeDefIR.
6. Let (callableTypeIR, typeId*) be functionTypeDefIR specialized by $typeArgumentIR^*$.
7. Check that (callableTypeIR, typeId*) has type (functionTypeIR, typeId*).
8. Let (functionTypeIR, $typeId_{impl}^*$) be (callableTypeIR, typeId*).
9. The result is callable type functionTypeIR with fresh type variables $typeId_{impl}^*$, defaults $id_{default}^*$ and optionals $id_{optional}^*$.

During runtime evaluation, functions are resolved by:

```
<span id="definedFunctionCallee"></span>
<span id="externFunctionCallee"></span>
```

```
functionCallee
    : definedFunctionCallee | externFunctionCallee
    ;

definedFunctionCallee
    : FUNCTION nameIR < typeParameterListIR > ( parameterListIR # id* )
        blockStatementIR
    ;

externFunctionCallee
    : EXTERN_FUNCTION nameIR < typeParameterListIR >
        ( parameterListIR # id* # id* )
    ;
```

For extern functions,

```
<span id="Callee_eval-externFunctionCallee"></span>
```

Finding callee referenceExpressionIR `( argumentListIR `), under context EC and state ARCH at p:

1. Let callResolution<(cursor, callableDef)>? be the overloaded callable definition of referenceExpressionIR(argumentListIR) in the p layer of EC.
2. If callResolution<(cursor, callableDef)>? is defined:
    a. Let _ ':' (_, callableDef) '#' $id_{default}^*$ '#' $id_{optional}^*$ be callResolution<(cursor, callableDef)>?.
    b. If let externFunctionDef of type externFunctionDef be callableDef:
        i. Let EXTERN_FUNCTION nameIR `< typeParameterListIR `> `( parameterListIR `) be externFunctionDef.
        ii. Let externFunctionCallee be EXTERN_FUNCTION nameIR `< typeParameterListIR `> `( parameterListIR '#' $id_{default}^*$ '#' $id_{optional}^*$ `).
        iii. The result is context EC, state ARCH, and callee $_{CONT}$ externFunctionCallee.

For user-defined functions,

```
<span id="Callee_eval-definedFunctionCallee"></span>
```

Finding callee referenceExpressionIR `( argumentListIR `), under context EC and state ARCH at p:

1. Let callResolution<(cursor, callableDef)>? be the overloaded callable definition of referenceExpressionIR(argumentListIR) in the p layer of EC.
2. If callResolution<(cursor, callableDef)>? is defined:
    a. Let _ ':' (_, callableDef) '#' $id_{default}^*$ '#' $_^*$ be callResolution<(cursor, callableDef)>?.

b. If let definedFunctionDef of type definedFunctionDef be callableDef:

i. Let FUNCTION nameIR `< typeParameterListIR `> `( parameterListIR `) blockStatementIR be definedFunctionDef.

ii. Let definedFunctionCallee be FUNCTION nameIR `< typeParameterListIR `> `( parameterListIR '#' $id_{default}^{*}$ `) blockStatementIR.

iii. The result is context EC, state ARCH, and callee $_{CONT}$ definedFunctionCallee.

19.4.2.3. Built-in methods

During type checking, built-in methods are resolved as:

```
builtinMethodTypeIR
    : builtinIsValidMethodTypeIR | builtinSetValidMethodTypeIR
    | builtinSetInvalidMethodTypeIR | builtinPushFrontMethodTypeIR
    | builtinPopFrontMethodTypeIR | builtinSizeMethodTypeIR
    ;
```

And during runtime evaluation, built-in methods are resolved as:

```
builtinMethodCallee
    : builtinIsValidMethodCallee | builtinSetValidMethodCallee
    | builtinSetInvalidMethodCallee | builtinPushFrontMethodCallee
    | builtinPopFrontMethodCallee | builtinSizeMethodCallee
    ;
```

19.4.2.3.1. minSizeInBits, maxSizeInBits, minSizeInBytes, and maxSizeInBytes

During type checking,

Finding type of callable call typedExpressionIR$_{base}$ '.' nameIR `< · `> `( · `), under cursor p and context TC:

1. If nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"]:
    a. Let typeIR$_{base}$ be the type of typedExpressionIR$_{base}$.
    b. If typeIR$_{base}$ has a defined size:
        i. Let methodTypeIR be BUILTIN_SIZE_OF.
        ii. The result is callable type methodTypeIR with fresh type variables ·, defaults · and optionals ·.

During runtime evaluation,

Finding callee typedExpressionIR$_{base}$ '.' nameIR `( · `), under context EC and state ARCH at p:

1. If nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"]:
    a. Let a pair of type typeIR$_{base}$ and compile-time known-ness _ be the note of typedExpressionIR$_{base}$.
    b. Let builtinSizeMethodCallee be BUILTIN_SIZE_OF nameIR typeIR$_{base}$.

c. The result is context EC, state ARCH, and callee $_{CONT}$ builtinSizeMethodCallee.

19.4.2.3.2. push_front and pop_front on header stacks

During type checking,

Finding type of callable call typedExpressionIR$_{base}$ '.' "push_front" `< · `> `( argumentIR `), under cursor p and context TC:

1. Let typeIR$_{base}$ be the type of typedExpressionIR$_{base}$.
2. Let typeIR be typeIR$_{base}$ with typedefs unrolled.
3. If typeIR has type headerStackTypeIR:
   a. Let typedLvalueIR$^{?}$ be the l-value form of typedExpressionIR$_{base}$.
   b. If typedLvalueIR$^{?}$ is defined:
      i. Let typedLvalueIR$_{base}$ be typedLvalueIR$^{?}$.
      ii. Let lvalue$_{base}$ be $lift_typedLvalueIR(typedLvalueIR$_{base}$).
      iii. Let _ be the result of typing lvalue$_{base}$, under context TC at p.
      iv. Let methodTypeIR be BUILTIN_PUSH_FRONT.
      v. The result is callable type methodTypeIR with fresh type variables ·, defaults · and optionals ·.

Finding type of callable call typedExpressionIR$_{base}$ '.' "pop_front" `< · `> `( argumentIR `), under cursor p and context TC:

1. Let typeIR$_{base}$ be the type of typedExpressionIR$_{base}$.
2. Let typeIR be typeIR$_{base}$ with typedefs unrolled.
3. If typeIR has type headerStackTypeIR:
   a. Let typedLvalueIR$^{?}$ be the l-value form of typedExpressionIR$_{base}$.
   b. If typedLvalueIR$^{?}$ is defined:
      i. Let typedLvalueIR$_{base}$ be typedLvalueIR$^{?}$.
      ii. Let lvalue$_{base}$ be $lift_typedLvalueIR(typedLvalueIR$_{base}$).
      iii. Let _ be the result of typing lvalue$_{base}$, under context TC at p.
      iv. Let methodTypeIR be BUILTIN_POP_FRONT.
      v. The result is callable type methodTypeIR with fresh type variables ·, defaults · and optionals ·.

During runtime evaluation,

Finding callee typedExpressionIR '.' "push_front" `( argumentListIR `), under context EC and state ARCH at p:

1. Try:
   a. Let typedLvalueIR$'^{?}$ be the l-value form of typedExpressionIR.
   b. If typedLvalueIR$'^{?}$ is defined:

i. Let typedLvalueIR be typedLvalueIR'?.

ii. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

iii. If let abortResult of type abortResult be storageReferenceResult:

A. The result is context EC_1, state $ARCH_1$, and callee abortResult.

2. Then, try:

a. If argumentListIR is a list of length 1:

i. Let typedLvalueIR'? be the l-value form of typedExpressionIR.

ii. If typedLvalueIR'? is defined:

A. Let typedLvalueIR be typedLvalueIR'?.

B. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

C. If let $_{CONT}$ storageReference'? of type continueResult<storageReference?> be storageReferenceResult:

I. If storageReference'? is defined:

1. Let storageReference be storageReference'?.

2. Let builtinPushFrontMethodCallee be BUILTIN_PUSH_FRONT storageReference.

3. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ builtinPushFrontMethodCallee.

Finding callee typedExpressionIR '.' "pop_front" `( argumentListIR `), under context EC and state ARCH at p:

1. Try:

a. Let typedLvalueIR'? be the l-value form of typedExpressionIR.

b. If typedLvalueIR'? is defined:

i. Let typedLvalueIR be typedLvalueIR'?.

ii. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

iii. If let abortResult of type abortResult be storageReferenceResult:

A. The result is context EC_1, state $ARCH_1$, and callee abortResult.

2. Then, try:

a. If argumentListIR is a list of length 1:

i. Let typedLvalueIR'? be the l-value form of typedExpressionIR.

ii. If typedLvalueIR'? is defined:

A. Let typedLvalueIR be typedLvalueIR'?.

B. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

C. If let $_{CONT}$ storageReference'? of type continueResult<storageReference?> be storageReferenceResult:

I. If storageReference'? is defined:

1. Let storageReference be storageReference'?.

2. Let builtinPopFrontMethodCallee be BUILTIN_POP_FRONT storageReference.

3. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ builtinPopFrontMethodCallee.

19.4.2.3.3. isValid, setValid, and setInvalid on headers and header unions

During type checking,

Finding type of callable call $typedExpressionIR_{base}$ '.' nameIR `< · `> `( · `), under cursor p and context TC:

1. If nameIR is equal to "isValid":
    a. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
    b. Let typeIR be $typeIR_{base}$ with typedefs unrolled.
    c. If typeIR has type headerTypeIR:
        i. Let methodTypeIR be BUILTIN_IS_VALID.
        ii. The result is callable type methodTypeIR with fresh type variables ·, defaults · and optionals ·.
2. Else if nameIR is equal to "setValid":
    a. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
    b. Let typeIR be $typeIR_{base}$ with typedefs unrolled.
    c. If typeIR has type headerTypeIR:
        i. Let $typedLvalueIR^{?}$ be the l-value form of $typedExpressionIR_{base}$.
        ii. If $typedLvalueIR^{?}$ is defined:
            A. Let $typedLvalueIR_{base}$ be $typedLvalueIR^{?}$.
            B. Let $lvalue_{base}$ be $lift_typedLvalueIR($typedLvalueIR_{base}$).
            C. Let _ be the result of typing $lvalue_{base}$, under context TC at p.
            D. Let methodTypeIR be BUILTIN_SET_VALID.
            E. The result is callable type methodTypeIR with fresh type variables ·, defaults · and optionals ·.
3. Else if nameIR is equal to "setInvalid":
    a. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
    b. Let typeIR be $typeIR_{base}$ with typedefs unrolled.
    c. If typeIR has type headerTypeIR:
        i. Let $typedLvalueIR^{?}$ be the l-value form of $typedExpressionIR_{base}$.
        ii. If $typedLvalueIR^{?}$ is defined:
            A. Let $typedLvalueIR_{base}$ be $typedLvalueIR^{?}$.
            B. Let $lvalue_{base}$ be $lift_typedLvalueIR($typedLvalueIR_{base}$).
            C. Let _ be the result of typing $lvalue_{base}$, under context TC at p.
            D. Let methodTypeIR be BUILTIN_SET_INVALID.
            E. The result is callable type methodTypeIR with fresh type variables ·, defaults · and optionals ·.

Finding type of callable call $typedExpressionIR_{base}$ '.' "isValid" `< · `> `( · `), under cursor p and context TC:

1. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
2. Let typeIR be $typeIR_{base}$ with typedefs unrolled.
3.

If typeIR has type headerUnionTypeIR:

a. Let methodTypeIR be BUILTIN_IS_VALID.

b. The result is callable type methodTypeIR with fresh type variables ·, defaults · and optionals ·.

During runtime evaluation,

Finding callee typedExpressionIR '.' "isValid" `( argumentListIR `), under context EC and state ARCH at p:

1. Try:
    a. Let typedLvalueIR'? be the l-value form of typedExpressionIR.
    b. If typedLvalueIR'? is defined:
        i. Let typedLvalueIR be typedLvalueIR'?.
        ii. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
        iii. If let abortResult of type abortResult be storageReferenceResult:
            A. The result is context EC_1, state $ARCH_1$, and callee abortResult.
2. Then, try:
    a. If argumentListIR is an empty list:
        i. Let typedLvalueIR'? be the l-value form of typedExpressionIR.
        ii. If typedLvalueIR'? is defined:
            A. Let typedLvalueIR be typedLvalueIR'?.
            B. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
            C. If let $_{CONT}$ storageReference'? of type continueResult<storageReference?> be storageReferenceResult:
                I. If storageReference'? is defined:
                    1. Let storageReference be storageReference'?.
                    2. Let builtinIsValidMethodCallee be BUILTIN_IS_VALID storageReference.
                    3. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ builtinIsValidMethodCallee.

Finding callee typedExpressionIR '.' "setValid" `( argumentListIR `), under context EC and state ARCH at p:

1. Try:
    a. Let typedLvalueIR'? be the l-value form of typedExpressionIR.
    b. If typedLvalueIR'? is defined:
        i. Let typedLvalueIR be typedLvalueIR'?.
        ii. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
        iii. If let abortResult of type abortResult be storageReferenceResult:
            A. The result is context EC_1, state $ARCH_1$, and callee abortResult.
2. Then, try:

a. If argumentListIR is an empty list:

i. Let typedLvalueIR'? be the l-value form of typedExpressionIR.

ii. If typedLvalueIR'? is defined:

A. Let typedLvalueIR be typedLvalueIR'?.

B. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

C. If let $_{CONT}$ storageReference'? of type continueResult<storageReference?> be storageReferenceResult:

I. If storageReference'? is defined:

1. Let storageReference be storageReference'?.

2. Let builtinSetValidMethodCallee be BUILTIN_SET_VALID storageReference.

3. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ builtinSetValidMethodCallee.

Finding callee typedExpressionIR '.' "setInvalid" `( argumentListIR `), under context EC and state ARCH at p:

1. Try:

a. Let typedLvalueIR'? be the l-value form of typedExpressionIR.

b. If typedLvalueIR'? is defined:

i. Let typedLvalueIR be typedLvalueIR'?.

ii. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

iii. If let abortResult of type abortResult be storageReferenceResult:

A. The result is context EC_1, state $ARCH_1$, and callee abortResult.

2. Then, try:

a. If argumentListIR is an empty list:

i. Let typedLvalueIR'? be the l-value form of typedExpressionIR.

ii. If typedLvalueIR'? is defined:

A. Let typedLvalueIR be typedLvalueIR'?.

B. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.

C. If let $_{CONT}$ storageReference'? of type continueResult<storageReference?> be storageReferenceResult:

I. If storageReference'? is defined:

1. Let storageReference be storageReference'?.

2. Let builtinSetInvalidMethodCallee be BUILTIN_SET_INVALID storageReference.

3. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ builtinSetInvalidMethodCallee.

19.4.2.4. Extern methods

During type checking, extern methods are resolved by:

```
externMethodTypeIR
    : EXTERN_METHOD nameIR ( parameterIR* ) : typeIR
    | EXTERN_METHOD ABSTRACT nameIR ( parameterIR* ) : typeIR
```

```
    ;
```

Finding type of callable call typedExpressionIR$_{base}$ '.' nameIR `< typeArgumentIR$^{*}$ `> `( argumentIR'$^{*}$ `), under cursor p and context TC:

1. Let typeIR$_{base}$ be the type of typedExpressionIR$_{base}$.
2. Let typeIR be typeIR$_{base}$ with typedefs unrolled.
3. If let EXTERN typeId `< _$^{*}$ `> `{ ( callableId$_{extern}$ ':' externMethodTypeDefIR$_{extern}$ )$^{*}$ `} of type externObjectTypeIR be typeIR:
    a. Let callTargetKey be the call target key for nameIR `( argumentIR'$^{*}$ `).
    b. Let callResolution<externMethodTypeDefIR>$^{?}$ be finding the item in `{ ( callableId$_{extern}$ ':' externMethodTypeDefIR$_{extern}$ )$^{*}$ `} matching callTargetKey using parameter getter $parameterListIR_of_externMethodTypeDefIR.
    c. If callResolution<externMethodTypeDefIR>$^{?}$ is defined:
        i. Let _ ':' externMethodTypeDefIR '#' id$_{default}^{*}$ '#' id$_{optional}^{*}$ be callResolution<externMethodTypeDefIR>$^{?}$.
        ii. Let (callableTypeIR, typeId*) be externMethodTypeDefIR specialized by typeArgumentIR*.
        iii. If let (externMethodTypeIR, typeId$_{impl}^{*}$) of type (externMethodTypeIR, typeId*) be (callableTypeIR, typeId*):
            A. The result is callable type externMethodTypeIR with fresh type variables typeId$_{impl}^{*}$, defaults id$_{default}^{*}$ and optionals id$_{optional}^{*}$.

During runtime evaluation, extern methods are resolved by:

```
externMethodCallee
    : EXTERN_METHOD objectId . nameIR < typeParameterListIR >
        ( parameterListIR # id* # id* ) { theta ; frame ; blockStatementIR? }
    ;
```

Finding callee typedExpressionIR '.' nameIR `( argumentListIR `), under context EC and state ARCH at p:

1. Let typedLvalueIR'$^{?}$ be the l-value form of typedExpressionIR.
2. If typedLvalueIR'$^{?}$ is defined:
    a. Let typedLvalueIR be typedLvalueIR'$^{?}$.
    b. Let context EC$_1$, state ARCH$_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
    c. If let abortResult of type abortResult be storageReferenceResult:
        i. The result is context EC$_1$, state ARCH$_1$, and callee abortResult.
    d. Else:
        i. Let $_{CONT}$ storageReference'$^{?}$ be storageReferenceResult.
        ii. If storageReference'$^{?}$ is defined:
            A. Let storageReference be storageReference'$^{?}$.
            B. Let value value be the result of reading from storageReference, under context EC$_1$ and state ARCH$_1$

at p.

C. If let REF objectId of type objectReferenceValue be value:

I. Let object'? be the object objectId in $ARCH_1$.

II. If object'? is defined:

1. Let object be object'?.

2. If let EXTERN _ `< theta `> `{ _ frame externMethodDefEnv `} of type externObject be object:

1. Let callTargetKey be the call target key for nameIR `( argumentListIR `).

2. Let callResolution<externMethodDef>? be finding the item in externMethodDefEnv matching callTargetKey using parameter getter $parameterListIR_of_externMethodDef.

3. If callResolution<externMethodDef>? is defined:

1. Let _ ':' externMethodDef '#' $id_{default}$* '#' $id_{optional}$* be callResolution<externMethodDef>?.

2. If let EXTERN_METHOD nameIR' `< typeParameterListIR `> `( parameterListIR `) blockStatementIR? be externMethodDef:

1. If nameIR' is equal to nameIR:

1. Let externMethodCallee be EXTERN_METHOD objectId '.' nameIR `< typeParameterListIR `> `( parameterListIR '#' $id_{default}$* '#' $id_{optional}$* `) `{ theta ';' frame ';' blockStatementIR? `}.

2. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ externMethodCallee.

19.4.2.5. Parser apply methods

During type checking, parser apply methods are resolved by:

```
parserApplyMethodTypeIR
    : PARSER_APPLY ( parameterIR* )
    ;
```

Finding type of callable call $typedExpressionIR_{base}$ '.' "apply" `< · `> `( argumentIR'* `), under cursor p and context TC:

1. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.

2. Let typeIR be $typeIR_{base}$ with typedefs unrolled.

3. If let PARSER _ `< _* `> `( parameterIR* `) of type parserObjectTypeIR be typeIR:

a. Let parserApplyMethodTypeIR be PARSER_APPLY `( parameterIR* `).

b. For each parameterIR in parameterIR*:

- Let _ _ _ id_{param} _ be parameterIR.

Let id_{param}* be the resulting list.

c. Let callableId be "apply" `( $id_{param}$* `).

d. Let callTargetKey be the call target key for "apply" `( argumentIR'* `).

e. Let callResolution<parserApplyMethodTypeIR>? be finding the item in `{ callableId ':' parserApplyMethodTypeIR `} matching callTargetKey using parameter getter

$parameterListIR_of_parserApplyMethodTypeIR.

f. If callResolution<parserApplyMethodTypeIR>? is defined:

i. Let _ ':' _ '#' $id_{default}$* '#' $id_{optional}$* be callResolution<parserApplyMethodTypeIR>?.

ii. The result is callable type parserApplyMethodTypeIR with fresh type variables ·, defaults $id_{default}$* and optionals $id_{optional}$*.

During runtime evaluation, parser apply methods are resolved by:

```
parserApplyMethodCallee
    : PARSER objectId . APPLY ( parameterListIR # id* )
        { theta ; frame ; parserLocalDeclarationListIR ; stateEnv }
    ;
```

Finding callee typedExpressionIR '.' "apply" `( argumentListIR `), under context EC and state ARCH at p:

1. Let typedLvalueIR'? be the l-value form of typedExpressionIR.
2. If typedLvalueIR'? is defined:
   a. Let typedLvalueIR be typedLvalueIR'?.
   b. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
   c. If let abortResult of type abortResult be storageReferenceResult:
      i. The result is context EC_1, state $ARCH_1$, and callee abortResult.
   d. Else:
      i. Let $_{CONT}$ storageReference'? be storageReferenceResult.
      ii. If storageReference'? is defined:
         A. Let storageReference be storageReference'?.
         B. Let value value be the result of reading from storageReference, under context EC_1 and state $ARCH_1$ at p.
         C. If let REF objectId of type objectReferenceValue be value:
            I. Let object'? be the object objectId in $ARCH_1$.
            II. If object'? is defined:
               1. Let object be object'?.
               2. If let PARSER `< theta `> `( parameterListIR `) `{ frame parserLocalDeclarationListIR stateEnv `} of type parserObject be object:
                  1. Let callableId be the callable identifier for "apply" `( parameterListIR `).
                  2. Let parserApplyMethodDef be PARSER_APPLY `( parameterListIR `) `{ parserLocalDeclarationListIR ';' stateEnv `}.
                  3. Let callTargetKey be the call target key for "apply" `( argumentListIR `).
                  4. Let callResolution<parserApplyMethodDef>? be finding the item in `{ callableId ':' parserApplyMethodDef `} matching callTargetKey using parameter getter $parameterListIR_of_parserApplyMethodDef.
                  5. If callResolution<parserApplyMethodDef>? is defined:

1. Let _ ':' _ '#' $id_{default}^{*}$ '#' _* be callResolution<parserApplyMethodDef>?.
2. Let parserApplyMethodCallee be PARSER objectId '.' APPLY `( parameterListIR '#' $id_{default}^{*}$ `) `{ theta ';' frame ';' parserLocalDeclarationListIR ';' stateEnv `}.
3. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ parserApplyMethodCallee.

19.4.2.6. Control apply methods

During type checking, control apply methods are resolved by:

```
controlApplyMethodTypeIR
    : CONTROL_APPLY ( parameterIR* )
    ;
```

Finding type of callable call $typedExpressionIR_{base}$ '.' "apply" `< · `> `( $argumentIR'^{*}$ `), under cursor p and context TC:

1. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
2. Let typeIR be $typeIR_{base}$ with typedefs unrolled.
3. If let CONTROL _ `< _* `> `( $parameterIR^{*}$ `) of type controlObjectTypeIR be typeIR:
   a. Let controlApplyMethodTypeIR be CONTROL_APPLY `( $parameterIR^{*}$ `).
   b. For each parameterIR in $parameterIR^{*}$:
      - Let _ _ _ id_{param} _ be parameterIR.

      Let id_{param}^{*} be the resulting list.
   c. Let callableId be "apply" `( $id_{param}^{*}$ `).
   d. Let callTargetKey be the call target key for "apply" `( $argumentIR'^{*}$ `).
   e. Let callResolution<controlApplyMethodTypeDefIR>? be finding the item in `{ callableId ':' controlApplyMethodTypeIR `} matching callTargetKey using parameter getter $parameterListIR_of_controlApplyMethodTypeIR.
   f. If callResolution<controlApplyMethodTypeDefIR>? is defined:
      i. Let _ ':' _ '#' $id_{default}^{*}$ '#' $id_{optional}^{*}$ be callResolution<controlApplyMethodTypeDefIR>?.
      ii. The result is callable type controlApplyMethodTypeIR with fresh type variables ·, defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$.

During runtime evaluation, control apply methods are resolved by:

```
controlApplyMethodCallee
    : CONTROL objectId . APPLY ( parameterListIR # id* )
        {
          theta ; frame ; controlLocalDeclarationListIR ; actionDefEnv ;
          controlBodyIR
        }
    ;
```

Finding callee typedExpressionIR '.' "apply" `( argumentListIR `), under context EC and state ARCH at p:

1. Let typedLvalueIR'? be the l-value form of typedExpressionIR.
2. If typedLvalueIR'? is defined:
    a. Let typedLvalueIR be typedLvalueIR'?.
    b. Let context EC_1, state $ARCH_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
    c. If let abortResult of type abortResult be storageReferenceResult:
        i. The result is context EC_1, state $ARCH_1$, and callee abortResult.
    d. Else:
        i. Let $_{CONT}$ storageReference'? be storageReferenceResult.
        ii. If storageReference'? is defined:
            A. Let storageReference be storageReference'?.
            B. Let value value be the result of reading from storageReference, under context EC_1 and state $ARCH_1$ at p.
            C. If let REF objectId of type objectReferenceValue be value:
                I. Let object'? be the object objectId in $ARCH_1$.
                II. If object'? is defined:
                    1. Let object be object'?.
                    2. If let CONTROL `< theta `> `( parameterListIR `) `{ frame controlLocalDeclarationListIR actionDefEnv controlBodyIR `} of type controlObject be object:
                        1. Let callableId be the callable identifier for "apply" `( parameterListIR `).
                        2. Let controlApplyMethodDef be CONTROL_APPLY `( parameterListIR `) `{ controlLocalDeclarationListIR ';' actionDefEnv ';' controlBodyIR `}.
                        3. Let callTargetKey be the call target key for "apply" `( argumentListIR `).
                        4. Let callResolution<controlApplyMethodDef>? be finding the item in `{ callableId ':' controlApplyMethodDef `} matching callTargetKey using parameter getter $parameterListIR_of_controlApplyMethodDef.
                        5. If callResolution<controlApplyMethodDef>? is defined:
                            1. Let _ ':' _ '#' $id_{default}$* '#' _* be callResolution<controlApplyMethodDef>?.
                            2. Let controlApplyMethodCallee be CONTROL objectId '.' APPLY `( parameterListIR '#' $id_{default}$* `) `{ theta ';' frame ';' controlLocalDeclarationListIR ';' actionDefEnv ';' controlBodyIR `}.
                            3. The result is context EC_1, state $ARCH_1$, and callee $_{CONT}$ controlApplyMethodCallee.

19.4.2.7. Table apply methods

During type checking, table apply methods are resolved by:

```
tableApplyMethodTypeIR
    : TABLE_APPLY : tableMetadataStructTypeIR
    ;
```

Finding type of callable call typedExpressionIR$_{base}$ '.' "apply" `< · `> `( · `), under cursor p and context TC:

1. Let typeIR$_{base}$ be the type of typedExpressionIR$_{base}$.
2. Let typeIR be typeIR$_{base}$ with typedefs unrolled.
3. Check that typeIR has type tableObjectTypeIR.
4. Let TABLE _ `{ tableMetadataStructTypeIR `} be typeIR.
5. Let tableApplyMethodTypeIR be TABLE_APPLY ':' tableMetadataStructTypeIR.
6. The result is callable type tableApplyMethodTypeIR with fresh type variables ·, defaults · and optionals ·.

During runtime evaluation, table apply methods are resolved by:

```
tableApplyMethodCallee
    : TABLE objectId . APPLY { frame ; tableObjectProperty }
    ;
```

Finding callee typedExpressionIR '.' "apply" `( · `), under context EC and state ARCH at p:

1. Let typedLvalueIR'$^?$ be the l-value form of typedExpressionIR.
2. If typedLvalueIR'$^?$ is defined:
   a. Let typedLvalueIR be typedLvalueIR'$^?$.
   b. Let context EC$_1$, state ARCH$_1$, and result storageReferenceResult be the result of evaluating typedLvalueIR, under context EC and state ARCH at p.
   c. If let abortResult of type abortResult be storageReferenceResult:
      i. The result is context EC$_1$, state ARCH$_1$, and callee abortResult.
   d. Else:
      i. Let $_{CONT}$ storageReference'$^?$ be storageReferenceResult.
      ii. If storageReference'$^?$ is defined:
         A. Let storageReference be storageReference'$^?$.
         B. Let value value be the result of reading from storageReference, under context EC$_1$ and state ARCH$_1$ at p.
         C. If let REF objectId of type objectReferenceValue be value:
            I. Let object'$^?$ be the object objectId in ARCH$_1$.
            II. If object'$^?$ is defined:
               1. Let object be object'$^?$.
               2. If let TABLE typeId `{ frame TBL `} of type tableObject be object:
                  1. Let tableApplyMethodCallee be TABLE objectId '.' APPLY `{ frame ';' TBL `}.
                  2. The result is context EC$_1$, state ARCH$_1$, and callee $_{CONT}$ tableApplyMethodCallee.

19.4.2.8. Parenthesized callables

During type checking, parenthesized callables are resolved by:

Finding type of callable call `( callableTargetIR `) `< typeArgumentIR* `> `( argumentIR'* `), under cursor p and context TC:

1. Let callable type callableTypeIR with fresh type variables $typeId_{inserted}^*$, defaults $id_{default}^*$ and optionals $id_{optional}^*$ be the result of finding type of callable call callableTargetIR `< typeArgumentIR* `> `( argumentIR'* `), under cursor p and context TC.
2. The result is callable type callableTypeIR with fresh type variables $typeId_{inserted}^*$, defaults $id_{default}^*$ and optionals $id_{optional}^*$.

During runtime evaluation, parenthesized callables are resolved by:

Finding callee `( callableTargetIR `) `( argumentListIR `), under context EC and state ARCH at p:

1. Let context EC_1, state $ARCH_1$, and callee calleeResult be the result of finding callee callableTargetIR `( argumentListIR `), under context EC and state ARCH at p.
2. The result is context EC_1, state $ARCH_1$, and callee calleeResult.

19.5. Type argument inference

When a call omits some type arguments, they are inferred from the types of the supplied arguments. The result of inference is an assignment of a type to each type variable to be inferred:

```
infer
    : KNOWN AS typeIR | UNKNOWN
    ;

constraint = map<typeId, infer>

inference = map<typeId, typeIR>
```

$empty_constraint(typeId*)

1. Return `{ ( typeId ':' UNKNOWN )* `}.

Type variables that are only used in @optional parameters are not strictly inferred, while other type variables are strictly inferred, meaning they must be determined by the arguments. The strictness of each type variable is represented by:

```
strict
    : STRICT | LAX
    ;
```

```
strictness = map<typeId, strict>
```

$empty_strictness(typeId*)

1. Return `{ ( typeId ':' STRICT )* `}.

Inference generates constraints from each parameter and argument pair, merges them, and resolves the merged constraint:

$infer(typeId*, parameterIR*, parameterIR$_{optional}$*, argumentIR*)

1. If typeId* matches pattern []: return an empty map.
2. Else:
   a. Let strictness$_{init}$ be $empty_strictness(typeId*).
   b. Let B be free type identifiers in parameterIR*.
   c. Let B$_{optional}$ be free type identifiers in parameterIR$_{optional}$*.
   d. Let `{ typeId$_{free}$* `} be the difference of the sets B$_{optional}$ and B.
   e. Let strict* be a list of the length of typeId$_{free}$* copies of LAX.
   f. Let strictness be strictness$_{init}$ where each of typeId$_{free}$* to strict* are added.
   g. Let constraint$_{init}$ be $empty_constraint(typeId*).
   h. Check that the length of argumentIR* is equal to the length of parameterIR*.
   i. For each argumentIR in argumentIR* and parameterIR in parameterIR*:
      - Let constraint$_{pair}$ be $infer_pair(constraint$_{init}$, parameterIR, argumentIR).

      Let constraint$_{pair}$* be the resulting list.
   j. Let constraint$_{inferred}$ be $merge_constraints(constraint$_{init}$, constraint$_{pair}$*).
   k. Let inference be $resolve_constraint(strictness, constraint$_{inferred}$).
   l. Let inference$_{resolved}$ be $resolve_inference(inference).
   m. Return inference$_{resolved}$.

$infer_pair(constraint, parameterIR, argumentIR)

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:
   a. Let typeIR$_{param}$ be the type of parameterIR.
   b. Let typeIR$_{arg}$ be the type of typedExpressionIR.
   c. Return $gen_constraint_pair(constraint, typeIR$_{param}$, typeIR$_{arg}$).
2. Else if let nameIR '=' typedExpressionIR be argumentIR:
   a. Let typeIR$_{param}$ be the type of parameterIR.
   b. Let typeIR$_{arg}$ be the type of typedExpressionIR.
   c. Return $gen_constraint_pair(constraint, typeIR$_{param}$, typeIR$_{arg}$).
3. Else if argumentIR matches pattern % '=' '_': return constraint.
4. Else: return constraint.

19.5.1. Constraint generation

Each parameter type is matched structurally against the argument type, recording a known type whenever a type variable is reached:

$gen_constraint_pair(constraint, typeIR, $typeIR_{arg'}$)

1. Try:
    a. If let $_{NAME}$ $typeId_{param}$ of type nameTypeIR be typeIR:
        i. If $typeId_{param}$ is an element of the set the domain of the map constraint:
            A. Return the map constraint with the value for $typeId_{param}$ is updated to KNOWN AS $typeIR_{arg'}$.
    b. Else if let TYPEDEF _ $typeIR_{param}$ of type typedefTypeIR be typeIR:
        i. Return $gen_constraint_pair(constraint, $typeIR_{param}$, $typeIR_{arg'}$).
    c. Else if let TYPE typeId $typeIR_{param}$ of type newTypeIR be typeIR:
        i. If let TYPE typeId' $typeIR_{arg}$ of type newTypeIR be $typeIR_{arg'}$:
            A. If typeId is equal to typeId':
                I. Return $gen_constraint_pair(constraint, $typeIR_{param}$, $typeIR_{arg}$).
    d. Else if let LIST `< $typeIR_{param}$ `> of type listTypeIR be typeIR:
        i. If let LIST `< $typeIR_{arg}$ `> of type listTypeIR be $typeIR_{arg'}$:
            A. Return $gen_constraint_pair(constraint, $typeIR_{param}$, $typeIR_{arg}$).
    e. Else if let TUPLE `< $typeIR_{param}^{*}$ `> of type tupleTypeIR be typeIR:
        i. If let TUPLE `< $typeIR_{arg}^{*}$ `> of type tupleTypeIR be $typeIR_{arg'}$:
            A. Return $gen_constraint(constraint, $typeIR_{param}^{*}$, $typeIR_{arg}^{*}$).
    f. Else if let ARRAY $typeIR_{param}$ `[ $n_{size}$ `] of type arrayTypeIR be typeIR:
        i. If let ARRAY $typeIR_{arg}$ `[ $n_{size'}$ `] of type arrayTypeIR be $typeIR_{arg'}$:
            A. If n_{size} is equal to $n_{size'}$:
                I. Return $gen_constraint_pair(constraint, $typeIR_{param}$, $typeIR_{arg}$).
    g. Else if let HEADER_STACK $typeIR_{param}$ `[ $n_{size}$ `] of type headerStackTypeIR be typeIR:
        i. If let HEADER_STACK $typeIR_{arg}$ `[ $n_{size'}$ `] of type headerStackTypeIR be $typeIR_{arg'}$:
            A. If n_{size} is equal to $n_{size'}$:
                I. Return $gen_constraint_pair(constraint, $typeIR_{param}$, $typeIR_{arg}$).
    h. Else if let STRUCT typeId `< $typeIR_{arg\_a}^{*}$ `> `{ _ $^{*}$ `} of type structTypeIR be typeIR:
        i. If let STRUCT typeId' `< $typeIR_{arg\_b}^{*}$ `> `{ _ $^{*}$ `} of type structTypeIR be $typeIR_{arg'}$:
            A. If typeId is equal to typeId':
                I. Return $gen_constraint(constraint, $typeIR_{arg_a}^{*}$, $typeIR_{arg_b}^{*}$).
    i. Else if let HEADER typeId `< $typeIR_{arg\_a}^{*}$ `> `{ _ $^{*}$ `} of type headerTypeIR be typeIR:
        i. If let HEADER typeId' `< $typeIR_{arg\_b}^{*}$ `> `{ _ $^{*}$ `} of type headerTypeIR be $typeIR_{arg'}$:
            A. If typeId is equal to typeId':
                I. Return $gen_constraint(constraint, $typeIR_{arg_a}^{*}$, $typeIR_{arg_b}^{*}$).
    j. Else if let HEADER_UNION typeId `< $typeIR_{arg\_a}^{*}$ `> `{ _ $^{*}$ `} of type headerUnionTypeIR be typeIR:
        i. If let HEADER_UNION typeId' `< $typeIR_{arg\_b}^{*}$ `> `{ _ $^{*}$ `} of type headerUnionTypeIR be $typeIR_{arg'}$:
            A. If typeId is equal to typeId':

I. Return $gen_constraint(constraint, typeIR$_{arg_a}^*$, typeIR$_{arg_b}^*$).

k. Else if let EXTERN typeId `< typeIR$_{arg\_a}^*$ `> `{ _ $^*$ `} of type externObjectTypeIR be typeIR:

i. If let EXTERN typeId' `< typeIR$_{arg\_b}^*$ `> `{ _ $^*$ `} of type externObjectTypeIR be typeIR$_{arg}$':

A. If typeId is equal to typeId':

I. Return $gen_constraint(constraint, typeIR$_{arg_a}^*$, typeIR$_{arg_b}^*$).

l. Else if let PARSER _ `< _ $^*$ `> `( parameterIR$_{param}^*$ `) of type parserObjectTypeIR be typeIR:

i. If let PARSER _ `< _ $^*$ `> `( parameterIR$_{arg}^*$ `) of type parserObjectTypeIR be typeIR$_{arg}$':

A. For each parameterIR$_{param}$ in parameterIR$_{param}^*$:

- Let _ direction typeIR$_{param}$ _ _ be parameterIR$_{param}$.

Let direction* and typeIR$_{param}^*$ be the resulting lists.

B. For each parameterIR$_{arg}$ in parameterIR$_{arg}^*$:

- Let _ direction' typeIR$_{arg}$ _ _ be parameterIR$_{arg}$.

Let direction'* and typeIR$_{arg}^*$ be the resulting lists.

C. If the length of direction* is equal to the length of direction'*:

I. If direction' is equal to direction, for all direction in direction* and direction' in direction'*:

1. Return $gen_constraint(constraint, typeIR$_{param}^*$, typeIR$_{arg}^*$).

m. Else if let CONTROL _ `< _ $^*$ `> `( parameterIR$_{param}^*$ `) of type controlObjectTypeIR be typeIR:

i. If let CONTROL _ `< _ $^*$ `> `( parameterIR$_{arg}^*$ `) of type controlObjectTypeIR be typeIR$_{arg}$':

A. For each parameterIR$_{param}$ in parameterIR$_{param}^*$:

- Let _ direction typeIR$_{param}$ _ _ be parameterIR$_{param}$.

Let direction* and typeIR$_{param}^*$ be the resulting lists.

B. For each parameterIR$_{arg}$ in parameterIR$_{arg}^*$:

- Let _ direction' typeIR$_{arg}$ _ _ be parameterIR$_{arg}$.

Let direction'* and typeIR$_{arg}^*$ be the resulting lists.

C. If the length of direction* is equal to the length of direction'*:

I. If direction' is equal to direction, for all direction in direction* and direction' in direction'*:

1. Return $gen_constraint(constraint, typeIR$_{param}^*$, typeIR$_{arg}^*$).

n. Else if let PACKAGE _ `< _ $^*$ `> `{ typeIR$_{param}^*$ `} of type packageObjectTypeIR be typeIR:

i. If let PACKAGE _ `< _ $^*$ `> `{ typeIR$_{arg}^*$ `} of type packageObjectTypeIR be typeIR$_{arg}$':

A. Return $gen_constraint(constraint, typeIR$_{param}^*$, typeIR$_{arg}^*$).

2. Then, try:

a. If let TYPEDEF _ typeIR$_{arg}$ of type typedefTypeIR be typeIR$_{arg}$':

i. If typeIR does not have type nameTypeIR:

A. If typeIR does not have type typedefTypeIR:

I. Return $gen_constraint_pair(constraint, typeIR, typeIR$_{arg}$).

3. Otherwise: return constraint.

$gen_constraint(constraint, $typeIR_{param}^{*}$, $typeIR_{arg}^{*}$)

1. Check that the length of $typeIR_{arg}^{*}$ is equal to the length of $typeIR_{param}^{*}$.
2. For each $typeIR_{arg}$ in $typeIR_{arg}^{*}$ and $typeIR_{param}$ in $typeIR_{param}^{*}$:

   - Let $constraint_{pair}$ be $gen_constraint_pair(constraint, $typeIR_{param}$, $typeIR_{arg}$).

   Let $constraint_{pair}^{*}$ be the resulting list.

3. Return $merge_constraints(constraint, $constraint_{pair}^{*}$).

19.5.2. Constraint merging

Constraints generated from different pairs are merged, requiring that a type variable constrained more than once be constrained consistently:

$merge_constraints(constraint$_{pre}$, constraint*)

1. If constraint* matches pattern []: return constraint$_{pre}$.
2. Else:
   a. Let constraint$_{post_h}$ :: constraint$_{post_t}^{*}$ be constraint*.
   b. Let constraint$_{pre_1}$ be $merge_constraint(constraint$_{pre}$, constraint$_{post_h}$).
   c. Let constraint$_{pre_2}$ be $merge_constraints(constraint$_{pre_1}$, constraint$_{post_t}^{*}$).
   d. Return constraint$_{pre_2}$.

$merge_constraint(constraint$_{pre}$, constraint$_{post}$)

1. Let `{ typeId$_{pre}^{*}$ `} be the domain of the map constraint$_{pre}$.
2. Let `{ typeId$_{post}^{*}$ `} be the domain of the map constraint$_{post}$.
3. Check that `{ typeId$_{pre}^{*}$ `} is equal to `{ typeId$_{post}^{*}$ `}.
4. Return $merge_constraint'(constraint$_{pre}$, constraint$_{post}$, typeId$_{pre}^{*}$, `{ · `}).

$merge_constraint'(constraint$_{pre}$, constraint$_{post}$, typeId*, constraint)

1. If typeId* matches pattern []: return constraint.
2. Else:
   a. Let typeId$_{h}$ :: typeId$_{t}^{*}$ be typeId*.
   b. Try:
      i. If UNKNOWN is equal to the value of typeId$_{h}$ in map constraint$_{pre}$:
         A. Try:
            I. If UNKNOWN is equal to the value of typeId$_{h}$ in map constraint$_{post}$:
               1. Let constraint$_{updated}$ be the map constraint with the value for typeId$_{h}$ is updated to UNKNOWN.
               2. Return $merge_constraint'(constraint$_{pre}$, constraint$_{post}$, typeId$_{t}^{*}$, constraint$_{updated}$).
         B. Then, try:

I. Let infer'? be the value of $typeId_h$ in map $constraint_{post}$.

II. If infer'? is defined:

1. Let infer be infer'?.

2. If let KNOWN AS $typeIR_{post}$ be infer:

1. Let $constraint_{updated}$ be the map constraint with the value for $typeId_h$ is updated to KNOWN AS $typeIR_{post}$.

2. Return $merge_constraint'($constraint_{pre}$, $constraint_{post}$, $typeId_t^*$, $constraint_{updated}$).

c. Then, try:

i. Let infer'? be the value of $typeId_h$ in map $constraint_{pre}$.

ii. Check that infer'? is defined.

iii. Let infer be infer'?.

iv. Check that infer matches pattern KNOWN AS %.

v. Let KNOWN AS $typeIR_{pre}$ be infer.

vi. Try:

A. If UNKNOWN is equal to the value of $typeId_h$ in map $constraint_{post}$:

I. Let $constraint_{updated}$ be the map constraint with the value for $typeId_h$ is updated to KNOWN AS $typeIR_{pre}$.

II. Return $merge_constraint'($constraint_{pre}$, $constraint_{post}$, $typeId_t^*$, $constraint_{updated}$).

vii. Then, try:

A. Let infer''? be the value of $typeId_h$ in map $constraint_{post}$.

B. Check that infer''? is defined.

C. Let infer'' be infer''?.

D. Check that infer'' matches pattern KNOWN AS %.

E. Let KNOWN AS $typeIR_{post}$ be infer''.

F. Let casts castResult be the result of checking if $typeIR_{pre}$ can be implicitly cast to $typeIR_{post}$.

G. If castResult matches pattern OK %:

I. Let $constraint_{updated}$ be the map constraint with the value for $typeId_h$ is updated to KNOWN AS $typeIR_{post}$.

II. Return $merge_constraint'($constraint_{pre}$, $constraint_{post}$, $typeId_t^*$, $constraint_{updated}$).

H. Else:

I. Let casts castResult' be the result of checking if $typeIR_{post}$ can be implicitly cast to $typeIR_{pre}$.

II. Check that castResult' matches pattern OK %.

III. Let $constraint_{updated}$ be the map constraint with the value for $typeId_h$ is updated to KNOWN AS $typeIR_{pre}$.

IV. Return $merge_constraint'($constraint_{pre}$, $constraint_{post}$, $typeId_t^*$, $constraint_{updated}$).

19.5.3. Constraint resolution

The merged constraint is resolved into an inference, checking that every strictly-inferred type variable is determined:

$resolve_constraint(strictness, set<pair<typeId, infer>>)

1. Try:

a. If set<pair<typeId, infer>> is equal to `{ · `}: return `{ · `}.

2. Then, try:
   a. Let `{ pair<typeId, infer>* `} be set<pair<typeId, infer>>.
   b. Check that pair<typeId, infer>* is a non-empty list.
   c. Let $typeId_h$ ':' $infer_h$ :: ($typeId_t$ ':' $infer_t$)* be pair<typeId, infer>*.
   d. If let KNOWN AS $typeIR_h$ be $infer_h$:
      i. Let `{ ( typeId ':' $typeIR_t$ )* `} be $resolve_constraint(strictness, `{ ( $typeId_t$ ':' $infer_t$ )* `}).
      ii. Check that the length of $typeId_t$* is equal to the length of typeId*.
      iii. Check that typeId is equal to $typeId_t$, for all $typeId_t$ in $typeId_t$* and typeId in typeId*.
      iv. Return `{ $typeId_h$ ':' $typeIR_h$ :: ( $typeId_t$ ':' $typeIR_t$ )* `}.
   e. Else:
      i. Check that the value of $typeId_h$ in map strictness is equal to LAX.
      ii. Let `{ ( typeId ':' $typeIR_t$ )* `} be $resolve_constraint(strictness, `{ ( $typeId_t$ ':' $infer_t$ )* `}).
      iii. Check that the length of $typeId_t$* is equal to the length of typeId*.
      iv. Check that typeId is equal to $typeId_t$, for all $typeId_t$ in $typeId_t$* and typeId in typeId*.
      v. Return `{ $typeId_h$ ':' BOOL :: ( $typeId_t$ ':' $typeIR_t$ )* `}.

Finally, any synthesized sequence type is resolved to a tuple type:

$resolve_inference(`{ ( typeId ':' typeIR )* `})

1. For each typeIR in typeIR*:
   - Let $typeIR_{resolved}$ be $resolve_inference'(typeIR).

   Let $typeIR_{resolved}$* be the resulting list.
2. Return `{ ( typeId ':' $typeIR_{resolved}$ )* `}.

$resolve_inference'(typeIR)

1. Let typeIR' be typeIR with typedefs unrolled.
2. If let sequenceTypeIR of type sequenceTypeIR be typeIR':
   a. If let SEQ `< $typeIR_{arg}$* `> be sequenceTypeIR:
      i. Let $typeIR_{resolved}$ be TUPLE `< $typeIR_{arg}$* `>.
      ii. Return $typeIR_{resolved}$.
3. Otherwise: return typeIR.

19.6. Call convention

Calls in P4 are executed using copy-in/copy-out semantics. For a high-level description of copy-in/copy-out semantics, see Section 6.5.

19.6.1. Type checking

Calling conventions are checked by the type system with:

Call_convention_ok:

- Checking calling convention of argumentListIR applied on parameterIR*, under context TC and action context actctxt at p:
- The result is casted argumentListIR.

Checking calling convention of argumentIR* applied on parameterIR*, under context TC and action context actctxt at p:

1. If parameterIR* matches pattern []:
   a. Check that argumentIR* is an empty list.
   b. The result is casted ·.
2. Else:
   a. Let $\text{parameterIR}_h :: \text{parameterIR}_t^*$ be parameterIR*.
   b. Check that argumentIR* is a non-empty list.
   c. Let $\text{argumentIR}_h :: \text{argumentIR}_t^*$ be argumentIR*.
   d. Let casted $\text{argumentIR}_{h_cast}$ be the result of checking calling convention of argumentIR_h applied to parameter type parameterIR_h, under context TC and action context actctxt at p.
   e. Let casted $\text{argumentIR}_{t_cast}^*$ be the result of checking calling convention of argumentIR_t^* applied on parameterIR_t^*, under context TC and action context actctxt at p.
   f. The result is casted $\text{argumentIR}_{h_cast} :: \text{argumentIR}_{t_cast}^*$.

Each argument-parameter pair is checked by:

Call_convention_arg_ok:

- Checking calling convention of argumentIR applied to parameter type parameterIR, under context TC and action context actctxt at p:
- The result is casted argumentIR.

Checking calling convention of argumentIR applied to parameter type parameterIR, under context TC and action context actctxt at p:

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:

a. Let casted typedExpressionIR$_{cast}$ be the result of checking calling convention of typedExpressionIR applied to parameterIR, under context TC and action context actctxt at p.

b. The result is casted typedExpressionIR$_{cast}$.

2. Else if let nameIR '=' typedExpressionIR be argumentIR:

a. Let casted typedExpressionIR$_{cast}$ be the result of checking calling convention of typedExpressionIR applied to parameterIR, under context TC and action context actctxt at p.

b. The result is casted nameIR '=' typedExpressionIR$_{cast}$.

3. Else if let nameIR '=' '_' be argumentIR:

a. Let direction be the direction of parameterIR.

b. Check that direction is OUT.

c. The result is casted nameIR '=' '_'.

4. Else:

a. Let direction be the direction of parameterIR.

b. Check that direction is OUT.

c. The result is casted '_'.

Expression arguments are checked, depending on the parameter's direction:

Call_convention_expr_ok:

- Checking calling convention of typedExpressionIR applied to parameterIR, under context TC and action context actctxt at p:
- The result is casted typedExpressionIR.

Call_convention_expr_ok dispatch:

1. Try:

a. If direction is IN: goto in

2. Then, try:

a. Goto out-inout

3. Then, try:

a. Check that direction is EMPTY.

b. Goto directionless

19.6.1.1. in

Checking calling convention of typedExpressionIR$_{arg}$ applied to annotationList IN typeIR$_{param}$ id parameterInitializerOptIR, under context TC and action context actctxt at p:

1. Let typedExpressionIR$^{?}$ be typedExpressionIR$_{arg}$ implicitly cast to typeIR$_{param}$.
2.

If $typedExpressionIR^?$ is defined:

a. Let $typedExpressionIR_{arg_cast}$ be $typedExpressionIR^?$.

b. The result is casted $typedExpressionIR_{arg_cast}$.

19.6.1.2. out and inout

`<span id="Call_convention_expr_ok-out-inout"></span>`

Checking calling convention of $typedExpressionIR_{arg}$ applied to annotationList direction $typeIR_{param}$ id parameterInitializerOptIR, under context TC and action context actctxt at p:

1. If direction is equal to OUT or direction is equal to INOUT:
    a. Let $typeIR_{arg}$ be the type of $typedExpressionIR_{arg}$.
    b. If $typeIR_{param}$ and $typeIR_{arg}$ are the same type:
        i. Let $typedLvalueIR^?$ be the l-value form of $typedExpressionIR_{arg}$.
        ii. If $typedLvalueIR^?$ is defined:
            A. Let $typedLvalueIR_{arg}$ be $typedLvalueIR^?$.
            B. Let $lvalue_{arg}$ be $lift_typedLvalueIR($typedLvalueIR_{arg}$).
            C. Let _ be the result of typing $lvalue_{arg}$, under context TC at p.
            D. The result is casted $typedExpressionIR_{arg}$.

19.6.1.3. Directionless

`<span id="Call_convention_expr_ok-directionless"></span>`

Checking calling convention of $typedExpressionIR_{arg}$ applied to annotationList EMPTY $typeIR_{param}$ id parameterInitializerOptIR, under context TC and action context actctxt at p:

1. If actctxt matches pattern ACTION:
    a. Let $typedExpressionIR^?$ be $typedExpressionIR_{arg}$ implicitly cast to $typeIR_{param}$.
    b. Check that $typedExpressionIR^?$ is defined.
    c. Let $typedExpressionIR_{arg_cast}$ be $typedExpressionIR^?$.
    d. The result is casted $typedExpressionIR_{arg_cast}$.
2. Else:
    a. Let $typedExpressionIR^?$ be $typedExpressionIR_{arg}$ implicitly cast to $typeIR_{param}$.
    b. Check that $typedExpressionIR^?$ is defined.
    c. Let $typedExpressionIR_{arg_cast}$ be $typedExpressionIR^?$.
    d. Check that the compile-time known-ness of $typedExpressionIR_{arg}$ does not match pattern DYN.
    e. The result is casted $typedExpressionIR_{arg_cast}$.

19.6.2. Runtime evaluation of copy-in

The copy-in phase of a call is evaluated by:

`<span id="relation_latex_Copy_in"></span>`

Copy_in:

- Copy-in from argumentListIR at p'' of EC' to parameterListIR in p' at EC, under state ARCH at shared p:
- The result is
  - state ARCH,
  - caller's context EC,
  - callee's context EC',
  - and copy-in result copyInResult.

The result of copy-in evaluation is:

```
copyInResult
    : continueResult<storageReference?*> | abortResult
    ;
```

Copy-in from argumentIR* at p_{callee} of EC_{callee_0} to parameterIR* in p_{caller} at EC, under state ARCH at shared p_{shared}:

1. If parameterIR* matches pattern []:
   a. Check that argumentIR* is an empty list.
   b. Let EC_{callee_1} be $copy_e($p_{shared}$, EC, EC_{callee_0}).
   c. The result is
      - state ARCH,
      - caller's context EC,
      - callee's context EC_{callee_1},
      - and copy-in result $_{CONT}\cdot$.
2. Else:
   a. Let $parameterIR_h$ :: $parameterIR_t^*$ be parameterIR*.
   b. Check that argumentIR* is a non-empty list.
   c. Let $argumentIR_h$ :: $argumentIR_t^*$ be argumentIR*.
   d. Let state $ARCH_1$, caller's context EC_{caller_1}, callee's context EC_{callee_1}, and copy-in result copyInArgumentResult be the result of copy-in from $argumentIR_h$ at p_{callee} of EC_{callee_0} to $parameterIR_h$ in p_{caller} at EC, under state ARCH.
   e. If let abortResult of type abortResult be copyInArgumentResult:
      i. The result is
         - state $ARCH_1$,
         - caller's context EC_{caller_1},
         - callee's context EC_{callee_1},
         - and copy-in result abortResult.
   f. Else:
      i. Let $_{CONT}$ $storageReference_h^?$ be copyInArgumentResult.

ii. Let state $ARCH_2$, caller's context EC_{caller_2}, callee's context EC_{callee_2}, and copy-in result copyInResult be the result of copy-in from $argumentIR_t^*$ at p_{callee} of EC_{callee_1} to $parameterIR_t^*$ in p_{caller} at EC_{caller_1}, under state $ARCH_1$ at shared p_{shared}.

iii. If let abortResult of type abortResult be copyInResult:

A. The result is

- state $ARCH_2$,
- caller's context EC_{caller_2},
- callee's context EC_{callee_2},
- and copy-in result abortResult.

iv. Else:

A. Let $_{CONT}\ storageReference_t^{?*}$ be copyInResult.

B. Let $storageReference^{?*}$ be $storageReference_h^{?} :: storageReference_t^{?*}$.

C. The result is

- state $ARCH_2$,
- caller's context EC_{caller_2},
- callee's context EC_{callee_2},
- and copy-in result $_{CONT}\ storageReference^{?*}$.

Default arguments are copied in by:

Copy_in_default:

- Copying defaults for parameterListIR, under context p and context EC:
- The result is EC.

Copying defaults for $parameterIR^*$, under context p_{callee} and context EC_{callee_0}:

1. For each parameterIR in $parameterIR^*$:

   - Let _ _ _ nameIR parameterInitializerOptIR be parameterIR.

   Let $nameIR^*$ and $parameterInitializerOptIR^*$ be the resulting lists.

2. Check that parameterInitializerOptIR is defined, for all parameterInitializerOptIR in $parameterInitializerOptIR^*$.
3. For each parameterInitializerOptIR in $parameterInitializerOptIR^*$:

   - Let parameterInitializerIR be parameterInitializerOptIR.

   Let $parameterInitializerIR^*$ be the resulting list.

4. Check that parameterInitializerIR has type constantInitializerIR, for all parameterInitializerIR in $parameterInitializerIR^*$.
5. For each parameterInitializerIR in $parameterInitializerIR^*$:

- Let '=' $_{VALUE}$ value be parameterInitializerIR.

Let value* be the resulting list.

6. Let EC_{callee_1} be $add_vars_e($p_{callee}$, EC_{callee_0}, ($_{BARE}$ nameIR)*, value*).
7. The result is EC_{callee_1}.

Otherwise, each argument-parameter pair is evaluated by:

Copy_in_arg:

- Copy-in from argumentIR at p' of EC' to parameterIR in p at EC, under state ARCH:
- The result is
  - state ARCH,
  - caller's context EC,
  - callee's context EC',
  - and copy-in result copyInArgumentResult.

Copy_in_arg dispatch:

1. Try:
   a. Goto directionless-in
2. Then, try:
   a. If direction matches pattern INOUT: goto inout
   b. Else if direction matches pattern OUT: goto out

Its result is:

```
copyInArgumentResult
    : continueResult<storageReference?> | abortResult
    ;
```

19.6.2.1. in and directionless

Copy-in from argumentIR at p_{callee} of EC_{callee_0} to annotationList direction typeIR nameIR parameterInitializerOptIR in p_{caller} at EC_{caller_0}, under state $ARCH_0$:

1. If direction is equal to $_{EMPTY}$ or direction is equal to IN:
   a. Let p_{caller} EC_{caller_0} $ARCH_0$ |- argumentIR : EC_{caller_1} $ARCH_1$ expressionResult.
   b. If let abortResult of type abortResult be expressionResult:
      i. The result is

- state $ARCH_1$,
- caller's context EC_{caller_1},
- callee's context EC_{callee_0},
- and copy-in result abortResult.

c. Else:

i. Let $_{CONT}$ value be expressionResult.

ii. Let EC_{callee_1} be EC_{callee_0} where $_{BARE}$ nameIR to value is added to the p_{callee} layer.

iii. The result is

- state $ARCH_1$,
- caller's context EC_{caller_1},
- callee's context EC_{callee_1},
- and copy-in result $_{CONT}$ ·.

19.6.2.2. inout

Copy-in from argumentIR at p_{callee} of EC_{callee_0} to annotationList INOUT typeIR nameIR parameterInitializerOptIR in p_{caller} at EC_{caller_0}, under state $ARCH_0$:

1. Let p_{caller} EC_{caller_0} $ARCH_0$ |- argumentIR : EC_{caller_1} $ARCH_1$ storageReferenceResult.
2. If let abortResult of type abortResult be storageReferenceResult:
   a. The result is
      - state $ARCH_1$,
      - caller's context EC_{caller_1},
      - callee's context EC_{callee_0},
      - and copy-in result abortResult.
3. Else:
   a. Let $_{CONT}$ storageReference$'^{?}$ be storageReferenceResult.
   b. Check that storageReference$'^{?}$ is defined.
   c. Let storageReference be storageReference$'^{?}$.
   d. Let value value be the result of reading from storageReference, under context EC_{caller_1} and state $ARCH_1$ at p_{caller}.
   e. Let EC_{callee_1} be EC_{callee_0} where $_{BARE}$ nameIR to value is added to the p_{callee} layer.
   f. The result is
      - state $ARCH_1$,
      - caller's context EC_{caller_1},
      - callee's context EC_{callee_1},
      - and copy-in result $_{CONT}$ storageReference.

19.6.2.3. out

Copy-in from argumentIR at p_{callee} of EC_{callee_0} to annotationList OUT typeIR nameIR parameterInitializerOptIR in p_{caller} at EC_{caller_0}, under state $ARCH_0$:

1. Let p_{caller} EC_{caller_0} $ARCH_0$ |- argumentIR : EC_{caller_1} $ARCH_1$ storageReferenceResult.
2. If let abortResult of type abortResult be storageReferenceResult:
   a. The result is
      - state $ARCH_1$,
      - caller's context EC_{caller_1},
      - callee's context EC_{callee_0},
      - and copy-in result abortResult.
3. Else:
   a. Let $_{CONT}$ storageReference$^?$ be storageReferenceResult.
   b. Let $typeIR_{subst}$ be typeIR substituted by bound type variables in EC_{callee_0} from the p_{callee} layer.
   c. Let $value_{init}$ be the default value for type $typeIR_{subst}$.
   d. Let EC_{callee_1} be EC_{callee_0} where $_{BARE}$ nameIR to $value_{init}$ is added to the p_{callee} layer.
   e. The result is
      - state $ARCH_1$,
      - caller's context EC_{caller_1},
      - callee's context EC_{callee_1},
      - and copy-in result $_{CONT}$ storageReference$^?$.

19.6.3. Compile-time evaluation of copy-in

The copy-in phase of a call is evaluated at compile time by:

Copy_in_inst:

- Copy-in from IC at p with argumentIR* to IC' at p' with id*, under store STO:
- The result is store STO and context IC.

Copy-in from IC_{caller} at p_{caller} with argumentIR* to IC at p_{callee} with id*, under store STO:

1. If argumentIR* matches pattern []:
   a. Check that id* is an empty list.
   b. The result is store STO and context IC.
2. Else:
   a. Let $argumentIR_h$ :: $argumentIR_t^*$ be argumentIR*.
   b. Check that id* is a non-empty list.
   c. Let id_h :: id_t^* be id*.

d. Let store STO_1 and context IC_{callee_1} be the result of copy-in from IC_{caller} at p_{caller} with $argumentIR_h$ to IC at p_{callee} with id_h, under store STO.

e. Let store STO_2 and context IC_{callee_2} be the result of copy-in from IC_{caller} at p_{caller} with $argumentIR_t^*$ to IC_{callee_1} at p_{callee} with id_t^*, under store STO_1.

f. The result is store STO_2 and context IC_{callee_2}.

Each argument-parameter pair is evaluated by:

Copy_in_arg_inst:

- Copy-in from IC at p with argumentIR to IC' at p' with id, under store STO:
- The result is store STO and context IC.

Copy-in from IC_{caller} at p_{caller} with argumentIR to IC_{callee_0} at p_{callee} with id, under store STO_0:

1. Let IC_{caller_inner} be IC_{caller} with id added to the path.
2. Let store STO_1 and value be the result of compile-time evaluation of argumentIR, under context IC_{caller_inner} and store STO_0 at p_{caller}.
3. Let IC_{callee_1} be IC_{callee_0} where id to value is added to the p_{callee} layer.
4. The result is store STO_1 and context IC_{callee_1}.

For default arguments, the argument is copied in by:

Copy_in_inst_default:

- Copying defaults $parameterIR^*$ from p at IC to IC' at p', under store STO:
- The result is store STO and context IC.

Copying defaults $parameterIR^*$ from p_{caller} at IC_{caller} to IC at p_{callee}, under store STO:

1. If $parameterIR^*$ matches pattern []:
   a. The result is store STO and context IC.
2. Else:
   a. Let $parameterIR_h :: parameterIR_t^*$ be $parameterIR^*$.
   b. Let store STO_1 and context IC_{callee_1} be the result of copying default $parameterIR_h$ from p_{caller} at IC_{caller} to IC at p_{callee}, under store STO.
   c.

Let store STO_2 and context IC_{callee_2} be the result of copying defaults $parameterIR_t^*$ from p_{caller} at IC_{caller} to IC_{callee_1} at p_{callee}, under store STO_1.

d. The result is store STO_2 and context IC_{callee_2}.

Copy_in_arg_inst_default:

- Copying default parameterIR from p at IC to IC' at p', under store STO:
- The result is store STO and context IC.

Copying default parameterIR from p_{caller} at IC_{caller} to IC_{callee_0} at p_{callee}, under store STO:

1. Let id and parameterInitializerOptIR be the name and the default initializer of parameterIR.
2. Check that parameterInitializerOptIR is defined.
3. Let parameterInitializerIR be parameterInitializerOptIR.
4. If let '=' typedExpressionIR of type initializerIR be parameterInitializerIR:
   a. Let IC_{caller_inner} be IC_{caller} with id added to the path.
   b. Let store STO_1 and value be the result of compile-time evaluation of typedExpressionIR, under context IC_{caller_inner} and store STO at p_{caller}.
   c. Let IC_{callee_1} be IC_{callee_0} where id to value is added to the p_{callee} layer.
   d. The result is store STO_1 and context IC_{callee_1}.
5. Else:
   a. Let '=' $_{VALUE}$ value be parameterInitializerIR.
   b. Let IC_{callee_1} be IC_{callee_0} where id to value is added to the p_{callee} layer.
   c. The result is store STO and context IC_{callee_1}.

Copy-out is unnecessary at compile time, because constructors do not have out or inout parameters.

19.6.4. Runtime evaluation of copy-out

The copy-out phase of a call is evaluated by:

Copy_out:

- Copy-out from parameterListIR at p' of EC to $storageReference^{?*}$ at p'' of EC', under state ARCH at shared p:
- The result is EC.

Copy-out from parameterListIR at p_{caller} of EC_{caller_0} to storageReference$^{?*}$ at p_{callee} of EC_{callee}, under state ARCH at shared p_{shared}:

1. Let EC_{caller_1} be $copy_e($$p_{shared}$, EC_{callee}, EC_{caller_0}).
2. Let caller's context EC_{caller_2} be the result of copy-in from storageReference$^{?*}$ at p_{callee} of EC_{callee} to parameterListIR in p_{caller} at EC_{caller_1}, under state ARCH.
3. The result is EC_{caller_2}.

It internally invokes:

Copy_out_inner:

- Copy-in from storageReference$^{?*}$ at p' of EC' to parameterListIR in p at EC, under state ARCH:
- The result is caller's context EC.

Copy-in from storageReference$^{?*}$ at p_{callee} of EC_{callee} to parameterIR* in p_{caller} at EC, under state ARCH:

1. If parameterIR* matches pattern []:
   a. Check that storageReference$^{?*}$ is an empty list.
   b. The result is caller's context EC.
2. Else:
   a. Let $parameterIR_h$ :: $parameterIR_t^{*}$ be parameterIR*.
   b. Let storageReference'$^{?*}$ be storageReference$^{?*}$.
   c. Check that storageReference'$^{?*}$ is a non-empty list.
   d. Let $storageReference_h^{?}$ :: $storageReference_t^{?*}$ be storageReference'$^{?*}$.
   e. Let context EC_{caller_1} be the result of copy-out from p_{callee} at EC of $parameterIR_h$ to p_{caller} at shared ARCH, under state p_{caller} at shared ARCH.
   f. Let caller's context EC_{caller_2} be the result of copy-in from $storageReference_t^{?*}$ at p_{callee} of EC_{callee} to $parameterIR_t^{*}$ in p_{caller} at EC_{caller_1}, under state ARCH.
   g. The result is caller's context EC_{caller_2}.

Where for each argument-parameter pair:

Copy_out_argument:

- Copy-out from p' at EC of parameterIR to p at shared ARCH, under state p at shared ARCH:
- The result is context EC.

Copy_out_argument dispatch:

1. Try:
    a. If storageReference'? is none: goto in-directionless
2. Then, try:
    a. Goto out-inout

19.6.4.1. in and directionless

Copy-out from p_{callee} at EC_{caller_0} of annotationList direction typeIR nameIR parameterInitializerOptIR to p_{caller} at shared ARCH, under state p_{caller} at shared ARCH:

1. If direction is equal to IN or direction is equal to $_{EMPTY}$:
    a. The result is context EC_{caller_0}.

19.6.4.2. out and inout

Copy-out from p_{callee} at EC_{caller_0} of annotationList direction typeIR nameIR parameterInitializerOptIR to p_{caller} at shared ARCH, under state p_{caller} at shared ARCH:

1. If storageReference'? matches pattern ():
    a. Check that direction is equal to OUT or direction is equal to INOUT.
    b. The result is context EC_{caller_0}.
2. Else:
    a. Let storageReference be storageReference'?.
    b. Check that direction is equal to OUT or direction is equal to INOUT.
    c. Let value be the value of $_{BARE}$ nameIR from the p_{callee} layer of EC_{callee}.
    d. Let context EC_{caller_1} be the result of writing value to storageReference, under context EC_{caller_0} and state ARCH at p_{caller}.
    e. The result is context EC_{caller_1}.

19.7. Constructor calls

Constructor calls are type checked as follows:

Inst_ok:

- Typing instantiation constructorTypeIR `< typeArgumentListIR `> `( argumentListIR `) with omitted type arguments typeId*, defaults id* and optionals id'* , under cursor p, context TC, and instctxt instantiation site context:

- The result is constructed type typeIR with inferred typeArgumentListIR and casted argumentListIR.

Typing instantiation constructorTypeIR `< typeArgumentIR* `> `( argumentIR* `) with omitted type arguments $typeId_{infer}$*, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p, context TC, and instctxt instantiation site context:

1. Let CONSTRUCTOR `( parameterIR* `) ':' $typeIR_{object}$ be constructorTypeIR.
2. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL $parameterIR_{optional}$* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
3. Let inference be $infer($typeId$_{infer}$*, parameterIR$_{aligned}$*, parameterIR$_{optional}$*, argumentIR*$)$.
4. For each typeArgumentIR in typeArgumentIR*:
   - Let $typeArgumentIR_{inferred}$ be typeArgumentIR substituted by inference.

   Let $typeArgumentIR_{inferred}$* be the resulting list.
5. For each $parameterIR_{aligned}$ in $parameterIR_{aligned}$*:
   - Let $parameterIR_{aligned_inferred}$ be $parameterIR_{aligned}$ substituted by inference.

   Let $parameterIR_{aligned_inferred}$* be the resulting list.
6. Let $typeIR_{object_inferred}$ be $typeIR_{object}$ substituted by inference.
7. Let $constructorTypeIR_{inferred}$ be CONSTRUCTOR `( $parameterIR_{aligned\_inferred}$* `) ':' $typeIR_{object_inferred}$.
8. Let bound be bound type variables from the p layer of TC.
9. If $constructorTypeIR_{inferred}$ is a well-formed constructor type, with bound type variables bound:
   a. Check that $instantiable(p, TC, instctxt, $typeIR$_{object_inferred})$.
   b. Let $typeIR_{object_unroll}$ be $typeIR_{object}$ with typedefs unrolled.
   c. If $typeIR_{object_unroll}$ has type packageObjectTypeIR:
      i. Check that p is GLOBAL.
      ii. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned_inferred}$*, under context TC and action context NOACTION at BLOCK.
      iii. The result is constructed type $typeIR_{object_inferred}$ with inferred $typeArgumentIR_{inferred}$* and casted $argumentIR_{cast}$*.
   d. Else:
      i. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned_inferred}$*, under context TC and action context NOACTION at p.
      ii. The result is constructed type $typeIR_{object_inferred}$ with inferred $typeArgumentIR_{inferred}$* and casted $argumentIR_{cast}$*.

At compile-time, constructor calls are evaluated as follows:

Constructor_call:

- Instantiating constructorDef `< typeArgumentListIR `> `( argumentListIR `) with defaults id* and optionals id'*,

under context IC and store STO at p:

- The result is store STO and instantiated object.

Constructor_call dispatch:

1. If let externObjectConstructorDef of type externObjectConstructorDef be constructorDef: goto externConstructorDef
2. Else if let parserObjectConstructorDef of type parserObjectConstructorDef be constructorDef: goto parserConstructorDef
3. Else if let controlObjectConstructorDef of type controlObjectConstructorDef be constructorDef: goto controlConstructorDef
4. Else:
   a. Let packageObjectConstructorDef be constructorDef.
   b. Goto packageConstructorDef

The contexts in which each type may be instantiated are collected in Appendix E.

19.7.1. Extern objects

Instantiating externObjectConstructorDef `< typeArgumentIR* `> `( argumentIR* `) with defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$, under context IC and store STO_0 at p:

1. Let EXTERN nameIR `< typeParameterIR* `> `( constructorParameterIR* `) `{ externMethodPrototypeIR* `} be externObjectConstructorDef.
2. Let p_{callee} be BLOCK.
3. Let IC_{callee_0} be an instantiation context same as IC up to the GLOBAL layer.
4. Check that the length of typeArgumentIR* is equal to the length of typeParameterIR*.
5. Let theta be `{ ( typeParameterIR ':' typeArgumentIR )* `}.
6. Let IC_{callee_1} be IC_{callee_0} with BLOCK.TYPE set to theta.
7. Let GIVEN $constructorParameterIR_{aligned}^{*}$ DEFAULT $constructorParameterIR_{default}^{*}$ OPTIONAL _* be aligning constructorParameterIR* with argumentIR* where defaults are $id_{default}^{*}$ and optionals are $id_{optional}^{*}$.
8. For each $constructorParameterIR_{aligned}$ in $constructorParameterIR_{aligned}^{*}$:

   - Let _ _ _ $id_{aligned}$ _ be $constructorParameterIR_{aligned}$.

   Let $id_{aligned}^{*}$ be the resulting list.

9. Let store STO_1 and context IC_{callee_2} be the result of copy-in from IC at p with argumentIR* to IC_{callee_1} at p_{callee} with $id_{aligned}^{*}$, under store STO_0.
10. Let store STO_2 and context IC_{callee_3} be the result of copying defaults $constructorParameterIR_{default}^{*}$ from p at IC to IC_{callee_2} at p_{callee}, under store STO_1.
11. Let context IC_{callee_4} be the result of compile-time evaluation of externMethodPrototypeIR*, under context IC_{callee_3}.
12. Let `{ ( callableId ':' callableDef )* `} be IC_{callee_4}.BLOCK.CALLABLE.
13. Check that callableDef has type externMethodDef, for all callableDef in callableDef*.
14. For each callableDef in callableDef*:

    - Let externMethodDef be callableDef.

Let externMethodDef* be the resulting list.

15. For each constructorParameterIR$_{default}$ in constructorParameterIR$_{default}$*:
    - Let _ _ _ id _ be constructorParameterIR$_{default}$.

    Let id* be the resulting list.
16. Check that the length of id$_{default}$* is equal to the length of id*.
17. Check that id is equal to id$_{default}$, for all id$_{default}$ in id$_{default}$* and id in id*.
18. Let id$_{cparam}$* be id$_{aligned}$* concatenated with id$_{default}$*.
19. For each id$_{cparam}$ in id$_{cparam}$*:
    - Let value? be the value of id$_{cparam}$ in map IC$_{callee_4}$.BLOCK.FRAME.

    Let value?* be the resulting list.
20. Check that value? is defined, for all value? in value?*.
21. For each value? in value?*:
    - Let value$_{cparam}$ be value?.

    Let value$_{cparam}$* be the resulting list.
22. Let objectState be initialization of nameIR `< typeArgumentIR* `>` `(id$_{cparam}$* '=' value$_{cparam}$* `) in the target.
23. Let theta$_{extern}$ be IC$_{callee_4}$.BLOCK.TYPE.
24. Let frame$_{extern}$ be IC$_{callee_4}$.BLOCK.FRAME.
25. Let externMethodDefEnv$_{extern}$ be `{ ( callableId ':' externMethodDef )* `}.
26. Let object$_{extern}$ be EXTERN nameIR `< theta$_{extern}$ `> `{ objectState frame$_{extern}$ externMethodDefEnv$_{extern}$ `}.
27. The result is store STO$_2$ and instantiated object$_{extern}$.

Constructing an extern object initializes its object state through the target-defined operation:

Initialization of nameIR `< typeIR* `>` `(id* '=' value* `) in the target

19.7.2. Parser objects

Instantiating parserObjectConstructorDef `< typeArgumentIR* `>` `(argumentIR* `) with defaults id$_{default}$* and optionals id$_{optional}$*, under context IC and store STO$_0$ at p:

1. Let PARSER `< typeParameterIR* `>` `(parameterIR* `) `(constructorParameterIR* `) `{ parserLocalDeclarationIR* parserStateIR* `} be parserObjectConstructorDef.
2. Let p$_{callee}$ be BLOCK.
3. Let IC$_{callee_0}$ be an instantiation context same as IC up to the GLOBAL layer.
4. Check that the length of typeArgumentIR* is equal to the length of typeParameterIR*.

5. Let $theta_{parser}$ be `{ ( typeParameterIR ':' typeArgumentIR )* `}.
6. Let IC_{callee_1} be IC_{callee_0} with BLOCK.TYPE set to $theta_{parser}$.
7. Let GIVEN $constructorParameterIR_{aligned}^{*}$ DEFAULT $constructorParameterIR_{default}^{*}$ OPTIONAL _* be aligning $constructorParameterIR^{*}$ with $argumentIR^{*}$ where defaults are $id_{default}^{*}$ and optionals are $id_{optional}^{*}$.
8. For each $constructorParameterIR_{aligned}$ in $constructorParameterIR_{aligned}^{*}$:

    - Let _ _ _ $id_{aligned}$ _ be $constructorParameterIR_{aligned}$.

    Let $id_{aligned}^{*}$ be the resulting list.
9. Let store STO_1 and context IC_{callee_2} be the result of copy-in from IC at p with $argumentIR^{*}$ to IC_{callee_1} at p_{callee} with $id_{aligned}^{*}$, under store STO_0.
10. Let store STO_2 and context IC_{callee_3} be the result of copying defaults $constructorParameterIR_{default}^{*}$ from p at IC to IC_{callee_2} at p_{callee}, under store STO_1.
11. Let context IC_{local}, store STO_3, and $parserLocalDeclarationIR_{inst}^{*}$ be the result of compile-time evaluation of $parserLocalDeclarationIR^{*}$, under context IC_{callee_3} and store STO_2.
12. Let context IC_{state} and store STO_4 be the result of compile-time evaluation of $parserStateIR^{*}$, under context IC_{local} and store STO_3.
13. Let $frame_{parser}$ be IC_{callee_3}.BLOCK.FRAME.
14. Let $stateEnv_{parser}$ be IC_{state}.BLOCK.STATE.
15. Let $object_{parser}$ be PARSER `< $theta_{parser}$ `> `( $parameterIR^{*}$ `) `{ $frame_{parser}$ $parserLocalDeclarationIR_{inst}^{*}$ $stateEnv_{parser}$ `}.
16. The result is store STO_4 and instantiated $object_{parser}$.

19.7.3. Control objects

Instantiating controlObjectConstructorDef `< $typeArgumentIR^{*}$ `> `( $argumentIR^{*}$ `) with defaults $id_{default}^{*}$ and optionals $id_{optional}^{*}$, under context IC and store STO_0 at p:

1. Let CONTROL `< $typeParameterIR^{*}$ `> `( $parameterIR^{*}$ `) `( $constructorParameterIR^{*}$ `) `{ $controlLocalDeclarationIR^{*}$ APPLY controlBodyIR `} be controlObjectConstructorDef.
2. Let p_{callee} be BLOCK.
3. Let IC_{callee_0} be an instantiation context same as IC up to the GLOBAL layer.
4. Check that the length of $typeArgumentIR^{*}$ is equal to the length of $typeParameterIR^{*}$.
5. Let $theta_{control}$ be `{ ( typeParameterIR ':' typeArgumentIR )* `}.
6. Let IC_{callee_1} be IC_{callee_0} with BLOCK.TYPE set to $theta_{control}$.
7. Let GIVEN $constructorParameterIR_{aligned}^{*}$ DEFAULT $constructorParameterIR_{default}^{*}$ OPTIONAL _* be aligning $constructorParameterIR^{*}$ with $argumentIR^{*}$ where defaults are $id_{default}^{*}$ and optionals are $id_{optional}^{*}$.
8. For each $constructorParameterIR_{aligned}$ in $constructorParameterIR_{aligned}^{*}$:

    - Let _ _ _ $id_{aligned}$ _ be $constructorParameterIR_{aligned}$.

    Let $id_{aligned}^{*}$ be the resulting list.
9. Let store STO_1 and context IC_{callee_2} be the result of copy-in from IC at p with $argumentIR^{*}$ to IC_{callee_1} at p_{callee} with $id_{aligned}^{*}$, under store STO_0.
10. Let store STO_2 and context IC_{callee_3} be the result of copying defaults $constructorParameterIR_{default}^{*}$ from p at IC to IC_{callee_2} at p_{callee}, under store STO_1.

11. Let context IC_{local_0}, store STO_3, and $controlLocalDeclarationIR_{inst}^*$ be the result of compile-time evaluation of $controlLocalDeclarationIR^*$, under context $IC_{callee_3}$and store STO_2.
12. Let context IC_{local_1}, store STO_4, and $controlBodyIR_{inst}$ be the result of compile-time evaluation of controlBodyIR, under context IC_{local_0}, and store STO_3.
13. Let $frame_{control}$ be IC_{callee_3}.BLOCK.FRAME.
14. Let `{ ( callableId ':' callableDef )* `} be IC_{local_1}.BLOCK.CALLABLE.
15. Check that callableDef has type actionDef, for all callableDef in $callableDef^*$.
16. For each callableDef in $callableDef^*$:

    - Let actionDef be callableDef.

    Let $actionDef^*$ be the resulting list.
17. Let $object_{control}$ be CONTROL `< $theta_{control}$ `> `( $parameterIR^*$ `) `{ $frame_{control}$ $controlLocalDeclarationIR_{inst}^*$ `{ (callableId ':' actionDef)* `} $controlBodyIR_{inst}$ `}.
18. The result is store STO_4 and instantiated $object_{control}$.

19.7.4. Package objects

Instantiating packageObjectConstructorDef `< $typeArgumentIR^*$ `> `( $argumentIR^*$ `) with defaults $id_{default}^*$ and optionals $id_{optional}^*$, under context IC and store STO_0 at p:

1. Let PACKAGE `< $typeParameterIR^*$ `> `( $constructorParameterIR^*$ `) be packageObjectConstructorDef.
2. Let p_{callee} be BLOCK.
3. Let IC_{callee_0} be an instantiation context same as IC up to the GLOBAL layer.
4. Check that the length of $typeArgumentIR^*$ is equal to the length of $typeParameterIR^*$.
5. Let IC_{callee_1} be IC_{callee_0} with BLOCK.TYPE set to `{ ( typeParameterIR ':' typeArgumentIR )* `}.
6. Let GIVEN $constructorParameterIR_{aligned}^*$ DEFAULT $constructorParameterIR_{default}^*$ OPTIONAL _* be aligning $constructorParameterIR^*$ with $argumentIR^*$ where defaults are $id_{default}^*$ and optionals are $id_{optional}^*$.
7. For each $constructorParameterIR_{aligned}$ in $constructorParameterIR_{aligned}^*$:

    - Let _ _ _ $id_{aligned}$ _ be $constructorParameterIR_{aligned}$.

    Let $id_{aligned}^*$ be the resulting list.
8. Let store STO_1 and context IC_{callee_2} be the result of copy-in from IC at p with $argumentIR^*$ to IC_{callee_1} at p_{callee} with $id_{aligned}^*$, under store STO_0.
9. Let store STO_2 and context IC_{callee_3} be the result of copying defaults $constructorParameterIR_{default}^*$ from p at IC to IC_{callee_2} at p_{callee}, under store STO_1.
10. Let $theta_{package}$ be IC_{callee_3}.BLOCK.TYPE.
11. Let $frame_{package}$ be IC_{callee_3}.BLOCK.FRAME.
12. Let $object_{package}$ be PACKAGE `< $theta_{package}$ `> `{ $frame_{package}$ `}.
13. The result is store STO_2 and instantiated $object_{package}$.

19.8. Callable calls

Callable calls are type checked as follows:

Call_ok:

- Typing invocation callableTypeIR `< typeArgumentListIR `> `( argumentListIR `) with omitted type arguments typeId*, defaults id* and optionals id'*, under cursor p and context TC:
- The result is return type typeIR with inferred typeArgumentListIR and casted argumentListIR.

Call_ok dispatch:

1. If let actionTypeIR of type actionTypeIR be callableTypeIR:
    a. Check that typeArgumentIR* is an empty list.
    b. Check that typeId* is an empty list.
    c. Goto actionTypeIR
2. Else if let definedFunctionTypeIR of type definedFunctionTypeIR be callableTypeIR: goto definedFunctionTypeIR
3. Else if let externFunctionTypeIR of type externFunctionTypeIR be callableTypeIR: goto externFunctionTypeIR
4. Else if let builtinMethodTypeIR of type builtinMethodTypeIR be callableTypeIR:
    a. Check that typeId* is an empty list.
    b. Goto builtinMethodTypeIR
5. Else if let externMethodTypeIR of type externMethodTypeIR be callableTypeIR: goto externMethodTypeIR
6. Else if let parserApplyMethodTypeIR of type parserApplyMethodTypeIR be callableTypeIR:
    a. Check that typeArgumentIR* is an empty list.
    b. Check that typeId* is an empty list.
    c. Goto parserApplyMethodTypeIR
7. Else if let controlApplyMethodTypeIR of type controlApplyMethodTypeIR be callableTypeIR:
    a. Check that typeArgumentIR* is an empty list.
    b. Check that typeId* is an empty list.
    c. Goto controlApplyMethodTypeIR
8. Else:
    a. Let tableApplyMethodTypeIR be callableTypeIR.
    b. Check that typeArgumentIR* is an empty list.
    c. Check that typeId* is an empty list.
    d. Check that argumentIR* is an empty list.
    e. Check that $id_{default}$* is an empty list.
    f. Check that $id_{optional}$* is an empty list.
    g. Goto tableApplyMethodTypeIR

At runtime, callable callees are evaluated as follows:

Call_eval:

- Calling callee `< typeArgumentListIR `> `( argumentListIR `), under context EC and state ARCH at p:
- The result is context EC, state ARCH, and call result callResult.

Call_eval dispatch:

1. Try:
    a. Let typeArgumentIR* be typeArgumentListIR.
    b. Try:
        i. If let actionCallee of type actionCallee be callee:
            A. If typeArgumentIR* is an empty list: goto actionCallee
        ii. Else if let definedFunctionCallee of type definedFunctionCallee be callee: goto definedFunctionCallee
    c. Then, try:
        i. Let argumentIR* be argumentListIR.
        ii. If let externFunctionCallee of type externFunctionCallee be callee: goto externFunctionCallee
    d. Then, try:
        i. If let builtinPushFrontMethodCallee of type builtinPushFrontMethodCallee be callee:
            A. If typeArgumentIR* is an empty list:
                I. If let [+argumentIR]+ be argumentListIR: goto builtinPushFrontMethodCallee
        ii. Else if let builtinPopFrontMethodCallee of type builtinPopFrontMethodCallee be callee:
            A. If typeArgumentIR* is an empty list:
                I. If let [+argumentIR]+ be argumentListIR: goto builtinPopFrontMethodCallee
    e. Then, try:
        i. Let argumentIR* be argumentListIR.
        ii. If let builtinIsValidMethodCallee of type builtinIsValidMethodCallee be callee:
            A. If typeArgumentIR* is an empty list:
                I. If argumentIR* is an empty list:
                    1. Try:
                        1. Goto builtinIsValidMethodCallee-header
                    2. Then, try:
                        1. Goto builtinIsValidMethodCallee-header-union
        iii. Else if let builtinSetValidMethodCallee of type builtinSetValidMethodCallee be callee:
            A. If typeArgumentIR* is an empty list:
                I. If argumentIR* is an empty list: goto builtinSetValidMethodCallee
        iv. Else if let builtinSetInvalidMethodCallee of type builtinSetInvalidMethodCallee be callee:
            A. If typeArgumentIR* is an empty list:
                I. If argumentIR* is an empty list: goto builtinSetInvalidMethodCallee
        v. Else if let builtinSizeMethodCallee of type builtinSizeMethodCallee be callee:
            A. If typeArgumentIR* is an empty list:
                I. If argumentIR* is an empty list: goto builtinSizeMethodCallee
        vi. Else if let externMethodCallee of type externMethodCallee be callee: goto externMethodCallee

2. Then, try:
    a. If let parserApplyMethodCallee of type parserApplyMethodCallee be callee: goto parserApplyMethodCallee
    b. Else if let controlApplyMethodCallee of type controlApplyMethodCallee be callee: goto controlApplyMethodCallee
    c. Else if let tableApplyMethodCallee of type tableApplyMethodCallee be callee: goto tableApplyMethodCallee

The following subsections describe the type checking and evaluation of callable calls for different types of callees.

The contexts from which each kind of callee may be called are collected in Appendix E.

19.8.1. Actions

19.8.1.1. Type checking

A call to an action is type checked as follows:

Typing invocation actionTypeIR `< · `> `( argumentIR* `) with omitted type arguments ·, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p and context TC:

1. Let _ ACTION _ `( parameterIR* `) be actionTypeIR.
2. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
3. Check that $callable_action(p, TC).
4. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned}$*, under context TC and action context ACTION at p.
5. The result is return type VOID with inferred · and casted $argumentIR_{cast}$*.

19.8.1.2. Runtime evaluation

An action call is evaluated as follows:

Calling actionCallee `< · `> `( argumentListIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let ACTION p_{shared} '.' nameIR `( parameterIR* '#' $id_{default}$* `) blockStatementIR be actionCallee.
2. Let EC_{callee_0} be copy of EC_0 from the p_{shared} layer.
3. Let GIVEN $parameterIR_{aligned}$* DEFAULT $parameterIR_{default}$* OPTIONAL _* be aligning parameterIR* with argumentListIR where defaults are $id_{default}$* and optionals are ·.
4. Let state $ARCH_1$, caller's context EC_1, callee's context EC_{callee_1}, and copy-in result copyInResult be the result of copy-in from argumentListIR at LOCAL of EC_{callee_0} to $parameterIR_{aligned}$* in p at EC_0, under state $ARCH_0$ at shared p_{shared}.
5. If let exitResult of type exitResult be copyInResult:
    a. The result is context EC_1, state $ARCH_1$, and call result EXIT.
6. Else if let $_{CONT}$ storageReference?* of type continueResult<storageReference?*> be copyInResult:
    a. Let EC_{callee_2} be the result of copying defaults for $parameterIR_{default}$*, under context LOCAL and context EC_{callee_1}.

b. Let context EC_{callee_3}, state $ARCH_2$ and result actionResult be the result of evaluating action body blockStatementIR, under context EC_{callee_2} and state $ARCH_1$.

c. Try:

i. If let exitResult of type exitResult be actionResult:

A. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at LOCAL of EC_{callee_3}, under state $ARCH_2$ at shared p_{shared}.

B. The result is context EC_2, state $ARCH_2$, and call result EXIT.

d. Then, try:

i. If actionResult is equal to RETURN ·:

A. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at LOCAL of EC_{callee_3}, under state $ARCH_2$ at shared p_{shared}.

B. The result is context EC_2, state $ARCH_2$, and call result RETURN ·.

An action body is evaluated as follows:

ActionBody_eval:

- Evaluating action body blockStatementIR, under context EC and state ARCH:
- The result is context EC, state ARCH and result actionResult.

The result of evaluating an action body is either an exit or a return.

```
actionResult
    : exitResult | returnResult
    ;
```

Evaluating action body blockStatementIR, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of blockStatementIR, under context EC_0, and state $ARCH_0$.
2. Try:
   a. If let continueEmptyResult of type continueEmptyResult be statementResult:
      i. The result is context EC_1, state $ARCH_1$ and result RETURN ·.
   b. Else if let exitResult of type exitResult be statementResult:
      i. The result is context EC_1, state $ARCH_1$ and result EXIT.
3. Then, try:
   a. Check that statementResult is equal to RETURN ·.
   b. The result is context EC_1, state $ARCH_1$ and result RETURN ·.

19.8.2. Functions

19.8.2.1. Type checking

A call to a function is type checked as follows:

Typing invocation definedFunctionTypeIR `< typeArgumentIR* `> `( argumentIR* `) with omitted type arguments typeId*, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p and context TC:

1. Let FUNCTION nameIR `( parameterIR* `) ':' $typeIR_{ret}$ be definedFunctionTypeIR.
2. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL $parameterIR_{optional}$* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
3. Let inference be $infer(typeId*, $parameterIR_{aligned}$*, $parameterIR_{optional}$*, argumentIR*).
4. For each typeArgumentIR in typeArgumentIR*:
   - Let $typeArgumentIR_{inferred}$ be typeArgumentIR substituted by inference.

   Let $typeArgumentIR_{inferred}$* be the resulting list.
5. For each $parameterIR_{aligned}$ in $parameterIR_{aligned}$*:
   - Let $parameterIR_{aligned_inferred}$ be $parameterIR_{aligned}$ substituted by inference.

   Let $parameterIR_{aligned_inferred}$* be the resulting list.
6. Let $typeIR_{ret_inferred}$ be $typeIR_{ret}$ substituted by inference.
7. Let $functionTypeIR_{inferred}$ be FUNCTION nameIR `( $parameterIR_{aligned\_inferred}$* `) ':' $typeIR_{ret_inferred}$.
8. Let B be bound type variables from the p layer of TC.
9. If $functionTypeIR_{inferred}$ is a well-formed callable type, with bound type variables B:
   a. Check that $callable_definedFunction(p, TC).
   b. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned_inferred}$*, under context TC and action context NOACTION at p.
   c. The result is return type $typeIR_{ret_inferred}$ with inferred $typeArgumentIR_{inferred}$* and casted $argumentIR_{cast}$*.

19.8.2.2. Runtime evaluation

A function call is evaluated as follows:

Calling definedFunctionCallee `< typeArgumentIR* `> `( argumentListIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let FUNCTION nameIR `< typeParameterIR* `> `( parameterIR* '#' $id_{default}$* `) blockStatementIR be definedFunctionCallee.
2. Let EC_{callee_0} be copy of EC_0 from the GLOBAL layer.
3. If the length of typeArgumentIR* is equal to the length of typeParameterIR*:
   a. Let EC_{callee_1} be EC_{callee_0} with LOCAL.TYPE set to `{ ( typeParameterIR ':' typeArgumentIR )* `}.
   b. Let GIVEN $parameterIR_{aligned}$* DEFAULT $parameterIR_{default}$* OPTIONAL _* be aligning parameterIR* with argumentListIR where defaults are $id_{default}$* and optionals are ·.

c. Let p_{shared} be GLOBAL.

d. Let state $ARCH_1$, caller's context EC_1, callee's context EC_{callee_2}, and copy-in result copyInResult be the result of copy-in from argumentListIR at LOCAL of EC_{callee_1} to $parameterIR_{aligned}^{*}$ in p at EC_0, under state $ARCH_0$ at shared p_{shared}.

e. If let abortResult of type abortResult be copyInResult:

i. The result is context EC_1, state $ARCH_1$, and call result abortResult.

f. Else:

i. Let $_{CONT}$ $storageReference^{?*}$ be copyInResult.

ii. Let EC_{callee_3} be the result of copying defaults for $parameterIR_{default}^{*}$, under context LOCAL and context EC_{callee_2}.

iii. Let context EC_{callee_4}, state $ARCH_2$ and result definedFunctionResult be the result of evaluating function body blockStatementIR, under context EC_{callee_3} and state $ARCH_1$.

iv. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at LOCAL of EC_{callee_4}, under state $ARCH_2$ at shared p_{shared}.

v. The result is context EC_2, state $ARCH_2$, and call result definedFunctionResult.

A function body is evaluated as follows:

DefinedFunctionBody_eval:

- Evaluating function body blockStatementIR, under context EC and state ARCH:
- The result is context EC, state ARCH and result definedFunctionResult.

The result of evaluating an function body is either an exit or a return.

```
definedFunctionResult
    : exitResult | returnResult
    ;
```

Evaluating function body blockStatementIR, under context EC_0 and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of blockStatementIR, under context EC_0, and state $ARCH_0$.
2. If let continueEmptyResult of type continueEmptyResult be statementResult:
   a. The result is context EC_1, state $ARCH_1$ and result RETURN ·.
3. Else if let exitResult of type exitResult be statementResult:
   a. The result is context EC_1, state $ARCH_1$ and result EXIT.
4. Else if let returnResult of type returnResult be statementResult:
   a. The result is context EC_1, state $ARCH_1$ and result returnResult.

19.8.3. Extern functions

19.8.3.1. Type checking

A call to an extern function is type checked as follows:

Typing invocation externFunctionTypeIR `< typeArgumentIR* `> `( argumentIR* `) with omitted type arguments typeId*, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p and context TC:

1. Let EXTERN_FUNCTION nameIR `( parameterIR* `) ':' $typeIR_{ret}$ be externFunctionTypeIR.
2. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL $parameterIR_{optional}$* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
3. Let inference be $infer(typeId*, $parameterIR_{aligned}$*, $parameterIR_{optional}$*, argumentIR*).
4. For each typeArgumentIR in typeArgumentIR*:
   - Let $typeArgumentIR_{inferred}$ be typeArgumentIR substituted by inference.

   Let $typeArgumentIR_{inferred}$* be the resulting list.
5. For each $parameterIR_{aligned}$ in $parameterIR_{aligned}$*:
   - Let $parameterIR_{aligned_inferred}$ be $parameterIR_{aligned}$ substituted by inference.

   Let $parameterIR_{aligned_inferred}$* be the resulting list.
6. Let $typeIR_{ret_inferred}$ be $typeIR_{ret}$ substituted by inference.
7. Let $externFunctionTypeIR_{inferred}$ be EXTERN_FUNCTION nameIR `( $parameterIR_{aligned\_inferred}$* `) ':' $typeIR_{ret_inferred}$.
8. Let B be bound type variables from the p layer of TC.
9. If $externFunctionTypeIR_{inferred}$ is a well-formed callable type, with bound type variables B:
   a. Check that $callable_externFunction(p, TC, nameIR).
   b. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned_inferred}$*, under context TC and action context NOACTION at p.
   c. The result is return type $typeIR_{ret_inferred}$ with inferred $typeArgumentIR_{inferred}$* and casted $argumentIR_{cast}$*.

19.8.3.2. Runtime evaluation

An extern function call is evaluated as follows:

Calling externFunctionCallee `< typeArgumentIR* `> `( argumentIR* `), under context EC_0 and state $ARCH_0$ at p:

1. Let EXTERN_FUNCTION nameIR `< typeParameterIR* `> `( parameterIR* '#' $id_{default}$* '#' $id_{optional}$* `) be externFunctionCallee.
2. Let EC_{callee_0} be copy of EC_0 from the GLOBAL layer.
3. If the length of typeArgumentIR* is equal to the length of typeParameterIR*:
   a. Let EC_{callee_1} be EC_{callee_0} with LOCAL.TYPE set to `{ ( typeParameterIR ':' typeArgumentIR )* `}.
   b. Let GIVEN $parameterIR_{aligned}$* DEFAULT $parameterIR_{default}$* OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.

c. Let p_{shared} be GLOBAL.

d. Let state $ARCH_1$, caller's context EC_1, callee's context EC_{callee_2}, and copy-in result copyInResult be the result of copy-in from argumentIR* at LOCAL of EC_{callee_1} to $parameterIR_{aligned}$* in p at EC_0, under state $ARCH_0$ at shared p_{shared}.

e. If let abortResult of type abortResult be copyInResult:

i. The result is context EC_1, state $ARCH_1$, and call result abortResult.

f. Else:

i. Let $_{CONT}$ storageReference?* be copyInResult.

ii. Let EC_{callee_3} be the result of copying defaults for $parameterIR_{default}$*, under context LOCAL and context EC_{callee_2}.

iii. For each parameterIR in parameterIR*:

- Let _ _ _ $nameIR_{param}$ _ be parameterIR.

Let $nameIR_{param}$* be the resulting list.

iv. Let context EC_{callee_4}, state $ARCH_2$, and call result callResult be the result of calling an extern function nameIR `( $nameIR_{param}$* `), under context EC_{callee_3}, and state $ARCH_1$.

v. If callResult has type rejectTransitionResult:

A. Let rejectTransitionResult be callResult.

B. The result is context EC_1, state $ARCH_2$, and call result rejectTransitionResult.

vi. Else:

A. Let EC_2 be the result of copy-out from $parameterIR_{aligned}$* at p of EC_1 to storageReference?* at LOCAL of EC_{callee_4}, under state $ARCH_2$ at shared p_{shared}.

B. The result is context EC_2, state $ARCH_2$, and call result callResult.

19.8.4. Built-in methods

19.8.4.1. Type checking

A call to a built-in method is type checked as follows:

Typing invocation builtinMethodTypeIR `< typeArgumentIR* `> `( argumentIR* `) with omitted type arguments ·, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p and context TC:

1. Let (parameterIR*, $typeIR_{ret}$) be $signature_of_builtinMethodTypeIR(builtinMethodTypeIR).
2. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
3. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned}$*, under context TC and action context NOACTION at p.
4. The result is return type $typeIR_{ret}$ with inferred typeArgumentIR* and casted $argumentIR_{cast}$*.

19.8.4.2. Runtime evaluation

A built-in method call is evaluated as follows:

19.8.4.2.1. minSizeInBits, maxSizeInBits, minSizeInBytes, and maxSizeInBytes

Calling builtinSizeMethodCallee `< · >` `( · )`, under context EC_0 and state $ARCH_0$ at p:

1. Let BUILTIN_SIZE_OF nameIR typeIR$_{base}$ be builtinSizeMethodCallee.
2. Let value'' be $sizeof(typeIR$_{base}$, nameIR).
3. Let returnResult be RETURN value''.
4. The result is context EC_0, state $ARCH_0$, and call result returnResult.

19.8.4.2.2. push_front and pop_front on header stacks

Calling builtinPushFrontMethodCallee `< · >` `( argumentIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let BUILTIN_PUSH_FRONT storageReference be builtinPushFrontMethodCallee.
2. Let value value be the result of reading from storageReference, under context EC_0 and state $ARCH_0$ at p.
3. If let headerStackValue of type headerStackValue be value:
   a. Let HEADER_STACK `[ value$_{element}^{*}$ `(n_{idx} `) `] be headerStackValue.
   b. Try:
      i. Let p EC_0 $ARCH_0$ |- argumentIR : EC_1 $ARCH_1$ expressionResult.
      ii. If let abortResult of type abortResult be expressionResult:
         A. The result is context EC_1, state $ARCH_1$, and call result abortResult.
   c. Then, try:
      i. Let n_{size} be the length of value$_{element}^{*}$.
      ii. Let p EC_0 $ARCH_0$ |- argumentIR : EC_1 $ARCH_1$ expressionResult.
      iii. If let $_{CONT}$ value' of type continueResult<value> be expressionResult:
         A. If let integerValue$_{count}$ of type integerValue be value':
            I. Let nat$^{?}$ be the natural number representation of integerValue$_{count}$.
            II. If nat$^{?}$ is defined:
               1. Let n_{count} be nat$^{?}$.
               2. Try:
                  1. If n_{count} is less than n_{size}:
                     1. Let value$_{front}^{*}$ be value$_{element}^{*}$[0 : n_{count}].
                     2. For each value$_{front}$ in value$_{front}^{*}$:

                        - Let value$_{front_invalidated}$ be $invalidate_value(value$_{front}$).

                        Let value$_{front_invalidated}^{*}$ be the resulting list.

                  3. Let i be max`( [ 0, $n_{size}$ - $n_{count}$ ] `).
                  4. If let n_{back} of type nat be i:
                     1. Let value$_{back}^{*}$ be value$_{element}^{*}$[0 : n_{back}].
                     2. Let value$_{element_updated}^{*}$ be value$_{front_invalidated}^{*}$ concatenated with value$_{back}^{*}$.

3. Let $n_{idx_updated}$ be min`( [ $n_{size}$, $n_{idx}$ + $n_{count}$ ] `).
4. Let $headerStackValue_{updated}$ be HEADER_STACK `[ $value_{element\_updated}$* `($n_{idx_updated}$ `) `].
5. Let context EC_2 be the result of writing $headerStackValue_{updated}$ to storageReference, under context EC_1 and state $ARCH_1$ at p.
6. The result is context EC_2, state $ARCH_1$, and call result RETURN ·.

3. Then, try:
    1. If n_{count} is greater than or equal to n_{size}:
        1. For each $value_{element}$ in $value_{element}$*:
            - Let $value_{element_updated}$ be $invalidate_value($value_{element}$).

            Let $value_{element_updated}$* be the resulting list.
        2. Let $n_{idx_updated}$ be min`( [ $n_{size}$, $n_{idx}$ + $n_{count}$ ] `).
        3. Let $headerStackValue_{updated}$ be HEADER_STACK `[ $value_{element\_updated}$* `($n_{idx_updated}$ `) `].
        4. Let context EC_2 be the result of writing $headerStackValue_{updated}$ to storageReference, under context EC_1 and state $ARCH_1$ at p.
        5. The result is context EC_2, state $ARCH_1$, and call result RETURN ·.

Calling builtinPopFrontMethodCallee `< · `> `( argumentIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let BUILTIN_POP_FRONT storageReference be builtinPopFrontMethodCallee.
2. Let value value be the result of reading from storageReference, under context EC_0 and state $ARCH_0$ at p.
3. If let headerStackValue of type headerStackValue be value:
    a. Let HEADER_STACK `[ $value_{element}$* `(n_{idx} `) `] be headerStackValue.
    b. Try:
        i. Let p EC_0 $ARCH_0$ |- argumentIR : EC_1 $ARCH_1$ expressionResult.
        ii. If let abortResult of type abortResult be expressionResult:
            A. The result is context EC_1, state $ARCH_1$, and call result abortResult.
    c. Then, try:
        i. Let n_{size} be the length of $value_{element}$*.
        ii. Let p EC_0 $ARCH_0$ |- argumentIR : EC_1 $ARCH_1$ expressionResult.
        iii. If let $_{CONT}$ value' of type continueResult<value> be expressionResult:
            A. If let $integerValue_{count}$ of type integerValue be value':
                I. Let $nat^?$ be the natural number representation of $integerValue_{count}$.
                II. If $nat^?$ is defined:
                    1. Let n_{count} be $nat^?$.
                    2. Try:
                        1. If n_{count} is less than n_{size}:
                            1. Let i be max`( [ 0, $n_{size}$ - $n_{count}$ ] `).
                            2. If let n_{front} of type nat be i:
                                1. Let $value_{front}$* be $value_{element}$*[n_{count} : n_{front}].

2. Let $value_{back}^{*}$ be $value_{element}^{*}[0 : n_{count}]$.
3. For each $value_{back}$ in $value_{back}^{*}$:

   - Let $value_{back_invalidated}$ be $invalidate_value($value_{back}$).

   Let $value_{back_invalidated}^{*}$ be the resulting list.

4. Let $value_{element_updated}^{*}$ be $value_{front}^{*}$ concatenated with $value_{back_invalidated}^{*}$.
5. Let $headerStackValue_{updated}$ be HEADER_STACK `[ $value_{element\_updated}^{*}$ `(n_{front} `) `].
6. Let context EC_2 be the result of writing $headerStackValue_{updated}$ to storageReference, under context EC_1 and state $ARCH_1$ at p.
7. The result is context EC_2, state $ARCH_1$, and call result RETURN ·.

3. Then, try:
   1. If n_{count} is greater than or equal to n_{size}:
      1. For each $value_{element}$ in $value_{element}^{*}$:

         - Let $value_{element_updated}$ be $invalidate_value($value_{element}$).

         Let $value_{element_updated}^{*}$ be the resulting list.

      2. Let i be max`( [ 0, $n_{size}$ - $n_{count}$ ] `).
      3. If let $n_{idx_updated}$ of type nat be i:
         1. Let $headerStackValue_{updated}$ be HEADER_STACK `[ $value_{element\_updated}^{*}$ `($n_{idx_updated}$ `) `].
         2. Let context EC_2 be the result of writing $headerStackValue_{updated}$ to storageReference, under context EC_1 and state $ARCH_1$ at p.
         3. The result is context EC_2, state $ARCH_1$, and call result RETURN ·.

19.8.4.2.3. isValid, setValid, and setInvalid on headers

Calling builtinIsValidMethodCallee `< · `> `( · `), under context EC_0 and state $ARCH_0$ at p:

1. Let BUILTIN_IS_VALID storageReference be builtinIsValidMethodCallee.
2. Let value value be the result of reading from storageReference, under context EC_0 and state $ARCH_0$ at p.
3. If let headerValue of type headerValue be value:
   a. Let HEADER _ `{ $b_{valid}$ ';' _$^{*}$ `} be headerValue.
   b. Let returnResult be RETURN $_{B}$ b_{valid}.
   c. The result is context EC_0, state $ARCH_0$, and call result returnResult.

Calling builtinSetValidMethodCallee `< · `> `( · `), under context EC_0 and state $ARCH_0$ at p:

1. Let BUILTIN_SET_VALID storageReference be builtinSetValidMethodCallee.
2. Let value value' be the result of reading from storageReference, under context EC_0 and state $ARCH_0$ at p.
3. If let headerValue of type headerValue be value':

a. Let HEADER typeId `{ _ ';' fieldValue* `} be headerValue.

b. Let value be HEADER typeId `{ true ';' fieldValue* `}.

c. Let context EC_1 be the result of writing value to storageReference, under context EC_0 and state $ARCH_0$ at p.

d. Let returnResult be RETURN ·.

e. The result is context EC_1, state $ARCH_0$, and call result returnResult.

Calling builtinSetInvalidMethodCallee `< · `> `( · `), under context EC_0 and state $ARCH_0$ at p:

1. Let BUILTIN_SET_INVALID storageReference be builtinSetInvalidMethodCallee.
2. Let value value' be the result of reading from storageReference, under context EC_0 and state $ARCH_0$ at p.
3. If let headerValue of type headerValue be value':
   a. Let HEADER typeId `{ _ ';' fieldValue* `} be headerValue.
   b. Let value be HEADER typeId `{ false ';' fieldValue* `}.
   c. Let context EC_1 be the result of writing value to storageReference, under context EC_0 and state $ARCH_0$ at p.
   d. Let returnResult be RETURN ·.
   e. The result is context EC_1, state $ARCH_0$, and call result returnResult.

19.8.4.2.4. isValid on header unions

Calling builtinIsValidMethodCallee `< · `> `( · `), under context EC_0 and state $ARCH_0$ at p:

1. Let BUILTIN_IS_VALID storageReference be builtinIsValidMethodCallee.
2. Let value value be the result of reading from storageReference, under context EC_0 and state $ARCH_0$ at p.
3. If let headerUnionValue of type headerUnionValue be value:
   a. Let HEADER_UNION typeId `{ fieldValue* `} be headerUnionValue.
   b. For each fieldValue in fieldValue*:
      - Let $value_{field}$ _ ';' be fieldValue.

      Let $value_{field}$* be the resulting list.
   c. For each $value_{field}$ in $value_{field}$*:
      - Let bool be $isValid_header($value_{field}$).

      Let bool* be the resulting list.
   d. Let $fieldValue_{valid}$* be fieldValue* filtered by bool*.
   e. Let b_{valid} be $fieldValue_{valid}$* is equal to ·.
   f. Let returnResult be RETURN $_B$ b_{valid}.
   g. The result is context EC_0, state $ARCH_0$, and call result returnResult.

19.8.5. Extern methods

19.8.5.1. Type checking

A call to an extern method is type checked as follows:

Typing invocation externMethodTypeIR `< typeArgumentIR* `> `( argumentIR* `) with omitted type arguments typeId*, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p and context TC:

1. If let EXTERN_METHOD nameIR `( parameterIR* `) ':' $typeIR_{ret}$ be externMethodTypeIR:
   a. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL $parameterIR_{optional}$* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
   b. Let inference be $infer(typeId*, $parameterIR_{aligned}$*, $parameterIR_{optional}$*, argumentIR*).
   c. For each typeArgumentIR in typeArgumentIR*:
      - Let $typeArgumentIR_{inferred}$ be typeArgumentIR substituted by inference.

      Let $typeArgumentIR_{inferred}$* be the resulting list.
   d. For each $parameterIR_{aligned}$ in $parameterIR_{aligned}$*:
      - Let $parameterIR_{aligned_inferred}$ be $parameterIR_{aligned}$ substituted by inference.

      Let $parameterIR_{aligned_inferred}$* be the resulting list.
   e. Let $typeIR_{ret_inferred}$ be $typeIR_{ret}$ substituted by inference.
   f. Let $externMethodTypeIR_{inferred}$ be EXTERN_METHOD nameIR `( $parameterIR_{aligned\_inferred}$* `) ':' $typeIR_{ret_inferred}$.
   g. Let B be bound type variables from the p layer of TC.
   h. If $externMethodTypeIR_{inferred}$ is a well-formed callable type, with bound type variables B:
      i. Check that $callable_externMethod(p, TC).
      ii. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned_inferred}$*, under context TC and action context NOACTION at p.
      iii. The result is return type $typeIR_{ret_inferred}$ with inferred $typeArgumentIR_{inferred}$* and casted $argumentIR_{cast}$*.
2. Else:
   a. Let EXTERN_METHOD ABSTRACT nameIR `( parameterIR* `) ':' $typeIR_{ret}$ be externMethodTypeIR.
   b. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL $parameterIR_{optional}$* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
   c. Let inference be $infer(typeId*, $parameterIR_{aligned}$*, $parameterIR_{optional}$*, argumentIR*).
   d. For each typeArgumentIR in typeArgumentIR*:
      - Let $typeArgumentIR_{inferred}$ be typeArgumentIR substituted by inference.

      Let $typeArgumentIR_{inferred}$* be the resulting list.
   e. For each $parameterIR_{aligned}$ in $parameterIR_{aligned}$*:
      - Let $parameterIR_{aligned_inferred}$ be $parameterIR_{aligned}$ substituted by inference.

      Let $parameterIR_{aligned_inferred}$* be the resulting list.
   f.

Let typeIR$_{ret_inferred}$ be typeIR$_{ret}$ substituted by inference.

g. Let externMethodTypeIR$_{inferred}$ be EXTERN_METHOD ABSTRACT nameIR `( parameterIR$_{aligned\_inferred}^{*}$ `) ':' typeIR$_{ret_inferred}$.

h. Let B be bound type variables from the p layer of TC.

i. If externMethodTypeIR$_{inferred}$ is a well-formed callable type, with bound type variables B:

   i. Check that $callable_externMethod(p, TC).

   ii. Let casted argumentIR$_{cast}^{*}$ be the result of checking calling convention of argumentIR* applied on parameterIR$_{aligned_inferred}^{*}$, under context TC and action context NOACTION at p.

   iii. The result is return type typeIR$_{ret_inferred}$ with inferred typeArgumentIR$_{inferred}^{*}$ and casted argumentIR$_{cast}^{*}$.

19.8.5.2. Runtime evaluation

An extern method call is evaluated as follows:

Calling externMethodCallee `< typeArgumentIR$^{*}$ `> `( argumentIR$^{*}$ `), under context EC$_0$ and state ARCH$_0$ at p:

1. Let EXTERN_METHOD objectId '.' nameIR `< typeParameterIR$^{*}$ `> `( parameterIR$^{*}$ '#' id$_{default}^{*}$ '#' id$_{optional}^{*}$ `) `{ theta ';' frame ';' blockStatementIR'$^{?}$ `} be externMethodCallee.
2. If blockStatementIR'$^{?}$ matches pattern (_):
   a. Let blockStatementIR be blockStatementIR'$^{?}$.
   b. Let EC$_{callee_0}$ be copy of EC$_0$ from the GLOBAL layer.
   c. Let EC$_{callee_1}$ be EC$_{callee_0}$ with BLOCK.TYPE set to theta.
   d. Let EC$_{callee_2}$ be EC$_{callee_1}$ with BLOCK.FRAME set to frame.
   e. If the length of typeArgumentIR* is equal to the length of typeParameterIR*:
      i. Let EC$_{callee_3}$ be EC$_{callee_2}$ with LOCAL.TYPE set to `{ ( typeParameterIR ':' typeArgumentIR )$^{*}$ `}.
      ii. Let GIVEN parameterIR$_{aligned}^{*}$ DEFAULT parameterIR$_{default}^{*}$ OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are id$_{default}^{*}$ and optionals are id$_{optional}^{*}$.
      iii. Let p$_{shared}$ be GLOBAL.
      iv. Let state ARCH$_1$, caller's context EC$_1$, callee's context EC$_{callee_4}$, and copy-in result copyInResult be the result of copy-in from argumentIR* at LOCAL of EC$_{callee_3}$ to parameterIR$_{aligned}^{*}$ in p at EC$_0$, under state ARCH$_0$ at shared p$_{shared}$.
      v. If let abortResult of type abortResult be copyInResult:
         A. The result is context EC$_1$, state ARCH$_1$, and call result abortResult.
      vi. Else:
         A. Let $_{CONT}$ storageReference$^{?*}$ be copyInResult.
         B. Let EC$_{callee_5}$ be the result of copying defaults for parameterIR$_{default}^{*}$, under context LOCAL and context EC$_{callee_4}$.
         C. Let context EC$_{callee_6}$, state ARCH$_2$ and result definedFunctionResult be the result of evaluating function body blockStatementIR, under context EC$_{callee_5}$ and state ARCH$_1$.
         D. Let EC$_2$ be the result of copy-out from parameterIR$_{aligned}^{*}$ at p of EC$_1$ to storageReference$^{?*}$ at LOCAL of EC$_{callee_6}$, under state ARCH$_2$ at shared p$_{shared}$.
         E. The result is context EC$_2$, state ARCH$_2$, and call result definedFunctionResult.
3. Else:
   a. Let EC$_{callee_0}$ be copy of EC$_0$ from the GLOBAL layer.

b. Let EC_{callee_1} be EC_{callee_0} with BLOCK.TYPE set to theta.

c. Let EC_{callee_2} be EC_{callee_1} with BLOCK.FRAME set to frame.

d. If the length of typeArgumentIR* is equal to the length of typeParameterIR*:

i. Let EC_{callee_3} be EC_{callee_2} with LOCAL.TYPE set to `{ ( typeParameterIR ':' typeArgumentIR )* `}.

ii. Let GIVEN $parameterIR_{aligned}$* DEFAULT $parameterIR_{default}$* OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.

iii. Let p_{shared} be GLOBAL.

iv. Let state $ARCH_1$, caller's context EC_1, callee's context EC_{callee_4}, and copy-in result copyInResult be the result of copy-in from argumentIR* at LOCAL of EC_{callee_3} to $parameterIR_{aligned}$* in p at EC_0, under state $ARCH_0$ at shared p_{shared}.

v. If let abortResult of type abortResult be copyInResult:

A. The result is context EC_1, state $ARCH_1$, and call result abortResult.

vi. Else:

A. Let CONT storageReference?* be copyInResult.

B. Let EC_{callee_5} be the result of copying defaults for $parameterIR_{default}$*, under context LOCAL and context EC_{callee_4}.

C. For each parameterIR in parameterIR*:

- Let _ _ _ $nameIR_{param}$ _ be parameterIR.

Let $nameIR_{param}$* be the resulting list.

D. Let context EC_{callee_6}, state $ARCH_2$, and call result callResult be the result of calling an extern method nameIR `( $nameIR_{param}$* `) on object objectId, under context EC_{callee_5}, and state $ARCH_1$.

E. If callResult has type rejectTransitionResult:

I. Let rejectTransitionResult be callResult.

II. The result is context EC_1, state $ARCH_2$, and call result rejectTransitionResult.

F. Else:

I. Let EC_2 be the result of copy-out from $parameterIR_{aligned}$* at p of EC_1 to storageReference?* at LOCAL of EC_{callee_6}, under state $ARCH_2$ at shared p_{shared}.

II. The result is context EC_2, state $ARCH_2$, and call result callResult.

19.8.6. Parser apply methods

19.8.6.1. Type checking

A call to a parser apply method is type checked as follows:

Typing invocation parserApplyMethodTypeIR `< · `> `( argumentIR* `) with omitted type arguments ·, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p and context TC:

1. Let PARSER_APPLY `( parameterIR* `) be parserApplyMethodTypeIR.
2. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
3. Check that $callable_parserApplyMethod(p, TC).
4. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on

parameterIR$_{aligned}^{*}$, under context TC and action context NOACTION at p.

5. The result is return type VOID with inferred · and casted argumentIR$_{cast}^{*}$.

19.8.6.2. Runtime evaluation

A parser apply method call is evaluated as follows:

Calling parserApplyMethodCallee `< typeArgumentListIR `> `( argumentListIR `), under context EC$_0$ and state ARCH$_0$ at p:

1. Let PARSER _ '.' APPLY `( parameterListIR '#' id$_{default}^{*}$ `) `{ theta ';' frame ';' parserLocalDeclarationListIR ';' stateEnv `} be parserApplyMethodCallee.
2. Let EC$_{callee_0}$ be copy of EC$_0$ from the GLOBAL layer.
3. Let EC$_{callee_1}$ be EC$_{callee_0}$[BLOCK.TYPE = theta][BLOCK.FRAME = frame] with BLOCK.STATE set to stateEnv.
4. Let GIVEN parameterIR$_{aligned}^{*}$ DEFAULT parameterIR$_{default}^{*}$ OPTIONAL _* be aligning parameterListIR with argumentListIR where defaults are id$_{default}^{*}$ and optionals are ·.
5. Let p$_{shared}$ be GLOBAL.
6. Let state ARCH$_1$, caller's context EC$_1$, callee's context EC$_{callee_2}$, and copy-in result copyInResult be the result of copy-in from argumentListIR at BLOCK of EC$_{callee_1}$ to parameterIR$_{aligned}^{*}$ in p at EC$_0$, under state ARCH$_0$ at shared p$_{shared}$.
7. If let rejectTransitionResult of type rejectTransitionResult be copyInResult:
   a. The result is context EC$_1$, state ARCH$_1$, and call result rejectTransitionResult.
8. Else if let CONT storageReference$^{?*}$ of type continueResult<storageReference?*> be copyInResult:
   a. Let EC$_{callee_3}$ be the result of copying defaults for parameterIR$_{default}^{*}$, under context BLOCK and context EC$_{callee_2}$.
   b. Let context EC$_{callee_4}$, state ARCH$_2$, and result parserDeclarationResult be the result of evaluating parser local declarations parserLocalDeclarationListIR, under context EC$_{callee_3}$, and state ARCH$_1$.
   c. If let rejectTransitionResult of type rejectTransitionResult be parserDeclarationResult:
      i. Let EC$_2$ be the result of copy-out from parameterIR$_{aligned}^{*}$ at p of EC$_1$ to storageReference$^{?*}$ at BLOCK of EC$_{callee_4}$, under state ARCH$_2$ at shared p$_{shared}$.
      ii. The result is context EC$_2$, state ARCH$_2$, and call result rejectTransitionResult.
   d. Else:
      i. Let context EC$_{callee_5}$, state ARCH$_3$, and transitionResult be the result of runtime evaluation of parser transition "start", under context EC$_{callee_4}$ and state ARCH$_2$.
      ii. If let rejectTransitionResult of type rejectTransitionResult be transitionResult:
         A. Let EC$_2$ be the result of copy-out from parameterIR$_{aligned}^{*}$ at p of EC$_1$ to storageReference$^{?*}$ at BLOCK of EC$_{callee_5}$, under state ARCH$_3$ at shared p$_{shared}$.
         B. The result is context EC$_2$, state ARCH$_3$, and call result rejectTransitionResult.
      iii. Else if let acceptTransitionResult of type acceptTransitionResult be transitionResult:
         A. Let EC$_2$ be the result of copy-out from parameterIR$_{aligned}^{*}$ at p of EC$_1$ to storageReference$^{?*}$ at BLOCK of EC$_{callee_5}$, under state ARCH$_3$ at shared p$_{shared}$.
         B. The result is context EC$_2$, state ARCH$_3$, and call result RETURN ·.

19.8.7. Control apply methods

19.8.7.1. Type checking

A call to a control apply method is type checked as follows:

Typing invocation controlApplyMethodTypeIR `< · `> `( argumentIR* `) with omitted type arguments ·, defaults $id_{default}$* and optionals $id_{optional}$*, under cursor p and context TC:

1. Let CONTROL_APPLY `( parameterIR* `) be controlApplyMethodTypeIR.
2. Let GIVEN $parameterIR_{aligned}$* DEFAULT _* OPTIONAL _* be aligning parameterIR* with argumentIR* where defaults are $id_{default}$* and optionals are $id_{optional}$*.
3. Check that $callable_controlApplyMethod(p, TC).
4. Let casted $argumentIR_{cast}$* be the result of checking calling convention of argumentIR* applied on $parameterIR_{aligned}$*, under context TC and action context NOACTION at p.
5. The result is return type VOID with inferred · and casted $argumentIR_{cast}$*.

19.8.7.2. Runtime evaluation

A control apply method call is evaluated as follows:

Calling controlApplyMethodCallee `< typeArgumentListIR `> `( argumentListIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let CONTROL _ '.' APPLY `( parameterListIR '#' $id_{default}$* `) `{ theta ';' frame ';' controlLocalDeclarationListIR ';' actionDefEnv ';' controlBodyIR `} be controlApplyMethodCallee.
2. Let EC_{callee_0} be copy of EC_0 from the GLOBAL layer.
3. Let `{ ( callableId ':' actionDef )* `} be actionDefEnv.
4. Let EC_{callee_1} be EC_{callee_0}[BLOCK.TYPE = theta][BLOCK.FRAME = frame] with BLOCK.CALLABLE set to `{ ( callableId ':' actionDef )* `}.
5. Let GIVEN $parameterIR_{aligned}$* DEFAULT $parameterIR_{default}$* OPTIONAL _* be aligning parameterListIR with argumentListIR where defaults are $id_{default}$* and optionals are ·.
6. Let p_{shared} be GLOBAL.
7. Let state $ARCH_1$, caller's context EC_1, callee's context EC_{callee_2}, and copy-in result copyInResult be the result of copy-in from argumentListIR at BLOCK of EC_{callee_1} to $parameterIR_{aligned}$* in p at EC_0, under state $ARCH_0$ at shared p_{shared}.
8. If let exitResult of type exitResult be copyInResult:
   a. The result is context EC_1, state $ARCH_1$, and call result EXIT.
9. Else if let CONT storageReference?* of type continueResult<storageReference?*> be copyInResult:
   a. Let EC_{callee_3} be the result of copying defaults for $parameterIR_{default}$*, under context BLOCK and context EC_{callee_2}.
   b. Let context EC_{callee_4}, state $ARCH_2$, and result controlLocalDeclarationResult be the result of evaluating controlLocalDeclarationListIR, under context EC_{callee_3}, and state $ARCH_1$.
   c. If let exitResult of type exitResult be controlLocalDeclarationResult:
      i. Let EC_2 be the result of copy-out from $parameterIR_{aligned}$* at p of EC_1 to storageReference?* at BLOCK of EC_{callee_4}, under state $ARCH_2$ at shared p_{shared}.
      ii. The result is context EC_2, state $ARCH_2$, and call result EXIT.
   d. Else:
      i. Let continueEmptyResult be controlLocalDeclarationResult.
      ii. Let context EC_{callee_5}, state $ARCH_3$, and result controlBodyResult be the result of evaluating controlBodyIR, under context EC_{callee_4}, and state $ARCH_2$.
      iii. Try:
         A. If let exitResult of type exitResult be controlBodyResult:

I. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_5}, under state $ARCH_3$ at shared p_{shared}.

II. The result is context EC_2, state $ARCH_3$, and call result EXIT.

iv. Then, try:

A. Check that controlBodyResult is equal to RETURN ·.

B. Let EC_2 be the result of copy-out from $parameterIR_{aligned}^{*}$ at p of EC_1 to $storageReference^{?*}$ at BLOCK of EC_{callee_5}, under state $ARCH_3$ at shared p_{shared}.

C. The result is context EC_2, state $ARCH_3$, and call result RETURN ·.

A control apply method body is evaluated as follows:

ControlBody_eval:

- Evaluating controlBodyIR, under context EC, and state ARCH:
- The result is context EC, state ARCH, and result controlBodyResult.

Evaluating controlBodyIR, under context EC_0, and state $ARCH_0$:

1. Let context EC_1, state $ARCH_1$, and statementResult be the result of runtime evaluation of controlBodyIR, under context EC_0, and state $ARCH_0$.
2. Try:
   a. If let continueEmptyResult of type continueEmptyResult be statementResult:
      i. The result is context EC_1, state $ARCH_1$, and result RETURN ·.
   b. Else if let exitResult of type exitResult be statementResult:
      i. The result is context EC_1, state $ARCH_1$, and result EXIT.
3. Then, try:
   a. Check that statementResult is equal to RETURN ·.
   b. The result is context EC_1, state $ARCH_1$, and result RETURN ·.

The result of evaluating a control body is either an exit or a return.

```
controlBodyResult
    : exitResult | returnResult
    ;
```

19.8.8. Table apply methods

19.8.8.1. Type checking

A call to a table apply method is type checked as follows:

Typing invocation tableApplyMethodTypeIR `< · `> `( · `) with omitted type arguments ·, defaults · and optionals ·, under cursor p and context TC:

1. Let TABLE_APPLY ':' tableMetadataStructTypeIR be tableApplyMethodTypeIR.
2. Check that $callable_tableApplyMethod(p, TC).
3. The result is return type tableMetadataStructTypeIR with inferred · and casted ·.

19.8.8.2. Runtime evaluation

A table apply method call is evaluated as follows:

Calling tableApplyMethodCallee `< typeArgumentListIR `> `( argumentListIR `), under context EC_0 and state $ARCH_0$ at p:

1. Let TABLE objectId '.' APPLY `{ frame ';' TBL `} be tableApplyMethodCallee.
2. Let EC_{callee_0} be copy of EC_0 from the BLOCK layer.
3. Let EC_{callee_1} be EC_{callee_0} with LOCAL.FRAMES set to frame.
4. Let $typeId'^*$ be the reversal of objectId.
5. Check that $typeId'^*$ is a non-empty list.
6. Let typeId :: $_^*$ be $typeId'^*$.
7. Let EC_{callee_1} $ARCH_0$ |- typeId TBL : EC_{callee_2} $ARCH_1$ tableResult.
8. If let exitResult of type exitResult be tableResult:
   a. Let EC_1 be $copy_e(BLOCK, EC_{callee_2}, EC_0).
   b. The result is context EC_1, state $ARCH_1$, and call result EXIT.
9. Else:
   a. Let returnResult be tableResult.
   b. Let EC_1 be $copy_e(BLOCK, EC_{callee_2}, EC_0).
   c. The result is context EC_1, state $ARCH_1$, and call result returnResult.

20. Operations

Unary and binary operations rely on two primitive integer helpers, the power of two and integer negation:

2 to the power of nat

The two's complement negation of i

20.1. Unary operations

Unary operations are evaluated with:

unop value

1. If unop matches pattern '~': return '~' value.
2. Else if unop matches pattern '!': return '!' value.
3. Else if unop matches pattern '+': return '+' value.
4. Else: return '-' value.

Negation

'!' value

1. Check that value has type boolValue.
2. Let $_{B}$b be value.
3. Return $_{B}$~b.

Bitwise complement

'~' value

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let w W i be integerLiteral:
    a. Let i' be the two's complement negation of i.
    b. Let int be 2 to the power of w.
    c. Let i'' be i' if i' is greater than or equal to 0 otherwise int + i'.
    d. Return w W i''.
4. Else if let w S i be integerLiteral:
    a. Let i' be the two's complement negation of i.
    b. Let int' be 2 to the power of w.
    c. Let i'' be i' if i' is greater than or equal to 0 otherwise int' + i'.
    d. Return w S i''.

Plus

'+' value

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i be integerLiteral: return D i.
4. Else if let w W i be integerLiteral: return w W i.
5. Else:
    a. Let w S i be integerLiteral.
    b. Return w S i.

Minus

'-' value

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i be integerLiteral: return D -i.
4. Else if let w W i be integerLiteral:
    a. Let int be 2 to the power of w.
    b. Let i' be the bitstring of width w representing the integer int - i.
    c. Return w W i'.
5. Else:
    a. Let w S i be integerLiteral.
    b. Let int' be the two's complement representation of bitstring -i of width w.
    c. Let i' be the bitstring of width w representing the integer int'.
    d. Return w S i'.

20.2. Binary operations

Binary operations are evaluated with:

$value_l$ binop $value_r$

1. If binop matches pattern '+':
    a. Return $value_l$ '+' $value_r$.
2. Else if binop matches pattern '|+|':
    a. Return $value_l$ '|+|' $value_r$.
3. Else if binop matches pattern '-': return $value_l$ '-' $value_r$.
4. Else if binop matches pattern '|-|': return $value_l$ '|-|' $value_r$.
5. Else if binop matches pattern '*': return $value_l$ '*' $value_r$.
6. Else if binop matches pattern '/': return $value_l$ '/' $value_r$.
7. Else if binop matches pattern '%': return $value_l$ '%' $value_r$.
8. Else if binop matches pattern '<<': return $value_l$ '<<' $value_r$.
9. Else if binop matches pattern '>>': return $value_l$ '>>' $value_r$.

10. Else if binop matches pattern '<=':
    a. Let bool be $value_l$ '<=' $value_r$.
    b. Return $_B$ bool.
11. Else if binop matches pattern '>=':
    a. Let bool' be $value_l$ '>=' $value_r$.
    b. Return $_B$ bool'.
12. Else if binop matches pattern '<':
    a. Let bool'' be $value_l$ '<' $value_r$.
    b. Return $_B$ bool''.
13. Else if binop matches pattern '>':
    a. Let bool''' be $value_l$ '>' $value_r$.
    b. Return $_B$ bool'''.
14. Else if binop matches pattern '==':
    a. Let bool'''' be $value_l$ '==' $value_r$.
    b. Return $_B$ bool''''.
15. Else if binop matches pattern '!=':
    a. Let bool''''' be $value_l$ '!=' $value_r$.
    b. Return $_B$ bool'''''.
16. Else if binop matches pattern '&': return $value_l$ '&' $value_r$.
17. Else if binop matches pattern '^': return $value_l$ '^' $value_r$.
18. Else if binop matches pattern '|': return $value_l$ '|' $value_r$.
19. Else if binop matches pattern '++':
    a. Return $value_l$ '++' $value_r$.

Addition

value '+' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern D %.
    d. Let D i_r be integerLiteral'.
    e. Return D i_l + i_r.
4. Else if let w W i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % W %.
    d. Let w' W i_r be integerLiteral'.

e. Check that w is equal to w'.

f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.

g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.

h. Let i be the bitstring of width w representing the integer $i_{l'} + i_{r'}$.

i. Return w W i.

5. Else:

a. Let w S i_l be integerLiteral.

b. Check that value' has type integerLiteral.

c. Let integerLiteral' be value'.

d. Check that integerLiteral' matches pattern % S %.

e. Let w' S i_r be integerLiteral'.

f. Check that w is equal to w'.

g. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.

h. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.

i. Let i be the bitstring of width w representing the integer $i_{l'} + i_{r'}$.

j. Return w S i.

Saturated addition

value '|+|' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let w W i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. Check that integerLiteral' matches pattern % W %.

d. Let w' W i_r be integerLiteral'.

e. Check that w is equal to w'.

f. Let i be $i_l + i_r$.

g. Let i_{max} be 2 to the power of w.

h. Let i' be i if i is less than i_{max} otherwise $i_{max} - 1$.

i. Return w W i'.

4. Else if let w S i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. Check that integerLiteral' matches pattern % S %.

d. Let w'' S i_r be integerLiteral'.

e. Check that w is equal to w''.

f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.

g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.

h. Let i' be $i_{l'} + i_{r'}$.

i. Try:

   i. If i' is greater than 0:

      A. Let i be w - 1.

      B. If let w' of type nat be i:

         I. Let i_{max} be 2 to the power of w'.

         II. Let i'' be i' if i' is less than i_{max} otherwise i_{max} - 1.

         III. Let i''' be the bitstring of width w representing the integer i''.

         IV. Return w S i'''.

j. Then, try:

   i. Check that i' is less than or equal to 0.

   ii. Let i be w - 1.

   iii. Check that i has type nat.

   iv. Let w' be i.

   v. Let int be 2 to the power of w'.

   vi. Let i_{min} be -int.

   vii. Let i'' be i' if i' is greater than or equal to i_{min} otherwise i_{min}.

   viii. Let i''' be the bitstring of width w representing the integer i''.

   ix. Return w S i'''.

Subtraction

value '-' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern D %.
   d. Let D i_r be integerLiteral'.
   e. Return D i_l - i_r.
4. Else if let w W i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern % W %.
   d. Let w' W i_r be integerLiteral'.
   e. Check that w is equal to w'.
   f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
   g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
   h. Let i be the bitstring of width w representing the integer $i_{l'}$ - $i_{r'}$.

i. Return w W i.

5. Else:
    a. Let w S i_l be integerLiteral.
    b. Check that value' has type integerLiteral.
    c. Let integerLiteral' be value'.
    d. Check that integerLiteral' matches pattern % S %.
    e. Let w' S i_r be integerLiteral'.
    f. Check that w is equal to w'.
    g. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
    h. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
    i. Let i be the bitstring of width w representing the integer $i_{l'}$ - $i_{r'}$.
    j. Return w S i.

Saturated subtraction

value '|-|' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let w W i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % W %.
    d. Let w' W i_r be integerLiteral'.
    e. Check that w is equal to w'.
    f. Let i be i_l - i_r.
    g. Let i' be i if i is greater than or equal to 0 otherwise 0.
    h. Return w W i'.
4. Else if let w S i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % S %.
    d. Let w'' S i_r be integerLiteral'.
    e. Check that w is equal to w''.
    f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
    g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
    h. Let i' be $i_{l'}$ - $i_{r'}$.
    i. Try:
        i. If i' is greater than 0:
            A. Let i be w - 1.
            B. If let w' of type nat be i:

I. Let i_{max} be 2 to the power of w'.

II. Let i'' be i' if i' is less than i_{max} otherwise i_{max} - 1.

III. Let i''' be the bitstring of width w representing the integer i''.

IV. Return w S i'''.

j. Then, try:

i. Check that i' is less than or equal to 0.

ii. Let i be w - 1.

iii. Check that i has type nat.

iv. Let w' be i.

v. Let int be 2 to the power of w'.

vi. Let i_{min} be -int.

vii. Let i'' be i' if i' is greater than or equal to i_{min} otherwise i_{min}.

viii. Let i''' be the bitstring of width w representing the integer i''.

ix. Return w S i'''.

Multiplication

value '*' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern D %.
   d. Let D i_r be integerLiteral'.
   e. Return D i_l * i_r.
4. Else if let w W i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern % W %.
   d. Let w' W i_r be integerLiteral'.
   e. Check that w is equal to w'.
   f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
   g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
   h. Let i be the bitstring of width w representing the integer $i_{l'}$ * $i_{r'}$.
   i. Return w W i.
5. Else:
   a. Let w S i_l be integerLiteral.
   b. Check that value' has type integerLiteral.
   c. Let integerLiteral' be value'.

d. Check that integerLiteral' matches pattern % S %.
e. Let w' S i_r be integerLiteral'.
f. Check that w is equal to w'.
g. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
h. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
i. Let i be the bitstring of width w representing the integer $i_{l'} * i_{r'}$.
j. Return w S i.

Division

value '/' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern D %.
   d. Let D i_r be integerLiteral'.
   e. Return D i_l / i_r.
4. Else if let w W i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern % W %.
   d. Let w' W i_r be integerLiteral'.
   e. Check that w is equal to w'.
   f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
   g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
   h. Let i be the bitstring of width w representing the integer $i_{l'}$ / $i_{r'}$.
   i. Return w W i.

Modulus

value '%' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern D %.
   d. Let D i_r be integerLiteral'.

e. Return D i_l \ i_r.

4. Else if let w W i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. Check that integerLiteral' matches pattern % W %.
   d. Let w' W i_r be integerLiteral'.
   e. Check that w is equal to w'.
   f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
   g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
   h. Let i be the bitstring of width w representing the integer $i_{l'}$ \ $i_{r'}$.
   i. Return w W i.

Shift left

value '<<' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. If let D i_r be integerLiteral':
      i. Let int be the left shift of i_l by i_r.
      ii. Return D int.
   d. Else if let w_r W i_r be integerLiteral':
      i. Let int' be the left shift of i_l by i_r.
      ii. Return D int'.
   e. Else:
      i. Let w_r S i_r be integerLiteral'.
      ii. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w_r.
      iii. Let int'' be the left shift of i_l by $i_{r'}$.
      iv. Return D int''.
4. Else if let w_l W i_l be integerLiteral:
   a. Check that value' has type integerLiteral.
   b. Let integerLiteral' be value'.
   c. If let D i_r be integerLiteral':
      i. Let int''' be the two's complement representation of bitstring i_r of width w_l.
      ii. Return w_l W i_l '<<' w_l W int'''.
   d. Else if let w_r W i_r be integerLiteral':
      i. Let int'''' be the left shift of i_l by i_r.
      ii. Let i be the bitstring of width w_l representing the integer int''''.

iii. Return w_l W i.

e. Else:

i. Let w_r S i_r be integerLiteral'.

ii. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w_r.

iii. Let int'''' be the left shift of i_l by $i_{r'}$.

iv. Let i be the bitstring of width w_l representing the integer int''''.

v. Return w_l W i.

5. Else:

a. Let w_l S i_l be integerLiteral.

b. Check that value' has type integerLiteral.

c. Let integerLiteral' be value'.

d. If let D i_r be integerLiteral':

i. Let int''''' be the two's complement representation of bitstring i_r of width w_l.

ii. Return w_l S i_l '<<' w_l S int'''''.

e. Else if let w_r W i_r be integerLiteral':

i. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w_l.

ii. Let int'''''' be the left shift of $i_{l'}$ by i_r.

iii. Let i be the bitstring of width w_l representing the integer int''''''.

iv. Return w_l S i.

f. Else:

i. Let w_r S i_r be integerLiteral'.

ii. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w_l.

iii. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w_r.

iv. Let int''''''' be the left shift of $i_{l'}$ by $i_{r'}$.

v. Let i be the bitstring of width w_l representing the integer int'''''''.

vi. Return w_l S i.

Shift right

value '>>' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. If let D i_r be integerLiteral':

i. Let int be the logical right shift of i_l by i_r.

ii. Return D int.

d. Else if let w_r W i_r be integerLiteral':

i. Let int' be the logical right shift of i_l by i_r.

ii. Return D int'.

e. Else:

i. Let w_r S i_r be integerLiteral'.

ii. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w_r.

iii. Let int'' be the logical right shift of i_l by $i_{r'}$.

iv. Return D int''.

4. Else if let w_l W i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. If let D i_r be integerLiteral':

i. Let int''' be the two's complement representation of bitstring i_r of width w_l.

ii. Return w_l W i_l '>>' w_l W int'''.

d. Else if let w_r W i_r be integerLiteral':

i. Let int'''' be the logical right shift of i_l by i_r.

ii. Let i be the bitstring of width w_l representing the integer int''''.

iii. Return w_l W i.

e. Else:

i. Let w_r S i_r be integerLiteral'.

ii. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w_r.

iii. Let int''''' be the logical right shift of i_l by $i_{r'}$.

iv. Let i be the bitstring of width w_l representing the integer int'''''.

v. Return w_l W i.

5. Else:

a. Let w_l S i_l be integerLiteral.

b. Check that value' has type integerLiteral.

c. Let integerLiteral' be value'.

d. If let D i_r be integerLiteral':

i. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w_l.

ii. Try:

A. If $i_{l'}$ is less than 0:

I. Let i be w_l - 1.

II. If let $w_{l'}$ of type nat be i:

1. Let int'''''' be 2 to the power of $w_{l'}$.

2. Let i' be the arithmetic right shift of i_l by i_r with sign extension int''''''.

3. Let i'' be the bitstring of width w_l representing the integer i'.

4. Return w_l S i''.

iii. Then, try:

A. Check that $i_{l'}$ is greater than or equal to 0.

B. Let i be the logical right shift of i_l by i_r.

C. Let i' be the bitstring of width w_l representing the integer i.

D. Return w_l S i'.

e. Else if let w_r W i_r be integerLiteral':
   i. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w_l.
   ii. Try:
      A. If $i_{l'}$ is less than 0:
         I. Let i be w_l - 1.
         II. If let $w_{l'}$ of type nat be i:
            1. Let int'''''' be 2 to the power of $w_{l'}$.
            2. Let i' be the arithmetic right shift of i_l by i_r with sign extension int''''''.
            3. Let i'' be the bitstring of width w_l representing the integer i'.
            4. Return w_l S i''.
   iii. Then, try:
      A. Check that $i_{l'}$ is greater than or equal to 0.
      B. Let i be the logical right shift of i_l by i_r.
      C. Let i' be the bitstring of width w_l representing the integer i.
      D. Return w_l S i'.
f. Else:
   i. Let w_r S i_r be integerLiteral'.
   ii. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w_l.
   iii. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w_r.
   iv. Try:
      A. If $i_{l'}$ is less than 0:
         I. Let i be w_l - 1.
         II. If let $w_{l'}$ of type nat be i:
            1. Let int''''''' be 2 to the power of $w_{l'}$.
            2. Let i' be the arithmetic right shift of i_l by i_r with sign extension int'''''''.
            3. Let i'' be the bitstring of width w_l representing the integer i'.
            4. Return w_l S i''.
   v. Then, try:
      A. Check that $i_{l'}$ is greater than or equal to 0.
      B. Let i be the logical right shift of i_l by i_r.
      C. Let i' be the bitstring of width w_l representing the integer i.
      D. Return w_l S i'.

Equality and inequality

value '==' value'

1. Try:
   a. If let $_B$ b_a of type boolValue be value:
      i. If let $_B$ b_b of type boolValue be value': return b_a is equal to b_b.
   b. Else if let ERROR '.' id_a of type errorValue be value:
      i. If let ERROR '.' id_b of type errorValue be value': return id_a is equal to id_b.

c. Else if let MATCH_KIND '.' id_a of type matchKindValue be value:
    i. If let MATCH_KIND '.' id_b of type matchKindValue be value': return id_a is equal to id_b.
d. Else if let $stringValue_a$ of type stringValue be value:
    i. If let $stringValue_b$ of type stringValue be value':
        A. Return $stringValue_a$ is equal to $stringValue_b$.
e. Else if let integerLiteral of type integerLiteral be value:
    i. If let D i_a be integerLiteral:
        A. If let integerLiteral' of type integerLiteral be value':
            I. If let D i_b be integerLiteral': return i_a is equal to i_b.
    ii. Else if let w_a W i_a be integerLiteral:
        A. If let integerLiteral' of type integerLiteral be value':
            I. If let w_b W i_b be integerLiteral':
                1. Return w_a is equal to w_b and i_a is equal to i_b.
    iii. Else:
        A. Let w_a S i_a be integerLiteral.
        B. If let integerLiteral' of type integerLiteral be value':
            I. If let w_b S i_b be integerLiteral':
                1. Return w_a is equal to w_b and i_a is equal to i_b.
f. Else if let LIST `[ $value_a^*$ `] of type listValue be value:
    i. If let LIST `[ $value_b^*$ `] of type listValue be value':
        A. Try:
            I. If the length of $value_a^*$ is not equal to the length of $value_b^*$: return false.
        B. Then, try:
            I. If the length of $value_a^*$ is equal to the length of $value_b^*$:
                1. For each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$:

                    - Let bool be $value_a$ '==' $value_b$.

                    Let $bool^*$ be the resulting list.

                2. Return all $bool^*$ are true.
g. Else if let TUPLE `( $value_a^*$ `) of type tupleValue be value:
    i. If let TUPLE `( $value_b^*$ `) of type tupleValue be value':
        A. If the length of $value_a^*$ is equal to the length of $value_b^*$:
            I. For each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$:

                - Let bool'' be $value_a$ '==' $value_b$.

                Let $bool''^*$ be the resulting list.

            II. Let bool' be all $bool''^*$ are true.
            III. Return the length of $value_a^*$ is equal to the length of $value_b^*$ and bool'.
h. Else if let sequenceValue of type sequenceValue be value:
    i. If let SEQ `( $value_a^*$ `) be sequenceValue:
        A. If let sequenceValue' of type sequenceValue be value':

I. If let SEQ `( $value_b^*$ `) be sequenceValue':

1. Try:
    1. If the length of $value_a^*$ is not equal to the length of $value_b^*$: return false.
2. Then, try:
    1. If the length of $value_a^*$ is equal to the length of $value_b^*$:
        1. For each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$:

            • Let bool''' be $value_a$ '==' $value_b$.

            Let bool'''* be the resulting list.

        2. Return all bool'''* are true.

i. Else if let ARRAY `[ $value_a^*$ `] of type arrayValue be value:

i. If let ARRAY `[ $value_b^*$ `] of type arrayValue be value':

A. Try:

I. If the length of $value_a^*$ is not equal to the length of $value_b^*$: return false.

B. Then, try:

I. If the length of $value_a^*$ is equal to the length of $value_b^*$:

1. For each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$:

    • Let bool'''' be $value_a$ '==' $value_b$.

    Let bool''''* be the resulting list.

2. Return all bool''''* are true.

j. Else if let HEADER_STACK `[ $value_a^*$ `(_ `) `] of type headerStackValue be value:

i. If let HEADER_STACK `[ $value_b^*$ `(_ `) `] of type headerStackValue be value':

A. Try:

I. If the length of $value_a^*$ is not equal to the length of $value_b^*$: return false.

B. Then, try:

I. If the length of $value_a^*$ is equal to the length of $value_b^*$:

1. For each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$:

    • Let bool''''' be $value_a$ '==' $value_b$.

    Let bool'''''* be the resulting list.

2. Return all bool'''''* are true.

k. Else if let STRUCT $typeId_a$ `{ $fieldValue_a^*$ `} of type structValue be value:

i. If let STRUCT $typeId_b$ `{ $fieldValue_b^*$ `} of type structValue be value':

A. For each $fieldValue_a$ in $fieldValue_a^*$:

• Let $value_{field_a}$ $nameIR_{field_a}$ ';' be $fieldValue_a$.

Let $nameIR_{field_a}^*$ and $value_{field_a}^*$ be the resulting lists.

B. For each $fieldValue_b$ in $fieldValue_b^*$:

• Let $value_{field_b}$ $nameIR_{field_b}$ ';' be $fieldValue_b$.

Let $nameIR_{field_b}^*$ and $value_{field_b}^*$ be the resulting lists.

C. If the length of $value_{field_a}^{*}$ is equal to the length of $value_{field_b}^{*}$:

I. If the length of $nameIR_{field_a}^{*}$ is equal to the length of $nameIR_{field_b}^{*}$:

1. For each $value_{field_a}$ in $value_{field_a}^{*}$ and $value_{field_b}$ in $value_{field_b}^{*}$:

- Let $bool'''''''$ be $value_{field_a}$ '==' $value_{field_b}$.

Let $bool'''''''^{*}$ be the resulting list.

2. Let $bool'''''$ be all $bool'''''''^{*}$ are true.
3. Let $bool'''''$ be all $(nameIR_{field_a} = nameIR_{field_b})^{*}$ are true.
4. Return $typeId_a$ is equal to $typeId_b$ and the length of $fieldValue_a^{*}$ is equal to the length of $fieldValue_b^{*}$ and $bool'''''$ and $bool'''''$.

l. Else if let HEADER $typeId_a$ `{ _ ';' $fieldValue_a^{*}$ `} of type headerValue be value:

i. If let HEADER $typeId_b$ `{ _ ';' $fieldValue_b^{*}$ `} of type headerValue be value':

A. For each $fieldValue_a$ in $fieldValue_a^{*}$:

- Let $value_{field_a}$ $nameIR_{field_a}$ ';' be $fieldValue_a$.

Let $nameIR_{field_a}^{*}$ and $value_{field_a}^{*}$ be the resulting lists.

B. For each $fieldValue_b$ in $fieldValue_b^{*}$:

- Let $value_{field_b}$ $nameIR_{field_b}$ ';' be $fieldValue_b$.

Let $nameIR_{field_b}^{*}$ and $value_{field_b}^{*}$ be the resulting lists.

C. If the length of $value_{field_a}^{*}$ is equal to the length of $value_{field_b}^{*}$:

I. If the length of $nameIR_{field_a}^{*}$ is equal to the length of $nameIR_{field_b}^{*}$:

1. For each $value_{field_a}$ in $value_{field_a}^{*}$ and $value_{field_b}$ in $value_{field_b}^{*}$:

- Let $bool'''''''''$ be $value_{field_a}$ '==' $value_{field_b}$.

Let $bool'''''''''^{*}$ be the resulting list.

2. Let $bool'''''''$ be all $bool'''''''''^{*}$ are true.
3. Let $bool'''''''$ be all $(nameIR_{field_a} = nameIR_{field_b})^{*}$ are true.
4. Return $typeId_a$ is equal to $typeId_b$ and the length of $fieldValue_a^{*}$ is equal to the length of $fieldValue_b^{*}$ and $bool'''''''$ and $bool'''''''$.

m. Else if let HEADER_UNION $typeId_a$ `{ $fieldValue_a^{*}$ `} of type headerUnionValue be value:

i. If let HEADER_UNION $typeId_b$ `{ $fieldValue_b^{*}$ `} of type headerUnionValue be value':

A. For each $fieldValue_a$ in $fieldValue_a^{*}$:

- Let $value_{field_a}$ $nameIR_{field_a}$ ';' be $fieldValue_a$.

Let $nameIR_{field_a}^{*}$ and $value_{field_a}^{*}$ be the resulting lists.

B. For each $fieldValue_b$ in $fieldValue_b^{*}$:

- Let $value_{field_b}$ $nameIR_{field_b}$ ';' be $fieldValue_b$.

Let $nameIR_{field_b}^{*}$ and $value_{field_b}^{*}$ be the resulting lists.

C. If the length of $value_{field_a}^{*}$ is equal to the length of $value_{field_b}^{*}$:

I. If the length of $nameIR_{field_a}$* is equal to the length of $nameIR_{field_b}$*:

1. For each $value_{field_a}$ in $value_{field_a}$* and $value_{field_b}$ in $value_{field_b}$*:

   - Let bool"""""""""" be $value_{field_a}$ '==' $value_{field_b}$.

   Let bool"""""""""* be the resulting list.

2. Let bool"""""""""" be all bool"""""""""* are true.
3. Let bool"""""""""" be all ($nameIR_{field_a}$ = $nameIR_{field_b}$)* are true.
4. Return $typeId_a$ is equal to $typeId_b$ and the length of $fieldValue_a$* is equal to the length of $fieldValue_b$* and bool"""""""""" and bool"""""""""".

n. Else if let enumValue of type enumValue be value:

i. If let $typeId_a$ '.' $nameIR_{field_a}$ be enumValue:

A. If let enumValue' of type enumValue be value':

I. If let $typeId_b$ '.' $nameIR_{field_b}$ be enumValue':

1. Return $typeId_a$ is equal to $typeId_b$ and $nameIR_{field_a}$ is equal to $nameIR_{field_b}$.

ii. Else:

A. Let $typeId_a$ '.' _ '.' $value_{field_a}$ be enumValue.

B. If let enumValue' of type enumValue be value':

I. If let $typeId_b$ '.' _ '.' $value_{field_b}$ be enumValue':

1. Let bool"""""""""" be $value_{field_a}$ '==' $value_{field_b}$.
2. Return $typeId_a$ is equal to $typeId_b$ and bool"""""""""".

o. Else if let invalidHeaderValue of type invalidHeaderValue be value:

i. If let invalidHeaderValue' of type invalidHeaderValue be value': return true.

2. Then, try:

a. Check that value has type integerValue.

b. Let integerValue be value.

c. Check that integerValue matches pattern % '.' % V %.

d. Let w_{max_a} '.' _ V i_a be integerValue.

e. Check that value' has type integerValue.

f. Let integerValue' be value'.

g. Check that integerValue' matches pattern % '.' % V %.

h. Let w_{max_b} '.' _ V i_b be integerValue'.

i. Return w_{max_a} is equal to w_{max_b} and i_a is equal to i_b.

$value_l$ '!=' $value_r$

1. Let bool be $value_l$ '==' $value_r$.
2. Return ~bool.

Comparisons

value '<' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern D %.
    d. Let D i_r be integerLiteral'.
    e. Return i_l is less than i_r.
4. Else if let w W i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % W %.
    d. Let w' W i_r be integerLiteral'.
    e. Check that w is equal to w'.
    f. Return i_l is less than i_r.
5. Else:
    a. Let w S i_l be integerLiteral.
    b. Check that value' has type integerLiteral.
    c. Let integerLiteral' be value'.
    d. Check that integerLiteral' matches pattern % S %.
    e. Let w' S i_r be integerLiteral'.
    f. Check that w is equal to w'.
    g. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
    h. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
    i. Return $i_{l'}$ is less than $i_{r'}$.

value '<=' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern D %.
    d. Let D i_r be integerLiteral'.
    e. Return i_l is less than or equal to i_r.
4. Else if let w W i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % W %.
    d. Let w' W i_r be integerLiteral'.

e. Check that w is equal to w'.

f. Return i_l is less than or equal to i_r.

5. Else:

a. Let w S i_l be integerLiteral.

b. Check that value' has type integerLiteral.

c. Let integerLiteral' be value'.

d. Check that integerLiteral' matches pattern % S %.

e. Let w' S i_r be integerLiteral'.

f. Check that w is equal to w'.

g. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.

h. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.

i. Return $i_{l'}$ is less than or equal to $i_{r'}$.

value '>' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. Check that integerLiteral' matches pattern D %.

d. Let D i_r be integerLiteral'.

e. Return i_l is greater than i_r.

4. Else if let w W i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. Check that integerLiteral' matches pattern % W %.

d. Let w' W i_r be integerLiteral'.

e. Check that w is equal to w'.

f. Return i_l is greater than i_r.

5. Else:

a. Let w S i_l be integerLiteral.

b. Check that value' has type integerLiteral.

c. Let integerLiteral' be value'.

d. Check that integerLiteral' matches pattern % S %.

e. Let w' S i_r be integerLiteral'.

f. Check that w is equal to w'.

g. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.

h. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.

i. Return $i_{l'}$ is greater than $i_{r'}$.

value '>=' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern D %.
    d. Let D i_r be integerLiteral'.
    e. Return i_l is greater than or equal to i_r.
4. Else if let w W i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % W %.
    d. Let w' W i_r be integerLiteral'.
    e. Check that w is equal to w'.
    f. Return i_l is greater than or equal to i_r.
5. Else:
    a. Let w S i_l be integerLiteral.
    b. Check that value' has type integerLiteral.
    c. Let integerLiteral' be value'.
    d. Check that integerLiteral' matches pattern % S %.
    e. Let w' S i_r be integerLiteral'.
    f. Check that w is equal to w'.
    g. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
    h. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
    i. Return $i_{l'}$ is greater than or equal to $i_{r'}$.

Bitwise

value '&' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let w W i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % W %.
    d. Let w' W i_r be integerLiteral'.
    e. Check that w is equal to w'.
    f. Let int be the bitwise and of i_l and i_r.

g. Let i be the bitstring of width w representing the integer int.

h. Return w W i.

4. Else if let w S i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. Check that integerLiteral' matches pattern % S %.

d. Let w' S i_r be integerLiteral'.

e. Check that w is equal to w'.

f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.

g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.

h. Let int' be the bitwise and of $i_{l'}$ and $i_{r'}$.

i. Let i be the bitstring of width w representing the integer int'.

j. Return w S i.

value '|' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let w W i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. Check that integerLiteral' matches pattern % W %.

d. Let w' W i_r be integerLiteral'.

e. Check that w is equal to w'.

f. Let int be the bitwise or of i_l and i_r.

g. Let i be the bitstring of width w representing the integer int.

h. Return w W i.

4. Else if let w S i_l be integerLiteral:

a. Check that value' has type integerLiteral.

b. Let integerLiteral' be value'.

c. Check that integerLiteral' matches pattern % S %.

d. Let w' S i_r be integerLiteral'.

e. Check that w is equal to w'.

f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.

g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.

h. Let int' be the bitwise or of $i_{l'}$ and $i_{r'}$.

i. Let i be the bitstring of width w representing the integer int'.

j. Return w S i.

value '^' value'

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let w W i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % W %.
    d. Let w' W i_r be integerLiteral'.
    e. Check that w is equal to w'.
    f. Let int be the bitwise xor of i_l and i_r.
    g. Let i be the bitstring of width w representing the integer int.
    h. Return w W i.
4. Else if let w S i_l be integerLiteral:
    a. Check that value' has type integerLiteral.
    b. Let integerLiteral' be value'.
    c. Check that integerLiteral' matches pattern % S %.
    d. Let w' S i_r be integerLiteral'.
    e. Check that w is equal to w'.
    f. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w.
    g. Let $i_{r'}$ be the two's complement representation of bitstring i_r of width w.
    h. Let int' be the bitwise xor of $i_{l'}$ and $i_{r'}$.
    i. Let i be the bitstring of width w representing the integer int'.
    j. Return w S i.

Concatenation

value '++' value'

1. If let '"' t_l '"' of type stringLiteral be value:
    a. Check that value' has type stringLiteral.
    b. Let '"' t_r '"' be value'.
    c. Return '"' t_l ++ t_r '"'.
2. Else if let integerLiteral of type integerLiteral be value:
    a. If let w_l W i_l be integerLiteral:
        i. Check that value' has type integerLiteral.
        ii. Let integerLiteral' be value'.
        iii. If let w_r W i_r be integerLiteral':
            A. Let $i_{l'}$ be the left shift of i_l by w_r.
            B. Let $i_{l''}$ be $i_{l'}$ + i_r.
            C. Let w be w_l + w_r.
            D. Let i be the bitstring of width w representing the integer $i_{l''}$.
            E. Return w W i.

iv. Else if let w_r S i_r be integerLiteral':

A. Let $i_{l'}$ be the left shift of i_l by w_r.

B. Let $i_{l''}$ be $i_{l'}$ + i_r.

C. Let w be w_l + w_r.

D. Let i be the bitstring of width w representing the integer $i_{l''}$.

E. Return w W i.

b. Else if let w_l S i_l be integerLiteral:

i. Check that value' has type integerLiteral.

ii. Let integerLiteral' be value'.

iii. If let w_r W i_r be integerLiteral':

A. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w_l.

B. Let $i_{l''}$ be the left shift of $i_{l'}$ by w_r.

C. Let $i_{l'''}$ be $i_{l''}$ + i_r.

D. Let w be w_l + w_r.

E. Let i be the bitstring of width w representing the integer $i_{l'''}$.

F. Return w S i.

iv. Else if let w_r S i_r be integerLiteral':

A. Let $i_{l'}$ be the two's complement representation of bitstring i_l of width w_l.

B. Let $i_{l''}$ be the left shift of $i_{l'}$ by w_r.

C. Let $i_{l'''}$ be $i_{l''}$ + i_r.

D. Let w be w_l + w_r.

E. Let i be the bitstring of width w representing the integer $i_{l'''}$.

F. Return w S i.

The type of a concatenation is determined from the operand types, summing their widths, with:

$result_concat(typeIR, typeIR')

1. Try:

a. If let stringTypeIR of type stringTypeIR be typeIR:

i. If let stringTypeIR' of type stringTypeIR be typeIR': return STRING.

b. Else if let INT `< $w_a$ `> of type fixedIntTypeIR be typeIR:

i. If let INT `< $w_b$ `> of type fixedIntTypeIR be typeIR':

A. Return INT `< $w_a$ + $w_b$ `>.

ii. Else if let BIT `< $w_b$ `> of type fixedBitTypeIR be typeIR':

A. Return INT `< $w_a$ + $w_b$ `>.

c. Else if let BIT `< $w_a$ `> of type fixedBitTypeIR be typeIR:

i. If let INT `< $w_b$ `> of type fixedIntTypeIR be typeIR':

A. Return BIT `< $w_a$ + $w_b$ `>.

ii. Else if let BIT `< $w_b$ `> of type fixedBitTypeIR be typeIR':

A. Return BIT `< $w_a$ + $w_b$ `>.

d. Else if let TYPEDEF _ $typeIR_l$ of type typedefTypeIR be typeIR:

i. Return $result_concat(typeIR_l, typeIR').

2. Then, try:

a. Check that typeIR' has type typedefTypeIR.

b. Let TYPEDEF _ $typeIR_r$ be typeIR'.

c. Check that typeIR does not have type typedefTypeIR.

d. Return $result_concat(typeIR, $typeIR_r$).

20.3. Default values

Some P4 types define a *default value*, which can be used to automatically initialize values of that type. The high-level description of default values are as follows:

- For int, bit<N> and int<N> types the default value is 0.
- For bool the default value is false.
- For error the default value is error.NoError (defined in core.p4)
- For string the default value is the empty string ""
- For varbit<N> the default value is a string of zero bits (there is currently no P4 literal to represent such a value).
- For enum values with an underlying type the default value is 0, even if 0 is actually not one of the named values in the enum.
- For enum values without an underlying type the default value is the first value that appears in the enum type declaration.
- For header types the default value is invalid.
- For header stacks the default value is that all elements are invalid and the nextIndex is 0.
- For header_union values the default value is that all union elements are invalid.
- For struct types the default value is a struct where each field has the default value of the suitable field type — if all such default values are defined.
- For a tuple type the default value is a tuple where each field has the default value of the suitable type — if all such default values are defined.

Note that some types do not have default values, e.g., match_kind, set types, function types, extern types, parser types, control types, and package types.

Default values are derived from:

The default value for type typeIR'

The default value for type typeIR'

1. If let boolTypeIR of type boolTypeIR be typeIR': return $_B$ false.
2. Else if let errorTypeIR of type errorTypeIR be typeIR': return ERROR '.' "NoError".

3. Else if let stringTypeIR of type stringTypeIR be typeIR':
    a. Let $text_{empty}$ be "".
    b. Return "" $text_{empty}$ "".
4. Else if let intTypeIR of type intTypeIR be typeIR': return D 0.
5. Else if let BIT `< w `> of type fixedBitTypeIR be typeIR': return w W 0.
6. Else if let INT `< w `> of type fixedIntTypeIR be typeIR': return w S 0.
7. Else if let VARBIT `< w `> of type varBitTypeIR be typeIR': return w '.' 0 V 0.
8. Else if let TYPEDEF _ typeIR of type typedefTypeIR be typeIR':
    a. Return the default value for type typeIR.
9. Else if let TYPE _ typeIR of type newTypeIR be typeIR':
    a. Return the default value for type typeIR.
10. Else if let TUPLE `< $typeIR^*$ `> of type tupleTypeIR be typeIR':
    a. For each typeIR in $typeIR^*$:
        - Let value' be the default value for type typeIR.

        Let $value'^*$ be the resulting list.
    b. Return TUPLE `( $value'^*$ `).
11. Else if let ARRAY typeIR `[ $n_s$ `] of type arrayTypeIR be typeIR':
    a. Let value'' be the default value for type typeIR.
    b. Let $value^*$ be a list of n_s copies of value''.
    c. Return ARRAY `[ $value^*$ `].
12. Else if let HEADER_STACK typeIR `[ $n_s$ `] of type headerStackTypeIR be typeIR':
    a. Let value''' be the default value for type typeIR.
    b. Let $value^*$ be a list of n_s copies of value'''.
    c. Return HEADER_STACK `[ $value^*$ `(0 `) `].
13. Else if let STRUCT typeId `< $_^*$ `> `{ ( _ $typeIR_f$ $id_f$ ';' )$^*$ `} of type structTypeIR be typeIR':
    a. For each $typeIR_f$ in $typeIR_f^*$:
        - Let value'''' be the default value for type $typeIR_f$.

        Let $value''''^*$ be the resulting list.
    b. Return STRUCT typeId `{ ( value'''' $id_f$ ';' )$^*$ `}.
14. Else if let HEADER typeId `< $_^*$ `> `{ ( _ $typeIR_f$ $id_f$ ';' )$^*$ `} of type headerTypeIR be typeIR':
    a. For each $typeIR_f$ in $typeIR_f^*$:
        - Let value''''' be the default value for type $typeIR_f$.

        Let $value'''''^*$ be the resulting list.
    b. Return HEADER typeId `{ false ';' ( value''''' $id_f$ ';' )$^*$ `}.
15. Else if let HEADER_UNION typeId `< $_^*$ `> `{ ( _ $typeIR_f$ $id_f$ ';' )$^*$ `} of type headerUnionTypeIR be typeIR':
    a. For each $typeIR_f$ in $typeIR_f^*$:
        - Let value'''''' be the default value for type $typeIR_f$.

Let value''''''* be the resulting list.

b. Return HEADER_UNION typeId `{ ( value'''''' $id_f$ ';' )* `}.

16. Else if let ENUM typeId `{ id* `} of type simpleEnumTypeIR be typeIR':
    a. Check that id* is a non-empty list.
    b. Let id_{f_h} :: _* be id*.
    c. Return typeId '.' id_{f_h}.
17. Else if let ENUM typeId `< typeIR `> `{ ( $nameIR_{field}$ '=' $value_{field}$ ';' )* `} of type serializableEnumTypeIR be typeIR':
    a. Let $value_{zero}$ be D 0 casted to type typeIR.
    b. Try:
        i. Let id? be the value associated to $value_{zero}$ in ($value_{field}$, $nameIR_{field}$)*.
        ii. If id? is defined:
            A. Let id_{zero} be id?.
            B. Return typeId '.' id_{zero} '.' $value_{zero}$.
    c. Then, try:
        i. Check that the value associated to $value_{zero}$ in ($value_{field}$, $nameIR_{field}$)* is none.
        ii. Let id_{zero} be "__UNSPECIFIED".
        iii. Return typeId '.' id_{zero} '.' $value_{zero}$.

20.4. Bit-string access operations

Bit-string access operations read or replace a contiguous range of bits within a numeric value. A range is given by its high and low indices, and an offset is given by its low index and width.

Access and replacement select the underlying bits with the primitives:

The bitslice of i from i' to i''

The bitslice of i from i' to i'' replaced with i'''

Range access and replacement

value `[ value' ':' value'' `]

1. Check that value has type integerValue.
2. Let $integerValue_b$ be value.
3. Check that value' has type integerValue.

4. Let $integerValue_h$ be value'.
5. Check that value'' has type integerValue.
6. Let $integerValue_l$ be value''.
7. Let i_b be the integer representation of $integerValue_b$.
8. Let i_h be the integer representation of $integerValue_h$.
9. Let i_l be the integer representation of $integerValue_l$.
10. Let i' be i_h + 1 - i_l.
11. Check that i' has type nat.
12. Let w be i'.
13. Let i be the bitslice of i_b from i_h to i_l.
14. Return w W i.

value `[ value' ':' value'' `] replaced with value'''

1. Check that value has type integerValue.
2. Let $integerValue_b$ be value.
3. Check that value' has type integerValue.
4. Let $integerValue_h$ be value'.
5. Check that value'' has type integerValue.
6. Let $integerValue_l$ be value''.
7. Check that value''' has type integerValue.
8. Let $integerValue_r$ be value'''.
9. Check that $integerValue_b$ has type integerLiteral.
10. Let integerLiteral be $integerValue_b$.
11. If let w W i_b be integerLiteral:
    a. Let i_h be the integer representation of $integerValue_h$.
    b. Let i_l be the integer representation of $integerValue_l$.
    c. Check that $integerValue_r$ has type integerLiteral.
    d. Let integerLiteral' be $integerValue_r$.
    e. Check that integerLiteral' matches pattern % W %.
    f. Let _ W i_r be integerLiteral'.
    g. Let i be the bitslice of i_b from i_h to i_l replaced with i_r.
    h. Return w W i.
12. Else if let w S i_b be integerLiteral:
    a. Let i_h be the integer representation of $integerValue_h$.
    b. Let i_l be the integer representation of $integerValue_l$.
    c. Check that $integerValue_r$ has type integerLiteral.
    d. Let integerLiteral' be $integerValue_r$.
    e. Check that integerLiteral' matches pattern % W %.
    f. Let _ W i_r be integerLiteral'.
    g.

Let i be the bitslice of i_b from i_h to i_l replaced with i_r.

h. Return w S i.

Offset access and replacement

value `[ value' '+:' value'' `]

1. Check that value has type integerValue.
2. Let $integerValue_b$ be value.
3. Check that value' has type integerValue.
4. Let $integerValue_l$ be value'.
5. Check that value'' has type integerValue.
6. Let $integerValue_w$ be value''.
7. Let i_b be the integer representation of $integerValue_b$.
8. Let i_l be the integer representation of $integerValue_l$.
9. Let $nat^?$ be the natural number representation of $integerValue_w$.
10. Check that $nat^?$ is defined.
11. Let w be $nat^?$.
12. Let i_h be i_l + w - 1.
13. Let i be the bitslice of i_b from i_h to i_l.
14. Return w W i.

value `[ value' '+:' value'' `] replaced with value'''

1. Check that value has type integerValue.
2. Let $integerValue_b$ be value.
3. Check that value' has type integerValue.
4. Let $integerValue_l$ be value'.
5. Check that value'' has type integerValue.
6. Let $integerValue_w$ be value''.
7. Check that value''' has type integerValue.
8. Let $integerValue_r$ be value'''.
9. Check that $integerValue_b$ has type integerLiteral.
10. Let integerLiteral be $integerValue_b$.
11. If let w W i_b be integerLiteral:
    a. Let i_l be the integer representation of $integerValue_l$.
    b. Let $nat^?$ be the natural number representation of $integerValue_w$.
    c. Check that $nat^?$ is defined.
    d. Let n_w be $nat^?$.
    e. Let i_h be i_l + n_w - 1.
    f. Check that $integerValue_r$ has type integerLiteral.

g. Let integerLiteral' be $integerValue_r$.

h. Check that integerLiteral' matches pattern % W %.

i. Let _ W i_r be integerLiteral'.

j. Let i be the bitslice of i_b from i_h to i_l replaced with i_r.

k. Return w W i.

12. Else if let w S i_b be integerLiteral:

a. Let i_l be the integer representation of $integerValue_l$.

b. Let $nat^?$ be the natural number representation of $integerValue_w$.

c. Check that $nat^?$ is defined.

d. Let n_w be $nat^?$.

e. Let i_h be i_l + n_w - 1.

f. Check that $integerValue_r$ has type integerLiteral.

g. Let integerLiteral' be $integerValue_r$.

h. Check that integerLiteral' matches pattern % W %.

i. Let _ W i_r be integerLiteral'.

j. Let i be the bitslice of i_b from i_h to i_l replaced with i_r.

k. Return w S i.

20.5. Compile-time size determination

The method calls minSizeInBits, minSizeInBytes, maxSizeInBits, and maxSizeInBytes can be applied to certain expressions. These method calls return the minimum (or maximum) size in bits (or bytes) required to store the expression. Thus, the result type of these methods has type int. Except in certain situations involving type variables, discussed below, these method calls produce local compile-time known values; otherwise they produce compile-time known values. None of these methods evaluate the expression that is the receiver of the method call, so it may be invalid (e.g., an out-of-bounds header stack access).

The method minSizeInBytes returns the result of minSizeInBits rounded up to the next whole number of bytes. In other words, for any expression e, e.minSizeInBytes() is equal to (e.minSizeInBits() + 7) >> 3.

The method maxSizeInBytes always returns the result of maxSizeInBits rounded up to the next whole number of bytes. In other words, for any expression e, e.maxSizeInBytes() is equal to (e.maxSizeInBits() + 7) >> 3.

They dispatch by method name with:

$sizeof(typeIR, t)

1. If t is equal to "minSizeInBits": return $sizeof_minSizeInBits(typeIR).
2. Else if t is equal to "minSizeInBytes": return $sizeof_minSizeInBytes(typeIR).
3. Else if t is equal to "maxSizeInBits": return $sizeof_maxSizeInBits(typeIR).
4. Else if t is equal to "maxSizeInBytes": return $sizeof_maxSizeInBytes(typeIR).

Minimum size in bits

$sizeof_minSizeInBits(typeIR)

1. Let nat be $sizeof_minSizeInBits'(typeIR).
2. Return D nat.

$sizeof_minSizeInBits'(typeIR')

1. If let boolTypeIR of type boolTypeIR be typeIR': return 1.
2. Else if let INT `< w `> of type fixedIntTypeIR be typeIR': return w.
3. Else if let BIT `< w `> of type fixedBitTypeIR be typeIR': return w.
4. Else if typeIR' has type varBitTypeIR: return 0.
5. Else if let TYPEDEF _ typeIR of type typedefTypeIR be typeIR': return $sizeof_minSizeInBits'(typeIR).
6. Else if let TYPE _ typeIR of type newTypeIR be typeIR': return $sizeof_minSizeInBits'(typeIR).
7. Else if let ENUM _ `< typeIR `> `{ _* `} of type serializableEnumTypeIR be typeIR': return $sizeof_minSizeInBits'(typeIR).
8. Else if let TUPLE `< typeIR* `> of type tupleTypeIR be typeIR':
   a. For each typeIR in typeIR*:
      - Let nat be $sizeof_minSizeInBits'(typeIR).

      Let nat* be the resulting list.
   b. Return sum`( nat* `).
9. Else if let ARRAY typeIR `[ $n_{size}$ `] of type arrayTypeIR be typeIR':
   a. Let nat' be $sizeof_minSizeInBits'(typeIR).
   b. Return nat' * n_{size}.
10. Else if let HEADER_STACK typeIR `[ $n_{size}$ `] of type headerStackTypeIR be typeIR':
    a. Let nat'' be $sizeof_minSizeInBits'(typeIR).
    b. Return nat'' * n_{size}.
11. Else if let STRUCT _ `< _* `> `{ ( _ typeIR _ ';' )* `} of type structTypeIR be typeIR':
    a. For each typeIR in typeIR*:
       - Let nat''' be $sizeof_minSizeInBits'(typeIR).

       Let nat'''* be the resulting list.
    b. Return sum`( nat'''* `).
12. Else if let HEADER _ `< _* `> `{ ( _ typeIR _ ';' )* `} of type headerTypeIR be typeIR':
    a. For each typeIR in typeIR*:
       - Let nat'''' be $sizeof_minSizeInBits'(typeIR).

       Let nat''''* be the resulting list.
    b. Return sum`( nat''''* `).
13. Else if typeIR' has type headerUnionTypeIR:
    a. Try:
       i. Let HEADER_UNION _ `< _* `> `{ fieldTypeIR* `} be typeIR'.
       ii. If fieldTypeIR* is an empty list: return 0.

b. Then, try:

i. Let HEADER_UNION _ `< _* `> `{ ( _ typeIR _ ';' )* `} be typeIR'.

ii. Check that typeIR* is a non-empty list.

iii. For each typeIR in typeIR*:

- Let nat'''' be $sizeof_minSizeInBits'(typeIR).

Let nat''''* be the resulting list.

iv. Return max`( nat''''* `).

Minimum size in bytes

$sizeof_minSizeInBytes(typeIR)

1. Let n_{size} be $sizeof_minSizeInBits'(typeIR).
2. Return D n_{size} / 8.

Maximum size in bits

$sizeof_maxSizeInBits(typeIR)

1. Let nat be $sizeof_maxSizeInBits'(typeIR).
2. Return D nat.

$sizeof_maxSizeInBits'(typeIR')

1. If let boolTypeIR of type boolTypeIR be typeIR': return 1.
2. Else if let INT `< w `> of type fixedIntTypeIR be typeIR': return w.
3. Else if let BIT `< w `> of type fixedBitTypeIR be typeIR': return w.
4. Else if let VARBIT `< w `> of type varBitTypeIR be typeIR': return w.
5. Else if let TYPEDEF _ typeIR of type typedefTypeIR be typeIR': return $sizeof_maxSizeInBits'(typeIR).
6. Else if let TYPE _ typeIR of type newTypeIR be typeIR': return $sizeof_maxSizeInBits'(typeIR).
7. Else if let ENUM _ `< typeIR `> `{ _* `} of type serializableEnumTypeIR be typeIR': return $sizeof_maxSizeInBits'(typeIR).
8. Else if let TUPLE `< typeIR* `> of type tupleTypeIR be typeIR':
   a. For each typeIR in typeIR*:
      - Let nat be $sizeof_maxSizeInBits'(typeIR).

      Let nat* be the resulting list.

   b. Return sum`( nat* `).
9. Else if let ARRAY typeIR `[ $n_{size}$ `] of type arrayTypeIR be typeIR':
   a. Let nat' be $sizeof_maxSizeInBits'(typeIR).
   b. Return nat' * n_{size}.
10. Else if let HEADER_STACK typeIR `[ $n_{size}$ `] of type headerStackTypeIR be typeIR':

a. Let nat'' be $sizeof_maxSizeInBits'(typeIR).

b. Return nat'' * n_{size}.

11. Else if let STRUCT _ `< _* `> `{ ( _ typeIR _ ';' )* `} of type structTypeIR be typeIR':

a. For each typeIR in typeIR*:

- Let nat''' be $sizeof_maxSizeInBits'(typeIR).

Let nat'''* be the resulting list.

b. Return sum`( nat'''* `).

12. Else if let HEADER _ `< _* `> `{ ( _ typeIR _ ';' )* `} of type headerTypeIR be typeIR':

a. For each typeIR in typeIR*:

- Let nat'''' be $sizeof_maxSizeInBits'(typeIR).

Let nat''''* be the resulting list.

b. Return sum`( nat''''* `).

13. Else if typeIR' has type headerUnionTypeIR:

a. Try:

i. Let HEADER_UNION _ `< _* `> `{ fieldTypeIR* `} be typeIR'.

ii. If fieldTypeIR* is an empty list: return 0.

b. Then, try:

i. Let HEADER_UNION _ `< _* `> `{ ( _ typeIR _ ';' )* `} be typeIR'.

ii. Check that typeIR* is a non-empty list.

iii. For each typeIR in typeIR*:

- Let nat''''' be $sizeof_maxSizeInBits'(typeIR).

Let nat'''''* be the resulting list.

iv. Return max`( nat'''''* `).

Maximum size in bytes

$sizeof_maxSizeInBytes(typeIR)

1. Let n_{size} be $sizeof_maxSizeInBits'(typeIR).
2. Return D n_{size} / 8.

20.6. Keyset matching

Keyset matching tests whether a value belongs to a keyset. It is used both when matching parser select cases and when matching table entries.

A tuple of keysets matches a tuple of values when each keyset matches its corresponding value:

value* matches setValue'*

1. Try:
    a. If setValue'* matches pattern []:
        i. If value* is an empty list: return true.
    b. Else if setValue'* matches pattern [_/1]:
        i. Let setValue be setValue'*.
        ii. If value* is a list of length 1:
            A. Let $value_{key}$ be value*.
            B. If setValue does not match pattern SET `{ '...' `}:
                I. Return $match_keyset(setValue, $value_{key}$).
    c. Else if setValue'* matches pattern :::
        i. Let $setValue_{h}$ :: $setValue_{t}^{*}$ be setValue'*.
        ii. If value* is a non-empty list:
            A. Let $value_{key_h}$ :: $value_{key_t}^{*}$ be value*.
            B. If $setValue_{t}^{*}$ is a non-empty list:
                I. If $value_{key_t}^{*}$ is a non-empty list:
                    1. If $match_keyset($setValue_{h}$, $value_{key_h}$):
                        1. Return $value_{key_t}^{*}$ matches $setValue_{t}^{*}$.
                    2. Else: return false.
2. Then, try:
    a. Check that setValue'* is equal to SET `{ '...' `}.
    b. Return true.

A single keyset matches a value when the value is a member of the keyset's set:

$match_keyset(setValue, $value_{key}$)

1. If setValue matches pattern SET `{ '...' `}: return true.
2. Else if let SET `{ value `} be setValue: return value '==' $value_{key}$.
3. Else if let SET `{ value* `(_ `) `} be setValue:
    a. For each value in value*:

        - Let bool be value '==' $value_{key}$.

        Let bool* be the resulting list.

    b. Return some of bool* is true.
4. Else if let SET `{ $value_{base}$ '&&&' $value_{mask}$ `} be setValue:
    a. Let value' be $value_{base}$ '&' $value_{mask}$.
    b. Let value'' be $value_{key}$ '&' $value_{mask}$.
    c. Return value' '==' value''.
5. Else:
    a. Let SET `{ $value_{base}$ '..' $value_{limit}$ `} be setValue.
    b. Let bool' be $value_{base}$ '<=' $value_{key}$.
    c. Let bool'' be $value_{key}$ '<=' $value_{limit}$.

d. Return bool' and bool''.

21. Annotations

Annotations are a simple mechanism for extending the P4 language to some limited degree without changing the grammar. Annotations are attached to types, fields, variables, etc. using the @ syntax (as shown explicitly in the P4 grammar). Unstructured annotations, or just "annotations," have an optional body; structured annotations have a mandatory body, containing at least a pair of square brackets [].

```
annotationBody
    : /* empty */ | annotationBody ( annotationBody )
    | annotationBody annotationToken
    ;

structuredAnnotationBody
    : sequenceOrRecordElementExpression trailingCommaOpt
    ;

annotation
    : @ name | @ name ( annotationBody ) | @ name [ structuredAnnotationBody ]
    | @ PRAGMA name annotationBody
    ;
```

Structured annotations and unstructured annotations on any one element must not use the same name. Thus, a given name can only be applied to one type of annotation or the other for any one element. An annotation used on one element does not affect the annotation on another because they have different scope.

This is legal:

```
@my_anno(1) table T { /* body omitted */ }
@my_anno[2] table U { /* body omitted */ } // OK - different scope than previous
                                          // use of my_anno
```

This is illegal:

```
@my_anno(1)
@my_anno[2] table U { /* body omitted */ } // Error - changed type of anno
                                          // on an element
```

Multiple unstructured annotations using the same name can appear on a given element; they are cumulative. Each one will be bound to that element. In contrast, only one structured annotation using a given name may appear on an element; multiple uses of the same name will produce an error.

This is legal:

```
@my_anno(1)
@my_anno(2) table U { /* body omitted */ }  // OK - unstructured annos accumulate
```

This is illegal:

```
@my_anno[1]
@my_anno[2] table U { /* body omitted */ } // Error - reused the same structured
                                           // anno on an element
```

21.1. Bodies of Unstructured Annotations

The flexibility of P4 unstructured annotations comes from the minimal structure mandated by the P4 grammar: unstructured annotation bodies may contain any sequence of terminals, so long as parentheses are balanced. In the following grammar fragment, the annotationToken non-terminal represents any terminal produced by the lexer, including keywords, identifiers, string and integer literals, and symbols, but excluding parentheses.

```
annotationBody
    : /* empty */ | annotationBody ( annotationBody )
    | annotationBody annotationToken
    ;
```

Unstructured annotations may impose additional structure on their bodies, and are not confined to the P4 language. For example, the P4Runtime specification [2 - Chapter 10] defines a @pkginfo annotation that expects key-value pairs. === Bodies of Structured Annotations

Unlike unstructured annotations, structured annotations use square brackets [...] and have a restricted format. They are commonly used to declare custom metadata, consisting of expression lists or key-value lists but not both. An expressionList may be empty or contain a comma-separated list of member expressions. A namedExpressionList consists of one or more namedExpressions, each consisting of a key and a value expression. Note the syntax for expression is rich, see Appendix F for details.

All expressions within a structuredAnnotationBody must be compile-time known values with a result type that is either: string, int, or bool. In particular, structured expressions (e.g. an expression containing an expressionList, a namedExpressionList, etc.) are not allowed. Note that P4Runtime information (P4Info) may stipulate additional restrictions. For example, an integer expression might be limited to 64-bit values.

It is illegal to duplicate a name within the namedExpressionList of a structured annotation.

```
structuredAnnotationBody
    : sequenceOrRecordElementExpression trailingCommaOpt
    ;

sequenceOrRecordElementExpression
    : sequenceElementExpression | recordElementExpression
    ;

sequenceElementExpression = expressionList

recordElementExpression
    : name = expression | name = expression , ...
    | name = expression , namedExpressionList
    | name = expression , namedExpressionList , ...
    ;

namedExpression
    : name = expression
    ;
```

21.1.1. Structured Annotation Examples

Empty Expression List

The following example produces an empty annotation:

```
@Empty[]
table t {
    /* body omitted */
}
```

Mixed Expression List

The following example will produce an effective expression list as follows:

```
[1,"hello",true, false, 11]
```

```
#define TEXT_CONST "hello"
#define NUM_CONST 6
@MixedExprList[1,TEXT_CONST,true,1==2,5+NUM_CONST]
table t {
    /* body omitted */
}
```

namedExprssionList of Strings

```
@Labels[short="Short Label", hover="My Longer Table Label to appear in hover-help"]
table t {
    /* body omitted */
}
```

namedExpressionList of Mixed Expressions

The following example will produce an effective namedExpressionList as follows.

```
[label="text", my_bool=true, int_val=6]
```

```
@MixedKV[label="text", my_bool=true, int_val=2*3]
table t {
    /* body omitted */
}
```

Illegal Mixing of namedExpression and expressionList

The following example is invalid because the body contains both a namedExpression and an expression:

```
@IllegalMixing[key=4, 5] // illegal mixing
table t {
    /* body omitted */
}
```

Illegal Duplicate Key

The following example is invalid because the same key occurs more than once:

```
@DupKey[k1=4,k1=5] // illegal duplicate key
table t {
```

```
    /* body omitted */
}
```

Illegal Duplicate Structured Annotation

The following example is invalid because the annotation name occurs more than once on the same element, e.g. table t:

```
@DupAnno[k1=4]
@DupAnno[k2=5] // illegal duplicate name
table t {
    /* body omitted */
}
```

Illegal Simultaneous Use of Both Structured and Unstructured Annotation

The following example is invalid because the annotation name is used by both an unstructured and structured annotation on the same element table t:

```
@MixAnno("Anything")
@MixAnno[k2=5] // illegal use in both annotation types
table t {
    /* body omitted */
}
```

21.2. Predefined annotations

Annotation names that start with lowercase letters are reserved for the standard library and architecture. This document pre-defines a set of "standard" annotations in Appendix C. We expect that this list will grow. We encourage custom architectures to define annotations starting with a manufacturer prefix: e.g., an organization named X would use annotations named like @X_annotation

21.2.1. Optional parameter annotations

A parameter to a package, parser type, control type, extern method, extern function or extern object constructor can be annotated with @optional to indicate that the user does not need to provide a corresponding argument for that parameter. The meaning of a parameter with no supplied value is target-dependent.

Whether a parameter carries the @optional annotation is checked by:

The annotations of annotationList _ _ _ _

1. Return annotationList.

parameterIR is optional

1. Let annotationList be the annotations of parameterIR.
2. Return annotationList contains an optional annotation.

annotationList contains an optional annotation

1. Let annotation* be annotationList flattened.
2. For each annotation in annotation*:
   - Let b be annotation is an optional annotation.

   Let b* be the resulting list.
3. Return some of b* is true.

annotation is an optional annotation

1. If let '@' name be annotation:
   a. If "optional" is equal to the name of name: return true.
2. Otherwise: return false.

21.2.2. Annotations on the table action list

The following two annotations can be used to give additional information to the compiler and control-plane about actions in a table. These annotations have no bodies.

- @tableonly: actions with this annotation can only appear within the table, and never as default action.
- @defaultonly: actions with this annotation can only appear in the default action, and never in the table.

```
table t {
    actions = {
       a,              // can appear anywhere
       @tableonly b,   // can only appear in the table
       @defaultonly c, // can only appear in the default action
    }
    /* body omitted */
}
```

21.2.3. Control-plane API annotations

The @name annotation directs the compiler to use a different local name when generating the external APIs used to manipulate a language element from the control plane. This annotation takes a local compile-time known value of type string (typically a string literal). In the following example, the fully-qualified name of the table is c_inst.t1.

```
control c( /* parameters omitted */ )() {
    @name("t1") table t { /* body omitted */ }
    apply { /* body omitted */ }
}
c() c_inst;
```

The @hidden annotation hides a controllable entity, e.g. a table, key, action, or extern, from the control plane. This effectively removes its fully-qualified name (Section 10.1.1). This annotation does not have a body.

21.2.3.1. Restrictions

Each element may be annotated with at most one @name or @hidden annotation, and each control plane

name must refer to at most one controllable entity. This is of special concern when using an absolute @name annotation: if a type containing a @name annotation with an absolute pathname (i.e., one starting with a dot) is instantiated more than once, it will result in the same name referring to two controllable entities.

```
control noargs();
package top(noargs c1, noargs c2);

control c() {
    @name(".foo.bar") table t { /* body omitted */ }
    apply { /* body omitted */ }
}
top(c(), c()) main;
```

Without the @name annotation, this program would produce two controllable entities with fully-qualified names main.c1.t and main.c2.t. However, the @name(".foo.bar") annotation renames table t in both instances to foo.bar, resulting in one name that refers to two controllable entities, which is illegal.

The @name annotation of an element is located, and the control-plane name is derived from it, with:

$find_name_annotation_opt(annotationList)

1. Try:
    a. If annotationList matches pattern $_{\text{EMPTY}}$: return ·.
    b. Else if let annotation of type annotation be annotationList:
        i. If let '@' name `( annotationBody `) be annotation:
            A. If "name" is equal to the name of name: return annotation.
            B. Else: return ·.
2. Then, try:
    a. If let annotationListNonEmpty' of type annotationListNonEmpty be annotationList:
        i. If let annotationListNonEmpty annotation be annotationListNonEmpty':
            A. Try:
                I. If $find_name_annotation_opt(annotation) is none:
                    1. Return $find_name_annotation_opt(annotationListNonEmpty).
            B. Then, try:
                I. Let $\text{annotation}^{!?}$ be $find_name_annotation_opt(annotation).
                II. If $\text{annotation}^{!?}$ is defined:
                    1. Let $\text{annotation}_{\text{found}}$ be $\text{annotation}^{!?}$.
                    2. Return $\text{annotation}_{\text{found}}$.
3. Otherwise: return ·.

$name_annotation_opt(annotation)

1. If let '@' name `( annotationBody `) be annotation:
    a. If "name" is equal to the name of name:
        i. If let annotationBody' annotationToken be annotationBody:
            A. If annotationBody' is $_{\text{EMPTY}}$:
                I.

Let nameIR be annotationToken as text.

II. Let text' be the text nameIR with the suffix "{quot}" removed.

III. Let text be the text text' with the prefix "{quot}" removed.

IV. Return text.

2. Otherwise: return ·.

$name_annotation(annotationList)

1. Let annotation* be annotationList flattened.
2. For each annotation in annotation*:
   - Let nameIR$_{opt}^{?}$ be $name_annotation_opt(annotation).

   Let nameIR$_{opt}^{?*}$ be the resulting list.
3. Let nameIR$_{found}^{**}$ be $opt_as_seq_<nameIR>(nameIR$_{opt}^{?}$)*.
4. Return $concat_<nameIR>(nameIR$_{found}^{**}$).

$name_annotation_default(annotationList, nameIR)

1. Try:
   a. Let nameIR'* be $name_annotation(annotationList).
   b. If nameIR'* is a list of length 1:
      i. Let nameIR$_{found}$ be nameIR'*.
      ii. Return nameIR$_{found}$.
2. Then, try:
   a. Check that $name_annotation(annotationList) is an empty list.
   b. Return nameIR.

21.2.4. Concurrency control annotations

The @atomic annotation, described in Section 7.4.5 can be used to enforce the atomic execution of a code block.

21.2.5. Branch prediction annotations

The @likely and @unlikely annotation can be used on blocks that are used in if-else or switch statements to hint when a branch is likely or unlikely to be taken.

```
if (error_failure) @unlikely {
    /* code that is unlikely to execute as the condition is probably false */
}

switch(value) {
    1: @likely {
        /* code for the most common case */
       }
    2: { /* ... */ }
    default: @unlikely {
```

```
        /* code for rare case */
    }
}
```

These annotations have no effect on the semantics of the program; they are just hints to help an optimizer. A warning may be issued if a @likely block is found to never execute or an @unlikely block will always execute, but is not required.

21.2.6. Value set annotations

The @match annotation, described in Section 16.6, is used to specify a match_kind value other than the default match_kind of exact for a field of a value_set.

21.2.7. Extern function/method annotations

Various annotations may appear on extern function and method declarations to describe limitations on the behavior and interactions of those functions. By default extern functions might have any effect on the environment of the P4 program and might interact in non-trivial ways (subject to a few limitations—see [sec-calling-convention]). Since externs are architecture-specific and their behavior is known to the architecture definition, these annotations are not strictly necessary (an implementation can have knowledge of how externs interact based on their names built into it), but these annotations provide a uniform way of describing certain well-defined interactions (or their absence), allowing architecture-independent analysis of P4 programs.

- @pure - Describes a function that depends solely on its in parameter values, and has no effect other than returning a value, and copy-out behavior on its out and inout parameters. No hidden state is recorded between calls, and its value does not depend on any hidden state that may be changed by other calls. An example is a hash function that computes a deterministic hash of its arguments, and its return value does not depend upon any control-plane writable seed or initialization vector value. A @pure function whose results are unused may be safely eliminated with no adverse effects, and multiple calls with identical arguments may be combined into a single call (subject to the limits imposed by copy-out behavior of out and inout parameters). @pure functions may also be reordered with respect to other computations that are not data dependent.
- @noSideEffects - Weaker than @pure and describes a function that does not change any hidden state, but may depend on hidden state. One example is a hash function that computes a deterministic hash of its arguments, plus some internal state that can be modified via control plane API calls such as a seed or initialization vector. Another example is a read of one element of a register array extern object. Such a function may be dead code eliminated, and may be reordered or combined with other @noSideEffects or @pure calls (subject to the limits imposed by copy-out behavior of out and inout parameters), but not with other function calls that may have side effects that affect the function.

21.2.8. Deprecated annotation

The deprecated annotation has a required string argument that is a message that will be printed by a compiler when a program is using the deprecated construct. This is mostly useful for annotating library constructs, such as externs. The parameter must be a local compile-time known value of type string.

```
#define DEPR_V1_2_2 "Deprecated in v1.2.2"
@deprecated("Please use the 'check' function instead." ++ DEPR_V1_2_2)
extern Checker {
   /* body omitted */
}
```

21.2.9. No warnings annotation

The noWarn annotation has a required string argument that indicates a compiler warning that will be inhibited. For example @noWarn("unused") on a declaration will prevent a compiler warning if that declaration is not used. The parameter must be a local compile-time known value of type string. === Target-specific annotations

Each P4 compiler implementation can define additional annotations specific to the target of the compiler. The syntax of the annotations should conform to the above description. The semantics of such annotations is target-specific. They could be used in a similar way to pragmas in other languages.

The P4 compiler should provide:

- Errors when annotations are used incorrectly (e.g., an annotation expecting a parameter but used without arguments, or with arguments of the wrong type)
- Warnings for unknown annotations.

Appendix A: Revision History

A.1. Summary of changes made in unreleased version

- Clarified that numeric priorities cannot be assigned to entries of a table that has const entries ([sec-entries]).
- Clarified that switch statements are allowed in action and function bodies, and that switch statements with action_run expressions are only allowed in control apply blocks ([sec-stmts] and [sec-switch-stmt]).
- Added standard branch prediction annotations (Section 21.2.5)
- Added support for compile-time string concatenation using ++ operator ([sec-string-type] and [sec-string-ops]).
- Added compound assignment statements ([sec-assignment])
- Update the specification grammar for for loops to match the grammar used in the p4c implementation ([sec-loop-stmt]).

A.2. Summary of changes made in version 1.2.5, released October 11, 2024

- Improved type nesting rules ([sec-type-nesting]).
- Clarified that directionless extern parameters are passed by reference.
- Introduced distinction between local compile-time known and compile-time known values (Section 7.5).

A.3. Summary of changes made in version 1.2.4,released May 15, 2023

- Added header stack expressions ([sec-hs-init]).
- Allow casts from a type to itself ([sec-casts]).
- Added an invalid header or header union expression {#} ([sec-ops-on-hdrs] and [sec-expr-hu]).

- Added a concept of numeric values ([sec-numeric-values]).
- Added a section on operations on extern objects ([sec-extern-operations]).
- Added note in sections operations on types for types that support compile-time size determination.
- Clarified that header stacks are arrays of headers or header unions.
- Added distinctness of fields for types that have fields including error, match kind, struct, header, and header union.
- Clarified types bit<W>, int<W>, and varbit<W> encompass the case where the width is a compile-time known expression evaluating to an appropriate integer ([sec-unsigned-integers], [sec-signed-integers], [sec-dynamically-sized-integers]).
- Clarified restrictions for parameters with default values ([sec-calling-convention]).
- Added optional trailing commas ([sec-trailing-commas]).
- Clarified the scope of parser namespaces ([sec-parser-decl]).
- Specified that algorithm for generating control-plane names for keys is optional ([sec-cp-keys]).
- Clarified types of expressions that may appear in select ([sec-select]).
- Added description of semantics of the core.p4 match kinds ([sec-table-keys]).
- Explicitly disallow overloading of parsers, controls, and packages ([sec-extern-objects]).
- Clarified implicit casts present in select expressions ([sec-select]).
- Clarified that slices can be applied to arbitrary-precision integers ([sec-varint-ops]).
- Clarified that direct invocation is not possible for objects that have constructor arguments ([sec-direct-invocation]).
- Added comparison for tuples as a legal operation ([sec-tuple-exprs]).
- Clarified the behavior of lookahead on header-typed values (Section 16.8.3).
- Added static_assert function ([sec-static-assert]).
- Clarified semantics of ranges where the start is bigger than the end ([sec-ranges]).
- Allow ranges to be specified by serializable enums ([sec-ranges]).
- Specified type produced by the *sizeInB* methods ([sec-minsizeinbits]).
- Added section with operations on match_kind values ([sec-match-kind-exprs]).
- Renamed infinite-precision integers to arbitrary-precision integers ([sec-arbitrary-precision-integers]).
- compiler-inserted default_action is not const ([sec-tables]).
- Clarified the restrictions on run time for tables with const entries ([sec-entries]).
- renamed list expressions to tuple expressions
- Added list type ([sec-list-types]).
- Defined entries table property without const, for entries installed when the P4 program is loaded, but the control plane can later change them or add to them ([sec-entries]).
- Clarified behavior of table with no key property, or if its list of keys is empty ([sec-table-keys]).

A.4. Summary of changes made in version 1.2.3, released July 11, 2022.

- Extended minSizeInBits and minSizeInBytes to apply to more expressions ([sec-minsizeinbits]).

- Added support for maxSizeInBits and maxSizeInBytes ([sec-minsizeinbits]).
- Added support for empty lists of const entries in tables ([sec-entries]).
- Added support for switch statements in actions ([sec-actions]).
- Added support for direct invocation of controls and parsers ([sec-parameterization]).
- Added parser value_set to list of control-plane visible names ([sec-cp-names]).
- Added match_kind as a base type ([sec-match-kind-type]).
- Removed structure initializers as they are subsumed by structure-valued expressions ([sec-structure-expressions]).
- Specified operations on values typed as type variables ([sec-type-variable-operations]).
- Clarified semantics of compile-time known values (Section 7.5).
- Clarified semantics of directionless parameters ([sec-calling-convention]).
- Clarified semantics of arbitrary precision integers ([sec-arbitrary-precision-integers]).
- Clarified semantics of bit slices, shifts, and concatenation ([sec-bit-ops]).
- Clarified semantics of optional parameters ([sec-optional-parameters]).
- Clarified restrictions on extern method and function invocation (Appendix E).
- Clarified semantics of implicit casts ([sec-implicit-casts]).

A.5. Summary of changes made in version 1.2.2, released May 17, 2021

- Added support for accessing tuple fields ([sec-tuple-exprs]).
- Added support for generic structures ([sec-type-spec]).
- Added support for integers, enums, and errors in switch statements ([sec-switch-stmt]).
- Added support for additional enumeration types ([sec-enum-types]).
- Added support for abstract methods ([sec-abstract-methods]).
- Added support for conditional statements and empty statements in parsers ([sec-parser-state-stmt]).
- Added support for casts from int to bool ([sec-casts]).
- Added support for 0-width bitstrings and varbits ([sec-uninitialized-values-and-writing-invalid-headers]).
- Clarified that default_action is NoAction if otherwise unspecified ([sec-tables]).
- Clarified the types of expressions that may be used as indexes for header stacks ([sec-expr-hs]).
- Clarified representation of Booleans in headers ([sec-header-types]).
- Clarified representation of empty types ([sec-uninitialized-values-and-writing-invalid-headers]).
- Clarified that action data can be specified by the control plane, default_action table property, or const entries table property ([sec-actions]).
- Fixed several typos and inconsistencies in grammar (Appendix F).
- Eliminated annotations on const entries in grammar (Appendix F).

A.6. Summary of changes made in version 1.2.1, released June 11, 2020

- Added structure-value expressions ([sec-structure-expressions]).
- Added support for default values ([sec-default-values]).
- Added support for concatenating signed strings ([sec-concatenation]).
- Added key-value and list-structured annotations ([sec-annotations]).
- Added @pure and @noSideEffects annotations (Section 21.2.7).
- Added @noWarn annotation (Section 21.2.9).
- Generalized typing for masks to allow serializable enums ([sec-cubes]).
- Restricted the right operands of bit shifts involving arbitrary-precision integers to be constant and positive ([sec-varint-ops]).
- Clarified copy-out behavior for return ([sec-return-stmt]) and exit ([sec-exit-stmt]) statements.
- Clarified semantics of invalid header stacks ([sec-uninitialized-values-and-writing-invalid-headers]).
- Clarified initialization semantics ([sec-lvalues] and [sec-calling-convention]), especially for headers and local variables.
- Clarified evaluation order for table keys ([sec-mau-semantics]).
- Fixed grammar to clarify parsing of right shift operator (>>), allow empty statements in parser ([sec-parser-state-stmt]), and eliminate annotations on const entries ([sec-entries]).

A.7. Summary of changes made in version 1.2.0, released October 14, 2019

- Added table.apply().miss ([sec-invoke-mau]).
- Added string type ([sec-string-type]).
- Added implicit casts from enum values ([sec-enum-exprs]).
- Allow 1-bit signed values
- Define the type of bit slices from signed and unsigned values to be unsigned.
- Constrain default label position for switch statements.
- Allow empty tuples.
- Added @deprecated annotation.
- Relaxed the structure of annotation bodies.
- Removed the @pkginfo annotation, which is now defined by the P4Runtime specification.
- Added int type ([sec-arbitrary-precision-integers]).
- Added error ParserInvalidArgument ([sec-packet-extract-two], Section 16.8.4).
- Clarified the significance of order of entries in const entries ([sec-entries]).
- Added methods to calculate header size ([sec-ops-on-hdrs]).

A.8. Summary of changes made in version 1.1.0, released November 26, 2017.

- Top-level functions ([sec-functions])
  - Functions may be declared at the top-level of a P4 program.
- Optional and named parameters ([sec-calling-convention])
  - Parameters may be specified by name, with a default value, or designated as optional.
- `enum` representations ([sec-enum-exprs])
  - `enum` values to be specified with a concrete representation.
- Parser values sets ([sec-value-set])
  - `value_set` objects for control-plane programmable `select` labels.
- Type definitions (Section 8.5.8)
  - New types may be introduced in programs.
- Saturating arithmetic ([sec-bit-ops])
  - Saturating arithmetic is supported on some targets.
- Structured annotations ([sec-annotations])
  - Annotations may be specified as lists of key-value pairs
- Globalname ([sec-name-annotations])
  - The reserved `globalname` annotation has been removed.
- Table `size` property ([sec-size-table-property])
  - Meaning of optional `size` property for tables has been defined.
- Invalid headers ([sec-ops-on-hdrs])
  - Clarified semantics of operations on invalid headers.
- Calling restrictions (Appendix E)
  - Added restrictions on kinds of values that may be passed as arguments to calls.
- Bitwise operator precedence (Appendix F)
  - Modified precedence conventions so that bitwise operators `&` `|` and `^` have higher precedence than relation operators `<` `>` `<=` `>=`.
- Computed bitwidths ([sec-base-types])
  - Added support for specifying widths using expressions in `bit` and `varbit` types.

A.9. Initial version 1.0.0, released May 17, 2017

Appendix B: P4 reserved keywords

The following table shows all P4 reserved keywords. Some identifiers are treated as keywords only in specific contexts (e.g., the keyword `actions`).

abstract	action	apply	bit
bool	const	control	default

else	enum	error	extern
exit	false	header	header_union
if	in	inout	int
list	match_kind	package	parser
out	return	select	state
string	struct	switch	table
this	transition	true	tuple
type	typedef	value_set	varbit
verify	void		

Appendix C: P4 reserved annotations

The following table shows all P4 reserved annotations.

Annotation	Purpose	See Section
atomic	specify atomic execution	[sec-concurrency]
defaultonly	action can only appear in the default action	[sec-annotations]
hidden	hides a controllable entity from the control plane	[sec-name-annotations]
match	specify match_kind of a field in a value_set	[sec-value-set]
name	assign local control-plane name	[sec-name-annotations]
optional	parameter is optional	[sec-optional-parameters]
tableonly	action cannot be a default_action	[sec-annotations]
deprecated	Construct has been deprecated	Section 21.2.8
pure	pure function	Section 21.2.7
noSideEffects	function with no side effects	Section 21.2.7
noWarn	Has a string argument; inhibits compiler warnings	Section 21.2.9

Appendix D: P4 core library

The P4 core library contains declarations that are useful to most programs.

For example, the core library includes the declarations of the predefined packet_in and packet_out extern objects, used in parsers and deparsers to access packet data.

```
/// Standard error codes.  New error codes can be declared by users.
error {
    NoError,           /// No error.
    PacketTooShort,    /// Not enough bits in packet for 'extract'.
    NoMatch,           /// 'select' expression has no matches.
    StackOutOfBounds,  /// Reference to invalid element of a header stack.
    HeaderTooShort,    /// Extracting too many bits into a varbit field.
    ParserTimeout,     /// Parser execution time limit exceeded.
```

```
    ParserInvalidArgument  /// Parser operation was called with a value
                           /// not supported by the implementation.
}
extern packet_in {
    /// Read a header from the packet into a fixed-sized header @hdr
    /// and advance the cursor.
    /// May trigger error PacketTooShort or StackOutOfBounds.
    /// @T must be a fixed-size header type
    void extract<T>(out T hdr);
    /// Read bits from the packet into a variable-sized header @variableSizeHeader
    /// and advance the cursor.
    /// @T must be a header containing exactly 1 varbit field.
    /// May trigger errors PacketTooShort, StackOutOfBounds, or HeaderTooShort.
    void extract<T>(out T variableSizeHeader,
                    in bit<32> variableFieldSizeInBits);
    /// Read bits from the packet without advancing the cursor.
    /// @returns: the bits read from the packet.
    /// T may be an arbitrary fixed-size type.
    T lookahead<T>();
    /// Advance the packet cursor by the specified number of bits.
    void advance(in bit<32> sizeInBits);
    /// @return packet length in bytes.  This method may be unavailable on
    /// some target architectures.
    bit<32> length();
}
extern packet_out {
    /// Write @data into the output packet, skipping invalid headers
    /// and advancing the cursor
    /// @T can be a header type, a header stack, a header_union, or a struct
    /// containing fields with such types.
    void emit<T>(in T data);
}
action NoAction() {}
/// Standard match kinds for table key fields.
/// Some architectures may not support all these match kinds.
/// Architectures can declare additional match kinds.
match_kind {
    /// Match bits exactly.
    exact,
    /// Ternary match, using a mask.
    ternary,
    /// Longest-prefix match.
    lpm
}

/// Static assert evaluates a boolean expression
/// at compilation time.  If the expression evaluates to
/// false, compilation is stopped and the corresponding message is printed.
/// The function returns a boolean, so that it can be used
/// as a global constant value in a program, e.g.:
/// const version = static_assert(V1MODEL_VERSION > 20180000, "Expected a v1 model
version >= 20180000");
extern bool static_assert(bool check, string message);

/// Like the above but using a default message.
extern bool static_assert(bool check);
```

Appendix E: Restrictions on compile time and run time calls

This appendix summarizes restrictions on compile time and run time calls that can be made.

Some guiding principles:

- Controls are not allowed to call parsers, and vice versa, so there is no use in passing one type to the other in constructor parameters or run-time parameters.
- At run time, after a control is called, and before that call is complete, there can be no recursive calls between controls, nor from a control to itself. Similarly for parsers. There can be loops among states within a single parser.
- Externs are not allowed to call parsers or controls, so there is no use in passing objects of those types to them.
- Tables are always instantiated directly in their enclosing control, and cannot be instantiated at the top level. There is no syntax for specifying parameters that are tables. Tables are only intended to be used from within the control where they are defined.
- Value-sets can be instantiated in an enclosing parser or at the top level. There is no syntax for specifying parameters that are value-sets. Value-sets can be shared between the parsers as long as they are in the scope.

A note on recursion: It is expected that some architectures will define capabilities for recirculating a packet to be processed again as if it were a newly arriving packet, or to make "clones" of packets that are then processed by parsers and/or control blocks that the original packet has already completed. This does not change the notes above on recursion that apply while a parser or control is executing.

E.1. Restrictions on run time calls

This section lists restrictions on what kinds of calls can be made from which places in a P4 program. Calling a parser, control, or table means invoking its apply() method.

One way that an extern can be called from the top level of a parser or control is in an initializer expression for a declared variable, e.g. bit<32> x = rand.get();. Here, the top level of a parser or control means either within a parser declaration, but outside of any of its parser state declarations, or within a control declaration, but outside its apply { ... } block.

There may not be any recursion in calls, neither by a thing calling itself directly, nor mutual recursion.

An extern can never cause any other type of P4 program object to be called. See Section 19.6.

Note that particular externs may have additional restrictions not listed in this table. Any such restrictions should be documented in the description for each extern, as part of the documentation for the architecture that defines the extern.

In many cases, the restriction will be "from a parser state only" or "from a control apply block or action only", but it may be even more restrictive, e.g. only from a particular kind of control block instantiated in a particular role in an architecture.

Actions may be called directly from a control apply block. An action is callable only in the following contexts:

$callable_action(p, TC)

1. If p matches pattern GLOBAL: return false.
2. Else if p matches pattern BLOCK: return false.
3. Else:
    a. Let localKind be TC.LOCAL.KIND.
    b. Return localKind is equal to ACTION or localKind is equal to CONTROL_APPLY_METHOD.

A function is callable only in the following contexts:

$callable_definedFunction(p, TC)

1. If p matches pattern GLOBAL: return false.
2. Else if p matches pattern BLOCK:
    a. Let blockKind be TC.BLOCK.KIND.
    b. Return blockKind is equal to PARSER or blockKind is equal to CONTROL.
3. Else:
    a. Let localKind be TC.LOCAL.KIND.
    b. Return localKind does not have type externMethodLocalKind.

An extern function is callable only in the following contexts:

$callable_externFunction(p, TC, nameIR)

1. If p matches pattern GLOBAL:
    a. If nameIR is equal to "verify": return false.
    b. Else: return true.
2. Else if p matches pattern BLOCK:
    a. If nameIR is equal to "verify":
        i. Let blockKind be TC.BLOCK.KIND.
        ii. Return blockKind is equal to PARSER.
    b. Else:
        i. Let blockKind be TC.BLOCK.KIND.
        ii. Return blockKind is equal to PARSER or blockKind is equal to CONTROL.
3. Else:
    a. If nameIR is equal to "verify":
        i. Let localKind be TC.LOCAL.KIND.
        ii. Return localKind is equal to PARSER_STATE.
    b. Else: return true.

An extern method is callable only in the following contexts:

$callable_externMethod(p, TC)

1. If p matches pattern GLOBAL: return false.
2. Else if p matches pattern BLOCK:
    a. Let blockKind be TC.BLOCK.KIND.
    b. Return blockKind is equal to PARSER or blockKind is equal to CONTROL.
3. Else: return true.

A parser apply method is callable only in the following contexts:

$callable_parserApplyMethod(p, TC)

1. If p matches pattern GLOBAL: return false.
2. Else if p matches pattern BLOCK: return false.
3. Else:
    a. Let localKind be TC.LOCAL.KIND.
    b. Return localKind is equal to PARSER_STATE.

A control apply method is callable only in the following contexts:

$callable_controlApplyMethod(p, TC)

1. If p matches pattern GLOBAL: return false.
2. Else if p matches pattern BLOCK: return false.
3. Else:
    a. Let localKind be TC.LOCAL.KIND.
    b. Return localKind is equal to CONTROL_APPLY_METHOD.

A table apply method is callable only in the following contexts:

$callable_tableApplyMethod(p, TC)

1. If p matches pattern GLOBAL: return false.
2. Else if p matches pattern BLOCK: return false.
3. Else:
    a. Let localKind be TC.LOCAL.KIND.
    b. Return localKind is equal to CONTROL_APPLY_METHOD or localKind is equal to TABLE_APPLY_METHOD.

E.2. Restrictions on instantiations

This section lists restrictions on where one may perform instantiations (see Section 7.3) of different types.

You may declare variables and constants of any of the value types within a parser, control, or function (see Section 11.2 for more details). Declaring a variable or constant is not the same as instantiation. Variables may not be declared at the top level of your program, but constants may.

Instantiation of a type dispatches to the rule for its object kind:

$instantiable(p, TC, instctxt, typeIR)

1. Let $typeIR_{unroll}$ be typeIR with typedefs unrolled.
2. If $typeIR_{unroll}$ has type externObjectTypeIR:
    a. Return $instantiable_extern(p, TC, instctxt).
3. Else if $typeIR_{unroll}$ has type parserObjectTypeIR:
    a. Return $instantiable_parser(p, TC, instctxt).
4. Else if $typeIR_{unroll}$ has type controlObjectTypeIR:
    a. Return $instantiable_control(p, TC, instctxt).

5. Else if $typeIR_{unroll}$ has type packageObjectTypeIR:
    a. Return $instantiable_package(p, TC, instctxt).
6. Else if $typeIR_{unroll}$ has type tableObjectTypeIR:
    a. Return $instantiable_table(p, TC, instctxt).

For externs, one can only specify their interface in $P4_{16}$, not their implementation. Thus there is no place to instantiate objects within an extern. An extern object is instantiable only in the following contexts:

$instantiable_extern(p, TC, instctxt)

1. If p matches pattern GLOBAL: return true.
2. Else if p matches pattern BLOCK: return true.
3. Else: return true.

A parser object is instantiable only in the following contexts:

$instantiable_parser(p, TC, instctxt)

1. If p matches pattern GLOBAL: return instctxt is equal to ANON.
2. Else if p matches pattern BLOCK:
    a. Let blockKind be TC.BLOCK.KIND.
    b. Return blockKind is equal to PARSER or blockKind is equal to PACKAGE.
3. Else:
    a. Let localKind be TC.LOCAL.KIND.
    b. Return localKind is equal to PARSER_STATE.

A control object is instantiable only in the following contexts:

$instantiable_control(p, TC, instctxt)

1. If p matches pattern GLOBAL: return instctxt is equal to ANON.
2. Else if p matches pattern BLOCK:
    a. Let blockKind be TC.BLOCK.KIND.
    b. Return blockKind is equal to CONTROL or blockKind is equal to PACKAGE.
3. Else:
    a. Let localKind be TC.LOCAL.KIND.
    b. Return localKind is equal to CONTROL_APPLY_METHOD.

The answer for package is always "no" because there is no "inside a package" where instantiations can be written in $P4_{16}$. One can definitely make constructor calls and use instances of stateful types as parameters when instantiating a package. A package object is instantiable only in the following contexts:

$instantiable_package(p, TC, instctxt)

1. If p matches pattern GLOBAL: return true.

2. Else if p matches pattern BLOCK:
    a. Let blockKind be TC.BLOCK.KIND.
    b. Return blockKind is equal to PACKAGE.
3. Else: return false.

A table object is instantiable only in the following contexts:

$instantiable_table(p, TC, instctxt)

1. If p matches pattern GLOBAL: return false.
2. Else if p matches pattern BLOCK:
    a. Let blockKind be TC.BLOCK.KIND.
    b. Return blockKind is equal to CONTROL.
3. Else: return false.

Appendix F: P4 grammar

This is the grammar of $P4_{16}$ written using the YACC/bison language. Absent from this grammar is the precedence of various operations.

The grammar is actually ambiguous, so the lexer and the parser must collaborate for parsing the language. In particular, the lexer must be able to distinguish two kinds of identifiers:

- Type names previously introduced (TYPE_IDENTIFIER tokens)
- Regular identifiers (IDENTIFIER token)

The parser has to use a symbol table to indicate to the lexer how to parse subsequent appearances of identifiers. For example, given the following program fragment:

```
typedef bit<4> t;
struct s { /* body omitted */}
t x;
parser p(bit<8> b) { /* body omitted */ }
```

The lexer has to return the following terminal kinds:

```
t - TYPE_IDENTIFIER
s - TYPE_IDENTIFIER
x - IDENTIFIER
p - TYPE_IDENTIFIER
b - IDENTIFIER
```

This grammar has been heavily influenced by limitations of the Bison parser generator tool.

The STRING_LITERAL token corresponds to a string literal enclosed within double quotes, as described in Section 6.2.2.3.

All other terminals are uppercase spellings of the corresponding keywords. For example, RETURN is the terminal returned by the lexer when parsing the keyword return.

```
trailingCommaOpt
    : /* empty */ | ,
    ;

booleanLiteral
    : TRUE | FALSE
    ;

integerLiteral
    : D int | nat W int | nat S int
    ;

stringLiteral
    : " text "
    ;

identifier
    : _ID text
    ;

typeIdentifier
    : _TID text
    ;

nonTypeName
    : identifier | APPLY | KEY | ACTIONS | STATE | ENTRIES | TYPE | PRIORITY
    ;

prefixedNonTypeName
    : nonTypeName | _ID . nonTypeName
    ;

typeName = typeIdentifier

prefixedTypeName
    : typeName | _TID . typeName
```

```
    ;

tableCustomName
    : identifier | typeIdentifier | APPLY | STATE | TYPE | PRIORITY
    ;

name
    : nonTypeName | typeName | LIST
    ;

nameList
    : name | nameList , name
    ;

member = name

direction
    : /* empty */ | IN | OUT | INOUT
    ;

baseType
    : BOOL | ERROR | MATCH_KIND | STRING | INT | INT < int >
    | INT < ( expression ) > | BIT | BIT < int > | BIT < ( expression ) >
    | VARBIT < int > | VARBIT < ( expression ) >
    ;

specializedType
    : prefixedTypeName < typeArgumentList >
    ;

namedType
    : prefixedTypeName | specializedType
    ;

listType
    : LIST < typeArgument >
    ;

tupleType
    : TUPLE < typeArgumentList >
    ;

type
    : baseType | namedType | arrayType | listType | tupleType
    ;

typeOrVoid
    : type | VOID | identifier
    ;

typeParameter = name

typeParameterList
    : typeParameter | typeParameterList , typeParameter
    ;

typeParameterListOpt
    : /* empty */ | < typeParameterList >
    ;

parameter
    : annotationList direction type name initializerOpt
    ;

nonEmptyParameterList
    : parameter | nonEmptyParameterList , parameter
    ;

parameterList
```

```
    : /* empty */ | nonEmptyParameterList
    ;

constructorParameter = parameter

constructorParameterList = parameterList

constructorParameterListOpt
    : /* empty */ | ( parameterList )
    ;

namedExpression
    : name = expression
    ;

namedExpressionList
    : namedExpression | namedExpressionList , namedExpression
    ;

literalExpression
    : booleanLiteral | integerLiteral | stringLiteral
    ;

referenceExpression
    : prefixedNonTypeName | THIS
    ;

defaultExpression
    : ...
    ;

unop
    : ! | ~ | - | +
    ;

unaryExpression
    : unop expression
    ;

binop
    : * | / | % | + | - | |+| | |-| | << | >> | <= | >= | < | > | != | == | &
    | ^ | | | ++ | && | ||
    ;

binaryExpression
    : expression binop expression
    ;

binaryExpressionNonBrace
    : expressionNonBrace binop expression
    ;

ternaryExpression
    : expression ? expression : expression
    ;

ternaryExpressionNonBrace
    : expressionNonBrace ? expression : expression
    ;

castExpression
    : ( type ) expression
    ;

dataExpression
    : invalidHeaderExpression | sequenceOrRecordExpression
    ;

errorAccessExpression
```

```
    : ERROR . member
    ;

memberAccessExpression
    : memberAccessBase . member
    ;

indexAccessExpression
    : expression [ expression ]
    ;

sliceop
    : : | +:
    ;

sliceAccessExpression
    : expression [ expression sliceop expression ]
    ;

accessExpression
    : errorAccessExpression | memberAccessExpression | indexAccessExpression
    | sliceAccessExpression
    ;

memberAccessExpressionNonBrace
    : memberAccessBaseNonBrace . member
    ;

indexAccessExpressionNonBrace
    : expressionNonBrace [ expression ]
    ;

sliceAccessExpressionNonBrace
    : expressionNonBrace [ expression sliceop expression ]
    ;

accessExpressionNonBrace
    : errorAccessExpression | memberAccessExpressionNonBrace
    | indexAccessExpressionNonBrace | sliceAccessExpressionNonBrace
    ;

callExpression
    : callTarget ( argumentList )
    | callableTarget < realTypeArgumentList > ( argumentList )
    ;

callExpressionNonBrace
    : callTargetNonBrace ( argumentList )
    | callableTargetNonBrace < realTypeArgumentList > ( argumentList )
    ;

parenthesizedExpression
    : ( expression )
    ;

expression
    : literalExpression | referenceExpression | defaultExpression
    | unaryExpression | binaryExpression | ternaryExpression | castExpression
    | dataExpression | accessExpression | callExpression
    | parenthesizedExpression
    ;

expressionList
    : /* empty */ | expression | expressionList , expression
    ;

memberAccessBase
    : prefixedTypeName | expression
    ;
```

```
sequenceElementExpression = expressionList

recordElementExpression
    : name = expression | name = expression , ...
    | name = expression , namedExpressionList
    | name = expression , namedExpressionList , ...
    ;

sequenceOrRecordElementExpression
    : sequenceElementExpression | recordElementExpression
    ;

callableTarget = expression

constructorTarget = namedType

callTarget
    : callableTarget | constructorTarget
    ;

expressionNonBrace
    : literalExpression | referenceExpression | unaryExpression
    | binaryExpressionNonBrace | ternaryExpressionNonBrace | castExpression
    | accessExpressionNonBrace | callExpressionNonBrace
    | parenthesizedExpression
    ;

memberAccessBaseNonBrace
    : prefixedTypeName | expressionNonBrace
    ;

callableTargetNonBrace = expressionNonBrace

callTargetNonBrace
    : callableTargetNonBrace | constructorTarget
    ;

simpleKeysetExpression
    : expression | expression &&& expression | expression .. expression
    | DEFAULT | _
    ;

simpleKeysetExpressionList
    : simpleKeysetExpression
    | simpleKeysetExpressionList , simpleKeysetExpression
    ;

tupleKeysetExpression
    : ( expression &&& expression ) | ( expression .. expression ) | ( DEFAULT )
    | ( _ ) | ( simpleKeysetExpression , simpleKeysetExpressionList )
    ;

keysetExpression
    : simpleKeysetExpression | tupleKeysetExpression
    ;

realTypeArgument
    : type | VOID | _
    ;

realTypeArgumentList
    : realTypeArgument | realTypeArgumentList , realTypeArgument
    ;

typeArgument
    : realTypeArgument | nonTypeName
    ;
```

```
typeArgumentList
    : /* empty */ | typeArgument | typeArgumentList , typeArgument
    ;

argument
    : expression | name = expression | name = _ | _
    ;

argumentListNonEmpty
    : argument | argumentListNonEmpty , argument
    ;

argumentList
    : /* empty */ | argumentListNonEmpty
    ;

lvalue
    : referenceExpression | lvalue . member | lvalue [ expression ]
    | lvalue [ expression sliceop expression ] | ( lvalue )
    ;

emptyStatement
    : ;
    ;

assignmentStatement
    : lvalue assignop expression ;
    ;

callStatement
    : lvalue ( argumentList ) ; | lvalue < typeArgumentList > ( argumentList ) ;
    ;

directApplicationStatement
    : namedType . APPLY ( argumentList ) ;
    ;

returnStatement
    : RETURN ; | RETURN expression ;
    ;

exitStatement
    : EXIT ;
    ;

blockStatement
    : annotationList { blockElementStatementList }
    ;

conditionalStatement
    : IF ( expression ) statement | IF ( expression ) statement ELSE statement
    ;

forUpdateStatement
    : lvalue ( argumentList ) | lvalue < typeArgumentList > ( argumentList )
    | lvalue assignop expression
    ;

forUpdateStatementListNonEmpty
    : forUpdateStatement | forUpdateStatementListNonEmpty , forUpdateStatement
    ;

forUpdateStatementList
    : /* empty */ | forUpdateStatementListNonEmpty
    ;

forInitStatement
    : annotationList type name initializerOpt | forUpdateStatement
    ;
```

```
forInitStatementListNonEmpty
    : forInitStatement | forInitStatementListNonEmpty , forInitStatement
    ;

forInitStatementList
    : /* empty */ | forInitStatementListNonEmpty
    ;

forCollectionExpression
    : expression | expression .. expression
    ;

forStatement
    : annotationList FOR
        ( forInitStatementList ; expression ; forUpdateStatementList ) statement
    | annotationList FOR ( type name IN forCollectionExpression ) statement
    | annotationList FOR
        ( annotationListNonEmpty type name IN forCollectionExpression )
        statement
    ;

switchLabel
    : DEFAULT | expressionNonBrace
    ;

switchCase
    : switchLabel : blockStatement | switchLabel :
    ;

switchCaseList
    : /* empty */ | switchCaseList switchCase
    ;

switchStatement
    : SWITCH ( expression ) { switchCaseList }
    ;

breakStatement
    : BREAK ;
    ;

continueStatement
    : CONTINUE ;
    ;

statement
    : emptyStatement | assignmentStatement | callStatement
    | directApplicationStatement | returnStatement | exitStatement
    | blockStatement | conditionalStatement | forStatement | breakStatement
    | continueStatement | switchStatement
    ;

initializer
    : = expression
    ;

constantDeclaration
    : annotationList CONST type name initializer ;
    ;

initializerOpt
    : /* empty */ | initializer
    ;

variableDeclaration
    : annotationList type name initializerOpt ;
    ;
```

```
blockElementStatement
    : constantDeclaration | variableDeclaration | statement
    ;

blockElementStatementList
    : /* empty */ | blockElementStatementList blockElementStatement
    ;

functionPrototype
    : typeOrVoid name typeParameterListOpt ( parameterList )
    ;

functionDeclaration
    : annotationList functionPrototype blockStatement
    ;

actionDeclaration
    : annotationList ACTION name ( parameterList ) blockStatement
    ;

objectInitializer
    : = { objectDeclarationList }
    ;

instantiation
    : annotationList type ( argumentList ) name ;
    | annotationList type ( argumentList ) name objectInitializer ;
    ;

objectDeclaration
    : functionDeclaration | instantiation
    ;

objectDeclarationList
    : /* empty */ | objectDeclarationList objectDeclaration
    ;

errorDeclaration
    : ERROR { nameList }
    ;

matchKindDeclaration
    : MATCH_KIND { nameList trailingCommaOpt }
    ;

enumTypeDeclaration
    : annotationList ENUM name { nameList trailingCommaOpt }
    | annotationList ENUM type name { namedExpressionList trailingCommaOpt }
    ;

typeField
    : annotationList type name ;
    ;

typeFieldList
    : /* empty */ | typeFieldList typeField
    ;

structTypeDeclaration
    : annotationList STRUCT name typeParameterListOpt { typeFieldList }
    ;

headerTypeDeclaration
    : annotationList HEADER name typeParameterListOpt { typeFieldList }
    ;

headerUnionTypeDeclaration
    : annotationList HEADER_UNION name typeParameterListOpt { typeFieldList }
    ;
```

```
derivedTypeDeclaration
    : enumTypeDeclaration | structTypeDeclaration | headerTypeDeclaration
    | headerUnionTypeDeclaration
    ;

typedef
    : type | derivedTypeDeclaration
    ;

typedefDeclaration
    : annotationList TYPEDEF typedef name ; | annotationList TYPE type name ;
    ;

externFunctionDeclaration
    : annotationList EXTERN functionPrototype ;
    ;

externConstructorPrototype
    : annotationList typeIdentifier ( parameterList ) ;
    ;

externMethodPrototype
    : annotationList functionPrototype ;
    | annotationList ABSTRACT functionPrototype ;
    ;

externConstructorOrMethodPrototype
    : externConstructorPrototype | externMethodPrototype
    ;

externConstructorOrMethodPrototypeList
    : /* empty */
    | externConstructorOrMethodPrototypeList externConstructorOrMethodPrototype
    ;

externObjectDeclaration
    : annotationList EXTERN nonTypeName typeParameterListOpt
        { externConstructorOrMethodPrototypeList }
    ;

externDeclaration
    : externFunctionDeclaration | externObjectDeclaration
    ;

selectCase
    : keysetExpression : name ;
    ;

selectCaseList
    : /* empty */ | selectCaseList selectCase
    ;

selectExpression
    : SELECT ( expressionList ) { selectCaseList }
    ;

stateExpression
    : name ; | selectExpression
    ;

transitionStatement
    : /* empty */ | TRANSITION stateExpression
    ;

valueSetType
    : baseType | tupleType | prefixedTypeName
    ;
```

```
valueSetDeclaration
    : annotationList VALUE_SET < valueSetType > ( expression ) name ;
    ;

parserTypeDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList ) ;
    ;

parserBlockStatement
    : annotationList { parserBlockElementStatementList }
    ;

parserBlockElementStatement
    : constantDeclaration | variableDeclaration | parserStatement
    ;

parserBlockElementStatementList
    : /* empty */ | parserBlockElementStatementList parserBlockElementStatement
    ;

parserConditionalStatement
    : IF ( expression ) parserStatement
    | IF ( expression ) parserStatement ELSE parserStatement
    ;

parserStatement
    : emptyStatement | assignmentStatement | callStatement
    | directApplicationStatement | parserBlockStatement
    | parserConditionalStatement
    ;

parserState
    : annotationList STATE name
        { parserBlockElementStatementList transitionStatement }
    ;

parserStateList
    : parserState | parserStateList parserState
    ;

parserLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | valueSetDeclaration
    ;

parserLocalDeclarationList
    : /* empty */ | parserLocalDeclarationList parserLocalDeclaration
    ;

parserDeclaration
    : annotationList PARSER name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { parserLocalDeclarationList parserStateList }
    ;

const
    : CONST
    ;

constOpt
    : /* empty */ | const
    ;

tableKey
    : expression : name annotationList ;
    ;

tableKeyList
    : /* empty */ | tableKeyList tableKey
```

```
    ;

tableActionReference
    : prefixedNonTypeName | prefixedNonTypeName ( argumentList )
    ;

tableAction
    : annotationList tableActionReference ;
    ;

tableActionList
    : /* empty */ | tableActionList tableAction
    ;

tableEntryPriority
    : PRIORITY = integerLiteral : | PRIORITY = ( expression ) :
    ;

tableEntry
    : constOpt tableEntryPriority keysetExpression : tableActionReference
        annotationList ;
    | constOpt keysetExpression : tableActionReference annotationList ;
    ;

tableEntryList
    : /* empty */ | tableEntryList tableEntry
    ;

tableProperty
    : KEY = { tableKeyList } | ACTIONS = { tableActionList }
    | annotationList constOpt ENTRIES = { tableEntryList }
    | annotationList constOpt tableCustomName initializer ;
    ;

tablePropertyList
    : /* empty */ | tablePropertyList tableProperty
    ;

tableDeclaration
    : annotationList TABLE name { tablePropertyList }
    ;

controlTypeDeclaration
    : annotationList CONTROL name typeParameterListOpt ( parameterList ) ;
    ;

controlBody = blockStatement

controlLocalDeclaration
    : constantDeclaration | instantiation | variableDeclaration
    | actionDeclaration | tableDeclaration
    ;

controlLocalDeclarationList
    : /* empty */ | controlLocalDeclarationList controlLocalDeclaration
    ;

controlDeclaration
    : annotationList CONTROL name typeParameterListOpt ( parameterList )
        constructorParameterListOpt
        { controlLocalDeclarationList APPLY controlBody }
    ;

packageTypeDeclaration
    : annotationList PACKAGE name typeParameterListOpt ( parameterList ) ;
    ;

typeDeclaration
    : derivedTypeDeclaration | typedefDeclaration | parserTypeDeclaration
```

```
    | controlTypeDeclaration | packageTypeDeclaration
    ;

declaration
    : constantDeclaration | instantiation | functionDeclaration
    | actionDeclaration | errorDeclaration | matchKindDeclaration
    | externDeclaration | parserDeclaration | controlDeclaration
    | typeDeclaration
    ;

annotationToken
    : UNEXPECTED_TOKEN | ABSTRACT | ACTION | ACTIONS | APPLY | BOOL | BIT
    | BREAK | CONST | CONTINUE | CONTROL | DEFAULT | ELSE | ENTRIES | ENUM
    | ERROR | EXIT | EXTERN | FALSE | FOR | HEADER | HEADER_UNION | IF | IN
    | INOUT | INT | KEY | MATCH_KIND | TYPE | OUT | PARSER | PACKAGE | PRAGMA
    | RETURN | SELECT | STATE | STRING | STRUCT | SWITCH | TABLE | THIS
    | TRANSITION | TRUE | TUPLE | TYPEDEF | VARBIT | VALUE_SET | LIST | VOID | _
    | identifier | typeIdentifier | stringLiteral | integerLiteral | &&& | ..
    | << | && | || | == | != | >= | <= | ++ | + | |+| | - | |-| | * | / | % | |
    | & | ^ | ~ | [ | ] | { | } | < | > | ! | : | , | ? | . | = | ; | @
    ;

annotationBody
    : /* empty */ | annotationBody ( annotationBody )
    | annotationBody annotationToken
    ;

structuredAnnotationBody
    : sequenceOrRecordElementExpression trailingCommaOpt
    ;

annotation
    : @ name | @ name ( annotationBody ) | @ name [ structuredAnnotationBody ]
    | @ PRAGMA name annotationBody
    ;

annotationListNonEmpty
    : annotation | annotationListNonEmpty annotation
    ;

annotationList
    : /* empty */ | annotationListNonEmpty
    ;

p4program
    : /* empty */ | p4program declaration | p4program ;
    ;
```

Appendix G: $P4_{IR}$ grammar

This is the grammar of $P4_{IR}$, which is produced after type checking a $P4_{16}$ program. It is largely the same as the $P4_{16}$ grammar, but with additional information, such as type annotations for expressions.

```
prefixedNameIR
    : _BARE nameIR | . nameIR
    ;

nameListIR = nameIR*

voidTypeIR
    : VOID
    ;

boolTypeIR
    : BOOL
    ;

errorTypeIR
    : ERROR
    ;

matchKindTypeIR
    : MATCH_KIND
    ;

stringTypeIR
    : STRING
    ;

intTypeIR
    : INT
    ;

fixedIntTypeIR
    : INT < nat >
    ;

fixedBitTypeIR
    : BIT < nat >
```

```
    ;

varBitTypeIR
    : VARBIT < nat >
    ;

integerTypeIR
    : intTypeIR | fixedIntTypeIR | fixedBitTypeIR | varBitTypeIR
    ;

baseTypeIR
    : voidTypeIR | boolTypeIR | errorTypeIR | matchKindTypeIR | stringTypeIR
    | integerTypeIR
    ;

nameTypeIR
    : _NAME typeId
    ;

typedefTypeIR
    : TYPEDEF typeId typeIR
    ;

newTypeIR
    : TYPE typeId typeIR
    ;

aliasTypeIR
    : typedefTypeIR | newTypeIR
    ;

listTypeIR
    : LIST < typeIR >
    ;

tupleTypeIR
    : TUPLE < typeIR* >
    ;

headerStackTypeIR
    : HEADER_STACK typeIR [ nat ]
    ;

fieldTypeIR
    : annotationList typeIR nameIR ;
    ;

structTypeIR
    : STRUCT typeId < typeArgumentIR* > { fieldTypeIR* }
    ;

headerTypeIR
    : HEADER typeId < typeArgumentIR* > { fieldTypeIR* }
    ;

headerUnionTypeIR
    : HEADER_UNION typeId < typeArgumentIR* > { fieldTypeIR* }
    ;

valueFieldIR
    : nameIR = value ;
    ;

simpleEnumTypeIR
    : ENUM typeId { nameIR* }
    ;

serializableEnumTypeIR
    : ENUM typeId < typeIR > { valueFieldIR* }
```

```
    ;

enumTypeIR
    : simpleEnumTypeIR | serializableEnumTypeIR
    ;

dataTypeIR
    : listTypeIR | tupleTypeIR | arrayTypeIR | headerStackTypeIR | structTypeIR
    | headerTypeIR | headerUnionTypeIR | enumTypeIR
    ;

externMethodTypeDefEnv = map<callableId, externMethodTypeDefIR>

externObjectTypeIR
    : EXTERN typeId < typeArgumentIR* > externMethodTypeDefEnv
    ;

parserObjectTypeIR
    : PARSER typeId < typeArgumentIR* > ( parameterIR* )
    ;

controlObjectTypeIR
    : CONTROL typeId < typeArgumentIR* > ( parameterIR* )
    ;

packageObjectTypeIR
    : PACKAGE typeId < typeArgumentIR* > { typeIR* }
    ;

tableObjectTypeIR
    : TABLE typeId { tableMetadataStructTypeIR }
    ;

objectTypeIR
    : externObjectTypeIR | parserObjectTypeIR | controlObjectTypeIR
    | packageObjectTypeIR | tableObjectTypeIR
    ;

defaultTypeIR
    : DEFAULT
    ;

invalidHeaderTypeIR
    : HEADER_INVALID
    ;

sequenceTypeIR
    : SEQ < typeIR* > | SEQ < typeIR* , ... >
    ;

recordTypeIR
    : RECORD { fieldTypeIR* } | RECORD { fieldTypeIR* , ... }
    ;

setTypeIR
    : SET < typeIR >
    ;

tableMetadataEnumTypeIR
    : TABLE_ENUM typeId { nameIR* }
    ;

tableMetadataStructTypeIR
    : TABLE_STRUCT typeId
        {
          HIT boolTypeIR ; MISS boolTypeIR ; ACTION_RUN tableMetadataEnumTypeIR
          ;
        }
    ;
```

```
tableMetadataTypeIR
    : tableMetadataEnumTypeIR | tableMetadataStructTypeIR
    ;

synthesizedTypeIR
    : defaultTypeIR | invalidHeaderTypeIR | sequenceTypeIR | recordTypeIR
    | setTypeIR | tableMetadataTypeIR
    ;

typeIR
    : baseTypeIR | nameTypeIR | aliasTypeIR | dataTypeIR | objectTypeIR
    | synthesizedTypeIR
    ;

typeParameterIR = typeId

typeParameterListIR = typeParameterIR*

parameterIR
    : annotationList direction typeIR id parameterInitializerOptIR
    ;

parameterListIR = parameterIR*

constructorParameterIR = parameterIR

constructorParameterListIR = parameterIR*

namedExpressionIR
    : nameIR = typedExpressionIR
    ;

namedExpressionListIR = namedExpressionIR*

literalExpressionIR = literalExpression

referenceExpressionIR = prefixedNameIR

defaultExpressionIR = defaultExpression

unaryExpressionIR
    : unop typedExpressionIR
    ;

binaryExpressionIR
    : typedExpressionIR binop typedExpressionIR
    ;

ternaryExpressionIR
    : typedExpressionIR ? typedExpressionIR : typedExpressionIR
    ;

castExpressionIR
    : ( typeIR ) typedExpressionIR
    ;

dataExpressionIR
    : invalidHeaderExpressionIR | sequenceExpressionIR | recordExpressionIR
    ;

errorAccessExpressionIR
    : ERROR . nameIR
    ;

memberAccessExpressionIR
    : memberAccessBaseIR . nameIR
    ;
```

```
indexAccessExpressionIR
    : typedExpressionIR [ typedExpressionIR ]
    ;

sliceAccessExpressionIR
    : typedExpressionIR [ typedExpressionIR sliceop typedExpressionIR ]
    ;

accessExpressionIR
    : errorAccessExpressionIR | memberAccessExpressionIR
    | indexAccessExpressionIR | sliceAccessExpressionIR
    ;

callableTargetIR
    : referenceExpressionIR | typedExpressionIR . nameIR
    | TYPE prefixedNameIR . nameIR | ( callableTargetIR )
    ;

constructorTargetIR
    : prefixedNameIR < typeArgumentListIR >
    ;

callExpressionIR
    : constructorTargetIR ( argumentListIR )
    | callableTargetIR < typeArgumentListIR > ( argumentListIR )
    ;

parenthesizedExpressionIR
    : ( typedExpressionIR )
    ;

expressionIR
    : literalExpressionIR | referenceExpressionIR | defaultExpressionIR
    | unaryExpressionIR | binaryExpressionIR | ternaryExpressionIR
    | castExpressionIR | dataExpressionIR | accessExpressionIR
    | callExpressionIR | parenthesizedExpressionIR
    ;

expressionNoteIR
    : ( typeIR ctk )
    ;

typedExpressionIR
    : expressionIR # expressionNoteIR
    ;

typedExpressionListIR = typedExpressionIR*

memberAccessBaseIR
    : TYPE prefixedNameIR | typedExpressionIR
    ;

simpleKeysetExpressionIR
    : typedExpressionIR | typedExpressionIR &&& typedExpressionIR
    | typedExpressionIR .. typedExpressionIR | DEFAULT | _
    ;

simpleKeysetExpressionListIR = simpleKeysetExpressionIR*

tupleKeysetExpressionIR
    : ( simpleKeysetExpressionListIR )
    ;

keysetExpressionIR
    : simpleKeysetExpressionIR | tupleKeysetExpressionIR
    ;

typeArgumentIR = typeIR
```

```
typeArgumentListIR = typeArgumentIR*

argumentIR
    : typedExpressionIR | nameIR = typedExpressionIR | nameIR = _ | _
    ;

argumentListIR = argumentIR*

lvalueIR
    : referenceExpressionIR | typedLvalueIR . nameIR
    | typedLvalueIR [ typedExpressionIR ]
    | typedLvalueIR [ typedExpressionIR sliceop typedExpressionIR ]
    | ( typedLvalueIR )
    ;

emptyStatementIR = emptyStatement

assignmentStatementIR
    : typedLvalueIR assignop typedExpressionIR ;
    ;

callStatementIR
    : callableTargetIR < typeArgumentListIR > ( argumentListIR ) ;
    ;

directApplicationStatementIR
    : constructorTargetIR . APPLY ( argumentListIR ) ;
    ;

returnStatementIR
    : RETURN ; | RETURN typedExpressionIR ;
    ;

exitStatementIR = exitStatement

blockStatementIR
    : annotationList { blockElementStatementListIR }
    ;

conditionalStatementIR
    : IF ( typedExpressionIR ) statementIR
    | IF ( typedExpressionIR ) statementIR ELSE statementIR
    ;

forUpdateStatementIR
    : callableTargetIR < typeArgumentListIR > ( argumentListIR )
    | typedLvalueIR assignop typedExpressionIR
    ;

forUpdateStatementListIR = forUpdateStatementIR*

forInitStatementIR
    : annotationList typeIR nameIR initializerOptIR | forUpdateStatementIR
    ;

forInitStatementListIR = forInitStatementIR*

forCollectionExpressionIR
    : typedExpressionIR | typedExpressionIR .. typedExpressionIR
    ;

forStatementIR
    : annotationList FOR
        (
          forInitStatementListIR ; typedExpressionIR ; forUpdateStatementListIR
        ) statementIR
    | annotationList FOR ( typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    | annotationList FOR
```

```
        ( annotationList typeIR nameIR IN forCollectionExpressionIR )
        statementIR
    ;

switchLabelIR
    : DEFAULT | typedExpressionIR
    ;

switchCaseIR
    : switchLabelIR : blockStatementIR | switchLabelIR :
    ;

switchCaseListIR = switchCaseIR*

switchStatementIR
    : SWITCH ( typedExpressionIR ) { switchCaseListIR }
    ;

breakStatementIR = breakStatement

continueStatementIR = continueStatement

statementIR
    : emptyStatementIR | assignmentStatementIR | callStatementIR
    | directApplicationStatementIR | returnStatementIR | exitStatementIR
    | blockStatementIR | conditionalStatementIR | forStatementIR
    | breakStatementIR | continueStatementIR | switchStatementIR
    ;

constantInitializerIR
    : = _VALUE value
    ;

constantDeclarationIR
    : annotationList CONST typeIR nameIR constantInitializerIR ;
    ;

initializerOptIR = initializerIR?

variableDeclarationIR
    : annotationList typeIR nameIR initializerOptIR ;
    ;

blockElementStatementIR
    : constantDeclarationIR | variableDeclarationIR | statementIR
    ;

blockElementStatementListIR = blockElementStatementIR*

functionPrototypeIR
    : typeIR nameIR < typeParameterListIR , typeParameterListIR >
        ( parameterListIR )
    ;

functionDeclarationIR
    : annotationList functionPrototypeIR blockStatementIR
    ;

actionDeclarationIR
    : annotationList ACTION nameIR ( parameterListIR ) blockStatementIR
    ;

objectInitializerIR
    : = { objectDeclarationListIR }
    ;

instantiationIR
    : annotationList typeIR constructorTargetIR ( argumentListIR ) nameIR
        objectInitializerOptIR ;
```

```
    ;

objectDeclarationIR
    : functionDeclarationIR | instantiationIR
    ;

objectDeclarationListIR = objectDeclarationIR*

errorDeclarationIR
    : ERROR { nameListIR }
    ;

matchKindDeclarationIR
    : MATCH_KIND { nameListIR }
    ;

enumTypeDeclarationIR
    : annotationList ENUM nameIR { nameListIR }
    | annotationList ENUM typeIR nameIR { namedValueListIR }
    ;

typeFieldIR
    : annotationList typeIR nameIR ;
    ;

typeFieldListIR = typeFieldIR*

structTypeDeclarationIR
    : annotationList STRUCT nameIR < typeParameterListIR > { typeFieldListIR }
    ;

headerTypeDeclarationIR
    : annotationList HEADER nameIR < typeParameterListIR > { typeFieldListIR }
    ;

headerUnionTypeDeclarationIR
    : annotationList HEADER_UNION nameIR < typeParameterListIR >
        { typeFieldListIR }
    ;

derivedTypeDeclarationIR
    : enumTypeDeclarationIR | structTypeDeclarationIR | headerTypeDeclarationIR
    | headerUnionTypeDeclarationIR
    ;

typedefIR
    : typeIR | derivedTypeDeclarationIR
    ;

typedefDeclarationIR
    : annotationList TYPEDEF typedefIR nameIR ;
    | annotationList TYPE typeIR nameIR ;
    ;

externFunctionDeclarationIR
    : annotationList EXTERN functionPrototypeIR ;
    ;

externConstructorPrototypeIR
    : annotationList nameIR < , typeParameterListIR > ( parameterListIR ) ;
    ;

externMethodPrototypeIR
    : annotationList functionPrototypeIR ;
    | annotationList ABSTRACT functionPrototypeIR ;
    ;

externObjectDeclarationIR
    : annotationList EXTERN nameIR < typeParameterListIR , >
```

```
        { externConstructorPrototypeListIR externMethodPrototypeListIR }
    ;

externDeclarationIR
    : externFunctionDeclarationIR | externObjectDeclarationIR
    ;

selectCaseIR
    : keysetExpressionIR : nameIR ;
    ;

selectCaseListIR = selectCaseIR*

selectExpressionIR
    : SELECT ( typedExpressionListIR ) { selectCaseListIR }
    ;

stateExpressionIR
    : nameIR ; | selectExpressionIR
    ;

transitionStatementIR
    : TRANSITION stateExpressionIR
    ;

valueSetDeclarationIR
    : annotationList VALUE_SET < typeIR > ( typedExpressionIR ) nameIR ;
    ;

parserTypeDeclarationIR
    : annotationList PARSER nameIR < typeParameterListIR , typeParameterListIR >
        ( parameterListIR ) ;
    ;

parserBlockStatementIR
    : annotationList { parserBlockElementStatementListIR }
    ;

parserBlockElementStatementIR
    : constantDeclarationIR | variableDeclarationIR | parserStatementIR
    ;

parserBlockElementStatementListIR = parserBlockElementStatementIR*

parserConditionalStatementIR
    : IF ( typedExpressionIR ) parserStatementIR
    | IF ( typedExpressionIR ) parserStatementIR ELSE parserStatementIR
    ;

parserStatementIR
    : emptyStatementIR | assignmentStatementIR | callStatementIR
    | directApplicationStatementIR | parserBlockStatementIR
    | parserConditionalStatementIR
    ;

parserStateIR
    : annotationList STATE nameIR
        { parserBlockElementStatementListIR transitionStatementIR }
    ;

parserStateListIR = parserStateIR*

parserLocalDeclarationIR
    : constantDeclarationIR | instantiationIR | variableDeclarationIR
    | valueSetDeclarationIR
    ;

parserLocalDeclarationListIR = parserLocalDeclarationIR*
```

```
parserDeclarationIR
    : annotationList PARSER nameIR < typeParameterListIR , typeParameterListIR >
        ( parameterListIR ) ( constructorParameterListIR )
        { parserLocalDeclarationListIR parserStateListIR }
    ;

constIR = const

constOptIR = constIR?

tableKeyIR
    : typedExpressionIR # nameIR : nameIR annotationList ;
    ;

tableKeyListIR = tableKeyIR*

controlPlaneNameIR = nameIR?

tableActionReferenceIR
    : prefixedNameIR controlPlaneNameIR ( argumentListIR )
    ;

tableActionIR
    : annotationList tableActionReferenceIR #
        ( parameterListIR , parameterListIR ) ;
    ;

tableActionListIR = tableActionIR*

tableEntryPriorityIR
    : PRIORITY = integerLiteral :
    ;

tableEntryIR
    : constOptIR tableEntryPriorityOptIR keysetExpressionIR :
        tableActionReferenceIR annotationList ;
    ;

tableEntryListIR = tableEntryIR*

tablePropertyIR
    : tableKeysPropertyIR | tableActionsPropertyIR
    | tableDefaultActionPropertyIR | tableEntriesPropertyIR
    | tableCustomPropertyIR
    ;

tablePropertyListIR = tablePropertyIR*

tableDeclarationIR
    : annotationList TABLE typeIR nameIR { tablePropertyListIR }
    ;

controlTypeDeclarationIR
    : annotationList CONTROL nameIR
        < typeParameterListIR , typeParameterListIR > ( parameterListIR ) ;
    ;

controlBodyIR = blockStatementIR

controlLocalDeclarationIR
    : constantDeclarationIR | instantiationIR | variableDeclarationIR
    | actionDeclarationIR | tableDeclarationIR
    ;

controlLocalDeclarationListIR = controlLocalDeclarationIR*

controlDeclarationIR
    : annotationList CONTROL nameIR
        < typeParameterListIR , typeParameterListIR > ( parameterListIR )
```

```
        ( constructorParameterListIR )
        { controlLocalDeclarationListIR APPLY controlBodyIR }
    ;

packageTypeDeclarationIR
    : annotationList PACKAGE nameIR
        < typeParameterListIR , typeParameterListIR > ( parameterListIR ) ;
    ;

typeDeclarationIR
    : derivedTypeDeclarationIR | typedefDeclarationIR | parserTypeDeclarationIR
    | controlTypeDeclarationIR | packageTypeDeclarationIR
    ;

declarationIR
    : constantDeclarationIR | instantiationIR | functionDeclarationIR
    | actionDeclarationIR | errorDeclarationIR | matchKindDeclarationIR
    | externDeclarationIR | parserDeclarationIR | controlDeclarationIR
    | typeDeclarationIR
    ;

p4programIR
    : declarationIR* ;
    ;
```

Appendix H: Built-in meta-functions

This appendix collects the built-in, low-level meta-functions on options, sequences, sets, maps, and text that are used throughout the formal specification. They are largely mechanical and are gathered here so that the main specification remains readable.

H.1. Core notation and collections

A conditional chooses between two values:

X_t if b otherwise X_f

1. If b is equal to true: return X_t.
2. Else: return X_f.

Options are inspected and converted to sequences with:

$is_some_<X>($X^?$)

1. If $X^?$ matches pattern $()$: return false.
2. Else: return true.

$opt_as_seq_<X>($X'^?$)

1. If $X'^?$ matches pattern $()$: return $\cdot$.
2. Else:
    a. Let X be $X'^?$.
    b. Return X.

A sequence of Booleans is reduced with:

Some of b^* is true

1. If b^* matches pattern []: return false.
2. Else:
    a. Let $b_h :: b_t^*$ be b^*.
    b. Let bool be some of b_t^* is true.
    c. Return b_h or bool.

All b^* are true

1. If b^* matches pattern []: return true.
2. Else:
    a. Let $b_h :: b_t^*$ be b^*.
    b. Let bool be all b_t^* are true.
    c. Return b_h and bool.

Sequences are constructed and filtered with:

The list of the first nat natural numbers

1. If nat is equal to 0: return ·.
2. Else:
    a. Let i be nat - 1.
    b. Check that i has type nat.
    c. Let n be i.
    d. Let nat'^* be the list of the first n natural numbers.
    e. Return n :: nat'^*.

A list of nat copies of X

1. If nat is equal to 0: return ·.
2. Else:
    a. Let i be nat - 1.
    b. Check that i has type nat.
    c. Let n be i.
    d. Let X'^* be a list of n copies of X.
    e. Return X concatenated with X'^*.

X^* filtered by b^*

1. If X^* matches pattern []:

a. Check that b^* is an empty list.

b. Return $\cdot$.

2. Else:

a. Let $X_h :: X_t^*$ be X^*.

b. Check that b^* is a non-empty list.

c. Let $b_h :: b_t^*$ be b^*.

d. If b_h:

i. Let X'^* be X_t^* filtered by b_t^*.

ii. Return $X_h :: X'^*$.

e. Else:

i. Return X_t^* filtered by b_t^*.

Sets support the following operations:

$empty_set<K>

1. Return `{ · `}.

K is an element of the set `{ $K_e^*$ `}

1. Return K is in K_e^*.

Maps support the following operations:

An empty map

1. Return `{ · `}.

The domain of the map `{ ( K ':' V )* `}

1. Return `{ $K^*$ `}.

The codomain of the map `{ ( K ':' V )* `}

1. Return `{ $V^*$ `}.

$extend_map<K, V>(`{ ( $K_a$ ':' $V_a$ )* `}, `{ ( $K_b$ ':' $V_b$ )* `})

1. Let `{ ( $K_c$ ':' $V_c$ )* `} be `{ ( $K_a$ ':' $V_a$ )* `} where each of K_b^* to V_b^* are added.
2. Return `{ ( $K_c$ ':' $V_c$ )* `}.

Text is concatenated, directly or with a separator, with:

The concatenation of t^*

1. If t^* matches pattern []: return "".
2. Else:
    a. Let $t_h :: t_t^*$ be t^*.
    b. Let text be the concatenation of t_t^*.
    c. Return t_h concatenated with text.

The result of joining t'^* with separator t_{sep}

1. If t'^* matches pattern []: return "".
2. Else if t'^* matches pattern [_/1]:
    a. Let t_h be t'^*.
    b. Return t_h.
3. Else if t'^* matches pattern ::::
    a. Let $t_{h1} :: t^*$ be t'^*.
    b. Check that t^* is a non-empty list.
    c. Let $t_{h2} :: t_t^*$ be t^*.
    d. Let text be the result of joining $t_{h2} :: t_t^*$ with separator t_{sep}.
    e. Return t_{h1} concatenated with t_{sep} concatenated with text.

Substrings are replaced, optionally leaving a marked region untouched, with:

The text t with all occurrences of t_{old} replaced by t_{new}

1. Return $replace_text'(0, the length of t, t, t_{old}, t_{new}).

$replace_text'($n_{len}$, $n_{len'}$, t, t_{old}, t_{new})

1. Try:
    a. If n_{len} is equal to $n_{len'}$: return t.
2. Then, try:
    a. Check that n_{len} is less than $n_{len'}$.
    b. Check that n_{len} is less than the length of t.
    c. Let $t_{replaced}$ be t with [n_{len}] set to t_{new} if t[n_{len}] is equal to t_{old} otherwise t.
    d. Return $replace_text'($n_{len}$ + 1, $n_{len'}$, $t_{replaced}$, t_{old}, t_{new}).

The text t with all occurrences of t_{old} not replaced by t_{new}

1. Return $replace_text_except'(0, the length of t, t, t_{old}, t_{new}).

$replace_text_except'($n_{len}$, $n_{len'}$, t, t_{old}, t_{new})

1. Try:
    a. If n_{len} is equal to $n_{len'}$: return t.
2. Then, try:
    a. Check that n_{len} is less than $n_{len'}$.
    b. Check that n_{len} is less than the length of t.
    c. Let $t_{replaced}$ be t if t[n_{len}] is equal to t_{old} otherwise t with [n_{len}] set to t_{new}.
    d. Return $replace_text_except'($n_{len}$ + 1, $n_{len'}$, $t_{replaced}$, t_{old}, t_{new}).

Prefixes and suffixes are tested and stripped with:

The text t starts with t_{prefix}

1. If the length of t_{prefix} is less than or equal to the length of t:
    a. If t[0 : the length of t_{prefix}] is equal to t_{prefix}: return true.
2. Otherwise: return false.

The text t ends with t_{suffix}

1. If the length of t_{suffix} is less than or equal to the length of t:
    a. Let i be the length of t - the length of t_{suffix}.
    b. If let n_{start} of type nat be i:
        i. If t[n_{start} : the length of t_{suffix}] is equal to t_{suffix}: return true.
2. Otherwise: return false.

The text t with the prefix t_{prefix} removed many times as possible

1. If the text t starts with t_{prefix}:
    a. Let $t_{stripped}$ be the text t with the prefix t_{prefix} removed.
    b. Return the text $t_{stripped}$ with the prefix t_{prefix} removed many times as possible.
2. Else: return t.

The text t with the suffix t_{suffix} removed many times as possible

1. If the text t ends with t_{suffix}:
    a. Let $t_{stripped}$ be the text t with the suffix t_{suffix} removed.
    b. Return the text $t_{stripped}$ with the suffix t_{suffix} removed many times as possible.
2. Else: return t.

Substring containment is tested with:

The text t contains t_c

1. Return $contains'(0, the length of t, t, t_c).

$$contains'(n_{len}, n_{len'}, t, t_c)$$

1. Try:
   a. If n_{len} is equal to $n_{len'}$: return false.
2. Then, try:
   a. Check that n_{len} is less than $n_{len'}$.
   b. Check that n_{len} is less than the length of t.
   c. If $t[n_{len}]$ is equal to t_c: return true.
   d. Else:
      i. Return $contains'($n_{len}$ + 1, $n_{len'}$, t, t_c).

Appendix I: Type meta-functions

This appendix collects the meta-functions that analyze and transform types, including free type variable computation, fresh variable generation, and substitution.

I.1. Free type variables

The free type variables of a type, its definition, a callable type, and a parameter are collected with:

Free type identifiers in typeIR'

1. If typeIR' has type baseTypeIR: return `{ · `}.
2. Else if let NAME typeId of type nameTypeIR be typeIR': return `{ typeId `}.
3. Else if let TYPEDEF _ typeIR of type typedefTypeIR be typeIR':
   a. Return free type identifiers in typeIR.
4. Else if let TYPE _ typeIR of type newTypeIR be typeIR':
   a. Return free type identifiers in typeIR.
5. Else if let LIST `< typeIR `> of type listTypeIR be typeIR':
   a. Return free type identifiers in typeIR.
6. Else if let TUPLE `< typeIR* `> of type tupleTypeIR be typeIR':
   a. For each typeIR in typeIR*:
      - Let bound be free type identifiers in typeIR.

      Let bound* be the resulting list.
   b. Return the union of the sets bound*.
7. Else if let ARRAY typeIR `[ _ `] of type arrayTypeIR be typeIR':
   a. Return free type identifiers in typeIR.

8. Else if let HEADER_STACK typeIR `[ _ `] of type headerStackTypeIR be typeIR':
    a. Return free type identifiers in typeIR.
9. Else if let STRUCT _ `< typeArgumentIR* `> `{ ( _ typeIR$_{field}$ _ ';' )* `} of type structTypeIR be typeIR':
    a. For each typeArgumentIR in typeArgumentIR*:
        - Let bound'' be free type identifiers in typeArgumentIR.

      Let bound''* be the resulting list.
    b. Let set<typeId> be the union of the sets bound''*.
    c. For each typeIR$_{field}$ in typeIR$_{field}$*:
        - Let bound' be free type identifiers in typeIR$_{field}$.

      Let bound'* be the resulting list.
    d. Let set<typeId>' be the union of the sets bound'*.
    e. Return the union of the sets set<typeId> and set<typeId>'.
10. Else if let HEADER _ `< typeArgumentIR* `> `{ ( _ typeIR$_{field}$ _ ';' )* `} of type headerTypeIR be typeIR':
    a. For each typeArgumentIR in typeArgumentIR*:
        - Let bound'''' be free type identifiers in typeArgumentIR.

      Let bound''''* be the resulting list.
    b. Let set<typeId>'' be the union of the sets bound''''*.
    c. For each typeIR$_{field}$ in typeIR$_{field}$*:
        - Let bound''' be free type identifiers in typeIR$_{field}$.

      Let bound'''* be the resulting list.
    d. Let set<typeId>''' be the union of the sets bound'''*.
    e. Return the union of the sets set<typeId>'' and set<typeId>'''.
11. Else if let HEADER_UNION _ `< typeArgumentIR* `> `{ ( _ typeIR$_{field}$ _ ';' )* `} of type headerUnionTypeIR be typeIR':
    a. For each typeArgumentIR in typeArgumentIR*:
        - Let bound'''''' be free type identifiers in typeArgumentIR.

      Let bound''''''* be the resulting list.
    b. Let set<typeId>'''' be the union of the sets bound''''''*.
    c. For each typeIR$_{field}$ in typeIR$_{field}$*:
        - Let bound''''' be free type identifiers in typeIR$_{field}$.

      Let bound'''''* be the resulting list.
    d. Let set<typeId>''''' be the union of the sets bound'''''*.
    e. Return the union of the sets set<typeId>'''' and set<typeId>'''''.
12. Else if typeIR' has type simpleEnumTypeIR: return `{ · `}.
13. Else if let ENUM _ `< typeIR `> `{ _* `} of type serializableEnumTypeIR be typeIR':
    a. Return free type identifiers in typeIR.

14. Else if let EXTERN _ `< typeArgumentIR* `> `{ ( _ ':' externMethodTypeDefIR )* `} of type externObjectTypeIR be typeIR':
    a. For each typeArgumentIR in typeArgumentIR*:
        - Let bound''''''' be free type identifiers in typeArgumentIR.

        Let bound'''''''* be the resulting list.
    b. Let set<typeId>''''' be the union of the sets bound'''''''*.
    c. For each externMethodTypeDefIR in externMethodTypeDefIR*:
        - Let bound''''' be free type identifiers in externMethodTypeDefIR.

        Let bound'''''''* be the resulting list.
    d. Let set<typeId>''''' be the union of the sets bound'''''''*.
    e. Return the union of the sets set<typeId>''''' and set<typeId>'''''''.
15. Else if let PARSER _ `< typeArgumentIR* `> `( parameterIR* `) of type parserObjectTypeIR be typeIR':
    a. For each typeArgumentIR in typeArgumentIR*:
        - Let bound''''''''' be free type identifiers in typeArgumentIR.

        Let bound'''''''''* be the resulting list.
    b. Let set<typeId>''''''' be the union of the sets bound'''''''''*.
    c. For each parameterIR in parameterIR*:
        - Let bound''''''' be free type identifiers in parameterIR.

        Let bound'''''''* be the resulting list.
    d. Let set<typeId>''''''' be the union of the sets bound'''''''''*.
    e. Return the union of the sets set<typeId>''''''' and set<typeId>'''''''''.
16. Else if let CONTROL _ `< typeArgumentIR* `> `( parameterIR* `) of type controlObjectTypeIR be typeIR':
    a. For each typeArgumentIR in typeArgumentIR*:
        - Let bound''''''''' be free type identifiers in typeArgumentIR.

        Let bound'''''''''* be the resulting list.
    b. Let set<typeId>''''''''' be the union of the sets bound'''''''''*.
    c. For each parameterIR in parameterIR*:
        - Let bound''''''''' be free type identifiers in parameterIR.

        Let bound'''''''''* be the resulting list.
    d. Let set<typeId>''''''''' be the union of the sets bound'''''''''*.
    e. Return the union of the sets set<typeId>''''''''' and set<typeId>'''''''''.
17. Else if let PACKAGE _ `< typeArgumentIR* `> `{ typeIR* `} of type packageObjectTypeIR be typeIR':
    a. For each typeArgumentIR in typeArgumentIR*:
        - Let bound''''''''''' be free type identifiers in typeArgumentIR.

Let bound''''''''''''* be the resulting list.

b. Let set<typeId>'''''''''' be the union of the sets bound''''''''''''*.

c. For each typeIR in typeIR*:

- Let bound'''''''''' be free type identifiers in typeIR.

Let bound''''''''''''* be the resulting list.

d. Let set<typeId>'''''''''' be the union of the sets bound''''''''''''*.

e. Return the union of the sets set<typeId>'''''''''' and set<typeId>''''''''''.

18. Else if typeIR' has type tableObjectTypeIR: return `{ · `}.

19. Else if let defaultTypeIR of type defaultTypeIR be typeIR': return `{ · `}.

20. Else if let invalidHeaderTypeIR of type invalidHeaderTypeIR be typeIR': return `{ · `}.

21. Else if let sequenceTypeIR of type sequenceTypeIR be typeIR':

a. If let SEQ `< typeIR* `> be sequenceTypeIR:

i. For each typeIR in typeIR*:

- Let bound''''''''''''' be free type identifiers in typeIR.

Let bound'''''''''''''* be the resulting list.

ii. Return the union of the sets bound'''''''''''''*.

b. Else:

i. Let SEQ `< typeIR* ',' '...' `> be sequenceTypeIR.

ii. For each typeIR in typeIR*:

- Let bound''''''''''''' be free type identifiers in typeIR.

Let bound'''''''''''''* be the resulting list.

iii. Return the union of the sets bound'''''''''''''*.

22. Else if let recordTypeIR of type recordTypeIR be typeIR':

a. If let RECORD `{ ( _ typeIR _ ';' )* `} be recordTypeIR:

i. For each typeIR in typeIR*:

- Let bound''''''''''''''' be free type identifiers in typeIR.

Let bound'''''''''''''''* be the resulting list.

ii. Return the union of the sets bound'''''''''''''''*.

b. Else:

i. Let RECORD `{ ( _ typeIR _ ';' )* ',' '...' `} be recordTypeIR.

ii. For each typeIR in typeIR*:

- Let bound''''''''''''''' be free type identifiers in typeIR.

Let bound'''''''''''''''* be the resulting list.

iii. Return the union of the sets bound'''''''''''''''*.

23. Else if let SET `< typeIR `> of type setTypeIR be typeIR':

a. Return free type identifiers in typeIR.

24. Else: return `{ · `}.

Free type identifiers in typeDefIR

1. Let (typeParameterIR$_{expl}^{*}$, typeParameterIR$_{impl}^{*}$) be the type parameters of typeDefIR.
2. Let $B_{tparams}$ be `{ typeParameterIR$_{expl}^{*}$ ++ typeParameterIR$_{impl}^{*}$ `}.
3. Let typeIR be the underlying type of typeDefIR.
4. Let B_{base} be free type identifiers in typeIR.
5. Return the difference of the sets B_{base} and $B_{tparams}$.

Free type identifiers in callableTypeIR

1. If let _ ACTION _ `( parameterIR$^{*}$ `) of type actionTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:
      - Let bound be free type identifiers in parameterIR.

      Let bound* be the resulting list.
   b. Let B_{params} be the union of the sets bound*.
   c. Return B_{params}.
2. Else if let FUNCTION _ `( parameterIR$^{*}$ `) ':' typeIR of type definedFunctionTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:
      - Let bound' be free type identifiers in parameterIR.

      Let bound'* be the resulting list.
   b. Let B_{params} be the union of the sets bound'*.
   c. Let B_{ret} be free type identifiers in typeIR.
   d. Return the union of the sets B_{params} and B_{ret}.
3. Else if let EXTERN_FUNCTION _ `( parameterIR$^{*}$ `) ':' typeIR of type externFunctionTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:
      - Let bound'' be free type identifiers in parameterIR.

      Let bound''* be the resulting list.
   b. Let B_{params} be the union of the sets bound''*.
   c. Let B_{ret} be free type identifiers in typeIR.
   d. Return the union of the sets B_{params} and B_{ret}.
4. Else if callableTypeIR has type builtinMethodTypeIR: return `{ · `}.
5. Else if let externMethodTypeIR of type externMethodTypeIR be callableTypeIR:
   a. If let EXTERN_METHOD _ `( parameterIR$^{*}$ `) ':' typeIR be externMethodTypeIR:
      i. For each parameterIR in parameterIR*:
         - Let bound''' be free type identifiers in parameterIR.

Let bound'''* be the resulting list.

ii. Let B_{params} be the union of the sets bound'''*.

iii. Let B_{ret} be free type identifiers in typeIR.

iv. Return the union of the sets B_{params} and B_{ret}.

b. Else:

i. Let EXTERN_METHOD ABSTRACT _ `( parameterIR* `) ':' typeIR be externMethodTypeIR.

ii. For each parameterIR in parameterIR*:

- Let bound'''' be free type identifiers in parameterIR.

Let bound''''* be the resulting list.

iii. Let B_{params} be the union of the sets bound''''*.

iv. Let B_{ret} be free type identifiers in typeIR.

v. Return the union of the sets B_{params} and B_{ret}.

6. Else if let PARSER_APPLY `( parameterIR* `) of type parserApplyMethodTypeIR be callableTypeIR:

a. For each parameterIR in parameterIR*:

- Let bound''''' be free type identifiers in parameterIR.

Let bound'''''* be the resulting list.

b. Let B_{params} be the union of the sets bound'''''*.

c. Return B_{params}.

7. Else if let CONTROL_APPLY `( parameterIR* `) of type controlApplyMethodTypeIR be callableTypeIR:

a. For each parameterIR in parameterIR*:

- Let bound'''''' be free type identifiers in parameterIR.

Let bound''''''* be the resulting list.

b. Let B_{params} be the union of the sets bound''''''*.

c. Return B_{params}.

8. Else: return `{ · `}.

Free type identifiers in callableTypeDefIR

1. Let ($typeId_{tparam_expl}$* , $typeId_{tparam_impl}$*) be the type parameters of callableTypeDefIR.
2. Let $B_{tparams}$ be `{ $typeId_{tparam\_expl}$* ++ $typeId_{tparam\_impl}$* `}.
3. Let callableTypeIR be the underlying type of callableTypeDefIR.
4. Let B_{base} be free type identifiers in callableTypeIR.
5. Return the difference of the sets B_{base} and $B_{tparams}$.

Free type identifiers in _ _ typeIR _ _

1. Return free type identifiers in typeIR.

Free type identifiers in parameterIR*

1. For each parameterIR in parameterIR*:
   - Let bound be free type identifiers in parameterIR.

   Let bound* be the resulting list.
2. Return the union of the sets bound*.

The type variables bound in a typing context are given by:

Bound type variables from the p layer of TC

1. If p matches pattern GLOBAL: return `{ · `}.
2. Else if p matches pattern BLOCK:
   a. Return the domain of the map TC.BLOCK.TYPE.
3. Else:
   a. Let B_{block} be bound type variables from the BLOCK layer of TC.
   b. Let B_{local} be the domain of the map TC.LOCAL.TYPE.
   c. Return the union of the sets B_{block} and B_{local}.

Fresh type variables are generated with:

nat fresh type variables

1. If nat is equal to 0: return ·.
2. Else:
   a. Let i be nat - 1.
   b. Check that i has type nat.
   c. Let n be i.
   d. Let typeId* be n fresh type variables.
   e. Return $fresh_typeId :: typeId*.

I.2. Substitution

A substitution maps type variables to types. Applying a substitution must avoid variable capture, which is ensured by:

$capture_avoiding(theta, $typeParameterIR_{expl}^{*}$, $typeParameterIR_{impl}^{*}$, B)

1. Let `{ $typeId_{free}^{*}$ `} be the difference of the sets B and `{ $typeParameterIR_{expl}^{*}$ ++ $typeParameterIR_{impl}^{*}$ `}.
2. For each $typeId_{free}$ in $typeId_{free}^{*}$:
   - Let typeIR? be the value of $typeId_{free}$ in map theta.

   Let typeIR?* be the resulting list.

3. Check that typeIR$^{?}$ is defined, for all typeIR$^{?}$ in typeIR$^{?*}$.
4. For each typeIR$^{?}$ in typeIR$^{?*}$:

    - Let typeIR$_{free}$ be typeIR$^{?}$.

    Let typeIR$_{free}^{*}$ be the resulting list.

5. For each typeIR$_{free}$ in typeIR$_{free}^{*}$:

    - Let bound be free type identifiers in typeIR$_{free}$.

    Let bound* be the resulting list.

6. Let B$_{capture}$ be the union of the sets bound*.
7. Let typeId$_{fresh}^{*}$ be the length of typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$ fresh type variables.
8. Check that $_{\$in_set<typeId>(typeId}$fresh~, B$_{capture}$), for all typeId$_{fresh}$ in typeId$_{fresh}^{*}$.
9. Let typeParameterIR$_{expl_fresh}^{*}$ be typeId$_{fresh}^{*}$[0 : the length of typeParameterIR$_{expl}^{*}$].
10. Let typeParameterIR$_{impl_fresh}^{*}$ be typeId$_{fresh}^{*}$[the length of typeParameterIR$_{expl}^{*}$: the length of typeParameterIR$_{impl}^{*}$].
11. Let theta$_{fresh}$ be theta where each of typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$ to (NAME typeParameterIR$_{expl_fresh}$)* concatenated with (NAME typeParameterIR$_{impl_fresh}$)* are added.
12. Return (theta$_{fresh}$, typeParameterIR$_{expl_fresh}^{*}$, typeParameterIR$_{impl_fresh}^{*}$).

$capture_avoiding_(theta, typeParameterIR*, B)

1. Let `{ typeId$_{free}^{*}$ `} be the difference of the sets B and `{ typeParameterIR$^{*}$ `}.
2. For each typeId$_{free}$ in typeId$_{free}^{*}$:

    - Let typeIR$^{?}$ be the value of typeId$_{free}$ in map theta.

    Let typeIR$^{?*}$ be the resulting list.

3. Check that typeIR$^{?}$ is defined, for all typeIR$^{?}$ in typeIR$^{?*}$.
4. For each typeIR$^{?}$ in typeIR$^{?*}$:

    - Let typeIR$_{free}$ be typeIR$^{?}$.

    Let typeIR$_{free}^{*}$ be the resulting list.

5. For each typeIR$_{free}$ in typeIR$_{free}^{*}$:

    - Let bound be free type identifiers in typeIR$_{free}$.

    Let bound* be the resulting list.

6. Let B$_{capture}$ be the union of the sets bound*.
7. Let typeParameterIR$_{fresh}^{*}$ be the length of typeParameterIR* fresh type variables.
8. Check that $_{\$in_set<typeId>(typeParameterIR}$fresh~, B$_{capture}$), for all typeParameterIR$_{fresh}$ in typeParameterIR$_{fresh}^{*}$.
9. Let theta$_{fresh}$ be theta where each of typeParameterIR* to (NAME typeParameterIR$_{fresh}$)* are added.
10. Return (theta$_{fresh}$, typeParameterIR$_{fresh}^{*}$).

An empty substitution map is constructed with:

$empty_theta

1. Return an empty map.

Substitution is applied to a type and a parameter with:

typeIR substituted by set<pair<typeId, typeIR>>

1. If set<pair<typeId, typeIR>> is equal to `{ · `}: return typeIR.
2. Otherwise:
    a. Return typeIR substituted by a non-empty set<pair<typeId, typeIR>>.

typeIR' substituted by a non-empty theta

1. If let baseTypeIR of type baseTypeIR be typeIR': return baseTypeIR.
2. Else if let NAME typeId of type nameTypeIR be typeIR':
    a. Try:
        i. Let $typeIR''^{?}$ be the value of typeId in map theta.
        ii. If $typeIR''^{?}$ is defined:
            A. Let typeIR be $typeIR''^{?}$.
            B. Return typeIR.
    b. Then, try:
        i. Check that the value of typeId in map theta is none.
        ii. Return NAME typeId.
3. Else if let TYPEDEF typeId typeIR of type typedefTypeIR be typeIR':
    a. Let $typeIR_{subst}$ be typeIR substituted by a non-empty theta.
    b. Return TYPEDEF typeId $typeIR_{subst}$.
4. Else if let TYPE typeId typeIR of type newTypeIR be typeIR':
    a. Let $typeIR_{subst}$ be typeIR substituted by a non-empty theta.
    b. Return TYPE typeId $typeIR_{subst}$.
5. Else if let LIST `< typeIR `> of type listTypeIR be typeIR':
    a. Let $typeIR_{subst}$ be typeIR substituted by a non-empty theta.
    b. Return LIST `< $typeIR_{subst}$ `>.
6. Else if let TUPLE `< $typeIR^{*}$ `> of type tupleTypeIR be typeIR':
    a. For each typeIR in $typeIR^{*}$:

        - Let $typeIR_{subst}$ be typeIR substituted by a non-empty theta.

        Let $typeIR_{subst}^{*}$ be the resulting list.

    b. Return TUPLE `< $typeIR_{subst}^{*}$ `>.
7. Else if let ARRAY typeIR `[ $n_{size}$ `] of type arrayTypeIR be typeIR':
    a. Let $typeIR_{subst}$ be typeIR substituted by a non-empty theta.
    b. Return ARRAY $typeIR_{subst}$ `[ $n_{size}$ `].

8. Else if let HEADER_STACK typeIR `[ $n_{size}$ `] of type headerStackTypeIR be typeIR':
    a. Let $typeIR_{subst}$ be typeIR substituted by a non-empty theta.
    b. Return HEADER_STACK $typeIR_{subst}$ `[ $n_{size}$ `].
9. Else if let structTypeIR of type structTypeIR be typeIR':
    a. Let STRUCT typeId `< $typeArgumentIR^{*}$ `> `{ ( annotationList $typeIR_{field}$ $nameIR_{field}$ ';' )$^{*}$ `} be structTypeIR.
    b. For each typeArgumentIR in $typeArgumentIR^{*}$:
        - Let $typeArgumentIR_{subst}$ be typeArgumentIR substituted by a non-empty theta.

        Let $typeArgumentIR_{subst}^{*}$ be the resulting list.
    c. For each $typeIR_{field}$ in $typeIR_{field}^{*}$:
        - Let $typeIR_{field_subst}$ be $typeIR_{field}$ substituted by a non-empty theta.

        Let $typeIR_{field_subst}^{*}$ be the resulting list.
    d. Let $structTypeIR_{subst}$ be STRUCT typeId `< $typeArgumentIR_{subst}^{*}$ `> `{ ( annotationList $typeIR_{field\_subst}$ $nameIR_{field}$ ';' )$^{*}$ `}.
    e. Return $structTypeIR_{subst}$.
10. Else if let headerTypeIR of type headerTypeIR be typeIR':
    a. Let HEADER typeId `< $typeArgumentIR^{*}$ `> `{ ( annotationList $typeIR_{field}$ $nameIR_{field}$ ';' )$^{*}$ `} be headerTypeIR.
    b. For each typeArgumentIR in $typeArgumentIR^{*}$:
        - Let $typeArgumentIR_{subst}$ be typeArgumentIR substituted by a non-empty theta.

        Let $typeArgumentIR_{subst}^{*}$ be the resulting list.
    c. For each $typeIR_{field}$ in $typeIR_{field}^{*}$:
        - Let $typeIR_{field_subst}$ be $typeIR_{field}$ substituted by a non-empty theta.

        Let $typeIR_{field_subst}^{*}$ be the resulting list.
    d. Let $headerTypeIR_{subst}$ be HEADER typeId `< $typeArgumentIR_{subst}^{*}$ `> `{ ( annotationList $typeIR_{field\_subst}$ $nameIR_{field}$ ';' )$^{*}$ `}.
    e. Return $headerTypeIR_{subst}$.
11. Else if let headerUnionTypeIR of type headerUnionTypeIR be typeIR':
    a. Let HEADER_UNION typeId `< $typeArgumentIR^{*}$ `> `{ ( annotationList $typeIR_{field}$ $nameIR_{field}$ ';' )$^{*}$ `} be headerUnionTypeIR.
    b. For each typeArgumentIR in $typeArgumentIR^{*}$:
        - Let $typeArgumentIR_{subst}$ be typeArgumentIR substituted by a non-empty theta.

        Let $typeArgumentIR_{subst}^{*}$ be the resulting list.
    c. For each $typeIR_{field}$ in $typeIR_{field}^{*}$:
        - Let $typeIR_{field_subst}$ be $typeIR_{field}$ substituted by a non-empty theta.

        Let $typeIR_{field_subst}^{*}$ be the resulting list.
    d. Let $headerUnionTypeIR_{subst}$ be HEADER_UNION typeId `< $typeArgumentIR_{subst}^{*}$ `> `{ ( annotationList $typeIR_{field\_subst}$ $nameIR_{field}$ ';' )$^{*}$ `}.

e. Return headerUnionTypeIR$_{subst}$.

12. Else if let ENUM typeId `{ id$_{field}^{*}$ `} of type simpleEnumTypeIR be typeIR':
    a. Return ENUM typeId `{ id$_{field}^{*}$ `}.
13. Else if let ENUM typeId `< typeIR `> `{ valueFieldIR$^{*}$ `} of type serializableEnumTypeIR be typeIR':
    a. Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.
    b. Return ENUM typeId `< typeIR$_{subst}$ `> `{ valueFieldIR$^{*}$ `}.
14. Else if let externObjectTypeIR of type externObjectTypeIR be typeIR':
    a. Let EXTERN typeId `< typeArgumentIR$^{*}$ `> `{ ( callableId ':' externMethodTypeDefIR )$^{*}$ `} be externObjectTypeIR.
    b. For each typeArgumentIR in typeArgumentIR*:

       - Let typeArgumentIR$_{subst}$ be typeArgumentIR substituted by a non-empty theta.

       Let typeArgumentIR$_{subst}^{*}$ be the resulting list.

    c. For each externMethodTypeDefIR in externMethodTypeDefIR*:

       - Let callableTypeDefIR be externMethodTypeDefIR substituted by theta.

       Let callableTypeDefIR* be the resulting list.

    d. Check that callableTypeDefIR has type externMethodTypeDefIR, for all callableTypeDefIR in callableTypeDefIR*.
    e. For each callableTypeDefIR in callableTypeDefIR*:

       - Let externMethodTypeDefIR$_{subst}$ be callableTypeDefIR.

       Let externMethodTypeDefIR$_{subst}^{*}$ be the resulting list.

    f. Let externObjectTypeIR$_{subst}$ be EXTERN typeId `< typeArgumentIR$_{subst}^{*}$ `> `{ ( callableId ':' externMethodTypeDefIR$_{subst}$ )$^{*}$ `}.
    g. Return externObjectTypeIR$_{subst}$.
15. Else if let parserObjectTypeIR of type parserObjectTypeIR be typeIR':
    a. Let PARSER typeId `< typeArgumentIR$^{*}$ `> `( parameterIR$^{*}$ `) be parserObjectTypeIR.
    b. For each typeArgumentIR in typeArgumentIR*:

       - Let typeArgumentIR$_{subst}$ be typeArgumentIR substituted by a non-empty theta.

       Let typeArgumentIR$_{subst}^{*}$ be the resulting list.

    c. For each parameterIR in parameterIR*:

       - Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

       Let parameterIR$_{subst}^{*}$ be the resulting list.

    d. Let parserObjectTypeIR$_{subst}$ be PARSER typeId `< typeArgumentIR$_{subst}^{*}$ `> `( parameterIR$_{subst}^{*}$ `).
    e. Return parserObjectTypeIR$_{subst}$.
16. Else if let controlObjectTypeIR of type controlObjectTypeIR be typeIR':
    a. Let CONTROL typeId `< typeArgumentIR$^{*}$ `> `( parameterIR$^{*}$ `) be controlObjectTypeIR.
    b. For each typeArgumentIR in typeArgumentIR*:

       - Let typeArgumentIR$_{subst}$ be typeArgumentIR substituted by a non-empty theta.

Let typeArgumentIR$_{subst}^{*}$ be the resulting list.

c. For each parameterIR in parameterIR*:

- Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

Let parameterIR$_{subst}^{*}$ be the resulting list.

d. Let controlObjectTypeIR$_{subst}$ be CONTROL typeId `< typeArgumentIR$_{subst}^{*}$ `> `( parameterIR$_{subst}^{*}$ `).

e. Return controlObjectTypeIR$_{subst}$.

17. Else if let packageObjectTypeIR of type packageObjectTypeIR be typeIR':

a. Let PACKAGE typeId `< typeArgumentIR$^{*}$ `> `{ typeIR$^{*}$ `} be packageObjectTypeIR.

b. For each typeArgumentIR in typeArgumentIR*:

- Let typeArgumentIR$_{subst}$ be typeArgumentIR substituted by a non-empty theta.

Let typeArgumentIR$_{subst}^{*}$ be the resulting list.

c. For each typeIR in typeIR*:

- Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.

Let typeIR$_{subst}^{*}$ be the resulting list.

d. Let packageObjectTypeIR$_{subst}$ be PACKAGE typeId `< typeArgumentIR$_{subst}^{*}$ `> `{ typeIR$_{subst}^{*}$ `}.

e. Return packageObjectTypeIR$_{subst}$.

18. Else if let tableObjectTypeIR of type tableObjectTypeIR be typeIR': return tableObjectTypeIR.

19. Else if let defaultTypeIR of type defaultTypeIR be typeIR': return DEFAULT.

20. Else if let invalidHeaderTypeIR of type invalidHeaderTypeIR be typeIR': return HEADER_INVALID.

21. Else if let sequenceTypeIR of type sequenceTypeIR be typeIR':

a. If let SEQ `< typeIR$^{*}$ `> be sequenceTypeIR:

i. For each typeIR in typeIR*:

- Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.

Let typeIR$_{subst}^{*}$ be the resulting list.

ii. Return SEQ `< typeIR$_{subst}^{*}$ `>.

b. Else:

i. Let SEQ `< typeIR$^{*}$ ',' '...' `> be sequenceTypeIR.

ii. For each typeIR in typeIR*:

- Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.

Let typeIR$_{subst}^{*}$ be the resulting list.

iii. Return SEQ `< typeIR$_{subst}^{*}$ ',' '...' `>.

22. Else if let recordTypeIR of type recordTypeIR be typeIR':

a. If let RECORD `{ ( annotationList typeIR$_{field}$ id$_{field}$ ';' )$^{*}$ `} be recordTypeIR:

i. For each typeIR$_{field}$ in typeIR$_{field}^{*}$:

- Let typeIR$_{field_subst}$ be typeIR$_{field}$ substituted by a non-empty theta.

Let typeIR$_{field_subst}$* be the resulting list.

ii. Return RECORD `{ ( annotationList typeIR$_{field\_subst}$ id$_{field}$ ';' )* `}.

b. Else:

i. Let RECORD `{ ( annotationList typeIR$_{field}$ id$_{field}$ ';' )* ',' '...' `} be recordTypeIR.

ii. For each typeIR$_{field}$ in typeIR$_{field}$*:

- Let typeIR$_{field_subst}$ be typeIR$_{field}$ substituted by a non-empty theta.

Let typeIR$_{field_subst}$* be the resulting list.

iii. Return RECORD `{ ( annotationList typeIR$_{field\_subst}$ id$_{field}$ ';' )* ',' '...' `}.

23. Else if let SET `< typeIR `> of type setTypeIR be typeIR':

a. Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.

b. Return SET `< typeIR$_{subst}$ `>.

24. Else:

a. Let tableMetadataTypeIR be typeIR'.

b. Return tableMetadataTypeIR.

parameterIR substituted by set<pair<typeId, typeIR>>

1. If set<pair<typeId, typeIR>> is equal to `{ · `}: return parameterIR.
2. Otherwise:
   a. Return parameterIR substituted by a non-empty set<pair<typeId, typeIR>>.

annotationList direction typeIR id parameterInitializerOptIR substituted by a non-empty theta

1. Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.
2. Return annotationList direction typeIR$_{subst}$ id parameterInitializerOptIR.

Substitution is applied to callable types, callable type definitions, and constructor types with:

callableTypeIR substituted by set<pair<typeId, typeIR>>

1. If set<pair<typeId, typeIR>> is equal to `{ · `}: return callableTypeIR.
2. Otherwise:
   a. Return callableTypeIR substituted by a non-empty set<pair<typeId, typeIR>>.

callableTypeIR substituted by a non-empty theta

1. If let annotationList ACTION nameIR `( parameterIR* `) of type actionTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:

- Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

Let parameterIR$_{subst}$* be the resulting list.

b. Return annotationList ACTION nameIR `( parameterIR$_{subst}^{*}$ `).

2. Else if let FUNCTION nameIR `( parameterIR$^{*}$ `) ':' typeIR of type definedFunctionTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:
      - Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

      Let parameterIR$_{subst}^{*}$ be the resulting list.
   b. Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.
   c. Return FUNCTION nameIR `( parameterIR$_{subst}^{*}$ `) ':' typeIR$_{subst}$.
3. Else if let EXTERN_FUNCTION nameIR `( parameterIR$^{*}$ `) ':' typeIR of type externFunctionTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:
      - Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

      Let parameterIR$_{subst}^{*}$ be the resulting list.
   b. Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.
   c. Return EXTERN_FUNCTION nameIR `( parameterIR$_{subst}^{*}$ `) ':' typeIR$_{subst}$.
4. Else if let builtinMethodTypeIR of type builtinMethodTypeIR be callableTypeIR: return builtinMethodTypeIR.
5. Else if let externMethodTypeIR of type externMethodTypeIR be callableTypeIR:
   a. If let EXTERN_METHOD nameIR `( parameterIR$^{*}$ `) ':' typeIR be externMethodTypeIR:
      i. For each parameterIR in parameterIR*:
         - Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

         Let parameterIR$_{subst}^{*}$ be the resulting list.
      ii. Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.
      iii. Return EXTERN_METHOD nameIR `( parameterIR$_{subst}^{*}$ `) ':' typeIR$_{subst}$.
   b. Else:
      i. Let EXTERN_METHOD ABSTRACT nameIR `( parameterIR$^{*}$ `) ':' typeIR be externMethodTypeIR.
      ii. For each parameterIR in parameterIR*:
         - Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

         Let parameterIR$_{subst}^{*}$ be the resulting list.
      iii. Let typeIR$_{subst}$ be typeIR substituted by a non-empty theta.
      iv. Return EXTERN_METHOD ABSTRACT nameIR `( parameterIR$_{subst}^{*}$ `) ':' typeIR$_{subst}$.
6. Else if let PARSER_APPLY `( parameterIR$^{*}$ `) of type parserApplyMethodTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:
      - Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

      Let parameterIR$_{subst}^{*}$ be the resulting list.
   b. Return PARSER_APPLY `( parameterIR$_{subst}^{*}$ `).
7. Else if let CONTROL_APPLY `( parameterIR$^{*}$ `) of type controlApplyMethodTypeIR be callableTypeIR:
   a. For each parameterIR in parameterIR*:

- Let parameterIR$_{subst}$ be parameterIR substituted by a non-empty theta.

Let parameterIR$_{subst}^{*}$ be the resulting list.

b. Return CONTROL_APPLY `( parameterIR$_{subst}^{*}$ `).

8. Else:
   a. Let tableApplyMethodTypeIR be callableTypeIR.
   b. Return tableApplyMethodTypeIR.

callableTypeDefIR substituted by set<pair<typeId, typeIR>>

1. If set<pair<typeId, typeIR>> is equal to `{ · `}: return callableTypeDefIR.
2. Otherwise:
   a. Return callableTypeDefIR substituted by a non-empty set<pair<typeId, typeIR>>.

callableTypeDefIR substituted by a non-empty theta

1. If let actionTypeDefIR of type actionTypeDefIR be callableTypeDefIR:
   a. Let callableTypeIR be actionTypeDefIR substituted by a non-empty theta.
   b. Check that callableTypeIR has type actionTypeDefIR.
   c. Let actionTypeDefIR$_{subst}$ be callableTypeIR.
   d. Return actionTypeDefIR$_{subst}$.
2. Else if let FUNCTION nameIR `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) ':' typeIR of type definedFunctionTypeDefIR be callableTypeDefIR:
   a. Let B be free type identifiers in FUNCTION nameIR `( parameterIR$^{*}$ `) ':' typeIR.
   b. Let (theta$_{fresh}$, typeParameterIR$_{expl_fresh}^{*}$, typeParameterIR$_{impl_fresh}^{*}$) be $capture_avoiding(theta, typeParameterIR$_{expl}^{*}$, typeParameterIR$_{impl}^{*}$, B).
   c. For each parameterIR in parameterIR*:

      - Let parameterIR$_{subst}$ be parameterIR substituted by theta$_{fresh}$.

      Let parameterIR$_{subst}^{*}$ be the resulting list.

   d. Let typeIR$_{subst}$ be typeIR substituted by theta$_{fresh}$.
   e. Return FUNCTION nameIR `< typeParameterIR$_{expl\_fresh}^{*}$ ',' typeParameterIR$_{impl\_fresh}^{*}$ `> `( parameterIR$_{subst}^{*}$ `) ':' typeIR$_{subst}$.
3. Else if let EXTERN_FUNCTION nameIR `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) ':' typeIR of type externFunctionTypeDefIR be callableTypeDefIR:
   a. Let B be free type identifiers in EXTERN_FUNCTION nameIR `( parameterIR$^{*}$ `) ':' typeIR.
   b. Let (theta$_{fresh}$, typeParameterIR$_{expl_fresh}^{*}$, typeParameterIR$_{impl_fresh}^{*}$) be $capture_avoiding(theta, typeParameterIR$_{expl}^{*}$, typeParameterIR$_{impl}^{*}$, B).
   c. For each parameterIR in parameterIR*:

      - Let parameterIR$_{subst}$ be parameterIR substituted by theta$_{fresh}$.

      Let parameterIR$_{subst}^{*}$ be the resulting list.

   d. Let typeIR$_{subst}$ be typeIR substituted by theta$_{fresh}$.

e. Return EXTERN_FUNCTION nameIR `< $typeParameterIR_{expl\_fresh}^{*}$ ',' $typeParameterIR_{impl\_fresh}^{*}$ `> `( $parameterIR_{subst}^{*}$ `) ':' $typeIR_{subst}$.

4. Else if let externMethodTypeDefIR of type externMethodTypeDefIR be callableTypeDefIR:
   a. If let EXTERN_METHOD nameIR `< $typeParameterIR_{expl}^{*}$ ',' $typeParameterIR_{impl}^{*}$ `> `( $parameterIR^{*}$ `) ':' typeIR be externMethodTypeDefIR:
      i. Let B be free type identifiers in EXTERN_METHOD nameIR `( $parameterIR^{*}$ `) ':' typeIR.
      ii. Let ($theta_{fresh}$, $typeParameterIR_{expl_fresh}^{*}$, $typeParameterIR_{impl_fresh}^{*}$) be $capture_avoiding(theta, $typeParameterIR_{expl}^{*}$, $typeParameterIR_{impl}^{*}$, B).
      iii. For each parameterIR in $parameterIR^{*}$:
         • Let $parameterIR_{subst}$ be parameterIR substituted by $theta_{fresh}$.

         Let $parameterIR_{subst}^{*}$ be the resulting list.
      iv. Let $typeIR_{subst}$ be typeIR substituted by $theta_{fresh}$.
      v. Return EXTERN_METHOD nameIR `< $typeParameterIR_{expl\_fresh}^{*}$ ',' $typeParameterIR_{impl\_fresh}^{*}$ `> `( $parameterIR_{subst}^{*}$ `) ':' $typeIR_{subst}$.
   b. Else:
      i. Let EXTERN_METHOD ABSTRACT nameIR `< $typeParameterIR_{expl}^{*}$ ',' $typeParameterIR_{impl}^{*}$ `> `( $parameterIR^{*}$ `) ':' typeIR be externMethodTypeDefIR.
      ii. Let B be free type identifiers in EXTERN_METHOD ABSTRACT nameIR `( $parameterIR^{*}$ `) ':' typeIR.
      iii. Let ($theta_{fresh}$, $typeParameterIR_{expl_fresh}^{*}$, $typeParameterIR_{impl_fresh}^{*}$) be $capture_avoiding(theta, $typeParameterIR_{expl}^{*}$, $typeParameterIR_{impl}^{*}$, B).
      iv. For each parameterIR in $parameterIR^{*}$:
         • Let $parameterIR_{subst}$ be parameterIR substituted by $theta_{fresh}$.

         Let $parameterIR_{subst}^{*}$ be the resulting list.
      v. Let $typeIR_{subst}$ be typeIR substituted by $theta_{fresh}$.
      vi. Return EXTERN_METHOD ABSTRACT nameIR `< $typeParameterIR_{expl\_fresh}^{*}$ ',' $typeParameterIR_{impl\_fresh}^{*}$ `> `( $parameterIR_{subst}^{*}$ `) ':' $typeIR_{subst}$.
5. Else if let parserApplyMethodTypeDefIR of type parserApplyMethodTypeDefIR be callableTypeDefIR:
   a. Let callableTypeIR be parserApplyMethodTypeDefIR substituted by a non-empty theta.
   b. Check that callableTypeIR has type parserApplyMethodTypeDefIR.
   c. Let $parserApplyMethodTypeDefIR_{subst}$ be callableTypeIR.
   d. Return $parserApplyMethodTypeDefIR_{subst}$.
6. Else if let controlApplyMethodTypeDefIR of type controlApplyMethodTypeDefIR be callableTypeDefIR:
   a. Let callableTypeIR be controlApplyMethodTypeDefIR substituted by a non-empty theta.
   b. Check that callableTypeIR has type controlApplyMethodTypeDefIR.
   c. Let $controlApplyMethodTypeDefIR_{subst}$ be callableTypeIR.
   d. Return $controlApplyMethodTypeDefIR_{subst}$.

constructorTypeIR substituted by set<pair<typeId, typeIR>>

1. If set<pair<typeId, typeIR>> is equal to `{ · `}: return constructorTypeIR.
2. Otherwise:

a. Return constructorTypeIR substituted by a non-empty set<pair<typeId, typeIR>>.

CONSTRUCTOR `( constructorParameterIR* `) ':' typeIR substituted by a non-empty theta

1. For each constructorParameterIR in constructorParameterIR*:
   - Let $constructorParameterIR_{subst}$ be constructorParameterIR substituted by a non-empty theta.

   Let $constructorParameterIR_{subst}^{*}$ be the resulting list.
2. Let $typeIR_{subst}$ be typeIR substituted by a non-empty theta.
3. Return CONSTRUCTOR `( $constructorParameterIR_{subst}^{*}$ `) ':' $typeIR_{subst}$.

Substitution is also applied to extern method environments and to the type bindings of a context with:

Substitution of abstract methods in $externMethodTypeDefEnv_{extern}$ by implementations in set<pair<callableId, externMethodTypeDefIR>>

1. Try:
   a. If set<pair<callableId, externMethodTypeDefIR>> is equal to `{ · `}: return $externMethodTypeDefEnv_{extern}$.
2. Then, try:
   a. Let `{ pair<callableId, externMethodTypeDefIR>* `} be set<pair<callableId, externMethodTypeDefIR>>.
   b. Check that pair<callableId, externMethodTypeDefIR>* is a non-empty list.
   c. Let $callableId_{init_h}$ ':' $externMethodTypeDefIR_{init_h}$ :: ($callableId_{init_t}$ ':' $externMethodTypeDefIR_{init_t}$)* be pair<callableId, externMethodTypeDefIR>*.
   d. Let $nameIR_{init_h}$ `( _* `) be $callableId_{init_h}$.
   e. Let (callableId, externMethodTypeDefIR)? be $find_non_overloaded<externMethodTypeDefIR>($externMethodTypeDefEnv_{extern}$, $nameIR_{init_h}$).
   f. Check that (callableId, externMethodTypeDefIR)? is defined.
   g. Let (callableId, $externMethodTypeDefIR_{abstract}$) be (callableId, externMethodTypeDefIR)?.
   h. Check that $externMethodTypeDefIR_{abstract}$ matches pattern EXTERN_METHOD ABSTRACT % `< % ',' % `> `( % `) ':' %.
   i. Let EXTERN_METHOD ABSTRACT nameIR `< $typeId_{expl}^{*}$ ',' $typeId_{impl}^{*}$ `> `( $parameterIR^{*}$ `) ':' $typeIR_{ret}$ be $externMethodTypeDefIR_{abstract}$.
   j. Let $externMethodTypeDefIR_{concrete}$ be EXTERN_METHOD nameIR `< $typeId_{expl}^{*}$ ',' $typeId_{impl}^{*}$ `> `( $parameterIR^{*}$ `) ':' $typeIR_{ret}$.
   k. If $externMethodTypeDefIR_{concrete}$ and $externMethodTypeDefIR_{init_h}$ are the same type:
      i. Let $externMethodTypeDefEnv_{extern_subst}$ be the map $externMethodTypeDefEnv_{extern}$ with the value for callableId is updated to $externMethodTypeDefIR_{concrete}$.
      ii. Return substitution of abstract methods in $externMethodTypeDefEnv_{extern_subst}$ by implementations in `{ ( $callableId_{init\_t}$ ':' $externMethodTypeDefIR_{init\_t}$ )* `}.

typeIR substituted by bound type variables in EC from the p layer

1. If p matches pattern GLOBAL: return typeIR.
2. Else if p matches pattern BLOCK:
   a.

Let theta be EC.BLOCK.TYPE.

b. Let $typeIR_{subst}$ be typeIR substituted by theta.

c. Return $typeIR_{subst}$.

3. Else:

a. Let $theta_{block}$ be EC.BLOCK.TYPE.

b. Let $theta_{local}$ be EC.LOCAL.TYPE.

c. Let theta be $extend_map<typeId, typeIR>($theta_{block}$, $theta_{local}$).

d. Let $typeIR_{subst}$ be typeIR substituted by theta.

e. Return $typeIR_{subst}$.

typeIR substituted by bound type variables in IC from the p layer

1. If p matches pattern GLOBAL: return typeIR.
2. Else if p matches pattern BLOCK:

a. Let theta be IC.BLOCK.TYPE.

b. Let $typeIR_{subst}$ be typeIR substituted by theta.

c. Return $typeIR_{subst}$.

3. Else:

a. Let $theta_{block}$ be IC.BLOCK.TYPE.

b. Let $theta_{local}$ be IC.LOCAL.TYPE.

c. Let theta be $extend_map<typeId, typeIR>($theta_{block}$, $theta_{local}$).

d. Let $typeIR_{subst}$ be typeIR substituted by theta.

e. Return $typeIR_{subst}$.

Appendix J: Subexpressions

The immediate subexpressions of a typed expression, an expression, and an argument are collected with:

The sub-expressions of typedExpressionIR

1. Let expressionIR be the expression of typedExpressionIR.
2. Let typedExpressionListIR be the sub-expressions of expressionIR.
3. Return typedExpressionIR :: typedExpressionListIR.

The sub-expressions of $typedExpressionIR^*$

1. For each typedExpressionIR in $typedExpressionIR^*$:

- Let typedExpressionListIR be the sub-expressions of typedExpressionIR.

Let $typedExpressionListIR^*$ be the resulting list.

2. Return $concat_<typedExpressionIR>($typedExpressionListIR^*$).

The sub-expressions of expressionIR

1. If let unop typedExpressionIR of type unaryExpressionIR be expressionIR:
    a. Return the sub-expressions of typedExpressionIR.
2. Else if let typedExpressionIR$_l$ binop typedExpressionIR$_r$ of type binaryExpressionIR be expressionIR:
    a. Let typedExpressionListIR' be the sub-expressions of typedExpressionIR$_l$.
    b. Let typedExpressionListIR'' be the sub-expressions of typedExpressionIR$_r$.
    c. Return typedExpressionListIR' concatenated with typedExpressionListIR''.
3. Else if let typedExpressionIR$_{cond}$ '?' typedExpressionIR$_{then}$ ':' typedExpressionIR$_{else}$ of type ternaryExpressionIR be expressionIR:
    a. Let typedExpressionListIR''' be the sub-expressions of typedExpressionIR$_{cond}$.
    b. Let typedExpressionListIR'''' be the sub-expressions of typedExpressionIR$_{then}$.
    c. Let typedExpressionListIR''''' be the sub-expressions of typedExpressionIR$_{else}$.
    d. Return typedExpressionListIR''' concatenated with typedExpressionListIR'''' concatenated with typedExpressionListIR'''''.
4. Else if let `( typeIR `) typedExpressionIR of type castExpressionIR be expressionIR:
    a. Return the sub-expressions of typedExpressionIR.
5. Else if let sequenceExpressionIR of type sequenceExpressionIR be expressionIR:
    a. If let SEQ `{ typedExpressionListIR `} be sequenceExpressionIR:
        i. Return the sub-expressions of typedExpressionListIR.
    b. Else:
        i. Let SEQ `{ typedExpressionListIR ',' '...' `} be sequenceExpressionIR.
        ii. Return the sub-expressions of typedExpressionListIR.
6. Else if let recordExpressionIR of type recordExpressionIR be expressionIR:
    a. If let RECORD `{ ( nameIR '=' typedExpressionIR )$^*$ `} be recordExpressionIR:
        i. Return the sub-expressions of typedExpressionIR*.
    b. Else:
        i. Let RECORD `{ ( nameIR '=' typedExpressionIR )$^*$ ',' '...' `} be recordExpressionIR.
        ii. Return the sub-expressions of typedExpressionIR*.
7. Else if let memberAccessBaseIR '.' nameIR of type memberAccessExpressionIR be expressionIR:
    a. If let typedExpressionIR of type typedExpressionIR be memberAccessBaseIR:
        i. Return the sub-expressions of typedExpressionIR.
8. Else if let typedExpressionIR$_{base}$ `[ typedExpressionIR$_{index}$ `] of type indexAccessExpressionIR be expressionIR:
    a. Let typedExpressionListIR'''''' be the sub-expressions of typedExpressionIR$_{base}$.
    b. Let typedExpressionListIR''''''' be the sub-expressions of typedExpressionIR$_{index}$.
    c. Return typedExpressionListIR'''''' concatenated with typedExpressionListIR'''''''.
9. Else if let typedExpressionIR$_{base}$ `[ typedExpressionIR$_{hi}$ sliceop typedExpressionIR$_{lo}$ `] of type sliceAccessExpressionIR be expressionIR:
    a. If sliceop is ':':
        i. Let typedExpressionListIR'''''''' be the sub-expressions of typedExpressionIR$_{base}$.
        ii. Let typedExpressionListIR''''''''' be the sub-expressions of typedExpressionIR$_{hi}$.
        iii. Let typedExpressionListIR'''''''''' be the sub-expressions of typedExpressionIR$_{lo}$.

iv. Return typedExpressionListIR'''''''' concatenated with typedExpressionListIR''''''''' concatenated with typedExpressionListIR'''''''''.

10. Else if let callExpressionIR of type callExpressionIR be expressionIR:
    a. If let constructorTargetIR `( argumentIR* `) be callExpressionIR:
        i. Return the sub-expressions of argumentIR*.
    b. Else:
        i. Let callableTargetIR `< typeArgumentIR* `> `( argumentIR* `) be callExpressionIR.
        ii. Return the sub-expressions of argumentIR*.
11. Else if let `( typedExpressionIR `) of type parenthesizedExpressionIR be expressionIR:
    a. Return the sub-expressions of typedExpressionIR.
12. Otherwise: return ·.

The sub-expressions of argumentIR

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:
    a. Return the sub-expressions of typedExpressionIR.
2. Else if let nameIR '=' typedExpressionIR be argumentIR:
    a. Return the sub-expressions of typedExpressionIR.
3. Otherwise: return ·.

The sub-expressions of argumentIR*

1. For each argumentIR in argumentIR*:
    - Let typedExpressionListIR be the sub-expressions of argumentIR.

    Let typedExpressionListIR* be the resulting list.
2. Return $concat_<typedExpressionIR>(typedExpressionListIR*).

Appendix K: Flattening and unflattening

The surface grammar and $P4_{IR}$ represent list-shaped constructs with recursive productions. A right-recursive list associates to the right, so its head is the first element and its tail is the remaining list; a left-recursive list associates to the left, accumulating elements as it grows. To manipulate lists uniformly, each recursive list is flattened into a sequence and later reconstructed into its original left- or right-recursive shape by an unflattening function. Unflattening a left-recursive list uses a primed helper that carries an accumulator, and some constructs distinguish an empty from a non-empty list.

Names

nameList' flattened

1. If let name of type name be nameList': return name.
2. Else:

a. Let nameList ',' name be nameList'.

b. Let name'* be nameList flattened.

c. Return name'* concatenated with name.

$unflatten_nameList(name*)

1. Check that name* is a non-empty list.
2. Let $name_h$ :: $name_t^*$ be name*.
3. Return $unflatten_nameList'(name_h, name_t^*).

$unflatten_nameList'(nameList, name*)

1. If name* matches pattern []: return nameList.
2. Else:
   a. Let $name_h$ :: $name_t^*$ be name*.
   b. Return $unflatten_nameList'(nameList ',' $name_h$, $name_t^*$).

Prefixed names

The string form of prefixedNameIR

1. If let $_{BARE}$ nameIR be prefixedNameIR: return nameIR.
2. Else:
   a. Let '.' nameIR be prefixedNameIR.
   b. Return "." concatenated with nameIR.

Type fields

typeFieldList' flattened

1. If typeFieldList' matches pattern $_{EMPTY}$: return ·.
2. Else:
   a. Let typeFieldList typeField be typeFieldList'.
   b. Let typeField'* be typeFieldList flattened.
   c. Return typeField'* concatenated with typeField.

$unflatten_typeFieldList(typeField*)

1. Return $unflatten_typeFieldList'($_{EMPTY}$, typeField*).

$unflatten_typeFieldList'(typeFieldList, typeField*)

1. If typeField* matches pattern []: return typeFieldList.
2. Else:
    a. Let $typeField_h$ :: $typeField_t^*$ be typeField*.
    b. Return $unflatten_typeFieldList'(typeFieldList $typeField_h$, $typeField_t^*$).

Type parameters

typeParameterList' flattened

1. If let typeParameter of type typeParameter be typeParameterList': return typeParameter.
2. Else:
    a. Let typeParameterList ',' typeParameter be typeParameterList'.
    b. Let typeParameter'* be typeParameterList flattened.
    c. Return typeParameter'* concatenated with typeParameter.

$unflatten_typeParameterList(typeParameter*)

1. Check that typeParameter* is a non-empty list.
2. Let $typeParameter_h$ :: $typeParameter_t^*$ be typeParameter*.
3. Return $unflatten_typeParameterList'(typeParameter$_h$, $typeParameter_t^*$).

$unflatten_typeParameterList'(typeParameterList, typeParameter*)

1. If typeParameter* matches pattern []: return typeParameterList.
2. Else:
    a. Let $typeParameter_h$ :: $typeParameter_t^*$ be typeParameter*.
    b. Return $unflatten_typeParameterList'(typeParameterList ',' typeParameter$_h$, $typeParameter_t^*$).

Optional type parameters

typeParameterListOpt flattened

1. If typeParameterListOpt matches pattern $_{EMPTY}$: return ·.
2. Else:
    a. Let `< typeParameterList `> be typeParameterListOpt.
    b. Return typeParameterList flattened.

$unflatten_typeParameterListOpt(typeParameter*)

1. If typeParameter* matches pattern []: return $_{EMPTY}$.
2. Else:
    a. Let $typeParameter_h$ :: $typeParameter_t^*$ be typeParameter*.

b. Let typeParameterList be $unflatten_typeParameterList(typeParameter$_h$ :: typeParameter$_t^*$).

c. Return `< typeParameterList `>.

Parameters

parameterList flattened

1. Try:
   a. If parameterList matches pattern $_{\text{EMPTY}}$: return ·.
   b. Else if let parameter of type parameter be parameterList: return parameter.
2. Then, try:
   a. Check that parameterList has type nonEmptyParameterList.
   b. Let nonEmptyParameterList' be parameterList.
   c. Check that nonEmptyParameterList' matches pattern % ',' %.
   d. Let nonEmptyParameterList ',' parameter be nonEmptyParameterList'.
   e. Let parameter'* be nonEmptyParameterList flattened.
   f. Return parameter'* concatenated with parameter.

$unflatten_nonEmptyParameterList(parameter*)

1. Check that parameter* is a non-empty list.
2. Let parameter$_h$ :: parameter$_t^*$ be parameter*.
3. Return $unflatten_nonEmptyParameterList'(parameter$_h$, parameter$_t^*$).

$unflatten_nonEmptyParameterList'(nonEmptyParameterList, parameter*)

1. If parameter* matches pattern []: return nonEmptyParameterList.
2. Else:
   a. Let parameter$_h$ :: parameter$_t^*$ be parameter*.
   b. Return $unflatten_nonEmptyParameterList'(nonEmptyParameterList ',' parameter$_h$, parameter$_t^*$).

$unflatten_parameterList(parameter*)

1. If parameter* matches pattern []: return $_{\text{EMPTY}}$.
2. Else:
   a. Let parameter$_h$ :: parameter$_t^*$ be parameter*.
   b. Let nonEmptyParameterList be $unflatten_nonEmptyParameterList(parameter$_h$ :: parameter$_t^*$).
   c. Return nonEmptyParameterList.

Constructor parameters

constructorParameterListOpt flattened

1. If constructorParameterListOpt matches pattern $_{\text{EMPTY}}$: return ·.
2. Else:
   a. Let `( parameterList `) be constructorParameterListOpt.
   b. Return parameterList flattened.

$unflatten_constructorParameterList(constructorParameter*)

1. If constructorParameter* matches pattern []: return $_{\text{EMPTY}}$.
2. Else:
   a. Let parameterList be $unflatten_parameterList(constructorParameter*).
   b. Return `( parameterList `).

Named expressions

namedExpressionList' flattened

1. If let namedExpression of type namedExpression be namedExpressionList': return namedExpression.
2. Else:
   a. Let namedExpressionList ',' namedExpression be namedExpressionList'.
   b. Let namedExpression'* be namedExpressionList flattened.
   c. Return namedExpression'* concatenated with namedExpression.

$unflatten_namedExpressionList(namedExpression*)

1. Check that namedExpression* is a non-empty list.
2. Let namedExpression_h :: namedExpression_t* be namedExpression*.
3. Return $unflatten_namedExpressionList'(namedExpression_h, namedExpression_t*).

$unflatten_namedExpressionList'(namedExpressionList, namedExpression*)

1. If namedExpression* matches pattern []: return namedExpressionList.
2. Else:
   a. Let namedExpression_h :: namedExpression_t* be namedExpression*.
   b. Return $unflatten_namedExpressionList'(namedExpressionList ',' namedExpression_h, namedExpression_t*).

Expressions

expressionList' flattened

1. If expressionList' matches pattern $_{\text{EMPTY}}$: return ·.
2. Else if let expression of type expression be expressionList': return expression.
3. Else:
    a. Let expressionList ',' expression be expressionList'.
    b. Let expression'* be expressionList flattened.
    c. Return expression'* concatenated with expression.

$unflatten_expressionList(expression*)

1. If expression* matches pattern []: return $_{\text{EMPTY}}$.
2. Else:
    a. Let expression$_{h}$ :: expression$_{t}^{*}$ be expression*.
    b. Return $unflatten_expressionList'(expression$_{h}$, expression$_{t}^{*}$).

$unflatten_expressionList'(expressionList, expression*)

1. If expression* matches pattern []: return expressionList.
2. Else:
    a. Let expression$_{h}$ :: expression$_{t}^{*}$ be expression*.
    b. Return $unflatten_expressionList'(expressionList ',' expression$_{h}$, expression$_{t}^{*}$).

Record element expressions

$unflatten_recordElementExpression(namedExpression*)

1. If namedExpression* matches pattern [_/1]:
    a. Let namedExpression$_{h}$ be namedExpression*.
    b. Let name$_{h}$ '=' expression$_{h}$ be namedExpression$_{h}$.
    c. Return name$_{h}$ '=' expression$_{h}$.
2. Else if namedExpression* matches pattern :::
    a. Let namedExpression$_{h}$ :: namedExpression$_{t}^{*}$ be namedExpression*.
    b. Check that namedExpression$_{t}^{*}$ is a non-empty list.
    c. Let name$_{h}$ '=' expression$_{h}$ be namedExpression$_{h}$.
    d. Let namedExpressionList be $unflatten_namedExpressionList(namedExpression$_{t}^{*}$).
    e. Return name$_{h}$ '=' expression$_{h}$ ',' namedExpressionList.

$unflatten_recordElementExpression_with_default(namedExpression*)

1. If namedExpression* matches pattern [_/1]:
    a. Let namedExpression$_{h}$ be namedExpression*.
    b. Let name$_{h}$ '=' expression$_{h}$ be namedExpression$_{h}$.
    c. Return name$_{h}$ '=' expression$_{h}$ ',' '...'.

2. Else if namedExpression* matches pattern :::
    a. Let $namedExpression_h$ :: $namedExpression_t^*$ be namedExpression*.
    b. Check that $namedExpression_t^*$ is a non-empty list.
    c. Let $name_h$ '=' $expression_h$ be $namedExpression_h$.
    d. Let namedExpressionList be $unflatten_namedExpressionList($namedExpression_t^*$).
    e. Return $name_h$ '=' $expression_h$ ',' namedExpressionList ',' '...'.

Simple keyset expressions

simpleKeysetExpressionList' flattened

1. If let simpleKeysetExpression of type simpleKeysetExpression be simpleKeysetExpressionList': return simpleKeysetExpression.
2. Else:
    a. Let simpleKeysetExpressionList ',' simpleKeysetExpression be simpleKeysetExpressionList'.
    b. Let simpleKeysetExpression'* be simpleKeysetExpressionList flattened.
    c. Return simpleKeysetExpression'* concatenated with simpleKeysetExpression.

$unflatten_simpleKeysetExpressionList(simpleKeysetExpression*)

1. Check that simpleKeysetExpression* is a non-empty list.
2. Let $simpleKeysetExpression_h$ :: $simpleKeysetExpression_t^*$ be simpleKeysetExpression*.
3. Return $unflatten_simpleKeysetExpressionList'($simpleKeysetExpression_h$, $simpleKeysetExpression_t^*$).

$unflatten_simpleKeysetExpressionList'(simpleKeysetExpressionList, simpleKeysetExpression*)

1. If simpleKeysetExpression* matches pattern []: return simpleKeysetExpressionList.
2. Else:
    a. Let $simpleKeysetExpression_h$ :: $simpleKeysetExpression_t^*$ be simpleKeysetExpression*.
    b. Return $unflatten_simpleKeysetExpressionList'(simpleKeysetExpressionList ',' $simpleKeysetExpression_h$, $simpleKeysetExpression_t^*$).

Keyset expressions

The flattened keyset expression list of keysetExpressionIR

1. If let simpleKeysetExpressionIR of type simpleKeysetExpressionIR be keysetExpressionIR: return simpleKeysetExpressionIR.
2. Else:
    a. Let `( simpleKeysetExpressionListIR `) be keysetExpressionIR.
    b. Return simpleKeysetExpressionListIR.

Real type arguments

realTypeArgumentList' flattened

1. If let realTypeArgument of type realTypeArgument be realTypeArgumentList': return realTypeArgument.
2. Else:
    a. Let realTypeArgumentList ',' realTypeArgument be realTypeArgumentList'.
    b. Let realTypeArgument'* be realTypeArgumentList flattened.
    c. Return realTypeArgument'* concatenated with realTypeArgument.

$unflatten_realTypeArgumentList(realTypeArgument*)

1. Check that realTypeArgument* is a non-empty list.
2. Let $\text{realTypeArgument}_h$ :: $\text{realTypeArgument}_t^*$ be realTypeArgument*.
3. Return $\$\text{unflatten_realTypeArgumentList}'(\text{realTypeArgument}_h, \text{realTypeArgument}_t^*)$.

$unflatten_realTypeArgumentList'(realTypeArgumentList, realTypeArgument*)

1. If realTypeArgument* matches pattern []: return realTypeArgumentList.
2. Else:
    a. Let $\text{realTypeArgument}_h$ :: $\text{realTypeArgument}_t^*$ be realTypeArgument*.
    b. Return $\$\text{unflatten_realTypeArgumentList}'(\text{realTypeArgumentList ',' realTypeArgument}_h, \text{realTypeArgument}_t^*)$.

Type arguments

typeArgumentList' flattened

1. If typeArgumentList' matches pattern EMPTY: return ·.
2. Else if let typeArgument of type typeArgument be typeArgumentList': return typeArgument.
3. Else:
    a. Let typeArgumentList ',' typeArgument be typeArgumentList'.
    b. Let typeArgument'* be typeArgumentList flattened.
    c. Return typeArgument'* concatenated with typeArgument.

$unflatten_typeArgumentList(typeArgument*)

1. If typeArgument* matches pattern []: return EMPTY.
2. Else:
    a. Let typeArgument_h :: typeArgument_t^* be typeArgument*.
    b. Return $\$\text{unflatten_typeArgumentList}'(\text{typeArgument}_h, \text{typeArgument}_t^*)$.

$unflatten_typeArgumentList'(typeArgumentList, typeArgument*)

1. If typeArgument* matches pattern []: return typeArgumentList.
2. Else:
    a. Let $typeArgument_h$ :: $typeArgument_t^*$ be typeArgument*.
    b. Return $unflatten_typeArgumentList'(typeArgumentList ',' $typeArgument_h$, $typeArgument_t^*$).

Arguments

argumentList flattened

1. Try:
    a. If argumentList matches pattern $_{EMPTY}$: return ·.
    b. Else if let argument of type argument be argumentList: return argument.
2. Then, try:
    a. Check that argumentList has type argumentListNonEmpty.
    b. Let argumentListNonEmpty' be argumentList.
    c. Check that argumentListNonEmpty' matches pattern % ',' %.
    d. Let argumentListNonEmpty ',' argument be argumentListNonEmpty'.
    e. Let argument'* be argumentListNonEmpty flattened.
    f. Return argument'* concatenated with argument.

$unflatten_argumentList(argument*)

1. If argument* matches pattern []: return $_{EMPTY}$.
2. Else:
    a. Let $argument_h$ :: $argument_t^*$ be argument*.
    b. Let argumentListNonEmpty be $unflatten_argumentListNonEmpty($argument_h$ :: $argument_t^*$).
    c. Return argumentListNonEmpty.

$unflatten_argumentListNonEmpty(argument*)

1. Check that argument* is a non-empty list.
2. Let $argument_h$ :: $argument_t^*$ be argument*.
3. Return $unflatten_argumentListNonEmpty'($argument_h$, $argument_t^*$).

$unflatten_argumentListNonEmpty'(argumentListNonEmpty, argument*)

1. If argument* matches pattern []: return argumentListNonEmpty.
2. Else:
    a. Let $argument_h$ :: $argument_t^*$ be argument*.
    b. Return $unflatten_argumentListNonEmpty'(argumentListNonEmpty ',' $argument_h$, $argument_t^*$).

For-update statements

$flatten_forUpdateStatementList(forUpdateStatementList)

1. Try:
   a. If forUpdateStatementList matches pattern $_{\text{EMPTY}}$: return ·.
   b. Else if let forUpdateStatement of type forUpdateStatement be forUpdateStatementList: return forUpdateStatement.
2. Then, try:
   a. Check that forUpdateStatementList has type forUpdateStatementListNonEmpty.
   b. Let forUpdateStatementListNonEmpty' be forUpdateStatementList.
   c. Check that forUpdateStatementListNonEmpty' matches pattern % ',' %.
   d. Let forUpdateStatementListNonEmpty ',' forUpdateStatement be forUpdateStatementListNonEmpty'.
   e. Let forUpdateStatement'* be $flatten_forUpdateStatementList(forUpdateStatementListNonEmpty).
   f. Return forUpdateStatement'* concatenated with forUpdateStatement.

For-init statements

$flatten_forInitStatementList(forInitStatementList)

1. Try:
   a. If forInitStatementList matches pattern $_{\text{EMPTY}}$: return ·.
   b. Else if let forInitStatement of type forInitStatement be forInitStatementList: return forInitStatement.
2. Then, try:
   a. Check that forInitStatementList has type forInitStatementListNonEmpty.
   b. Let forInitStatementListNonEmpty' be forInitStatementList.
   c. Check that forInitStatementListNonEmpty' matches pattern % ',' %.
   d. Let forInitStatementListNonEmpty ',' forInitStatement be forInitStatementListNonEmpty'.
   e. Let forInitStatement'* be $flatten_forInitStatementList(forInitStatementListNonEmpty).
   f. Return forInitStatement'* concatenated with forInitStatement.

Switch cases

switchCaseList' flattened

1. If switchCaseList' matches pattern $_{\text{EMPTY}}$: return ·.
2. Else:
   a. Let switchCaseList switchCase be switchCaseList'.
   b. Let switchCase'* be switchCaseList flattened.
   c. Return switchCase'* concatenated with switchCase.

$unflatten_switchCaseList(switchCase*)

1. Return $unflatten_switchCaseList'($_{\text{EMPTY}}$, switchCase*).

$unflatten_switchCaseList'(switchCaseList, switchCase*)

1. If switchCase* matches pattern []: return switchCaseList.
2. Else:
   a. Let switchCase$_h$ :: switchCase$_t^*$ be switchCase*.
   b. Return $unflatten_switchCaseList'(switchCaseList switchCase$_h$, switchCase$_t^*$).

Block element statements

blockElementStatementList' flattened

1. If blockElementStatementList' matches pattern $_{\text{EMPTY}}$: return ·.
2. Else:
   a. Let blockElementStatementList blockElementStatement be blockElementStatementList'.
   b. Let blockElementStatement'* be blockElementStatementList flattened.
   c. Return blockElementStatement'* concatenated with blockElementStatement.

$unflatten_blockElementStatementList(blockElementStatement*)

1. Return $unflatten_blockElementStatementList'($_{\text{EMPTY}}$, blockElementStatement*).

$unflatten_blockElementStatementList'(blockElementStatementList, blockElementStatement*)

1. If blockElementStatement* matches pattern []: return blockElementStatementList.
2. Else:
   a. Let blockElementStatement$_h$ :: blockElementStatement$_t^*$ be blockElementStatement*.
   b. Return $unflatten_blockElementStatementList'(blockElementStatementList blockElementStatement$_h$, blockElementStatement$_t^*$).

Object declarations

objectDeclarationList' flattened

1. If objectDeclarationList' matches pattern $_{\text{EMPTY}}$: return ·.
2. Else:
   a. Let objectDeclarationList objectDeclaration be objectDeclarationList'.
   b. Let objectDeclaration'* be objectDeclarationList flattened.
   c. Return objectDeclaration'* concatenated with objectDeclaration.

$unflatten_objectDeclarationList(objectDeclaration*)

1. Return $unflatten_objectDeclarationList'($_{EMPTY}$, objectDeclaration*).

$unflatten_objectDeclarationList'(objectDeclarationList, objectDeclaration*)

1. If objectDeclaration* matches pattern []: return objectDeclarationList.
2. Else:
   a. Let objectDeclaration$_{h}$ :: objectDeclaration$_{t}^{*}$ be objectDeclaration*.
   b. Return $unflatten_objectDeclarationList'(objectDeclarationList objectDeclaration$_{h}$, objectDeclaration$_{t}^{*}$).

Optional object initializer

The list of object declarations in objectInitializerOptIR

1. Try:
   a. Let objectInitializerIR$^{?}$ be objectInitializerOptIR.
   b. If objectInitializerIR$^{?}$ is none: return ·.
2. Then, try:
   a. Check that objectInitializerOptIR is defined.
   b. Let '=' `{ objectDeclarationListIR `} be objectInitializerOptIR.
   c. Return objectDeclarationListIR.

Extern constructor and method prototypes

externConstructorOrMethodPrototypeList' flattened

1. If externConstructorOrMethodPrototypeList' matches pattern $_{EMPTY}$: return ·.
2. Else:
   a. Let externConstructorOrMethodPrototypeList externConstructorOrMethodPrototype be externConstructorOrMethodPrototypeList'.
   b. Let externConstructorOrMethodPrototype'* be externConstructorOrMethodPrototypeList flattened.
   c. Return externConstructorOrMethodPrototype'* concatenated with externConstructorOrMethodPrototype.

$unflatten_externConstructorOrMethodPrototypeList(externConstructorOrMethodPrototype*)

1. Return $unflatten_externConstructorOrMethodPrototypeList'($_{EMPTY}$, externConstructorOrMethodPrototype*).

$unflatten_externConstructorOrMethodPrototypeList'(externConstructorOrMethodPrototypeList, externConstructorOrMethodPrototype*)

1. If externConstructorOrMethodPrototype* matches pattern []:
   a. Return externConstructorOrMethodPrototypeList.
2. Else:
   a. Let externConstructorOrMethodPrototype$_{h}$ :: externConstructorOrMethodPrototype$_{t}^{*}$ be

externConstructorOrMethodPrototype*.

b. Return $unflatten_externConstructorOrMethodPrototypeList'(externConstructorOrMethodPrototypeList externConstructorOrMethodPrototype$_h$, externConstructorOrMethodPrototype$_t^*$).

Select cases

selectCaseList' flattened

1. If selectCaseList' matches pattern $_{\text{EMPTY}}$: return ·.
2. Else:
   a. Let selectCaseList selectCase be selectCaseList'.
   b. Let selectCase'* be selectCaseList flattened.
   c. Return selectCase'* concatenated with selectCase.

$unflatten_selectCaseList(selectCase*)

1. Return $unflatten_selectCaseList'($_{\text{EMPTY}}$, selectCase*).

$unflatten_selectCaseList'(selectCaseList, selectCase*)

1. If selectCase* matches pattern []: return selectCaseList.
2. Else:
   a. Let selectCase$_h$ :: selectCase$_t^*$ be selectCase*.
   b. Return $unflatten_selectCaseList'(selectCaseList selectCase$_h$, selectCase$_t^*$).

Parser block element statements

parserBlockElementStatementList' flattened

1. If parserBlockElementStatementList' matches pattern $_{\text{EMPTY}}$: return ·.
2. Else:
   a. Let parserBlockElementStatementList parserBlockElementStatement be parserBlockElementStatementList'.
   b. Let parserBlockElementStatement'* be parserBlockElementStatementList flattened.
   c. Return parserBlockElementStatement'* concatenated with parserBlockElementStatement.

$unflatten_parserBlockElementStatementList(parserBlockElementStatement*)

1. Return $unflatten_parserBlockElementStatementList'($_{\text{EMPTY}}$, parserBlockElementStatement*).

$unflatten_parserBlockElementStatementList'(parserBlockElementStatementList, parserBlockElementStatement*)

1. If parserBlockElementStatement* matches pattern []:
   a. Return parserBlockElementStatementList.

2. Else:
    a. Let parserBlockElementStatement$_h$ :: parserBlockElementStatement$_t^*$ be parserBlockElementStatement*.
    b. Return $unflatten_parserBlockElementStatementList'(parserBlockElementStatementList parserBlockElementStatement$_h$, parserBlockElementStatement$_t^*$).

Parser states

parserStateList' flattened

1. If let parserState of type parserState be parserStateList': return parserState.
2. Else:
    a. Let parserStateList parserState be parserStateList'.
    b. Let parserState'* be parserStateList flattened.
    c. Return parserState'* concatenated with parserState.

$unflatten_parserStateList(parserState*)

1. Check that parserState* is a non-empty list.
2. Let parserState$_h$ :: parserState$_t^*$ be parserState*.
3. Return $unflatten_parserStateList'(parserState$_h$, parserState$_t^*$).

$unflatten_parserStateList'(parserStateList, parserState*)

1. If parserState* matches pattern []: return parserStateList.
2. Else:
    a. Let parserState$_h$ :: parserState$_t^*$ be parserState*.
    b. Return $unflatten_parserStateList'(parserStateList parserState$_h$, parserState$_t^*$).

Parser local declarations

parserLocalDeclarationList' flattened

1. If parserLocalDeclarationList' matches pattern $_{EMPTY}$: return ·.
2. Else:
    a. Let parserLocalDeclarationList parserLocalDeclaration be parserLocalDeclarationList'.
    b. Let parserLocalDeclaration'* be parserLocalDeclarationList flattened.
    c. Return parserLocalDeclaration'* concatenated with parserLocalDeclaration.

$unflatten_parserLocalDeclarationList(parserLocalDeclaration*)

1. Return $unflatten_parserLocalDeclarationList'($_{EMPTY}$, parserLocalDeclaration*).

$unflatten_parserLocalDeclarationList'(parserLocalDeclarationList, parserLocalDeclaration*)

1. If parserLocalDeclaration* matches pattern []: return parserLocalDeclarationList.
2. Else:
    a. Let $parserLocalDeclaration_h :: parserLocalDeclaration_t^*$ be parserLocalDeclaration*.
    b. Return $unflatten_parserLocalDeclarationList'(parserLocalDeclarationList parserLocalDeclaration_h, parserLocalDeclaration_t^*)$.

Optional constant

The optional const qualifier in constOpt

1. If constOpt matches pattern $_{EMPTY}$: return ·.
2. Else:
    a. Let const be constOpt.
    b. Return const.

Table keys

tableKeyList' flattened

1. If tableKeyList' matches pattern $_{EMPTY}$: return ·.
2. Else:
    a. Let tableKeyList tableKey be tableKeyList'.
    b. Let tableKey'* be tableKeyList flattened.
    c. Return tableKey'* concatenated with tableKey.

$unflatten_tableKeyList(tableKey*)

1. Return $unflatten_tableKeyList'($_{EMPTY}$, tableKey*).

$unflatten_tableKeyList'(tableKeyList, tableKey*)

1. If tableKey* matches pattern []: return tableKeyList.
2. Else:
    a. Let $tableKey_h :: tableKey_t^*$ be tableKey*.
    b. Return $unflatten_tableKeyList'(tableKeyList tableKey_h, tableKey_t^*)$.

Table actions

tableActionList' flattened

1. If tableActionList' matches pattern $_{EMPTY}$: return ·.

2. Else:
    a. Let tableActionList tableAction be tableActionList'.
    b. Let tableAction'* be tableActionList flattened.
    c. Return tableAction'* concatenated with tableAction.

$unflatten_tableActionList(tableAction*)

1. Return $unflatten_tableActionList'(EMPTY, tableAction*).

$unflatten_tableActionList'(tableActionList, tableAction*)

1. If tableAction* matches pattern []: return tableActionList.
2. Else:
    a. Let $tableAction_h$ :: $tableAction_t^*$ be tableAction*.
    b. Return $unflatten_tableActionList'(tableActionList $tableAction_h$, $tableAction_t^*$).

Table entries

tableEntryList' flattened

1. If tableEntryList' matches pattern EMPTY: return ·.
2. Else:
    a. Let tableEntryList tableEntry be tableEntryList'.
    b. Let tableEntry'* be tableEntryList flattened.
    c. Return tableEntry'* concatenated with tableEntry.

$unflatten_tableEntryList(tableEntry*)

1. Return $unflatten_tableEntryList'(EMPTY, tableEntry*).

$unflatten_tableEntryList'(tableEntryList, tableEntry*)

1. If tableEntry* matches pattern []: return tableEntryList.
2. Else:
    a. Let $tableEntry_h$ :: $tableEntry_t^*$ be tableEntry*.
    b. Return $unflatten_tableEntryList'(tableEntryList $tableEntry_h$, $tableEntry_t^*$).

Table properties

tablePropertyList' flattened

1. If tablePropertyList' matches pattern EMPTY: return ·.

2. Else:
    a. Let tablePropertyList tableProperty be tablePropertyList'.
    b. Let tableProperty'* be tablePropertyList flattened.
    c. Return tableProperty'* concatenated with tableProperty.

$unflatten_tablePropertyList(tableProperty*)

1. Return $unflatten_tablePropertyList'($_{EMPTY}$, tableProperty*).

$unflatten_tablePropertyList'(tablePropertyList, tableProperty*)

1. If tableProperty* matches pattern []: return tablePropertyList.
2. Else:
    a. Let $tableProperty_h$ :: $tableProperty_t$* be tableProperty*.
    b. Return $unflatten_tablePropertyList'(tablePropertyList $tableProperty_h$, $tableProperty_t$*).

Control local declarations

controlLocalDeclarationList' flattened

1. If controlLocalDeclarationList' matches pattern $_{EMPTY}$: return ·.
2. Else:
    a. Let controlLocalDeclarationList controlLocalDeclaration be controlLocalDeclarationList'.
    b. Let controlLocalDeclaration'* be controlLocalDeclarationList flattened.
    c. Return controlLocalDeclaration'* concatenated with controlLocalDeclaration.

$unflatten_controlLocalDeclarationList(controlLocalDeclaration*)

1. Return $unflatten_controlLocalDeclarationList'($_{EMPTY}$, controlLocalDeclaration*).

$unflatten_controlLocalDeclarationList'(controlLocalDeclarationList, controlLocalDeclaration*)

1. If controlLocalDeclaration* matches pattern []: return controlLocalDeclarationList.
2. Else:
    a. Let $controlLocalDeclaration_h$ :: $controlLocalDeclaration_t$* be controlLocalDeclaration*.
    b. Return $unflatten_controlLocalDeclarationList'(controlLocalDeclarationList $controlLocalDeclaration_h$, $controlLocalDeclaration_t$*).

Annotations

annotationList flattened

1. Try:

a. If annotationList matches pattern $_{\text{EMPTY}}$: return ·.

b. Else if let annotation of type annotation be annotationList: return annotation.

2. Then, try:

a. Check that annotationList has type annotationListNonEmpty.

b. Let annotationListNonEmpty' be annotationList.

c. Check that annotationListNonEmpty' matches pattern % %.

d. Let annotationListNonEmpty annotation be annotationListNonEmpty'.

e. Let annotation'* be annotationListNonEmpty flattened.

f. Return annotation'* concatenated with annotation.

P4 programs

$unflatten_p4program(declaration*)

1. Return $unflatten_p4program'($_{\text{EMPTY}}$, declaration*).

$unflatten_p4program'(p4program, declaration*)

1. If declaration* matches pattern []: return p4program.
2. Else:

a. Let declaration_h :: declaration_t* be declaration*.

b. Return $unflatten_p4program'(p4program declaration_h, declaration_t*).

Appendix L: Lifting P4IR to surface syntax

Each $P4_{IR}$ node is lifted back into the corresponding surface syntax by the following functions, grouped by construct category.

Names

$lift_nameIR_as_identifier(nameIR)

1. Return $_{\text{ID}}$ nameIR.

$lift_nameIR_as_typeIdentifier(nameIR)

1. Return $_{\text{TID}}$ nameIR.

$lift_nameIR_as_nonTypeName(t)

1. If t is equal to "apply": return APPLY.
2. Else if t is equal to "key": return KEY.
3. Else if t is equal to "actions": return ACTIONS.

4. Else if t is equal to "state": return STATE.
5. Else if t is equal to "entries": return ENTRIES.
6. Else if t is equal to "type": return TYPE.
7. Else if t is equal to "priority": return PRIORITY.
8. Otherwise: return $_{ID}$ t.

$lift_prefixedNameIR_as_prefixedNonTypeName(prefixedNameIR)

1. If let $_{BARE}$ nameIR be prefixedNameIR:
   a. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
   b. Return nonTypeName.
2. Else:
   a. Let '.' nameIR be prefixedNameIR.
   b. Let nonTypeName' be $lift_nameIR_as_nonTypeName(nameIR).
   c. Return $_{ID}$ '.' nonTypeName'.

$lift_nameIR_as_typeName(nameIR)

1. Return $lift_nameIR_as_typeIdentifier(nameIR).

$lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR)

1. If let $_{BARE}$ nameIR be prefixedNameIR:
   a. Let typeName be $lift_nameIR_as_typeName(nameIR).
   b. Return typeName.
2. Else:
   a. Let '.' nameIR be prefixedNameIR.
   b. Let typeName' be $lift_nameIR_as_typeName(nameIR).
   c. Return $_{TID}$ '.' typeName'.

$lift_nameIR_as_tableCustomName(t)

1. If t is equal to "apply": return APPLY.
2. Else if t is equal to "state": return STATE.
3. Else if t is equal to "type": return TYPE.
4. Else if t is equal to "priority": return PRIORITY.
5. Otherwise:
   a. Let identifier be $lift_nameIR_as_identifier(t).
   b. Return identifier.

$lift_nameListIR_as_nonTypeNameList(nameIR*)

1. For each nameIR in nameIR*:
    - Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).

    Let nonTypeName* be the resulting list.
2. For each nonTypeName in nonTypeName*:
    - Let name be nonTypeName.

    Let name* be the resulting list.
3. Let nameList be $unflatten_nameList(name*).
4. Return nameList.

$lift_nameIR_as_member(nameIR)

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Return nonTypeName.

Values

$lift_value(value)

1. Otherwise:
    a. (debug: value)
    b. Return $sink<expression>.

Types

$lift_baseTypeIR(baseTypeIR)

1. If let boolTypeIR of type boolTypeIR be baseTypeIR: return BOOL.
2. Else if let errorTypeIR of type errorTypeIR be baseTypeIR:
    a. Try:
        i. Return ERROR.
    b. Then, try:
        i. Return ERROR.
3. Else if let matchKindTypeIR of type matchKindTypeIR be baseTypeIR: return MATCH_KIND.
4. Else if let stringTypeIR of type stringTypeIR be baseTypeIR: return STRING.
5. Else if let intTypeIR of type intTypeIR be baseTypeIR: return INT.
6. Else if let INT `< w `> of type fixedIntTypeIR be baseTypeIR: return INT `< w `>.
7. Else if let BIT `< w `> of type fixedBitTypeIR be baseTypeIR: return BIT `< w `>.
8. Else if let VARBIT `< w `> of type varBitTypeIR be baseTypeIR: return VARBIT `< w `>.

$lift_nameTypeIR(⌷$_{NAME}$ typeId)

1. Let typeName be $lift_nameIR_as_typeName(typeId).
2. Return typeName.

$lift_aliasTypeIR(aliasTypeIR)

1. If let TYPEDEF typeId _ of type typedefTypeIR be aliasTypeIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.
2. Else:
   a. Let TYPE typeId _ be aliasTypeIR.
   b. Let typeName be $lift_nameIR_as_typeName(typeId).
   c. Return typeName.

$lift_dataTypeIR(dataTypeIR)

1. If let LIST `< typeIR `> of type listTypeIR be dataTypeIR:
   a. Let typeArgument be $lift_typeArgumentIR(typeIR).
   b. Return LIST `< typeArgument `>.
2. Else if let TUPLE `< typeIR$^*$ `> of type tupleTypeIR be dataTypeIR:
   a. Let typeArgumentList be $lift_typeArgumentListIR(typeIR*).
   b. Return TUPLE `< typeArgumentList `>.
3. Else if let ARRAY typeIR `[ n `] of type arrayTypeIR be dataTypeIR:
   a. Let type be $lift_typeIR(typeIR).
   b. Return type `[ D n `].
4. Else if let HEADER_STACK typeIR `[ n `] of type headerStackTypeIR be dataTypeIR:
   a. Let type be $lift_typeIR(typeIR).
   b. Return type `[ D n `].
5. Else if let ENUM typeId `{ _$^*$ `} of type simpleEnumTypeIR be dataTypeIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.
6. Else if let ENUM typeId `< typeIR `> `{ _$^*$ `} of type serializableEnumTypeIR be dataTypeIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Let typeArgument be $lift_typeArgumentIR(typeIR).
   c. Return typeName `< typeArgument `>.

$lift_typeIR(typeIR)

1. If let baseTypeIR of type baseTypeIR be typeIR:
   a. Let baseType be $lift_baseTypeIR(baseTypeIR).

b. Return baseType.

2. Else if let nameTypeIR of type nameTypeIR be typeIR:
   a. Let namedType be $lift_nameTypeIR(nameTypeIR).
   b. Return namedType.
3. Else if let aliasTypeIR of type aliasTypeIR be typeIR: return $lift_aliasTypeIR(aliasTypeIR).
4. Else if let dataTypeIR of type dataTypeIR be typeIR: return $lift_dataTypeIR(dataTypeIR).
5. Otherwise:
   a. (debug: typeIR)
   b. Return $sink<type>.

$lift_typeIR_as_typeOrVoid(typeIR)

1. Try:
   a. If let voidTypeIR of type voidTypeIR be typeIR: return VOID.
2. Then, try:
   a. Check that typeIR is not equal to VOID.
   b. Let type be $lift_typeIR(typeIR).
   c. Return type.

Type definitions

$lift_typeDefIR_as_typeName(typeDefIR)

1. If let NAME typeId of type nameTypeIR be typeDefIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.
2. Else if let TYPEDEF typeId _ of type typedefTypeIR be typeDefIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.
3. Else if let TYPE typeId _ of type newTypeIR be typeDefIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.
4. Else if let STRUCT typeId `< _* `> `{ _* `} of type structTypeDefIR be typeDefIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.
5. Else if let HEADER typeId `< _* `> `{ _* `} of type headerTypeDefIR be typeDefIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.
6. Else if let HEADER_UNION typeId `< _* `> `{ _* `} of type headerUnionTypeDefIR be typeDefIR:
   a. Let typeName be $lift_nameIR_as_typeName(typeId).
   b. Return typeName.

7. Else if let ENUM typeId `{ _* `} of type simpleEnumTypeIR be typeDefIR:
    a. Let typeName be $lift_nameIR_as_typeName(typeId).
    b. Return typeName.
8. Else if let ENUM typeId `< _ `> `{ _* `} of type serializableEnumTypeIR be typeDefIR:
    a. Let typeName be $lift_nameIR_as_typeName(typeId).
    b. Return typeName.
9. Else if let EXTERN typeId `< _* ',' _* `> _ of type externObjectTypeDefIR be typeDefIR:
    a. Let typeName be $lift_nameIR_as_typeName(typeId).
    b. Return typeName.
10. Else if let PARSER typeId `< _* ',' _* `> `( _* `) of type parserObjectTypeDefIR be typeDefIR:
    a. Let typeName be $lift_nameIR_as_typeName(typeId).
    b. Return typeName.
11. Else if let CONTROL typeId `< _* ',' _* `> `( _* `) of type controlObjectTypeDefIR be typeDefIR:
    a. Let typeName be $lift_nameIR_as_typeName(typeId).
    b. Return typeName.
12. Else if let PACKAGE typeId `< _* ',' _* `> `{ _* `} of type packageObjectTypeDefIR be typeDefIR:
    a. Let typeName be $lift_nameIR_as_typeName(typeId).
    b. Return typeName.
13. Else:
    a. Let TABLE typeId `{ _ `} be typeDefIR.
    b. Let typeName be $lift_nameIR_as_typeName(typeId).
    c. Return typeName.
14. Otherwise:
    a. (debug: typeDefIR)
    b. Return $sink<typeName>.

Type parameters

$lift_typeParameterIR(typeId)

1. Let typeName be $lift_nameIR_as_typeName(typeId).
2. Let typeParameter be typeName.
3. Return typeParameter.

$lift_typeParameterListIR(typeParameterIR*)

1. For each typeParameterIR in typeParameterIR*:
    - Let typeParameter be $lift_typeParameterIR(typeParameterIR).

    Let typeParameter* be the resulting list.
2. Let typeParameterListOpt be $unflatten_typeParameterListOpt(typeParameter*).

3. Return typeParameterListOpt.

Parameters

$lift_parameterIR(annotationList direction typeIR nameIR parameterInitializerOptIR)

1. Check that parameterInitializerOptIR has type constantInitializerIR?.
2. Let constantInitializerIR? be parameterInitializerOptIR.
3. Let type be $lift_typeIR(typeIR).
4. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
5. Let name be nonTypeName.
6. Let initializerOpt be $lift_constantInitializerOptIR_as_initializerOpt(constantInitializerIR?).
7. Return annotationList direction type name initializerOpt.

$lift_parameterListIR(parameterIR*)

1. For each parameterIR in parameterIR*:
   - Let parameter be $lift_parameterIR(parameterIR).

   Let parameter* be the resulting list.
2. Let parameterList be $unflatten_parameterList(parameter*).
3. Return parameterList.

Constructor parameters

$lift_constructorParameterIR(constructorParameterIR)

1. Return $lift_parameterIR(constructorParameterIR).

$lift_constructorParameterListIR(constructorParameterIR*)

1. For each constructorParameterIR in constructorParameterIR*:
   - Let constructorParameter be $lift_constructorParameterIR(constructorParameterIR).

   Let constructorParameter* be the resulting list.
2. Let constructorParameterListOpt be $unflatten_constructorParameterList(constructorParameter*).
3. Return constructorParameterListOpt.

Initializers

$lift_initializerIR('=' typedExpressionIR)

1. Let expression be $lift_typedExpressionIR(typedExpressionIR).
2. Return '=' expression.

$lift_initializerOptIR(initializerIR'?)

1. If initializerIR'? matches pattern (): return EMPTY.
2. Else:
   a. Let initializerIR be initializerIR'?.
   b. Let initializer be $lift_initializerIR(initializerIR).
   c. Return initializer.

$lift_constantInitializerIR_as_initializer('=' VALUE value)

1. Let expression be $lift_value(value).
2. Return '=' expression.

$lift_constantInitializerOptIR_as_initializerOpt(constantInitializerIR'?)

1. If constantInitializerIR'? matches pattern (): return EMPTY.
2. Else:
   a. Let constantInitializerIR be constantInitializerIR'?.
   b. Let initializer be $lift_constantInitializerIR_as_initializer(constantInitializerIR).
   c. Return initializer.

Expression key-value pairs

$lift_namedExpressionIR(nameIR '=' typedExpressionIR)

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let expression be $lift_typedExpressionIR(typedExpressionIR).
4. Return name '=' expression.

$lift_namedExpressionListIR(namedExpressionIR*)

1. For each namedExpressionIR in namedExpressionIR*:
   - Let namedExpression be $lift_namedExpressionIR(namedExpressionIR).

   Let namedExpression* be the resulting list.
2. Let namedExpressionList be $unflatten_namedExpressionList(namedExpression*).
3. Return namedExpressionList.

$lift_namedValueIR(nameIR '=' value)

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let expression be $lift_value(value).
4. Return name '=' expression.

$lift_namedValueListIR(namedValueIR*)

1. For each namedValueIR in namedValueIR*:
   - Let namedExpression be $lift_namedValueIR(namedValueIR).

   Let namedExpression* be the resulting list.
2. Let namedExpressionList be $unflatten_namedExpressionList(namedExpression*).
3. Return namedExpressionList.

Expressions

$lift_literalExpressionIR(literalExpression)

1. Return literalExpression.

$lift_referenceExpressionIR(prefixedNameIR)

1. If let $_{\text{BARE}}$ nameIR be prefixedNameIR:
   a. If nameIR is equal to "this": return THIS.
   b. Else:
      i. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
      ii. Return nonTypeName.
2. Else:
   a. Let '.' nameIR be prefixedNameIR.
   b. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
   c. Return $_{\text{ID}}$ '.' nonTypeName.

$lift_defaultExpressionIR(defaultExpression)

1. Return '...'.

$lift_unaryExpressionIR(unop typedExpressionIR)

1. Let expression be $lift_typedExpressionIR(typedExpressionIR).
2. Return unop expression.

$lift_binaryExpressionIR(typedExpressionIR$_l$ binop typedExpressionIR$_r$)

1. Let expression$_l$ be $lift_typedExpressionIR(typedExpressionIR$_l$).
2. Let expression$_r$ be $lift_typedExpressionIR(typedExpressionIR$_r$).
3. Return expression$_l$ binop expression$_r$.

$lift_ternaryExpressionIR(typedExpressionIR$_{cond}$ '?' typedExpressionIR$_{then}$ ':' typedExpressionIR$_{else}$)

1. Let expression$_{cond}$ be $lift_typedExpressionIR(typedExpressionIR$_{cond}$).
2. Let expression$_{then}$ be $lift_typedExpressionIR(typedExpressionIR$_{then}$).
3. Let expression$_{else}$ be $lift_typedExpressionIR(typedExpressionIR$_{else}$).
4. Return expression$_{cond}$ '?' expression$_{then}$ ':' expression$_{else}$.

$lift_castExpressionIR(` ( typeIR `) typedExpressionIR)

1. Let typeIR$_{unroll}$ be typeIR with typedefs unrolled.
2. If typeIR$_{unroll}$ has type setTypeIR:
   a. Let expression be $lift_typedExpressionIR(typedExpressionIR).
   b. Return expression.
3. Else:
   a. Let type be $lift_typeIR(typeIR).
   b. Let expression be $lift_typedExpressionIR(typedExpressionIR).
   c. Return `( type `) expression.

$lift_dataExpressionIR(dataExpressionIR)

1. If let invalidHeaderExpression of type invalidHeaderExpression be dataExpressionIR: return '{#}'.
2. Else if let sequenceExpressionIR of type sequenceExpressionIR be dataExpressionIR:
   a. If let SEQ `{ typedExpressionIR* `} be sequenceExpressionIR:
      i. Let expressionList be $lift_typedExpressionListIR(typedExpressionIR*).
      ii. Return `{ expressionList $_{EMPTY}$ `}.
   b. Else:
      i. Let SEQ `{ typedExpressionIR* ',' '...' `} be sequenceExpressionIR.
      ii. For each typedExpressionIR in typedExpressionIR*:
          - Let expression be $lift_typedExpressionIR(typedExpressionIR).

          Let expression* be the resulting list.
      iii. Let expressionList be $lift_typedExpressionListIR(typedExpressionIR*).
      iv. Return `{ expressionList $_{EMPTY}$ `}.
3. Else:
   a. Let recordExpressionIR be dataExpressionIR.
   b. If let RECORD `{ namedExpressionIR* `} be recordExpressionIR:

i. For each namedExpressionIR in namedExpressionIR*:

- Let namedExpression be $lift_namedExpressionIR(namedExpressionIR).

Let namedExpression* be the resulting list.

ii. Let recordElementExpression be $unflatten_recordElementExpression(namedExpression*).

iii. Return `{ recordElementExpression EMPTY `}.

c. Else:

i. Let RECORD `{ namedExpressionIR* ',' '...' `} be recordExpressionIR.

ii. For each namedExpressionIR in namedExpressionIR*:

- Let namedExpression be $lift_namedExpressionIR(namedExpressionIR).

Let namedExpression* be the resulting list.

iii. Let recordElementExpression be $unflatten_recordElementExpression_with_default(namedExpression*).

iv. Return `{ recordElementExpression EMPTY `}.

Member and index access expressions

$lift_errorAccessExpressionIR(ERROR '.' nameIR)

1. Let member be $lift_nameIR_as_member(nameIR).
2. Return ERROR '.' member.

$lift_memberAccessBaseIR(memberAccessBaseIR)

1. If let TYPE prefixedNameIR be memberAccessBaseIR:
   a. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
   b. Return prefixedTypeName.
2. Else:
   a. Let typedExpressionIR be memberAccessBaseIR.
   b. Let expression be $lift_typedExpressionIR(typedExpressionIR).
   c. Return expression.

$lift_memberAccessExpressionIR(memberAccessBaseIR '.' nameIR)

1. Let memberAccessBase be $lift_memberAccessBaseIR(memberAccessBaseIR).
2. Let member be $lift_nameIR_as_member(nameIR).
3. Return memberAccessBase '.' member.

$lift_indexAccessExpressionIR($typedExpressionIR_{base}$ `[ $typedExpressionIR_{index}$ `])

1. Let $expression_{base}$ be $lift_typedExpressionIR($typedExpressionIR_{base}$).
2. Let $expression_{index}$ be $lift_typedExpressionIR($typedExpressionIR_{index}$).

3. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{index}}$ `].

$lift_sliceAccessExpressionIR(typedExpressionIR$_{\text{base}}$ `[ typedExpressionIR$_{\text{hi}}$ sliceop typedExpressionIR$_{\text{lo}}$ `])

1. Check that sliceop is ':'.
2. Let $\text{expression}_{\text{base}}$ be $lift_typedExpressionIR(typedExpressionIR$_{\text{base}}$).
3. Let $\text{expression}_{\text{hi}}$ be $lift_typedExpressionIR(typedExpressionIR$_{\text{hi}}$).
4. Let $\text{expression}_{\text{lo}}$ be $lift_typedExpressionIR(typedExpressionIR$_{\text{lo}}$).
5. Return $\text{expression}_{\text{base}}$ `[ $\text{expression}_{\text{hi}}$ ':' $\text{expression}_{\text{lo}}$ `].

$lift_accessExpressionIR(accessExpressionIR)

1. If let errorAccessExpressionIR of type errorAccessExpressionIR be accessExpressionIR:
    a. Let errorAccessExpression be $lift_errorAccessExpressionIR(errorAccessExpressionIR).
    b. Return errorAccessExpression.
2. Else if let memberAccessExpressionIR of type memberAccessExpressionIR be accessExpressionIR:
    a. Let memberAccessExpression be $lift_memberAccessExpressionIR(memberAccessExpressionIR).
    b. Return memberAccessExpression.
3. Else if let indexAccessExpressionIR of type indexAccessExpressionIR be accessExpressionIR:
    a. Let indexAccessExpression be $lift_indexAccessExpressionIR(indexAccessExpressionIR).
    b. Return indexAccessExpression.
4. Else:
    a. Let sliceAccessExpressionIR be accessExpressionIR.
    b. Let sliceAccessExpression be $lift_sliceAccessExpressionIR(sliceAccessExpressionIR).
    c. Return sliceAccessExpression.

$lift_callExpressionIR(callExpressionIR)

1. If let constructorTargetIR `( $\text{argumentIR}^*$ `) be callExpressionIR:
    a. Let prefixedNameIR `< $\text{typeArgumentIR}^*$ `> be constructorTargetIR.
    b. If typeArgumentIR^* matches pattern []:
        i. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
        ii. Let namedType be prefixedTypeName.
        iii. Let argumentList be $lift_argumentListIR($\text{argumentIR}^*$).
        iv. Return namedType `( argumentList `).
    c. Else:
        i. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
        ii. Let typeArgumentList be $lift_typeArgumentListIR($\text{typeArgumentIR}^*$).
        iii. Let namedType be prefixedTypeName `< typeArgumentList `>.
        iv. Let argumentList be $lift_argumentListIR($\text{argumentIR}^*$).
        v. Return namedType `( argumentList `).

2. Else:
    a. Let callableTargetIR `< typeArgumentIR* `> `( argumentIR* `) be callExpressionIR.
    b. Try:
        i. If typeArgumentIR* is an empty list:
            A. Let expression be $lift_callableTargetIR_as_expression(callableTargetIR).
            B. Let argumentList be $lift_argumentListIR(argumentIR*).
            C. Return expression `( argumentList `).
    c. Then, try:
        i. Let expression be $lift_callableTargetIR_as_expression(callableTargetIR).
        ii. Check that typeArgumentIR* is a non-empty list.
        iii. Let realTypeArgumentList be $lift_typeArgumentListIR_as_realTypeArgumentList(typeArgumentIR*).
        iv. Let argumentList be $lift_argumentListIR(argumentIR*).
        v. Return expression `< realTypeArgumentList `> `( argumentList `).

$lift_parenthesizedExpressionIR(`( typedExpressionIR `))

1. Let expression be $lift_typedExpressionIR(typedExpressionIR).
2. Return `( expression `).

$lift_expressionIR(expressionIR)

1. If let literalExpressionIR of type literalExpressionIR be expressionIR:
    a. Let literalExpression be $lift_literalExpressionIR(literalExpressionIR).
    b. Return literalExpression.
2. Else if let referenceExpressionIR of type referenceExpressionIR be expressionIR:
    a. Let referenceExpression be $lift_referenceExpressionIR(referenceExpressionIR).
    b. Return referenceExpression.
3. Else if let defaultExpressionIR of type defaultExpressionIR be expressionIR:
    a. Let defaultExpression be $lift_defaultExpressionIR(defaultExpressionIR).
    b. Return defaultExpression.
4. Else if let unaryExpressionIR of type unaryExpressionIR be expressionIR:
    a. Let unaryExpression be $lift_unaryExpressionIR(unaryExpressionIR).
    b. Return unaryExpression.
5. Else if let binaryExpressionIR of type binaryExpressionIR be expressionIR:
    a. Let binaryExpression be $lift_binaryExpressionIR(binaryExpressionIR).
    b. Return binaryExpression.
6. Else if let ternaryExpressionIR of type ternaryExpressionIR be expressionIR:
    a. Let ternaryExpression be $lift_ternaryExpressionIR(ternaryExpressionIR).
    b. Return ternaryExpression.
7. Else if let castExpressionIR of type castExpressionIR be expressionIR:
    a. Return $lift_castExpressionIR(castExpressionIR).

8. Else if let dataExpressionIR of type dataExpressionIR be expressionIR:
    a. Let dataExpression be $lift_dataExpressionIR(dataExpressionIR).
    b. Return dataExpression.
9. Else if let accessExpressionIR of type accessExpressionIR be expressionIR:
    a. Let accessExpression be $lift_accessExpressionIR(accessExpressionIR).
    b. Return accessExpression.
10. Else if let callExpressionIR of type callExpressionIR be expressionIR:
    a. Let callExpression be $lift_callExpressionIR(callExpressionIR).
    b. Return callExpression.
11. Else:
    a. Let parenthesizedExpressionIR be expressionIR.
    b. Let parenthesizedExpression be $lift_parenthesizedExpressionIR(parenthesizedExpressionIR).
    c. Return parenthesizedExpression.

$lift_typedExpressionIR(expressionIR '#' _)

1. Return $lift_expressionIR(expressionIR).

$lift_typedExpressionListIR(typedExpressionIR*)

1. For each typedExpressionIR in typedExpressionIR*:
    - Let expression be $lift_typedExpressionIR(typedExpressionIR).

    Let expression* be the resulting list.
2. Let expressionList be $unflatten_expressionList(expression*).
3. Return expressionList.

Keyset expressions

$lift_simpleKeysetExpressionIR(simpleKeysetExpressionIR)

1. If let typedExpressionIR of type typedExpressionIR be simpleKeysetExpressionIR:
    a. Let expression be $lift_typedExpressionIR(typedExpressionIR).
    b. Return expression.
2. Else if let $typedExpressionIR_{base}$ '&&&' $typedExpressionIR_{mask}$ be simpleKeysetExpressionIR:
    a. Let $expression_{base}$ be $lift_typedExpressionIR($typedExpressionIR_{base}$).
    b. Let $expression_{mask}$ be $lift_typedExpressionIR($typedExpressionIR_{mask}$).
    c. Return $expression_{base}$ '&&&' $expression_{mask}$.
3. Else if let $typedExpressionIR_{lb}$ '..' $typedExpressionIR_{ub}$ be simpleKeysetExpressionIR:
    a. Let $expression_{lb}$ be $lift_typedExpressionIR($typedExpressionIR_{lb}$).
    b. Let $expression_{ub}$ be $lift_typedExpressionIR($typedExpressionIR_{ub}$).

c. Return expression$_{lb}$ '..' expression$_{ub}$.

4. Else if simpleKeysetExpressionIR matches pattern DEFAULT: return DEFAULT.
5. Else: return '_'.

$lift_simpleKeysetExpressionListIR(simpleKeysetExpressionIR*)

1. For each simpleKeysetExpressionIR in simpleKeysetExpressionIR*:

   - Let simpleKeysetExpression be $lift_simpleKeysetExpressionIR(simpleKeysetExpressionIR).

   Let simpleKeysetExpression* be the resulting list.

2. Let simpleKeysetExpressionList be $unflatten_simpleKeysetExpressionList(simpleKeysetExpression*).
3. Return simpleKeysetExpressionList.

$lift_tupleKeysetExpressionIR(tupleKeysetExpressionIR)

1. Try:
   a. Let `( simpleKeysetExpressionListIR `) be tupleKeysetExpressionIR.
   b. If let [+simpleKeysetExpressionIR]+ be simpleKeysetExpressionListIR:
      i. If let typedExpressionIR$_{base}$ '&&&' typedExpressionIR$_{mask}$ be simpleKeysetExpressionIR:
         A. Let expression$_{base}$ be $lift_typedExpressionIR(typedExpressionIR$_{base}$).
         B. Let expression$_{mask}$ be $lift_typedExpressionIR(typedExpressionIR$_{mask}$).
         C. Return `( expression$_{base}$ '&&&' expression$_{mask}$ `).
      ii. Else if let typedExpressionIR$_{lb}$ '..' typedExpressionIR$_{ub}$ be simpleKeysetExpressionIR:
         A. Let expression$_{lb}$ be $lift_typedExpressionIR(typedExpressionIR$_{lb}$).
         B. Let expression$_{ub}$ be $lift_typedExpressionIR(typedExpressionIR$_{ub}$).
         C. Return `( expression$_{lb}$ '..' expression$_{ub}$ `).
2. Then, try:
   a. If tupleKeysetExpressionIR is equal to `( DEFAULT `): return `( DEFAULT `).
   b. Else if tupleKeysetExpressionIR is equal to `( '_' `): return `( '_' `).
3. Then, try:
   a. Let `( simpleKeysetExpressionIR$^{*}$ `) be tupleKeysetExpressionIR.
   b. Check that the length of simpleKeysetExpressionIR* is greater than 1.
   c. Check that simpleKeysetExpressionIR* is a non-empty list.
   d. Let simpleKeysetExpressionIR$_{h}$ :: simpleKeysetExpressionIR$_{t}^{*}$ be simpleKeysetExpressionIR*.
   e. Let simpleKeysetExpression$_{h}$ be $lift_simpleKeysetExpressionIR(simpleKeysetExpressionIR$_{h}$).
   f. Let simpleKeysetExpressionList$_{t}$ be $lift_simpleKeysetExpressionListIR(simpleKeysetExpressionIR$_{t}^{*}$).
   g. Return `( simpleKeysetExpression$_{h}$ ',' simpleKeysetExpressionList$_{t}$ `).

$lift_keysetExpressionIR(keysetExpressionIR)

1. If let simpleKeysetExpressionIR of type simpleKeysetExpressionIR be keysetExpressionIR:
   a.

Let simpleKeysetExpression be $lift_simpleKeysetExpressionIR(simpleKeysetExpressionIR).

b. Return simpleKeysetExpression.

2. Else:

a. Let tupleKeysetExpressionIR be keysetExpressionIR.

b. Let tupleKeysetExpression be $lift_tupleKeysetExpressionIR(tupleKeysetExpressionIR).

c. Return tupleKeysetExpression.

Type arguments

$lift_typeArgumentIR_as_realTypeArgument(typeIR)

1. If let voidTypeIR of type voidTypeIR be typeIR: return VOID.
2. Otherwise:

a. Let type be $lift_typeIR(typeIR).

b. Return type.

$lift_typeArgumentIR(typeIR)

1. Let realTypeArgument be $lift_typeArgumentIR_as_realTypeArgument(typeIR).
2. Return realTypeArgument.

$lift_typeArgumentListIR(typeArgumentIR*)

1. For each typeArgumentIR in typeArgumentIR*:

- Let typeArgument be $lift_typeArgumentIR(typeArgumentIR).

Let typeArgument* be the resulting list.

2. Let typeArgumentList be $unflatten_typeArgumentList(typeArgument*).
3. Return typeArgumentList.

$lift_typeArgumentListIR_as_realTypeArgumentList(typeArgumentIR*)

1. For each typeArgumentIR in typeArgumentIR*:

- Let realTypeArgument be $lift_typeArgumentIR_as_realTypeArgument(typeArgumentIR).

Let realTypeArgument* be the resulting list.

2. Let realTypeArgumentList be $unflatten_realTypeArgumentList(realTypeArgument*).
3. Return realTypeArgumentList.

Arguments

$lift_argumentIR(argumentIR)

1. If let typedExpressionIR of type typedExpressionIR be argumentIR:
    a. Let expression be $lift_typedExpressionIR(typedExpressionIR).
    b. Return expression.
2. Else if let nameIR '=' typedExpressionIR be argumentIR:
    a. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
    b. Let name be nonTypeName.
    c. Let expression be $lift_typedExpressionIR(typedExpressionIR).
    d. Return name '=' expression.
3. Else if let nameIR '=' '_' be argumentIR:
    a. Let nonTypeName' be $lift_nameIR_as_nonTypeName(nameIR).
    b. Let name be nonTypeName'.
    c. Return name '=' '_'.
4. Else: return '_'.

$lift_argumentListIR(argumentIR*)

1. For each argumentIR in argumentIR*:
    - Let argument be $lift_argumentIR(argumentIR).

    Let argument* be the resulting list.
2. Let argumentList be $unflatten_argumentList(argument*).
3. Return argumentList.

Statements

$lift_emptyStatementIR(emptyStatement)

1. Return ';'.

$lift_assignmentStatementIR(typedLvalueIR assignop typedExpressionIR ';')

1. Let lvalue be $lift_typedLvalueIR(typedLvalueIR).
2. Let expression be $lift_typedExpressionIR(typedExpressionIR).
3. Return lvalue assignop expression ';'.

$lift_callStatementIR(callableTargetIR `< typeArgumentIR* `> `( argumentIR* `) ';')

1. Try:

a. If typeArgumentIR* is an empty list:
    i. Let lvalue be $lift_callableTargetIR_as_lvalue(callableTargetIR).
    ii. Let argumentList be $lift_argumentListIR(argumentIR*).
    iii. Return lvalue `( argumentList `) ';'.
2. Then, try:
    a. Let lvalue be $lift_callableTargetIR_as_lvalue(callableTargetIR).
    b. Let typeArgumentList be $lift_typeArgumentListIR(typeArgumentIR*).
    c. Let argumentList be $lift_argumentListIR(argumentIR*).
    d. Return lvalue `< typeArgumentList `> `( argumentList `) ';'.

$lift_directApplicationStatementIR(constructorTargetIR '.' APPLY `( argumentIR* `) ';')

1. Let prefixedNameIR `< typeArgumentIR* `> be constructorTargetIR.
2. Try:
    a. If typeArgumentIR* is an empty list:
        i. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
        ii. Let argumentList be $lift_argumentListIR(argumentIR*).
        iii. Return prefixedTypeName '.' APPLY `( argumentList `) ';'.
3. Then, try:
    a. Check that the length of typeArgumentIR* is greater than 0.
    b. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
    c. Let typeArgumentList be $lift_typeArgumentListIR(typeArgumentIR*).
    d. Let namedType be prefixedTypeName `< typeArgumentList `>.
    e. Let argumentList be $lift_argumentListIR(argumentIR*).
    f. Return namedType '.' APPLY `( argumentList `) ';'.

$lift_returnStatementIR(returnStatementIR)

1. If returnStatementIR matches pattern RETURN ';': return RETURN ';'.
2. Else:
    a. Let RETURN typedExpressionIR ';' be returnStatementIR.
    b. Let expression be $lift_typedExpressionIR(typedExpressionIR).
    c. Return RETURN expression ';'.

$lift_exitStatementIR(exitStatement)

1. Return EXIT ';'.

$lift_blockElementStatementIR(blockElementStatementIR)

1. If let constantDeclarationIR of type constantDeclarationIR be blockElementStatementIR:
    a. Let constantDeclaration be $lift_constantDeclarationIR(constantDeclarationIR).

b. Return constantDeclaration.

2. Else if let variableDeclarationIR of type variableDeclarationIR be blockElementStatementIR:
   a. Let variableDeclaration be $lift_variableDeclarationIR(variableDeclarationIR).
   b. Return variableDeclaration.
3. Else:
   a. Let statementIR be blockElementStatementIR.
   b. Let statement be $lift_statementIR(statementIR).
   c. Return statement.

$lift_blockElementStatementListIR(blockElementStatementIR*)

1. For each blockElementStatementIR in blockElementStatementIR*:
   - Let blockElementStatement be $lift_blockElementStatementIR(blockElementStatementIR).

   Let blockElementStatement* be the resulting list.
2. Let blockElementStatementList be $unflatten_blockElementStatementList(blockElementStatement*).
3. Return blockElementStatementList.

$lift_blockStatementIR(annotationList `{ blockElementStatementIR* `})

1. For each blockElementStatementIR in blockElementStatementIR*:
   - Let blockElementStatement be $lift_blockElementStatementIR(blockElementStatementIR).

   Let blockElementStatement* be the resulting list.
2. Let blockElementStatementList be $unflatten_blockElementStatementList(blockElementStatement*).
3. Return annotationList `{ blockElementStatementList `}.

$lift_conditionalStatementIR(conditionalStatementIR)

1. If let IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then}$ be conditionalStatementIR:
   a. Let expression$_{cond}$ be $lift_typedExpressionIR(typedExpressionIR$_{cond}$).
   b. Let statement$_{then}$ be $lift_statementIR(statementIR$_{then}$).
   c. Return IF `( expression$_{cond}$ `) statement$_{then}$.
2. Else:
   a. Let IF `( typedExpressionIR$_{cond}$ `) statementIR$_{then}$ ELSE statementIR$_{else}$ be conditionalStatementIR.
   b. Let expression$_{cond}$ be $lift_typedExpressionIR(typedExpressionIR$_{cond}$).
   c. Let statement$_{then}$ be $lift_statementIR(statementIR$_{then}$).
   d. Let statement$_{else}$ be $lift_statementIR(statementIR$_{else}$).
   e. Return IF `( expression$_{cond}$ `) statement$_{then}$ ELSE statement$_{else}$.

$lift_switchLabelIR(switchLabelIR)

1. If switchLabelIR matches pattern DEFAULT: return DEFAULT.
2. Else:
    a. Let typedExpressionIR be switchLabelIR.
    b. Let expression be $lift_typedExpressionIR(typedExpressionIR).
    c. Let expressionNonBrace be the non-brace expression corresponding to expression.
    d. Return expressionNonBrace.

$lift_switchCaseIR(switchCaseIR)

1. If let switchLabelIR ':' blockStatementIR be switchCaseIR:
    a. Let switchLabel be $lift_switchLabelIR(switchLabelIR).
    b. Let blockStatement be $lift_blockStatementIR(blockStatementIR).
    c. Return switchLabel ':' blockStatement.
2. Else:
    a. Let switchLabelIR ':' be switchCaseIR.
    b. Let switchLabel be $lift_switchLabelIR(switchLabelIR).
    c. Return switchLabel ':'.

$lift_switchCaseListIR(switchCaseIR*)

1. For each switchCaseIR in switchCaseIR*:
    - Let switchCase be $lift_switchCaseIR(switchCaseIR).

    Let switchCase* be the resulting list.
2. Let switchCaseList be $unflatten_switchCaseList(switchCase*).
3. Return switchCaseList.

$lift_switchStatementIR(SWITCH `( typedExpressionIR `) `{ switchCaseIR* `})

1. Let expression be $lift_typedExpressionIR(typedExpressionIR).
2. Let switchCaseList be $lift_switchCaseListIR(switchCaseIR*).
3. Return SWITCH `( expression `) `{ switchCaseList `}.

$lift_breakStatementIR(breakStatement)

1. Return BREAK ';'.

$lift_continueStatementIR(continueStatement)

1. Return CONTINUE ';'.

$lift_statementIR(statementIR)

1. If let emptyStatementIR of type emptyStatementIR be statementIR:
    a. Let emptyStatement be $lift_emptyStatementIR(emptyStatementIR).
    b. Return emptyStatement.
2. Else if let assignmentStatementIR of type assignmentStatementIR be statementIR:
    a. Let assignmentStatement be $lift_assignmentStatementIR(assignmentStatementIR).
    b. Return assignmentStatement.
3. Else if let callStatementIR of type callStatementIR be statementIR:
    a. Let callStatement be $lift_callStatementIR(callStatementIR).
    b. Return callStatement.
4. Else if let directApplicationStatementIR of type directApplicationStatementIR be statementIR:
    a. Let directApplicationStatement be $lift_directApplicationStatementIR(directApplicationStatementIR).
    b. Return directApplicationStatement.
5. Else if let returnStatementIR of type returnStatementIR be statementIR:
    a. Let returnStatement be $lift_returnStatementIR(returnStatementIR).
    b. Return returnStatement.
6. Else if let exitStatementIR of type exitStatementIR be statementIR:
    a. Let exitStatement be $lift_exitStatementIR(exitStatementIR).
    b. Return exitStatement.
7. Else if let blockStatementIR of type blockStatementIR be statementIR:
    a. Let blockStatement be $lift_blockStatementIR(blockStatementIR).
    b. Return blockStatement.
8. Else if let conditionalStatementIR of type conditionalStatementIR be statementIR:
    a. Let conditionalStatement be $lift_conditionalStatementIR(conditionalStatementIR).
    b. Return conditionalStatement.
9. Else if let switchStatementIR of type switchStatementIR be statementIR:
    a. Let switchStatement be $lift_switchStatementIR(switchStatementIR).
    b. Return switchStatement.
10. Otherwise:
    a. (debug: statementIR)
    b. Return $sink<statement>.

L-values

$lift_typedLvalueIR(lvalueIR '#' _)

1. Try:
    a. If lvalueIR is equal to BARE "this": return THIS.
2. Then, try:
    a. If let prefixedNameIR of type prefixedNameIR be lvalueIR:

i. If let $_{BARE}$ nameIR be prefixedNameIR:

A. Check that nameIR is not equal to "this".

B. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).

C. Return nonTypeName.

ii. Else:

A. Let '.' nameIR be prefixedNameIR.

B. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).

C. Return $_{ID}$ '.' nonTypeName.

b. Else if let typedLvalueIR '.' nameIR be lvalueIR:

i. Let lvalue be $lift_typedLvalueIR(typedLvalueIR).

ii. Let member be $lift_nameIR_as_member(nameIR).

iii. Return lvalue '.' member.

c. Else if let typedLvalueIR `[ typedExpressionIR `] be lvalueIR:

i. Let lvalue be $lift_typedLvalueIR(typedLvalueIR).

ii. Let expression be $lift_typedExpressionIR(typedExpressionIR).

iii. Return lvalue `[ expression `].

d. Else if let typedLvalueIR `[ $typedExpressionIR_{hi}$ sliceop $typedExpressionIR_{lo}$ `] be lvalueIR:

i. Check that sliceop is ':'.

ii. Let lvalue be $lift_typedLvalueIR(typedLvalueIR).

iii. Let $expression_{hi}$ be $lift_typedExpressionIR($typedExpressionIR_{hi}$).

iv. Let $expression_{lo}$ be $lift_typedExpressionIR($typedExpressionIR_{lo}$).

v. Return lvalue `[ $expression_{hi}$ ':' $expression_{lo}$ `].

e. Else:

i. Let `( typedLvalueIR `) be lvalueIR.

ii. Let lvalue be $lift_typedLvalueIR(typedLvalueIR).

iii. Return `( lvalue `).

$lift_callableTargetIR_as_expression(callableTargetIR')

1. Try:

a. If callableTargetIR' is equal to $_{BARE}$ "this": return THIS.

2. Then, try:

a. If let prefixedNameIR of type prefixedNameIR be callableTargetIR':

i. If let $_{BARE}$ nameIR be prefixedNameIR:

A. Check that nameIR is not equal to "this".

B. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).

C. Return nonTypeName.

ii. Else:

A. Let '.' nameIR be prefixedNameIR.

B. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).

C. Return $_{ID}$ '.' nonTypeName.

b. Else if let typedExpressionIR '.' nameIR be callableTargetIR':
    i. Let expression be $lift_typedExpressionIR(typedExpressionIR).
    ii. Let member be $lift_nameIR_as_member(nameIR).
    iii. Return expression '.' member.
c. Else if let TYPE prefixedNameIR '.' nameIR be callableTargetIR':
    i. Try:
        A. If prefixedNameIR is equal to BARE "this":
            I. Let member be $lift_nameIR_as_member(nameIR).
            II. Return THIS '.' member.
    ii. Then, try:
        A. If let BARE nameIR' be prefixedNameIR:
            I. Check that nameIR' is not equal to "this".
            II. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR').
            III. Let member be $lift_nameIR_as_member(nameIR).
            IV. Return nonTypeName '.' member.
        B. Else:
            I. Let '.' nameIR' be prefixedNameIR.
            II. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR').
            III. Let member be $lift_nameIR_as_member(nameIR).
            IV. Return ID '.' nonTypeName '.' member.
d. Else:
    i. Let `( callableTargetIR `) be callableTargetIR'.
    ii. Let expression be $lift_callableTargetIR_as_expression(callableTargetIR).
    iii. Return `( expression `).

$lift_callableTargetIR_as_lvalue(callableTargetIR')

1. Try:
    a. If callableTargetIR' is equal to BARE "this": return THIS.
2. Then, try:
    a. If let prefixedNameIR of type prefixedNameIR be callableTargetIR':
        i. If let BARE nameIR be prefixedNameIR:
            A. Check that nameIR is not equal to "this".
            B. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
            C. Return nonTypeName.
        ii. Else:
            A. Let '.' nameIR be prefixedNameIR.
            B. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
            C. Return ID '.' nonTypeName.
    b. Else if let typedExpressionIR '.' nameIR be callableTargetIR':
        i. Let typedLvalueIR'? be the l-value form of typedExpressionIR.
        ii.

Check that typedLvalueIR? is defined.

iii. Let typedLvalueIR be typedLvalueIR?.

iv. Let lvalue be $lift_typedLvalueIR(typedLvalueIR).

v. Let member be $lift_nameIR_as_member(nameIR).

vi. Return lvalue '.' member.

c. Else if let TYPE prefixedNameIR '.' nameIR be callableTargetIR':

i. Try:

A. If prefixedNameIR is equal to BARE "this":

I. Let member be $lift_nameIR_as_member(nameIR).

II. Return THIS '.' member.

ii. Then, try:

A. If let BARE nameIR' be prefixedNameIR:

I. Check that nameIR' is not equal to "this".

II. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR').

III. Let member be $lift_nameIR_as_member(nameIR).

IV. Return nonTypeName '.' member.

B. Else:

I. Let '.' nameIR' be prefixedNameIR.

II. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR').

III. Let member be $lift_nameIR_as_member(nameIR).

IV. Return ID '.' nonTypeName '.' member.

d. Else:

i. Let `( callableTargetIR `) be callableTargetIR'.

ii. Let lvalue be $lift_callableTargetIR_as_lvalue(callableTargetIR).

iii. Return `( lvalue `).

Declarations

$lift_constantDeclarationIR(constantDeclarationIR)

1. Let annotationList CONST typeIR nameIR constantInitializerIR ';' be constantDeclarationIR.
2. Let type be $lift_typeIR(typeIR).
3. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
4. Let name be nonTypeName.
5. Let initializer be $lift_constantInitializerIR_as_initializer(constantInitializerIR).
6. Return annotationList CONST type name initializer ';'.

$lift_variableDeclarationIR(annotationList typeIR nameIR initializerIR? ';')

1. Let type be $lift_typeIR(typeIR).
2. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
3. Let name be nonTypeName.

4. Let initializerOpt be $lift_initializerOptIR(initializerIR?).
5. Return annotationList type name initializerOpt ';'.

$lift_functionPrototypeIR(typeIR nameIR `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR* `))

1. Let typeOrVoid be $lift_typeIR_as_typeOrVoid(typeIR).
2. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
3. Let name be nonTypeName.
4. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
5. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
6. Let parameterList be $lift_parameterListIR(parameterIR*).
7. Return typeOrVoid name typeParameterListOpt `( parameterList `).

$lift_functionDeclarationIR(annotationList functionPrototypeIR blockStatementIR)

1. Let functionPrototype be $lift_functionPrototypeIR(functionPrototypeIR).
2. Let blockStatement be $lift_blockStatementIR(blockStatementIR).
3. Return annotationList functionPrototype blockStatement.

$lift_actionDeclarationIR(annotationList ACTION nameIR `( parameterIR* `) blockStatementIR)

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let parameterList be $lift_parameterListIR(parameterIR*).
4. Let blockStatement be $lift_blockStatementIR(blockStatementIR).
5. Return annotationList ACTION name `( parameterList `) blockStatement.

$lift_objectDeclarationIR(objectDeclarationIR)

1. If let functionDeclarationIR of type functionDeclarationIR be objectDeclarationIR:
    a. Let functionDeclaration be $lift_functionDeclarationIR(functionDeclarationIR).
    b. Return functionDeclaration.
2. Else:
    a. Let instantiationIR be objectDeclarationIR.
    b. Let instantiation be $lift_instantiationIR(instantiationIR).
    c. Return instantiation.

$lift_objectDeclarationListIR(objectDeclarationIR*)

1. For each objectDeclarationIR in objectDeclarationIR*:
    - Let objectDeclaration be $lift_objectDeclarationIR(objectDeclarationIR).

Let objectDeclaration* be the resulting list.

2. Let objectDeclarationList be $unflatten_objectDeclarationList(objectDeclaration*).
3. Return objectDeclarationList.

$lift_instantiationIR(annotationList typeIR constructorTargetIR `( argumentIR* `) nameIR objectInitializerOptIR ';')

1. Try:
    a. Let objectInitializerIR? be objectInitializerOptIR.
    b. If objectInitializerIR? is none:
        i. Let prefixedNameIR `< typeArgumentIR* `> be constructorTargetIR.
        ii. Try:
            A. If typeArgumentIR* is an empty list:
                I. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
                II. Let type be prefixedTypeName.
                III. Let argumentList be $lift_argumentListIR(argumentIR*).
                IV. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
                V. Let name be nonTypeName.
                VI. Return annotationList type `( argumentList `) name ';'.
        iii. Then, try:
            A. If the length of typeArgumentIR* is greater than 0:
                I. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
                II. Let typeArgumentList be $lift_typeArgumentListIR(typeArgumentIR*).
                III. Let type be prefixedTypeName `< typeArgumentList `>.
                IV. Let argumentList be $lift_argumentListIR(argumentIR*).
                V. Let nonTypeName' be $lift_nameIR_as_nonTypeName(nameIR).
                VI. Let name be nonTypeName'.
                VII. Return annotationList type `( argumentList `) name ';'.
2. Then, try:
    a. Check that objectInitializerOptIR is defined.
    b. Let '=' `{ objectDeclarationIR* `} be objectInitializerOptIR.
    c. Let prefixedNameIR `< typeArgumentIR* `> be constructorTargetIR.
    d. Try:
        i. If typeArgumentIR* is an empty list:
            A. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
            B. Let type be prefixedTypeName.
            C. Let argumentList be $lift_argumentListIR(argumentIR*).
            D. Let nonTypeName'' be $lift_nameIR_as_nonTypeName(nameIR).
            E. Let name be nonTypeName''.
            F. Let objectDeclarationList be $lift_objectDeclarationListIR(objectDeclarationIR*).
            G. Return annotationList type `( argumentList `) name '=' `{ objectDeclarationList `} ';'.
    e. Then, try:

i. Check that the length of typeArgumentIR* is greater than 0.
ii. Let prefixedTypeName be $lift_prefixedNameIR_as_prefixedTypeName(prefixedNameIR).
iii. Check that typeArgumentIR* is a non-empty list.
iv. Let typeArgumentList be $lift_typeArgumentListIR(typeArgumentIR*).
v. Let type be prefixedTypeName `< typeArgumentList `>.
vi. Let argumentList be $lift_argumentListIR(argumentIR*).
vii. Let nonTypeName''' be $lift_nameIR_as_nonTypeName(nameIR).
viii. Let name be nonTypeName'''.
ix. Let objectDeclarationList be $lift_objectDeclarationListIR(objectDeclarationIR*).
x. Return annotationList type `( argumentList `) name '=' `{ objectDeclarationList `} ';'.

$lift_errorDeclarationIR(ERROR `{ nameIR* `})

1. Let nameList be $lift_nameListIR_as_nonTypeNameList(nameIR*).
2. Return ERROR `{ nameList `}.

$lift_matchKindDeclarationIR(MATCH_KIND `{ nameIR* `})

1. Let nameList be $lift_nameListIR_as_nonTypeNameList(nameIR*).
2. Return MATCH_KIND `{ nameList $_{EMPTY}$ `}.

$lift_enumTypeDeclarationIR(enumTypeDeclarationIR)

1. If let annotationList ENUM nameIR `{ $nameIR_{field}$* `} be enumTypeDeclarationIR:
   a. Let typeName be $lift_nameIR_as_typeName(nameIR).
   b. Let name be typeName.
   c. Let $nameList_{field}$ be $lift_nameListIR_as_nonTypeNameList($nameIR_{field}$*).
   d. Return annotationList ENUM name `{ $nameList_{field}$ $_{EMPTY}$ `}.
2. Else:
   a. Let annotationList ENUM typeIR nameIR `{ namedValueListIR `} be enumTypeDeclarationIR.
   b. Let type be $lift_typeIR(typeIR).
   c. Let typeName' be $lift_nameIR_as_typeName(nameIR).
   d. Let name be typeName'.
   e. Let namedExpressionList be $lift_namedValueListIR(namedValueListIR).
   f. Return annotationList ENUM type name `{ namedExpressionList $_{EMPTY}$ `}.

$lift_typeFieldIR(annotationList typeIR nameIR ';')

1. Let type be $lift_typeIR(typeIR).
2. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
3. Let name be nonTypeName.

4. Return annotationList type name ';'.

$lift_typeFieldListIR(typeFieldIR*)

1. For each typeFieldIR in typeFieldIR*:
   - Let typeField be $lift_typeFieldIR(typeFieldIR).

   Let typeField* be the resulting list.
2. Let typeFieldList be $unflatten_typeFieldList(typeField*).
3. Return typeFieldList.

$lift_structTypeDeclarationIR(annotationList STRUCT nameIR `< typeParameterIR* `> `{ typeFieldIR* `})

1. Let typeName be $lift_nameIR_as_typeName(nameIR).
2. Let name be typeName.
3. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
4. Let typeFieldList be $lift_typeFieldListIR(typeFieldIR*).
5. Return annotationList STRUCT name typeParameterListOpt `{ typeFieldList `}.

$lift_headerTypeDeclarationIR(annotationList HEADER nameIR `< typeParameterIR* `> `{ typeFieldIR* `})

1. Let typeName be $lift_nameIR_as_typeName(nameIR).
2. Let name be typeName.
3. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
4. Let typeFieldList be $lift_typeFieldListIR(typeFieldIR*).
5. Return annotationList HEADER name typeParameterListOpt `{ typeFieldList `}.

$lift_headerUnionTypeDeclarationIR(annotationList HEADER_UNION nameIR `< typeParameterIR* `> `{ typeFieldIR* `})

1. Let typeName be $lift_nameIR_as_typeName(nameIR).
2. Let name be typeName.
3. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
4. Let typeFieldList be $lift_typeFieldListIR(typeFieldIR*).
5. Return annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList `}.

$lift_derivedTypeDeclarationIR(derivedTypeDeclarationIR)

1. If let enumTypeDeclarationIR of type enumTypeDeclarationIR be derivedTypeDeclarationIR:
   a. Let enumTypeDeclaration be $lift_enumTypeDeclarationIR(enumTypeDeclarationIR).
   b. Return enumTypeDeclaration.
2. Else if let structTypeDeclarationIR of type structTypeDeclarationIR be derivedTypeDeclarationIR:

a. Let structTypeDeclaration be $lift_structTypeDeclarationIR(structTypeDeclarationIR).

b. Return structTypeDeclaration.

3. Else if let headerTypeDeclarationIR of type headerTypeDeclarationIR be derivedTypeDeclarationIR:

a. Let headerTypeDeclaration be $lift_headerTypeDeclarationIR(headerTypeDeclarationIR).

b. Return headerTypeDeclaration.

4. Else:

a. Let headerUnionTypeDeclarationIR be derivedTypeDeclarationIR.

b. Let headerUnionTypeDeclaration be $lift_headerUnionTypeDeclarationIR(headerUnionTypeDeclarationIR).

c. Return headerUnionTypeDeclaration.

$lift_typedefIR(typedefIR)

1. If let typeIR of type typeIR be typedefIR:

a. Let type be $lift_typeIR(typeIR).

b. Return type.

2. Else:

a. Let derivedTypeDeclarationIR be typedefIR.

b. Let derivedTypeDeclaration be $lift_derivedTypeDeclarationIR(derivedTypeDeclarationIR).

c. Return derivedTypeDeclaration.

$lift_typedefDeclarationIR(typedefDeclarationIR)

1. If let annotationList TYPEDEF typedefIR nameIR ';' be typedefDeclarationIR:

a. If let typeIR of type typeIR be typedefIR:

i. Let type be $lift_typeIR(typeIR).

ii. Let typeName be $lift_nameIR_as_typeName(nameIR).

iii. Let name be typeName.

iv. Return annotationList TYPEDEF type name ';'.

b. Else:

i. Let derivedTypeDeclarationIR be typedefIR.

ii. Let derivedTypeDeclaration be $lift_derivedTypeDeclarationIR(derivedTypeDeclarationIR).

iii. Let typeName' be $lift_nameIR_as_typeName(nameIR).

iv. Let name be typeName'.

v. Return annotationList TYPEDEF derivedTypeDeclaration name ';'.

2. Else:

a. Let annotationList TYPE typeIR nameIR ';' be typedefDeclarationIR.

b. Let type be $lift_typeIR(typeIR).

c. Let typeName'' be $lift_nameIR_as_typeName(nameIR).

d. Let name be typeName''.

e. Return annotationList TYPE type name ';'.

$lift_externFunctionDeclarationIR(annotationList EXTERN functionPrototypeIR ';')

1. Let functionPrototype be $lift_functionPrototypeIR(functionPrototypeIR).
2. Return annotationList EXTERN functionPrototype ';'.

Object, parser, control, and table declarations

$lift_externConstructorPrototypeIR(annotationList nameIR `< ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) ';')

1. Let typeIdentifier be $lift_nameIR_as_typeIdentifier(nameIR).
2. Let parameterList be $lift_parameterListIR(parameterIR*).
3. Return annotationList typeIdentifier `( parameterList `) ';'.

$lift_externMethodPrototypeIR(externMethodPrototypeIR)

1. If let annotationList functionPrototypeIR ';' be externMethodPrototypeIR:
   a. Let functionPrototype be $lift_functionPrototypeIR(functionPrototypeIR).
   b. Return annotationList functionPrototype ';'.
2. Else:
   a. Let annotationList ABSTRACT functionPrototypeIR ';' be externMethodPrototypeIR.
   b. Let functionPrototype be $lift_functionPrototypeIR(functionPrototypeIR).
   c. Return annotationList ABSTRACT functionPrototype ';'.

$lift_externObjectDeclarationIR(annotationList EXTERN nameIR `< typeParameterIR$^{*}$ ',' `> `{ externConstructorPrototypeIR$^{*}$ externMethodPrototypeIR$^{*}$ `})

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
3. For each externConstructorPrototypeIR in externConstructorPrototypeIR*:

   - Let externConstructorPrototype be $lift_externConstructorPrototypeIR(externConstructorPrototypeIR).

   Let externConstructorPrototype* be the resulting list.

4. For each externMethodPrototypeIR in externMethodPrototypeIR*:

   - Let externMethodPrototype be $lift_externMethodPrototypeIR(externMethodPrototypeIR).

   Let externMethodPrototype* be the resulting list.

5. Let externConstructorOrMethodPrototype* be externConstructorPrototype* concatenated with externMethodPrototype*.
6. Let externConstructorOrMethodPrototypeList be $unflatten_externConstructorOrMethodPrototypeList(externConstructorOrMethodPrototype*).
7. Return annotationList EXTERN nonTypeName typeParameterListOpt `{ externConstructorOrMethodPrototypeList `}.

$lift_externDeclarationIR(externDeclarationIR)

1. If let externFunctionDeclarationIR of type externFunctionDeclarationIR be externDeclarationIR:
    a. Let externFunctionDeclaration be $lift_externFunctionDeclarationIR(externFunctionDeclarationIR).
    b. Return externFunctionDeclaration.
2. Else:
    a. Let externObjectDeclarationIR be externDeclarationIR.
    b. Let externObjectDeclaration be $lift_externObjectDeclarationIR(externObjectDeclarationIR).
    c. Return externObjectDeclaration.

$lift_selectCaseIR(keysetExpressionIR ':' nameIR ';')

1. Let keysetExpression be $lift_keysetExpressionIR(keysetExpressionIR).
2. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
3. Let name be nonTypeName.
4. Return keysetExpression ':' name ';'.

$lift_selectCaseListIR(selectCaseIR*)

1. For each selectCaseIR in selectCaseIR*:
    - Let selectCase be $lift_selectCaseIR(selectCaseIR).

    Let selectCase* be the resulting list.
2. Let selectCaseList be $unflatten_selectCaseList(selectCase*).
3. Return selectCaseList.

$lift_selectExpressionIR(SELECT `( typedExpressionListIR `) `{ selectCaseIR* `})

1. Let expressionList be $lift_typedExpressionListIR(typedExpressionListIR).
2. Let selectCaseList be $lift_selectCaseListIR(selectCaseIR*).
3. Return SELECT `( expressionList `) `{ selectCaseList `}.

$lift_stateExpressionIR(stateExpressionIR)

1. If let nameIR ';' be stateExpressionIR:
    a. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
    b. Let name be nonTypeName.
    c. Return name ';'.
2. Else:
    a. Let selectExpressionIR be stateExpressionIR.
    b. Let selectExpression be $lift_selectExpressionIR(selectExpressionIR).
    c. Return selectExpression.

$lift_transitionStatementIR(TRANSITION stateExpressionIR)

1. Let stateExpression be $lift_stateExpressionIR(stateExpressionIR).
2. Return TRANSITION stateExpression.

$lift_typeIR_as_valueSetType(typeIR)

1. Let type be $lift_typeIR(typeIR).
2. If let baseType of type baseType be type: return baseType.
3. Else if let tupleType of type tupleType be type: return tupleType.
4. Else if let prefixedTypeName of type prefixedTypeName be type: return prefixedTypeName.

$lift_valueSetDeclarationIR(annotationList VALUE_SET `< typeIR `> `( typedExpressionIR `) nameIR ';')

1. Let valueSetType be $lift_typeIR_as_valueSetType(typeIR).
2. Let expression be $lift_typedExpressionIR(typedExpressionIR).
3. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
4. Let name be nonTypeName.
5. Return annotationList VALUE_SET `< valueSetType `> `( expression `) name ';'.

$lift_parserTypeDeclarationIR(annotationList PARSER nameIR `< $typeParameterIR_{expl}^{*}$ ',' $typeParameterIR_{impl}^{*}$ `> `( $parameterIR^{*}$ `) ';')

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let $typeParameterIR^{*}$ be $typeParameterIR_{expl}^{*}$ concatenated with $typeParameterIR_{impl}^{*}$.
4. Let typeParameterListOpt be $lift_typeParameterListIR($typeParameterIR^{*}$).
5. Let parameterList be $lift_parameterListIR($parameterIR^{*}$).
6. Return annotationList PARSER name typeParameterListOpt `( parameterList `) ';'.

$lift_parserBlockElementStatementIR(parserBlockElementStatementIR)

1. If let constantDeclarationIR of type constantDeclarationIR be parserBlockElementStatementIR:
   a. Let constantDeclaration be $lift_constantDeclarationIR(constantDeclarationIR).
   b. Return constantDeclaration.
2. Else if let variableDeclarationIR of type variableDeclarationIR be parserBlockElementStatementIR:
   a. Let variableDeclaration be $lift_variableDeclarationIR(variableDeclarationIR).
   b. Return variableDeclaration.
3. Else:
   a. Let parserStatementIR be parserBlockElementStatementIR.
   b. Let parserStatement be $lift_parserStatementIR(parserStatementIR).
   c. Return parserStatement.

$lift_parserBlockElementStatementListIR(parserBlockElementStatementIR*)

1. For each parserBlockElementStatementIR in parserBlockElementStatementIR*:
    - Let parserBlockElementStatement be $lift_parserBlockElementStatementIR(parserBlockElementStatementIR).

    Let parserBlockElementStatement* be the resulting list.
2. Let parserBlockElementStatementList be $unflatten_parserBlockElementStatementList(parserBlockElementStatement*).
3. Return parserBlockElementStatementList.

$lift_parserBlockStatementIR(annotationList `{ parserBlockElementStatementIR* `})

1. Let parserBlockElementStatementList be $lift_parserBlockElementStatementListIR(parserBlockElementStatementIR*).
2. Return annotationList `{ parserBlockElementStatementList `}.

$lift_parserConditionalStatementIR(parserConditionalStatementIR)

1. If let IF `( typedExpressionIR$_{cond}$ `) parserStatementIR$_{then}$ be parserConditionalStatementIR:
    a. Let expression$_{cond}$ be $lift_typedExpressionIR(typedExpressionIR$_{cond}$).
    b. Let parserStatement$_{then}$ be $lift_parserStatementIR(parserStatementIR$_{then}$).
    c. Return IF `( expression$_{cond}$ `) parserStatement$_{then}$.
2. Else:
    a. Let IF `( typedExpressionIR$_{cond}$ `) parserStatementIR$_{then}$ ELSE parserStatementIR$_{else}$ be parserConditionalStatementIR.
    b. Let expression$_{cond}$ be $lift_typedExpressionIR(typedExpressionIR$_{cond}$).
    c. Let parserStatement$_{then}$ be $lift_parserStatementIR(parserStatementIR$_{then}$).
    d. Let parserStatement$_{else}$ be $lift_parserStatementIR(parserStatementIR$_{else}$).
    e. Return IF `( expression$_{cond}$ `) parserStatement$_{then}$ ELSE parserStatement$_{else}$.

$lift_parserStatementIR(parserStatementIR)

1. If let emptyStatementIR of type emptyStatementIR be parserStatementIR:
    a. Let emptyStatement be $lift_emptyStatementIR(emptyStatementIR).
    b. Return emptyStatement.
2. Else if let assignmentStatementIR of type assignmentStatementIR be parserStatementIR:
    a. Let assignmentStatement be $lift_assignmentStatementIR(assignmentStatementIR).
    b. Return assignmentStatement.
3. Else if let callStatementIR of type callStatementIR be parserStatementIR:
    a. Let callStatement be $lift_callStatementIR(callStatementIR).
    b. Return callStatement.
4. Else if let directApplicationStatementIR of type directApplicationStatementIR be parserStatementIR:
    a.

Let directApplicationStatement be $lift_directApplicationStatementIR(directApplicationStatementIR).

b. Return directApplicationStatement.

5. Else if let parserBlockStatementIR of type parserBlockStatementIR be parserStatementIR:
   a. Let parserBlockStatement be $lift_parserBlockStatementIR(parserBlockStatementIR).
   b. Return parserBlockStatement.
6. Else:
   a. Let parserConditionalStatementIR be parserStatementIR.
   b. Let parserConditionalStatement be $lift_parserConditionalStatementIR(parserConditionalStatementIR).
   c. Return parserConditionalStatement.

$lift_parserStateIR(annotationList STATE nameIR `{ parserBlockElementStatementIR* transitionStatementIR `})

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let parserBlockElementStatementList be $lift_parserBlockElementStatementListIR(parserBlockElementStatementIR*).
4. Let transitionStatement be $lift_transitionStatementIR(transitionStatementIR).
5. Return annotationList STATE name `{ parserBlockElementStatementList transitionStatement `}.

$lift_parserStateListIR(parserStateIR*)

1. For each parserStateIR in parserStateIR*:
   - Let parserState be $lift_parserStateIR(parserStateIR).

   Let parserState* be the resulting list.
2. Let parserStateList be $unflatten_parserStateList(parserState*).
3. Return parserStateList.

$lift_parserLocalDeclarationIR(parserLocalDeclarationIR)

1. If let constantDeclarationIR of type constantDeclarationIR be parserLocalDeclarationIR:
   a. Let constantDeclaration be $lift_constantDeclarationIR(constantDeclarationIR).
   b. Return constantDeclaration.
2. Else if let instantiationIR of type instantiationIR be parserLocalDeclarationIR:
   a. Let instantiation be $lift_instantiationIR(instantiationIR).
   b. Return instantiation.
3. Else if let variableDeclarationIR of type variableDeclarationIR be parserLocalDeclarationIR:
   a. Let variableDeclaration be $lift_variableDeclarationIR(variableDeclarationIR).
   b. Return variableDeclaration.
4. Else:
   a. Let valueSetDeclarationIR be parserLocalDeclarationIR.
   b. Let valueSetDeclaration be $lift_valueSetDeclarationIR(valueSetDeclarationIR).

c. Return valueSetDeclaration.

$lift_parserLocalDeclarationListIR(parserLocalDeclarationIR*)

1. For each parserLocalDeclarationIR in parserLocalDeclarationIR*:

   - Let parserLocalDeclaration be $lift_parserLocalDeclarationIR(parserLocalDeclarationIR).

   Let parserLocalDeclaration* be the resulting list.

2. Let parserLocalDeclarationList be $unflatten_parserLocalDeclarationList(parserLocalDeclaration*).
3. Return parserLocalDeclarationList.

$lift_parserDeclarationIR(annotationList PARSER nameIR `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `( parameterIR* `) `( constructorParameterIR* `) `{ parserLocalDeclarationIR* parserStateIR* `})

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let typeParameterIR* be typeParameterIR$_{expl}$* concatenated with typeParameterIR$_{impl}$*.
4. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
5. Let parameterList be $lift_parameterListIR(parameterIR*).
6. Let constructorParameterListOpt be $lift_constructorParameterListIR(constructorParameterIR*).
7. Let parserLocalDeclarationList be $lift_parserLocalDeclarationListIR(parserLocalDeclarationIR*).
8. Let parserStateList be $lift_parserStateListIR(parserStateIR*).
9. Return annotationList PARSER name typeParameterListOpt `( parameterList `) constructorParameterListOpt `{ parserLocalDeclarationList parserStateList `}.

$lift_constOptIR(constOptIR)

1. Try:
   a. Let constIR? be constOptIR.
   b. If constIR? is none: return $_{EMPTY}$.
2. Then, try:
   a. Check that constOptIR is defined.
   b. Return CONST.

$lift_tableKeyIR(typedExpressionIR '#' _ ':' nameIR annotationList ';')

1. Let expression be $lift_typedExpressionIR(typedExpressionIR).
2. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
3. Let name be nonTypeName.
4. Return expression ':' name annotationList ';'.

$lift_tableKeyListIR(tableKeyIR*)

1. For each tableKeyIR in tableKeyIR*:
    - Let tableKey be $lift_tableKeyIR(tableKeyIR).

    Let tableKey* be the resulting list.
2. Let tableKeyList be $unflatten_tableKeyList(tableKey*).
3. Return tableKeyList.

$lift_tableActionReferenceIR(prefixedNameIR _ `( argumentIR* `))

1. Let prefixedNonTypeName be $lift_prefixedNameIR_as_prefixedNonTypeName(prefixedNameIR).
2. Let argumentList be $lift_argumentListIR(argumentIR*).
3. Return prefixedNonTypeName `( argumentList `).

$lift_tableActionReferenceIR_as_expression(prefixedNameIR _ `( argumentIR* `))

1. Let prefixedNonTypeName be $lift_prefixedNameIR_as_prefixedNonTypeName(prefixedNameIR).
2. Let argumentList be $lift_argumentListIR(argumentIR*).
3. Return prefixedNonTypeName `( argumentList `).

$lift_tableActionIR(annotationList tableActionReferenceIR '#' `( _ ',' _ `) ';')

1. Let tableActionReference be $lift_tableActionReferenceIR(tableActionReferenceIR).
2. Return annotationList tableActionReference ';'.

$lift_tableActionListIR(tableActionIR*)

1. For each tableActionIR in tableActionIR*:
    - Let tableAction be $lift_tableActionIR(tableActionIR).

    Let tableAction* be the resulting list.
2. Let tableActionList be $unflatten_tableActionList(tableAction*).
3. Return tableActionList.

$lift_tableEntryPriorityIR(PRIORITY '=' integerLiteral ':')

1. Return PRIORITY '=' integerLiteral ':'.

$lift_tableEntryIR(constOptIR tableEntryPriorityOptIR keysetExpressionIR ':' tableActionReferenceIR annotationList ';')

1. Try:
    a. Let tableEntryPriorityIR? be tableEntryPriorityOptIR.

b. If tableEntryPriorityIR? is none:

i. Let constOpt be $lift_constOptIR(constOptIR).

ii. Let keysetExpression be $lift_keysetExpressionIR(keysetExpressionIR).

iii. Let tableActionReference be $lift_tableActionReferenceIR(tableActionReferenceIR).

iv. Return constOpt keysetExpression ':' tableActionReference annotationList ';'.

2. Then, try:

a. Check that tableEntryPriorityOptIR is defined.

b. Let tableEntryPriorityIR be tableEntryPriorityOptIR.

c. Let constOpt be $lift_constOptIR(constOptIR).

d. Let tableEntryPriority be $lift_tableEntryPriorityIR(tableEntryPriorityIR).

e. Let keysetExpression be $lift_keysetExpressionIR(keysetExpressionIR).

f. Let tableActionReference be $lift_tableActionReferenceIR(tableActionReferenceIR).

g. Return constOpt tableEntryPriority keysetExpression ':' tableActionReference annotationList ';'.

$lift_tableEntryListIR(tableEntryIR*)

1. For each tableEntryIR in tableEntryIR*:

- Let tableEntry be $lift_tableEntryIR(tableEntryIR).

Let tableEntry* be the resulting list.

2. Let tableEntryList be $unflatten_tableEntryList(tableEntry*).
3. Return tableEntryList.

$lift_tableCustomPropertyIR(tableCustomPropertyIR)

1. Check that tableCustomPropertyIR matches pattern % % CUSTOM % % ';'.
2. Let annotationList constOptIR CUSTOM nameIR initializerIR ';' be tableCustomPropertyIR.
3. Let constOpt be $lift_constOptIR(constOptIR).
4. Let tableCustomName be $lift_nameIR_as_tableCustomName(nameIR).
5. Let initializer be $lift_initializerIR(initializerIR).
6. Return annotationList constOpt tableCustomName initializer ';'.

$lift_tableKeysPropertyIR(KEY '=' `{ tableKeyIR* `})

1. Let tableKeyList be $lift_tableKeyListIR(tableKeyIR*).
2. Return KEY '=' `{ tableKeyList `}.

$lift_tableActionsPropertyIR(ACTIONS '=' `{ tableActionIR* `})

1. Let tableActionList be $lift_tableActionListIR(tableActionIR*).
2. Return ACTIONS '=' `{ tableActionList `}.

$lift_tableDefaultActionPropertyIR(annotationList constOptIR DEFAULT_ACTION '=' tableActionReferenceIR ';')

1. Let constOpt be $lift_constOptIR(constOptIR).
2. Let expression be $lift_tableActionReferenceIR_as_expression(tableActionReferenceIR).
3. Return annotationList constOpt $_{ID}$ "default_action" '=' expression ';'.

$lift_tableEntriesPropertyIR(annotationList constOptIR ENTRIES '=' `{ tableEntryIR* `})

1. Let constOpt be $lift_constOptIR(constOptIR).
2. Let tableEntryList be $lift_tableEntryListIR(tableEntryIR*).
3. Return annotationList constOpt ENTRIES '=' `{ tableEntryList `}.

$lift_tablePropertyIR(tablePropertyIR)

1. If let tableKeysPropertyIR of type tableKeysPropertyIR be tablePropertyIR:
   a. Return $lift_tableKeysPropertyIR(tableKeysPropertyIR).
2. Else if let tableActionsPropertyIR of type tableActionsPropertyIR be tablePropertyIR:
   a. Return $lift_tableActionsPropertyIR(tableActionsPropertyIR).
3. Else if let tableDefaultActionPropertyIR of type tableDefaultActionPropertyIR be tablePropertyIR:
   a. Return $lift_tableDefaultActionPropertyIR(tableDefaultActionPropertyIR).
4. Else if let tableEntriesPropertyIR of type tableEntriesPropertyIR be tablePropertyIR:
   a. Return $lift_tableEntriesPropertyIR(tableEntriesPropertyIR).
5. Else:
   a. Let tableCustomPropertyIR be tablePropertyIR.
   b. Return $lift_tableCustomPropertyIR(tableCustomPropertyIR).

$lift_tablePropertyListIR(tablePropertyIR*)

1. For each tablePropertyIR in tablePropertyIR*:

   - Let tableProperty be $lift_tablePropertyIR(tablePropertyIR).

   Let tableProperty* be the resulting list.
2. Let tablePropertyList be $unflatten_tablePropertyList(tableProperty*).
3. Return tablePropertyList.

$lift_tableDeclarationIR(annotationList TABLE typeIR nameIR `{ tablePropertyIR* `})

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let tablePropertyList be $lift_tablePropertyListIR(tablePropertyIR*).
4. Return annotationList TABLE name `{ tablePropertyList `}.

$lift_controlTypeDeclarationIR(annotationList CONTROL nameIR `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) ';')

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
4. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
5. Let parameterList be $lift_parameterListIR(parameterIR*).
6. Return annotationList CONTROL name typeParameterListOpt `( parameterList `) ';'.

$lift_controlBodyIR(controlBodyIR)

1. Return $lift_blockStatementIR(controlBodyIR).

$lift_controlLocalDeclarationIR(controlLocalDeclarationIR)

1. If let constantDeclarationIR of type constantDeclarationIR be controlLocalDeclarationIR:
   a. Let constantDeclaration be $lift_constantDeclarationIR(constantDeclarationIR).
   b. Return constantDeclaration.
2. Else if let instantiationIR of type instantiationIR be controlLocalDeclarationIR:
   a. Let instantiation be $lift_instantiationIR(instantiationIR).
   b. Return instantiation.
3. Else if let variableDeclarationIR of type variableDeclarationIR be controlLocalDeclarationIR:
   a. Let variableDeclaration be $lift_variableDeclarationIR(variableDeclarationIR).
   b. Return variableDeclaration.
4. Else if let actionDeclarationIR of type actionDeclarationIR be controlLocalDeclarationIR:
   a. Let actionDeclaration be $lift_actionDeclarationIR(actionDeclarationIR).
   b. Return actionDeclaration.
5. Else:
   a. Let tableDeclarationIR be controlLocalDeclarationIR.
   b. Let tableDeclaration be $lift_tableDeclarationIR(tableDeclarationIR).
   c. Return tableDeclaration.

$lift_controlLocalDeclarationListIR(controlLocalDeclarationIR*)

1. For each controlLocalDeclarationIR in controlLocalDeclarationIR*:

   - Let controlLocalDeclaration be $lift_controlLocalDeclarationIR(controlLocalDeclarationIR).

   Let controlLocalDeclaration* be the resulting list.

2. Let controlLocalDeclarationList be $unflatten_controlLocalDeclarationList(controlLocalDeclaration*).
3. Return controlLocalDeclarationList.

$lift_controlDeclarationIR(annotationList CONTROL nameIR `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) `( constructorParameterIR$^{*}$ `) `{ controlLocalDeclarationIR$^{*}$ APPLY controlBodyIR `})

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
4. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
5. Let parameterList be $lift_parameterListIR(parameterIR*).
6. Let constructorParameterListOpt be $lift_constructorParameterListIR(constructorParameterIR*).
7. Let controlLocalDeclarationList be $lift_controlLocalDeclarationListIR(controlLocalDeclarationIR*).
8. Let controlBody be $lift_controlBodyIR(controlBodyIR).
9. Return annotationList CONTROL name typeParameterListOpt `( parameterList `) constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody `}.

$lift_packageTypeDeclarationIR(annotationList PACKAGE nameIR `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) ';')

1. Let nonTypeName be $lift_nameIR_as_nonTypeName(nameIR).
2. Let name be nonTypeName.
3. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
4. Let typeParameterListOpt be $lift_typeParameterListIR(typeParameterIR*).
5. Let parameterList be $lift_parameterListIR(parameterIR*).
6. Return annotationList PACKAGE name typeParameterListOpt `( parameterList `) ';'.

$lift_typeDeclarationIR(typeDeclarationIR)

1. If let derivedTypeDeclarationIR of type derivedTypeDeclarationIR be typeDeclarationIR:
   a. Let derivedTypeDeclaration be $lift_derivedTypeDeclarationIR(derivedTypeDeclarationIR).
   b. Return derivedTypeDeclaration.
2. Else if let typedefDeclarationIR of type typedefDeclarationIR be typeDeclarationIR:
   a. Let typedefDeclaration be $lift_typedefDeclarationIR(typedefDeclarationIR).
   b. Return typedefDeclaration.
3. Else if let parserTypeDeclarationIR of type parserTypeDeclarationIR be typeDeclarationIR:
   a. Let parserTypeDeclaration be $lift_parserTypeDeclarationIR(parserTypeDeclarationIR).
   b. Return parserTypeDeclaration.
4. Else if let controlTypeDeclarationIR of type controlTypeDeclarationIR be typeDeclarationIR:
   a. Let controlTypeDeclaration be $lift_controlTypeDeclarationIR(controlTypeDeclarationIR).
   b. Return controlTypeDeclaration.
5. Else:
   a. Let packageTypeDeclarationIR be typeDeclarationIR.
   b. Let packageTypeDeclaration be $lift_packageTypeDeclarationIR(packageTypeDeclarationIR).
   c. Return packageTypeDeclaration.

$lift_declarationIR(declarationIR)

1. If let constantDeclarationIR of type constantDeclarationIR be declarationIR:
    a. Let constantDeclaration be $lift_constantDeclarationIR(constantDeclarationIR).
    b. Return constantDeclaration.
2. Else if let instantiationIR of type instantiationIR be declarationIR:
    a. Let instantiation be $lift_instantiationIR(instantiationIR).
    b. Return instantiation.
3. Else if let functionDeclarationIR of type functionDeclarationIR be declarationIR:
    a. Let functionDeclaration be $lift_functionDeclarationIR(functionDeclarationIR).
    b. Return functionDeclaration.
4. Else if let actionDeclarationIR of type actionDeclarationIR be declarationIR:
    a. Let actionDeclaration be $lift_actionDeclarationIR(actionDeclarationIR).
    b. Return actionDeclaration.
5. Else if let errorDeclarationIR of type errorDeclarationIR be declarationIR:
    a. Let errorDeclaration be $lift_errorDeclarationIR(errorDeclarationIR).
    b. Return errorDeclaration.
6. Else if let matchKindDeclarationIR of type matchKindDeclarationIR be declarationIR:
    a. Let matchKindDeclaration be $lift_matchKindDeclarationIR(matchKindDeclarationIR).
    b. Return matchKindDeclaration.
7. Else if let externDeclarationIR of type externDeclarationIR be declarationIR:
    a. Let externDeclaration be $lift_externDeclarationIR(externDeclarationIR).
    b. Return externDeclaration.
8. Else if let parserDeclarationIR of type parserDeclarationIR be declarationIR:
    a. Let parserDeclaration be $lift_parserDeclarationIR(parserDeclarationIR).
    b. Return parserDeclaration.
9. Else if let controlDeclarationIR of type controlDeclarationIR be declarationIR:
    a. Let controlDeclaration be $lift_controlDeclarationIR(controlDeclarationIR).
    b. Return controlDeclaration.
10. Else:
    a. Let typeDeclarationIR be declarationIR.
    b. Let typeDeclaration be $lift_typeDeclarationIR(typeDeclarationIR).
    c. Return typeDeclaration.
11. Otherwise:
    a. (debug: declarationIR)
    b. Return $sink<declaration>.

P4 program

$lift_p4programIR(declarationIR* ';')

1. For each declarationIR in declarationIR*:

    - Let declaration be $lift_declarationIR(declarationIR).

   Let declaration* be the resulting list.

2. Return $unflatten_p4program(declaration*).

Appendix M: Context and environment operations

This appendix collects the operations on the three contexts threaded through the specification — the typing, instantiation, and execution contexts — and on the environments they contain. Each context provides an empty value, operations to add and look up bindings for the various kinds of entity it tracks, and operations to enter and exit scopes.

M.1. Typing context

The typing context is threaded through type checking. It holds a stack of scopes binding variables to their types, environments of type, callable-type, and constructor-type definitions, and the type frame used for type-argument inference. Its operations are suffixed with _t.

Empty environments

$empty_callableTypeDefEnv

1. Return an empty map.

$empty_constructorTypeDefEnv

1. Return an empty map.

$empty_frame

1. Return an empty map.

$empty_typeDefEnv

1. Return an empty map.

$empty_typeFrame

1. Return an empty map.

An empty typing context

1. Let globalTypingLayer be {+CONSTRUCTOR+ $empty_constructorTypeDefEnv, TYPE $empty_typeDefEnv, CALLABLE $empty_callableTypeDefEnv, FRAME $empty_typeFrame}.
2. Let blockTypingLayer be {+KIND+ $_{\text{EMPTY}}$, TYPE $empty_typeDefEnv, CALLABLE $empty_callableTypeDefEnv, FRAME $empty_typeFrame}.
3. Let localTypingLayer be {+KIND+ $_{\text{EMPTY}}$, TYPE $empty_typeDefEnv, FRAMES $empty_typeFrame}.
4. Let TC be {+CONTROL+ CONT, LOOP OUT, GLOBAL globalTypingLayer, BLOCK blockTypingLayer, LOCAL localTypingLayer}.
5. Return TC.

Variables

TC where id to varTypeIR is added to the p layer

1. If p matches pattern GLOBAL:
   a. Let typeFrame be TC.GLOBAL.FRAME.
   b. Check that ~$in_set<id>(id, $dom_map<id, varTypeIR>(typeFrame)).
   c. Let typeIR be the type of varTypeIR.
   d. Let $\text{typeIR}_{\text{unroll}}$ be typeIR with typedefs unrolled.
   e. Check that if id is equal to "main", then $\text{typeIR}_{\text{unroll}}$ has type packageObjectTypeIR.
   f. Let $\text{typeFrame}_{\text{update}}$ be typeFrame where id to varTypeIR is added.
   g. Let TC' be TC with GLOBAL.FRAME set to $\text{typeFrame}_{\text{update}}$.
   h. Return TC'.
2. Else if p matches pattern BLOCK:
   a. Let typeFrame be TC.BLOCK.FRAME.
   b. Check that ~$in_set<id>(id, $dom_map<id, varTypeIR>(typeFrame)).
   c. Check that id is not equal to "main".
   d. Let $\text{typeFrame}_{\text{update}}$ be TC.BLOCK.FRAME where id to varTypeIR is added.
   e. Let TC' be TC with BLOCK.FRAME set to $\text{typeFrame}_{\text{update}}$.
   f. Return TC'.
3. Else:
   a. Let typeFrame'^{*} be TC.LOCAL.FRAMES.
   b. Check that typeFrame'^{*} is a non-empty list.
   c. Let $\text{typeFrame}_{h} :: \text{typeFrame}_{t}^{*}$ be typeFrame'^{*}.
   d. Check that ~$in_set<id>(id, $dom_map<id, varTypeIR>($\text{typeFrame}_{h}$)).
   e. Check that id is not equal to "main".
   f. Let $\text{typeFrame}_{\text{h_update}}$ be typeFrame_{h} where id to varTypeIR is added.
   g. Let typeFrame^{*} be $\text{typeFrame}_{\text{h_update}} :: \text{typeFrame}_{t}^{*}$.
   h. Let TC' be TC with LOCAL.FRAMES set to typeFrame^{*}.
   i. Return TC'.

TC where each of id^{*} to each of varTypeIR^{*} are added to the p layer

1. If id^* matches pattern []:
    a. Check that $varTypeIR^*$ is an empty list.
    b. Return TC.
2. Else:
    a. Let $id_h :: id_t^*$ be id^*.
    b. Check that $varTypeIR^*$ is a non-empty list.
    c. Let $varTypeIR_h :: varTypeIR_t^*$ be $varTypeIR^*$.
    d. Let TC' be TC where id_h to $varTypeIR_h$ is added to the p layer.
    e. Let TC'' be TC' where each of id_t^* to each of $varTypeIR_t^*$ are added to the p layer.
    f. Return TC''.

The value of variable prefixedNameIR from the p layer of TC

1. Let $varTypeIR^?$ be the type of variable prefixedNameIR from the p layer of TC.
2. Check that $varTypeIR^?$ is defined.
3. Let _ _ _ $value'^?$ be $varTypeIR^?$.
4. Check that $value'^?$ is defined.
5. Let value be $value'^?$.
6. Return value.

Types

Type parameters $typeId^*$ added to the p layer of TC

1. Return TC where each of $typeId^*$ to each of (NAME typeId)* are added to the p layer.

Type definitions

TC where typeId to typeDefIR is added to the p layer

1. If p matches pattern GLOBAL:
    a. Let typeDefEnv be TC.GLOBAL.TYPE.
    b. Check that ~$in_set<typeId>(typeId, $dom_map<typeId, typeDefIR>(typeDefEnv)).
    c. Let $typeDefEnv_{update}$ be typeDefEnv where typeId to typeDefIR is added.
    d. Let TC' be TC with GLOBAL.TYPE set to $typeDefEnv_{update}$.
    e. Return TC'.
2. Else if p matches pattern BLOCK:
    a. Let typeDefEnv be TC.BLOCK.TYPE.
    b. Check that ~$in_set<typeId>(typeId, $dom_map<typeId, typeDefIR>(typeDefEnv)).
    c. Let $typeDefEnv_{update}$ be typeDefEnv where typeId to typeDefIR is added.
    d. Let TC' be TC with BLOCK.TYPE set to $typeDefEnv_{update}$.

e. Return TC'.

3. Else:
   a. Let typeDefEnv be TC.LOCAL.TYPE.
   b. Check that ~$in_set<typeId>(typeId, $dom_map<typeId, typeDefIR>(typeDefEnv)).
   c. Let $typeDefEnv_{update}$ be typeDefEnv where typeId to typeDefIR is added.
   d. Let TC' be TC with LOCAL.TYPE set to $typeDefEnv_{update}$.
   e. Return TC'.

TC where each of $typeId^*$ to each of $typeDefIR^*$ are added to the p layer

1. If $typeId^*$ matches pattern []:
   a. Check that $typeDefIR^*$ is an empty list.
   b. Return TC.
2. Else:
   a. Let $typeId_h$ :: $typeId_t^*$ be $typeId^*$.
   b. Check that $typeDefIR^*$ is a non-empty list.
   c. Let $typeDefIR_h$ :: $typeDefIR_t^*$ be $typeDefIR^*$.
   d. Let TC' be TC where $typeId_h$ to $typeDefIR_h$ is added to the p layer.
   e. Let TC'' be TC' where each of $typeId_t^*$ to each of $typeDefIR_t^*$ are added to the p layer.
   f. Return TC''.

The type definition of prefixedNameIR from the p layer of TC

1. Try:
   a. If let '.' typeId be prefixedNameIR:
      i. Let typeDefEnv be TC.GLOBAL.TYPE.
      ii. Return the value of typeId in map typeDefEnv.
2. Then, try:
   a. If p matches pattern GLOBAL:
      i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
      ii. Let $_{BARE}$ typeId be prefixedNameIR.
      iii. Let typeDefEnv be TC.GLOBAL.TYPE.
      iv. Return the value of typeId in map typeDefEnv.
   b. Else if p matches pattern BLOCK:
      i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
      ii. Let $_{BARE}$ typeId be prefixedNameIR.
      iii. Let typeDefEnv be TC.BLOCK.TYPE.
      iv. Try:
         A. Let $typeDefIR'^?$ be the value of typeId in map typeDefEnv.
         B. If $typeDefIR'^?$ is defined:
            I. Let typeDefIR be $typeDefIR'^?$.

II. Return typeDefIR.

v. Then, try:

A. Check that the value of typeId in map typeDefEnv is none.

B. Return the type definition of $_{\text{BARE}}$ typeId from the GLOBAL layer of TC.

c. Else:

i. Check that prefixedNameIR matches pattern $_{\text{BARE}}$ %.

ii. Let $_{\text{BARE}}$ typeId be prefixedNameIR.

iii. Let typeDefEnv* be TC.LOCAL.TYPE.

iv. Try:

A. Let typeDefIR'? be the value of typeId in maps typeDefEnv*.

B. If typeDefIR'? is defined:

I. Let typeDefIR be typeDefIR'?.

II. Return typeDefIR.

v. Then, try:

A. Check that the value of typeId in maps typeDefEnv* is none.

B. Return the type definition of $_{\text{BARE}}$ typeId from the BLOCK layer of TC.

Callable definitions

TC where callableId to callableTypeDefIR is added as a non-overloaded callable to the p layer

1. If p matches pattern GLOBAL:
   a. Let callableTypeDefEnv be TC.GLOBAL.CALLABLE.
   b. Let nameIR `( _* `) be callableId.
   c. Let `{ ( nameIR$_k$ `(_* `) )* `} be the domain of the map callableTypeDefEnv.
   d. Check that ~$in_set<nameIR>(nameIR, `{ nameIR$_k$* `}).
   e. Let callableTypeDefEnv$_{\text{update}}$ be callableTypeDefEnv where callableId to callableTypeDefIR is added.
   f. Let TC' be TC with GLOBAL.CALLABLE set to callableTypeDefEnv$_{\text{update}}$.
   g. Return TC'.
2. Else if p matches pattern BLOCK:
   a. Let callableTypeDefEnv be TC.BLOCK.CALLABLE.
   b. Let nameIR `( _* `) be callableId.
   c. Let `{ ( nameIR$_k$ `(_* `) )* `} be the domain of the map callableTypeDefEnv.
   d. Check that ~$in_set<nameIR>(nameIR, `{ nameIR$_k$* `}).
   e. Let callableTypeDefEnv$_{\text{update}}$ be callableTypeDefEnv where callableId to callableTypeDefIR is added.
   f. Let TC' be TC with BLOCK.CALLABLE set to callableTypeDefEnv$_{\text{update}}$.
   g. Return TC'.

TC where callableId to callableTypeDefIR is added as an overloaded callable to the p layer

1. If p matches pattern GLOBAL:

a. Let callableTypeDefEnv be TC.GLOBAL.CALLABLE.
b. Check that ~$in_set<callableId>(callableId, $dom_map<callableId, callableTypeDefIR>(callableTypeDefEnv)).
c. Let $callableTypeDefEnv_{update}$ be callableTypeDefEnv where callableId to callableTypeDefIR is added.
d. Let TC' be TC with GLOBAL.CALLABLE set to $callableTypeDefEnv_{update}$.
e. Return TC'.

2. Else if p matches pattern BLOCK:
   a. Let callableTypeDefEnv be TC.BLOCK.CALLABLE.
   b. Check that ~$in_set<callableId>(callableId, $dom_map<callableId, callableTypeDefIR>(callableTypeDefEnv)).
   c. Let $callableTypeDefEnv_{update}$ be callableTypeDefEnv where callableId to callableTypeDefIR is added.
   d. Let TC' be TC with BLOCK.CALLABLE set to $callableTypeDefEnv_{update}$.
   e. Return TC'.

The non-overloaded callable definition of prefixedNameIR from the p layer of TC

1. Try:
   a. If let '.' nameIR be prefixedNameIR:
      i. Return $find_non_overloaded<callableTypeDefIR>(TC.GLOBAL.CALLABLE, nameIR).
2. Then, try:
   a. If p matches pattern GLOBAL:
      i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
      ii. Let $_{BARE}$ nameIR be prefixedNameIR.
      iii. Return $find_non_overloaded<callableTypeDefIR>(TC.GLOBAL.CALLABLE, nameIR).
   b. Else if p matches pattern BLOCK:
      i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
      ii. Let $_{BARE}$ nameIR be prefixedNameIR.
      iii. Try:
         A. Let (callableId, callableTypeDefIR)$^{?}$ be $find_non_overloaded<callableTypeDefIR>(TC.BLOCK.CALLABLE, nameIR).
         B. If (callableId, callableTypeDefIR)$^{?}$ is defined:
            I. Let (callableId, callableTypeDefIR) be (callableId, callableTypeDefIR)$^{?}$.
            II. Return (callableId, callableTypeDefIR).
      iv. Then, try:
         A. Check that $find_non_overloaded<callableTypeDefIR>(TC.BLOCK.CALLABLE, nameIR) is none.
         B. Return the non-overloaded callable definition of $_{BARE}$ nameIR from the GLOBAL layer of TC.
   c. Else:
      i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
      ii. Let $_{BARE}$ nameIR be prefixedNameIR.
      iii. Return the non-overloaded callable definition of $_{BARE}$ nameIR from the BLOCK layer of TC.

The overloaded callable definition of prefixedNameIR `( argumentIR$^{*}$ `) from the p layer of TC

1. If p matches pattern GLOBAL:

a. If let '.' nameIR be prefixedNameIR:
    i. Let callableTypeDefEnv be TC.GLOBAL.CALLABLE.
    ii. Let callTargetKey be the call target key for nameIR `( argumentIR* `).
    iii. Return finding the item in callableTypeDefEnv matching callTargetKey using parameter getter $parameterListIR_of_callableTypeDefIR.
b. Else:
    i. Let $_{BARE}$ nameIR be prefixedNameIR.
    ii. Let callableTypeDefEnv be TC.GLOBAL.CALLABLE.
    iii. Let callTargetKey be the call target key for nameIR `( argumentIR* `).
    iv. Return finding the item in callableTypeDefEnv matching callTargetKey using parameter getter $parameterListIR_of_callableTypeDefIR.

2. Else if p matches pattern BLOCK:
a. If let $_{BARE}$ nameIR be prefixedNameIR:
    i. Let callableTypeDefEnv be TC.BLOCK.CALLABLE.
    ii. Let callTargetKey be the call target key for nameIR `( argumentIR* `).
    iii. Try:
        A. Let callResolution<callableTypeDefIR>? be finding the item in callableTypeDefEnv matching callTargetKey using parameter getter $parameterListIR_of_callableTypeDefIR.
        B. If callResolution<callableTypeDefIR>? is defined:
            I. Let callableId ':' callableTypeDefIR '#' $id_{default}$* '#' $id_{optional}$* be callResolution<callableTypeDefIR>?.
            II. Return callableId ':' callableTypeDefIR '#' $id_{default}$* '#' $id_{optional}$*.
    iv. Then, try:
        A. Check that finding the item in callableTypeDefEnv matching callTargetKey using parameter getter $parameterListIR_of_callableTypeDefIR is none.
        B. Return the overloaded callable definition of $_{BARE}$ nameIR `( argumentIR* `) from the GLOBAL layer of TC.
b. Else:
    i. Let '.' nameIR be prefixedNameIR.
    ii. Return the overloaded callable definition of '.' nameIR `( argumentIR* `) from the GLOBAL layer of TC.

3. Else:
a. If let $_{BARE}$ nameIR be prefixedNameIR:
    i. Return the overloaded callable definition of $_{BARE}$ nameIR `( argumentIR* `) from the BLOCK layer of TC.
b. Else:
    i. Let '.' nameIR be prefixedNameIR.
    ii. Return the overloaded callable definition of '.' nameIR `( argumentIR* `) from the GLOBAL layer of TC.

Constructor definitions

Where callableId to constructorTypeDefIR is added to TC

1. Let constructorTypeDefEnv be TC.GLOBAL.CONSTRUCTOR.
2. Let nameIR `( _* `) be callableId.
3. Let `{ ( $nameIR_k$ `(_* `) )* `} be the domain of the map constructorTypeDefEnv.

4. Check that ~$in_set<nameIR>(nameIR, `{ $nameIR_k^* `}).
5. Let constructorTypeDefEnv_update be constructorTypeDefEnv where callableId to constructorTypeDefIR is added.
6. Let TC' be TC with GLOBAL.CONSTRUCTOR set to constructorTypeDefEnv_update.
7. Return TC'.

Where callableId to constructorTypeDefIR is added to TC

1. Let constructorTypeDefEnv be TC.GLOBAL.CONSTRUCTOR.
2. Check that ~$in_set<callableId>(callableId, $dom_map<callableId, constructorTypeDefIR>(constructorTypeDefEnv)).
3. Let constructorTypeDefEnv_update be constructorTypeDefEnv where callableId to constructorTypeDefIR is added.
4. Let TC' be TC with GLOBAL.CONSTRUCTOR set to constructorTypeDefEnv_update.
5. Return TC'.

Where each of callableId^* to each of constructorTypeDefIR^* are added to TC

1. If callableId^* matches pattern []:
   a. Check that constructorTypeDefIR^* is an empty list.
   b. Return TC.
2. Else:
   a. Let callableId_h :: callableId_t^* be callableId^*.
   b. Check that constructorTypeDefIR^* is a non-empty list.
   c. Let constructorTypeDefIR_h :: constructorTypeDefIR_t^* be constructorTypeDefIR^*.
   d. Let TC' be where callableId_h to constructorTypeDefIR_h is added to TC.
   e. Let TC'' be where each of callableId_t^* to each of constructorTypeDefIR_t^* are added to TC'.
   f. Return TC''.

The overloaded constructor definition of prefixedNameIR `( argumentIR^* `) in TC

1. If let '.' nameIR be prefixedNameIR:
   a. Let callTargetKey be the call target key for nameIR `( argumentIR^* `).
   b. Return finding the item in TC.GLOBAL.CONSTRUCTOR matching callTargetKey using parameter getter $parameterListIR_of_constructorTypeDefIR.
2. Else:
   a. Let _BARE nameIR be prefixedNameIR.
   b. Let callTargetKey be the call target key for nameIR `( argumentIR^* `).
   c. Return finding the item in TC.GLOBAL.CONSTRUCTOR matching callTargetKey using parameter getter $parameterListIR_of_constructorTypeDefIR.

Constructor parameters

$add_constructorParameter_t(TC, _ direction typeIR id _)

1. Let varTypeIR be direction typeIR CTK ·.
2. Let TC' be TC where id to varTypeIR is added to the BLOCK layer.
3. Return TC'.

parameterIR* added to TC

1. If parameterIR* matches pattern []: return TC.
2. Else:
   a. Let $parameterIR_h$ :: $parameterIR_t^*$ be parameterIR*.
   b. Let TC' be $add_constructorParameter_t(TC, $parameterIR_h$).
   c. Let TC'' be $parameterIR_t^*$ added to TC'.
   d. Return TC''.

Parameters

_ direction typeIR id _ added to the cursor layer of TC

1. If direction is EMPTY:
   a. Let varTypeIR be EMPTY typeIR CTK ·.
   b. Let TC' be TC where id to varTypeIR is added to the cursor layer.
   c. Return TC'.
2. Else:
   a. Let varTypeIR be direction typeIR DYN ·.
   b. Let TC' be TC where id to varTypeIR is added to the cursor layer.
   c. Return TC'.

$add_parameters_t(cursor, TC, parameterIR*)

1. If parameterIR* matches pattern []: return TC.
2. Else:
   a. Let $parameterIR_h$ :: $parameterIR_t^*$ be parameterIR*.
   b. Let TC' be $parameterIR_h$ added to the cursor layer of TC.
   c. Let TC'' be $add_parameters_t(cursor, TC', $parameterIR_t^*$).
   d. Return TC''.

Return types

The expected return type in TC

1. Try:
   a. Let localKind be TC.LOCAL.KIND.
   b. If let FUNCTION ':' $typeIR_{ret}$ of type definedFunctionLocalKind be localKind: return $typeIR_{ret}$.

c. Else if let EXTERN_FUNCTION ':' $typeIR_{ret}$ of type externFunctionLocalKind be localKind: return $typeIR_{ret}$.

2. Then, try:
    a. If ACTION is equal to TC.LOCAL.KIND: return VOID.
3. Then, try:
    a. Let localKind be TC.LOCAL.KIND.
    b. If let externMethodLocalKind of type externMethodLocalKind be localKind:
        i. If let EXTERN_METHOD ':' $typeIR_{ret}$ be externMethodLocalKind: return $typeIR_{ret}$.
        ii. Else:
            A. Let EXTERN_METHOD ABSTRACT ':' $typeIR_{ret}$ be externMethodLocalKind.
            B. Return $typeIR_{ret}$.
4. Then, try:
    a. If CONTROL_APPLY_METHOD is equal to TC.LOCAL.KIND: return VOID.
5. Otherwise: return ·.

Scope entry and exit

TC with a new local frame

1. Return TC with LOCAL.FRAMES set to $empty_typeFrame :: TC.LOCAL.FRAMES.

TC with the innermost frame removed

1. Let $typeFrame^*$ be TC.LOCAL.FRAMES.
2. Check that $typeFrame^*$ is a non-empty list.
3. Let $typeFrame_h$ :: $typeFrame_t^*$ be $typeFrame^*$.
4. Return TC with LOCAL.FRAMES set to $typeFrame_t^*$.

Copying and merging

The merged extern method definition environment of externMethodDefEnv and set<pair<callableId, callableDef>>

1. Try:
    a. If set<pair<callableId, callableDef>> is equal to `{ · `}: return externMethodDefEnv.
2. Then, try:
    a. Let `{ $pair<callableId, callableDef>^*$ `} be set<pair<callableId, callableDef>>.
    b. Check that $pair<callableId, callableDef>^*$ is a non-empty list.
    c. Let $callableId_h$ ':' $callableDef_h$ :: ($callableId_t$ ':' $callableDef_t$)* be $pair<callableId, callableDef>^*$.
    d. Check that $callableDef_h$ has type definedFunctionDef.
    e. Let FUNCTION nameIR `< typeParameterListIR `> `( parameterListIR `) blockStatementIR be $callableDef_h$.
    f. Let $externMethodDef_h$ be EXTERN_METHOD nameIR `< typeParameterListIR `> `( parameterListIR `) blockStatementIR.
    g. Let $externMethodDefEnv_{post}$ be the map externMethodDefEnv with the value for $callableId_h$ is updated to $externMethodDef_h$.

h. Return the merged extern method definition environment of $externMethodDefEnv_{post}$ and `{ ( $callableId_t$ ':' $callableDef_t$ )* `}.

The merged frame of frame and `{ ( id ':' value )* `}

1. Return frame where each of id* to value* are added.

M.2. Instantiation context

The instantiation context is threaded through compile-time evaluation, when constructor calls allocate objects. It binds the variables, types, and definitions available at instantiation time. Its operations are suffixed with _i.

Empty environments

An empty instantiation context

1. Let globalInstLayer be {+CONSTRUCTOR+ $empty_constructorDefEnv, TYPE $empty_typeDefEnv, CALLABLE $empty_callableDefEnv, FRAME $empty_frame}.
2. Let blockInstLayer be {+TYPE+ $empty_theta, CALLABLE $empty_callableDefEnv, STATE $empty_stateEnv, FRAME $empty_frame}.
3. Let localInstLayer be {+TYPE+ $empty_theta, FRAMES $empty_frame}.
4. Let IC be {+PATH+ ·, GLOBAL globalInstLayer, BLOCK blockInstLayer, LOCAL localInstLayer}.
5. Return IC.

Variables

IC where id to value is added to the p layer

1. If p matches pattern GLOBAL:
   a. Check that ~$in_set<id>(id, $dom_map<id, value>(IC.GLOBAL.FRAME)).
   b. Let frame be IC.GLOBAL.FRAME where id to value is added.
   c. Return IC with GLOBAL.FRAME set to frame.
2. Else if p matches pattern BLOCK:
   a. Check that ~$in_set<id>(id, $dom_map<id, value>(IC.BLOCK.FRAME)).
   b. Let frame be IC.BLOCK.FRAME where id to value is added.
   c. Return IC with BLOCK.FRAME set to frame.
3. Else:
   a. Let frame* be IC.LOCAL.FRAMES.
   b. Check that frame* is a non-empty list.
   c. Let $frame_h$ :: $frame_t^*$ be frame*.
   d. Check that ~$in_set<id>(id, $dom_map<id, value>($frame_h$)).
   e. Let $frame_{h'}$ be $frame_h$ where id to value is added.
   f. Return IC with LOCAL.FRAMES set to $frame_{h'}$ :: $frame_t^*$.

IC where each of id^* to each of $value^*$ are added to the p layer

1. If id^* matches pattern []:
    a. Check that $value^*$ is an empty list.
    b. Return IC.
2. Else:
    a. Let $id_h :: id_t^*$ be id^*.
    b. Check that $value^*$ is a non-empty list.
    c. Let $value_h :: value_t^*$ be $value^*$.
    d. Let IC' be IC where id_h to $value_h$ is added to the p layer.
    e. Let IC'' be IC' where each of id_t^* to each of $value_t^*$ are added to the p layer.
    f. Return IC''.

Types

$add_type_i(p, IC, typeId, typeIR)

1. If p matches pattern BLOCK:
    a. Let theta be IC.BLOCK.TYPE where typeId to typeIR is added.
    b. Return IC with BLOCK.TYPE set to theta.
2. Else if p matches pattern LOCAL:
    a. Let theta be IC.LOCAL.TYPE where typeId to typeIR is added.
    b. Return IC with LOCAL.TYPE set to theta.

$add_types_i(p, IC, $typeId^*$, $typeIR^*$)

1. If $typeId^*$ matches pattern []:
    a. Check that $typeIR^*$ is an empty list.
    b. Return IC.
2. Else:
    a. Let $typeId_h :: typeId_t^*$ be $typeId^*$.
    b. Check that $typeIR^*$ is a non-empty list.
    c. Let $typeIR_h :: typeIR_t^*$ be $typeIR^*$.
    d. Let IC' be $add_type_i(p, IC, $typeId_h$, $typeIR_h$).
    e. Return $add_types_i(p, IC', $typeId_t^*$, $typeIR_t^*$).

Type definitions

IC where typeId to typeDefIR is added to the p layer

1. Check that p is GLOBAL.
2. Let typeDefEnv be IC.GLOBAL.TYPE where typeId to typeDefIR is added.

3. Return IC with GLOBAL.TYPE set to typeDefEnv.

The type definition of prefixedNameIR from the _ layer of IC

1. If let $_{\text{BARE}}$ nameIR be prefixedNameIR:
   a. Return the value of nameIR in map IC.GLOBAL.TYPE.
2. Else:
   a. Let '.' nameIR be prefixedNameIR.
   b. Return the value of nameIR in map IC.GLOBAL.TYPE.

Callable definitions

IC where callableId to callableDef is added as a non-overloaded callable to the p layer

1. If p matches pattern GLOBAL:
   a. Let id `( _* `) be callableId.
   b. Let `{ ( $id_k$ `(_* `) )* `} be the domain of the map IC.GLOBAL.CALLABLE.
   c. Check that ~$in_set<id>(id, `{ $id_k$* `}).
   d. Let callableDefEnv be IC.GLOBAL.CALLABLE where callableId to callableDef is added.
   e. Return IC with GLOBAL.CALLABLE set to callableDefEnv.
2. Else if p matches pattern BLOCK:
   a. Let id `( _* `) be callableId.
   b. Let `{ ( $id_k$ `(_* `) )* `} be the domain of the map IC.BLOCK.CALLABLE.
   c. Check that ~$in_set<id>(id, `{ $id_k$* `}).
   d. Let callableDefEnv be IC.BLOCK.CALLABLE where callableId to callableDef is added.
   e. Return IC with BLOCK.CALLABLE set to callableDefEnv.

IC where callableId to callableDef is added as an overloaded callable to the p layer

1. If p matches pattern GLOBAL:
   a. Check that ~$in_set<callableId>(callableId, $dom_map<callableId, callableDef>(IC.GLOBAL.CALLABLE)).
   b. Let callableDefEnv be IC.GLOBAL.CALLABLE where callableId to callableDef is added.
   c. Return IC with GLOBAL.CALLABLE set to callableDefEnv.
2. Else if p matches pattern BLOCK:
   a. Check that ~$in_set<callableId>(callableId, $dom_map<callableId, callableDef>(IC.BLOCK.CALLABLE)).
   b. Let callableDefEnv be IC.BLOCK.CALLABLE where callableId to callableDef is added.
   c. Return IC with BLOCK.CALLABLE set to callableDefEnv.

The non-overloaded callable definition of prefixedNameIR from the p layer of IC

1. Try:

a. If let '.' nameIR be prefixedNameIR:
    i. Try:
        A. If $find_non_overloaded<callableDef>(IC.GLOBAL.CALLABLE, nameIR) is none: return ·.
    ii. Then, try:
        A. Let (callableId, callableDef)? be $find_non_overloaded<callableDef>(IC.GLOBAL.CALLABLE, nameIR).
        B. If (callableId, callableDef)? is defined:
            I. Let (callableId, callableDef) be (callableId, callableDef)?.
            II. Return (GLOBAL, callableId, callableDef).
2. Then, try:
    a. If p matches pattern GLOBAL:
        i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
        ii. Let $_{BARE}$ nameIR be prefixedNameIR.
        iii. Return the non-overloaded callable definition of '.' nameIR from the GLOBAL layer of IC.
    b. Else if p matches pattern BLOCK:
        i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
        ii. Let $_{BARE}$ nameIR be prefixedNameIR.
        iii. Try:
            A. Let (callableId, callableDef)? be $find_non_overloaded<callableDef>(IC.BLOCK.CALLABLE, nameIR).
            B. If (callableId, callableDef)? is defined:
                I. Let (callableId, callableDef) be (callableId, callableDef)?.
                II. Return (BLOCK, callableId, callableDef).
        iv. Then, try:
            A. Check that $find_non_overloaded<callableDef>(IC.BLOCK.CALLABLE, nameIR) is none.
            B. Return the non-overloaded callable definition of $_{BARE}$ nameIR from the GLOBAL layer of IC.
    c. Else:
        i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
        ii. Let $_{BARE}$ nameIR be prefixedNameIR.
        iii. Return the non-overloaded callable definition of $_{BARE}$ nameIR from the BLOCK layer of IC.

Constructor definitions

Where callableId to constructorDef is added to IC

1. Let constructorDefEnv be IC.GLOBAL.CONSTRUCTOR where callableId to constructorDef is added.
2. Return IC with GLOBAL.CONSTRUCTOR set to constructorDefEnv.

Where each of callableId* to each of constructorDef* are added to IC

1. If callableId* matches pattern []:
    a. Check that constructorDef* is an empty list.
    b. Return IC.
2. Else:

a. Let callableId$_h$ :: callableId$_t^*$ be callableId*.

b. Check that constructorDef* is a non-empty list.

c. Let constructorDef$_h$ :: constructorDef$_t^*$ be constructorDef*.

d. Let IC' be where callableId$_h$ to constructorDef$_h$ is added to IC.

e. Let IC'' be where each of callableId$_t^*$ to each of constructorDef$_t^*$ are added to IC'.

f. Return IC''.

The overloaded constructor definition of prefixedNameIR `( argumentIR$^*$ `) in IC

1. If let $_{BARE}$ nameIR be prefixedNameIR:
   a. Let callTargetKey be the call target key for nameIR `( argumentIR$^*$ `).
   b. Return finding the item in IC.GLOBAL.CONSTRUCTOR matching callTargetKey using parameter getter $parameterListIR_of_constructorDef.
2. Else:
   a. Let '.' nameIR be prefixedNameIR.
   b. Let callTargetKey be the call target key for nameIR `( argumentIR$^*$ `).
   c. Return finding the item in IC.GLOBAL.CONSTRUCTOR matching callTargetKey using parameter getter $parameterListIR_of_constructorDef.

Parser states

IC where nameIR to parserStateIR is added

1. Check that ~$in_set<nameIR>(nameIR, $dom_map<nameIR, parserStateIR>(IC.BLOCK.STATE)).
2. Let stateEnv be IC.BLOCK.STATE where nameIR to parserStateIR is added.
3. Return IC with BLOCK.STATE set to stateEnv.

Scope entry and exit

IC with a new local frame

1. Return IC with LOCAL.FRAMES set to $empty_frame :: IC.LOCAL.FRAMES.

IC with id added to the path

1. Return IC with PATH set to IC.PATH ++ id.

IC with the innermost frame removed

1. Let frame* be IC.LOCAL.FRAMES.
2. Check that frame* is a non-empty list.
3. Let frame$_h$ :: frame$_t^*$ be frame*.

4. Return IC with LOCAL.FRAMES set to $frame_t^*$.

An instantiation context same as IC up to the p layer

1. If p matches pattern GLOBAL:
    a. Let blockInstLayer be {+TYPE+ $empty_theta, CALLABLE $empty_callableDefEnv, STATE $empty_stateEnv, FRAME $empty_frame}.
    b. Let localInstLayer be {+TYPE+ $empty_theta, FRAMES $empty_frame}.
    c. Return IC[BLOCK = blockInstLayer] with LOCAL set to localInstLayer.
2. Else if p matches pattern BLOCK:
    a. Let localInstLayer be {+TYPE+ $empty_theta, FRAMES $empty_frame}.
    b. Return IC with LOCAL set to localInstLayer.
3. Else: return IC.

M.3. Execution context

The execution context holds the runtime information: the value environment, or store, mapping variables to values, the object and architecture state, the parser state, and the callable and constructor definition environments. Its operations are suffixed with _e.

Empty environments

$empty_callableDefEnv

1. Return an empty map.

$empty_constructorDefEnv

1. Return an empty map.

$empty_stateEnv

1. Return an empty map.

An empty store

1. Let store be an empty map.
2. Return store.

Variables

EC where prefixedNameIR to value is added to the p layer

1. If p matches pattern GLOBAL:

a. If let '.' id be prefixedNameIR:
    i. Let frame be EC.GLOBAL.FRAME where id to value is added.
    ii. Return EC with GLOBAL.FRAME set to frame.
b. Else:
    i. Let $_{\text{BARE}}$ id be prefixedNameIR.
    ii. Let frame be EC.GLOBAL.FRAME where id to value is added.
    iii. Return EC with GLOBAL.FRAME set to frame.

2. Else if p matches pattern BLOCK:
    a. Check that prefixedNameIR matches pattern $_{\text{BARE}}$ %.
    b. Let $_{\text{BARE}}$ id be prefixedNameIR.
    c. Let frame be EC.BLOCK.FRAME where id to value is added.
    d. Return EC with BLOCK.FRAME set to frame.
3. Else:
    a. Check that prefixedNameIR matches pattern $_{\text{BARE}}$ %.
    b. Let $_{\text{BARE}}$ id be prefixedNameIR.
    c. Let $frame^*$ be EC.LOCAL.FRAMES.
    d. Check that $frame^*$ is a non-empty list.
    e. Let $frame_h :: frame_t^*$ be $frame^*$.
    f. Let $frame_{h'}$ be $frame_h$ where id to value is added.
    g. Return EC with LOCAL.FRAMES set to $frame_{h'} :: frame_t^*$.

$add_vars_e(p, EC, $prefixedNameIR^*$, $value^*$)

1. If $prefixedNameIR^*$ matches pattern []:
    a. Check that $value^*$ is an empty list.
    b. Return EC.
2. Else:
    a. Let $prefixedNameIR_h :: prefixedNameIR_t^*$ be $prefixedNameIR^*$.
    b. Check that $value^*$ is a non-empty list.
    c. Let $value_h :: value_t^*$ be $value^*$.
    d. Let EC' be EC where $prefixedNameIR_h$ to $value_h$ is added to the p layer.
    e. Let EC'' be $add_vars_e(p, EC', $prefixedNameIR_t^*$, $value_t^*$).
    f. Return EC''.

EC where prefixedNameIR is updated to value from the p layer

1. Try:
    a. If let '.' id be prefixedNameIR:
        i. Let frame be the map EC.GLOBAL.FRAME with the value for id is updated to value.
        ii. Return EC with GLOBAL.FRAME set to frame.
2. Then, try:

a. If p matches pattern GLOBAL:
    i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
    ii. Let $_{BARE}$ id be prefixedNameIR.
    iii. Let frame be EC.GLOBAL.FRAME.
    iv. Check that id is an element of the set the domain of the map frame.
    v. Let frame' be the map frame with the value for id is updated to value.
    vi. Return EC with GLOBAL.FRAME set to frame'.
b. Else if p matches pattern BLOCK:
    i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
    ii. Let $_{BARE}$ id be prefixedNameIR.
    iii. Let frame be EC.BLOCK.FRAME.
    iv. If id is an element of the set the domain of the map frame:
        A. Let frame' be the map frame with the value for id is updated to value.
        B. Return EC with BLOCK.FRAME set to frame'.
    v. Else:
        A. Return EC where $_{BARE}$ id is updated to value from the GLOBAL layer.
c. Else:
    i. Check that prefixedNameIR matches pattern $_{BARE}$ %.
    ii. Let $_{BARE}$ id be prefixedNameIR.
    iii. Try:
        A. If EC.LOCAL.FRAMES is an empty list:
            I. Return EC where $_{BARE}$ id is updated to value from the BLOCK layer.
    iv. Then, try:
        A. Let $frame^*$ be EC.LOCAL.FRAMES.
        B. Check that $frame^*$ is a non-empty list.
        C. Let $frame_h :: frame_t^*$ be $frame^*$.
        D. If id is an element of the set the domain of the map $frame_h$:
            I. Let $frame_{h'}$ be the map $frame_h$ with the value for id is updated to value.
            II. Return EC with LOCAL.FRAMES set to $frame_{h'} :: frame_t^*$.
        E. Else:
            I. Let EC_{pop} be EC with LOCAL.FRAMES set to $frame_t^*$.
            II. Let $EC_{pop'}$ be EC_{pop} where $_{BARE}$ id is updated to value from the LOCAL layer.
            III. Return $EC_{pop'}$ with LOCAL.FRAMES set to $frame_h :: EC_{pop'}$.LOCAL.FRAMES.

Types

$find_type_e(p, EC, nameIR)

1. If p matches pattern BLOCK:
    a. Return the value of nameIR in map EC.BLOCK.TYPE.
2. Else if p matches pattern LOCAL:
    a. Try:

i. Let typeIR'? be the value of nameIR in map EC.LOCAL.TYPE.

ii. If typeIR'? is defined:

A. Let typeIR be typeIR'?.

B. Return typeIR.

b. Then, try:

i. Check that the value of nameIR in map EC.LOCAL.TYPE is none.

ii. Return $find_type_e(BLOCK, EC, nameIR).

Type definitions

$find_typeDef_e(_, EC, prefixedNameIR)

1. If let BARE nameIR be prefixedNameIR:
   a. Return the value of nameIR in map EC.GLOBAL.TYPE.
2. Else:
   a. Let '.' nameIR be prefixedNameIR.
   b. Return the value of nameIR in map EC.GLOBAL.TYPE.

Callable definitions

The overloaded callable definition of prefixedNameIR(argumentIR*) in the p layer of EC

1. If p matches pattern GLOBAL:
   a. If let BARE nameIR be prefixedNameIR:
      i. Let callTargetKey be the call target key for nameIR `( argumentIR* `).
      ii. Try:
         A. Let callResolution<callableDef>? be finding the item in EC.GLOBAL.CALLABLE matching callTargetKey using parameter getter $parameterListIR_of_callableDef.
         B. If callResolution<callableDef>? is defined:
            I. Let callableId ':' callableDef '#' $id_{default}$* '#' $id_{optional}$* be callResolution<callableDef>?.
            II. Return callableId ':' (GLOBAL, callableDef) '#' $id_{default}$* '#' $id_{optional}$*.
      iii. Then, try:
         A. Check that finding the item in EC.GLOBAL.CALLABLE matching callTargetKey using parameter getter $parameterListIR_of_callableDef is none.
         B. Return ·.
   b. Else:
      i. Let '.' nameIR be prefixedNameIR.
      ii. Let callTargetKey be the call target key for nameIR `( argumentIR* `).
      iii. Try:
         A. Let callResolution<callableDef>? be finding the item in EC.GLOBAL.CALLABLE matching callTargetKey using parameter getter $parameterListIR_of_callableDef.
         B. If callResolution<callableDef>? is defined:
            I. Let callableId ':' callableDef '#' $id_{default}$* '#' $id_{optional}$* be callResolution<callableDef>?.

II. Return callableId ':' (GLOBAL, callableDef) '#' $id_{default}^{*}$ '#' $id_{optional}^{*}$.

iv. Then, try:

A. Check that finding the item in EC.GLOBAL.CALLABLE matching callTargetKey using parameter getter $parameterListIR_of_callableDef is none.

B. Return ·.

2. Else if p matches pattern BLOCK:

a. If let $_{BARE}$ nameIR be prefixedNameIR:

i. Let callTargetKey be the call target key for nameIR `( $argumentIR^{*}$ `).

ii. Try:

A. Let callResolution<callableDef>$^{?}$ be finding the item in EC.BLOCK.CALLABLE matching callTargetKey using parameter getter $parameterListIR_of_callableDef.

B. If callResolution<callableDef>$^{?}$ is defined:

I. Let callableId ':' callableDef '#' $id_{default}^{*}$ '#' $id_{optional}^{*}$ be callResolution<callableDef>$^{?}$.

II. Return callableId ':' (BLOCK, callableDef) '#' $id_{default}^{*}$ '#' $id_{optional}^{*}$.

iii. Then, try:

A. Check that finding the item in EC.BLOCK.CALLABLE matching callTargetKey using parameter getter $parameterListIR_of_callableDef is none.

B. Return the overloaded callable definition of $_{BARE}$ nameIR($argumentIR^{*}$) in the GLOBAL layer of EC.

b. Else:

i. Let '.' nameIR be prefixedNameIR.

ii. Return the overloaded callable definition of '.' nameIR($argumentIR^{*}$) in the GLOBAL layer of EC.

3. Else:

a. If let $_{BARE}$ nameIR be prefixedNameIR:

i. Return the overloaded callable definition of $_{BARE}$ nameIR($argumentIR^{*}$) in the BLOCK layer of EC.

b. Else:

i. Let '.' nameIR be prefixedNameIR.

ii. Return the overloaded callable definition of '.' nameIR($argumentIR^{*}$) in the GLOBAL layer of EC.

Constructor definitions

$find_constructorDef_e(EC, prefixedNameIR, $argumentIR^{*}$)

1. If let $_{BARE}$ nameIR be prefixedNameIR:

a. Let callTargetKey be the call target key for nameIR `( $argumentIR^{*}$ `).

b. Return finding the item in EC.GLOBAL.CONSTRUCTOR matching callTargetKey using parameter getter $parameterListIR_of_constructorDef.

2. Else:

a. Let '.' nameIR be prefixedNameIR.

b. Let callTargetKey be the call target key for nameIR `( $argumentIR^{*}$ `).

c. Return finding the item in EC.GLOBAL.CONSTRUCTOR matching callTargetKey using parameter getter $parameterListIR_of_constructorDef.

Objects

$add_object_e(ARCH, objectId, object)

1. Let STO be ARCH.STORE where objectId to object is added.
2. Return ARCH with STORE set to STO.

The object objectId in ARCH

1. Return the value of objectId in map ARCH.STORE.

$find_object_qualified_e(ARCH, objectId)

1. Return the object objectId in ARCH.

$find_object_unqualified_e(ARCH, id)

1. Let `{ ( objectId ':' object )* `} be ARCH.STORE.
2. For each objectId in objectId*:

   - Let bool be objectId is an object name that ends with id.

   Let bool* be the resulting list.
3. For each bool in bool* and object in object*:

   - Let $object_{id}^{?}$ be object if bool otherwise ·.

   Let $object_{id}^{?*}$ be the resulting list.
4. Let $object''^{?*}$ be $object_{id}^{?*}$ filtered by $(object_{id}^{?} =/= \cdot)^{*}$.
5. If $object''^{?*}$ is a list of length 1:
   a. Let $object'^{?}$ be $object''^{?*}$.
   b. If $object'^{?}$ is defined:
      i. Let $object_{id_found}$ be $object'^{?}$.
      ii. Return $object_{id_found}$.
6. Otherwise: return ·.

$update_object_qualified_e($ARCH_0$, objectId, object)

1. Check that $find_object_qualified_e($ARCH_0$, objectId) is defined.
2. Let $ARCH_1$ be $add_object_e($ARCH_0$, objectId, object).
3. Return $ARCH_1$.

$update_object_unqualified_e($ARCH_0$, id, object)

1. Let `{ ( objectId ':' _ )* `} be $ARCH_0$.STORE.
2. For each objectId in objectId*:
   - Let bool be objectId is an object name that ends with id.

   Let bool* be the resulting list.
3. For each bool in bool* and objectId in objectId*:
   - Let $objectId_{id}^?$ be objectId if bool otherwise ·.

   Let $objectId_{id}^{?*}$ be the resulting list.
4. Let $objectId''^{?*}$ be $objectId_{id}^{?*}$ filtered by $(objectId_{id}^? =/= ·)^*$.
5. Check that $objectId''^{?*}$ is a list of length 1.
6. Let $objectId'^?$ be $objectId''^{?*}$.
7. Check that $objectId'^?$ is defined.
8. Let $objectId_{found}$ be $objectId'^?$.
9. Let $ARCH_1$ be $add_object_e($ARCH_0$, $objectId_{found}$, object).
10. Return $ARCH_1$.

Object state

$find_objectState_e(ARCH, objectId)

1. Let $object'^?$ be the object objectId in ARCH.
2. If $object'^?$ is defined:
   a. Let object be $object'^?$.
   b. If let externObject of type externObject be object:
      i. Let EXTERN _ `< _ `> `{ objectState _ _ `} be externObject.
      ii. Return objectState.
3. Otherwise: return ·.

$update_objectState_e($ARCH_0$, objectId, objectState)

1. Let $object'^?$ be the object objectId in $ARCH_0$.
2. If $object'^?$ is defined:
   a. Let object be $object'^?$.
   b. If let externObject of type externObject be object:
      i. Let EXTERN typeId `< theta `> `{ _ frame externMethodDefEnv `} be externObject.
      ii. Let $externObject_{update}$ be EXTERN typeId `< theta `> `{ objectState frame externMethodDefEnv `}.
      iii. Let $ARCH_1$ be $add_object_e($ARCH_0$, objectId, $externObject_{update}$).
      iv. Return $ARCH_1$.
3. Otherwise: return ·.

Store

store where objectId to object is added

1. Let store' be store where objectId to object is added.
2. Return store'.

The object objectId in store

1. Let object'? be the value of objectId in map store.
2. Check that object'? is defined.
3. Let object be object'?.
4. Return object.

Parser states

The parser state of nameIR in EC

1. Return the value of nameIR in map EC.BLOCK.STATE.

Actions

$find_action(TBLC, prefixedNameIR)

1. Return $find_action'(TBLC.ACTIONS, prefixedNameIR).

$find_action'(matchAction*, prefixedNameIR)

1. If matchAction* matches pattern []: return ·.
2. Else:
   a. Let $matchAction_h$ :: $matchAction_t^*$ be matchAction*.
   b. Let prefixedNameIR' `( parameterIR* '@' argumentListIR `) be $matchAction_h$.
   c. If prefixedNameIR' is equal to prefixedNameIR:
      i. Return (parameterIR*, argumentListIR).
   d. Else:
      i. Return $find_action'(matchAction_t*, prefixedNameIR).

Architecture state

$find_archState_e(ARCH)

1. Return ARCH.STATE.

$update_archState_e(ARCH, archState)

1. Return ARCH with STATE set to archState.

Annotations

$add_annotationList(annotationList, annotation'?)

1. Try:
    a. If annotation'? is none: return annotationList.
2. Then, try:
    a. If annotationList matches pattern EMPTY:
        i. Check that annotation'? is defined.
        ii. Let annotation be annotation'?.
        iii. Return annotation.
    b. Else:
        i. Let annotationListNonEmpty be annotationList.
        ii. Check that annotation'? is defined.
        iii. Let annotation be annotation'?.
        iv. Return annotationListNonEmpty annotation.

Scope entry and exit

EC with a new local frame

1. Return EC with LOCAL.FRAMES set to $empty_frame :: EC.LOCAL.FRAMES.

EC with the innermost frame removed

1. Let frame* be EC.LOCAL.FRAMES.
2. Check that frame* is a non-empty list.
3. Let $frame_h$ :: $frame_t^*$ be frame*.
4. Return EC with LOCAL.FRAMES set to $frame_t^*$.

Copy of EC from the p layer

1. If p matches pattern GLOBAL:
    a. Let blockEvalLayer be {+TYPE+ $empty_theta, CALLABLE $empty_callableDefEnv, STATE $empty_stateEnv, FRAME $empty_frame}.
    b. Let localEvalLayer be {+TYPE+ $empty_theta, FRAMES $empty_frame}.
    c. Return EC[BLOCK = blockEvalLayer] with LOCAL set to localEvalLayer.
2. Else if p matches pattern BLOCK:

a. Let localEvalLayer be {+TYPE+ $empty_theta, FRAMES $empty_frame}.

b. Return EC with LOCAL set to localEvalLayer.

3. Else: return EC.

Copying and merging

$copy_e(p, EC_{from}, EC_{to})

1. If p matches pattern GLOBAL:
   a. Return EC_{to} with GLOBAL set to EC_{from}.GLOBAL.
2. Else if p matches pattern BLOCK:
   a. Return EC_{to}[GLOBAL = EC_{from}.GLOBAL] with BLOCK set to EC_{from}.BLOCK.
3. Else:
   a. Return EC_{to}[GLOBAL = EC_{from}.GLOBAL][BLOCK = EC_{from}.BLOCK] with LOCAL set to EC_{from}.LOCAL.

Appendix N: Structural projections and identifiers

This appendix collects the projection functions that extract identifiers and sub-structures — such as parameter lists, type parameter lists, and signatures — from $P4_{IR}$ definitions, types, and extern prototypes. These projections let the specification treat the many kinds of callable and constructor uniformly.

Callable identifiers

The identifier of a callable is projected from a callable definition, a callable type, and an extern method prototype with:

The callable identifier for name `( parameterList `)

1. Let nameIR be the name of name.
2. Let id^* be $callableId'(parameterList).
3. Return nameIR `( $id^*$ `).

$callableId'(parameterList)

1. Try:
   a. If parameterList matches pattern $_{EMPTY}$: return ·.
   b. Else if let _ _ _ name _ of type parameter be parameterList:
      i. Let nameIR be the name of name.
      ii. Return nameIR.
2. Then, try:
   a. Check that parameterList has type nonEmptyParameterList.

b. Let nonEmptyParameterList' be parameterList.

c. Check that nonEmptyParameterList' matches pattern % ',' %.

d. Let nonEmptyParameterList ',' _ _ _ name _ be nonEmptyParameterList'.

e. Let id* be $callableId'(nonEmptyParameterList).

f. Let nameIR' be the name of name.

g. Return id* concatenated with nameIR'.

The callable identifier for nameIR `( parameterIR* `)

1. For each parameterIR in parameterIR*:

   - Let _ _ _ id _ be parameterIR.

   Let id* be the resulting list.

2. Return nameIR `( id* `).

The constructor id of _ nameIR `< ',' _ `> `( parameterIR* `) ';'

1. Return the callable identifier for nameIR `( parameterIR* `).

Callable id of externMethodPrototype

1. If let _ _ name _ `( parameterList `) ';' be externMethodPrototype:
   a. Return the callable identifier for name `( parameterList `).
2. Else:
   a. Let _ ABSTRACT _ name _ `( parameterList `) ';' be externMethodPrototype.
   b. Return the callable identifier for name `( parameterList `).

The callable id of externMethodPrototypeIR

1. If let _ _ nameIR `< _ ',' _ `> `( parameterListIR `) ';' be externMethodPrototypeIR:
   a. Return the callable identifier for nameIR `( parameterListIR `).
2. Else:
   a. Let _ ABSTRACT _ nameIR `< _ ',' _ `> `( parameterListIR `) ';' be externMethodPrototypeIR.
   b. Return the callable identifier for nameIR `( parameterListIR `).

Constructor identifiers

The identifier of a constructor is projected from a constructor definition and an extern constructor prototype with:

The constructor identifier for name constructorParameterListOpt

1. If constructorParameterListOpt matches pattern $_{\text{EMPTY}}$:
    a. Let nameIR be the name of name.
    b. Return nameIR `( · `).
2. Else:
    a. Let `( parameterList `) be constructorParameterListOpt.
    b. Return the callable identifier for name `( parameterList `).

Constructor id of _ typeIdentifier `( parameterList `) ';'

1. Return the callable identifier for typeIdentifier `( parameterList `).

Type and object identifiers

The identifier of a type definition, and the test on object identifiers, are:

objectId is an object name that ends with nameIR

1. Let nameIR''* be the reversal of objectId.
2. If nameIR''* is a non-empty list:
    a. Let nameIR' :: _* be nameIR''*.
    b. If nameIR' is equal to nameIR: return true.
3. Otherwise: return false.

The name of typeDefIR

1. If let $_{\text{NAME}}$ typeId of type nameTypeIR be typeDefIR: return typeId.
2. Else if let TYPEDEF typeId _ of type typedefTypeIR be typeDefIR: return typeId.
3. Else if let TYPE typeId _ of type newTypeIR be typeDefIR: return typeId.
4. Else if let STRUCT typeId `< _$^*$ `> `{ _$^*$ `} of type structTypeDefIR be typeDefIR: return typeId.
5. Else if let HEADER typeId `< _$^*$ `> `{ _$^*$ `} of type headerTypeDefIR be typeDefIR: return typeId.
6. Else if let HEADER_UNION typeId `< _$^*$ `> `{ _$^*$ `} of type headerUnionTypeDefIR be typeDefIR: return typeId.
7. Else if let ENUM typeId `{ _$^*$ `} of type simpleEnumTypeIR be typeDefIR: return typeId.
8. Else if let ENUM typeId `< _ `> `{ _$^*$ `} of type serializableEnumTypeIR be typeDefIR: return typeId.
9. Else if let EXTERN typeId `< _$^*$ ',' _$^*$ `> `{ _$^*$ `} of type externObjectTypeDefIR be typeDefIR: return typeId.
10. Else if let PARSER typeId `< _$^*$ ',' _$^*$ `> `( _$^*$ `) of type parserObjectTypeDefIR be typeDefIR: return typeId.
11. Else if let CONTROL typeId `< _$^*$ ',' _$^*$ `> `( _$^*$ `) of type controlObjectTypeDefIR be typeDefIR: return typeId.
12. Else if let PACKAGE typeId `< _$^*$ ',' _$^*$ `> `{ _$^*$ `} of type packageObjectTypeDefIR be typeDefIR: return typeId.
13. Else:
    a. Let TABLE typeId `{ _ `} be typeDefIR.
    b. Return typeId.

Parameter lists

The parameter list is projected from every kind of callable and constructor definition, and from their corresponding types, with:

$parameterListIR_of_actionDef(annotationList ACTION _ `( parameterListIR `) _)

1. Return parameterListIR.

$parameterListIR_of_actionTypeDefIR(_ ACTION _ `( parameterIR* `))

1. Return parameterIR*.

$parameterListIR_of_callableDef(callableDef)

1. If let actionDef of type actionDef be callableDef:
    a. Return $parameterListIR_of_actionDef(actionDef).
2. Else if let functionDef of type functionDef be callableDef:
    a. Return $parameterListIR_of_functionDef(functionDef).
3. Else:
    a. Let methodDef be callableDef.
    b. Return $parameterListIR_of_methodDef(methodDef).

The parameters of callableTypeDefIR

1. If let actionTypeDefIR of type actionTypeDefIR be callableTypeDefIR:
    a. Return $parameterListIR_of_actionTypeDefIR(actionTypeDefIR).
2. Else if let functionTypeDefIR of type functionTypeDefIR be callableTypeDefIR:
    a. Return $parameterListIR_of_functionTypeDefIR(functionTypeDefIR).
3. Else:
    a. Let methodTypeDefIR be callableTypeDefIR.
    b. Return $parameterListIR_of_methodTypeDefIR(methodTypeDefIR).

$parameterListIR_of_constructorDef(constructorDef)

1. If let EXTERN _ `< _ `> `( constructorParameterListIR `) `{ _ `} of type externObjectConstructorDef be constructorDef: return constructorParameterListIR.
2. Else if let PARSER `< _ `> `( _ `) `( constructorParameterListIR `) `{ _ _ `} of type parserObjectConstructorDef be constructorDef: return constructorParameterListIR.
3. Else if let CONTROL `< _ `> `( _ `) `( constructorParameterListIR `) `{ _ APPLY _ `} of type controlObjectConstructorDef be constructorDef: return constructorParameterListIR.
4. Else:
    a. Let PACKAGE `< _ `> `( constructorParameterListIR `) be constructorDef.
    b. Return constructorParameterListIR.

The parameters of CONSTRUCTOR `< _* ',' _* `> `( constructorParameterIR* `) ':' _

1. Return constructorParameterIR*.

$parameterListIR_of_controlApplyMethodDef(CONTROL_APPLY `( parameterListIR `) `{ _ ';' _ ';' _ `})

1. Return parameterListIR.

$parameterListIR_of_controlApplyMethodTypeIR(CONTROL_APPLY `( parameterIR* `))

1. Return parameterIR*.

$parameterListIR_of_definedFunctionDef(FUNCTION _ `< _ `> `( parameterListIR `) _)

1. Return parameterListIR.

$parameterListIR_of_definedFunctionTypeDefIR(FUNCTION _ `< _* ',' _* `> `( parameterIR* `) ':' _)

1. Return parameterIR*.

$parameterListIR_of_externFunctionDef(EXTERN_FUNCTION _ `< _ `> `( parameterListIR `))

1. Return parameterListIR.

$parameterListIR_of_externFunctionTypeDefIR(EXTERN_FUNCTION _ `< _* ',' _* `> `( parameterIR* `) ':' _)

1. Return parameterIR*.

$parameterListIR_of_externMethodDef(externMethodDef)

1. Check that externMethodDef matches pattern EXTERN_METHOD % `< % `> `( % `) %.
2. Let EXTERN_METHOD _ `< _ `> `( parameterListIR `) _? be externMethodDef.
3. Return parameterListIR.

$parameterListIR_of_externMethodTypeDefIR(externMethodTypeDefIR)

1. If let EXTERN_METHOD _ `< _* ',' _* `> `( parameterIR* `) ':' _ be externMethodTypeDefIR: return parameterIR*.
2. Else:
    a. Let EXTERN_METHOD ABSTRACT _ `< _* ',' _* `> `( parameterIR* `) ':' _ be externMethodTypeDefIR.
    b. Return parameterIR*.

$parameterListIR_of_functionDef(functionDef)

1. If let definedFunctionDef of type definedFunctionDef be functionDef:
    a. Return $parameterListIR_of_definedFunctionDef(definedFunctionDef).
2. Else:
    a. Let externFunctionDef be functionDef.
    b. Return $parameterListIR_of_externFunctionDef(externFunctionDef).

$parameterListIR_of_functionTypeDefIR(functionTypeDefIR)

1. If let definedFunctionTypeDefIR of type definedFunctionTypeDefIR be functionTypeDefIR:
    a. Return $parameterListIR_of_definedFunctionTypeDefIR(definedFunctionTypeDefIR).
2. Else:
    a. Let externFunctionTypeDefIR be functionTypeDefIR.
    b. Return $parameterListIR_of_externFunctionTypeDefIR(externFunctionTypeDefIR).

$parameterListIR_of_methodDef(methodDef)

1. If let externMethodDef of type externMethodDef be methodDef:
    a. Return $parameterListIR_of_externMethodDef(externMethodDef).
2. Else if let parserApplyMethodDef of type parserApplyMethodDef be methodDef:
    a. Return $parameterListIR_of_parserApplyMethodDef(parserApplyMethodDef).
3. Else if let controlApplyMethodDef of type controlApplyMethodDef be methodDef:
    a. Return $parameterListIR_of_controlApplyMethodDef(controlApplyMethodDef).
4. Else:
    a. Let tableApplyMethodDef be methodDef.
    b. Return $parameterListIR_of_tableApplyMethodDef(tableApplyMethodDef).

$parameterListIR_of_methodTypeDefIR(methodTypeDefIR)

1. If let externMethodTypeDefIR of type externMethodTypeDefIR be methodTypeDefIR:
    a. Return $parameterListIR_of_externMethodTypeDefIR(externMethodTypeDefIR).
2. Else if let parserApplyMethodTypeIR of type parserApplyMethodTypeIR be methodTypeDefIR:
    a. Return $parameterListIR_of_parserApplyMethodTypeIR(parserApplyMethodTypeIR).
3. Else if let controlApplyMethodTypeIR of type controlApplyMethodTypeIR be methodTypeDefIR:
    a. Return $parameterListIR_of_controlApplyMethodTypeIR(controlApplyMethodTypeIR).
4. Else:
    a. Let tableApplyMethodTypeDefIR be methodTypeDefIR.
    b. Return $parameterListIR_of_tableApplyMethodTypeDefIR(tableApplyMethodTypeDefIR).

$parameterListIR_of_parserApplyMethodDef(PARSER_APPLY `( parameterListIR `) `{ _ ';' _ `})

1. Return parameterListIR.

$parameterListIR_of_parserApplyMethodTypeIR(PARSER_APPLY `( parameterIR* `))

1. Return parameterIR*.

Type parameter lists

The type parameter list is projected from a callable type definition and a type definition with:

The type parameters of callableTypeDefIR

1. If callableTypeDefIR has type actionTypeDefIR: return (·, ·).
2. Else if let FUNCTION _ `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `( _* `) ':' _ of type definedFunctionTypeDefIR be callableTypeDefIR:
   a. Return (typeParameterIR$_{expl}$*, typeParameterIR$_{impl}$*).
3. Else if let EXTERN_FUNCTION _ `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `( _* `) ':' _ of type externFunctionTypeDefIR be callableTypeDefIR:
   a. Return (typeParameterIR$_{expl}$*, typeParameterIR$_{impl}$*).
4. Else if let externMethodTypeDefIR of type externMethodTypeDefIR be callableTypeDefIR:
   a. If let EXTERN_METHOD _ `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `( _* `) ':' _ be externMethodTypeDefIR:
      i. Return (typeParameterIR$_{expl}$*, typeParameterIR$_{impl}$*).
   b. Else:
      i. Let EXTERN_METHOD ABSTRACT _ `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `( _* `) ':' _ be externMethodTypeDefIR.
      ii. Return (typeParameterIR$_{expl}$*, typeParameterIR$_{impl}$*).
5. Else if callableTypeDefIR has type parserApplyMethodTypeDefIR: return (·, ·).
6. Else if callableTypeDefIR has type controlApplyMethodTypeDefIR: return (·, ·).

The type parameters of typeDefIR

1. If typeDefIR has type nameTypeDefIR: return (·, ·).
2. Else if typeDefIR has type aliasTypeDefIR: return (·, ·).
3. Else if let STRUCT _ `< typeParameterIR* `> `{ _* `} of type structTypeDefIR be typeDefIR:
   a. Return (typeParameterIR*, ·).
4. Else if let HEADER _ `< typeParameterIR* `> `{ _* `} of type headerTypeDefIR be typeDefIR:
   a. Return (typeParameterIR*, ·).
5. Else if let HEADER_UNION _ `< typeParameterIR* `> `{ _* `} of type headerUnionTypeDefIR be typeDefIR:
   a. Return (typeParameterIR*, ·).
6. Else if typeDefIR has type enumTypeDefIR: return (·, ·).
7. Else if let EXTERN _ `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `{ _* `} of type externObjectTypeDefIR be typeDefIR:
   a. Return (typeParameterIR$_{expl}$*, typeParameterIR$_{impl}$*).
8. Else if let PARSER _ `< typeParameterIR$_{expl}$* ',' typeParameterIR$_{impl}$* `> `( _* `) of type parserObjectTypeDefIR be typeDefIR:
   a. Return (typeParameterIR$_{expl}$*, typeParameterIR$_{impl}$*).

9. Else if let CONTROL _ `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( _$^{*}$ `) of type controlObjectTypeDefIR be typeDefIR:
    a. Return (typeParameterIR$_{expl}^{*}$, typeParameterIR$_{impl}^{*}$).
10. Else if let PACKAGE _ `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `{ _$^{*}$ `} of type packageObjectTypeDefIR be typeDefIR:
    a. Return (typeParameterIR$_{expl}^{*}$, typeParameterIR$_{impl}^{*}$).
11. Else: return (·, ·).

Types of definitions and prototypes

The type underlying a type definition, the callable type underlying a callable type definition, and the type of an extern method prototype are projected with:

The underlying type of typeDefIR

1. If let nameTypeDefIR of type nameTypeDefIR be typeDefIR: return nameTypeDefIR.
2. Else if let aliasTypeDefIR of type aliasTypeDefIR be typeDefIR: return aliasTypeDefIR.
3. Else if let STRUCT typeId `< typeParameterIR$^{*}$ `> `{ fieldTypeIR$^{*}$ `} of type structTypeDefIR be typeDefIR:
    a. Return STRUCT typeId `< ( NAME typeParameterIR )$^{*}$ `> `{ fieldTypeIR$^{*}$ `}.
4. Else if let HEADER typeId `< typeParameterIR$^{*}$ `> `{ fieldTypeIR$^{*}$ `} of type headerTypeDefIR be typeDefIR:
    a. Return HEADER typeId `< ( NAME typeParameterIR )$^{*}$ `> `{ fieldTypeIR$^{*}$ `}.
5. Else if let HEADER_UNION typeId `< typeParameterIR$^{*}$ `> `{ fieldTypeIR$^{*}$ `} of type headerUnionTypeDefIR be typeDefIR:
    a. Return HEADER_UNION typeId `< ( NAME typeParameterIR )$^{*}$ `> `{ fieldTypeIR$^{*}$ `}.
6. Else if let enumTypeDefIR of type enumTypeDefIR be typeDefIR: return enumTypeDefIR.
7. Else if let EXTERN typeId `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `{ ( callableId ':' externMethodTypeDefIR )$^{*}$ `} of type externObjectTypeDefIR be typeDefIR:
    a. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
    b. Return EXTERN typeId `< ( NAME typeParameterIR )$^{*}$ `> `{ ( callableId ':' externMethodTypeDefIR )$^{*}$ `}.
8. Else if let PARSER typeId `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) of type parserObjectTypeDefIR be typeDefIR:
    a. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
    b. Return PARSER typeId `< ( NAME typeParameterIR )$^{*}$ `> `( parameterIR$^{*}$ `).
9. Else if let CONTROL typeId `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `( parameterIR$^{*}$ `) of type controlObjectTypeDefIR be typeDefIR:
    a. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
    b. Return CONTROL typeId `< ( NAME typeParameterIR )$^{*}$ `> `( parameterIR$^{*}$ `).
10. Else if let PACKAGE typeId `< typeParameterIR$_{expl}^{*}$ ',' typeParameterIR$_{impl}^{*}$ `> `{ typeIR$^{*}$ `} of type packageObjectTypeDefIR be typeDefIR:
    a. Let typeParameterIR* be typeParameterIR$_{expl}^{*}$ concatenated with typeParameterIR$_{impl}^{*}$.
    b. Return PACKAGE typeId `< ( NAME typeParameterIR )$^{*}$ `> `{ typeIR$^{*}$ `}.
11. Else:
    a. Let tableObjectTypeDefIR be typeDefIR.
    b. Return tableObjectTypeDefIR.

The underlying type of callableTypeDefIR

1. If let actionTypeDefIR of type actionTypeDefIR be callableTypeDefIR: return actionTypeDefIR.
2. Else if let FUNCTION nameIR `< _* ',' _* `> `( parameterIR* `) ':' typeIR of type definedFunctionTypeDefIR be callableTypeDefIR:
   a. Return FUNCTION nameIR `( parameterIR* `) ':' typeIR.
3. Else if let EXTERN_FUNCTION nameIR `< _* ',' _* `> `( parameterIR* `) ':' typeIR of type externFunctionTypeDefIR be callableTypeDefIR:
   a. Return EXTERN_FUNCTION nameIR `( parameterIR* `) ':' typeIR.
4. Else if let externMethodTypeDefIR of type externMethodTypeDefIR be callableTypeDefIR:
   a. If let EXTERN_METHOD nameIR `< _* ',' _* `> `( parameterIR* `) ':' typeIR be externMethodTypeDefIR:
      i. Return EXTERN_METHOD nameIR `( parameterIR* `) ':' typeIR.
   b. Else:
      i. Let EXTERN_METHOD ABSTRACT nameIR `< _* ',' _* `> `( parameterIR* `) ':' typeIR be externMethodTypeDefIR.
      ii. Return EXTERN_METHOD ABSTRACT nameIR `( parameterIR* `) ':' typeIR.
5. Else if let parserApplyMethodTypeDefIR of type parserApplyMethodTypeDefIR be callableTypeDefIR: return parserApplyMethodTypeDefIR.
6. Else if let controlApplyMethodTypeDefIR of type controlApplyMethodTypeDefIR be callableTypeDefIR: return controlApplyMethodTypeDefIR.

Callable type definition of externMethodPrototypeIR

1. If let _ $\text{typeIR}_{\text{ret}}$ nameIR `< $\text{typeParameterListIR}_{\text{expl}}$ ',' $\text{typeParameterListIR}_{\text{impl}}$ `> `( parameterIR* `) ';' be externMethodPrototypeIR:
   a. Let externMethodTypeDefIR be EXTERN_METHOD nameIR `< $\text{typeParameterListIR}_{\text{expl}}$ ',' $\text{typeParameterListIR}_{\text{impl}}$ `> `( parameterIR* `) ':' $\text{typeIR}_{\text{ret}}$.
   b. Return externMethodTypeDefIR.
2. Else:
   a. Let _ ABSTRACT $\text{typeIR}_{\text{ret}}$ nameIR `< $\text{typeParameterListIR}_{\text{expl}}$ ',' $\text{typeParameterListIR}_{\text{impl}}$ `> `( parameterIR* `) ';' be externMethodPrototypeIR.
   b. Let externMethodTypeDefIR be EXTERN_METHOD ABSTRACT nameIR `< $\text{typeParameterListIR}_{\text{expl}}$ ',' $\text{typeParameterListIR}_{\text{impl}}$ `> `( parameterIR* `) ':' $\text{typeIR}_{\text{ret}}$.
   c. Return externMethodTypeDefIR.

Appendix O: Value serialization

This appendix collects the operations that convert between P4 values and their representations, used when parsing and deparsing packets.

Integer values

An integer value is projected to a signed or an unsigned mathematical integer with:

The integer representation of integerValue

1. If let integerLiteral of type integerLiteral be integerValue:
    a. If let D i be integerLiteral: return i.
    b. Else if let w S i be integerLiteral:
        i. Return the two's complement representation of bitstring i of width w.
    c. Else:
        i. Let w W i be integerLiteral.
        ii. Return i.
2. Else:
    a. Let _ '.' _ V i be integerValue.
    b. Return i.

The natural number representation of integerValue

1. Let i be the integer representation of integerValue.
2. If let n of type nat be i: return n.
3. Otherwise: return ·.

Bit-string serialization

A value is serialized to a bit-string, and reconstructed from a bit-string, with:

$write_bits_from_value(value')

1. Try:
    a. If let $_B$ b of type boolValue be value': return b.
    b. Else if let integerLiteral of type integerLiteral be value':
        i. If let w S i be integerLiteral: return $int_to_bits_signed(w, i).
        ii. Else if let w W i be integerLiteral: return $int_to_bits_unsigned(w, i).
    c. Else if let ARRAY `[ $value_{element}^{*}$ `] of type arrayValue be value':
        i. For each $value_{element}$ in $value_{element}^{*}$:
            - Let $bool^{*}$ be $write_bits_from_value($value_{element}$).

            Let $bool^{**}$ be the resulting list.
        ii. Return $concat_<bool>($bool^{**}$).
    d. Else if let HEADER_STACK `[ $value_{element}^{*}$ `(_ `) `] of type headerStackValue be value':
        i. For each $value_{element}$ in $value_{element}^{*}$:
            - Let $bool'^{*}$ be $write_bits_from_value($value_{element}$).

            Let $bool'^{**}$ be the resulting list.
        ii. Return $concat_<bool>($bool'^{**}$).
    e. Else if let STRUCT _ `{ ( $value_{field}$ $id_{field}$ ';' )$^{*}$ `} of type structValue be value':
        i. For each $value_{field}$ in $value_{field}^{*}$:

- Let bool''* be $write_bits_from_value(value_field).

Let bool''** be the resulting list.

ii. Return $concat_<bool>(bool''**).

f. Else if value' has type headerValue:

i. Try:

A. Let HEADER _ `{ b ';' ( value_field id_field ';' )* `} be value'.

B. If b is equal to true:

I. For each value_field in value_field*:

- Let bool'''* be $write_bits_from_value(value_field).

Let bool'''** be the resulting list.

II. Return $concat_<bool>(bool'''**).

ii. Then, try:

A. Let HEADER _ `{ b ';' _* `} be value'.

B. If b is equal to false: return ·.

g. Else if let HEADER_UNION _ `{ ( value id ';' )* `} of type headerUnionValue be value':

i. For each value in value*:

- Let bool''''* be $write_bits_from_value(value).

Let bool''''** be the resulting list.

ii. Return $concat_<bool>(bool''''**).

h. Else if let enumValue of type enumValue be value':

i. If let typeId '.' id '.' value be enumValue: return $write_bits_from_value(value).

2. Then, try:

a. Check that value' has type integerValue.

b. Let integerValue be value'.

c. Check that integerValue matches pattern % '.' % V %.

d. Let _ '.' w V i be integerValue.

e. Return $int_to_bits_unsigned(w, i).

$write_value_from_bits(value, n_var, b*)

1. Let (value_updated, b'*) be $write_value_from_bits'(value, n_var, b*).
2. Check that b'* is an empty list.
3. Return value_updated.

$write_value_from_bits'(value', n_var, b*)

1. Try:

a. If value' has type boolValue:

i. If b* is a non-empty list:

A. Let $b_h :: b_t^*$ be b^*.

B. Return ($_B\ b_h, b_t^*$).

b. Else if let integerLiteral of type integerLiteral be value':

i. If let w W _ be integerLiteral:

A. If the length of b^* is greater than or equal to w:

I. Let b_h^* be $b^*[0 : w]$.

II. Let i' be the length of b^* - w.

III. If let n_t of type nat be i':

1. Let b_t^* be $b^*[w : n_t]$.

2. Let int be $bits_to_int_unsigned($b_h^*$).

3. Let i be the bitstring of width w representing the integer int.

4. Return (w W i, b_t^*).

ii. Else if let w S _ be integerLiteral:

A. If the length of b^* is greater than or equal to w:

I. Let b_h^* be $b^*[0 : w]$.

II. Let i' be the length of b^* - w.

III. If let n_t of type nat be i':

1. Let b_t^* be $b^*[w : n_t]$.

2. Let int' be $bits_to_int_signed($b_h^*$).

3. Let i be the bitstring of width w representing the integer int'.

4. Return (w S i, b_t^*).

c. Else if let STRUCT typeId `{ fieldValue* `} of type structValue be value':

i. Let ($fieldValue_{updated}^*$, b_{rest}^*) be $write_value_fields_from_bits'(fieldValue*, n_{var}, b^*).

ii. Return (STRUCT typeId `{ $fieldValue_{updated}^*$ `}, b_{rest}^*).

d. Else if let HEADER typeId `{ _ ';' fieldValue* `} of type headerValue be value':

i. Let ($fieldValue_{updated}^*$, b_{rest}^*) be $write_value_fields_from_bits'(fieldValue*, n_{var}, b^*).

ii. Return (HEADER typeId `{ true ';' $fieldValue_{updated}^*$ `}, b_{rest}^*).

e. Else if let enumValue of type enumValue be value':

i. If let typeId '.' nameIR '.' value be enumValue:

A. Let ($value_{updated}$, b_{rest}^*) be $write_value_from_bits'(value, n_{var}, b^*).

B. Return (typeId '.' "__UNSPECIFIED" '.' $value_{updated}$, b_{rest}^*).

f. Else if let ARRAY `[ value* `] of type arrayValue be value':

i. Let ($value_{updated}^*$, b_{rest}^*) be $write_values_from_bits'(value*, n_{var}, b^*).

ii. Return (ARRAY `[ $value_{updated}^*$ `], b_{rest}^*).

g. Else if let HEADER_STACK `[ value* `(n_{idx} `) `] of type headerStackValue be value':

i. Let ($value_{updated}^*$, b_{rest}^*) be $write_values_from_bits'(value*, n_{var}, b^*).

ii. Return (HEADER_STACK `[ $value_{updated}^*$ `(n_{idx} `) `], b_{rest}^*).

2. Then, try:

a. Check that value' has type integerValue.

b. Let integerValue be value'.

c. Check that integerValue matches pattern % '.' % V %.

d. Let n_{max} '.' _ V i be integerValue.

e. If n_{var} is equal to 0:

   i. Return (n_{max} '.' 0 V i, b^*).

f. Else:

   i. Check that the length of b^* is greater than or equal to n_{var}.

   ii. Let b_h^* be $b^*[0 : n_{var}]$.

   iii. Let i' be the length of b^* - n_{var}.

   iv. Check that i' has type nat.

   v. Let n_t be i'.

   vi. Let b_t^* be $b^*[n_{var} : n_t]$.

   vii. Let int'' be $bits_to_int_signed($b_h^*$).

   viii. Let i' be the bitstring of width n_{var} representing the integer int''.

   ix. Return (n_{max} '.' n_{var} V i', b_t^*).

$write_values_from_bits'(value*, n_{var}, b^*)

1. If value* matches pattern []: return (·, b^*).
2. Else:
   a. Let $value_h$ :: $value_t^*$ be value*.
   b. Let ($value_{h_updated}$, b_t^*) be $write_value_from_bits'($value_h$, n_{var}, b^*).
   c. Let ($value_{t_updated}^*$, b_{rest}^*) be $write_values_from_bits'($value_t^*$, n_{var}, b_t^*).
   d. Return ($value_{h_updated}$ :: $value_{t_updated}^*$, b_{rest}^*).

$write_value_fields_from_bits'(fieldValue*, n_{var}, b^*)

1. If fieldValue* matches pattern []: return (·, b^*).
2. Else:
   a. Let $fieldValue_h$ :: $fieldValue_t^*$ be fieldValue*.
   b. Let ($fieldValue_{h_updated}$, b_t^*) be $write_value_field_from_bits'($fieldValue_h$, n_{var}, b^*).
   c. Let ($fieldValue_{t_updated}^*$, b_{rest}^*) be $write_value_fields_from_bits'($fieldValue_t^*$, n_{var}, b_t^*).
   d. Return ($fieldValue_{h_updated}$ :: $fieldValue_{t_updated}^*$, b_{rest}^*).

$write_value_field_from_bits'(value id ';', n_{var}, b^*)

1. Let ($value_{updated}$, b_{rest}^*) be $write_value_from_bits'(value, n_{var}, b^*).
2. Return ($value_{updated}$ id ';', b_{rest}^*).